

Tina4 for Node.js Developers

One Framework, Native Conventions, Shared Contracts

v3.13.138 • tina4.com

The Intelligent Native Application 4ramework

Table of Contents

Getting Started with Tina4 Node.js 71

1. What Is Tina4 Node.js . . . 71

2. Prerequisites and Installation . . . 71

What You Need . . . 71

Installing the Tina4 CLI . . . 72

Creating a New Project . . . 72

Starting the Dev Server . . . 73

3. Project Structure Walkthrough . . . 73

4. Your First Route . . . 74

Explicit Route Registration . . . 74

File-Based Routing Alternative . . . 75

Test It . . . 75

Understanding What Happened . . . 75

Adding More HTTP Methods . . . 76

5. Your First Template . . . 76

Create a Base Layout . . . 76

Create a Product Listing Page . . . 77

Create the Route That Renders the Template . . . 78

See It in the Browser . . . 78

How Template Rendering Works . . . 79

About tina4css . . . 79

6. Understanding .env . . . 79

7. The Dev Dashboard . . . 80

8. The app.ts Entry Point . . . 81

9. Manual Setup (No CLI) . . . 82
Step 1: Initialize and Install . . . 82
Step 2: Create app.ts . . . 82
Step 3: Create tsconfig.json . . . 82
Step 4: Create the Folder Structure . . . 82
Step 5: Create .env . . . 83
Step 6: Run It . . . 83
10. Request & Response Fundamentals . . . 83
Reading Query Parameters . . . 83
Reading URL Path Parameters . . . 83
Reading the Request Body . . . 83
Reading Headers . . . 84
Sending JSON Responses . . . 84
Sending HTML / Template Responses . . . 84
Status Codes . . . 84
Worked Example: A Complete Route File . . . 84
11. Exercise: Greeting API + Product List Template . . . 86
Exercise Part A: Greeting API . . . 86
Exercise Part B: Product List Page . . . 87
12. Solutions . . . 87
Solution A: Greeting API . . . 87
Solution B: Product List Page . . . 88
13. Gotchas . . . 90
1. File not auto-discovered . . . 90
2. "Module not found" errors . . . 90
3. JSON response shows HTML . . . 90
4. Template not found . . . 90

5. Port already in use . . . 90
6. Changes not reflected . . . 90
7. .env not loaded . . . 91

Routing **92**

1. How Routing Works in Tina4 . . . 92
2. HTTP Methods . . . 92
HEAD and OPTIONS: automatic, no boilerplate . . . 93
Explicit router.head() and router.options() registration . . . 94
3. Path Parameters . . . 94
Typed Parameters . . . 95
Typed Parameters in Action . . . 96
4. Query Parameters . . . 97
5. Route Groups . . . 97
6. Middleware on Routes . . . 98
Middleware on a Single Route . . . 98
Blocking Middleware . . . 99
Middleware on a Group . . . 99
Multiple Middleware . . . 99
7. Route Decorators: @noauth and @secured . . . 100
@noauth -- Public Routes . . . 100
@secured -- Protected GET Routes . . . 100
8. Route Chaining: .secure() and .cache() . . . 100
.secure() . . . 101
.cache() . . . 101
Chaining Both . . . 101
9. Wildcard and Catch-All Routes . . . 101
Wildcard Routes . . . 101

Catch-All Route (Custom 404) . . . 102
10. Route Listing via CLI . . . 102
11. Organizing Route Files . . . 103
Pattern 1: One File Per Resource . . . 103
Pattern 2: File-Based Routing by Feature . . . 103
12. Exercise: Build a Full CRUD API for Products . . . 103
Requirements . . . 103
13. Solution . . . 105
14. Gotchas . . . 107
1. Trailing Slashes Matter . . . 107
2. Parameter Names Must Be Unique in a Path . . . 107
3. Method Conflicts . . . 107
4. Route Handler Must Return a Response . . . 107
5. Async Handlers . . . 107
6. Middleware Must Be Passed as Function References in an Array . . . 107
7. Group Prefix Must Start with a Slash . . . 108
Request & Response **109**
1. The Two Objects You Always Get . . . 109
2. The Request Object . . . 109
method . . . 109
path . . . 109
params . . . 109
query . . . 109
body . . . 110
headers . . . 110
ip . . . 110
cookies . . . 110

files . . . 110
Inspecting the Full Request . . . 110
3. The Response Object . . . 111
json() -- JSON Response . . . 111
html() -- Template or Raw HTML Response . . . 111
text() -- Plain Text Response . . . 112
redirect() -- Redirect Response . . . 112
file() -- File Download Response . . . 112
4. Status Codes . . . 112
5. Custom Headers . . . 113
CORS Headers . . . 113
6. Cookies . . . 113
7. File Uploads . . . 114
Multiple File Uploads . . . 114
Handling a Single File Upload . . . 115
Saving the Uploaded File . . . 116
8. File Downloads . . . 116
9. XML Responses . . . 117
10. Content Negotiation . . . 117
11. Input Validation . . . 118
The Validator Class . . . 118
The Error Response Envelope . . . 118
Upload Size Limits . . . 118
12. Exercise: Build an Image Upload API . . . 119
Requirements . . . 119
13. Solution . . . 119
14. Gotchas . . . 121

1. Forgetting return . . . 121
2. Body Is Undefined for JSON Requests . . . 121
3. File Uploads Return Empty . . . 121
4. Redirect Loops . . . 121
5. Cookie Not Set . . . 121
6. Large Request Body Rejected . . . 121
7. Headers Are Lowercase in Node.js . . . 122

Templates **123**

1. Why Templates . . . 123
2. Variables and Expressions . . . 123
Accessing Nested Data . . . 123
Method Calls on Objects . . . 124
Expressions . . . 124
3. Template Inheritance . . . 124
Base Layout . . . 124
Child Template . . . 125
Using {{ parent() }} . . . 125
4. Includes . . . 126
Passing Variables to Includes . . . 126
5. For Loops . . . 126
The loop Variable . . . 127
Empty Lists . . . 127
Looping Over Key-Value Pairs . . . 127
6. Conditionals . . . 128
if / elseif / else . . . 128
Ternary Operator . . . 128
Testing for Existence . . . 128

{% set %} -- Local Variables . . . 128
7. Filters . . . 128
Complete Filter Reference . . . 129
String Filters . . . 129
Array Filters . . . 130
Encoding Filters . . . 131
Numeric Filters . . . 131
JSON Filters . . . 131
Dict Filters . . . 132
Other Filters . . . 132
Chaining Filters . . . 132
Dumping Values for Debugging . . . 132
8. Macros . . . 133
Defining a Macro . . . 133
Using Macros . . . 134
9. Special Tags . . . 134
{% raw %} -- Literal Output . . . 134
{% spaceless %} -- Remove Whitespace Between Tags . . . 134
{% autoescape %} -- Control HTML Escaping . . . 134
Whitespace Control . . . 135
Comments . . . 135
10. Template Route Export Pattern . . . 135
11. Exercise: Build a Product Catalog Page . . . 136
Requirements . . . 136
Data . . . 136
12. Solution . . . 136
13. Gotchas . . . 139

1. {% extends %} Must Be the First Tag . . . 139
2. Undefined Variables Show Nothing . . . 139
3. Auto-Escaping Prevents HTML Output . . . 139
4. Variable Scope in Includes . . . 139
5. Macro Arguments Are Positional . . . 139
6. Template File Extension Does Not Matter . . . 139
7. Filters Are Not JavaScript Functions . . . 140

Database **141**

1. From Arrays to Real Data . . . 141
2. Connecting to a Database . . . 141
The Default: SQLite . . . 141
Connection Strings for Other Databases . . . 141
Firebird URL Forms . . . 142
Separate Credentials . . . 142
Connection Pooling . . . 142
Verifying the Connection . . . 142
3. Getting the Database Object . . . 143
4. Raw Queries . . . 143
fetch() -- Get Multiple Rows . . . 143
DatabaseResult . . . 143
Properties . . . 144
Iteration . . . 144
Index Access . . . 144
Countable . . . 144
Conversion Methods . . . 144
Schema Metadata with columnInfo() . . . 144
fetchOne() -- Get a Single Row . . . 145

execute() -- Run a Statement . . . 145
Full Example: A Simple Query Route . . . 145
5. Parameterised Queries . . . 145
A Safe Search Endpoint . . . 146
6. Transactions . . . 146
7. Schema Inspection . . . 147
getTables() . . . 147
getColumns() . . . 147
tableExists() . . . 148
getDatabaseType() . . . 148
Schema Info Endpoint . . . 148
8. FakeData Seeder . . . 148
Seeding a Table . . . 149
9. Batch Operations with executeMany() . . . 149
10. Helper Methods: insert(), update(), delete() . . . 150
insert() . . . 150
update() . . . 150
delete() . . . 150
11. Migrations . . . 150
File Naming . . . 150
Generating a Migration . . . 150
Running Migrations . . . 151
Checking Status . . . 151
Rolling Back . . . 151
The Tracking Table . . . 151
Advanced SQL Splitting . . . 152
12. Query Caching . . . 152

13. Exercise: Build a Notes App . . . 152
Requirements . . . 152
14. Solution . . . 153
Migration . . . 153
Routes . . . 153
15. Gotchas . . . 155
1. Forgetting await . . . 155
2. Connection String Formats . . . 156
3. SQLite File Paths . . . 156
4. Parameterised Queries with LIKE . . . 156
5. Boolean Values in SQLite . . . 156
6. Migration Already Applied . . . 156
8. Down Migration Missing on Rollback . . . 156
7. fetch() Returns Empty Array, Not Null . . . 157
ORM **158**
1. From SQL to Objects . . . 158
ORM at a Glance: Four Languages, One Shape . . . 158
Defining a Model . . . 158
Common Query Operations . . . 159
2. Defining a Model . . . 160
Field Types . . . 161
Field Options . . . 161
Field Mapping . . . 161
autoMap and Case Conversion . . . 162
getDbColumn and getDbData . . . 163
find() vs where() -- naming convention . . . 163
3. createTable -- Schema from Models . . . 163

4. CRUD Operations . . . 164
save -- Create or Update . . . 164
create -- Build and Save in One Step . . . 164
findById -- Fetch One Record by Primary Key . . . 164
find -- Query by Filter Dict . . . 165
where -- Query with SQL Conditions . . . 165
delete -- Remove a Record . . . 165
Listing Records . . . 166
ModelCollection -- The Page and the Total . . . 166
selectOne -- Fetch a Single Record by SQL . . . 167
load -- Populate an Existing Instance . . . 167
count -- Count Records . . . 167
5. toDict, toJson, and Other Serialisation . . . 167
toDict . . . 167
toJson . . . 168
Other Serialisation Methods . . . 168
6. Relationships . . . 168
foreignKey Field Type: Auto-Wired Relationships . . . 168
hasMany . . . 169
hasOne . . . 170
belongsTo . . . 170
7. Eager Loading . . . 171
Declarative Relationships . . . 171
Nested Eager Loading . . . 172
toDict with Nested Includes . . . 172
8. Soft Delete . . . 173
Deleting and Restoring . . . 173

Including Soft-Deleted Records . . . 173
Counting with Soft Delete . . . 174
When to Use Soft Delete . . . 174
9. Auto-CRUD . . . 174
The autoCrud Flag . . . 174
Manual Registration . . . 174
What the Generated Routes Do . . . 175
Custom Routes Alongside Auto-CRUD . . . 175
Table Filter . . . 175
10. Scopes . . . 176
11. Input Validation . . . 177
12. Exercise: Build a Blog with Relationships . . . 178
Requirements . . . 178
13. Solution . . . 178
14. Gotchas . . . 182
1. Forgetting to call save() . . . 182
2. findById() returns null . . . 182
3. find() vs findById() . . . 182
4. all() not findAll() . . . 182
5. toDict() includes everything . . . 182
6. save() returns false on invalid data . . . 183
7. Foreign key not enforced . . . 183
8. N+1 query problem . . . 183
9. Auto-CRUD endpoint conflicts . . . 183
10. Soft-deleted records appearing in queries . . . 183
15. QueryBuilder Integration . . . 184
Multiple Database Connections . . . 184

QueryBuilder 185

1. Why a Query Builder? . . . 185

2. Creating a QueryBuilder . . . 185

Standalone: QueryBuilder.fromTable() . . . 185

From an ORM Model: Model.query() . . . 185

3. Selecting Columns . . . 186

4. WHERE Conditions . . . 186

where() -- AND . . . 186

orWhere() -- OR . . . 186

Combining AND and OR . . . 187

5. Joins . . . 187

Inner Join . . . 187

Left Join . . . 187

Multiple Joins . . . 187

6. Grouping and Aggregation . . . 188

groupBy() . . . 188

having() . . . 188

7. Ordering . . . 188

8. Limit and Offset . . . 189

Limit Only . . . 189

Limit with Offset . . . 189

9. Executing Queries . . . 189

get\<T\>() -- All Rows . . . 189

first\<T\>() -- Single Row . . . 190

count() -- Row Count . . . 190

exists() -- Boolean Check . . . 190

10. Inspecting SQL with toSql() . . . 190

11. Using QueryBuilder in Route Handlers . . . 191
List Endpoint with Filters . . . 191
Single Record Lookup . . . 191
Dashboard with Aggregation . . . 192
12. QueryBuilder vs ORM select() . . . 192
13. Exercise: Sales Report API . . . 193
Setup . . . 193
Requirements . . . 193
14. Solution . . . 193
Revenue by Category . . . 193
Top Customers . . . 194
Summary . . . 195
Testing It . . . 195
15. NoSQL: MongoDB Queries . . . 195
Operator Mapping . . . 196
Example . . . 196
16. Gotchas . . . 197
1. Forgetting the Database Adapter . . . 197
2. Parameter Order Matters . . . 197
3. toSql() Does Not Include LIMIT . . . 197
4. Reusing a QueryBuilder . . . 197
5. Mixing QueryBuilder with ORM select() . . . 197
6. OR Without AND . . . 198
Authentication **199**
1. Locking the Door . . . 199
2. JWT Tokens . . . 199
Generating a Token . . . 199

Token Expiry . . . 199
Validating a Token . . . 200
Reading the Payload . . . 200
The Secret Key and Algorithm . . . 200
3. Password Hashing . . . 201
Hashing a Password . . . 201
Checking a Password . . . 201
Registration Example . . . 202
4. The Login Flow . . . 202
5. Using Tokens in Requests . . . 204
6. Protecting Routes . . . 204
Auth Middleware . . . 204
Applying Middleware to Routes . . . 205
Applying Middleware to a Group . . . 205
7. @noauth and @secured Decorators . . . 205
@noauth -- Skip Authentication . . . 205
@secured -- Require Authentication for GET Routes . . . 206
Role-Based Authorization . . . 206
8. CSRF Protection . . . 207
Generating a Token . . . 207
Validating the Token . . . 207
CSRF Middleware . . . 208
When to Use CSRF Tokens . . . 208
9. Sessions . . . 208
Session Configuration . . . 208
Using Sessions . . . 209
Session Options . . . 209

10. Token Refresh . . . 209
11. Exercise: Build Login, Register, and Profile . . . 210
Requirements . . . 210
Test with: . . . 211
12. Solution . . . 211
Migration . . . 211
Routes . . . 211
13. Gotchas . . . 215
1. Token Expiry Confusion . . . 215
2. Secret Key Management . . . 215
3. CORS with Authentication . . . 215
4. Storing Tokens in localStorage . . . 216
5. Forgetting @noauth on Login . . . 216
6. Password Hash Column Too Short . . . 216
7. Token in URL Query Parameters . . . 216
Sessions & Cookies **217**
1. State in a Stateless World . . . 217
2. Session Configuration . . . 217
Available Backends . . . 217
Redis Configuration . . . 217
MongoDB Configuration . . . 218
Valkey Configuration . . . 218
Database Sessions . . . 218
Session Lifetime . . . 218
Session Cookie . . . 218
3. The Session API . . . 218
Full API Reference . . . 219

4. Reading and Writing Session Data . . . 219
Writing to the Session . . . 219
Reading from the Session . . . 220
Deleting Session Data . . . 220
Checking if a Key Exists . . . 220
Getting All Session Data . . . 220
Preferences Example . . . 221
Visit Counter Example . . . 221
5. Flash Messages . . . 221
Setting a Flash Message . . . 222
Reading a Flash Message . . . 222
Flash Messages in Templates . . . 222
6. Cookies . . . 222
Setting a Cookie . . . 223
Reading a Cookie . . . 223
Deleting a Cookie . . . 223
Cookie Options . . . 224
When to Use Cookies vs Sessions . . . 224
7. Remember Me . . . 225
8. Session Security . . . 226
Regenerate Session ID After Login . . . 226
Destroy a Session . . . 227
Secure Cookie Settings . . . 227
Session Configuration Options . . . 228
9. Exercise: Build a Shopping Cart . . . 228
Requirements . . . 228
Test with: . . . 229

10. Solution . . . 229

11. Gotchas . . . 232

1. Session lost between requests . . . 232

2. Session data is not JSON-serializable . . . 233

3. Cookie not sent on cross-origin requests . . . 233

4. File-based sessions on multi-server deployments . . . 233

5. Session cookie overwritten by another app . . . 233

6. Secure cookies on localhost . . . 233

7. Flash messages shown twice . . . 233

8. Session data disappears after server restart . . . 234

9. Large session data causes slow requests . . . 234

Middleware & Security **235**

1. The Pipeline Pattern . . . 235

2. What Middleware Is . . . 235

3. Built-in CorsMiddleware . . . 236

4. Built-in RateLimiterMiddleware . . . 237

5. Built-in RequestLogger . . . 238

Built-in SecurityHeadersMiddleware . . . 238

Combining All Four Built-In Middleware . . . 239

6. Writing Custom Middleware . . . 239

Function-Based Middleware . . . 239

Class-Based Middleware . . . 239

Example: Request Timer (Class-Based) . . . 240

Example: Request Logger (Function-Based) . . . 240

Example: Security Headers (Class-Based) . . . 240

Example: JSON Content-Type Enforcer (Function-Based) . . . 241

Example: Request ID (Class-Based) . . . 241

Example: Input Sanitization (Class-Based) . . . 241
Example: IP Whitelist (Function-Based) . . . 242
7. Applying Middleware to Routes . . . 242
8. Middleware on Route Groups . . . 242
9. Execution Order . . . 243
10. Short-Circuiting . . . 244
Conditional Short-Circuit . . . 245
11. Modifying Request and Response . . . 245
Adding Data to the Request . . . 245
Modifying the Response . . . 246
Adding a Request ID . . . 246
12. Real-World Example: JWT Authentication Middleware . . . 246
13. Real-World Example: API Key Middleware with Database Lookup . . . 248
14. Real-World Middleware Stack . . . 249
15. Exercise: Build an API Key Middleware System . . . 249
Requirements . . . 249
Test with: . . . 250
16. Solution . . . 250
Migration . . . 250
Routes . . . 251
17. Gotchas . . . 253
1. Forgetting to Call next() . . . 253
2. Middleware Modifies Response After It Is Sent . . . 253
3. Middleware Applied to Wrong Routes . . . 253
4. Middleware Execution Order Surprises . . . 253
5. Error in Middleware Breaks the Chain . . . 254
6. Database Connections in Middleware . . . 254

7. Modifying Request in Middleware Does Not Persist . . . 254
8. CORS Preflight Returns 404 . . . 254
9. Rate Limiter Counts Preflight Requests . . . 255
10. Middleware File Not Auto-Loaded . . . 255
11. Short-Circuiting Skips Cleanup . . . 255
12. Middleware Must Be Passed as Function References . . . 255
18. Security . . . 255
1. Secure-by-Default Routing . . . 256
Making a Write Route Public . . . 256
Protecting a GET Route . . . 256
The Rule . . . 257
2. CSRF Protection . . . 257
How It Works . . . 257
The Template . . . 258
The Middleware . . . 258
AJAX Requests . . . 258
Tokens in Query Strings: Blocked . . . 258
Skipping CSRF Validation . . . 258
Turning CSRF On . . . 259
3. Session-Bound Tokens . . . 259
How Stolen Tokens Fail . . . 259
4. Security Headers . . . 259
HSTS: Enforcing HTTPS . . . 260
Customising Headers . . . 260
5. SameSite Cookies . . . 260
6. Login Flow: Complete Example . . . 261
The Login Route . . . 262

The Login Form . . . 263
Protected Pages: Checking the Session . . . 263
Logout: Destroying the Session . . . 264
7. Handling Expired Sessions . . . 264
The Pattern: Redirect to Login, Then Back . . . 264
Token Refresh . . . 265
8. Rate Limiting . . . 265
9. CORS and Credentials . . . 266
10. Security Checklist . . . 266
Gotchas . . . 267
1. "My POST route returns 401 but I didn't add auth" . . . 267
2. "CSRF validation fails on AJAX requests" . . . 267
3. "I disabled CSRF but forms still fail" . . . 267
4. "My Content-Security-Policy blocks inline scripts" . . . 267
5. "User stays logged in after session expires" . . . 267
Exercise: Secure Contact Form . . . 268
Solution . . . 268
Caching **270**
1. From 800ms to 3ms . . . 270
2. Layer 1: Request-Scoped Query Cache (On by Default) . . . 270
3. Layer 2: Persistent Database Cache (Opt-In) . . . 271
Choosing the backend . . . 271
When to use it . . . 272
When not to use it . . . 272
Skipping the cache for one query . . . 272
4. Layer 3: Response Cache . . . 272
String form in the middleware list . . . 272

Function form via .middleware(...) . . . 273
What happens during the TTL . . . 273
Cache headers . . . 273
Caching with query parameters . . . 274
What not to cache . . . 274
5. Backends . . . 274
Memory (default) . . . 275
Redis (and Valkey) . . . 275
File . . . 275
Graceful fallback . . . 276
6. Direct Key/Value API . . . 276
cacheSet . . . 276
cacheGet . . . 276
cacheDelete . . . 277
clearCache . . . 277
Cache-aside pattern . . . 277
7. Cache Invalidation Strategies . . . 278
Strategy 1: Time-Based Expiry (TTL) . . . 278
Strategy 2: Event-Based Invalidation . . . 278
Strategy 3: Write-Through Cache . . . 278
Choosing a strategy . . . 279
8. TTL Management . . . 279
Dynamic TTL . . . 280
9. Cache Statistics . . . 280
Key/value backend stats (async) . . . 280
Database query-cache stats (sync) . . . 281
10. Combining Cache Layers . . . 281

11. Exercise: Cache an Expensive Product Listing Endpoint . . . 282
Requirements . . . 282
Test with . . . 283
12. Solution . . . 283
13. Gotchas . . . 286
1. Forgetting to await a cache call . . . 286
2. Caching authenticated responses . . . 286
3. Memory cache lost on restart . . . 286
4. Stale data after a database update . . . 286
5. Cache key collisions . . . 286
6. Serialization overhead . . . 287
7. Forgetting to set a TTL . . . 287
Queue System **288**
1. Do Not Make the User Wait . . . 288
2. Why Queues Matter . . . 288
3. File Queue (Default) . . . 288
Creating a Queue and Pushing a Job . . . 289
Convenience Method: produce . . . 289
Push with Priority . . . 289
Queue Size . . . 289
4. Pushing from Route Handlers . . . 290
5. Consuming Jobs . . . 290
Automatic Retry . . . 291
Manual Retry with Delay . . . 291
Manual Pop . . . 291
6. Job Lifecycle . . . 292
Job Methods . . . 292

Job Properties . . . 292
7. Retry and Dead Letters . . . 292
Max Retries . . . 292
Failed vs Dead-Lettered Jobs . . . 292
Dead Letters . . . 293
Reviving Jobs . . . 293
Purging Jobs . . . 293
8. Switching Backends . . . 293
Default: File . . . 293
RabbitMQ . . . 294
Kafka . . . 294
MongoDB . . . 294
9. Multiple Jobs from One Action . . . 295
10. Exercise: Build an Email Queue . . . 295
Requirements . . . 295
Test with: . . . 295
11. Solution . . . 296
12. Gotchas . . . 297
1. Use for await, not for . . . 297
2. Always call complete or fail . . . 297
3. consume runs forever . . . 297
4. Worker not picking up jobs . . . 297
5. Payload must be JSON-serializable . . . 298
6. Dead letters pile up . . . 298
7. File backend for production . . . 298
Events System **299**
1. Decouple Without Wiring Everything Together . . . 299

2. Listening for Events . . . 299
3. Emitting Events . . . 299
4. One-Shot Listeners with once() . . . 300
5. Priority . . . 300
6. Removing Listeners with off() . . . 301
7. Clearing All Listeners with clear() . . . 301
8. Listing Active Listeners . . . 301
9. Organising Events in a Real Application . . . 301
10. Exercise: Order Processing Pipeline . . . 302
Requirements . . . 302
Test with: . . . 302
11. Solution . . . 303
12. Gotchas . . . 304
1. Async listeners are not awaited . . . 304
2. Same function reference required for off() . . . 304
3. Listeners persist across requests . . . 304
4. Forgetting to import the events file . . . 304
Localization **305**
1. Your App Speaks One Language. Your Users Speak Many. . . . 305
2. Locale Files . . . 305
3. Creating an I18n Instance . . . 306
4. Translating Keys with t() . . . 306
5. Interpolation . . . 306
6. Locale Switching Per Request . . . 307
7. Fallback Locale . . . 307
8. Listing Available Locales . . . 308

9. Using I18n in Templates . . . 308
10. Exercise: Multilingual API Errors . . . 308
Requirements . . . 308
Test with: . . . 309
11. Solution . . . 309
12. Gotchas . . . 310
1. Locale files must be valid JSON . . . 310
2. Missing keys return the key string . . . 311
3. setLocale is not thread-safe for concurrent requests . . . 311
4. Placeholder names are case-sensitive . . . 311
Structured Logging **312**
1. console.log Is Not a Logging Strategy . . . 312
2. The Four Log Levels . . . 312
3. Controlling Verbosity with TINA4LOGLEVEL . . . 312
4. Adding Context Fields . . . 313
5. Logging Errors . . . 313
6. Request-Scoped Logging . . . 314
7. Performance Logging . . . 315
8. Exercise: Add Logging to an Existing API . . . 315
Requirements . . . 315
Expected log output (debug level): . . . 315
9. Solution . . . 316
10. Gotchas . . . 317
1. Logging sensitive data . . . 317
2. Logging in hot paths adds latency . . . 317
3. Circular references in context objects . . . 317
4. TINA4LOGLEVEL defaults to info . . . 317

Email with Messenger 318

1. Every App Sends Email . . . 318

2. Messenger Configuration via .env . . . 318

Staging delivery safety . . . 319

Common Provider Configurations . . . 319

3. Constructor Override Pattern . . . 319

4. Sending Plain Text Email . . . 320

The send() Method Signature . . . 321

5. Sending HTML Email with Text Fallback . . . 321

6. Adding Attachments . . . 322

Attachments with Custom Names and Binary Content . . . 323

7. CC, BCC, and Reply-To . . . 323

8. Custom Headers . . . 323

Default Headers on an Instance . . . 324

Per-Email Headers . . . 324

9. Reading Inbox via IMAP . . . 324

Listing Inbox Messages . . . 325

Reading a Specific Message . . . 326

Searching Messages . . . 326

Other IMAP Operations . . . 327

Read from a Specific Folder . . . 327

Testing IMAP Connectivity . . . 327

10. Dev Mailbox Capture . . . 327

Sending from a development environment . . . 328

11. Using Templates for Email Content . . . 328

Rendering and Sending . . . 329

12. Sending Email via Queues . . . 330
13. Exercise: Build a Contact Form with Email Notification . . . 331
Requirements . . . 331
Test with: . . . 331
14. Solution . . . 331
15. Gotchas . . . 334
1. Gmail Blocks "Less Secure" Apps . . . 334
2. Emails Go to Spam . . . 335
3. HTML Email Looks Broken . . . 335
4. Attachment File Not Found . . . 335
5. Dev Mode Silently Intercepts Emails . . . 335
6. Email Template Variables Not Substituted . . . 335
7. Connection Timeout on Send . . . 336
8. IMAP Host Not Configured . . . 336
Frontend Integration **337**
1. Beyond JSON APIs . . . 337
2. What Ships with Tina4 . . . 337
3. The Grid System . . . 337
4. Components . . . 338
Navbar . . . 338
Cards . . . 338
Buttons . . . 339
Tables . . . 339
Alerts . . . 339
Modals . . . 340
Progress Bars . . . 340
Badges . . . 340

Forms . . . 341
Utility Classes . . . 341
5. SCSS Customization . . . 341
Live SCSS Compilation . . . 342
6. frond.js -- The JavaScript Helper . . . 342
AJAX Requests . . . 342
Form Submission via AJAX . . . 343
Token Management . . . 343
Loading Indicators . . . 343
WebSocket (Covered in Chapter 23) . . . 344
7. JavaScript API Reference . . . 344
Including the Scripts . . . 344
7.1 tina4.min.js -- Core Utilities . . . 344
loadPage(url, targetId) . . . 344
saveForm(formId, url, method) . . . 344
sendRequest(url, method, data, callback) . . . 345
7.2 frond.min.js -- Template Engine Client-Side Helpers . . . 345
AJAX Form Handling . . . 345
WebSocket Auto-Reconnect . . . 345
Token Refresh . . . 345
Dynamic Template Loading . . . 345
7.3 tina4js.min.js -- Reactive Frontend Framework . . . 346
Reactive State: signal(), computed(), effect() . . . 346
DOM Rendering: html Tagged Template . . . 346
Web Components: Tina4Element . . . 346
Fetch Wrapper: api() . . . 347
WebSocket Client: ws() . . . 347

Client-Side Routing: route(), navigate() . . . 347

8. Building an Admin Dashboard . . . 347

Base Template . . . 347

Dashboard Page . . . 348

Dashboard Route . . . 350

9. Dark Mode . . . 351

Respecting System Preference . . . 351

10. Responsive Design . . . 351

Responsive Sidebar . . . 351

Responsive Tables . . . 352

Hiding Elements by Screen Size . . . 352

11. Building a Users Page with AJAX . . . 352

The Template . . . 352

The Route . . . 354

12. Building a CRUD Interface . . . 354

13. Static File Serving . . . 355

14. Integrating with React, Vue, or Svelte . . . 356

15. Exercise: Build an Admin Dashboard . . . 356

Requirements . . . 356

Test by: . . . 356

16. Solution . . . 357

17. Gotchas . . . 357

1. CSS Not Loading . . . 357

2. frond.js Functions Not Found . . . 358

3. Dark Mode Flickers on Page Load . . . 358

4. SCSS Not Compiling . . . 358

5. Modal Does Not Open . . . 358

6. Grid Columns Do Not Stack on Mobile . . . 358
7. Static Files Return 404 . . . 358
8. Form Data Not Reaching the Server . . . 359
9. Loading Indicator Never Disappears . . . 359
10. CORS Errors with Separate Frontend . . . 359
11. Large Bundle Sizes . . . 359
12. React Router Conflicts . . . 359
18. HtmlElement: Programmatic HTML Builder . . . 359
Testing **361**
1. Why Tests Matter More Than You Think . . . 361
2. Your First Test . . . 361
How It Works . . . 362
3. Assertion Functions . . . 362
expectEqual . . . 362
expectRaises . . . 362
expectTrue / expectFalse . . . 363
4. Testing Business Logic . . . 363
5. Testing ORM Models . . . 363
Test Database . . . 364
6. Testing Routes . . . 364
TestClient Methods . . . 365
TestResponse . . . 365
7. Testing Authentication . . . 366
8. Resetting Tests Between Files . . . 366
9. Runner Options . . . 366
CI Integration . . . 367
10. Running Tests . . . 367

Run All Tests . . . 367
Run a Specific Test File . . . 367
With npm Scripts . . . 367
11. Testing Best Practices . . . 368
Test One Thing Per Function . . . 368
Use Named Functions for Readable Output . . . 368
Isolate Tests . . . 368
Use Unique Values . . . 368
12. Failed Test Output . . . 369
13. Exercise: Write Tests for Utility Functions . . . 369
14. Solution . . . 370
15. Gotchas . . . 370
1. The tests() Decorator Returns the Original Function . . . 370
2. Arguments Are Passed as an Array . . . 370
3. Named Functions Are Required for Readable Output . . . 370
4. Call reset() Between Separate Test Files . . . 371
5. runAll() Returns Results . . . 371
6. Database State Carries Between Tests . . . 371
7. Async Functions Need Special Handling . . . 371
Scaffolding **372**
The CRUD Generator . . . 372
What Each File Contains . . . 372
Run It . . . 374
Individual Generators . . . 375
Model . . . 375
Route . . . 375
Migration . . . 375

Middleware . . . 375
Test . . . 375
Form . . . 375
View . . . 375
CRUD . . . 376
Auth . . . 376
AutoCRUD . . . 376
Usage . . . 376
The Auth Generator . . . 376
Field Types . . . 377
Table Naming Convention . . . 377
Combining Generators . . . 378
Exercise: Scaffold a Blog . . . 378
Gotchas . . . 378
Generators Do Not Overwrite . . . 378
Run Migrate After Generate . . . 379
File Naming Matters . . . 379
Field Changes Need New Migrations . . . 379
Singular Table Names . . . 379
npx Alternative . . . 379
API Documentation with Swagger 380
1. The 47-Endpoint Problem . . . 380
2. What Swagger/OpenAPI Is . . . 380
3. Enabling Swagger . . . 380
Swagger in Production . . . 380
4. Adding Descriptions to Routes . . . 381

Path Parameters . . . 381
Query Parameters . . . 381
5. Documenting Request Bodies . . . 381
Supported Types in @body and @response . . . 382
6. Documenting Responses . . . 382
7. Tags for Grouping . . . 383
8. Example Values . . . 383
9. Authentication in Swagger . . . 384
Public and Secured Routes . . . 384
Authorize Button . . . 384
Workflow . . . 384
10. Try-It-Out from the Swagger UI . . . 385
11. A Complete Documented API . . . 385
12. Hiding Routes . . . 386
13. Customizing the Swagger Info Block . . . 387
14. Generating Client SDKs . . . 387
Available Generators . . . 387
15. Auto-CRUD Routes in Swagger . . . 388
16. Exercise: Document a Complete User API . . . 388
Requirements . . . 388
17. Solution . . . 389
18. Gotchas . . . 390
1. Annotations Must Be Directly Above the Route . . . 390
2. Missing @tags Makes Endpoints Hard to Find . . . 390
3. @body Must Be Valid JSON . . . 390
4. Swagger Shows Routes You Did Not Annotate . . . 390
5. Response Examples Do Not Match Actual Responses . . . 390

6. Swagger UI Not Available in Production . . . 390
7. SDK Generation Produces Incorrect Types . . . 390
8. @secured Routes Not Sending Auth Token . . . 390

API Client **392**

1. Calling External APIs Without the Boilerplate . . . 392
2. The Api Class . . . 392
3. Configuring the Client . . . 392
4. GET Requests . . . 393
5. POST Requests . . . 394
6. PUT and PATCH Requests . . . 394
7. DELETE Requests . . . 395
8. Response Format . . . 395
9. Per-Request Content Type and Generic Requests . . . 395
10. Using Api Inside Route Handlers . . . 396
11. Exercise: GitHub User Profile Proxy . . . 396
    Requirements . . . 397
    Test with: . . . 397
12. Solution . . . 397
13. Gotchas . . . 398
    1. http_code vs error . . . 398
    2. Timeout is in seconds . . . 398
    3. Reuse the instance . . . 398
    4. Sending secrets in URLs . . . 398

GraphQL and SOAP **399**

1. The Problem GraphQL Solves . . . 399
2. GraphQL vs REST -- A Quick Comparison . . . 399

3. Enabling GraphQL . . . 400
4. Defining a Schema . . . 400
5. Writing Resolvers . . . 401
Testing the Queries . . . 401
6. Mutations . . . 402
Mutation Resolvers . . . 403
Testing Mutations . . . 404
7. Nested Types and Relationships . . . 405
8. Auto-Generating Schema from ORM Models . . . 407
9. The GraphiQL Playground . . . 407
10. Query Variables . . . 408
11. Exercise: Build a GraphQL API for a Blog . . . 408
Requirements . . . 408
Test with these queries: . . . 409
12. Solution . . . 409
Schema . . . 409
Resolvers . . . 410
13. Input Validation . . . 412
Example: Missing Required Argument . . . 412
14. Field-Level Auth Directives . . . 413
Passing Auth Context . . . 413
Schema Example . . . 413
Behavior on Failed Auth . . . 413
15. Gotchas . . . 414
1. Schema File Not Found . . . 414
2. Resolver Not Called . . . 414
3. Nested Resolver Returns Wrong Data . . . 414

4. Mutation Input Not Parsed . . . 414
5. N+1 Query Problem . . . 415
6. GraphQL Playground Returns 404 . . . 415
7. Type Mismatch Between Schema and Resolver . . . 415
14. SOAP / WSDL Services . . . 416
What is SOAP? . . . 416
Defining a SOAP Service . . . 416
Registering the Service . . . 417
Auto-Generated WSDL . . . 417
Type Mappings . . . 417
Lifecycle Hooks . . . 418
Testing with curl . . . 418
A More Complete Example . . . 419
SOAP Gotchas . . . 419
When to Use SOAP vs REST vs GraphQL . . . 420
Real-time with WebSocket **421**
1. The Refresh Button Problem . . . 421
2. What WebSocket Is . . . 421
3. Router.websocket() -- WebSocket as a Route . . . 421
Starting the Server . . . 422
4. Connection Events . . . 422
Open . . . 422
Message . . . 423
Close . . . 423
A Complete Handler . . . 423
5. Sending to a Single Client . . . 424
6. Broadcasting to All Clients . . . 425

Broadcast Excluding Sender . . . 425
Sending JSON . . . 426
Closing a Connection . . . 426
Connection Methods Summary . . . 427
7. Path-Scoped Isolation . . . 427
8. Building a Live Chat . . . 428
WebSocket Handler . . . 428
9. Live Notifications . . . 429
Use Cases . . . 430
Scoped Notifications . . . 430
10. Connecting from JavaScript . . . 430
Basic Connection . . . 431
Auto-Reconnect . . . 431
Using Native WebSocket . . . 432
Manual Auto-Reconnect with Native WebSocket . . . 432
11. A Complete Chat Page . . . 432
12. Exercise: Build a Real-Time Chat Room . . . 434
Requirements . . . 434
Test by: . . . 435
13. Solution . . . 435
14. Scaling with a Backplane . . . 437
Configuration . . . 438
Requirements . . . 438
When to Choose a Backplane . . . 438
15. Gotchas . . . 439
1. WebSocket Needs a Persistent Server . . . 439
2. Messages Are Strings, Not Objects . . . 439

3. Connection Count Is Per-Path . . . 439
4. Broadcasting Does Not Scale Across Servers . . . 439
5. Large Messages Cause Disconnects . . . 440
6. Memory Leak from Tracking Connected Users . . . 440
7. No Authentication on WebSocket Connect . . . 440
8. Route Params Use {id} Not :id . . . 440

Server-Sent Events (SSE) 441

1. The Polling Problem . . . 441
2. What SSE Is . . . 441
3. Your First Stream . . . 442
4. The SSE Protocol . . . 443
The data: Field . . . 443
The event: Field . . . 443
The id: Field . . . 443
The retry: Field . . . 443
The Delimiter . . . 444
5. Frontend: EventSource . . . 444
Basic Usage . . . 444
Named Events . . . 444
Closing the Connection . . . 445
With Authentication . . . 445
6. Real Examples . . . 445
Live Dashboard . . . 445
Build Progress . . . 446
LLM Chat Streaming . . . 446
File Processing Progress . . . 447
7. Custom Content Types . . . 448

NDJSON Streaming . . . 448
Consuming NDJSON on the Client . . . 448
8. SSE vs WebSocket . . . 449
9. Production Considerations . . . 450
Nginx Proxy Buffering . . . 450
Connection Limits . . . 450
Heartbeat Messages . . . 450
10. Exercise: Build a Live Metrics Dashboard . . . 451
Requirements . . . 451
Test by: . . . 451
11. Solution . . . 451
12. What You Learned . . . 453
WSDL / SOAP **454**
1. Legacy Services Are Still Services . . . 454
2. Consuming an External SOAP Service . . . 454
Calling an Operation . . . 454
Handling Errors . . . 455
3. The @WSDLOperation Decorator . . . 455
4. Publishing a SOAP Service . . . 455
Auto WSDL . . . 456
5. Calling Your Published Service . . . 457
6. Exercise: Expose a Product Catalogue as a SOAP Service . . . 458
Requirements . . . 458
Test with: . . . 458
7. Solution . . . 459
8. Gotchas . . . 460

1. SOAP is XML -- whitespace matters in some parsers . . . 460
2. Namespace mismatches break everything . . . 460
3. SOAP requests get a 401 . . . 460
4. Decimal types lose precision in JavaScript . . . 460

DI Container **461**

1. Stop Passing Dependencies Through Every Function Call . . . 461
2. The Container Class . . . 461
3. Registering a Service . . . 461
4. Singleton Registration . . . 462
5. Resolving Services with get() . . . 462
6. Checking Registration with has() . . . 462
7. Resetting the Container . . . 462
8. Services That Depend on Other Services . . . 463
9. Application-Wide Container Pattern . . . 463
10. Testing with the DI Container . . . 464
11. Exercise: Build a Service Container for a Store API . . . 464
Requirements . . . 464
Test with: . . . 465
12. Solution . . . 465
13. Gotchas . . . 466
1. Registering after first get() . . . 466
2. Circular dependencies . . . 466
3. Singletons holding stale state . . . 466
4. Over-registering transient services . . . 467

Service Runner **468**

1. Work That Never Stops . . . 468

2. Defining a Service . . . 468
3. Registering and Starting Services . . . 468
4. Service Lifecycle . . . 469
Stopping a Service . . . 469
Restart on Crash . . . 469
5. Service Options . . . 470
6. A Real-World Example: Queue Drainer . . . 470
7. Health Check Endpoint . . . 471
8. Integrating with the Application . . . 471
9. Exercise: Daily Report Service . . . 472
Requirements . . . 472
Test by setting the interval to 5 seconds so you can see it run: . . . 472
10. Solution . . . 472
11. Gotchas . . . 474
1. Services run immediately by default . . . 474
2. Uncaught exceptions outside run() crash the process . . . 474
3. interval is wall-clock delay between runs, not between start times . . . 474
4. Services do not share state safely . . . 474
MCP Dev Tools **475**
1. What is MCP? . . . 475
2. How It Works . . . 475
Localhost-Only Security . . . 475
3. Connecting AI Tools . . . 475
Claude Code . . . 475
Cursor . . . 476
Manual Connection . . . 476
4. Built-in Tools . . . 476

Database . . . 476
Routes . . . 477
Templates . . . 477
Files . . . 477
Migrations . . . 478
Data and ORM . . . 478
Infrastructure . . . 478
Debugging . . . 478
Project introspection . . . 479
Persistent code index . . . 479
Framework documentation (prose) . . . 479
Live API RAG (reflection) . . . 479
Plan management . . . 480
5. Protocol Details . . . 480
Streamable HTTP (current) . . . 481
Legacy HTTP+SSE (still supported) . . . 481
Messages . . . 481
Lifecycle . . . 481
6. Troubleshooting . . . 482
Building Custom MCP Servers **483**
1. Beyond Dev Tools . . . 483
2. Creating an MCP Server . . . 483
3. Registering Tools with mcpTool . . . 483
4. Registering Resources with mcpResource . . . 484
5. Class-Based MCP Services . . . 484
6. Securing MCP Endpoints . . . 485

7. Testing MCP Tools . . . 486
8. Complete Example: CRM MCP Server . . . 486
9. Best Practices . . . 488

Dev Tools **489**

1. Debugging at 2am . . . 489
2. Enabling the Dev Dashboard . . . 489
3. Dashboard Overview . . . 489
    - System Overview . . . 489
    - Request Inspector . . . 490
    - Error Log . . . 490
    - Queue Manager . . . 490
    - WebSocket Monitor . . . 490
    - Routes . . . 490
    - Mail . . . 490
4. The Dev Toolbar . . . 490
    - Disabling the Toolbar . . . 491
5. Error Overlay . . . 491
    - Example Error . . . 491
    - Error Overlay in Production . . . 492
6. The Gallery . . . 492
7. Live Reload . . . 492
    - How It Works . . . 493
8. Request Inspector . . . 493
    - Request Details Panel . . . 493
    - Filtering Requests . . . 494
    - Request Replay . . . 494
9. SQL Query Runner . . . 494

Safety . . . 495
10. Queue Monitor . . . 495
11. System Info . . . 495
12. Logging . . . 496
13. Health Check . . . 496
14. Exercise: Debug a Failing Route . . . 496
Setup . . . 497
Requirements . . . 497
Solution . . . 498
15. Gotchas . . . 498
1. Dev Dashboard Accessible on Network . . . 498
2. Live Reload Causes Connection Drops . . . 498
3. Error Overlay Shows in Production . . . 499
4. Request Inspector Slows Down the Server . . . 499
5. SQL Runner Executes Destructive Queries . . . 499
6. Template Changes Not Reflected . . . 499
7. Queue Monitor Shows No Jobs . . . 499
8. Debug Mode in Version Control . . . 499
9. Logs Fill Up Disk . . . 500
10. Performance Overhead . . . 500
Tina4 CLI **501**
1. Getting a New Developer Up to Speed . . . 501
2. tina4 init -- Project Scaffolding . . . 501
Language Detection . . . 501
Explicit Language Selection . . . 502
Init into an Existing Directory . . . 502

3. tina4 serve -- Dev Server . . . 502
Options . . . 503
Serving a Project by Name . . . 503
Bypassing the CLI . . . 503
4. tina4 generate model -- ORM Scaffolding . . . 503
Adding Fields . . . 504
Field Types . . . 505
Options . . . 505
5. tina4 generate route -- CRUD Route Scaffolding . . . 505
Options . . . 507
6. tina4 generate migration -- Migration Scaffolding . . . 507
When to Generate a Migration . . . 507
7. tina4 generate middleware -- Middleware Scaffolding . . . 508
8. Generate a Model and Its Routes . . . 508
9. tina4 doctor -- Health Check . . . 509
10. tina4 test -- Running Tests . . . 509
Test Options . . . 510
11. tina4 routes -- Route Listing . . . 510
12. tina4 migrate -- Database Migrations . . . 510
Rollback . . . 510
Migration Table Auto-Upgrade . . . 511
Status . . . 511
13. tina4 queue -- Queue Management . . . 511
14. tina4 build -- Production Build . . . 511
15. Custom CLI Commands . . . 511
16. Exercise: Scaffold a Feature in 5 Commands . . . 512
Requirements . . . 512

Expected Commands . . . 513
Solution . . . 513
17. Gotchas . . . 514
1. tina4 Command Not Found . . . 514
2. Wrong Language Detected . . . 514
3. Generated Files Overwrite Existing Code . . . 514
4. Migration Order Issues . . . 515
5. tina4 serve Uses Wrong Port . . . 515
6. Doctor Shows False Warnings . . . 515
7. Model Name Must Be PascalCase . . . 515
8. Build Fails . . . 515
9. Port Conflict . . . 515
18. Documentation . . . 516
19. Test Port (Dual-Port Development) . . . 516
Vibe Coding with AI **517**
Why Tina4 is Built for AI-Assisted Development . . . 517
The Zero-Dependency Advantage . . . 517
CLAUDE.md -- The AI's Instruction Manual . . . 517
Skills -- Deep Framework Knowledge . . . 518
tina4-maintainer . . . 518
tina4-developer . . . 518
tina4-js . . . 518
The Convention Advantage . . . 518
Real-World Vibe Coding Session . . . 518
TypeScript-Specific Advantages . . . 519
Supported AI Tools . . . 520

The AI Chat in Dev Dashboard . . . 520
Tips for Effective Vibe Coding with Tina4 . . . 521
Exercise . . . 521
Gotchas . . . 521
The Philosophy . . . 522
Environment Variables 523
Core Server . . . 524
Secrets and Authentication . . . 525
Database . . . 526
CORS . . . 527
Security Headers . . . 528
Rate Limiting . . . 528
Sessions . . . 529
Redis/Valkey session backend . . . 530
MongoDB session backend . . . 531
Cache . . . 532
Queues . . . 533
Kafka queue backend . . . 533
RabbitMQ queue backend . . . 533
MongoDB queue backend . . . 534
WebSocket . . . 534
Email . . . 535
Logging . . . 536
File logging . . . 537
Localisation . . . 537
Uploads . . . 538

Services (background tasks) . . . 538
Routing . . . 538
Templates . . . 538
GraphQL . . . 539
Application AI Client and Developer AI Tools . . . 539
MCP . . . 541
Swagger / OpenAPI . . . 541
Configuration Recipes . . . 541
PostgreSQL in production . . . 541
A shared cache on Redis . . . 542
Sessions that survive more than one box . . . 542
A queue on RabbitMQ or Kafka . . . 542
WebSocket broadcasts across instances . . . 542
Locked-down production headers . . . 543
The dev dashboard AI, kept local . . . 543
Minimal .env for Development . . . 543
Minimal .env for Production . . . 543
Feature Module Settings . . . 544
Deployment **545**
1. From Development to Production . . . 545
2. Production .env Configuration . . . 545
Key Differences from Development . . . 546
Sensitive Values . . . 546
3. Building for Production . . . 546
4. Docker Deployment . . . 546
Dockerfile . . . 546
.dockerignore . . . 547

Building and Running . . . 547
Docker Compose . . . 547
5. One Event Loop, and How to Get More . . . 548
What "slow" means here . . . 549
The three fixes . . . 549
Cluster mode . . . 549
What clustering costs you . . . 550
6. Health Checks . . . 550
Broken File Check . . . 551
7. Graceful Shutdown . . . 551
Configuring Shutdown Timeout . . . 551
Docker Stop Grace Period . . . 551
8. Log Rotation . . . 552
Using logrotate (Linux) . . . 552
Docker Logging . . . 552
9. Nginx Reverse Proxy . . . 552
10. SSL/TLS with Let's Encrypt . . . 554
With Nginx and Certbot . . . 554
With Docker (Using Traefik as Reverse Proxy) . . . 554
Certificate Monitoring . . . 555
11. Process Management with systemd . . . 555
12. Queue Workers in Production . . . 556
13. Zero-Downtime Deployment . . . 556
14. Scaling . . . 556
Vertical Scaling . . . 557
Load Balancing with Nginx . . . 557
Docker Scaling . . . 557

Horizontal Scaling . . . 557
Scaling Considerations . . . 558
15. Monitoring . . . 558
Log Aggregation . . . 558
Uptime Monitoring . . . 559
Application Performance Monitoring (APM) . . . 559
16. Exercise: Docker Deploy . . . 559
Requirements . . . 559
Solution . . . 560
HTTP Caching, Compression, and ETags . . . 561
17. Gotchas . . . 561
1. .env Not Loaded in Docker . . . 561
2. SQLite Database Lost on Container Restart . . . 561
3. WebSocket Connections Drop Behind Nginx . . . 561
4. Static Files Slow . . . 562
5. Memory Usage Grows Over Time . . . 562
6. Container Starts Before Database Is Ready . . . 562
7. SSL Certificate Not Renewing . . . 562
8. Scaled Instances Have Different State . . . 562
9. Debug Mode in Production . . . 563
10. .env in Version Control . . . 563
11. Missing Migrations . . . 563
12. Port Conflicts . . . 563
Building a Complete App **564**
1. Putting It All Together . . . 564
2. Planning the App . . . 564
Models . . . 564

Relationships . . . 564
Routes . . . 565
Templates . . . 565
3. Step 1: Init Project and Set Up Database . . . 565
Create Migrations . . . 565
4. Step 2: Define Models . . . 566
5. Step 3: Auth Middleware . . . 568
6. Step 4: Auth Routes . . . 568
Test Registration and Login . . . 570
7. Step 5: Task Routes . . . 571
Test the Task API . . . 575
8. Step 6: Dashboard Stats with Caching . . . 575
9. Step 7: WebSocket for Real-Time Updates . . . 577
10. Step 8: Email Notifications via Queue . . . 578
11. Step 9: Dashboard Template . . . 579
12. Step 10: Tests . . . 581
13. Step 11: Docker Deployment . . . 584
14. The Complete Project Structure . . . 586
15. What You Built . . . 586
16. What to Build Next . . . 587
17. Closing Thoughts -- The Tina4 Philosophy . . . 588
Release Notes **589**
v3.13.138 (2026-09-24) - Safer requests, templates, and database operations . . . 589
v3.13.105 (2026-08-19) - Cross-API invariants, honest logs, portable tests . . . 590
Queue and ORM audit bugs — five fixed . . . 590
Route inspection scans, never boots . . . 590

Startup log tells the truth about @noauth / @secured . . . 591
Hot reload reaches every module that imports the changed one . . . 591
Firebird migration ledger is case-agnostic . . . 591
Test-harness fixes . . . 591
Skills grow a spine . . . 591
Upgrade notes . . . 592
v3.13.104 (2026-08-17) - OIDC SSO and PostGIS land . . . 592
OIDC / OpenID Connect — provider-neutral, session handoff . . . 592
PostGIS spatial support . . . 592
Swagger is SSO-aware . . . 592
v3.13.102 (2026-08-15) - Unified client and CI hardening . . . 592
v3.13.103 (2026-08-16) - Metrics you can trust, releases that cannot lie . . . 593
What changed . . . 593
What ships . . . 593
Failure is data . . . 594
Upgrade notes . . . 594
v3.13.100 (2026-08-14) - Frond keeps the whole page . . . 594
Template inheritance . . . 594
Bounded caches . . . 594
Frond extension scope . . . 594
AI skill installer . . . 594
Release consistency . . . 594
Upgrade notes . . . 595
v3.13.99 (2026-08-13) - Stability, parity, and secure by default . . . 595
Security . . . 595
Data integrity . . . 596
ORM and validation . . . 596

Request model - the big one . . . 597
Migrations . . . 597
HTTP . . . 597
Dev tooling . . . 597
Proven in all four . . . 598
Bug fixes . . . 598
Possible breaking . . . 598
Coming in 3.13.100 . . . 599
v3.13.98 (2026-08-11) - Skills discipline . . . 599
v3.13.97 (2026-08-07) - Behaviour corrections . . . 600
Sessions . . . 600
Queues . . . 600
Proven . . . 600
Possible breaking . . . 600
v3.13.96 (2026-08-07) - One paginate envelope, one message shape . . . 601
Pagination . . . 601
Messenger . . . 601
Swagger . . . 602
Migrations . . . 602
Server . . . 602
Build and skills . . . 602
Proven in all four . . . 602
Possible breaking . . . 602
v3.13.95 (2026-08-06) - Preparing for the 3.14 stable release . . . 603
Queues . . . 603
Logging . . . 603
Databases . . . 603

Sessions . . . 603
Document store . . . 604
Local by default, production by environment variable . . . 604
Bulk insert . . . 604
Tooling . . . 604
Possible breaking . . . 604
v3.13.94 (2026-07-29) - A write with no filter is an error . . . 605
The write path (BREAKING) . . . 605
The Frond sandbox revokes capability, it does not skip a step . . . 606
Messenger: one name, one signature . . . 606
Base images that boot . . . 606
Frond, measured against the engines it replaces . . . 606
Also in this release . . . 607
Node.js specifics . . . 607
v3.13.92 (2026-07-27) - One algorithm, from the env var to the middleware . . . 608
The header names the algorithm that signed . . . 608
authMiddleware stops overriding your configuration . . . 608
authenticateRequest honours the overrides it accepts . . . 608
nbf is enforced, with 60 seconds of leeway (nodejs#39) . . . 609
The header cannot choose the verifier . . . 609
A misconfigured algorithm throws (Breaking) . . . 609
The tests that hold it . . . 610
v3.13.91 (2026-07-27) - The offenders list finally points at code worth fixing . . . 610
A function is no longer charged for the functions inside it . . . 610
Ruby complexity was double what it should have been . . . 611
LOC means one thing again . . . 611
The dev dashboard reads the coupling it is given . . . 611

v3.13.90 (2026-07-27) - A scaffolded model and its migration finally agree . . . 612
One default, defined once (php#186, ruby#33, nodejs#38) . . . 612
A failed write no longer reports success . . . 612
v3.13.89 (2026-07-27) - A typo'd tag no longer renders what it was hiding . . . 612
An unknown tag raises instead of leaking its body (Breaking, Security) . . . 613
set works as a block (Fixed) . . . 613
The expression corpus grew to 82 . . . 614
v3.13.88 (2026-07-27) - json_encode gives you JSON again . . . 614
json_encode emits JSON, not entities (Breaking, reverts 3.13.87) . . . 614
A value JSON cannot represent becomes null (Fixed) . . . 614
to_json and tojson are the same filter . . . 615
The expression corpus grew to 79 . . . 615
What to change in your templates . . . 615
v3.13.87 (2026-07-27) - Frond expressions agree in all four frameworks . . . 615
Booleans print as true or false . . . 616
{% import "file" as alias %} now works (New) . . . 616
The not operator works in output (Fixed) . . . 616
json_encode escaping . . . 616
The gate that keeps this true . . . 617
v3.13.86 (2026-07-25) - The package imports under plain Node, not just tsx . . . 617
v3.13.86 (2026-07-25) - Write-result field names now match the family . . . 617
v3.13.85 (2026-07-24) - The dev-admin bundle ships once . . . 617
Why the surviving file is still called .min.js . . . 618
A gate, so the duplicate cannot come back . . . 618
v3.13.84 (2026-07-24) - Every generated Dockerfile actually starts . . . 618
serve no longer kills PID 1 (all four frameworks) . . . 619
Node.js: the package ships built JavaScript . . . 619

Node.js: the CLI reported a version that had not shipped . . . 619

A gate, so this cannot rot again . . . 620

Also in the CLI (tina4 v3.8.58) . . . 620

v3.13.83 (2026-07-24) - Swagger UI stops serving itself in production . . . 620

The banner advertises only what answers (python#99) . . . 620

The CLI runs no watcher in production (tina4 v3.8.57) . . . 621

A stale CA no longer turns a missing certificate into six failures (python#98) . . . 621

v3.13.82 (2026-07-23) - MQTT 3.1.1: talk to any broker, no dependency . . . 621

v3.13.81 (2026-07-21) - tina4 test exit code, locked in . . . 622

v3.13.79 (2026-07-19) - Session cookies get Secure behind a proxy, and a renamed cookie is read back . . . 622

v3.13.77 (2026-07-16) - A slow background task no longer runs on top of itself . . . 623

v3.13.76 (2026-07-16) - Migrations apply again on a database created before 3.13.55 . . . 623

v3.13.75 (2026-07-14) - Static assets revalidate, so a deploy reaches users without a hard refresh . . . 624

v3.13.74 (2026-07-13) - The dev dashboard connection tester works again . . . 624

v3.13.73 (2026-07-13) - A failed migration re-applies cleanly . . . 624

v3.13.72 (2026-07-12) - Frond fixes, a sandbox hardening, and a malformed-path guard . . . 625

v3.13.71 (2026-07-11) - AI skills: sharper tina4_code guidance . . . 626

v3.13.70 (2026-07-11) - Installed-package imports, column defaults on INSERT, a Firebird charset override, and stacked Swagger metadata . . . 626

Installed-package imports resolve (#32) . . . 626

ORM honours column defaults on INSERT (#165) . . . 626

Firebird connection charset override (#160) . . . 626

Stacked Swagger metadata all survives (#59) . . . 627

v3.13.69 (2026-07-10) - The Api client grows up: uploads, downloads, and safe redirects . . . 627

Also shipping . . . 627

v3.13.68 (2026-07-10) - Parity release . . . 627

v3.13.67 (2026-07-10) - The MCP table browser, locked down . . . 628

v3.13.66 (2026-07-10) - A self-describing CLI, and generators that ship their tests 628
The self-describing CLI 628
Generators now ship a test with the code 628
Databases 629
Fixes 629
Breaking 629
v3.13.56 (2026-07-08) - Skills that own up when they drift 629
v3.13.55 (2026-07-07) - One migration tracking schema on every engine 630
v3.13.54 (2026-07-07) - Migrations honour the SET TERM directive 630
v3.13.53 (2026-07-06) - JSONField: JSON document columns 631
v3.13.52 (2026-07-04) - Frond live blocks, pgsql:// URL scheme, SCSS colour functions . . . 631
v3.13.51 (2026-07-03) - MCP Streamable HTTP transport, Firebird fixes 632
v3.13.50 (2026-07-02) - Path route params match INTEGER primary keys on SQLite (Ruby fix) 633
v3.13.47 (2026-06-25) - Open-issue batch: migration comment splitting, global middleware, SCSS interpolation 633
v3.13.46 (2026-06-24) - Atomic batch insert across MySQL, MSSQL and PostgreSQL 634
v3.13.45 (2026-06-24) - Real-service test hardening 634
v3.13.44 (2026-06-24) - Real-service bug-fix sweep (no mocks) 635
v3.13.43 (2026-06-22) - Queue: MongoDB no longer re-delivers completed jobs 636
v3.13.42 (2026-06-22) - Swagger configurability for external and public APIs 637
v3.13.41 (2026-06-22) - Queue reservation/visibility timeout (at-least-once delivery) 637
v3.13.40 (2026-06-22) - MCP security hardening + Swagger/OpenAPI overhaul 638
v3.13.39 (2026-06-21) - Auto-migration, unified critical log level, fail-loud ORM, per-route WebSocket auth 639
v3.13.38 (2026-06-19) - Coordinated security & robustness release 640
v3.13.37 (2026-06-18) - Dev-admin editor: TypeScript + Ruby highlighting fixed 640
v3.13.36 (2026-06-18) - Instant WebSocket dev-reload + dev-admin file browser fix 640

v3.13.35 (2026-06-17) - Live MCP endpoint + working DB tools for AI agents 641

v3.13.34 (2026-06-17) - Scaffolder fix + dual-port reload correction 641

v3.13.33 (2026-06-17) - Queues: priority pop + auto dead-lettering + TINA4QUEUEURL parity (Warning: behavioural change) 641

v3.13.32 (2026-06-17) - Caching: per-query bypass + string-middleware + X-Cache-TTL (chapter rewritten) 641

v3.13.31 (2026-06-17) - Version alignment (no functional change in Node) 642

v3.13.30 (2026-06-16) - Typed route params coerce + JWT expiry now in minutes (Warning: two breaking changes) 642

v3.13.29 (2026-06-16) - Live API search ranks qualified queries + resolves the public import path642

v3.13.27 (2026-06-16) - Frond template-engine parity fixes 642

v3.13.26 (2026-06-16) - pooling fix: standalone writes auto-commit; explicit transactions stay atomic 643

v3.13.25 (2026-06-16) - Node.js: distributed responseCache + persistent DB cache (async completion) 643

v3.13.24 (2026-06-15) - unified cache backends across response, KV, and persistent DB cache644

v3.13.23 (2026-06-15) - request-scoped DB query cache, on by default (+ cache fixes) 644

v3.13.21 (2026-06-15) - docs: render() corrections + version re-sync 645

v3.13.20 (2026-06-15) - Node.js: global class middleware (Router.use) now runs 645

v3.13.19 (2026-06-15) - return domain objects, construct from JSON, and one database binder645

response(...) serializes domain objects 646

Construct a model from a JSON object string 646

One database binder: bindDatabase (+ named connections) 646

v3.13.18 (2026-06-15) - ORM eager-load + include + aggregate fixes 646

v3.13.16 (2026-06-15) - Warning: Async database API (BREAKING) + createTable on PostgreSQL + result indexing 647

Warning: Breaking: the database / ORM / QueryBuilder API is now uniformly async 647

createTable engine-aware on PostgreSQL 647

result[0] index access 648

v3.13.14 (2026-06-13) - Logs reach stdout in containers + per-request logging + schema-qualified tables (#48) . . . 648
Per-request logging - now gated, routed through Log . . . 648
What changed (stdout) . . . 648
Why it spanned all four . . . 649
Schema-qualified tables (#48) + a PostgreSQL fetch() regression . . . 649
Tests . . . 650
v3.13.12 (2026-06-11) - SQL safety + implicit ORM binding + fetchAll correctness . . . 650
fetchAll actually fetches ALL rows now (no silent 100-row truncation) . . . 650
Trailing ; is now stripped from user SQL in fetch() / fetchOne() . . . 650
Implicit ORM binding from TINA4_DATABASE_URL . . . 651
Cross-framework parity . . . 651
Tests . . . 651
v3.13.11 (2026-06-11) - ORM correctness pass . . . 651
#50.2 - save() correctly INSERTs natural (non-auto-increment) PKs . . . 652
#50.1 - callable defaults (N/A in Node) . . . 652
BooleanField engine-aware DDL (already correct in Node) . . . 652
PG error-visibility fixes (Python only) . . . 652
Tests . . . 652
v3.13.9 (2026-06-10) . . . 653
The bug . . . 653
The fix . . . 653
Same algorithm in Python / PHP / Ruby . . . 653
Tests . . . 653
What you'll see when you re-install . . . 654
v3.13.7 (2026-06-10) . . . 654
NEW: tina4.request.error event . . . 654
FIX: Stack trace removed from production 500 body (CWE-209) . . . 654

Tests . . . 655
Background . . . 655
v3.13.6 (2026-06-09) . . . 655
Better driver install hints (#47) . . . 655
#46 - PostgreSQL transaction cascade (no fix needed) . . . 655
Tests . . . 656
v3.13.5 (2026-06-05) . . . 656
What changes . . . 656
Instance form still works . . . 656
clearRegistry() for test fixtures . . . 656
Implementation notes per framework . . . 657
Test count . . . 657
Upgrade . . . 657
v3.13.4 (2026-06-04) . . . 657
PY-10-02 - @middleware() no longer silently disables auth (SECURITY) . . . 658
PY-10-03 - request.headers is now case-insensitive . . . 658
PY-10-01 - Function-based middleware now runs . . . 658
Python chapter rewrites - book + docs . . . 659
Test count . . . 659
Upgrade . . . 659
v3.13.3 (2026-06-03) . . . 660
Env typed env-var helpers (tina4-python#43) . . . 660
Function-name in log lines (tina4-python#41) . . . 660
Test count . . . 661
Upgrade . . . 661
v3.13.2 (2026-06-03) . . . 661
SCSS calc() with mixed units (tina4-python#42, tina4-php#116, tina4-nodejs#1) . . . 661

Router.group docs taught a crashing pattern (tina4-python#44) . . . 661
DATABASEURL -> TINA4DATABASE_URL drift (tina4-python#45) . . . 661
Test count . . . 662
Upgrade . . . 662
v3.13.1 (2026-06-02) . . . 662
Convenience parity additions (Groups A / B) . . . 662
Decorator-style GraphQL resolvers across the family . . . 663
Class-based service pattern across the family . . . 663
PHP chapter rewrites (docs/php/ and book-2-php/) . . . 663
Test count . . . 664
Upgrade . . . 664
v3.13.0 (2026-06-01) . . . 664
The headline fire: Test class with HTTP helpers . . . 664
Auth.valid_token now returns the payload, not a bare bool - BREAKING . . . 665
Python-specific groups (mirrored to PHP/Ruby/Node in follow-up patches) . . . 665
PHP-specific: Tina4\Debug shim . . . 666
Documentation sweep . . . 666
Aspirational chapters rewritten . . . 667
tina4-js doc drift caught . . . 667
Test counts . . . 668
Upgrade . . . 668
v3.12.14 (2026-06-01) . . . 668
Python - tina4_python.test xUnit testing surface . . . 668
Cross-framework parity check (testing) . . . 669
PHP - :named placeholder translation across non-PDO adapters . . . 670
Cross-framework parity check (:named placeholders) . . . 670
Upgrade . . . 670

v3.12.13 (2026-05-29) . . . 670
Cross-framework dev-admin parity sweep (Tier 1-5) . . . 671
Folded-in from PHP 3.12.11 - file upload regression (tina4-book#139) . . . 672
Folded-in from PHP 3.12.12 + Python 3.12.13 - v2 tina4_migration auto-upgrade (#115) . . . 673
Folded-in from PHP 3.12.11 - request URL parity . . . 673
Upgrade . . . 673
v3.12.10 (2026-05-14) . . . 674
PHP - ORM->save() no longer swallows write failures (#114) . . . 674
Also in the PHP 3.12.7-3.12.9 patch line . . . 674
Python / Ruby / Node . . . 675
Upgrade . . . 675
v3.12.6 (2026-05-06) . . . 675
Python - psycopg2 % substitution no longer trips PL/pgSQL function bodies (#40) . . . 675
Long-standing tina4-js #37 confirmed fixed . . . 676
Upgrade . . . 676
v3.12.5 (2026-05-06) . . . 676
PHP - multipart bodies with file uploads now parse correctly (#135) . . . 676
Upgrade . . . 676
v3.12.4 (2026-05-06) . . . 676
25 new env vars across all 4 frameworks . . . 677
Logging - env-driven file output + rotation . . . 677
Documentation-truth CI gate now strict on both axes . . . 678
Tests added . . . 678
Upgrade path . . . 678
v3.12.3 (2026-05-05) . . . 678
Breaking changes (Ruby + PHP only) . . . 678
Doc parity - CLAUDE.md and book chapter 33 . . . 679

Other fixes . . . 679
Genuine gaps surfaced by the parity audit (follow-up, not blocking 3.12.3) . . . 679
Upgrade path . . . 679
v3.12.2 (2026-05-05) . . . 680
Firebird URL auto-detect . . . 680
Bug fix specific to PHP - mysqli localhost+port quirk . . . 680
Other fixes . . . 680
v3.12.1 (2026-05-04) . . . 681
v3.12.0 (2026-05-04) . . . 681
Why this release . . . 681
What changed . . . 681
Bug fixes shipped alongside the rename . . . 682
Migration - every renamed var . . . 683
Names that stay un-prefixed (not framework config) . . . 683
How to upgrade . . . 683
Parity . . . 684
v3.11.32 (2026-04-25) . . . 684
v3.11.13 (2026-04-16) . . . 685
v3.11.12 (2026-04-16) . . . 686
v3.11.11 (2026-04-16) . . . 687
v3.11.10 (2026-04-15) . . . 687
v3.11.9 (2026-04-15) . . . 688
v3.10.99 (2026-04-12) . . . 688
v3.10.97 (2026-04-11) . . . 689
v3.10.93 (2026-04-11) . . . 689
v3.10.92 (2026-04-10) . . . 689
v3.10.91 (2026-04-10) . . . 689

v3.10.90 (2026-04-09) . . . 690
v3.10.89 (2026-04-09) . . . 690
v3.10.88 (2026-04-09) . . . 690
v3.10.87 (2026-04-09) . . . 691
v3.10.86 (2026-04-09) . . . 691
v3.10.85 (2026-04-09) . . . 691
v3.10.84 (2026-04-09) . . . 691
v3.10.83 (2026-04-08) . . . 691
v3.10.70 (2026-04-06) . . . 692
v3.10.68 (2026-04-03) - Full Parity Release . . . 692
v3.10.67 (2026-04-03) . . . 692
v3.10.66 (2026-04-03) . . . 693
v3.10.65 (2026-04-03) . . . 693
v3.10.60 (2026-04-03) . . . 693
v3.10.57 (2026-04-02) . . . 693
v3.10.54 (2026-04-02) . . . 694
v3.10.48 - April 2, 2026 . . . 694
Bug Fixes . . . 694
v3.10.46 - April 1, 2026 . . . 694
Test Coverage . . . 694
v3.10.45 - April 1, 2026 . . . 695
Notes . . . 695
v3.10.44 - April 1, 2026 . . . 695
New Features . . . 695
Bug Fixes . . . 695
Test Coverage . . . 695

v3.10.40 - April 1, 2026 . . . 696
Bug Fixes . . . 696
v3.10.39 - April 1, 2026 . . . 696
New Features . . . 696
v3.10.38 - April 1, 2026 . . . 696
Code Metrics & Bubble Chart . . . 696
AI Context Installer . . . 696
Dashboard Improvements . . . 697
Cleanup . . . 697
v3.10.x - Previous Releases (March 28-31, 2026) . . . 697
Features . . . 697
Bug Fixes . . . 698
Firebird-Specific . . . 699
v3.9.x - QueryBuilder, Sessions, Path Injection (March 26-27, 2026) . . . 700
Features . . . 700
Breaking Changes . . . 701
Bug Fixes . . . 702
v3.8.x - Template Engine, Typed Params, Security (March 25-26, 2026) . . . 702
Features . . . 702
Breaking Changes . . . 703
v3.7.x - Template Auto-Serve, Firebird Migrations (March 25, 2026) . . . 704
Features . . . 704
v3.6.x - Architectural Parity (March 25, 2026) . . . 704
Features . . . 704
Bug Fixes . . . 704
v3.5.x - Bundled Frontend, Middleware (March 25, 2026) . . . 704
Features . . . 704

v3.4.x - Database, Auth, WebSocket, Uploads (March 24, 2026) . . . 705
Features . . . 705
Breaking Changes . . . 706
v3.3.x - Queue API, Field Mapping, Route Chaining (March 24, 2026) . . . 707
Features . . . 707
v3.2.x - Flexible Route Handlers (March 22, 2026) . . . 708
Features . . . 708
v3.1.x - Response Parity, Routing API (March 21-22, 2026) . . . 708
Features . . . 708
v3.0.0 - Initial Release (March 21, 2026) . . . 709
Features . . . 709
Pre-Release (rc.2-rc.5) . . . 710
Version Timeline . . . 711
Feature Catalog **712**
Read the status correctly . . . 712
Where the features live . . . 713
Current parity boundary . . . 713
Dependency boundary . . . 714
Numbered catalog . . . 714
Foundation . . . 714
Database and providers . . . 715
ORM and data layer . . . 716
HTTP core . . . 716
HTTP policies . . . 716
HTTP runtime . . . 717
Frond template engine . . . 717

Frontend assets . . . 718
Authentication . . . 718
Sessions and providers . . . 718
Cache and providers . . . 719
Integrations and storage . . . 720
Enterprise authentication . . . 721
Application runtime . . . 721
CLI . . . 722
Developer runtime . . . 722
Testing and verification tools . . . 723
What parity means . . . 723
Real-time Collaboration (WebRTC) 724
1. The Media Server You Do Not Have to Run . . . 724
2. What You Get, and How It Differs from Raw WebSocket . . . 724
3. Mounting the Module: realtime() . . . 725
What each feature wires . . . 727
4. The Config Bootstrap: GET {p}/api/rtc/config . . . 727
5. ICE and TURN Servers . . . 728
Environment variables . . . 728
6. Signalling: The Mesh Relay . . . 729
7. Chat: Secured Channels, Presence, History . . . 730
Chat history: GET {p}/api/channels/{id}/messages . . . 731
8. Files: Upload and Download . . . 732
POST {p}/api/files - upload (auth-required) . . . 732
GET {p}/api/files/{key} - download (.secure()) . . . 733
Storage backends . . . 733

9. Auth, Identity, and the Data Model . . . 734
The data model . . . 734
10. A Complete Example . . . 735
Backend - app.ts . . . 735
Browser - bootstrap from the config endpoint . . . 736
11. Footguns and Hard Rules . . . 737
Where to Go Next . . . 738
AI Client **739**
Configure a provider . . . 739
Complete one prompt . . . 739
Hold a chat . . . 740
Stream text . . . 740
Create embeddings . . . 741
Handle failures . . . 741
Timeouts and retries . . . 741
OpenID Connect SSO **743**
Configure SSO . . . 743
Use the identity . . . 743
Claims and provider recipes . . . 744
GIS and PostGIS **745**
Define a point field . . . 745
Query and return GeoJSON . . . 745
IoT and MQTT **746**
Configure, connect, and publish . . . 746

Getting Started with Tina4 Node.js

1. What Is Tina4 Node.js

Tina4 Node.js is a TypeScript-first web framework for Node.js 22+. Its core, Frond, and Swagger packages declare no required third-party packages. The ORM and root package declare optional packages for selected providers. Tina4 Node.js contributes to the framework family's 140-entry feature catalog; the catalog is an inventory, not a claim that every entry has reached parity.

SQLite support uses Node's built-in `node:sqlite` module, so the default database path needs no extra package. Remote database, MongoDB, Redis, and S3 providers install their optional driver or SDK packages when selected.

It belongs to the Tina4 family: four backend implementations in Python, PHP, Ruby, and Node.js. They share contracts, project structure, template syntax, CLI commands, and `.env` variables. The parity audit records where an implementation still differs.

Tina4 Node.js uses `camelCase` for method names (`fetchOne()`, `softDelete()`, `hasMany()`). JavaScript convention. Class names are `PascalCase`. Constants are `UPPER_SNAKE_CASE`.

By the end of this chapter, you will have a working project with an API endpoint and a rendered HTML page.

---

2. Prerequisites and Installation

What You Need

- **Node.js 22 or later** -- check with:

```
node -v
```

You should see output like:

```
v22.0.0
```

Anything below 22 means you need to upgrade first. Node 22+ is required because Tina4 uses the built-in `node:sqlite` module for SQLite support, which is not available in earlier versions.

- **npm** -- Node's package manager. Check with:

```
npm -v
```

You should see:

```
10.2.4
```

npm ships with Node.js. If Node.js is installed, npm is too.

- **The Tina4 CLI** -- a Rust-based binary that manages all four Tina4 frameworks:

macOS (Homebrew):

```
brew install tina4stack/tap/tina4
```

Linux / macOS (install script):

```
curl -fsSL https://raw.githubusercontent.com/tina4stack/tina4/main/install.sh | bash
```

Windows (PowerShell):

```
irm https://raw.githubusercontent.com/tina4stack/tina4/main/install.ps1 | iex
```

Verify the CLI:

```
tina4 --version
```

```
tina4 0.1.0
```

- **TypeScript and tsx** -- for running TypeScript directly:

```
npm install -g tsx
```

Verify:

```
tsx --version
```

Installing the Tina4 CLI

```
cargo install tina4
```

Or download from GitHub Releases.

The `tina4` CLI manages project scaffolding, development servers, migrations, and more across all Tina4 frameworks.

Creating a New Project

The Tina4 CLI scaffolds a new project in one command:

```
tina4 init nodejs my-store
```

`tina4 init` installs the Tina4 CLI globally (via cargo, homebrew, or direct download), then scaffolds a complete project with routes, templates, database, and configuration.

You should see:

```
Creating Tina4 project in ./my-store ...
  Detected language: Node.js (package.json)
  Created .env
  Created .env.example
  Created .gitignore
  Created src/routes/
  Created src/orm/
  Created migrations/
  Created src/seeds/
  Created src/templates/
  Created src/templates/errors/
  Created src/public/
  Created src/public/js/
  Created src/public/css/
  Created src/public/scss/
  Created src/public/images/
  Created src/public/icons/
  Created src/locales/
  Created data/
```

```
  Created logs/
  Created secrets/
  Created tests/

Project created! Next steps:
  cd my-store
  npm install
  tina4 serve
```

Install the Node.js dependencies:

```
cd my-store
npm install
```

```
added 1 package in 2s

1 package installed
```

One package. No dependency tree. No version conflicts. Just `tina4-nodejs`.

Starting the Dev Server

```
tina4 serve
```

```
 _____ _              _  _
|_   _(_)_ __   __ _| || |
  | | | | '_ \ / _` | || |_
  | | | | | | | (_| |__   _|
  |_| |_|_| |_|\__,_|  |_|

  Tina4 Node.js v3.10.3
  Server running at http://0.0.0.0:7148
  Debug mode: ON
  Database: sqlite:///data/app.db
  Press Ctrl+C to stop
```

Open your browser to `http://localhost:7148`. The Tina4 welcome page greets you.

Hit the health endpoint:

```
curl http://localhost:7148/health
```

```
{
  "status": "ok",
  "database": "connected",
  "uptime_seconds": 12,
  "version": "3.10.3",
  "framework": "tina4-nodejs"
}
```

Your project is alive.

---

3. Project Structure Walkthrough

Here is what `tina4 init` built:

```
my-store/
■■■ .env                    # Your configuration (gitignored)
■■■ .env.example            # Template for other developers
```

```
■■■ .gitignore              # Pre-configured
■■■ package.json            # npm package definition
■■■ package-lock.json       # Locked dependency versions
■■■ tsconfig.json           # TypeScript configuration
■■■ app.ts                  # Application entry point
■■■ node_modules/           # npm packages (gitignored)
■■■ migrations/             # SQL migration files (project root)
■■■ src/
■   ■■■ routes/             # Your route handlers go here
■   ■■■ orm/                # Your ORM model classes go here
■   ■■■ seeds/              # Database seed files
■   ■■■ templates/          # Frond/Twig templates
■   ■   ■■■ errors/         # Custom 404.html, 500.html
■   ■■■ public/             # Static files (CSS, JS, images)
■   ■   ■■■ js/
■   ■   ■   ■■■ frond.js    # Auto-provided JS helper library
■   ■   ■■■ css/
■   ■   ■   ■■■ tina4.css   # Built-in CSS utility framework
■   ■   ■■■ scss/
■   ■   ■■■ images/
■   ■   ■■■ icons/
■   ■■■ locales/            # Translation files
■       ■■■ en.json
■■■ data/                   # SQLite databases (gitignored)
■■■ logs/                   # Log files (gitignored)
■■■ secrets/                # JWT keys (gitignored)
■■■ tests/                  # Your test files
    ■■■ run-all.ts          # Test runner
```

Key directories:

- `src/routes/` -- Every `.ts` file here is auto-loaded at startup. Drop your route definitions here. Organize into subdirectories if you want. Tina4 also supports file-based routing: a file at `src/routes/api/users/get.ts` maps to `GET /api/users` with zero configuration.
- `src/orm/` -- Every `.ts` file here is auto-loaded. ORM model classes live here.
- `src/templates/` -- Frond (Tina4's built-in template engine -- see Chapter 4: Templates) looks here when you call `res.html()` with a template.
- `src/public/` -- Files served directly. `src/public/images/logo.png` becomes `/images/logo.png`.
- `data/` -- The default SQLite database (`app.db`) lives here. Gitignored because databases do not belong in version control.

---

4. Your First Route

Time to build an API endpoint that returns a JSON greeting.

Explicit Route Registration

Create the file `src/routes/greeting.ts`:

```
import { Router } from "tina4-nodejs";

Router.get("/api/greeting/{name}", async (req, res) => {
```

```
    const name = req.params.name;
    return res.json({
        message: `Hello, ${name}!`,
        timestamp: new Date().toISOString()
    });
});
```

Save the file. The dev server picks up the change. If live reload is off, restart with `tina4 serve`.

> ***Cross-language note.*** *Node, PHP, and Ruby read path parameters from* `req.params` / `$request->params` / `request.params`*. Python passes them as function arguments instead. Same concept, two shapes; pick the chapter that matches your runtime.*

File-Based Routing Alternative

Create the file `src/routes/api/greeting/[name]/get.ts`:

```
export default async (req, res) => {
    const name = req.params.name;
    return res.json({
        message: `Hello, ${name}!`,
        timestamp: new Date().toISOString()
    });
};
```

Both approaches produce identical results. File-based routing maps the file path to the URL. Dynamic segments go in brackets (`[name]`).

Test It

Open your browser to:

```
http://localhost:7148/api/greeting/Alice
```

You should see:

```
{
  "message": "Hello, Alice!",
  "timestamp": "2026-03-22T14:30:00.000Z"
}
```

Or use curl:

```
curl http://localhost:7148/api/greeting/Alice
```

```
{"message":"Hello, Alice!","timestamp":"2026-03-22T14:30:00.000Z"}
```

Understanding What Happened

- You created a file in `src/routes/`. Tina4 discovered it at startup.
- `Router.get("/api/greeting/{name}", ...)` registered a GET route with a path parameter `{name}`.
- A request to `/api/greeting/Alice` hit the router. Pattern matched. Handler fired.
- `req.params.name` delivered the value `"Alice"` from the URL.
- `res.json(...)` serialized the object, set `Content-Type: application/json`, and returned `200 OK`.

Adding More HTTP Methods

Add a POST endpoint. Update `src/routes/greeting.ts`:

```
import { Router } from "tina4-nodejs";

Router.get("/api/greeting/{name}", async (req, res) => {
    const name = req.params.name;
    return res.json({
        message: `Hello, ${name}!`,
        timestamp: new Date().toISOString()
    });
});

Router.post("/api/greeting", async (req, res) => {
    const name = req.body.name ?? "World";
    const language = req.body.language ?? "en";

    const greetings: Record<string, string> = {
        en: "Hello",
        es: "Hola",
        fr: "Bonjour",
        de: "Hallo",
        ja: "Konnichiwa"
    };

    const greeting = greetings[language] ?? greetings["en"];

    return res.status(201).json({
        message: `${greeting}, ${name}!`,
        language
    });
});
```

Test the POST endpoint:

```
curl -X POST http://localhost:7148/api/greeting \
  -H "Content-Type: application/json" \
  -d '{"name": "Carlos", "language": "es"}'
```

```
{"message":"Hola, Carlos!","language":"es"}
```

The HTTP status code is `201 Created` (set by `res.status(201)`).

---

5. Your First Template

Tina4 ships with **Frond** -- a zero-dependency, Twig-compatible template engine built from scratch. If you have used Twig, Jinja2, or Nunjucks, Frond will feel familiar.

Create a Base Layout

Create `src/templates/base.html`:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
```

```
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>{% block title %}My Store{% endblock %}</title>
    <link rel="stylesheet" href="/css/tina4.css">
    <style>
        body { font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, sans-serif; m
        .container { max-width: 960px; margin: 0 auto; padding: 20px; }
        .product-card { border: 1px solid #ddd; border-radius: 8px; padding: 16px; margin: 8px 0
        .product-card h3 { margin-top: 0; }
        .price { color: #2d8f2d; font-weight: bold; font-size: 1.2em; }
        .badge { display: inline-block; padding: 2px 8px; border-radius: 4px; font-size: 0.8em;
        .badge-success { background: #d4edda; color: #155724; }
        .badge-danger { background: #f8d7da; color: #721c24; }
        nav { background: #333; color: white; padding: 12px 20px; }
        nav a { color: white; text-decoration: none; margin-right: 16px; }
    </style>
</head>
<body>
    <nav>
        <a href="/">Home</a>
        <a href="/products">Products</a>
    </nav>
    <div class="container">
        {% block content %}{% endblock %}
    </div>
    <script src="/js/frond.js"></script>
</body>
</html>
```

This base layout defines two blocks (`title` and `content`) that child templates override. It includes `tina4.css` (the built-in CSS framework) and `frond.js` (the built-in JS helper library).

Create a Product Listing Page

Create `src/templates/products.html`:

```
{% extends "base.html" %}

{% block title %}Products - My Store{% endblock %}

{% block content %}
    <h1>Our Products</h1>
    <p>Showing {{ products | length }} product{{ products | length != 1 ? "s" : "" }}</p>

    {% if products | length > 0 %}
        {% for product in products %}
            <div class="product-card">
                <h3>{{ product.name }}</h3>
                <p>{{ product.description }}</p>
                <p class="price">${{ product.price | number_format(2) }}</p>
                {% if product.inStock %}
                    <span class="badge badge-success">In Stock</span>
                {% else %}
                    <span class="badge badge-danger">Out of Stock</span>
                {% endif %}
            </div>
        {% endfor %}
    {% else %}
        <p>No products available at the moment.</p>
    {% endif %}
```

```
{% endblock %}
```

Create the Route That Renders the Template

Create `src/routes/pages.ts`:

```
import { Router } from "tina4-nodejs";

Router.get("/products", async (req, res) => {
    const products = [
        {
            name: "Wireless Keyboard",
            description: "Ergonomic wireless keyboard with backlit keys.",
            price: 79.99,
            inStock: true
        },
        {
            name: "USB-C Hub",
            description: "7-port USB-C hub with HDMI, SD card reader, and Ethernet.",
            price: 49.99,
            inStock: true
        },
        {
            name: "Monitor Stand",
            description: "Adjustable aluminum monitor stand with cable management.",
            price: 129.99,
            inStock: false
        },
        {
            name: "Mechanical Mouse",
            description: "High-precision wireless mouse with 16,000 DPI sensor.",
            price: 59.99,
            inStock: true
        }
    ];

    return res.render("products.html", { products });
});
```

File-based routing works here too. Create `src/routes/products/get.ts`:

```
export const template = "products.html";

export default async (req, res) => {
    return {
        products: [
            { name: "Wireless Keyboard", description: "Ergonomic wireless keyboard.", price: 79.
            // ... more products
        ]
    };
};
```

Export a `template` constant and Tina4 renders it with the data returned from the handler.

See It in the Browser

Open `http://localhost:7148/products`. You should see:

- A dark navigation bar at the top with "Home" and "Products" links

- The heading "Our Products"
- A subheading showing "Showing 4 products"
- Four product cards, each with a name, description, price, and stock badge
- The "Monitor Stand" card wears a red "Out of Stock" badge
- The other three wear green "In Stock" badges

How Template Rendering Works

- `res.render("products.html", { products })` tells Frond to render `src/templates/products.html` with the given data.
- Frond sees `{% extends "base.html" %}` and loads the base template.
- The `{% block content %}` in `products.html` replaces the same block in `base.html`.
- `{{ product.name }}` outputs the value, auto-escaped for HTML safety.
- `{{ product.price | number_format(2) }}` formats the number with 2 decimal places.
- `{% for product in products %}` loops through the array.
- `{% if product.inStock %}` conditionally renders the stock badge.
- `{{ products | length }}` returns the count of items in the array.

About tina4css

The `tina4.css` file in the base template is Tina4's built-in CSS utility framework. Layout utilities, typography, and common UI patterns -- without Bootstrap or Tailwind. Auto-provided at scaffolding time. No separate download needed.

---

6. Understanding .env

Open the `.env` file at the root of your project:

```
TINA4_DEBUG=true
```

That is likely all you see. The scaffold creates a minimal `.env` with debug mode enabled. Everything else uses defaults.

The important defaults for development:

Variable	Default Value	What It Means
`TINA4_PORT`	`7148`	Server runs on port 7148
`TINA4_DATABASE_URL`	`sqlite:///data/app.db`	SQLite database in the `data/` directory
`TINA4_LOG_LEVEL`	`ALL`	All log messages are output

`TINA4_CORS_ORIGINS`	*(empty)*	No cross-origin browser requests; list your front end's origin to allow it
`TINA4_RATE_LIMIT`	`60`	60 requests per minute per IP

Log levels control how much output Tina4 produces:

Level	Behaviour
`ALL` / `DEBUG`	Full verbose output. DevReload active (live-reload, error overlay).
`INFO`	Standard logging. Startup messages, request summaries.
`WARNING`	Warnings and errors only.
`ERROR`	Errors only. Minimal output.

Set `TINA4_LOG_LEVEL=DEBUG` during development for maximum visibility. Use `WARNING` or `ERROR` in production.

To change the port, use the CLI flag or `.env`:

```
tina4 serve --port 8080
```

Or add it to your `.env` file:

```
TINA4_DEBUG=true
TINA4_PORT=8080
```

Restart the server (`Ctrl+C`, then `tina4 serve`). It now runs on port 8080.

How port resolution works: The Rust CLI (`tina4 serve`) determines the port using this priority order:

- **CLI flag** (highest priority): `tina4 serve --port 8080`
- `.env` **file**: `TINA4_PORT=8080`
- **Environment variable**: `PORT=8080`
- **Framework default** (PHP: 7145, Python: 7146, Ruby: 7147, Node.js: 7148)

The CLI reads your `.env` file and checks for `TINA4_PORT` (and falls back to `PORT`). The resolved port is passed to the Node.js server. All three methods work -- use whichever fits your workflow.

For the complete `.env` reference with all 68 variables, see Environment Variables.

---

7. The Dev Dashboard

With `TINA4_DEBUG=true` in your `.env`, Tina4 provides a built-in development dashboard. No additional environment variables are needed.

Restart the server and navigate to:

```
http://localhost:7148/__dev
```

The dashboard reveals:

- **System Overview** -- framework version, Node.js version, uptime, memory usage, database status
- **Request Inspector** -- recent HTTP requests with method, path, status, duration, and request ID. Click any request to see full headers, body, database queries, and template renders.
- **Error Log** -- unhandled exceptions with stack traces and occurrence counts
- **Queue Manager** -- queue status (pending, reserved, failed, dead-letter messages)
- **WebSocket Monitor** -- active WebSocket connections with metadata
- **Routes** -- all registered routes with their methods, paths, and middleware

The dev dashboard shows you what your application is doing without littering your code with `console.log` statements.

When you visit any HTML page (like `/products`), a **debug overlay** appears -- a toolbar at the bottom of the page showing:

- Request details (method, URL, duration)
- Database queries executed (with timing)
- Template renders (with timing)
- Session data
- Recent log entries

This overlay lives only in debug mode. Production never sees it.

---

8. The app.ts Entry Point

The `app.ts` file is the entry point:

```
import { startServer } from "tina4-nodejs";

startServer();
```

That is the entire file. Tina4 discovers routes, models, and templates from the `src/` directory. You register nothing manually.

The standard way to start the server is with the CLI:

```
tina4 serve

 _____ _             _  _
|_   _(_)_ __   __ _| || |
  | | | | '_ \ / _` | || |_
  | | | | | | | (_| |__   _|
  |_| |_|_| |_|\__,_|  |_|

```

```
Tina4 Node.js v3.10.3
Server running at http://0.0.0.0:7148
```

The CLI adds live reload and other development features. For direct Node.js execution (advanced usage), see Chapter 31: CLI.

---

9. Manual Setup (No CLI)

The `tina4` CLI scaffolds everything for you. But if you start from an empty folder (just Node.js and npm), here is the minimum you need.

Step 1: Initialize and Install

```
npm init -y
npm install tina4-nodejs typescript tsx
```

Step 2: Create `app.ts`

This is the entry point. Create a file called `app.ts` in your project root:

```
import { startServer } from "@tina4/core";

const port = parseInt(process.env.PORT || "7148", 10);
const host = process.env.HOST || "0.0.0.0";
startServer({ port, host });
```

Three lines of real code. `startServer` boots the framework, scans for routes, and starts listening.

Step 3: Create `tsconfig.json`

```
{
  "compilerOptions": {
    "target": "ES2022",
    "module": "ESNext",
    "moduleResolution": "bundler",
    "strict": true,
    "esModuleInterop": true,
    "outDir": "dist"
  },
  "include": ["src/**/*", "app.ts"]
}
```

Step 4: Create the Folder Structure

Tina4 expects this layout:

```
my-project/
■■■ app.ts
■■■ tsconfig.json
■■■ package.json
■■■ .env
■■■ src/
    ■■■ routes/       # Route files go here
    ■■■ templates/    # Twig templates go here
    ■■■ public/       # Static files (CSS, JS, images)
```

Create the directories:

```
mkdir -p src/routes src/templates src/public
```

Step 5: Create `.env`

```
TINA4_DEBUG=true
```

Step 6: Run It

```
tina4 serve
```

The server starts on `http://localhost:7148`. You should see the Tina4 welcome page. From here, add route files in `src/routes/` and templates in `src/templates/`, the same way as a CLI-scaffolded project.

> ***Note:*** *Tina4 Node.js refuses to start without the Rust CLI. To bypass it (for example inside a Docker image that already wraps the framework) set* `TINA4_OVERRIDE_CLIENT=true` *in* `.env` *and run* `npx tsx app.ts` *directly.*

---

10. Request & Response Fundamentals

Before jumping into the exercises, let's consolidate how route handlers work in Tina4 Node.js. Every handler receives two arguments: `req` (what the client sent) and `res` (what you send back). Here is the complete picture.

Reading Query Parameters

Query parameters are the key-value pairs after the `?` in a URL. Access them through `req.query`:

```
// URL: /api/search?q=laptop&page=2
req.query.q          // "laptop"
req.query.page       // "2" (always a string)
req.query.sort ?? "name"  // "name" (default -- param was not sent)
```

Reading URL Path Parameters

Route patterns like `/users/{id}` capture segments of the URL. Access them through `req.params`:

```
import { Router } from "tina4-nodejs";

Router.get("/users/{id:int}/posts/{slug}", async (req, res) => {
    const id = req.params.id;      // 5 (number, because of :int)
    const slug = req.params.slug;  // "hello-world" (string)
    return res.json({ user_id: id, slug: slug });
});
```

The `{id:int}` syntax tells Tina4 to convert the value to a number. Without `:int`, it stays a string.

Reading the Request Body

POST, PUT, and PATCH requests carry a body. Tina4 parses JSON bodies into an object automatically (as long as the client sends `Content-Type: application/json`):

```
Router.post("/api/items", async (req, res) => {
    const name = req.body.name ?? "";
    const price = req.body.price ?? 0;
    return res.json({ received_name: name, received_price: price });
});
```

Reading Headers

Headers are available as an object. In Node.js, header names are normalized to lowercase:

```
const contentType = req.headers["content-type"] ?? "not set";
const authToken = req.headers["authorization"] ?? "";
const custom = req.headers["x-custom-header"] ?? "";
```

Sending JSON Responses

`res.json()` converts an object to JSON and sets the correct `Content-Type`. Chain with `res.status()` for a custom status code:

```
return res.json({ id: 1, name: "Widget" });               // 200 OK (default)
return res.status(201).json({ id: 1, name: "Widget" });    // 201 Created
return res.status(404).json({ error: "Not found" });       // 404 Not Found
```

Sending HTML / Template Responses

`res.html()` renders a Frond template from `src/templates/` when given a filename and data:

```
return res.render("products.html", { products: productList, title: "Our Products" });
```

For raw HTML without a template file:

```
return res.html("<h1>Hello</h1><p>This works too.</p>");
```

Status Codes

The most common status codes you will use:

Code	Meaning	When to Use
`200`	OK	Successful GET (default)
`201`	Created	Successful POST that created something
`400`	Bad Request	Client sent invalid input
`404`	Not Found	Resource does not exist
`500`	Internal Server Error	Something broke on the server

Worked Example: A Complete Route File

Here is a full route file that ties everything together. It builds a small book lookup API with query parameters, path parameters, JSON responses, and proper status codes. Read through it before attempting the exercises -- it is your reference.

Create `src/routes/books.ts`:

```
import { Router } from "tina4-nodejs";

// In-memory data store
const books = [
    { id: 1, title: "Dune", author: "Frank Herbert", year: 1965 },
    { id: 2, title: "Neuromancer", author: "William Gibson", year: 1984 },
    { id: 3, title: "Snow Crash", author: "Neal Stephenson", year: 1992 }
];

Router.get("/api/books", async (req, res) => {
    // List all books. Supports ?author= filter and ?sort=year.
    const author = (req.query.author ?? "") as string;
    const sortBy = (req.query.sort ?? "") as string;

    let result = [...books];

    // Filter by author if the query param is present
    if (author) {
        result = result.filter(b =>
            b.author.toLowerCase().includes(author.toLowerCase())
        );
    }

    // Sort by year if requested
    if (sortBy === "year") {
        result.sort((a, b) => a.year - b.year);
    }

    return res.json({ books: result, count: result.length });
});

Router.get("/api/books/{id:int}", async (req, res) => {
    // Get a single book by ID. Returns 404 if not found.
    const id = req.params.id;
    const book = books.find(b => b.id === id);

    if (!book) {
        return res.status(404).json({ error: `Book with id ${id} not found` });
    }

    return res.json(book);
});

Router.post("/api/books", async (req, res) => {
    // Create a new book from the JSON body. Returns 201 on success.
    const title = req.body.title ?? "";
    const author = req.body.author ?? "";
    const year = req.body.year ?? 0;

    if (!title || !author) {
        return res.status(400).json({ error: "title and author are required" });
    }

    const newBook = {
        id: Math.max(...books.map(b => b.id)) + 1,
        title,
        author,
        year
    };
```

```
    books.push(newBook);

    return res.status(201).json(newBook);
});
```

Test it:

```
# List all books
curl http://localhost:7148/api/books

# Filter by author
curl "http://localhost:7148/api/books?author=gibson"

# Sort by year
curl "http://localhost:7148/api/books?sort=year"

# Get a single book
curl http://localhost:7148/api/books/2

# Get a book that does not exist (returns 404)
curl http://localhost:7148/api/books/99

# Create a new book
curl -X POST http://localhost:7148/api/books \
  -H "Content-Type: application/json" \
  -d '{"title": "Foundation", "author": "Isaac Asimov", "year": 1951}'
```

This example covers every building block the exercises use: reading query parameters, reading path parameters, reading the request body, returning JSON with different status codes, and handling missing data. Refer back to it as you work through the exercises below.

---

11. Exercise: Greeting API + Product List Template

Build the following two features from scratch, without looking at the examples above.

Exercise Part A: Greeting API

Create an API endpoint at `GET /api/greet` that:

- Accepts a query parameter `name` (e.g., `/api/greet?name=Sarah`)
- If `name` is missing, defaults to `"Stranger"`
- Returns JSON like:

```
{
  "greeting": "Welcome, Sarah!",
  "time_of_day": "afternoon"
}
```

- The `time_of_day` should be calculated from the server's current hour:
- 5:00 - 11:59 = "morning"
- 12:00 - 16:59 = "afternoon"
- 17:00 - 20:59 = "evening"

- 21:00 - 4:59 = "night"

Test your endpoint with:

```
curl "http://localhost:7148/api/greet?name=Sarah"
curl "http://localhost:7148/api/greet"
```

Exercise Part B: Product List Page

Create a page at `GET /store` that:

- Displays a list of at least 5 products (hardcoded for now)
- Each product has: name, category, price, and a boolean `featured` flag
- Featured products should be highlighted (different background color, border, or badge)
- The page should show the total number of products and the number of featured products
- Use template inheritance -- create a layout template and a page template that extends it
- Include `tina4.css` and `frond.js`

Your products data should look like this in your route handler:

```
const products = [
    { name: "Espresso Machine", category: "Kitchen", price: 299.99, featured: true },
    { name: "Yoga Mat", category: "Fitness", price: 29.99, featured: false },
    { name: "Standing Desk", category: "Office", price: 549.99, featured: true },
    { name: "Noise-Canceling Headphones", category: "Electronics", price: 199.99, featured: true
    { name: "Water Bottle", category: "Fitness", price: 24.99, featured: false }
];
```

Expected browser output:

- A page titled "Our Store"
- Text showing "5 products, 3 featured"
- A list of product cards with name, category, price, and a "Featured" badge on the highlighted items
- Featured products have a distinct visual style (your choice -- different border color, background, star icon, etc.)

---

12. Solutions

Solution A: Greeting API

Create `src/routes/greet.ts`:

```
import { Router } from "tina4-nodejs";

Router.get("/api/greet", async (req, res) => {
    const name = req.query.name ?? "Stranger";
    const hour = new Date().getHours();

    let timeOfDay: string;
    if (hour >= 5 && hour < 12) {
```

```
        timeOfDay = "morning";
    } else if (hour >= 12 && hour < 17) {
        timeOfDay = "afternoon";
    } else if (hour >= 17 && hour < 21) {
        timeOfDay = "evening";
    } else {
        timeOfDay = "night";
    }

    return res.json({
        greeting: `Welcome, ${name}!`,
        time_of_day: timeOfDay
    });
});
```

Test:

```
curl "http://localhost:7148/api/greet?name=Sarah"

{"greeting":"Welcome, Sarah!","time_of_day":"afternoon"}

curl "http://localhost:7148/api/greet"

{"greeting":"Welcome, Stranger!","time_of_day":"afternoon"}
```

Solution B: Product List Page

Create `src/templates/store-layout.html`:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>{% block title %}Store{% endblock %}</title>
    <link rel="stylesheet" href="/css/tina4.css">
    <style>
        body { font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, sans-serif; m
        .container { max-width: 960px; margin: 0 auto; padding: 20px; }
        header { background: #1a1a2e; color: white; padding: 16px 20px; }
        header h1 { margin: 0; }
        .stats { color: #888; margin: 8px 0 20px; }
        .product-grid { display: grid; gap: 16px; }
        .product-card { background: white; border: 2px solid #e0e0e0; border-radius: 8px; paddin
        .product-card.featured { border-color: #ffc107; background: #fffdf0; }
        .product-name { font-size: 1.2em; font-weight: bold; margin: 0 0 4px; }
        .product-category { color: #666; font-size: 0.9em; }
        .product-price { color: #2d8f2d; font-weight: bold; font-size: 1.1em; margin-top: 8px; }
        .featured-badge { background: #ffc107; color: #333; padding: 2px 8px; border-radius: 4px
    </style>
</head>
<body>
    <header>
        <h1>{% block header %}Store{% endblock %}</h1>
    </header>
    <div class="container">
        {% block content %}{% endblock %}
    </div>
    <script src="/js/frond.js"></script>
</body>
</html>
```

Create `src/templates/store.html`:

```
{% extends "store-layout.html" %}

{% block title %}Our Store{% endblock %}
{% block header %}Our Store{% endblock %}

{% block content %}
    <p class="stats">{{ products | length }} products, {{ featured_count }} featured</p>

    <div class="product-grid">
        {% for product in products %}
            <div class="product-card{{ product.featured ? ' featured' : '' }}">
                <p class="product-name">
                    {{ product.name }}
                    {% if product.featured %}
                        <span class="featured-badge">Featured</span>
                    {% endif %}
                </p>
                <p class="product-category">{{ product.category }}</p>
                <p class="product-price">${{ product.price | number_format(2) }}</p>
            </div>
        {% endfor %}
    </div>
{% endblock %}
```

Create `src/routes/store.ts`:

```
import { Router } from "tina4-nodejs";

Router.get("/store", async (req, res) => {
    const products = [
        { name: "Espresso Machine", category: "Kitchen", price: 299.99, featured: true },
        { name: "Yoga Mat", category: "Fitness", price: 29.99, featured: false },
        { name: "Standing Desk", category: "Office", price: 549.99, featured: true },
        { name: "Noise-Canceling Headphones", category: "Electronics", price: 199.99, featured:
        { name: "Water Bottle", category: "Fitness", price: 24.99, featured: false }
    ];

    const featuredCount = products.filter(p => p.featured).length;

    return res.render("store.html", {
        products,
        featured_count: featuredCount
    });
});
```

Open `http://localhost:7148/store` in your browser. You should see:

- A dark header reading "Our Store"
- Text showing "5 products, 3 featured"
- Five product cards in a grid
- Three cards (Espresso Machine, Standing Desk, Noise-Canceling Headphones) have a yellow border, light yellow background, and a "Featured" badge
- Two cards (Yoga Mat, Water Bottle) have a standard white background with gray border
- Each card shows the product name, category, and price formatted with two decimal places

---

13. Gotchas

1. File not auto-discovered

Problem: You created a route file but nothing happens when you visit the URL.

Cause: The file is not in `src/routes/`. It must be inside `src/routes/` (or a subdirectory of it), and the file must end with `.ts`.

Fix: Move the file to `src/routes/your-file.ts` and restart the server.

2. "Module not found" errors

Problem: `Cannot find module 'tina4-nodejs'` or similar.

Cause: Missing npm install or incorrect import path.

Fix: Run `npm install` in your project root. Make sure your import uses the exact package name: `import { Router } from "tina4-nodejs"`.

3. JSON response shows HTML

Problem: Your JSON endpoint returns HTML instead of JSON.

Cause: You returned a string instead of using `res.json()`. A plain string tells Tina4 to treat it as HTML.

Fix: Use `res.json(data)` for JSON endpoints. `console.log()` is not a response mechanism.

4. Template not found

Problem: `Template "my-page.html" not found` error.

Cause: The template file is not in `src/templates/`, or there is a typo in the filename.

Fix: Check that the file exists at `src/templates/my-page.html`. The name in `res.html()` is relative to `src/templates/`.

5. Port already in use

Problem: `Error: Address already in use (port 7148)`

Cause: Another process is occupying port 7148.

Fix: Stop the other process, or change the port:

```
TINA4_PORT=8080
```

Or use the CLI flag: `tina4 serve --port 8080`.

6. Changes not reflected

Problem: You edited a file but the browser shows the old version.

Cause: Live reload may not be active. Browser caching can serve stale versions.

Fix: Hard-refresh the browser (`Ctrl+Shift+R` or `Cmd+Shift+R`). If that fails, restart the dev server with `Ctrl+C` and `tina4 serve`.

7. .env not loaded

Problem: Environment variables have no effect.

Cause: The `.env` file must be at the project root (same directory as `package.json`).

Fix: Move `.env` to the project root.

Routing

1. How Routing Works in Tina4

Every web application maps URLs to code. A browser requests `/products`. The framework finds the handler for `/products`. Runs it. Sends back the result. That mapping is routing.

In Tina4 Node.js, you define routes in TypeScript files inside `src/routes/`. Every `.ts` file in that directory -- and its subdirectories -- is auto-loaded when the server starts. No registration files. No central config. Drop a file in. It works.

Tina4 supports two routing styles: **explicit registration** with the `Router` class, and **file-based routing** where the file path becomes the URL.

The simplest possible route using explicit registration:

```
import { Router } from "tina4-nodejs";

Router.get("/hello", async (req, res) => {
    return res.json({ message: "Hello, World!" });
});
```

Save that as `src/routes/hello.ts`, start the server with `tina4 serve`, and visit `http://localhost:7148/hello`:

```
{"message":"Hello, World!"}
```

The same thing using file-based routing. Create `src/routes/hello/get.ts`:

```
export default async (req, res) => {
    return res.json({ message: "Hello, World!" });
};
```

Both approaches produce identical results. Use whichever you prefer -- or mix them.

---

2. HTTP Methods

Tina4 supports all five standard HTTP methods. Each one has a static method on the `Router` class:

```
import { Router } from "tina4-nodejs";

Router.get("/products", async (req, res) => {
    return res.json({ action: "list all products" });
});

Router.post("/products", async (req, res) => {
    return res.status(201).json({ action: "create a product" });
});

Router.put("/products/{id}", async (req, res) => {
    const id = req.params.id;
    return res.json({ action: `replace product ${id}` });
});
```

```
Router.patch("/products/{id}", async (req, res) => {
    const id = req.params.id;
    return res.json({ action: `update product ${id}` });
});

Router.delete("/products/{id}", async (req, res) => {
    const id = req.params.id;
    return res.json({ action: `delete product ${id}` });
});
```

For file-based routing, the HTTP method is the filename:

```
src/routes/products/get.ts      → GET /products
src/routes/products/post.ts     → POST /products
src/routes/products/[id]/put.ts → PUT /products/{id}
```

Test each one:

```
curl http://localhost:7148/products
```

```
{"action":"list all products"}
```

```
curl -X POST http://localhost:7148/products \
  -H "Content-Type: application/json" \
  -d '{"name": "Widget"}'
```

```
{"action":"create a product"}
```

```
curl -X PUT http://localhost:7148/products/42
```

```
{"action":"replace product 42"}
```

```
curl -X PATCH http://localhost:7148/products/42
```

```
{"action":"update product 42"}
```

```
curl -X DELETE http://localhost:7148/products/42
```

```
{"action":"delete product 42"}
```

`GET` reads. `POST` creates. `PUT` replaces. `PATCH` patches. `DELETE` removes. REST convention. Predictable API.

HEAD and OPTIONS: automatic, no boilerplate

Tina4 handles `HEAD` and `OPTIONS` for you. **You don't register them.** They follow from your `router.get(...)` / `router.post(...)` / etc. routes:

- `HEAD` is identical to `GET` except the response carries no body (RFC 9110 §9.3.2). Every `GET` route automatically responds to `HEAD`: the framework strips the response body on the way out and preserves `Content-Length` pointing at the byte count the equivalent `GET` would have sent. Cache validators, link checkers, and uptime probes work without you writing anything.
- `OPTIONS` on any registered path returns `204 No Content` with an `Allow:` header listing every method the path supports (RFC 9110 §9.3.7).
- **Wrong method on an existing path** returns `405 Method Not Allowed` with the same `Allow:` header (RFC 9110 §15.5.6 + §10.2.1). Sending `PUT` to a `GET`-only route used to return `404`, which was semantically wrong and confusing for link checkers. Now you get a real `405`.
- `TRACE` **and** `CONNECT` are rejected with `405` (security default for origin servers).

```
# HEAD on a GET route - same headers, empty body
curl -sI http://localhost:7148/products
# HTTP/1.1 200 OK
# Content-Type: application/json
# Content-Length: 33

# OPTIONS - discover what the path supports
curl -sI -X OPTIONS http://localhost:7148/products
# HTTP/1.1 204 No Content
# Allow: GET, POST, HEAD, OPTIONS

# Wrong method - clear 405 with Allow header
curl -sI -X PUT http://localhost:7148/products
# HTTP/1.1 405 Method Not Allowed
# Allow: GET, POST, HEAD, OPTIONS
```

Explicit `router.head()` and `router.options()` registration

The automatic behaviour is enough for 95% of apps. When you need custom HEAD or OPTIONS handlers, register them explicitly:

```
import { Router } from "@tina4/core";
const router = new Router();

// HEAD handler that doesn't run the full GET body - useful for
// expensive endpoints where the client only needs to check existence
// or read validators (ETag, Last-Modified).
router.head("/expensive/{id}", async (req, res) => {
  res.raw.setHeader("ETag", computeEtag(req.params.id));
  return res({});
});

// OPTIONS handler that returns more than just Allow - for example
// a discovery endpoint.
router.options("/api/discovery", async (_req, res) => {
  return res({ version: "v1", endpoints: [] });
});
```

The framework still strips the response body for `HEAD` handlers (RFC 9110 §9.3.2 is a MUST), so you can't accidentally leak content even if your handler returns a body.

---

3. Path Parameters

Path parameters capture values from the URL. Wrap the parameter name in curly braces:

```
import { Router } from "tina4-nodejs";

Router.get("/users/{id}/posts/{postId}", async (req, res) => {
    const userId = req.params.id;
    const postId = req.params.postId;

    return res.json({
        user_id: userId,
        post_id: postId
    });
});
```

For file-based routing, wrap the parameter name in brackets:

```
src/routes/users/[id]/posts/[postId]/get.ts
```

```
curl http://localhost:7148/users/5/posts/99
```

```
{"user_id":"5","post_id":"99"}
```

Notice `user_id` came back as the string `"5"`, not the integer `5`. Path parameters are strings by default.

> ***Auto-casting:*** *Tina4 automatically casts path parameter values that are purely numeric to numbers. For example, requesting `/users/42/posts/99` will give you `req.params.id` as the number `42` and `req.params.postId` as the number `99` -- no explicit `:int` type hint required. The `:int` type hint adds validation (rejecting non-numeric values with a 404), but the auto-casting happens regardless.*

Typed Parameters

Enforce a type by adding a colon and the type after the parameter name:

```
import { Router } from "tina4-nodejs";

Router.get("/orders/{id:int}", async (req, res) => {
    const id = req.params.id; // This is now a number
    return res.json({
        order_id: id,
        type: typeof id
    });
});
```

```
curl http://localhost:7148/orders/42
```

```
{"order_id":42,"type":"number"}
```

Pass a non-integer value and the route refuses to match. You get a 404:

```
curl http://localhost:7148/orders/abc
```

```
{"error":"Not found","path":"/orders/abc","status":404}
```

Supported types:

Type	Matches	Auto-cast	Example
`int`	Digits only	Number	`{id:int}` matches `42` but not `abc`
`float`	Decimal numbers	Number	`{price:float}` matches `19.99`
`path`	All remaining path segments (catch-all)	String	`{slug:path}` matches `docs/api/auth`
`alpha`	Letters only	String	`{slug:alpha}` matches `hello` but not `hello123`

`alphanumeric`	Letters and digits	String	`{code:alphanumeric}` matches `abc123`

The `{name}` form (no type) matches any single path segment and returns it as a string.

Typed Parameters in Action

Here is a complete example showing the most commonly used typed parameters together:

```
import { Router } from "tina4-nodejs";

// Integer parameter -- only digits match, auto-cast to number
Router.get("/products/{id:int}", async (req, res) => {
    const id = req.params.id; // number, e.g. 42
    return res.json({
        product_id: id,
        type: typeof id
    });
});

// Float parameter -- decimal numbers, auto-cast to number
Router.get("/products/{id:int}/price/{price:float}", async (req, res) => {
    const id = req.params.id;
    const price = req.params.price;
    return res.json({
        product_id: id,
        price,
        type: typeof price
    });
});

// Path parameter -- catch-all, captures remaining segments as a string
Router.get("/files/{filepath:path}", async (req, res) => {
    const filepath = req.params.filepath;
    // filepath could be "images/photos/cat.jpg"
    return res.json({
        filepath,
        type: typeof filepath
    });
});
```

```
# Integer route -- matches digits, returns a number
curl http://localhost:7148/products/42
```

```
{"product_id":42,"type":"number"}
```

```
# Integer route -- non-integer gives a 404
curl http://localhost:7148/products/abc
```

```
{"error":"Not found","path":"/products/abc","status":404}
```

```
# Path catch-all -- captures everything after /files/
curl http://localhost:7148/files/images/photos/cat.jpg
```

```
{"filepath":"images/photos/cat.jpg","type":"string"}
```

The `:int` and `:float` types act as both a constraint and a converter. If the URL segment does not match the expected pattern, the route is skipped entirely and Tina4 moves on to the next

registered route (or returns 404 if nothing matches). The `:path` type is greedy -- it consumes all remaining segments, making it ideal for file paths and documentation URLs.

---

4. Query Parameters

Query parameters are the key-value pairs after the `?` in a URL. Access them via `req.query`:

```
import { Router } from "tina4-nodejs";

Router.get("/search", async (req, res) => {
    const q = req.query.q ?? "";
    const page = parseInt(req.query.page ?? "1", 10);
    const limit = parseInt(req.query.limit ?? "10", 10);

    return res.json({
        query: q,
        page,
        limit,
        offset: (page - 1) * limit
    });
});
```

```
curl "http://localhost:7148/search?q=keyboard&page=2&limit=20"
```

```
{"query":"keyboard","page":2,"limit":20,"offset":20}
```

A missing query parameter yields `undefined` from `req.query.key`. The nullish coalescing operator (`??`) provides defaults.

---

5. Route Groups

A set of routes sharing a common prefix belongs in a group. `Router.group()` eliminates repetition:

```
import { Router } from "tina4-nodejs";

Router.group("/api/v1", (group) => {

    group.get("/users", async (req, res) => {
        return res.json({ users: [] });
    });

    group.get("/users/{id:int}", async (req, res) => {
        const id = req.params.id;
        return res.json({ user: { id, name: "Alice" } });
    });

    group.post("/users", async (req, res) => {
        return res.status(201).json({ created: true });
    });

    group.get("/products", async (req, res) => {
        return res.json({ products: [] });
    });
```

```
});
```

The routes register as `/api/v1/users`, `/api/v1/users/{id}`, and `/api/v1/products`. Short paths inside the group. Tina4 prepends the prefix.

```
curl http://localhost:7148/api/v1/users
```

```
{"users":[]}
```

Groups nest:

```
import { Router } from "tina4-nodejs";

Router.group("/api", (api) => {
    api.group("/v1", (v1) => {
        v1.get("/status", async (req, res) => {
            return res.json({ version: "1.0" });
        });
    });

    api.group("/v2", (v2) => {
        v2.get("/status", async (req, res) => {
            return res.json({ version: "2.0" });
        });
    });
});
```

```
curl http://localhost:7148/api/v1/status
```

```
{"version":"1.0"}
```

```
curl http://localhost:7148/api/v2/status
```

```
{"version":"2.0"}
```

---

6. Middleware on Routes

Middleware is code that runs before (or after) your route handler. Authentication. Logging. Rate limiting. Input validation. Anything that belongs on multiple routes.

Middleware on a Single Route

Pass middleware as the third argument to any route method:

```
import { Router } from "tina4-nodejs";

function logRequest(req, res, next) {
    const start = Date.now();
    console.log(`[${new Date().toISOString()}] ${req.method} ${req.path}`);
    next();
    const duration = Date.now() - start;
    console.log(`  Completed in ${duration}ms`);
}

Router.get("/api/data", async (req, res) => {
    return res.json({ data: [1, 2, 3] });
}, [logRequest]);
```

The middleware function receives `req`, `res`, and `next`. Call `next()` to continue to the next middleware or route handler. Skip the call and the handler never runs -- the chain stops. A locked gate for unauthorized requests.

Blocking Middleware

Middleware that checks for an API key:

```
import { Router } from "tina4-nodejs";

function requireApiKey(req, res, next) {
    const apiKey = req.headers["x-api-key"] ?? "";

    if (apiKey !== "my-secret-key") {
        res({ error: "Invalid API key" }, 401);
        return;
    }

    next();
}

Router.get("/api/secret", async (req, res) => {
    return res.json({ secret: "The answer is 42" });
}, [requireApiKey]);
```

```
curl http://localhost:7148/api/secret
```

```
{"error":"Invalid API key"}
```

```
curl http://localhost:7148/api/secret -H "X-API-Key: my-secret-key"
```

```
{"secret":"The answer is 42"}
```

Middleware on a Group

Apply middleware to an entire group:

```
import { Router } from "tina4-nodejs";

Router.group("/api/admin", (group) => {

    group.get("/dashboard", async (req, res) => {
        return res.json({ page: "admin dashboard" });
    });

    group.get("/users", async (req, res) => {
        return res.json({ page: "user management" });
    });

}, [requireAuth]);
```

Multiple Middleware

Chain multiple middleware by passing an array:

```
Router.get("/api/important", async (req, res) => {
    return res.json({ data: "important stuff" });
}, [logRequest, requireApiKey, requireAuth]);
```

Middleware runs in order: `logRequest` first, then `requireApiKey`, then `requireAuth`, then the route handler.

---

7. Route Decorators: @noauth and @secured

Tina4 provides two special decorators for controlling authentication on routes.

@noauth -- Public Routes

When your application has global authentication middleware, `@noauth` marks specific routes as public:

```
import { Router } from "tina4-nodejs";

/**
 * @noauth
 */
Router.get("/api/public/info", async (req, res) => {
    return res.json({
        app: "My Store",
        version: "1.0.0"
    });
});
```

The `@noauth` decorator tells Tina4 to skip authentication checks for this route, even with global auth middleware configured in `.env`.

@secured -- Protected GET Routes

The `@secured` annotation marks a GET route as requiring authentication:

```
import { Router } from "tina4-nodejs";

/**
 * @secured
 */
Router.get("/api/profile", async (req, res) => {
    return res.json({
        user: req.user
    });
});
```

By default, `POST`, `PUT`, `PATCH`, and `DELETE` routes are considered secured. `GET` routes are not -- they are public unless you add `@secured`.

---

8. Route Chaining: .secure() and .cache()

Routes return a chainable object. Two methods you can call on any route: `.secure()` and `.cache()`.

.secure()

`.secure()` requires a valid bearer token in the `Authorization` header. If the token is missing or invalid, the route returns `401 Unauthorized` without ever reaching your handler:

```
import { Router } from "tina4-nodejs";

Router.get("/api/account", async (req, res) => {
    return res.json({ account: req.user });
}).secure();
```

```
curl http://localhost:7148/api/account
# 401 Unauthorized

curl http://localhost:7148/api/account -H "Authorization: Bearer eyJhbGci..."
# 200 OK
```

.cache()

`.cache()` enables response caching for the route. Once the handler runs and produces a response, subsequent requests to the same URL return the cached result without re-executing the handler:

```
Router.get("/api/catalog", async (req, res) => {
    // Expensive database query
    return res.json({ products });
}).cache();
```

Chaining Both

Chain `.secure()` and `.cache()` together:

```
Router.get("/api/data", async (req, res) => {
    return res.json({ data });
}).secure().cache();
```

This route requires a bearer token and caches the response. Order does not matter -- `.cache().secure()` produces the same result.

---

9. Wildcard and Catch-All Routes

Wildcard Routes

Use `*` at the end of a path to match anything after it:

```
import { Router } from "tina4-nodejs";

Router.get("/docs/*", async (req, res) => {
    const path = req.params["*"] ?? "";
    return res.json({
        section: "docs",
        path
    });
});
```

```
curl http://localhost:7148/docs/getting-started
```

```
{"section":"docs","path":"getting-started"}

curl http://localhost:7148/docs/api/authentication/jwt

{"section":"docs","path":"api/authentication/jwt"}
```

Catch-All Route (Custom 404)

Register a catch-all to handle any unmatched URL:

```
import { Router } from "tina4-nodejs";

Router.get("/*", async (req, res) => {
    return res.status(404).json({
        error: "Page not found",
        path: req.path
    });
});
```

Define this route last. Tina4 matches routes in registration order -- first match wins.

You can also create a custom 404 page by placing a template at `src/templates/errors/404.html`:

```
{% extends "base.html" %}

{% block title %}Not Found{% endblock %}

{% block content %}
    <h1>404 - Page Not Found</h1>
    <p>The page you are looking for does not exist.</p>
    <a href="/">Go back home</a>
{% endblock %}
```

Tina4 uses this template for any unmatched route when the file exists.

---

10. Route Listing via CLI

As your application grows, you need visibility into all registered routes. The CLI provides it:

```
tina4 routes

Method   Path                          Middleware          Auth
------   ----                          ----------          ----
GET      /hello                        -                   public
GET      /products                     -                   public
POST     /products                     -                   secured
PUT      /products/{id}                -                   secured
PATCH    /products/{id}                -                   secured
DELETE   /products/{id}                -                   secured
GET      /api/v1/users                 -                   public
GET      /api/v1/users/{id:int}        -                   public
POST     /api/v1/users                 -                   secured
GET      /api/admin/dashboard          requireAuth         public
GET      /api/admin/users              requireAuth         public
GET      /api/public/info              -                   @noauth
GET      /api/profile                  -                   @secured
GET      /search                       -                   public
```

```
GET      /docs/*                    -                  public
```

---

11. Organizing Route Files

Organize route files however you want. Tina4 loads every `.ts` file in `src/routes/` recursively. Two common patterns:

Pattern 1: One File Per Resource

```
src/routes/
■■■ products.ts     # All product routes
■■■ users.ts        # All user routes
■■■ orders.ts       # All order routes
■■■ pages.ts        # HTML page routes
```

Pattern 2: File-Based Routing by Feature

```
src/routes/
■■■ api/
■   ■■■ products/
■   ■   ■■■ get.ts          # GET /api/products
■   ■   ■■■ post.ts         # POST /api/products
■   ■   ■■■ [id]/
■   ■       ■■■ get.ts      # GET /api/products/{id}
■   ■       ■■■ put.ts      # PUT /api/products/{id}
■   ■       ■■■ delete.ts   # DELETE /api/products/{id}
■   ■■■ users/
■       ■■■ get.ts
■       ■■■ post.ts
■■■ admin/
■   ■■■ dashboard/get.ts
■   ■■■ settings/get.ts
■■■ pages/
    ■■■ home/get.ts
    ■■■ about/get.ts
```

Both patterns work identically. Choose whichever keeps your project navigable.

---

12. Exercise: Build a Full CRUD API for Products

Build a complete REST API for managing products. All data lives in a TypeScript array (no database yet -- that comes in Chapter 5).

Requirements

Create a file `src/routes/product-api.ts` with the following routes:

Method	Path	Description
`GET`	`/api/products`	List all products. Support `?category=` filter.

`GET`	`/api/products/{id:int}`	Get a single product by ID. Return 404 if not found.
`POST`	`/api/products`	Create a new product. Return 201.
`PUT`	`/api/products/{id:int}`	Replace a product. Return 404 if not found.
`DELETE`	`/api/products/{id:int}`	Delete a product. Return 204 with no body.

Each product has: `id` (number), `name` (string), `category` (string), `price` (number), `inStock` (boolean).

Start with this seed data:

```
let products = [
    { id: 1, name: "Wireless Keyboard", category: "Electronics", price: 79.99, inStock: true },
    { id: 2, name: "Yoga Mat", category: "Fitness", price: 29.99, inStock: true },
    { id: 3, name: "Coffee Grinder", category: "Kitchen", price: 49.99, inStock: false },
    { id: 4, name: "Standing Desk", category: "Office", price: 549.99, inStock: true },
    { id: 5, name: "Running Shoes", category: "Fitness", price: 119.99, inStock: true }
];
```

Test with:

```
# List all
curl http://localhost:7148/api/products

# Filter by category
curl "http://localhost:7148/api/products?category=Fitness"

# Get one
curl http://localhost:7148/api/products/3

# Create
curl -X POST http://localhost:7148/api/products \
  -H "Content-Type: application/json" \
  -d '{"name": "Desk Lamp", "category": "Office", "price": 39.99, "inStock": true}'

# Update
curl -X PUT http://localhost:7148/api/products/3 \
  -H "Content-Type: application/json" \
  -d '{"name": "Burr Coffee Grinder", "category": "Kitchen", "price": 59.99, "inStock": true}'

# Delete
curl -X DELETE http://localhost:7148/api/products/3

# Not found
curl http://localhost:7148/api/products/999
```

---

13. Solution

Create `src/routes/product-api.ts`:

```
import { Router } from "tina4-nodejs";

// In-memory product store (resets on server restart)
let products = [
    { id: 1, name: "Wireless Keyboard", category: "Electronics", price: 79.99, inStock: true },
    { id: 2, name: "Yoga Mat", category: "Fitness", price: 29.99, inStock: true },
    { id: 3, name: "Coffee Grinder", category: "Kitchen", price: 49.99, inStock: false },
    { id: 4, name: "Standing Desk", category: "Office", price: 549.99, inStock: true },
    { id: 5, name: "Running Shoes", category: "Fitness", price: 119.99, inStock: true }
];

let nextId = 6;

// List all products, optionally filter by category
Router.get("/api/products", async (req, res) => {
    const category = req.query.category ?? null;

    if (category !== null) {
        const filtered = products.filter(
            p => p.category.toLowerCase() === String(category).toLowerCase()
        );
        return res.json({ products: filtered, count: filtered.length });
    }

    return res.json({ products, count: products.length });
});

// Get a single product by ID
Router.get("/api/products/{id:int}", async (req, res) => {
    const id = req.params.id;
    const product = products.find(p => p.id === id);

    if (!product) {
        return res.status(404).json({ error: "Product not found", id });
    }

    return res.json(product);
});

// Create a new product
Router.post("/api/products", async (req, res) => {
    const body = req.body;

    if (!body.name) {
        return res.status(400).json({ error: "Name is required" });
    }

    const product = {
        id: nextId++,
        name: body.name,
        category: body.category ?? "Uncategorized",
        price: parseFloat(body.price ?? 0),
        inStock: Boolean(body.inStock ?? true)
    };
```

```
    products.push(product);

    return res.status(201).json(product);
});

// Replace a product
Router.put("/api/products/{id:int}", async (req, res) => {
    const id = req.params.id;
    const body = req.body;
    const index = products.findIndex(p => p.id === id);

    if (index === -1) {
        return res.status(404).json({ error: "Product not found", id });
    }

    products[index] = {
        id,
        name: body.name ?? products[index].name,
        category: body.category ?? products[index].category,
        price: parseFloat(body.price ?? products[index].price),
        inStock: Boolean(body.inStock ?? products[index].inStock)
    };

    return res.json(products[index]);
});

// Delete a product
Router.delete("/api/products/{id:int}", async (req, res) => {
    const id = req.params.id;
    const index = products.findIndex(p => p.id === id);

    if (index === -1) {
        return res.status(404).json({ error: "Product not found", id });
    }

    products.splice(index, 1);

    return res.status(204).json(null);
});
```

Expected output for the test commands:

List all:

```
{"products":[{"id":1,"name":"Wireless Keyboard","category":"Electronics","price":79.99,"inStock"
```

Filter by category:

```
{"products":[{"id":2,"name":"Yoga Mat","category":"Fitness","price":29.99,"inStock":true},{"id":
```

Create (Status: `201 Created`):

```
{"id":6,"name":"Desk Lamp","category":"Office","price":39.99,"inStock":true}
```

Not found (Status: `404 Not Found`):

```
{"error":"Product not found","id":999}
```

---

14. Gotchas

1. Trailing Slashes Matter

Problem: `/products` works but `/products/` returns a 404 (or vice versa).

Cause: Tina4 treats `/products` and `/products/` as different routes by default.

Fix: Pick one convention and stick with it. Set `TINA4_TRAILING_SLASH_REDIRECT=true` in `.env` and Tina4 redirects `/products/` to `/products`.

2. Parameter Names Must Be Unique in a Path

Problem: `/users/{id}/posts/{id}` gives wrong results -- both parameters share the same name.

Cause: The second `{id}` overwrites the first in `req.params`.

Fix: Use distinct names: `/users/{userId}/posts/{postId}`.

3. Method Conflicts

Problem: You defined `Router.get("/items/{id}", ...)` and `Router.get("/items/{action}", ...)` and the wrong handler runs.

Cause: Both patterns match `/items/42`. The first one registered wins.

Fix: Use typed parameters to disambiguate: `Router.get("/items/{id:int}", ...)` matches integers only, leaving `/items/export` free for the other route.

4. Route Handler Must Return a Response

Problem: Your route handler runs but the browser shows an empty page or a 500 error.

Cause: You forgot the `return` statement. Without `return`, the handler produces `undefined` and Tina4 has nothing to send back.

Fix: Every handler must `return res.json(...)` or `return res.html(...)`.

5. Async Handlers

Problem: Your handler calls an async function but the response returns before it completes.

Cause: You forgot to `await` the async operation. The handler returns before the work finishes.

Fix: `await` all async operations inside handlers. All Tina4 route handlers should be `async` functions.

6. Middleware Must Be Passed as Function References in an Array

Problem: Passing a middleware function name as a string causes an error.

Cause: Tina4 expects middleware as an array of function references, not as strings.

Fix: Pass middleware as an array of function references: `[myMiddleware]`, not `"myMiddleware"`.

---

7. Group Prefix Must Start with a Slash

Problem: `Router.group("api/v1", ...)` produces routes that do not match.

Cause: The group prefix needs a leading `/`.

Fix: Start group prefixes with `/`: `Router.group("/api/v1", ...)`.

Request & Response

1. The Two Objects You Always Get

Every route handler in Tina4 receives two arguments: `req` and `res`. The request tells you what the client sent. The response is how you talk back. Together they are the entire HTTP conversation.

```
import { Router } from "tina4-nodejs";

Router.get("/echo", async (req, res) => {
    return res.json({
        method: req.method,
        path: req.path,
        your_ip: req.ip
    });
});
```

```
curl http://localhost:7148/echo
```

```
{"method":"GET","path":"/echo","your_ip":"127.0.0.1"}
```

The pattern for every route: inspect the request, build the response, return it.

---

2. The Request Object

The `req` object gives you everything the client sent. Here is the complete inventory.

method

The HTTP method as an uppercase string: `"GET"`, `"POST"`, `"PUT"`, `"PATCH"`, or `"DELETE"`.

```
req.method // "GET"
```

path

The URL path without query parameters:

```
// Request to /api/users?page=2
req.path // "/api/users"
```

params

Path parameters from the URL pattern (see Chapter 2):

```
// Route: /users/{id}/posts/{postId}
// Request: /users/5/posts/42
req.params.id     // "5" (or 5 if typed as :id:int)
req.params.postId // "42"
```

query

Query string parameters as an object:

```
// Request: /search?q=keyboard&page=2&sort=price
req.query.q    // "keyboard"
```

```
req.query.page // "2"
req.query.sort // "price"
```

body

The parsed request body. JSON requests produce an object. Form submissions contain form fields:

```
// POST with {"name": "Widget", "price": 9.99}
req.body.name  // "Widget"
req.body.price // 9.99
```

headers

Request headers as an object. Header names are normalized to lowercase:

```
req.headers["content-type"]  // "application/json"
req.headers["authorization"] // "Bearer eyJhbGci..."
req.headers["x-custom"]      // "my-value"
```

ip

The client's IP address:

```
req.ip // "127.0.0.1"
```

Tina4 respects `X-Forwarded-For` and `X-Real-IP` headers when behind a reverse proxy.

cookies

Cookies sent by the client:

```
req.cookies.session_id  // "abc123"
req.cookies.preferences // "dark-mode"
```

files

Uploaded files (covered in detail in section 7):

```
req.files.avatar // { filename, type, size, content } - content is a Buffer
```

Inspecting the Full Request

A route that dumps everything:

```
import { Router } from "tina4-nodejs";

Router.post("/debug/request", async (req, res) => {
    return res.json({
        method: req.method,
        path: req.path,
        params: req.params,
        query: req.query,
        body: req.body,
        headers: req.headers,
        ip: req.ip,
        cookies: req.cookies
    });
});
```

```
curl -X POST "http://localhost:7148/debug/request?page=1" \
```

```
  -H "Content-Type: application/json" \
  -H "X-Custom: hello" \
  -d '{"name": "test"}'
```

```
{
  "method": "POST",
  "path": "/debug/request",
  "params": {},
  "query": {"page": "1"},
  "body": {"name": "test"},
  "headers": {
    "content-type": "application/json",
    "x-custom": "hello",
    "host": "localhost:7148",
    "user-agent": "curl/8.4.0",
    "accept": "*/*",
    "content-length": "16"
  },
  "ip": "127.0.0.1",
  "cookies": {}
}
```

---

3. The Response Object

The `res` object is your toolkit for sending data back to the client. Every method returns the response, so you can chain calls.

json() -- JSON Response

The workhorse for APIs. Pass any object or value and it becomes JSON:

```
return res.json({ name: "Alice", age: 30 });
```

```
{"name":"Alice","age":30}
```

Chain with `status()` for a custom status code:

```
return res.status(201).json({ id: 7, name: "Widget" });
```

This returns `201 Created` with the JSON body.

html() -- Template or Raw HTML Response

Render a Frond template with data (Chapter 4: Templates goes deep):

```
return res.render("products.html", {
    products,
    title: "Our Products"
});
```

Or return raw HTML:

```
return res.html("<h1>Hello</h1><p>This is HTML.</p>");
```

Sets `Content-Type: text/html; charset=utf-8`.

text() -- Plain Text Response

Return plain text:

```
return res.text("Just a plain string.");
```

Sets `Content-Type: text/plain; charset=utf-8`.

redirect() -- Redirect Response

Send the client to a different URL:

```
return res.redirect("/login");
```

This sends a `302 Found` redirect by default. Pass a different status code for permanent redirects:

```
return res.redirect("/new-location", 301);
```

file() -- File Download Response

Send a file to the client:

```
return res.file("/path/to/report.pdf");
```

Tina4 sets the appropriate `Content-Type` based on the file extension. Pass `{ contentType }` to override it.

Force a download (`Content-Disposition: attachment`, saved under the file's own name):

```
return res.file("/path/to/report.pdf", { download: true });
```

---

4. Status Codes

Every response method chains with `status()`. The most common ones:

Code	Meaning	When to Use
`200`	OK	Default. Successful GET, PUT, PATCH.
`201`	Created	Successful POST that created a resource.
`204`	No Content	Successful DELETE. No body needed.
`301`	Moved Permanently	URL has changed forever.
`302`	Found	Temporary redirect.
`400`	Bad Request	Invalid input from the client.
`401`	Unauthorized	Missing or invalid authentication.

403	Forbidden	Authenticated but not allowed.
404	Not Found	Resource does not exist.
409	Conflict	Duplicate or conflicting data.
422	Unprocessable Entity	Valid JSON but fails business rules.
500	Internal Server Error	Something broke on the server.

```
return res.status(201).json({ id: 7, created: true });
```

---

5. Custom Headers

Set response headers with the `header()` method:

```
Router.get("/api/data", async (req, res) => {
    return res
        .header("X-Request-Id", crypto.randomUUID())
        .header("X-Rate-Limit-Remaining", "57")
        .header("Cache-Control", "no-cache")
        .json({ data: [1, 2, 3] });
});
```

CORS Headers

Tina4 handles CORS based on the `TINA4_CORS_ORIGINS` setting in `.env`. By default it is empty, which denies every cross-origin request. List the origins your front end is served from:

```
TINA4_CORS_ORIGINS=https://myapp.com,https://admin.myapp.com
```

---

6. Cookies

Set cookies on the response:

```
Router.post("/login", async (req, res) => {
    return res
        .cookie("session_id", "abc123xyz", {
            httpOnly: true,
            secure: true,
            sameSite: "Strict",
            maxAge: 3600,
            path: "/"
        })
        .json({ message: "Logged in" });
});
```

Read cookies from the request:

```
Router.get("/profile", async (req, res) => {
```

```
    const sessionId = req.cookies.session_id ?? null;

    if (sessionId === null) {
        return res.status(401).json({ error: "Not logged in" });
    }

    return res.json({ session: sessionId });
});
```

Delete a cookie by setting its `maxAge` to `0`:

```
return res
    .cookie("session_id", "", { maxAge: 0, path: "/" })
    .json({ message: "Logged out" });
```

---

7. File Uploads

Uploaded files arrive via `req.files`, keyed by field name. Each file is an object with `filename`, `type`, `size` and `content`, a `Buffer` holding the bytes; several files under one field name arrive as an array. Write one to disk with `saveUpload(file, dir, name?)`, which strips directory parts from the name and refuses to write outside `dir`.

Multiple File Uploads

Handle multiple files from a single form field:

```
import { Router, saveUpload } from "tina4-nodejs";
import { join, extname } from "path";
import { randomUUID } from "crypto";

Router.post("/api/gallery", async (req, res) => {
    const files = req.files?.photos;

    if (!files) {
        return res.status(400).json({ error: "No files uploaded" });
    }

    // One file arrives as an object, several under the same field name as an array
    const fileList = [files].flat();

    const results = [];
    for (const file of fileList) {
        const ext = extname(file.filename);
        const savedName = `gallery_${randomUUID()}${ext}`;
        saveUpload(file, join(process.cwd(), "src/public/uploads"), savedName);
        results.push({
            original: file.filename,
            saved: savedName,
            size: file.size,
            url: `/uploads/${savedName}`,
        });
    }

    return res.status(201).json({ uploaded: results, count: results.length });
});
```

```
curl -X POST http://localhost:7148/api/gallery \
```

```
  -F "photos=@/path/to/photo1.jpg" \
  -F "photos=@/path/to/photo2.jpg" \
  -F "photos=@/path/to/photo3.jpg"
```

Handling a Single File Upload

```
import { Router } from "tina4-nodejs";

Router.post("/api/upload", async (req, res) => {
    if (!req.files?.image) {
        return res.status(400).json({ error: "No file uploaded" });
    }

    const file = req.files.image;

    return res.json({
        filename: file.filename,
        type: file.type,
        size: file.size
    });
});
```

```
curl -X POST http://localhost:7148/api/upload \
  -F "image=@/path/to/photo.jpg"
```

```
{
  "filename": "photo.jpg",
  "type": "image/jpeg",
  "size": 245760
}
```

Saving the Uploaded File

```
import { Router, saveUpload } from "tina4-nodejs";
import { join, extname } from "path";
import { randomUUID } from "crypto";

Router.post("/api/upload", async (req, res) => {
    if (!req.files?.image) {
        return res.status(400).json({ error: "No file uploaded" });
    }

    const file = req.files.image;

    // Validate file type
    const allowedTypes = ["image/jpeg", "image/png", "image/gif", "image/webp"];
    if (!allowedTypes.includes(file.type)) {
        return res.status(400).json({ error: "Invalid file type. Allowed: JPEG, PNG, GIF, WebP"
    }

    // Validate file size (max 5MB)
    const maxSize = 5 * 1024 * 1024;
    if (file.size > maxSize) {
        return res.status(400).json({ error: "File too large. Maximum size: 5MB" });
    }

    // Generate a unique filename
    const ext = extname(file.filename);
    const filename = `img_${randomUUID()}${ext}`;
    saveUpload(file, join(process.cwd(), "src/public/uploads"), filename);

    return res.status(201).json({
        message: "File uploaded successfully",
        filename,
        url: `/uploads/${filename}`,
        size: file.size
    });
});
```

---

8. File Downloads

Send files to the client using `res.file()`:

```
import { Router } from "tina4-nodejs";
import { existsSync } from "fs";
import { join } from "path";

Router.get("/api/reports/{filename}", async (req, res) => {
    const filename = req.params.filename;
    const filepath = join(process.cwd(), "data/reports", filename);

    if (!existsSync(filepath)) {
        return res.status(404).json({ error: "Report not found" });
    }

    return res.file(filepath);
});
```

To force a download (the browser saves it under the file's own name):

```
 return res.file(filepath, { download: true });
```

---

9. XML Responses

Return XML when your API needs to serve legacy clients or SOAP integrations:

```
 Router.get("/api/products/{id:int}.xml", async (req, res) => {
     const product = { id: req.params.id, name: "Wireless Keyboard", price: 79.99 };

     const xml = `<?xml version="1.0" encoding="UTF-8"?>
 <product>
     <id>${product.id}</id>
     <name>${product.name}</name>
     <price>${product.price}</price>
 </product>`;

     return res.header("Content-Type", "application/xml").text(xml);
 });
```

---

10. Content Negotiation

Check the `Accept` header to return different formats from the same endpoint:

```
 import { Router } from "tina4-nodejs";

 Router.get("/api/products/{id:int}", async (req, res) => {
     const id = req.params.id;
     const product = { id, name: "Wireless Keyboard", price: 79.99 };

     const accept = req.headers["accept"] ?? "application/json";

     if (accept.includes("text/html")) {
         return res.render("product-detail.html", { product });
     }

     if (accept.includes("application/xml")) {
         const xml = `<?xml version="1.0"?><product><id>${id}</id><name>${product.name}</name><pr
         return res.header("Content-Type", "application/xml").text(xml);
     }

     if (accept.includes("text/plain")) {
         return res.text(`Product #${id}: ${product.name} - $${product.price}`);
     }

     return res.json(product);
 });
```

---

11. Input Validation

Tina4 includes a `Validator` class for declarative input validation. Chain rules together and check the result. If validation fails, use `res.error()` to return a structured error envelope.

The Validator Class

```
import { Validator } from "tina4-nodejs";

Router.post("/api/users", (req, res) => {
    const v = new Validator(req.body);
    v.required("name").required("email").email("email").minLength("name", 2);

    if (!v.isValid()) {
        return res.error("VALIDATION_FAILED", v.errors()[0].message, 400);
    }

    // proceed with valid data
});
```

The `Validator` accepts the request body (an object) and provides chainable methods:

Method	Description
`required(field)`	Field must be present and non-empty
`email(field)`	Field must be a valid email address
`minLength(field, n)`	Field must have at least `n` characters
`maxLength(field, n)`	Field must have at most `n` characters
`numeric(field)`	Field must be a number
`inList(field, values)`	Field must be one of the allowed values

Call `v.isValid()` to check all rules. Call `v.errors()` to get the array of failures, each with a `field` and `message` property.

The Error Response Envelope

`res.error()` returns a consistent JSON error envelope:

```
return res.error("VALIDATION_FAILED", "Name is required", 400);
```

This produces:

```
{"error": true, "code": "VALIDATION_FAILED", "message": "Name is required", "status": 400}
```

The three arguments are: an error code string, a human-readable message, and the HTTP status code. Use this pattern across your API for consistent error handling.

Upload Size Limits

Tina4 enforces a maximum upload size via the `TINA4_MAX_UPLOAD_SIZE` environment variable. The value is in bytes. The default is `10485760` (10 MB).

```
TINA4_MAX_UPLOAD_SIZE=10485760
```

If a client sends a file larger than this limit, Tina4 returns a `413 Payload Too Large` response before your handler runs. To allow larger uploads, increase the value in `.env`:

```
TINA4_MAX_UPLOAD_SIZE=52428800
```

This sets the limit to 50 MB.

---

12. Exercise: Build an Image Upload API

Build an API that handles image uploads and serves them back.

Requirements

Method	Path	Description
`POST`	`/api/images`	Upload an image. Validate type and size. Return the image URL.
`GET`	`/api/images/{filename}`	Return the uploaded image file. Return 404 if not found.

Rules:

- Accept JPEG, PNG, and WebP files only
- Maximum file size: 2MB
- Save files to `src/public/uploads/` with a unique filename
- Return the original filename, the saved filename, file size in KB, and the URL
- The GET endpoint should serve the file directly (not JSON)

---

13. Solution

Create `src/routes/images.ts`:

```
import { Router, saveUpload } from "tina4-nodejs";
import { join, extname } from "path";
import { randomUUID } from "crypto";

Router.post("/api/images", async (req, res) => {
    if (!req.files?.image) {
        return res.status(400).json({ error: "No image file provided. Use field name 'image'." }
    }

    const file = req.files.image;

    const allowedTypes = ["image/jpeg", "image/png", "image/webp"];
    if (!allowedTypes.includes(file.type)) {
        return res.status(400).json({
            error: "Invalid file type",
```

```
            received: file.type,
            allowed: allowedTypes
        });
    }

    const maxSize = 2 * 1024 * 1024;
    if (file.size > maxSize) {
        return res.status(400).json({
            error: "File too large",
            size_bytes: file.size,
            max_bytes: maxSize
        });
    }

    const ext = extname(file.filename).toLowerCase();
    const savedName = `img_${randomUUID()}${ext}`;
    saveUpload(file, join(process.cwd(), "src/public/uploads"), savedName);

    return res.status(201).json({
        message: "Image uploaded successfully",
        original_name: file.filename,
        saved_name: savedName,
        size_kb: Math.round(file.size / 1024 * 10) / 10,
        type: file.type,
        url: `/uploads/${savedName}`
    });
});

Router.get("/api/images/{filename}", async (req, res) => {
    const filename = req.params.filename;

    if (filename.includes("..") || filename.includes("/")) {
        return res.status(400).json({ error: "Invalid filename" });
    }

    const filepath = join(process.cwd(), "src/public/uploads", filename);

    if (!existsSync(filepath)) {
        return res.status(404).json({ error: "Image not found", filename });
    }

    return res.file(filepath);
});
```

Expected output for upload:

```
{
  "message": "Image uploaded successfully",
  "original_name": "photo.jpg",
  "saved_name": "img_a1b2c3d4-e5f6-7890-abcd-ef1234567890.jpg",
  "size_kb": 240.0,
  "type": "image/jpeg",
  "url": "/uploads/img_a1b2c3d4-e5f6-7890-abcd-ef1234567890.jpg"
}
```

(Status: `201 Created`)

---

14. Gotchas

1. Forgetting `return`

Problem: Your handler runs (log output appears) but the browser shows an empty response or a 500 error.

Cause: You wrote `res.json({...})` without `return`.

Fix: Write `return res.json({...})`. The response object must be returned from the handler for Tina4 to send it.

2. Body Is Undefined for JSON Requests

Problem: `req.body` is `undefined` or empty even though you are sending JSON.

Cause: Missing `Content-Type: application/json` header. Without it, Tina4 does not parse the body as JSON.

Fix: Include `-H "Content-Type: application/json"` when sending JSON with curl. In frontend JavaScript, `fetch()` with `JSON.stringify()` requires `headers: {"Content-Type": "application/json"}`.

3. File Uploads Return Empty

Problem: `req.files` is empty even though you are uploading a file.

Cause: The form is not using `enctype="multipart/form-data"`, or the curl command uses `-d` instead of `-F`.

Fix: For HTML forms, use `<form enctype="multipart/form-data">`. For curl, use `-F "field=@file.jpg"` (with `@`), not `-d`.

4. Redirect Loops

Problem: The browser shows "too many redirects" or hangs.

Cause: Route A redirects to route B, which redirects back to route A.

Fix: Trace the redirect chain in the browser's network inspector. Break the cycle.

5. Cookie Not Set

Problem: You called `res.cookie(...)` but the browser does not show the cookie.

Cause: `secure: true` means the cookie travels only over HTTPS. Local development uses `http://localhost`. The cookie is dropped.

Fix: Set `secure: false` during development.

6. Large Request Body Rejected

Problem: POST requests with large bodies return a 413 error.

Cause: The request body exceeds the configured maximum size.

Fix: Increase `TINA4_MAX_UPLOAD_SIZE` in `.env`. The default is `10mb`.

7. Headers Are Lowercase in Node.js

Problem: `req.headers["Content-Type"]` is `undefined` even though the header was sent.

Cause: Node.js normalizes all header names to lowercase.

Fix: Use lowercase header names: `req.headers["content-type"]`.

Templates

1. Why Templates

In Chapter 1, you saw `res.render("products.html", data)` produce a full HTML page. That rendering was done by **Frond**, Tina4's built-in template engine. Zero dependencies. Twig-compatible. Built from scratch. If you know Twig, Jinja2, or Nunjucks, you know 90% of Frond.

Templates live in `src/templates/`. Call `res.render("page.html", data)` and Frond loads `src/templates/page.html`, processes the tags and expressions, and returns the final HTML.

This chapter covers every feature of the template engine. After this, you build real pages.

---

2. Variables and Expressions

Output a variable with double curly braces:

```
<h1>Hello, {{ name }}!</h1>
```

Route handler:

```
import { Router } from "tina4-nodejs";

Router.get("/welcome", async (req, res) => {
    return res.render("welcome.html", {
        name: "Alice"
    });
});
```

Create `src/templates/welcome.html`:

```
<!DOCTYPE html>
<html>
<head><title>Welcome</title></head>
<body>
    <h1>Hello, {{ name }}!</h1>
</body>
</html>
```

Expected browser output:

```
Hello, Alice!
```

Accessing Nested Data

Dot notation reaches into nested objects:

```
const data = {
    user: {
        name: "Alice",
        email: "alice@example.com",
        address: {
            city: "Cape Town",
```

```
            country: "South Africa"
        }
    }
};

return res.render("profile.html", data);
```

```
<p>{{ user.name }} lives in {{ user.address.city }}, {{ user.address.country }}.</p>
```

Output:

```
Alice lives in Cape Town, South Africa.
```

Method Calls on Objects

If an object value is a function, Frond calls it automatically when accessed via dot notation. You can also call methods with arguments:

```
const data = {
    user: {
        name: "Alice",
        t: (key: string) => translations[key] ?? key
    }
};

return res.render("page.html", data);
```

```
<p>{{ user.name }}</p>
<p>{{ user.t("welcome_message") }}</p>
```

This works with any callable property. Arguments are evaluated as expressions, so you can pass variables, strings, or numbers.

Expressions

Basic expressions work inside `{{ }}`:

```
<p>Total: ${{ price * quantity }}</p>
<p>Discounted: ${{ price * 0.9 }}</p>
<p>Full name: {{ first_name ~ " " ~ last_name }}</p>
```

The `~` operator concatenates strings.

---

3. Template Inheritance

Template inheritance is the engine's headline feature. Define a base layout once. Extend it in every page.

Base Layout

Create `src/templates/base.twig`:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>{% block title %}My App{% endblock %}</title>
```

```
    <link rel="stylesheet" href="/css/tina4.css">
    {% block head %}{% endblock %}
</head>
<body>
    <nav>
        <a href="/">Home</a>
        <a href="/about">About</a>
        <a href="/contact">Contact</a>
    </nav>

    <main>
        {% block content %}{% endblock %}
    </main>

    <footer>
        <p>&copy; 2026 My App. All rights reserved.</p>
    </footer>

    <script src="/js/frond.js"></script>
    {% block scripts %}{% endblock %}
</body>
</html>
```

Child Template

Create `src/templates/about.twig`:

```
{% extends "base.twig" %}

{% block title %}About Us{% endblock %}

{% block content %}
    <h1>About Us</h1>
    <p>We have been building things since {{ founded_year }}.</p>
    <p>Our team has {{ team_size }} members across {{ office_count }} offices.</p>
{% endblock %}
```

Route handler:

```
import { Router } from "tina4-nodejs";

Router.get("/about", async (req, res) => {
    return res.render("about.twig", {
        founded_year: 2020,
        team_size: 12,
        office_count: 3
    });
});
```

Using `{{ parent() }}`

Add to a block rather than replace it:

```
{% extends "base.twig" %}

{% block head %}
    {{ parent() }}
    <link rel="stylesheet" href="/css/contact-form.css">
{% endblock %}

{% block content %}
```

```
    <h1>Contact Us</h1>
    <form>...</form>
{% endblock %}
```

---

4. Includes

Break templates into reusable pieces with `{% include %}`:

Create `src/templates/partials/header.twig`:

```
<header>
    <div class="logo">{{ site_name | default("My App") }}</div>
    <nav>
        <a href="/">Home</a>
        <a href="/products">Products</a>
        <a href="/contact">Contact</a>
    </nav>
</header>
```

Create `src/templates/partials/product-card.twig`:

```
<div class="product-card{{ product.featured ? ' featured' : '' }}">
    <h3>{{ product.name }}</h3>
    <p class="price">${{ product.price | number_format(2) }}</p>
    {% if product.inStock %}
        <span class="badge-success">In Stock</span>
    {% else %}
        <span class="badge-danger">Out of Stock</span>
    {% endif %}
</div>
```

Use them in a page:

```
{% extends "base.twig" %}

{% block content %}
    {% include "partials/header.twig" %}

    <h1>Products</h1>
    {% for product in products %}
        {% include "partials/product-card.twig" %}
    {% endfor %}
{% endblock %}
```

Passing Variables to Includes

```
{% include "partials/header.twig" with {"site_name": "Cool Store"} %}
```

Use `only` to isolate the included template from the parent scope:

```
{% include "partials/header.twig" with {"site_name": "Cool Store"} only %}
```

---

5. For Loops

Loop through arrays with `{% for %}`:

```
<ul>
{% for item in items %}
    <li>{{ item }}</li>
{% endfor %}
</ul>
```

The `loop` Variable

Inside a for loop, Frond provides a special `loop` variable:

Property	Type	Description
`loop.index`	int	Current iteration (1-based)
`loop.index0`	int	Current iteration (0-based)
`loop.first`	bool	True on the first iteration
`loop.last`	bool	True on the last iteration
`loop.length`	int	Total number of items
`loop.revindex`	int	Iterations remaining (1-based)

```
<table>
    <thead>
        <tr><th>#</th><th>Name</th><th>Price</th></tr>
    </thead>
    <tbody>
    {% for product in products %}
        <tr class="{{ loop.index is odd ? 'row-light' : 'row-dark' }}">
            <td>{{ loop.index }}</td>
            <td>{{ product.name }}</td>
            <td>${{ product.price | number_format(2) }}</td>
        </tr>
    {% endfor %}
    </tbody>
</table>
```

Empty Lists

Handle empty lists with `{% else %}`:

```
{% for product in products %}
    <div class="product-card">
        <h3>{{ product.name }}</h3>
    </div>
{% else %}
    <p>No products found.</p>
{% endfor %}
```

Looping Over Key-Value Pairs

```
{% for key, value in metadata %}
    <dt>{{ key }}</dt>
    <dd>{{ value }}</dd>
{% endfor %}
```

---

6. Conditionals

if / elseif / else

```
{% if user.role == "admin" %}
    <a href="/admin">Admin Panel</a>
{% elseif user.role == "editor" %}
    <a href="/editor">Editor Dashboard</a>
{% else %}
    <a href="/profile">My Profile</a>
{% endif %}
```

Ternary Operator

```
<span class="{{ is_active ? 'text-green' : 'text-gray' }}">
    {{ is_active ? 'Active' : 'Inactive' }}
</span>
```

Testing for Existence

```
{% if error_message is defined %}
    <div class="alert alert-danger">{{ error_message }}</div>
{% endif %}
```

{% set %} -- Local Variables

Create or update a variable inside a template:

```
{% set greeting = "Hello" %}
{% set full_name = user.first_name ~ " " ~ user.last_name %}
{% set total = price * quantity %}
{% set discount = total - rebate %}

<p>{{ greeting }}, {{ full_name }}!</p>
<p>Total: {{ total }}, After discount: {{ discount }}</p>
```

The `~` operator concatenates strings. Arithmetic operators (`+`, `-`, `*`, `/`, `//`, `%`, `**`) work in `set` and expressions.

When combining filters with arithmetic, assign the filtered values first:

```
{% set dr = account.dr|default(0) %}
{% set cr = account.cr|default(0) %}
{% set balance = dr - cr %}
<p>Balance: {{ balance }}</p>
```

---

7. Filters

Filters transform values. Apply them with the `|` (pipe) character.

Complete Filter Reference

String Filters

Filter	Example	Description	
`upper`	`{{ name | upper }}`		Convert to uppercase
`lower`	`{{ name | lower }}`		Convert to lowercase
`capitalize`	`{{ name | capitalize }}`		Capitalize first letter
`title`	`{{ name | title }}`		Capitalize each word
`trim`	`{{ name | trim }}`		Strip leading/trailing whitespace
`ltrim`	`{{ name | ltrim }}`		Strip leading whitespace
`rtrim`	`{{ name | rtrim }}`		Strip trailing whitespace
`slug`	`{{ title | slug }}`		Convert to URL-friendly slug
`wordwrap(80)`	`{{ text | wordwrap(80) }}`		Wrap text at N characters
`truncate(100)`	`{{ text | truncate(100) }}`		Truncate to N characters with ellipsis
`nl2br`	`{{ text | nl2br }}`		Convert newlines to ` ` tags
`striptags`	`{{ html | striptags }}`		Remove all HTML tags
`replace("a", "b")`	`{{ text | replace("old", "new") }}`		Replace occurrences of a substring

Array Filters

Filter	Example	Description	
`length`	`{{ items \	length }}`	Count items in array or string length
`reverse`	`{{ items \	reverse }}`	Reverse order of items
`sort`	`{{ items \	sort }}`	Sort items ascending
`shuffle`	`{{ items \	shuffle }}`	Randomly shuffle items
`first`	`{{ items \	first }}`	Get the first item
`last`	`{{ items \	last }}`	Get the last item
`join(", ")`	`{{ items \	join(", ") }}`	Join array items with separator
`split(",")`	`{{ csv \	split(",") }}`	Split string into array
`unique`	`{{ items \	unique }}`	Remove duplicate values
`filter`	`{{ items \	filter }}`	Remove falsy values from array
`map("name")`	`{{ items \	map("name") }}`	Extract a property from each item
`column("name")`	`{{ items \	column("name") }}`	Extract a column from array of objects
`batch(3)`	`{{ items \	batch(3) }}`	Group items into batches of N
`slice(0, 3)`	`{{ items \	slice(0, 3) }}`	Extract a slice from offset with length

Encoding Filters

Filter	Example	Description	
`escape` (`e`)	`{{ text | escape }}`	HTML-escape special characters
`raw` (`safe`)	`{{ html | raw }}`	Output without auto-escaping
`url_encode`	`{{ text | url_encode }}`	URL-encode a string
`base64_encode` (`base64encode`)	`{{ text | base64_encode }}`	Base64-encode a string
`base64_decode` (`base64decode`)	`{{ data | base64_decode }}`	Base64-decode a string
`md5`	`{{ text | md5 }}`	Compute MD5 hash
`sha256`	`{{ text | sha256 }}`	Compute SHA-256 hash

Numeric Filters

Filter	Example	Description	
`abs`	`{{ num | abs }}`	Absolute value
`round(2)`	`{{ price | round(2) }}`	Round to N decimal places
`number_format(2)`	`{{ price | number_format(2) }}`	Format with decimals and thousands separator
`int`	`{{ val | int }}`	Cast to integer
`float`	`{{ val | float }}`	Cast to float
`string`	`{{ val | string }}`	Cast to string

JSON Filters

Filter	Example	Description	
`json_encode`	`{{ data | json_encode }}`	Encode value as JSON string
`to_json` (`tojson`)	`{{ data | to_json }}`	Encode value as JSON string (alias)
`json_decode`	`{{ str | json_decode }}`	Decode JSON string to object
`js_escape`	`{{ text | js_escape }}`	Escape string for safe use in JavaScript

Dict Filters

Filter	Example	Description	
`keys`	`{{ obj | keys }}`		Get dictionary keys as array
`values`	`{{ obj | values }}`		Get dictionary values as array
`merge(other)`	`{{ defaults | merge(overrides) }}`		Merge two dictionaries

Other Filters

Filter	Example	Description	
`default("fallback")`	`{{ name | default("Guest") }}`		Fallback when value is empty or undefined
`date("Y-m-d")`	`{{ created | date("Y-m-d") }}`		Format a date value
`format(val)`	`{{ "%.2f" | format(price) }}`		Format string with value (sprintf-style)
`data_uri`	`{{ content | data_uri }}`		Convert to a data URI string
`dump`	`{{ var | dump }}` or `{{ dump(var) }}`		Debug output - gated on `TINA4_DEBUG=true` (see *Dumping Values*)
`form_token`	`{{ form_token() }}`		Generate a CSRF hidden input with token
`formTokenValue`	`{{ formTokenValue("context") }}`		Return just the raw JWT token string
`to_json`	`{{ data | to_json }}`		JSON-encode a value (safe, no double-escaping)
`js_escape`	`{{ text | js_escape }}`		Escape for safe use in JavaScript strings

Chaining Filters

```
{{ name | trim | lower | capitalize }}
{# "  ALICE SMITH  " -> "Alice smith" #}
```

Dumping Values for Debugging

The `dump` helper lets you inspect any variable mid-template. Two interchangeable forms are supported:

```
{{ user | dump }}
{{ dump(user) }}
```

Both produce the same `<pre>`-wrapped, HTML-escaped inspection of the value. Unlike `JSON.stringify`, the built-in inspector handles everything safely:

- **Circular references** - marked `[Circular]` (no crash)
- **BigInt** - shown as `123n` (no crash)
- **Map / Set** - `Map(2) { "a" => 1, "b" => 2 }` / `Set(3) { 1, 2, 3 }`
- **Date** - `Date(2026-04-09T13:00:00.000Z)` (type retained)
- **Error** - `Error("message")` (type + message)
- **Class instances** - `User { name: "Alice" }` (class name preserved)
- **Functions / Symbols / undefined** - shown inline, not silently dropped

```
{{ dump(order) }}

{# Output: #}
{# <pre>Order { id: 42, items: [...], total: 99.99 }</pre> #}
```

dump is gated on `TINA4_DEBUG=true`. In production (env var unset or `false`) **both** the filter and function form silently return an empty `SafeString`. This prevents accidental leaks of internal state, object shapes, or sensitive values into rendered HTML if a developer leaves a `{{ dump(x) }}` call in a template.

```
# .env - dev
TINA4_DEBUG=true    # dump() outputs the value

# .env - production
TINA4_DEBUG=false   # dump() is a no-op
```

You can rely on this gate for safety, but treat `dump` as a development-only convenience. For structured output in production code paths, use `to_json`.

---

8. Macros

Macros are reusable template functions. Define once. Call many times.

Defining a Macro

Create `src/templates/macros.twig`:

```
{% macro button(text, url, style) %}
    <a href="{{ url | default('#') }}" class="btn btn-{{ style | default('primary') }}">
        {{ text }}
    </a>
{% endmacro %}

{% macro alert(message, type) %}
    <div class="alert alert-{{ type | default('info') }}">
        {{ message }}
    </div>
{% endmacro %}

{% macro input(name, label, type, value) %}
    <div class="form-group">
        <label for="{{ name }}">{{ label | default(name | capitalize) }}</label>
        <input type="{{ type | default('text') }}" id="{{ name }}" name="{{ name }}" value="{{ v
    </div>
```

```
{% endmacro %}
```

Using Macros

```
{% from "macros.twig" import button, alert, input %}

{% extends "base.twig" %}

{% block content %}
    {{ alert("Your profile has been updated.", "success") }}

    <form method="POST" action="/profile">
        {{ input("name", "Full Name", "text", user.name) }}
        {{ input("email", "Email Address", "email", user.email) }}

        {{ button("Save Changes", "", "primary") }}
        {{ button("Cancel", "/dashboard", "secondary") }}
    </form>
{% endblock %}
```

---

9. Special Tags

{% raw %} -- Literal Output

When you need to output literal `{{ }}` (for a Vue.js template, for example):

```
{% raw %}
    <div id="app">
        {{ message }}
    </div>
{% endraw %}
```

{% spaceless %} -- Remove Whitespace Between Tags

The `spaceless` tag strips whitespace between HTML tags. Useful for inline elements where whitespace affects layout:

```
{% spaceless %}
    <div>
        <span>Hello</span>
        <span>World</span>
    </div>
{% endspaceless %}
```

Output:

```
<div><span>Hello</span><span>World</span></div>
```

Only whitespace between tags is removed. Whitespace inside text content stays intact.

{% autoescape %} -- Control HTML Escaping

By default, Frond escapes HTML in `{{ }}` output to prevent XSS attacks. The `autoescape` tag controls this behavior for a block:

```
{% autoescape false %}
    {{ raw_html }}
{% endautoescape %}
```

With `autoescape false`, the variable outputs as raw HTML without escaping. Use this when you trust the content -- rendering Markdown-to-HTML output, for example.

To re-enable escaping inside a `false` block:

```
{% autoescape false %}
    {{ trusted_html }}
    {% autoescape true %}
        {{ user_input }}
    {% endautoescape %}
{% endautoescape %}
```

Whitespace Control

Use hyphens in tag delimiters to trim whitespace:

```
{%- if condition -%}
    No leading or trailing whitespace around this block
{%- endif -%}
```

The `-` on the left trims whitespace before the tag. The `-` on the right trims whitespace after the tag. Works with `{{ }}`, `{% %}`, and `{# #}` delimiters.

Comments

```
{# This comment will not appear in the HTML output #}
```

---

10. Template Route Export Pattern

Tina4 Node.js has a special pattern for file-based routes with templates. Export a `template` constant and return data from the handler:

Create `src/routes/catalog/get.ts`:

```
export const template = "catalog.twig";

export default async (req, res) => {
    const category = req.query.category ?? "";

    const products = [
        { name: "Espresso Machine", category: "Kitchen", price: 299.99, featured: true },
        { name: "Yoga Mat", category: "Fitness", price: 29.99, featured: false },
        { name: "Standing Desk", category: "Office", price: 549.99, featured: true }
    ];

    const filtered = category
        ? products.filter(p => p.category.toLowerCase() === category.toLowerCase())
        : products;

    return {
        products: filtered,
        active_category: category,
        categories: [...new Set(products.map(p => p.category))]
    };
};
```

Tina4 renders `src/templates/catalog.twig` with the returned data. The route handler stays clean -- it returns data, the framework handles rendering.

---

11. Exercise: Build a Product Catalog Page

Build a product catalog page with a base layout, product cards, category filters, and a reusable card macro.

Requirements

- Create a base layout at `src/templates/catalog-base.twig` with blocks for `title`, `content`, and `scripts`
- Create a macro file at `src/templates/catalog-macros.twig` with a `productCard(product)` macro and a `categoryFilter(categories, active)` macro
- Create a page template at `src/templates/catalog.twig` that extends the base, uses the macros, shows category filter buttons, and shows product cards in a grid
- Create a route at `GET /catalog` that accepts an optional `?category=` filter

Data

```
const products = [
    { name: "Espresso Machine", category: "Kitchen", price: 299.99, inStock: true, featured: tru
    { name: "Yoga Mat", category: "Fitness", price: 29.99, inStock: true, featured: false },
    { name: "Standing Desk", category: "Office", price: 549.99, inStock: true, featured: true },
    { name: "Blender", category: "Kitchen", price: 89.99, inStock: false, featured: false },
    { name: "Running Shoes", category: "Fitness", price: 119.99, inStock: true, featured: false
    { name: "Desk Lamp", category: "Office", price: 39.99, inStock: true, featured: true },
    { name: "Cast Iron Skillet", category: "Kitchen", price: 44.99, inStock: true, featured: fal
];
```

---

12. Solution

Create `src/templates/catalog-base.twig`:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>{% block title %}Product Catalog{% endblock %}</title>
    <link rel="stylesheet" href="/css/tina4.css">
    <style>
        body { font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, sans-serif; m
        .container { max-width: 1000px; margin: 0 auto; padding: 20px; }
        .header { background: #2c3e50; color: white; padding: 20px; margin-bottom: 24px; }
        .header h1 { margin: 0; }
        .filters { margin-bottom: 20px; }
        .filter-btn { display: inline-block; padding: 6px 14px; margin: 0 6px 6px 0; border-radi
        .filter-btn.active { background: #2c3e50; color: white; border-color: #2c3e50; }
        .product-grid { display: grid; grid-template-columns: repeat(auto-fill, minmax(280px, 1f
```

```
        .product-card { background: white; border: 2px solid #e9ecef; border-radius: 8px; paddin
        .product-card.featured { border-color: #f39c12; background: #fef9e7; }
        .product-name { font-size: 1.1em; font-weight: 600; margin: 0 0 4px; }
        .product-category { font-size: 0.85em; color: #6c757d; }
        .product-price { font-size: 1.2em; font-weight: bold; color: #27ae60; }
        .badge-featured { background: #f39c12; color: white; display: inline-block; padding: 2px
        .badge-stock { background: #d4edda; color: #155724; display: inline-block; padding: 2px
        .badge-nostock { background: #f8d7da; color: #721c24; display: inline-block; padding: 2p
        .empty-state { text-align: center; padding: 40px; color: #6c757d; }
    </style>
</head>
<body>
    {% block content %}{% endblock %}
    <script src="/js/frond.js"></script>
    {% block scripts %}{% endblock %}
</body>
</html>
```

Create `src/templates/catalog-macros.twig`:

```
{% macro productCard(product) %}
    <div class="product-card{{ product.featured ? ' featured' : '' }}">
        <p class="product-name">
            {{ product.name }}
            {% if product.featured %}
                <span class="badge-featured">Featured</span>
            {% endif %}
        </p>
        <p class="product-category">{{ product.category }}</p>
        <p class="product-price">
            ${{ product.price | number_format(2) }}
            {% if product.inStock %}
                <span class="badge-stock">In Stock</span>
            {% else %}
                <span class="badge-nostock">Out of Stock</span>
            {% endif %}
        </p>
    </div>
{% endmacro %}

{% macro categoryFilter(categories, active) %}
    <div class="filters">
        <a href="/catalog" class="filter-btn{{ active is not defined or active == '' ? ' active'
        {% for cat in categories %}
            <a href="/catalog?category={{ cat }}" class="filter-btn{{ active == cat ? ' active'
        {% endfor %}
    </div>
{% endmacro %}
```

Create `src/templates/catalog.twig`:

```
{% extends "catalog-base.twig" %}

{% from "catalog-macros.twig" import productCard, categoryFilter %}

{% block title %}{{ active_category | default("All") }} Products - Catalog{% endblock %}

{% block content %}
    <div class="header">
        <h1>Product Catalog</h1>
```

```
        <p>{{ products | length }} product{{ products | length != 1 ? 's' : '' }}{% if active_ca
    </div>

    <div class="container">
        {{ categoryFilter(categories, active_category) }}

        {% if products | length > 0 %}
            <div class="product-grid">
                {% for product in products %}
                    {{ productCard(product) }}
                {% endfor %}
            </div>
        {% else %}
            <div class="empty-state">
                <h2>No products found</h2>
                <p>Try a different category or <a href="/catalog">view all products</a>.</p>
            </div>
        {% endif %}
    </div>
{% endblock %}
```

Create `src/routes/catalog.ts`:

```
import { Router } from "tina4-nodejs";

Router.get("/catalog", async (req, res) => {
    const allProducts = [
        { name: "Espresso Machine", category: "Kitchen", price: 299.99, inStock: true, featured:
        { name: "Yoga Mat", category: "Fitness", price: 29.99, inStock: true, featured: false },
        { name: "Standing Desk", category: "Office", price: 549.99, inStock: true, featured: tru
        { name: "Blender", category: "Kitchen", price: 89.99, inStock: false, featured: false },
        { name: "Running Shoes", category: "Fitness", price: 119.99, inStock: true, featured: fa
        { name: "Desk Lamp", category: "Office", price: 39.99, inStock: true, featured: true },
        { name: "Cast Iron Skillet", category: "Kitchen", price: 44.99, inStock: true, featured:
    ];

    const categories = [...new Set(allProducts.map(p => p.category))].sort();
    const activeCategory = req.query.category ?? "";

    const products = activeCategory
        ? allProducts.filter(p => p.category.toLowerCase() === String(activeCategory).toLowerCas
        : allProducts;

    return res.render("catalog.twig", {
        products,
        categories,
        active_category: activeCategory
    });
});
```

Expected browser output for `/catalog`:

- A dark header with "Product Catalog" and "7 products"
- Filter buttons: All (active), Fitness, Kitchen, Office
- A grid of 7 product cards
- Three cards (Espresso Machine, Standing Desk, Desk Lamp) have a gold border and "Featured" badge

Expected browser output for `/catalog?category=Kitchen`:

- Header shows "3 products in Kitchen"
- The Kitchen filter button is active
- Three cards: Espresso Machine, Blender, Cast Iron Skillet

---

13. Gotchas

1. `{% extends %}` Must Be the First Tag

Problem: Template inheritance does not work. The page renders without the base layout.

Cause: `{% extends "base.twig" %}` must be the first tag in the template. No exceptions.

Fix: Make `{% extends %}` the absolute first thing in the file.

2. Undefined Variables Show Nothing

Problem: `{{ username }}` renders as empty instead of showing an error.

Cause: Frond outputs nothing for undefined variables. By design.

Fix: Use the `default` filter: `{{ username | default("Guest") }}`.

3. Auto-Escaping Prevents HTML Output

Problem: You pass HTML content but it appears as literal text.

Cause: Auto-escaping converts `<` to `<` for security.

Fix: For trusted content, use `{{ content | raw }}`. Never use `raw` on user-supplied input.

4. Variable Scope in Includes

Problem: A variable defined inside a `{% for %}` loop is not accessible after the loop ends.

Cause: Loop variables are scoped to the loop.

Fix: Use `{% set %}` before the loop to accumulate values.

5. Macro Arguments Are Positional

Problem: Calling `{{ button("Click", style="danger") }}` does not work.

Cause: Frond macros use positional arguments, not keyword arguments.

Fix: Pass arguments in the order defined: `{{ button("Click", "/url", "danger") }}`.

6. Template File Extension Does Not Matter

Problem: Not sure whether to use `.html`, `.twig`, or `.tpl`.

Cause: Frond does not care about the file extension. It processes any file in `src/templates/`.

Fix: Pick one extension. Be consistent. This book uses `.twig` for templates with Twig syntax and `.html` for simple HTML files.

7. Filters Are Not JavaScript Functions

Problem: You try `{{ items | count }}` or `{{ name | toUpperCase }}` and get an error.

Cause: Frond filters follow Twig conventions, not JavaScript conventions.

Fix: Use `{{ items | length }}` instead of `count`. Use `{{ name | upper }}` instead of `toUpperCase`.

Database

1. From Arrays to Real Data

In Chapters 2 and 3, all your data lived in TypeScript arrays. Server restart. Data gone. That works for learning routing and responses. Real applications need persistent storage.

This chapter covers Tina4's database layer: raw queries, parameterised queries, transactions, schema inspection, helper methods, and migrations.

Tina4 speaks to five database engines: SQLite, PostgreSQL, MySQL, Microsoft SQL Server, and Firebird. The API is identical across all of them. One line in `.env` switches the engine.

---

2. Connecting to a Database

The Default: SQLite

When you scaffold a project with `tina4 init`, Tina4 drops a SQLite database at `data/app.db`. The default `.env` includes:

```
TINA4_DEBUG=true
```

With no explicit `TINA4_DATABASE_URL`, Tina4 defaults to `sqlite:///data/app.db`. That is why the health check at `/health` shows `"database": "connected"` with zero configuration.

SQLite support uses Node's built-in `node:sqlite` module (Node 22+). No native C++ addons are needed. No `node-gyp`. No platform-specific binaries. This is what makes Tina4 Node.js truly zero runtime dependencies -- even the database driver ships with Node itself.

Connection Strings for Other Databases

Set `TINA4_DATABASE_URL` in `.env` to use a different engine:

```
# SQLite (explicit)
TINA4_DATABASE_URL=sqlite:///data/app.db

# PostgreSQL
TINA4_DATABASE_URL=postgres://localhost:5432/myapp

# MySQL
TINA4_DATABASE_URL=mysql://localhost:3306/myapp

# Microsoft SQL Server
TINA4_DATABASE_URL=mssql://localhost:1433/myapp

# Firebird
TINA4_DATABASE_URL=firebird://localhost:3050/path/to/database.fdb
```

The Firebird adapter requires the `node-firebird` package (`npm install node-firebird`). SQL dialect differences are handled automatically: `LIMIT`/`OFFSET` is translated to `ROWS X TO Y`, boolean values are converted to integers, and `ILIKE` is converted to `LOWER() LIKE LOWER()`.

Firebird URL Forms

Firebird is the awkward one: every other engine has a server-side database name (`postgres://host:port/dbname`), but Firebird wants either an absolute file path on the server, a Windows drive-letter path, or an alias. The classic URI form needs a double slash to keep the leading `/` of an absolute path through `new URL()`, which is unintuitive.

Tina4 normalises five equivalent forms. Pick whichever reads best:

```
# Classic double-slash absolute path -- the URL spec way
TINA4_DATABASE_URL=firebird://SYSDBA:masterkey@localhost:3050//firebird/data/app.fdb

# Single-slash absolute path -- what most people instinctively type
TINA4_DATABASE_URL=firebird://SYSDBA:masterkey@localhost:3050/firebird/data/app.fdb

# Windows drive-letter path (also accepts /C%3A/Data/app.fdb)
TINA4_DATABASE_URL=firebird://SYSDBA:masterkey@host:3050/C:/Data/app.fdb

# Firebird alias (single token, no slashes)
TINA4_DATABASE_URL=firebird://SYSDBA:masterkey@localhost:3050/employee
```

For ops setups that keep the server URL and database location in separate config layers -- or for Windows backslash paths -- set `TINA4_DATABASE_FIREBIRD_PATH`:

```
TINA4_DATABASE_FIREBIRD_PATH=C:\firebird\data\app.fdb
TINA4_DATABASE_URL=firebird://SYSDBA:masterkey@localhost:3050/ignored
```

The env override wins over whatever path is in the URL.

Separate Credentials

```
TINA4_DATABASE_URL=postgres://localhost:5432/myapp
TINA4_DATABASE_USERNAME=myuser
TINA4_DATABASE_PASSWORD=secretpassword
```

Connection Pooling

For applications that handle many concurrent requests, enable connection pooling by passing a pool size to `Database.create()`:

```
const db = await Database.create("postgres://localhost/mydb", undefined, undefined, 5);
```

The fourth argument controls how many database connections are maintained:

- `0` (the default) -- a single connection is used for all queries
- `N` (where N > 0) -- N connections are created and rotated round-robin across queries

Pooled connections are thread-safe. Each query is dispatched to the next available connection in the pool. This eliminates contention when multiple route handlers query the database simultaneously.

Verifying the Connection

```
curl http://localhost:7148/health
```

```
{
  "status": "ok",
  "database": "connected",
  "uptime_seconds": 3,
```

```
  "version": "3.10.3",
  "framework": "tina4-nodejs"
}
```

---

3. Getting the Database Object

In your route handlers, access the database via the `Database` class:

```
import { Router } from "tina4-nodejs";
import { Database } from "tina4-nodejs/orm";

Router.get("/api/test-db", async (req, res) => {
    const db = await Database.getConnection();

    const result = await db.fetch("SELECT 1 + 1 AS answer");

    return res.json(result);
});
```

```
curl http://localhost:7148/api/test-db
```

```
{"answer": 2}
```

`Database.getConnection()` returns the active database connection. Call `fetch()`, `execute()`, and `fetchOne()` on this object. All database methods are async. All return Promises.

---

4. Raw Queries

fetch() -- Get Multiple Rows

```
const db = await Database.getConnection();

const products = await db.fetch("SELECT * FROM products WHERE price > 50");
```

Each row is a plain object with column names as keys:

```
// products looks like:
[
    { id: 1, name: "Keyboard", price: 79.99 },
    { id: 4, name: "Standing Desk", price: 549.99 }
]
```

DatabaseResult

`fetch()` returns a `DatabaseResult` object. It behaves like an array but carries extra metadata about the query.

Properties

```
const result = await db.fetch("SELECT * FROM users WHERE active = ?", [1]);

result.records;      // [{ id: 1, name: "Alice" }, { id: 2, name: "Bob" }]
result.columns;      // ["id", "name", "email", "active"]
result.count;        // total number of matching rows
result.limit;        // query limit (if set)
result.offset;       // query offset (if set)
```

Iteration

A `DatabaseResult` is iterable. Use it directly in `for...of`:

```
for (const user of result) {
    console.log(user.name);
}
```

Index Access

Access rows by index like a regular array:

```
const firstUser = result[0];
```

Countable

The `length` property works on the result:

```
console.log(result.length); // number of records in this result set
```

Conversion Methods

```
result.toJson();      // JSON string of all records
result.toCsv();       // CSV string with column headers
result.toArray();     // plain array of objects
result.toPaginate();  // { records: [...], count: 42, limit: 10, offset: 0 }
```

`toPaginate()` is designed for building paginated API responses. It bundles the records with the total count, limit, and offset in a single object.

Schema Metadata with columnInfo()

`columnInfo()` returns detailed metadata about the columns in the result set. The data is lazy-loaded -- it only queries the database schema when you call the method for the first time:

```
const info = await result.columnInfo();
// [
//     { name: "id", type: "INTEGER", size: null, decimals: null, nullable: false, primaryKey: t
//     { name: "name", type: "TEXT", size: null, decimals: null, nullable: false, primaryKey: fa
//     { name: "email", type: "TEXT", size: 255, decimals: null, nullable: true, primaryKey: fal
//     ...
// ]
```

Each column entry contains:

Field	Description
`name`	Column name
`type`	Database type (e.g. `INTEGER`, `TEXT`, `REAL`)

`size`	Maximum size (or `null` if not applicable)
`decimals`	Decimal places (or `null`)
`nullable`	Whether the column allows `NULL`
`primaryKey`	Whether the column is part of the primary key

This is useful for building dynamic forms, generating documentation, or validating data before insert.

fetchOne() -- Get a Single Row

```
const product = await db.fetchOne("SELECT * FROM products WHERE id = 1");
// Returns: { id: 1, name: "Keyboard", price: 79.99 }
```

If no row matches, `fetchOne()` returns `null`.

execute() -- Run a Statement

For INSERT, UPDATE, DELETE, and DDL statements that do not return rows:

```
await db.execute("INSERT INTO products (name, price) VALUES ('Widget', 9.99)");
await db.execute("UPDATE products SET price = 89.99 WHERE id = 1");
await db.execute("DELETE FROM products WHERE id = 5");
await db.execute("CREATE TABLE IF NOT EXISTS logs (id INTEGER PRIMARY KEY, message TEXT, created
```

Full Example: A Simple Query Route

```
import { Router } from "tina4-nodejs";
import { Database } from "tina4-nodejs/orm";

Router.get("/api/products", async (req, res) => {
    const db = await Database.getConnection();

    const products = await db.fetch("SELECT * FROM products ORDER BY name");

    return res.json({
        products,
        count: products.length
    });
});
```

---

5. Parameterised Queries

Never concatenate user input into SQL strings. That door leads to SQL injection:

```
// NEVER do this:
await db.fetch(`SELECT * FROM products WHERE name = '${userInput}'`);
```

Instead, use parameterised queries:

```
const db = await Database.getConnection();

// Named parameters
const product = await db.fetchOne(
    "SELECT * FROM products WHERE id = :id",
```

```
    { id: 42 }
);

// Positional parameters
const products = await db.fetch(
    "SELECT * FROM products WHERE price BETWEEN ? AND ? ORDER BY price",
    [10.00, 100.00]
);
```

A Safe Search Endpoint

```
import { Router } from "tina4-nodejs";
import { Database } from "tina4-nodejs/orm";

Router.get("/api/products/search", async (req, res) => {
    const db = await Database.getConnection();

    const q = req.query.q ?? "";
    const maxPrice = parseFloat(req.query.max_price ?? "99999");

    if (!q) {
        return res.status(400).json({ error: "Query parameter 'q' is required" });
    }

    const products = await db.fetch(
        "SELECT * FROM products WHERE name LIKE :query AND price <= :maxPrice ORDER BY name",
        { query: `%${q}%`, maxPrice }
    );

    return res.json({
        query: q,
        max_price: maxPrice,
        results: products,
        count: products.length
    });
});
```

```
curl "http://localhost:7148/api/products/search?q=key&max_price=100"
```

```
{
  "query": "key",
  "max_price": 100,
  "results": [
    {"id": 1, "name": "Wireless Keyboard", "price": 79.99, "in_stock": 1}
  ],
  "count": 1
}
```

---

6. Transactions

When you need multiple operations to succeed or fail together, use transactions:

```
import { Router } from "tina4-nodejs";
import { Database } from "tina4-nodejs/orm";

Router.post("/api/orders", async (req, res) => {
    const db = await Database.getConnection();
    const body = req.body;
```

```
    try {
        await db.startTransaction();

        await db.execute(
            "INSERT INTO orders (customer_id, total, status) VALUES (:customerId, :total, 'pendi
            { customerId: body.customer_id, total: body.total }
        );

        const order = await db.fetchOne("SELECT last_insert_rowid() AS id");
        const orderId = order.id;

        for (const item of body.items) {
            await db.execute(
                "INSERT INTO order_items (order_id, product_id, quantity, price) VALUES (:orderI
                {
                    orderId,
                    productId: item.product_id,
                    qty: item.quantity,
                    price: item.price
                }
            );

            await db.execute(
                "UPDATE products SET stock = stock - :qty WHERE id = :productId",
                { qty: item.quantity, productId: item.product_id }
            );
        }

        await db.commit();

        return res.status(201).json({ order_id: orderId, status: "created" });
    } catch (e) {
        await db.rollback();
        return res.status(500).json({ error: `Order failed: ${e.message}` });
    }
});
```

---

7. Schema Inspection

getTables()

```
const db = await Database.getConnection();
const tables = await db.getTables();
// Returns: ["orders", "order_items", "products", "users"]
```

getColumns()

```
const columns = await db.getColumns("products");
// Returns: [
//     { name: "id", type: "INTEGER", nullable: false, primary: true },
//     { name: "name", type: "TEXT", nullable: false, primary: false },
//     ...
// ]
```

tableExists()

```
if (await db.tableExists("products")) {
    // Table exists, safe to query
}
```

getDatabaseType()

Identify the connected database engine at runtime:

```
const dbType = db.getDatabaseType();
// Returns: "sqlite", "postgresql", "mysql", or "mssql"
```

Useful when you need database-specific SQL syntax or want to display the engine in a status page.

Schema Info Endpoint

Combine schema inspection methods to build an introspection API:

```
import { Router } from "tina4-nodejs";
import { Database } from "tina4-nodejs/orm";

Router.get("/api/schema", async (req, res) => {
    const db = await Database.getConnection();

    const tables = await db.getTables();
    const schema: Record<string, any> = {};

    for (const table of tables) {
        schema[table] = await db.getColumns(table);
    }

    return res.json({
        database_type: db.getDatabaseType(),
        tables: schema,
        table_count: tables.length,
    });
});
```

This endpoint returns the full database schema -- every table and every column with its type and constraints. Useful for debugging, admin dashboards, and auto-generating documentation.

---

8. FakeData Seeder

Tina4 includes a `FakeData` generator for populating tables with realistic test data:

```
import { FakeData } from "tina4-nodejs";

const fake = new FakeData();

fake.name();       // "Alice Johnson"
fake.email();      // "alice.johnson@example.com"
fake.phone();      // "+1-555-0142"
fake.address();    // "742 Evergreen Terrace, Springfield"
fake.company();    // "Acme Corp"
fake.sentence();   // "The quick brown fox jumps over the lazy dog."
```

```
fake.paragraph();  // Multiple sentences of lorem text
fake.number(1, 100);   // Random integer between 1 and 100
fake.decimal(0, 1000); // Random decimal
fake.boolean();    // true or false
fake.date("2024-01-01", "2026-12-31"); // Random date in range
fake.uuid();       // "a1b2c3d4-e5f6-..."
```

Seeding a Table

```
import { Database } from "tina4-nodejs/orm";
import { FakeData } from "tina4-nodejs";

const db = await Database.getConnection();
const fake = new FakeData();

for (let i = 0; i < 50; i++) {
    await db.execute(
        "INSERT INTO users (name, email, company) VALUES (:name, :email, :company)",
        {
            name: fake.name(),
            email: fake.email(),
            company: fake.company(),
        }
    );
}

console.log("Seeded 50 users");
```

For ORM models, use the seeder helpers rather than writing the loop yourself:

```
import { seedOrm, seedModels } from "tina4-nodejs/orm";

await seedOrm(User, 20, undefined, 42, { clear: true });
await seedModels([User, Order], 20, { clear: true, seed: 42 });
```

`seedModels()` orders parents before children and clears in reverse order so foreign keys remain valid. `seedOrm()` and `seedModels()` create and seed their own `FakeData`. `seedTable()` deliberately rejects `opts.seed` because its generators belong to the caller; create `new FakeData(42)` and close over it in the field map instead.

FakeData is suitable for tests, development, demos, and load data. It is not cryptographically secure: never use it to create production passwords, tokens, API keys, or other secrets.

Run it as a script:

```
npx tsx scripts/seed-users.ts
```

FakeData generates consistent-looking data without external packages. Use it for development, demos, and test setup.

---

9. Batch Operations with executeMany()

Insert or update many rows efficiently:

```
const db = await Database.getConnection();

const products = [
```

```
    { name: "Widget A", price: 9.99 },
    { name: "Widget B", price: 14.99 },
    { name: "Widget C", price: 19.99 },
    { name: "Widget D", price: 24.99 }
];

await db.executeMany(
    "INSERT INTO products (name, price) VALUES (:name, :price)",
    products
);
```

---

10. Helper Methods: insert(), update(), delete()

insert()

```
const db = await Database.getConnection();

await db.insert("products", {
    name: "Wireless Mouse",
    price: 34.99,
    in_stock: 1
});

// Insert multiple rows - call insert() once per row
await db.insert("products", { name: "USB Cable", price: 9.99, in_stock: 1 });
await db.insert("products", { name: "HDMI Cable", price: 14.99, in_stock: 1 });
```

update()

```
// Filter is a column→value equality map (4th arg is optional positional params for raw filters)
await db.update("products", { price: 39.99, in_stock: 1 }, { id: 7 });
```

delete()

```
await db.delete("products", { id: 7 });
```

---

11. Migrations

Migrations are versioned SQL scripts. They evolve your database schema over time. Each migration runs once. Never again.

File Naming

Tina4 supports two naming patterns for migration files:

- **Sequential:** `000001_create_products.sql`
- **Timestamp:** `YYYYMMDDHHMMSS_create_products.sql`

Both patterns sort correctly. Tina4 uses BigInt comparison internally, so you can mix them in the same project without issues.

Generating a Migration

```
tina4 generate migration create_products_table
```

This creates two files in the `migrations/` folder:

```
migrations/20260324120000_create_products_table.sql
migrations/20260324120000_create_products_table.down.sql
```

The first file is the forward (up) migration. The second is the down migration used for rollbacks. Edit each one separately.

Forward migration (`20260324120000_create_products_table.sql`):

```
CREATE TABLE products (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    category TEXT NOT NULL DEFAULT 'Uncategorized',
    price REAL NOT NULL DEFAULT 0.00,
    in_stock INTEGER NOT NULL DEFAULT 1,
    created_at TEXT DEFAULT CURRENT_TIMESTAMP,
    updated_at TEXT DEFAULT CURRENT_TIMESTAMP
);
```

Down migration (`20260324120000_create_products_table.down.sql`):

```
DROP TABLE IF EXISTS products;
```

Down migrations are optional. If you skip them, rollback will warn you but still remove the tracking record.

Running Migrations

```
tina4 migrate
# or
tina4 migrate
```

Each run increments a batch number. Every migration applied during that run belongs to the same batch. This matters for rollback.

Checking Status

```
tina4 migrate:status
```

This shows which migrations have been applied and which are still pending.

Rolling Back

```
tina4 migrate:rollback
```

Rollback undoes the entire last batch. It finds each migration in the batch, runs its `.down.sql` file, and removes the tracking record. If a `.down.sql` file is missing, rollback warns but still cleans up the tracking entry.

The Tracking Table

Tina4 creates a `tina4_migration` table automatically. It has these columns:

Column	Purpose
`id`	Auto-incrementing primary key
`description`	The migration filename

`content`	Full SQL text of the migration (for audit)
`passed`	Whether the migration ran successfully
`batch`	Which batch this migration belongs to
`run_at`	When the migration was applied

Advanced SQL Splitting

Migration files can contain multiple statements. Tina4 splits them on semicolons, but it is smart about edge cases:

- `$$` **delimited blocks** for PostgreSQL stored procedures and functions
- `//` **blocks** for procedure definitions
- `/* */` **block comments** are preserved
- `--` **line comments** are preserved

This means you can write PostgreSQL stored procedures in your migration files without Tina4 breaking on internal semicolons.

---

12. Query Caching

Enable query caching in `.env`:

```
TINA4_DB_CACHE=true
```

Identical queries with identical parameters return cached results. The cache invalidates itself when you call `execute()`, `insert()`, `update()`, or `delete()` on the same table.

```
// fetch() takes (sql, params?, limit?, offset?) - there is no per-call cache bypass.
const products = await db.fetch("SELECT * FROM products", [], 10, 0);

// Clear the entire cache
db.cacheClear();
```

---

13. Exercise: Build a Notes App

Build a notes application backed by SQLite. Create the database table via a migration and build a full CRUD API.

Requirements

- Create a migration for a `notes` table with: `id`, `title`, `content`, `tag`, `created_at`, `updated_at`
- Build these API endpoints:

Method	Path	Description

`GET`	`/api/notes`	List all notes. Support `?tag=` and `?search=` filters.
`GET`	`/api/notes/{id:int}`	Get a single note. 404 if not found.
`POST`	`/api/notes`	Create a note. Validate title and content are not empty.
`PUT`	`/api/notes/{id:int}`	Update a note. 404 if not found.
`DELETE`	`/api/notes/{id:int}`	Delete a note. 204 on success, 404 if not found.

---

14. Solution

Migration

Generate the migration files:

```
tina4 generate migration create_notes_table
```

Edit `migrations/20260324120000_create_notes_table.sql`:

```
CREATE TABLE notes (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    title TEXT NOT NULL,
    content TEXT NOT NULL,
    tag TEXT NOT NULL DEFAULT 'general',
    created_at TEXT DEFAULT CURRENT_TIMESTAMP,
    updated_at TEXT DEFAULT CURRENT_TIMESTAMP
);
```

Edit `migrations/20260324120000_create_notes_table.down.sql`:

```
DROP TABLE IF EXISTS notes;
```

```
tina4 migrate
```

Routes

Create `src/routes/notes.ts`:

```
import { Router } from "tina4-nodejs";
import { Database } from "tina4-nodejs/orm";

Router.get("/api/notes", async (req, res) => {
    const db = await Database.getConnection();
    const tag = req.query.tag ?? "";
    const search = req.query.search ?? "";

    let sql = "SELECT * FROM notes";
    const params: Record<string, any> = {};
    const conditions: string[] = [];
```

```
    if (tag) {
        conditions.push("tag = :tag");
        params.tag = tag;
    }

    if (search) {
        conditions.push("(title LIKE :search OR content LIKE :search)");
        params.search = `%${search}%`;
    }

    if (conditions.length > 0) {
        sql += " WHERE " + conditions.join(" AND ");
    }

    sql += " ORDER BY updated_at DESC";

    const notes = await db.fetch(sql, params);

    return res.json({ notes, count: notes.length });
});

Router.get("/api/notes/{id:int}", async (req, res) => {
    const db = await Database.getConnection();
    const id = req.params.id;

    const note = await db.fetchOne("SELECT * FROM notes WHERE id = :id", { id });

    if (note === null) {
        return res.status(404).json({ error: "Note not found", id });
    }

    return res.json(note);
});

Router.post("/api/notes", async (req, res) => {
    const db = await Database.getConnection();
    const body = req.body;

    const errors: string[] = [];
    if (!body.title) errors.push("Title is required");
    if (!body.content) errors.push("Content is required");

    if (errors.length > 0) {
        return res.status(400).json({ errors });
    }

    await db.execute(
        "INSERT INTO notes (title, content, tag) VALUES (:title, :content, :tag)",
        {
            title: body.title,
            content: body.content,
            tag: body.tag ?? "general"
        }
    );

    const note = await db.fetchOne("SELECT * FROM notes WHERE id = last_insert_rowid()");

    return res.status(201).json(note);
});
```

```
Router.put("/api/notes/{id:int}", async (req, res) => {
    const db = await Database.getConnection();
    const id = req.params.id;
    const body = req.body;

    const existing = await db.fetchOne("SELECT * FROM notes WHERE id = :id", { id });

    if (existing === null) {
        return res.status(404).json({ error: "Note not found", id });
    }

    await db.execute(
        "UPDATE notes SET title = :title, content = :content, tag = :tag, updated_at = CURRENT_T
        {
            title: body.title ?? existing.title,
            content: body.content ?? existing.content,
            tag: body.tag ?? existing.tag,
            id
        }
    );

    const note = await db.fetchOne("SELECT * FROM notes WHERE id = :id", { id });

    return res.json(note);
});

Router.delete("/api/notes/{id:int}", async (req, res) => {
    const db = await Database.getConnection();
    const id = req.params.id;

    const existing = await db.fetchOne("SELECT * FROM notes WHERE id = :id", { id });

    if (existing === null) {
        return res.status(404).json({ error: "Note not found", id });
    }

    await db.execute("DELETE FROM notes WHERE id = :id", { id });

    return res.status(204).json(null);
});
```

Expected output for create (Status: `201 Created`):

```
{
  "id": 1,
  "title": "Shopping List",
  "content": "Milk, eggs, bread",
  "tag": "personal",
  "created_at": "2026-03-22 14:30:00",
  "updated_at": "2026-03-22 14:30:00"
}
```

---

15. Gotchas

1. Forgetting await

Problem: Database operations return ~~`Promise {<pending>}`~~ instead of results.

Cause: You forgot to `await` the database call. All Tina4 database methods are async.

Fix: Always `await` database calls: `const result = await db.fetch(...)`.

2. Connection String Formats

Problem: The database will not connect.

Cause: Each database engine expects a specific URL format. A common mistake is omitting the port.

Fix: Always include the port. Default ports: PostgreSQL 5432, MySQL 3306, MSSQL 1433, Firebird 3050.

3. SQLite File Paths

Problem: SQLite creates a new empty database instead of using the existing one.

Cause: Use three slashes for a relative path: `sqlite:///data/app.db`. Four slashes for absolute: `sqlite:////var/data/app.db`.

4. Parameterised Queries with LIKE

Problem: `WHERE name LIKE '%:q%'` does not work.

Cause: Parameters inside quotes are literal text, not placeholders.

Fix: Include the `%` in the parameter value: `{ q: "%" + search + "%" }`. The SQL should be `WHERE name LIKE :q`.

5. Boolean Values in SQLite

Problem: SQLite stores booleans as integers (1 and 0).

Fix: Cast in your TypeScript code: `inStock: Boolean(row.in_stock)`.

6. Migration Already Applied

Problem: You edited a migration file and ran `tina4 migrate` again, but nothing changed.

Cause: Once applied, a migration will not run again.

Fix: Create a new migration for schema changes. Do not edit applied migrations. If you need to undo, run `tina4 migrate:rollback` first, then fix the migration and re-run.

8. Down Migration Missing on Rollback

Problem: You ran `tina4 migrate:rollback` but the table was not dropped.

Cause: The `.down.sql` file is missing. Rollback removes the tracking record but cannot undo the schema change without it.

Fix: Always generate migrations with `tina4 generate migration`, which creates both files. If you created the migration manually, add the `.down.sql` file before you need to roll back.

7. fetch() Returns Empty Array, Not Null

Problem: You check `if (result === null)` but it never matches when the table is empty.

Cause: `fetch()` always returns an array. Only `fetchOne()` returns `null`.

Fix: Check with `if (result.length === 0)`.

ORM

1. From SQL to Objects

The last chapter was raw SQL. It works. It also gets repetitive. Every insert demands an INSERT statement. Every update demands an UPDATE. Every fetch maps column names to object keys. Over and over.

Tina4's ORM turns database rows into TypeScript objects. Define a model class with fields. The ORM writes the SQL. It stays SQL-first -- you can drop to raw SQL at any moment -- but for the 90% case of CRUD operations, the ORM handles the grunt work.

Picture a blog. Authors, posts, comments. Authors own many posts. Posts own many comments. Comments belong to posts. Modeling these relationships with raw SQL means JOINs and manual foreign key management. The ORM makes this declarative.

ORM at a Glance: Four Languages, One Shape

The ORM does the same job in every Tina4 book. Define a model. Save it. Query it. Each language wears its own clothes (PHP uses typed properties, Python uses field class instances, Ruby uses a DSL, Node uses config objects), but the operations line up. If you know the API in one book, you can read the others.

Defining a Model

The same `Post` model with `id`, `title`, `body`, and `created_at`:

Python - field class instances on the class body:

```
from tina4_python.orm import ORM, IntegerField, StringField, DateTimeField

class Post(ORM):
    table_name = "posts"

    id = IntegerField(primary_key=True, auto_increment=True)
    title = StringField(required=True, max_length=200)
    body = StringField(default="")
    created_at = DateTimeField()
```

PHP - native typed properties:

```
<?php
use Tina4\ORM;

class Post extends ORM
{
    public string $tableName = "posts";

    public int $id;
    public string $title;
    public string $body = "";
    public string $createdAt;
```

```
}
```

Ruby - class-level DSL declarations:

```
class Post < Tina4::ORM
  table_name "posts"

  integer_field :id, primary_key: true, auto_increment: true
  string_field :title, nullable: false, length: 200
  string_field :body, default: ""
  datetime_field :created_at
end
```

Node.js (TypeScript) - config objects in a `static fields` block:

```
import { BaseModel } from "tina4-nodejs/orm";

export default class Post extends BaseModel {
  static tableName = "posts";
  static fields = {
    id:        { type: "integer" as const, primaryKey: true, autoIncrement: true },
    title:     { type: "string"  as const, required: true, maxLength: 200 },
    body:      { type: "string"  as const, default: "" },
    createdAt: { type: "datetime" as const },
  };
}
```

Common Query Operations

Same operation, four shapes:

Operation	Python	PHP	Ruby	Node.js
Find by primary key	`Post.find_by_id(1)`	`Post::findById(1)`	`Post.find_by_id(1)`	`Post.findById(1)`
Filter by attributes	`Post.find({"title": "x"})`	`Post::find(["title" => "x"])`	`Post.find(title: "x")`	`Post.find({ title: "x" })`
Raw SQL where clause	`Post.where("title = ?", ["x"])`	`(new Post())->where("title = ?", ["x"])`	`Post.where("title = ?", ["x"])`	`Post.where("title = ?", ["x"])`
Build and save	`Post.create(title="x")`	`Post::create(["title" => "x"])`	`Post.create(title: "x")`	`Post.create({ title: "x" })`
Save an instance	`post.save()`	`$post->save()`	`post.save`	`post.save()`
Fetch every row	`Post.all()`	`(new Post())->all()`	`Post.all`	`Post.all()`

Delete a record	`post.delete()`	`$post->delete()`	`post.delete`	`post.delete()`
Count rows	`Post.count()`	`(new Post())->count()`	`Post.count`	`Post.count()`

A few details worth noting. `find()` takes attribute names and applies the field map; `where()` takes raw SQL and skips translation. PHP needs `(new Post())` for instance methods like `where()` and `all()`; the rest are static. Ruby methods drop the parentheses by convention.

For full detail on field options, relationships, eager loading, soft delete, validation, and Auto-CRUD, read the rest of this chapter; it shows the API for the language of this book.

---

2. Defining a Model

Create a model file in `src/models/`. Every `.ts` file in that directory is auto-loaded at startup.

Create `src/models/Note.ts`:

```
import { BaseModel } from "tina4-nodejs/orm";

export default class Note extends BaseModel {
  static tableName = "notes";
  static fields = {
    id:        { type: "integer" as const, primaryKey: true, autoIncrement: true },
    title:     { type: "string" as const, required: true, maxLength: 200 },
    content:   { type: "string" as const, default: "" },
    category:  { type: "string" as const, default: "general" },
    pinned:    { type: "boolean" as const, default: false },
    createdAt: { type: "datetime" as const },
    updatedAt: { type: "datetime" as const },
  };
}
```

A complete model. Here is what each piece does:

- `static tableName` -- the database table this model maps to. If omitted, the ORM uses the lowercase class name (e.g. `Contact` -> `contact`).
- `primaryKey: true` on a field marks it as the primary key (defaults to `id` if none is specified)
- Each field is a property in the `static fields` object with a config object describing its type and constraints

Field Types

Field Type	TypeScript Type	SQL Type	Description
`"integer"`	`number`	`INTEGER`	Whole numbers
`"string"`	`string`	`TEXT`	Text strings
`"text"`	`string`	`TEXT`	Long text
`"number"`	`number`	`REAL`	Decimal numbers
`"boolean"`	`boolean`	`INTEGER` (0/1)	True/False
`"datetime"`	`string`	`TEXT`	Date and time

For foreign keys, use `"integer"`. There is no separate foreign key type -- the relationship is defined through `hasMany`, `hasOne`, and `belongsTo` methods instead.

Field Options

Option	Type	Description
`primaryKey`	`boolean`	Marks this field as the primary key
`required`	`boolean`	Field must have a value (not undefined)
`default`	any	Default value when not provided
`maxLength`	`number`	Maximum string length
`minLength`	`number`	Minimum string length
`min`	`number`	Minimum numeric value
`max`	`number`	Maximum numeric value
`choices`	array	Allowed values
`autoIncrement`	`boolean`	Auto-incrementing integer
`pattern`	`string`	Regex pattern the value must match

Field Mapping

When your TypeScript property names do not match the database column names, use `static fieldMapping` to define the translation:

```
import { BaseModel } from "tina4-nodejs/orm";

export default class User extends BaseModel {
  static tableName = "user_accounts";
```

```
  static fieldMapping = {
    firstName: "fname",        // TS property -> DB column
    lastName:  "lname",
    emailAddress: "email",
  };

  static fields = {
    id:           { type: "integer" as const, primaryKey: true, autoIncrement: true },
    firstName:    { type: "string" as const, required: true },
    lastName:     { type: "string" as const, required: true },
    emailAddress: { type: "string" as const, required: true },
  };
}
```

With this mapping, `user.firstName` reads from and writes to the `fname` column. The ORM handles the conversion in both directions -- on `findById()`, `save()`, `select()`, and `toDict()`. This is useful with legacy databases or third-party schemas where you cannot rename the columns.

autoMap and Case Conversion

The `static autoMap = true` flag auto-generates `fieldMapping` entries from camelCase field names to snake_case database column names. Explicit `fieldMapping` entries always take precedence.

```
import { BaseModel } from "tina4-nodejs/orm";

export default class User extends BaseModel {
  static tableName = "users";
  static autoMap = true;  // firstName -> first_name, lastName -> last_name

  static fields = {
    id:        { type: "integer" as const, primaryKey: true, autoIncrement: true },
    firstName: { type: "string" as const, required: true },
    lastName:  { type: "string" as const, required: true },
    email:     { type: "string" as const, required: true },
  };
}
```

Two utility functions handle case conversion directly:

```
import { snakeToCamel, camelToSnake } from "tina4-nodejs/orm";

snakeToCamel("first_name");   // "firstName"
camelToSnake("firstName");    // "first_name"
```

A common use case is Firebird or Oracle, which store column names in uppercase:

```
import { BaseModel } from "tina4-nodejs/orm";

export default class Account extends BaseModel {
  static tableName    = "ACCOUNTS";
  static fieldMapping: Record<string, string> = {
    accountNo:   "ACCOUNTNO",
    storeName:   "STORENAME",
    creditLimit: "CREDITLIMIT",
  };

  static fields = {
```

```
    accountNo:   { type: "string" as const },
    storeName:   { type: "string" as const },
    creditLimit: { type: "number" as const, default: 0 },
  };
}
```

TypeScript code uses clean camelCase names (`account.accountNo`, `account.creditLimit`). The ORM maps them to the uppercase DB columns automatically.

getDbColumn and getDbData

Two helpers expose the fieldMapping in custom code:

```
// Get the DB column name for a JS property
const col = Account.getDbColumn("accountNo");   // "ACCOUNTNO"

// Get all instance fields using DB column names as keys
const data = account.getDbData();
// { ACCOUNTNO: "A001", STORENAME: "Main Store", CREDITLIMIT: 5000 }
```

These are used internally by `save()` and `createTable()`, but available for custom queries.

find() vs where() -- naming convention

The two query methods have a deliberate difference in how they handle column names:

- `find(filterObj)` uses **TypeScript property names**. The ORM translates them via `fieldMapping`.
- `where(sql)` uses **raw DB column names** in the SQL string. No translation is applied.

```
// find() -- use TS property names (fieldMapping applied)
const accounts = await Account.find({ accountNo: "A001" });   // translates to ACCOUNTNO = ?

// where() -- use DB column names directly in the SQL
const accounts2 = await Account.where("ACCOUNTNO = ?", ["A001"]);  // raw SQL, no translation
```

This means `find()` is portable across database engines, while `where()` gives you full control of the SQL.

---

3. createTable -- Schema from Models

You can create the database table directly from your model definition:

```
await Note.createTable();
```

This generates and runs the CREATE TABLE SQL based on your field definitions. It is good for development and testing. For production, use migrations (Chapter 5) for version-controlled schema changes.

If the table already exists, `createTable()` does nothing.

---

4. CRUD Operations

save -- Create or Update

```
// src/routes/api/notes/post.ts
import type { Tina4Request, Tina4Response } from "tina4-nodejs";
import Note from "../../models/Note.js";

export default async function (req: Tina4Request, res: Tina4Response) {
  const body = req.body as Record<string, unknown>;

  const note = new Note();
  note.title = body.title;
  note.content = body.content ?? "";
  note.category = body.category ?? "general";
  note.pinned = body.pinned ?? false;
  await note.save();

  res.json({ message: "Note created", note: note.toDict() }, 201);
}
```

`save()` detects whether the record is new (INSERT) or existing (UPDATE) based on whether the primary key has a value. It returns `this` on success for chaining. It returns `false` on failure.

create -- Build and Save in One Step

When you have a data object ready, `create()` builds the model and saves it in one call:

```
const note = await Note.create({
  title: "Quick Note",
  content: "Created in one step",
  category: "general",
});
```

findById -- Fetch One Record by Primary Key

```
// src/routes/api/notes/[id]/get.ts
import type { Tina4Request, Tina4Response } from "tina4-nodejs";
import Note from "../../../models/Note.js";

export default async function (req: Tina4Request, res: Tina4Response) {
  const note = await Note.findById(req.params.id);

  if (note === null) {
    return res.json({ error: "Note not found" }, 404);
  }

  res.json(note.toDict());
}
```

`findById()` takes a primary key value and returns a model instance, or `null` if no row matches. If soft delete is enabled, it excludes soft-deleted records.

Use `findOrFail()` when you want an Error thrown instead of `null`:

```
const note = await Note.findOrFail(id);  // Throws Error if not found
```

find -- Query by Filter Dict

The `find()` method accepts an object of column-value pairs and returns an array of matching records:

```
// Find all notes in the "work" category
const workNotes = await Note.find({ category: "work" });

// Find with pagination and ordering
const recent = await Note.find({ pinned: true }, 10, 0, "created_at DESC");

// Find all records (no filter)
const allNotes = await Note.find();
```

The full signature is `find(filter?, limit?, offset?, orderBy?, include?)`.

where -- Query with SQL Conditions

For more complex queries, `where()` takes a SQL WHERE clause with `?` placeholders:

```
const notes = await Note.where("category = ?", ["work"]);
```

delete -- Remove a Record

```
// src/routes/api/notes/[id]/delete.ts
import type { Tina4Request, Tina4Response } from "tina4-nodejs";
import Note from "../../../models/Note.js";

export default async function (req: Tina4Request, res: Tina4Response) {
  const note = await Note.findById(req.params.id);

  if (note === null) {
    return res.json({ error: "Note not found" }, 404);
  }

  await note.delete();

  res.json(null, 204);
}
```

Listing Records

```
// src/routes/api/notes/get.ts
import type { Tina4Request, Tina4Response } from "tina4-nodejs";
import Note from "../../models/Note.js";

export default async function (req: Tina4Request, res: Tina4Response) {
  const category = req.query.category;

  let notes;
  if (category) {
    notes = await Note.where("category = ?", [category]);
  } else {
    notes = await Note.all();
  }

  res.json({
    notes: notes.map((n) => n.toDict()),
    count: notes.length,
  });
}
```

`where()` takes a WHERE clause with `?` placeholders and an array of parameters. It returns a `ModelCollection` (covered just below). `all()` fetches all records. Both support pagination:

```
// With pagination
const notes = await Note.where("category = ?", ["work"], 20, 40);

// Fetch all with pagination -- all() takes an optional where clause string
const notes2 = await Note.all("category = ?", ["work"]);

// SQL-first query -- full control over the SQL
const notes3 = await Note.select(
  "SELECT * FROM notes WHERE pinned = ? ORDER BY created_at DESC",
  [1],
);
```

ModelCollection -- The Page and the Total

Pagination hides an awkward gap. You ask for 20 rows and you get 20 rows, but a pager needs to say "page 3 of 13", and that means knowing the total number of matching rows, not the 20 sitting on this page. The old answer was a second `COUNT(*)` query with the same filter written out again by hand.

Tina4 closes the gap. `where()`, `select()`, `find()` (the filter form), `all()`, and `withTrashed()` all return a `ModelCollection`. It extends `Array`, so `Array.isArray()` stays true and you iterate it, index it, `.map()` it, read `.length`, and `JSON.stringify` it exactly as before. It just carries one extra thing: the total for the filter, independent of `limit` and `offset`. Remember Node's `where()` takes `limit` and `offset` positionally, after the params:

```
const rows = await User.where("active = ?", [1], 20, 40);  // a page of up to 20 models
rows.getTotalRecords();   // 250 -- the whole matching set, ignoring limit/offset
```

That total is free. Every one of those methods already runs a `COUNT(*)` probe when it fetches the page, and the ORM used to hydrate the models and throw the count away. `ModelCollection` keeps it instead, so `getTotalRecords()` fires no second query. It is a method, not a property, so the accessor is spelled the same way across all four frameworks.

The single-record finders are untouched. `findById()`, `selectOne()`, and the primary-key finders still return one model or `null`.

Call `toPaginate()` for a ready-made pagination envelope. It hands back the same seven keys as `(await db.fetch(...)).toPaginate()`, so a route paginates the same way whether the data came through the ORM or through raw SQL:

```
// src/routes/api/notes/get.ts
export default async function (req: Tina4Request, res: Tina4Response) {
  const page = await Note.where("category = ?", ["work"], 20, 40);
  res.json(page.toPaginate());
  // { "records": [...20...], "total": 250, "page": 3, "per_page": 20,
  //   "total_pages": 13, "limit": 20, "offset": 40 }
}
```

The keys (`records`, `total`, `page`, `per_page`, `total_pages`, `limit`, `offset`) are snake_case and identical in all four frameworks, so a client reads the same JSON everywhere.

selectOne -- Fetch a Single Record by SQL

When you need exactly one record from a custom SQL query:

```
const note = await Note.selectOne("SELECT * FROM notes WHERE slug = ?", ["my-note"]);
```

Returns a model instance or `null`.

load -- Populate an Existing Instance

The `load()` method fills an existing model instance from the database:

```
const note = new Note();
note.id = 42;
await note.load();  // Loads data for id=42

// Or with a filter string
const note2 = new Note();
await note2.load("slug = ?", ["my-note"]);
```

Returns `true` if a record was found, `false` otherwise.

count -- Count Records

```
const total = await Note.count();
const workCount = await Note.count("category = ?", ["work"]);
```

Respects soft delete -- only counts non-deleted records.

---

5. toDict, toJson, and Other Serialisation

toDict

Convert a model instance to a plain object:

```
const note = await Note.findById(1);

const data = note.toDict();
// { id: 1, title: "Shopping List", content: "Milk, eggs", category: "personal", pinned: false,
```

The `include` parameter adds relationship data to the output (see Eager Loading below). Pass an array of relationship names:

```
// Include relationships in the output
const data = note.toDict(["comments"]);
```

toJson

Convert directly to a JSON string:

```
const jsonString = note.toJson();
// '{"id":1,"title":"Shopping List",...}'
```

Other Serialisation Methods

Method	Returns	Description
`toDict(include?)`	`Record<string, unknown>`	Primary dict method with optional relationship includes
`toAssoc(include?)`	`Record<string, unknown>`	Alias for `toDict()`
`toObject()`	`Record<string, unknown>`	Alias for `toDict()`
`toJson(include?)`	`string`	JSON string
`toArray()`	`unknown[]`	Flat array of values (no keys)
`toList()`	`unknown[]`	Alias for `toArray()`

---

6. Relationships

foreignKey Field Type: Auto-Wired Relationships

Declaring a field with `type: "foreignKey"` and a `references` model name automatically wires both sides of the relationship. The declaring model gets a `belongsTo` entry (the column name with `_id` stripped → the association name), and the referenced model gets a `hasMany` entry (the declaring model's table name, or whatever you pass via `relatedName`).

```
import { BaseModel } from "tina4-nodejs/orm";

export class User extends BaseModel {
    static tableName = "users";
    static fields = {
        id:   { type: "integer", primaryKey: true, autoIncrement: true },
        name: { type: "string", required: true },
    };
}

export class Post extends BaseModel {
    static tableName = "posts";
    static fields = {
```

```
        id:      { type: "integer", primaryKey: true, autoIncrement: true },
        title:   { type: "string", required: true },
        // Auto-wires Post.belongsTo User and User.hasMany Post
        user_id: { type: "foreignKey", references: "User" },
    };
}
```

With just the `foreignKey` field, both sides are accessible:

```
const post = await Post.findById(1);
const user = await post.belongsTo(User, "user_id");
console.log(user?.name);     // "Alice"

// Or via toDict with include
const postData = post.toDict(["user"]);

const alice = await User.findById(1);
const posts = await alice.hasMany(Post, "user_id");
posts.forEach(p => console.log(p.title));
```

For a custom `hasMany` key, pass `relatedName`:

```
user_id: { type: "foreignKey", references: "User", relatedName: "blog_posts" }
// User.hasMany entry will use "blog_posts" instead of the default
```

Models must be registered via `BaseModel.registerModel("User", User)` before eager loading can resolve the reference by name.

hasMany

An author has many posts:

Create `src/models/Author.ts`:

```
import { BaseModel } from "tina4-nodejs/orm";

export default class Author extends BaseModel {
  static tableName = "authors";
  static fields = {
    id:        { type: "integer" as const, primaryKey: true, autoIncrement: true },
    name:      { type: "string" as const, required: true },
    email:     { type: "string" as const, required: true },
    bio:       { type: "string" as const, default: "" },
    createdAt: { type: "datetime" as const },
  };
}
```

Create `src/models/BlogPost.ts`:

```
import { BaseModel } from "tina4-nodejs/orm";

export default class BlogPost extends BaseModel {
  static tableName = "posts";
  static fields = {
    id:        { type: "integer" as const, primaryKey: true, autoIncrement: true },
    authorId:  { type: "integer" as const, required: true },
    title:     { type: "string" as const, required: true, maxLength: 300 },
    slug:      { type: "string" as const, required: true },
    content:   { type: "string" as const, default: "" },
    status:    { type: "string" as const, default: "draft", choices: ["draft", "published", "arc
```

```
    createdAt: { type: "datetime" as const },
    updatedAt: { type: "datetime" as const },
  };
}
```

Now use `hasMany` to get an author's posts:

```
// src/routes/api/authors/[id]/get.ts
import type { Tina4Request, Tina4Response } from "tina4-nodejs";
import Author from "../../../models/Author.js";
import BlogPost from "../../../models/BlogPost.js";

export default async function (req: Tina4Request, res: Tina4Response) {
  const author = await Author.findById(req.params.id);

  if (author === null) {
    return res.json({ error: "Author not found" }, 404);
  }

  const posts = await author.hasMany(BlogPost, "author_id");

  const data = author.toDict();
  data.posts = posts.map((p) => p.toDict());

  res.json(data);
}
```

```
{
  "id": 1,
  "name": "Alice",
  "email": "alice@example.com",
  "bio": "Tech writer",
  "posts": [
    {"id": 1, "title": "Getting Started with Tina4", "slug": "getting-started", "status": "publi
    {"id": 2, "title": "Advanced Routing", "slug": "advanced-routing", "status": "draft"}
  ]
}
```

`hasMany()` accepts an optional `limit` and `offset` for pagination: `author.hasMany(BlogPost, "author_id", 10, 0)`.

hasOne

A user has one profile:

```
const profile = await user.hasOne(Profile, "user_id");
```

Returns a single model instance or `null`.

belongsTo

A post belongs to an author:

```
// src/routes/api/posts/[id]/get.ts
import type { Tina4Request, Tina4Response } from "tina4-nodejs";
import Author from "../../../models/Author.js";
import BlogPost from "../../../models/BlogPost.js";

export default async function (req: Tina4Request, res: Tina4Response) {
  const post = await BlogPost.findById(req.params.id);
```

```
  if (post === null) {
    return res.json({ error: "Post not found" }, 404);
  }

  const author = await post.belongsTo(Author, "author_id");

  const data = post.toDict();
  data.author = author ? author.toDict() : null;

  res.json(data);
}
```

```
{
  "id": 1,
  "authorId": 1,
  "title": "Getting Started with Tina4",
  "slug": "getting-started",
  "content": "...",
  "status": "published",
  "author": {
    "id": 1,
    "name": "Alice",
    "email": "alice@example.com"
  }
}
```

---

7. Eager Loading

Calling relationship methods inside a loop creates the N+1 problem. Load 10 authors. Call `hasMany(BlogPost, "author_id")` for each one. That fires 11 queries -- 1 for authors, 10 for posts. The page drags.

The `include` parameter on `all()`, `where()`, `findById()`, and `selectOne()` solves this. It eager-loads relationships in bulk.

Eager loading requires two things: declarative relationship definitions on the model, and model registration.

Declarative Relationships

Define relationships as static arrays on your model class:

```
import { BaseModel } from "tina4-nodejs/orm";

export default class Author extends BaseModel {
  static tableName = "authors";
  static hasMany = [{ model: "BlogPost", foreignKey: "author_id" }];
  static fields = {
    id:    { type: "integer" as const, primaryKey: true, autoIncrement: true },
    name:  { type: "string" as const, required: true },
    email: { type: "string" as const, required: true },
    bio:   { type: "string" as const, default: "" },
  };
}
```

```
import { BaseModel } from "tina4-nodejs/orm";
```

```
export default class BlogPost extends BaseModel {
  static tableName = "posts";
  static belongsTo = [{ model: "Author", foreignKey: "author_id" }];
  static hasMany = [{ model: "Comment", foreignKey: "post_id" }];
  static fields = {
    id:       { type: "integer" as const, primaryKey: true, autoIncrement: true },
    authorId: { type: "integer" as const, required: true },
    title:    { type: "string" as const, required: true },
    status:   { type: "string" as const, default: "draft" },
  };
}
```

Register the models so eager loading can resolve them by name:

```
BaseModel.registerModel("Author", Author);
BaseModel.registerModel("BlogPost", BlogPost);
BaseModel.registerModel("Comment", Comment);
```

Now use `include` to eager-load:

```
// Eager load posts when fetching all authors
const authors = await Author.all(undefined, undefined, ["posts"]);

// Eager load author and comments when finding a single post
const post = await BlogPost.findById(1, ["author", "comments"]);
```

Without eager loading, 10 authors and their posts cost 11 queries. With eager loading: 2 queries. That is the difference between a fast page and a slow one.

Nested Eager Loading

Dot notation loads multiple levels deep:

```
// Load authors, their posts, and each post's comments
const authors = await Author.all(undefined, undefined, ["posts", "posts.comments"]);
```

Authors, their posts, and each post's comments. Three queries total instead of hundreds.

toDict with Nested Includes

When eager loading is active, `toDict(include)` embeds the related data:

```
const post = await BlogPost.findById(1, ["author", "comments"]);
const data = post.toDict(["author", "comments"]);
```

```
{
  "id": 1,
  "title": "Getting Started with Tina4",
  "author": {
    "id": 1,
    "name": "Alice",
    "email": "alice@example.com"
  },
  "comments": [
    {"id": 1, "body": "Great post!", "authorName": "Bob"}
  ]
}
```

---

8. Soft Delete

Sometimes a record needs to disappear from queries without leaving the database. Soft delete handles this. The row stays. A flag marks it as deleted. Queries skip it.

```
import { BaseModel } from "tina4-nodejs/orm";

export default class Task extends BaseModel {
  static tableName = "tasks";
  static softDelete = true;  // Enable soft delete

  static fields = {
    id:        { type: "integer" as const, primaryKey: true, autoIncrement: true },
    title:     { type: "string" as const, required: true },
    completed: { type: "boolean" as const, default: false },
    isDeleted: { type: "integer" as const, default: 0 },  // Required for soft delete (0 = activ
    createdAt: { type: "string" as const },
  };
}
```

When `static softDelete = true`, the ORM changes its behaviour:

- `task.delete()` sets `is_deleted` to `1` instead of running a DELETE query
- `Task.all()`, `Task.where()`, and `Task.findById()` filter out records where `is_deleted = 1`
- `task.restore()` sets `is_deleted` back to `0` and makes the record visible again
- `task.forceDelete()` permanently removes the row from the database
- `Task.withTrashed()` includes soft-deleted records in query results

Deleting and Restoring

```
// Soft delete -- sets is_deleted = 1, row stays in the database
const task = await Task.findById(1);
await task.delete();

// Restore -- sets is_deleted = 0, record is visible again
await task.restore();

// Permanently delete -- removes the row, no recovery possible
await task.forceDelete();
```

`restore()` is the inverse of `delete()`. It sets `is_deleted` back to `0` and commits the change. The record reappears in all standard queries.

Including Soft-Deleted Records

Standard queries (`all()`, `where()`, `findById()`) exclude soft-deleted records. When you need to see everything -- for admin dashboards, audit logs, or data recovery -- use `withTrashed()`:

```
// All tasks, including soft-deleted ones
const allTasks = await Task.withTrashed();

// Soft-deleted tasks matching a condition
const deletedTasks = await Task.withTrashed("completed = ?", [1]);
```

`withTrashed()` accepts the same filter parameters as `where()`: `withTrashed(conditions?, params?, limit?, offset?)`. The only difference: it ignores the `is_deleted` filter that standard queries apply.

Counting with Soft Delete

The `count()` method respects soft delete. It only counts non-deleted records:

```
const activeCount = await Task.count();
const activeWork = await Task.count("category = ?", ["work"]);
```

When to Use Soft Delete

Soft delete suits data that users might want to recover -- emails, documents, user accounts. It also serves audit requirements where regulations demand retention. For temporary data (sessions, cache entries, logs), hard delete keeps the table lean.

---

9. Auto-CRUD

Writing the same five REST endpoints for every model gets tedious. Auto-CRUD generates them from your model class. Define the model. Set the flag. Five routes appear.

The autoCrud Flag

Set `static autoCrud = true` on your model class:

```
import { BaseModel } from "tina4-nodejs/orm";

export default class Note extends BaseModel {
  static tableName = "notes";
  static autoCrud = true;  // Generates REST endpoints automatically

  static fields = {
    id:      { type: "integer" as const, primaryKey: true, autoIncrement: true },
    title:   { type: "string" as const, required: true },
    content: { type: "string" as const, default: "" },
  };
}
```

When the ORM discovers this model at startup, it registers CRUD routes. Five routes appear.

Manual Registration

You can also register routes explicitly using `generateCrudRoutes()`:

```
import { generateCrudRoutes } from "tina4-nodejs/orm";
```

Both approaches produce the same result:

Method	Path	Description
`GET`	`/api/notes`	List all with filtering and pagination
`GET`	`/api/notes/{id}`	Get one by primary key

`POST`	`/api/notes`	Create a new record
`PUT`	`/api/notes/{id}`	Update a record
`DELETE`	`/api/notes/{id}`	Delete a record

The endpoint prefix derives from the table name. The `notes` table becomes `/api/notes`.

What the Generated Routes Do

GET /api/notes returns paginated results with filtering and sorting:

```
curl "http://localhost:7148/api/notes?limit=10&page=1"
```

```
{
  "records": [...],
  "data": [...],
  "total": 42,
  "count": 42,
  "limit": 10,
  "page": 1,
  "perPage": 10,
  "totalPages": 5
}
```

The list endpoint supports query parameters:

- `?filter[field]=value` -- filter by field value
- `?sort=-name` -- sort by field (prefix `-` for descending)
- `?page=2&limit=10` -- pagination

POST /api/notes validates input before saving:

```
curl -X POST http://localhost:7148/api/notes \
  -H "Content-Type: application/json" \
  -d '{"title": "New Note", "content": "Created via auto-CRUD"}'
```

If validation fails (for example, a required field is missing), the endpoint returns a 422 with error details:

```
{"error": "Validation failed", "statusCode": 422, "errors": ["title: This field is required"]}
```

DELETE /api/notes/1 respects soft delete. If the model has `static softDelete = true`, the record is marked deleted instead of removed.

Custom Routes Alongside Auto-CRUD

Custom routes defined in `src/routes/` load before auto-CRUD routes. They take precedence. If you need special logic for one endpoint (custom validation, side effects, complex queries), define that route as a file. Auto-CRUD handles the rest.

Table Filter

The `static tableFilter` property adds a permanent WHERE condition to all auto-CRUD queries:

```
export default class ActiveUser extends BaseModel {
```

```
  static tableName = "users";
  static tableFilter = "active = 1";
  static autoCrud = true;
  // ...
}
```

Every auto-CRUD query for this model appends `AND active = 1`.

---

10. Scopes

Scopes are reusable query filters baked into the model. In TypeScript, you can define them as static methods:

```
import { BaseModel } from "tina4-nodejs/orm";

export default class BlogPost extends BaseModel {
  static tableName = "posts";
  static fields = {
    id:        { type: "integer" as const, primaryKey: true, autoIncrement: true },
    title:     { type: "string" as const, required: true },
    status:    { type: "string" as const, default: "draft" },
    createdAt: { type: "datetime" as const },
  };

  static published() {
    return this.where("status = ?", ["published"]);
  }

  static drafts() {
    return this.where("status = ?", ["draft"]);
  }

  static recent(days = 7) {
    return this.where(
      "created_at > datetime('now', ?)",
      [`-${days} days`],
    );
  }
}
```

Use them in your routes:

```
// src/routes/api/posts/published/get.ts
import type { Tina4Request, Tina4Response } from "tina4-nodejs";
import BlogPost from "../../../models/BlogPost.js";

export default async function (req: Tina4Request, res: Tina4Response) {
  const posts = await BlogPost.published();
  res.json({ posts: posts.map((p) => p.toDict()) });
}
```

You can also register scopes dynamically with the `scope()` static method:

```
BlogPost.scope("active", "status != ?", ["archived"]);

// Now call it (cast needed since it's dynamically added):
const activePosts = await (BlogPost as any).active();
```

```
// With limit and offset:
const activePosts2 = await (BlogPost as any).active(10, 5);
```

`scope()` returns void. It registers a method on the class that calls `where()` with the given filter. The registered method accepts optional `limit` and `offset` parameters.

Scopes keep query logic in the model where it belongs. Route handlers stay thin.

---

11. Input Validation

Field definitions carry validation rules. Call `validate()` before `save()` and the ORM checks every constraint:

```
import { BaseModel } from "tina4-nodejs/orm";

export default class Product extends BaseModel {
  static tableName = "products";
  static fields = {
    id:       { type: "integer" as const, primaryKey: true, autoIncrement: true },
    name:     { type: "string" as const, required: true, minLength: 2, maxLength: 200 },
    sku:      { type: "string" as const, required: true, pattern: "^[A-Z]{2}-\\d{4}$" },  // e.g
    price:    { type: "number" as const, required: true, min: 0.01, max: 999999.99 },
    category: { type: "string" as const, choices: ["Electronics", "Kitchen", "Office", "Fitness"
  };
}
```

```
// src/routes/api/products/post.ts
import type { Tina4Request, Tina4Response } from "tina4-nodejs";
import Product from "../../models/Product.js";

export default async function (req: Tina4Request, res: Tina4Response) {
  const body = req.body as Record<string, unknown>;

  const product = new Product();
  product.name = body.name;
  product.sku = body.sku;
  product.price = body.price;
  product.category = body.category;

  const errors = product.validate();
  if (errors.length > 0) {
    return res.json({ errors }, 400);
  }

  await product.save();
  res.json({ product: product.toDict() }, 201);
}
```

If validation fails, `validate()` returns an array of error strings:

```
{
  "errors": [
    "name Must be at least 2 characters",
    "sku Must match pattern ^[A-Z]{2}-\\d{4}$",
    "price Must be at least 0.01",
    "category Must be one of: Electronics, Kitchen, Office, Fitness"
  ]
```

```
}
```

---

12. Exercise: Build a Blog with Relationships

Build a blog API with authors, posts, and comments.

Requirements

- Create these models:

Author: `id`, `name` (required), `email` (required), `bio`, `createdAt`

BlogPost: `id`, `authorId` (integer foreign key), `title` (required, max 300), `slug` (required), `content`, `status` (choices: draft/published/archived, default draft), `createdAt`, `updatedAt`

Comment: `id`, `postId` (integer foreign key), `authorName` (required), `authorEmail` (required), `body` (required, min 5 chars), `createdAt`

- Build these endpoints:

Method	Path	Description
`POST`	`/api/authors`	Create an author
`GET`	`/api/authors/{id}`	Get author with their posts
`POST`	`/api/posts`	Create a post (requires authorId)
`GET`	`/api/posts`	List published posts with author info
`GET`	`/api/posts/{id}`	Get post with author and comments
`POST`	`/api/posts/{id}/comments`	Add comment to a post

---

13. Solution

Create `src/models/Author.ts`:

```
import { BaseModel } from "tina4-nodejs/orm";

export default class Author extends BaseModel {
  static tableName = "authors";
  static fields = {
    id:        { type: "integer" as const, primaryKey: true, autoIncrement: true },
    name:      { type: "string" as const, required: true, minLength: 2 },
    email:     { type: "string" as const, required: true },
    bio:       { type: "string" as const, default: "" },
    createdAt: { type: "datetime" as const },
```

```
  };
}
```

Create `src/models/BlogPost.ts`:

```
import { BaseModel } from "tina4-nodejs/orm";

export default class BlogPost extends BaseModel {
  static tableName = "posts";
  static fields = {
    id:        { type: "integer" as const, primaryKey: true, autoIncrement: true },
    authorId:  { type: "integer" as const, required: true },
    title:     { type: "string" as const, required: true, maxLength: 300 },
    slug:      { type: "string" as const, required: true },
    content:   { type: "string" as const, default: "" },
    status:    { type: "string" as const, default: "draft", choices: ["draft", "published", "arc
    createdAt: { type: "datetime" as const },
    updatedAt: { type: "datetime" as const },
  };

  static published() {
    return this.where("status = ?", ["published"]);
  }
}
```

Create `src/models/Comment.ts`:

```
import { BaseModel } from "tina4-nodejs/orm";

export default class Comment extends BaseModel {
  static tableName = "comments";
  static fields = {
    id:          { type: "integer" as const, primaryKey: true, autoIncrement: true },
    postId:      { type: "integer" as const, required: true },
    authorName:  { type: "string" as const, required: true },
    authorEmail: { type: "string" as const, required: true },
    body:        { type: "string" as const, required: true, minLength: 5 },
    createdAt:   { type: "datetime" as const },
  };
}
```

Create `src/routes/api/authors/post.ts`:

```
import type { Tina4Request, Tina4Response } from "tina4-nodejs";
import Author from "../../../models/Author.js";

export default async function (req: Tina4Request, res: Tina4Response) {
  const body = req.body as Record<string, unknown>;

  const author = new Author();
  author.name = body.name;
  author.email = body.email;
  author.bio = body.bio ?? "";

  const errors = author.validate();
  if (errors.length > 0) {
    return res.json({ errors }, 400);
  }

  await author.save();
```

```
  res.json({ author: author.toDict() }, 201);
}
```

Create `src/routes/api/authors/[id]/get.ts`:

```
import type { Tina4Request, Tina4Response } from "tina4-nodejs";
import Author from "../../../../models/Author.js";
import BlogPost from "../../../../models/BlogPost.js";

export default async function (req: Tina4Request, res: Tina4Response) {
  const author = await Author.findById(req.params.id);

  if (author === null) {
    return res.json({ error: "Author not found" }, 404);
  }

  const posts = await BlogPost.where("author_id = ?", [author.id]);

  const data = author.toDict();
  data.posts = posts.map((p) => p.toDict());

  res.json(data);
}
```

Create `src/routes/api/posts/post.ts`:

```
import type { Tina4Request, Tina4Response } from "tina4-nodejs";
import Author from "../../../models/Author.js";
import BlogPost from "../../../models/BlogPost.js";

export default async function (req: Tina4Request, res: Tina4Response) {
  const body = req.body as Record<string, unknown>;

  // Verify author exists
  const author = await Author.findById(body.authorId);
  if (author === null) {
    return res.json({ error: "Author not found" }, 404);
  }

  const post = new BlogPost();
  post.authorId = body.authorId;
  post.title = body.title;
  post.slug = body.slug;
  post.content = body.content ?? "";
  post.status = body.status ?? "draft";

  const errors = post.validate();
  if (errors.length > 0) {
    return res.json({ errors }, 400);
  }

  await post.save();
  res.json({ post: post.toDict() }, 201);
}
```

Create `src/routes/api/posts/get.ts`:

```
import type { Tina4Request, Tina4Response } from "tina4-nodejs";
import Author from "../../../models/Author.js";
import BlogPost from "../../../models/BlogPost.js";
```

```
export default async function (req: Tina4Request, res: Tina4Response) {
  const posts = await BlogPost.published();
  const data = [];

  for (const p of posts) {
    const postDict = p.toDict();
    const author = await p.belongsTo(Author, "author_id");
    postDict.author = author ? author.toDict() : null;
    data.push(postDict);
  }

  res.json({ posts: data, count: data.length });
}
```

Create `src/routes/api/posts/[id]/get.ts`:

```
import type { Tina4Request, Tina4Response } from "tina4-nodejs";
import Author from "../../../../models/Author.js";
import BlogPost from "../../../../models/BlogPost.js";
import Comment from "../../../../models/Comment.js";

export default async function (req: Tina4Request, res: Tina4Response) {
  const post = await BlogPost.findById(req.params.id);

  if (post === null) {
    return res.json({ error: "Post not found" }, 404);
  }

  const author = await post.belongsTo(Author, "author_id");
  const comments = await post.hasMany(Comment, "post_id");

  const data = post.toDict();
  data.author = author ? author.toDict() : null;
  data.comments = comments.map((c) => c.toDict());
  data.commentCount = comments.length;

  res.json(data);
}
```

Create `src/routes/api/posts/[id]/comments/post.ts`:

```
import type { Tina4Request, Tina4Response } from "tina4-nodejs";
import BlogPost from "../../../../../models/BlogPost.js";
import Comment from "../../../../../models/Comment.js";

export default async function (req: Tina4Request, res: Tina4Response) {
  const post = await BlogPost.findById(req.params.id);

  if (post === null) {
    return res.json({ error: "Post not found" }, 404);
  }

  const body = req.body as Record<string, unknown>;

  const comment = new Comment();
  comment.postId = req.params.id;
  comment.authorName = body.authorName;
  comment.authorEmail = body.authorEmail;
  comment.body = body.body;
```

```
  const errors = comment.validate();
  if (errors.length > 0) {
    return res.json({ errors }, 400);
  }

  await comment.save();
  res.json({ comment: comment.toDict() }, 201);
}
```

---

14. Gotchas

1. Forgetting to call save()

Problem: You set properties on a model but the database does not change.

Cause: Setting `note.title = "New Title"` only changes the TypeScript object. The database remains unchanged until you call `note.save()`.

Fix: Call `save()` after modifying properties. Check the return value -- `save()` returns `this` on success and `false` on failure.

2. findById() returns null

Problem: You call `Note.findById(id)` but get `null` instead of a note object.

Cause: `findById()` returns `null` when no row matches the given primary key. If soft delete is enabled, `findById()` also excludes soft-deleted records.

Fix: Check for `null` after `findById()`: `if (note === null) return res.json({error: "Not found"}, 404)`. Use `findOrFail()` if you want an Error thrown instead.

3. find() vs findById()

Problem: You call `Note.find(42)` expecting a single record, but get unexpected results.

Cause: `find()` takes a filter object (`find({ id: 42 })`), not a bare primary key value. For single-record lookups by primary key, use `findById(42)`.

Fix: Use `findById(id)` for primary key lookups. Use `find({ column: value })` for filter-based queries.

4. all() not findAll()

Problem: You call `Note.findAll()` and get an error.

Cause: The method is `all()`, not `findAll()`. There is no `findAll()` method on BaseModel.

Fix: Use `Note.all()` to fetch all records.

5. toDict() includes everything

Problem: `user.toDict()` includes `passwordHash` in the API response.

Cause: `toDict()` includes all fields by default.

Fix: Build the response object manually, omitting sensitive fields: `{ id: user.id, name: user.name, email: user.email }`. Or create a helper method on your model class that returns only safe fields.

6. save() returns false on invalid data

Problem: `await model.save()` returns `false` and the row never reaches the database, but nothing threw.

Cause: `save()` runs `validate()` first. If any field fails its rules, it refuses the write, logs the reason, and returns `false` instead of throwing. The same happens when the database rejects the write (a missing table, a `NOT NULL` column).

Fix: Check the return value. `model.lastError` (or `model.getError()`) holds the reason. In a route handler, call `const errors = model.validate()` before `save()` when you want to send the field errors back to the client as a `400`.

7. Foreign key not enforced

Problem: You save a post with `authorId = 999` and it succeeds, even though no author with ID 999 exists.

Cause: SQLite does not enforce foreign key constraints by default. The ORM defines the relationship through `hasMany`/`belongsTo` methods, but the database itself may not enforce it.

Fix: Enable SQLite foreign keys with `PRAGMA foreign_keys = ON;` in a migration, or validate the foreign key in your route handler before saving.

8. N+1 query problem

Problem: Listing 100 authors with their posts runs 101 queries (1 for authors + 100 for posts), and the page loads slowly.

Cause: You call `author.hasMany(BlogPost, "author_id")` inside a loop for each author.

Fix: Use eager loading with the `include` parameter on `all()`, `where()`, or `findById()`. Define declarative relationships on the model and register models with `BaseModel.registerModel()`.

9. Auto-CRUD endpoint conflicts

Problem: Custom route at `/api/notes/{id}` stops working after enabling auto-CRUD for the Note model.

Cause: Both routes match the same path. The first registered route wins.

Fix: Custom routes in `src/routes/` load before auto-CRUD routes. They take precedence. If you want different behaviour, use a different path for the custom route.

10. Soft-deleted records appearing in queries

Problem: You soft-deleted a record, but queries still return it.

Cause: Soft delete requires the `static softDelete = true` flag on the model class and an `is_deleted` column in the database (integer, 0 or 1). Without both, soft delete is inactive.

Fix: Verify both the `static softDelete = true` flag and the `is_deleted` column exist. The column stores `0` for active records and `1` for deleted ones.

---

15. QueryBuilder Integration

ORM models provide a `query()` static method that returns a `QueryBuilder` pre-configured with the model's table name and database connection. This gives you a fluent API for building complex queries without writing raw SQL:

```
// Fluent query builder from ORM
const results = await User.query()
  .select("id", "name", "email")
  .where("active = ?", [1])
  .orderBy("name")
  .limit(50)
  .get();

// First matching record
const user = await User.query()
  .where("email = ?", ["alice@example.com"])
  .first();

// Count
const total = await User.query()
  .where("role = ?", ["admin"])
  .count();

// Check existence
const exists = await User.query()
  .where("email = ?", ["test@example.com"])
  .exists();
```

The `limit()` method accepts count and an optional offset: `limit(10, 20)`. There is no separate `offset()` method.

Note that `get()` returns plain objects, not model instances. Use `findById()`, `find()`, `all()`, or `where()` when you need model instances with `save()`, `delete()`, and relationship methods.

See the QueryBuilder chapter for the full fluent API including joins, grouping, having, and MongoDB support.

Multiple Database Connections

A model can target a named database connection using `static _db`:

```
export default class AuditLog extends BaseModel {
  static tableName = "audit_logs";
  static _db = "secondary";  // Uses the "secondary" named adapter
  static fields = {
    id:    { type: "integer" as const, primaryKey: true, autoIncrement: true },
    event: { type: "string" as const, required: true },
  };
}
```

Register the named adapter at startup with `initDatabase()`.

QueryBuilder

1. Why a Query Builder?

In Chapter 5 you wrote raw SQL. In Chapter 6 you used ORM models with `select()`. Both work. Both have limits.

Raw SQL gives you full control but it is brittle. Concatenate a WHERE clause wrong and you get a syntax error -- or worse, a SQL injection. The ORM's `select()` method handles simple cases, but building dynamic queries with optional filters, joins, and aggregations turns into a mess of string concatenation and empty-parameter guards.

The QueryBuilder sits between raw SQL and the ORM. It gives you a fluent, chainable API that builds correct SQL. You write TypeScript. It writes SQL. Every method returns `this`, so you chain calls in any order. When you are done, call `get()` to execute or `toSql()` to inspect.

---

2. Creating a QueryBuilder

There are two entry points.

Standalone: QueryBuilder.fromTable()

Import `QueryBuilder` from the ORM package and call the static `from()` factory:

```
import { QueryBuilder } from "tina4-nodejs/orm";

const users = await QueryBuilder.fromTable("users", db)
    .select("id", "name", "email")
    .where("active = ?", [1])
    .orderBy("name ASC")
    .limit(10)
    .get();
```

The first argument is the table name. The second is the database adapter -- the same `db` object you get from `Database.create()` or the globally initialised adapter. If you omit the adapter, the QueryBuilder falls back to the global adapter registered by `initDatabase()`.

From an ORM Model: Model.query()

Every model that extends `BaseModel` has a static `query()` method. It returns a QueryBuilder pre-configured with the model's table name and database adapter:

```
import { User } from "../orm/User";

const activeUsers = await User.query()
    .where("active = ?", [1])
    .orderBy("name ASC")
    .get();
```

No need to pass the table name or database. The model knows both.

---

3. Selecting Columns

By default, the QueryBuilder selects all columns (`*`). Use `select()` to pick specific columns:

```
await QueryBuilder.fromTable("products", db)
    .select("id", "name", "price")
    .get();
```

This generates:

```
SELECT id, name, price FROM products
```

Pass as many column names as you need. Each is a separate string argument, not a comma-separated string. If you call `select()` with no arguments, it keeps the default `*`.

You can also use expressions:

```
await QueryBuilder.fromTable("orders", db)
    .select("customer_id", "SUM(total) as revenue")
    .groupBy("customer_id")
    .get();
```

---

4. WHERE Conditions

where() -- AND

Add a condition with `where()`. Use `?` as the placeholder for parameters:

```
await QueryBuilder.fromTable("users", db)
    .where("age > ?", [18])
    .where("active = ?", [1])
    .get();
```

This generates:

```
SELECT * FROM users WHERE age > ? AND active = ?
```

With parameters `[18, 1]`.

Multiple `where()` calls are joined with AND. The first call starts the WHERE clause. Every subsequent call adds `AND condition`.

orWhere() -- OR

Use `orWhere()` to add an OR condition:

```
await QueryBuilder.fromTable("products", db)
    .where("category = ?", ["Electronics"])
    .orWhere("category = ?", ["Books"])
    .get();
```

This generates:

```
SELECT * FROM products WHERE category = ? OR category = ?
```

Combining AND and OR

```
await QueryBuilder.fromTable("products", db)
    .where("price < ?", [100])
    .where("in_stock = ?", [1])
    .orWhere("featured = ?", [1])
    .get();
```

Generates:

```
SELECT * FROM products WHERE price < ? AND in_stock = ? OR featured = ?
```

If you need grouping with parentheses, write the grouped expression as a single condition:

```
await QueryBuilder.fromTable("products", db)
    .where("price < ?", [100])
    .where("(category = ? OR category = ?)", ["Electronics", "Books"])
    .get();
```

Generates:

```
SELECT * FROM products WHERE price < ? AND (category = ? OR category = ?)
```

---

5. Joins

Inner Join

```
await QueryBuilder.fromTable("orders", db)
    .select("orders.id", "users.name", "orders.total")
    .join("users", "users.id = orders.user_id")
    .get();
```

Generates:

```
SELECT orders.id, users.name, orders.total FROM orders INNER JOIN users ON users.id = orders.use
```

Left Join

```
await QueryBuilder.fromTable("users", db)
    .select("users.name", "orders.total")
    .leftJoin("orders", "orders.user_id = users.id")
    .get();
```

Generates:

```
SELECT users.name, orders.total FROM users LEFT JOIN orders ON orders.user_id = users.id
```

Multiple Joins

Chain as many joins as you need:

```
await QueryBuilder.fromTable("orders", db)
    .select("orders.id", "users.name", "products.name as product_name")
    .join("users", "users.id = orders.user_id")
    .join("order_items", "order_items.order_id = orders.id")
    .join("products", "products.id = order_items.product_id")
    .where("orders.status = ?", ["completed"])
    .get();
```

---

6. Grouping and Aggregation

groupBy()

```
await QueryBuilder.fromTable("orders", db)
    .select("status", "COUNT(*) as total")
    .groupBy("status")
    .get();
```

Generates:

```
SELECT status, COUNT(*) as total FROM orders GROUP BY status
```

Call `groupBy()` multiple times to group by multiple columns:

```
await QueryBuilder.fromTable("orders", db)
    .select("status", "customer_id", "SUM(total) as revenue")
    .groupBy("status")
    .groupBy("customer_id")
    .get();
```

having()

Filter grouped results with `having()`:

```
await QueryBuilder.fromTable("products", db)
    .select("category", "AVG(price) as avg_price")
    .groupBy("category")
    .having("AVG(price) > ?", [50])
    .get();
```

Generates:

```
SELECT category, AVG(price) as avg_price FROM products GROUP BY category HAVING AVG(price) > ?
```

Multiple `having()` calls are joined with AND.

---

7. Ordering

```
await QueryBuilder.fromTable("products", db)
    .orderBy("price DESC")
    .get();
```

Generates:

```
SELECT * FROM products ORDER BY price DESC
```

Call `orderBy()` multiple times for multi-column sorting:

```
await QueryBuilder.fromTable("products", db)
    .orderBy("category ASC")
    .orderBy("price DESC")
    .get();
```

Generates:

```
SELECT * FROM products ORDER BY category ASC, price DESC
```

---

8. Limit and Offset

Limit Only

```
await QueryBuilder.fromTable("products", db)
    .limit(10)
    .get();
```

Limit with Offset

```
await QueryBuilder.fromTable("products", db)
    .limit(10, 20)
    .get();
```

The first argument is the maximum number of rows. The second is the number of rows to skip. This is how you implement pagination:

```
const page = 3;
const perPage = 25;
const offset = (page - 1) * perPage;

const products = await QueryBuilder.fromTable("products", db)
    .orderBy("name ASC")
    .limit(perPage, offset)
    .get();
```

---

9. Executing Queries

get\<T\>() -- All Rows

`get()` executes the query and returns an array of row objects:

```
const users = await QueryBuilder.fromTable("users", db)
    .where("active = ?", [1])
    .get();
// users: Record<string, unknown>[]
```

Use the TypeScript generic to type the result:

```
interface User {
    id: number;
    name: string;
    email: string;
    active: boolean;
}

const users = await QueryBuilder.fromTable("users", db)
    .where("active = ?", [1])
    .get<User>();
// users: User[]
```

The generic is optional. Without it, rows are typed as `Record<string, unknown>[]`.

first\<T\>() -- Single Row

`first()` returns one row or `null`:

```
const user = await QueryBuilder.fromTable("users", db)
    .where("email = ?", ["alice@example.com"])
    .first<User>();

if (!user) {
    // Not found
}
```

count() -- Row Count

`count()` returns the number of matching rows without fetching the data:

```
const total = await QueryBuilder.fromTable("users", db)
    .where("active = ?", [1])
    .count();
// total: number
```

It rewrites the query internally to `SELECT COUNT(*) as cnt` and extracts the value.

exists() -- Boolean Check

`exists()` returns `true` if at least one row matches:

```
const hasAdmin = await QueryBuilder.fromTable("users", db)
    .where("role = ?", ["admin"])
    .exists();
// hasAdmin: boolean
```

---

10. Inspecting SQL with toSql()

Call `toSql()` to get the generated SQL string without executing the query. Useful for debugging:

```
const sql = QueryBuilder.fromTable("users", db)
    .select("id", "name")
    .where("active = ?", [1])
    .orderBy("name ASC")
    .limit(10)
    .toSql();

console.log(sql);
// SELECT id, name FROM users WHERE active = ? ORDER BY name ASC
```

Note that `toSql()` returns the SQL with `?` placeholders. The actual parameter values are bound at execution time by the database adapter.

The `LIMIT` and `OFFSET` clauses are not included in `toSql()` output -- they are passed directly to the adapter's `fetch()` method as separate arguments.

---

11. Using QueryBuilder in Route Handlers

The QueryBuilder pairs naturally with route handlers and `res.json()`:

List Endpoint with Filters

```
// src/routes/api/products/get.ts
import { QueryBuilder } from "tina4-nodejs/orm";
import type { Tina4Request, Tina4Response } from "tina4-nodejs";

export default async function (req: Tina4Request, res: Tina4Response) {
    const category = req.query.category;
    const minPrice = req.query.min_price;
    const sort = req.query.sort ?? "name";
    const page = parseInt(req.query.page ?? "1", 10);
    const perPage = parseInt(req.query.per_page ?? "20", 10);

    const qb = await QueryBuilder.fromTable("products");

    if (category) {
        qb.where("category = ?", [category]);
    }

    if (minPrice) {
        qb.where("price >= ?", [parseFloat(minPrice)]);
    }

    const allowedSorts = ["name", "price", "category", "created_at"];
    const safeSort = allowedSorts.includes(sort) ? sort : "name";

    const total = await qb.count();
    const products = await qb
        .orderBy(`${safeSort} ASC`)
        .limit(perPage, (page - 1) * perPage)
        .get();

    return res.json({ products, total, page, per_page: perPage });
}
```

Single Record Lookup

```
// src/routes/api/users/[id]/get.ts
import { QueryBuilder } from "tina4-nodejs/orm";
import type { Tina4Request, Tina4Response } from "tina4-nodejs";

export default async function (req: Tina4Request, res: Tina4Response) {
    const user = await QueryBuilder.fromTable("users")
        .where("id = ?", [req.params.id])
        .first();

    if (!user) {
        return res.status(404).json({ error: "User not found" });
    }

    return res.json(user);
}
```

Dashboard with Aggregation

```
// src/routes/api/dashboard/get.ts
import { QueryBuilder } from "tina4-nodejs/orm";
import type { Tina4Request, Tina4Response } from "tina4-nodejs";

export default async function (req: Tina4Request, res: Tina4Response) {
    const totalUsers = await QueryBuilder.fromTable("users").count();

    const revenueByCategory = await QueryBuilder.fromTable("orders")
        .select("category", "SUM(total) as revenue", "COUNT(*) as order_count")
        .join("order_items", "order_items.order_id = orders.id")
        .join("products", "products.id = order_items.product_id")
        .where("orders.status = ?", ["completed"])
        .groupBy("category")
        .having("SUM(total) > ?", [0])
        .orderBy("revenue DESC")
        .get();

    const recentOrders = await QueryBuilder.fromTable("orders")
        .select("orders.id", "users.name as customer", "orders.total", "orders.created_at")
        .join("users", "users.id = orders.user_id")
        .orderBy("orders.created_at DESC")
        .limit(5)
        .get();

    return res.json({
        total_users: totalUsers,
        revenue_by_category: revenueByCategory,
        recent_orders: recentOrders,
    });
}
```

---

12. QueryBuilder vs ORM select()

Both query the database. When do you use which?

Scenario	Use
Simple CRUD on a single model	ORM `select()`
Need `toDict()` / `toObject()` on results	ORM `select()`
Eager loading relationships	ORM `select()` with `include`
Multi-table joins	QueryBuilder
Aggregations (SUM, COUNT, AVG)	QueryBuilder
Complex dynamic filters	QueryBuilder
Sub-selects in columns	QueryBuilder
You want typed results without a model	QueryBuilder with `get<T>()`

They are not mutually exclusive. Use ORM models for data that maps cleanly to a single table. Use the QueryBuilder when you need to reach across tables or aggregate.

---

13. Exercise: Sales Report API

Build a sales report endpoint using the QueryBuilder.

Setup

Assume these tables exist:

- `customers` -- id, name, email, region, created_at
- `orders` -- id, customer*id, status, total, created*at
- `order_items` -- id, order*id, product*id, quantity, unit_price
- `products` -- id, name, category, price

Requirements

Create three route files:

Method	Path	Description
`GET`	`/api/reports/revenue`	Revenue by product category with optional date range
`GET`	`/api/reports/top-customers`	Top 10 customers by total spend
`GET`	`/api/reports/summary`	Order count, total revenue, average order value

Query parameters for `/api/reports/revenue`:

- `after` -- only orders after this date (e.g. `2025-01-01`)
- `before` -- only orders before this date
- `min_revenue` -- only categories with revenue above this amount

---

14. Solution

Revenue by Category

Create `src/routes/api/reports/revenue/get.ts`:

```
import { QueryBuilder } from "tina4-nodejs/orm";
import type { Tina4Request, Tina4Response } from "tina4-nodejs";

export default async function (req: Tina4Request, res: Tina4Response) {
    const qb = await QueryBuilder.fromTable("order_items")
```

```
        .select(
            "products.category",
            "SUM(order_items.quantity * order_items.unit_price) as revenue",
            "SUM(order_items.quantity) as units_sold",
            "COUNT(DISTINCT orders.id) as order_count"
        )
        .join("orders", "orders.id = order_items.order_id")
        .join("products", "products.id = order_items.product_id")
        .where("orders.status = ?", ["completed"]);

    if (req.query.after) {
        qb.where("orders.created_at >= ?", [req.query.after]);
    }

    if (req.query.before) {
        qb.where("orders.created_at < ?", [req.query.before]);
    }

    qb.groupBy("products.category");

    if (req.query.min_revenue) {
        qb.having("SUM(order_items.quantity * order_items.unit_price) > ?", [
            parseFloat(req.query.min_revenue),
        ]);
    }

    const results = await qb.orderBy("revenue DESC").get();

    return res.json({ categories: results });
}
```

Top Customers

Create `src/routes/api/reports/top-customers/get.ts`:

```
import { QueryBuilder } from "tina4-nodejs/orm";
import type { Tina4Request, Tina4Response } from "tina4-nodejs";

export default async function (req: Tina4Request, res: Tina4Response) {
    const customers = await QueryBuilder.fromTable("orders")
        .select(
            "customers.id",
            "customers.name",
            "customers.email",
            "customers.region",
            "SUM(orders.total) as total_spend",
            "COUNT(orders.id) as order_count"
        )
        .join("customers", "customers.id = orders.customer_id")
        .where("orders.status = ?", ["completed"])
        .groupBy("customers.id")
        .groupBy("customers.name")
        .groupBy("customers.email")
        .groupBy("customers.region")
        .orderBy("total_spend DESC")
        .limit(10)
        .get();

    return res.json({ top_customers: customers });
```

```
}
```

Summary

Create `src/routes/api/reports/summary/get.ts`:

```
import { QueryBuilder } from "tina4-nodejs/orm";
import type { Tina4Request, Tina4Response } from "tina4-nodejs";

export default async function (req: Tina4Request, res: Tina4Response) {
    const stats = await QueryBuilder.fromTable("orders")
        .select(
            "COUNT(*) as order_count",
            "SUM(total) as total_revenue",
            "AVG(total) as avg_order_value"
        )
        .where("status = ?", ["completed"])
        .first();

    const ordersByStatus = await QueryBuilder.fromTable("orders")
        .select("status", "COUNT(*) as count")
        .groupBy("status")
        .orderBy("count DESC")
        .get();

    const hasOrders = await QueryBuilder.fromTable("orders").exists();

    return res.json({
        summary: stats,
        by_status: ordersByStatus,
        has_orders: hasOrders,
    });
}
```

Testing It

```
# Revenue by category
curl "http://localhost:7148/api/reports/revenue"

# Revenue with date range and minimum
curl "http://localhost:7148/api/reports/revenue?after=2025-01-01&before=2026-01-01&min_revenue=1

# Top customers
curl "http://localhost:7148/api/reports/top-customers"

# Summary
curl "http://localhost:7148/api/reports/summary"
```

---

15. NoSQL: MongoDB Queries

The QueryBuilder can generate MongoDB-compatible query documents with `toMongo()`. This returns an object containing the filter, projection, sort, limit, and skip -- ready to pass to the MongoDB Node.js driver.

Operator Mapping

SQL Operator	MongoDB Operator
`=`	Exact match
`!=`	`$ne`
`>`	`$gt`
`<`	`$lt`
`>=`	`$gte`
`<=`	`$lte`
`LIKE`	`$regex`
`IN`	`$in`
`IS NULL`	`$exists: false`
`IS NOT NULL`	`$exists: true`

Example

```
const query = await QueryBuilder.fromTable("users")
    .select("name", "email")
    .where("age > ?", [25])
    .where("status = ?", ["active"])
    .orderBy("name ASC")
    .limit(10)
    .offset(5);

const mongo = query.toMongo();
```

The returned object:

```
{
    filter: { age: { $gt: 25 }, status: "active" },
    projection: { name: 1, email: 1 },
    sort: { name: 1 },
    limit: 10,
    skip: 5,
}
```

Pass it directly to the MongoDB driver:

```
const collection = db.collection("users");
const cursor = collection.find(mongo.filter, {
    projection: mongo.projection,
    sort: mongo.sort,
    limit: mongo.limit,
    skip: mongo.skip,
});
```

---

16. Gotchas

1. Forgetting the Database Adapter

Problem: You call `QueryBuilder.fromTable("users").get()` and get `"QueryBuilder: No database adapter provided."`.

Cause: No database adapter was passed to `from()` and no global adapter has been initialised.

Fix: Either pass the adapter explicitly -- `QueryBuilder.fromTable("users", db)` -- or ensure `initDatabase()` has been called before any queries run. When Tina4 boots your server, the global adapter is initialised automatically from `TINA4_DATABASE_URL`.

2. Parameter Order Matters

Problem: Your WHERE clause has three `?` placeholders but the results are wrong.

Cause: Parameters are collected in the order you call `where()` and `orWhere()`. If you add conditions in one order but pass parameters assuming a different order, values bind to the wrong placeholders.

Fix: Each `where()` call takes its own parameter array. The QueryBuilder concatenates them in call order. Keep the parameters next to their condition:

```
// Correct
qb.where("age > ?", [18]).where("country = ?", ["ZA"]);

// Wrong -- do not batch parameters separately from conditions
```

3. toSql() Does Not Include LIMIT

Problem: You call `toSql()` and the output has no LIMIT clause, but `get()` returns limited rows.

Cause: LIMIT and OFFSET are passed to the database adapter's `fetch()` method as separate arguments, not appended to the SQL string.

Fix: This is by design. The adapter handles LIMIT/OFFSET according to the database engine's dialect. Use `toSql()` for debugging the WHERE, JOIN, and ORDER BY clauses.

4. Reusing a QueryBuilder

Problem: You call `count()` and then `get()` on the same QueryBuilder, but the results are inconsistent.

Cause: `count()` temporarily replaces the selected columns with `COUNT(*)` and restores them. This is safe for a single call, but interleaving multiple execution methods on the same builder can produce unexpected SQL if you modify the builder between calls.

Fix: For separate queries, create separate builders. Use `QueryBuilder.fromTable()` or `Model.query()` fresh for each independent query.

5. Mixing QueryBuilder with ORM select()

Problem: You expect `get()` to return model instances with `toDict()`.

Cause: The QueryBuilder returns plain objects (`Record<string, unknown>` or your generic type `T`). It does not return model instances.

Fix: Use `get<T>()` with an interface to type the results. If you need model methods like `toDict()`, `save()`, or relationship loading, use the ORM's `select()` method instead.

6. OR Without AND

Problem: You start with `orWhere()` instead of `where()` and the query behaves unexpectedly.

Cause: The first condition in the WHERE clause ignores the connector (AND/OR). Starting with `orWhere()` works syntactically but reads misleadingly.

Fix: Always start with `where()` for the first condition. Use `orWhere()` for subsequent alternatives.

Authentication

1. Locking the Door

Every endpoint you have built so far is public. Anyone with the URL can read, create, update, and delete data. Fine for a tutorial. Unacceptable in production.

A real application needs identity. Who is making this request? And are they allowed to make it?

This chapter covers Tina4's authentication system: JWT tokens, password hashing, middleware-based route protection, CSRF tokens for forms, and session management.

---

2. JWT Tokens

Tina4 uses JSON Web Tokens (JWT) for authentication. A JWT is a signed string carrying a payload -- a user ID, a role, whatever you need. The server creates the token at login. The client sends it with every request. The server verifies it without touching the database.

Generating a Token

```
import { Auth } from "tina4-nodejs";

const payload = {
    user_id: 42,
    email: "alice@example.com",
    role: "admin"
};

const token = Auth.getToken(payload, secret);
```

`getToken()` signs the payload with HS256 (HMAC-SHA256) and returns a JWT string like:

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjo0MiwiZW1haWwiOiJhbGljZUBleGFtcGxlLmNvbSIsInJ
```

The token has three parts separated by dots: header, payload, and signature. The signature ensures the token has not been tampered with.

The `secret` parameter is required -- pass your secret key directly or read it from the `TINA4_SECRET` env var.

> ***Legacy aliases:*** `Auth.getToken()` *and the standalone* `getToken()` *function still work. Use the primary name in new code.*

Token Expiry

Pass the expiry time as the third argument to `getToken()` in **minutes**:

```
const token = Auth.getToken(payload, secret, 60); // 1 hour (default)
```

Value	Duration
`15`	15 minutes

`60`	1 hour (default)
`1440`	24 hours
`10080`	7 days

You can also configure the default expiry in `.env`:

```
TINA4_TOKEN_LIMIT=1440
```

The value is in minutes. `1440` is 24 hours.

Validating a Token

```
const payload = Auth.validToken(token, secret);
// Returns the payload object if valid, null if invalid or expired
```

`validToken()` returns the decoded payload on success, not a boolean. This lets you validate and read the token in one step. Returns `null` if the token is invalid or expired.

> ***Legacy alias:*** `Auth.validateToken()` *and the standalone* `validToken()` *function work the same way.*

Reading the Payload

```
const payload = Auth.getPayload(token);
```

Returns the decoded payload **without validation** -- it just decodes the token:

```
{
    user_id: 42,
    email: "alice@example.com",
    role: "admin",
    iat: 1711112600,  // issued at (Unix timestamp)
    exp: 1711116200   // expires at (Unix timestamp)
}
```

If the token cannot be decoded, `getPayload()` returns `null`.

> ***Important:*** `getPayload()` *does not verify the signature or check expiry. Use* `validToken()` *when you need to confirm the token is trustworthy.*

The Secret Key and Algorithm

Tina4 Node.js uses **HS256** (HMAC-SHA256) for JWT signing. It uses only the standard library -- zero external dependencies.

Set the secret key in `.env`:

```
TINA4_SECRET=my-super-secret-key-at-least-32-chars
```

The `secret` parameter on `getToken()` and `validToken()` is optional -- if omitted, Tina4 reads from the `TINA4_SECRET` env var. If neither is set, Tina4 falls back to generating a random key at `secrets/jwt.key` on first run. Setting `TINA4_SECRET` explicitly is recommended for production so all server instances share the same key.

Keep this key secret. If someone gets it, they can forge tokens.

---

3. Password Hashing

Plain text passwords are a breach waiting to happen. Tina4 provides two functions for secure password handling.

Hashing a Password

```
import { Auth } from "tina4-nodejs";

const hash = await Auth.hashPassword("my-secure-password");
```

Uses PBKDF2 from the standard library -- no external dependencies. Each hash includes a random salt, so hashing the same password twice produces different results.

Checking a Password

```
const isCorrect = await Auth.checkPassword("my-secure-password", storedHash);
// Returns true if the password matches the hash
```

Registration Example

```
import { Router, Auth } from "tina4-nodejs";
import { Database } from "tina4-nodejs/orm";

/**
 * @noauth
 */
Router.post("/api/register", async (req, res) => {
    const body = req.body;

    // Validate input
    if (!body.name || !body.email || !body.password) {
        return res.status(400).json({ error: "Name, email, and password are required" });
    }

    if (body.password.length < 8) {
        return res.status(400).json({ error: "Password must be at least 8 characters" });
    }

    const db = await Database.getConnection();

    // Check if email already exists
    const existing = await db.fetchOne("SELECT id FROM users WHERE email = :email", { email: bod
    if (existing !== null) {
        return res.status(409).json({ error: "Email already registered" });
    }

    // Hash the password
    const passwordHash = await Auth.hashPassword(body.password);

    // Create the user
    await db.execute(
        "INSERT INTO users (name, email, password_hash) VALUES (:name, :email, :hash)",
        { name: body.name, email: body.email, hash: passwordHash }
    );

    const user = await db.fetchOne("SELECT id, name, email FROM users WHERE id = last_insert_row

    return res.status(201).json({ message: "Registration successful", user });
});
```

```
curl -X POST http://localhost:7148/api/register \
  -H "Content-Type: application/json" \
  -d '{"name": "Alice", "email": "alice@example.com", "password": "securePass123"}'
```

```
{
  "message": "Registration successful",
  "user": {"id": 1, "name": "Alice", "email": "alice@example.com"}
}
```

---

4. The Login Flow

The complete login flow. Client sends credentials. Server validates them. Server returns a JWT token.

```
import { Router, Auth } from "tina4-nodejs";
```

```
import { Database } from "tina4-nodejs/orm";

/**
 * @noauth
 */
Router.post("/api/login", async (req, res) => {
    const body = req.body;

    if (!body.email || !body.password) {
        return res.status(400).json({ error: "Email and password are required" });
    }

    const db = await Database.getConnection();

    // Find the user
    const user = await db.fetchOne(
        "SELECT id, name, email, password_hash FROM users WHERE email = :email",
        { email: body.email }
    );

    if (user === null) {
        return res.status(401).json({ error: "Invalid email or password" });
    }

    // Check the password
    if (!(await Auth.checkPassword(body.password, user.password_hash))) {
        return res.status(401).json({ error: "Invalid email or password" });
    }

    // Generate a token
    const secret = process.env.TINA4_SECRET || "tina4-default-secret";
    const token = Auth.getToken({
        user_id: user.id,
        email: user.email,
        name: user.name
    }, secret);

    return res.json({
        message: "Login successful",
        token,
        user: { id: user.id, name: user.name, email: user.email }
    });
});
```

Notice `@noauth` in the JSDoc comment. The login endpoint must be public. You cannot require a token to get a token.

```
curl -X POST http://localhost:7148/api/login \
  -H "Content-Type: application/json" \
  -d '{"email": "alice@example.com", "password": "securePass123"}'
```

```
{
  "message": "Login successful",
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "user": {
    "id": 1,
    "name": "Alice",
    "email": "alice@example.com"
  }
```

```
}
```

The client stores this token (in localStorage, a cookie, or memory) and sends it with subsequent requests.

---

5. Using Tokens in Requests

The client sends the token in the `Authorization` header with the `Bearer` prefix:

```
curl http://localhost:7148/api/profile \
  -H "Authorization: Bearer eyJhbGciOiJIUzI1NiIs..."
```

Or in frontend JavaScript:

```
// Frontend JavaScript
const response = await fetch("/api/profile", {
    headers: {
        "Authorization": `Bearer ${token}`,
        "Content-Type": "application/json",
    },
});
```

The token travels in the `Authorization` header with the `Bearer` prefix. The middleware extracts it, verifies it, and populates `req.user` with the decoded payload.

---

6. Protecting Routes

Auth Middleware

Create a reusable auth middleware:

```
import { Auth } from "tina4-nodejs";

function authMiddleware(req, res, next) {
    const authHeader = req.headers["authorization"] ?? "";

    if (!authHeader || !authHeader.startsWith("Bearer ")) {
        res({ error: "Authorization header required" }, 401);
        return;
    }

    const token = authHeader.substring(7);  // Remove "Bearer " prefix
    const secret = process.env.TINA4_SECRET || "tina4-default-secret";

    const payload = Auth.validToken(token, secret);
    if (payload === null) {
        res({ error: "Invalid or expired token" }, 401);
        return;
    }

    (req as any).user = payload;  // Attach user data to the request

    next();
}
```

Applying Middleware to Routes

```
import { Router } from "tina4-nodejs";

Router.get("/api/profile", async (req, res) => {
    return res.json({
        user_id: (req as any).user.user_id,
        email: (req as any).user.email,
        name: (req as any).user.name
    });
}, [authMiddleware]);
```

```
# Without token -- 401
curl http://localhost:7148/api/profile
```

```
{"error":"Authorization header required"}
```

```
# With valid token -- 200
curl http://localhost:7148/api/profile \
  -H "Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
```

```
{
  "user_id": 1,
  "email": "alice@example.com",
  "name": "Alice"
}
```

Applying Middleware to a Group

```
import { Router } from "tina4-nodejs";

Router.group("/api/admin", [authMiddleware], (router) => {

    router.get("/stats", async (req, res) => {
        return res.json({ active_users: 42 });
    });

    router.get("/logs", async (req, res) => {
        return res.json({ logs: [] });
    });

});
```

Every route in the group requires a valid token.

---

7. @noauth and @secured Decorators

These JSDoc annotations control authentication at the route level.

@noauth -- Skip Authentication

Use `@noauth` for public endpoints that should bypass any global or group-level auth:

```
import { Router } from "tina4-nodejs";

/**
 * @noauth
 */
Router.get("/api/public/health", async (req, res) => {
```

```
    return res.json({ status: "ok" });
});

/**
 * @noauth
 */
Router.post("/api/login", async (req, res) => {
    // Login logic
});

/**
 * @noauth
 */
Router.post("/api/register", async (req, res) => {
    // Registration logic
});
```

@secured -- Require Authentication for GET Routes

By default, POST, PUT, PATCH, and DELETE routes are considered secured. GET routes are public. Use `@secured` to explicitly protect a GET route:

```
import { Router } from "tina4-nodejs";

/**
 * @secured
 */
Router.get("/api/me", async (req, res) => {
    // This GET route requires authentication
    return res.json((req as any).user);
});
```

Role-Based Authorization

Combine auth middleware with role checks:

```
import { Auth } from "tina4-nodejs";

function requireRole(role: string) {
    return function (req, res, next) {
        const authHeader = req.headers["authorization"] ?? "";
        if (!authHeader || !authHeader.startsWith("Bearer ")) {
            res({ error: "Authorization required" }, 401);
            return;
        }

        const token = authHeader.substring(7);
        const secret = process.env.TINA4_SECRET || "tina4-default-secret";
        const payload = Auth.validToken(token, secret);
        if (payload === null) {
            res({ error: "Invalid or expired token" }, 401);
            return;
        }

        if ((payload.role ?? "") !== role) {
            res({ error: `Forbidden. Required role: ${role}` }, 403);
            return;
        }

        (req as any).user = payload;
```

```
        next();
    };
}
```

Use it as middleware:

```
import { Router } from "tina4-nodejs";

Router.delete("/api/users/:id", async (req, res) => {
    // Only admins can delete users
    return res.json({ deleted: true });
}, [requireRole("admin")]);
```

---

8. CSRF Protection

Traditional form-based applications (not SPAs) need CSRF protection. Cross-Site Request Forgery attacks trick a user's browser into submitting a form to your server. The browser sends cookies automatically -- the attacker rides on the user's session.

CSRF tokens stop this. Each form gets a unique token. When the form submits, the server checks the token. If it does not match, the request is rejected.

Generating a Token

In your template, include the CSRF token in every form:

```
<form method="POST" action="/profile/update">
    {{ form_token() }}

    <div class="form-group">
        <label for="name">Name</label>
        <input type="text" name="name" id="name" value="{{ user.name }}">
    </div>

    <button type="submit">Update Profile</button>
</form>
```

`{{ form_token() }}` renders a hidden input field:

```
<input type="hidden" name="_token" value="abc123randomtoken456">
```

Validating the Token

In your route handler, check the token:

```
import { Router, Auth } from "tina4-nodejs";

Router.post("/profile/update", async (req, res) => {
    // Validate CSRF token
    if (!Auth.validateFormToken(req.body._token ?? "")) {
        return res.status(403).json({ error: "Invalid form token. Please refresh and try again."
    }

    // Process the form...
    return res.redirect("/profile");
});
```

The CSRF token is tied to the session and expires after a single use. A malicious site cannot forge a form submission because it cannot guess the token.

CSRF Middleware

For automatic validation on all POST/PUT/DELETE requests, use the CSRF middleware:

```
import { csrfMiddleware } from "tina4-nodejs";

// Apply globally to all form-based routes
Router.post("/profile/update", async (req, res) => {
    // CSRF validation happens automatically
    return res.redirect("/profile");
}, [csrfMiddleware]);
```

When to Use CSRF Tokens

Use CSRF tokens for:

- HTML forms submitted by browsers
- Any POST/PUT/DELETE from server-rendered pages

You do not need CSRF tokens for:

- API endpoints that use JWT (the Bearer token already proves the request is intentional)
- Single-page applications that use `fetch()` with custom headers

---

9. Sessions

Tina4 supports server-side sessions for storing per-user state between requests. JWTs handle API authentication. Sessions handle stateful web pages. Both work side by side.

Session Configuration

Set the session backend in `.env`:

```
# File-based sessions (default)
TINA4_SESSION_BACKEND=file

# Redis
TINA4_SESSION_BACKEND=redis
TINA4_SESSION_REDIS_HOST=localhost
TINA4_SESSION_REDIS_PORT=6379

# MongoDB
TINA4_SESSION_BACKEND=mongodb
TINA4_SESSION_REDIS_HOST=localhost
TINA4_SESSION_REDIS_PORT=27017

# Valkey
TINA4_SESSION_BACKEND=valkey
TINA4_SESSION_REDIS_HOST=localhost
TINA4_SESSION_REDIS_PORT=6379
```

File-based sessions work out of the box. No extra dependencies. For production deployments with multiple servers, use Redis or Valkey so sessions are shared across instances.

Using Sessions

Access session data via `req.session`:

```
import { Router } from "tina4-nodejs";

Router.post("/login-form", async (req, res) => {
    // After validating credentials...
    req.session.userId = 42;
    req.session.userName = "Alice";
    req.session.loggedIn = true;

    return res.redirect("/dashboard");
});

Router.get("/dashboard", async (req, res) => {
    if (!req.session.loggedIn) {
        return res.redirect("/login");
    }

    return res.render("dashboard.html", {
        userName: req.session.userName
    });
});

Router.post("/logout", async (req, res) => {
    // Clear all session data
    req.session = {};

    return res.redirect("/login");
});
```

Sessions complement JWT tokens. Use JWT for stateless API authentication. Use sessions for traditional web applications with server-rendered pages.

Session Options

```
TINA4_SESSION_TTL=3600       # Session lifetime in seconds (default: 3600)
TINA4_SESSION_NAME=tina4_session  # Cookie name for the session ID
```

---

10. Token Refresh

Tokens expire. When they do, `Auth.validToken()` returns `null` and the middleware rejects the request with a `401`. The user must log in again -- unless you implement token refresh.

A refresh endpoint issues a new token before the current one expires:

```
import { Router, Auth } from "tina4-nodejs";

Router.post("/api/auth/refresh", async (req, res) => {
    // req.user is populated by auth middleware
    const secret = process.env.TINA4_SECRET || "tina4-default-secret";
    const newToken = Auth.getToken({
```

```
        user_id: req.user.user_id,
        email: req.user.email,
        role: req.user.role,
    }, secret);

    return res.json({ token: newToken });
}, [authMiddleware]);
```

The frontend can call this endpoint periodically to keep the session alive without requiring re-login. A common pattern: a short-lived access token (15 minutes) paired with a long-lived refresh token (7 days). The refresh token can mint new access tokens without re-entering credentials.

---

11. Exercise: Build Login, Register, and Profile

Build a complete authentication system with registration, login, profile viewing, and password changing.

Requirements

- Create a `users` table migration with: `id`, `name`, `email` (unique), `password_hash`, `role` (default "user"), `created_at`
- Build these endpoints:

Method	Path	Auth	Description
`POST`	`/api/register`	@noauth	Create an account. Validate name, email, password (min 8 chars).
`POST`	`/api/login`	@noauth	Login. Return JWT token.
`GET`	`/api/profile`	secured	Get current user's profile from token.
`PUT`	`/api/profile`	secured	Update name and email.
`PUT`	`/api/profile/password`	secured	Change password. Require current password.

- Create auth middleware that extracts the user from the JWT and attaches it to `req.user`.

Test with:

```
# Register
curl -X POST http://localhost:7148/api/register \
  -H "Content-Type: application/json" \
  -d '{"name": "Alice", "email": "alice@example.com", "password": "securePass123"}'

# Login
curl -X POST http://localhost:7148/api/login \
  -H "Content-Type: application/json" \
  -d '{"email": "alice@example.com", "password": "securePass123"}'

# Save the token from login response, then:

# Get profile
curl http://localhost:7148/api/profile \
  -H "Authorization: Bearer YOUR_TOKEN_HERE"

# Update profile
curl -X PUT http://localhost:7148/api/profile \
  -H "Authorization: Bearer YOUR_TOKEN_HERE" \
  -H "Content-Type: application/json" \
  -d '{"name": "Alice Smith"}'

# Change password
curl -X PUT http://localhost:7148/api/profile/password \
  -H "Authorization: Bearer YOUR_TOKEN_HERE" \
  -H "Content-Type: application/json" \
  -d '{"current_password": "securePass123", "new_password": "evenMoreSecure456"}'

# Try with no token (should fail)
curl http://localhost:7148/api/profile
```

---

12. Solution

Migration

Create `migrations/20260322160000_create_users_table.sql`:

```
-- UP
CREATE TABLE users (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    email TEXT NOT NULL UNIQUE,
    password_hash TEXT NOT NULL,
    role TEXT NOT NULL DEFAULT 'user',
    created_at TEXT DEFAULT CURRENT_TIMESTAMP
);

-- DOWN
DROP TABLE IF EXISTS users;
```

```
tina4 migrate
```

Routes

Create `src/routes/auth.ts`: _____

```
import { Router, Auth } from "tina4-nodejs";
import { Database } from "tina4-nodejs/orm";


function authMiddleware(req, res, next) {
    const authHeader = req.headers["authorization"] ?? "";

    if (!authHeader || !authHeader.startsWith("Bearer ")) {
        res({ error: "Authorization required. Send: Authorization: Bearer <token>" }, 401);
        return;
    }

    const token = authHeader.substring(7);
    const secret = process.env.TINA4_SECRET || "tina4-default-secret";

    const payload = Auth.validToken(token, secret);
    if (payload === null) {
        res({ error: "Invalid or expired token. Please login again." }, 401);
        return;
    }

    req.user = payload;

    next();
}


/**
 * @noauth
 */
Router.post("/api/register", async (req, res) => {
    const body = req.body;
    const errors: string[] = [];

    if (!body.name) errors.push("Name is required");
    if (!body.email) errors.push("Email is required");
    if (!body.password) errors.push("Password is required");
    else if (body.password.length < 8) errors.push("Password must be at least 8 characters");

    if (errors.length > 0) {
        return res.status(400).json({ errors });
    }

    const db = await Database.getConnection();

    const existing = await db.fetchOne("SELECT id FROM users WHERE email = :email", { email: bod
    if (existing !== null) {
        return res.status(409).json({ error: "Email already registered" });
    }

    const hash = await Auth.hashPassword(body.password);

    await db.execute(
        "INSERT INTO users (name, email, password_hash) VALUES (:name, :email, :hash)",
        { name: body.name, email: body.email, hash }
    );

    const user = await db.fetchOne("SELECT id, name, email, role, created_at FROM users WHERE id
```

```
    return res.status(201).json({ message: "Registration successful", user });
});


/**
 * @noauth
 */
Router.post("/api/login", async (req, res) => {
    const body = req.body;

    if (!body.email || !body.password) {
        return res.status(400).json({ error: "Email and password are required" });
    }

    const db = await Database.getConnection();

    const user = await db.fetchOne(
        "SELECT id, name, email, password_hash, role FROM users WHERE email = :email",
        { email: body.email }
    );

    if (user === null || !(await Auth.checkPassword(body.password, user.password_hash))) {
        return res.status(401).json({ error: "Invalid email or password" });
    }

    const secret = process.env.TINA4_SECRET || "tina4-default-secret";
    const token = Auth.getToken({
        user_id: user.id,
        email: user.email,
        name: user.name,
        role: user.role
    }, secret);

    return res.json({
        message: "Login successful",
        token,
        user: { id: user.id, name: user.name, email: user.email, role: user.role }
    });
});


Router.get("/api/profile", async (req, res) => {
    const db = await Database.getConnection();

    const user = await db.fetchOne(
        "SELECT id, name, email, role, created_at FROM users WHERE id = :id",
        { id: req.user.user_id }
    );

    if (user === null) {
        return res.status(404).json({ error: "User not found" });
    }

    return res.json(user);
}, [authMiddleware]);


Router.put("/api/profile", async (req, res) => {
    const db = await Database.getConnection();
```

```
    const body = req.body;
    const userId = req.user.user_id;

    // Check if the new email is already taken by another account
    if (body.email) {
        const existing = await db.fetchOne(
            "SELECT id FROM users WHERE email = :email AND id != :id",
            { email: body.email, id: userId }
        );
        if (existing !== null) {
            return res.status(409).json({ error: "Email already in use by another account" });
        }
    }

    const current = await db.fetchOne("SELECT * FROM users WHERE id = :id", { id: userId });

    await db.execute(
        "UPDATE users SET name = :name, email = :email WHERE id = :id",
        { name: body.name ?? current.name, email: body.email ?? current.email, id: userId }
    );

    const updated = await db.fetchOne(
        "SELECT id, name, email, role, created_at FROM users WHERE id = :id",
        { id: userId }
    );

    return res.json({ message: "Profile updated", user: updated });
}, [authMiddleware]);


Router.put("/api/profile/password", async (req, res) => {
    const db = await Database.getConnection();
    const body = req.body;
    const userId = req.user.user_id;

    if (!body.current_password || !body.new_password) {
        return res.status(400).json({ error: "Current password and new password are required" })
    }

    if (body.new_password.length < 8) {
        return res.status(400).json({ error: "New password must be at least 8 characters" });
    }

    const user = await db.fetchOne("SELECT password_hash FROM users WHERE id = :id", { id: userI

    if (!(await Auth.checkPassword(body.current_password, user.password_hash))) {
        return res.status(401).json({ error: "Current password is incorrect" });
    }

    const newHash = await Auth.hashPassword(body.new_password);
    await db.execute("UPDATE users SET password_hash = :hash WHERE id = :id", { hash: newHash, i

    return res.json({ message: "Password changed successfully" });
}, [authMiddleware]);
```

Expected output for register:

```
{
  "message": "Registration successful",
```

```
  "user": {
    "id": 1,
    "name": "Alice",
    "email": "alice@example.com",
    "role": "user",
    "created_at": "2026-03-22 16:00:00"
  }
}
```

(Status: `201 Created`)

Expected output for profile without token:

```
{"error":"Authorization required. Send: Authorization: Bearer <token>"}
```

(Status: `401 Unauthorized`)

---

13. Gotchas

1. Token Expiry Confusion

Problem: Tokens that worked yesterday now return 401.

Cause: The default token lifetime is 60 minutes. After that, the token is invalid even if the signature is correct.

Fix: Issue a new token at login. If your application needs long-lived sessions, use refresh tokens: a short-lived access token (15 minutes) paired with a long-lived refresh token (7 days) that can be used to get a new access token without re-entering credentials.

2. Secret Key Management

Problem: Tokens generated on one server are invalid on another, or tokens stop working after a deployment.

Cause: Each server generated its own random `secrets/jwt.key` file. Or the key file was deleted or regenerated during deployment.

Fix: Set `TINA4_SECRET` in `.env` explicitly and use the same value across all servers. Store it in your deployment secrets manager (not in version control). If the key changes, all existing tokens become invalid and users must log in again.

3. CORS with Authentication

Problem: Frontend requests with the `Authorization` header fail with a CORS error, even though `TINA4_CORS_ORIGINS=*` is set.

Cause: When the browser sends an `Authorization` header, it first sends a preflight `OPTIONS` request. The server must respond to the OPTIONS request with the correct CORS headers, including `Access-Control-Allow-Headers: Authorization`.

Fix: Tina4 handles preflight requests automatically. Make sure `TINA4_CORS_ORIGINS` is set correctly. If it is still failing, check that you are not overriding CORS headers in middleware.

---

4. Storing Tokens in localStorage

Problem: Your token is stolen via an XSS attack because it was stored in `localStorage`.

Cause: Any JavaScript on the page can read `localStorage`, including injected scripts from an XSS vulnerability.

Fix: Store tokens in `httpOnly` cookies when possible -- they cannot be accessed by JavaScript. For SPAs that must use `localStorage`, implement strict Content Security Policy headers and sanitize all user input.

5. Forgetting @noauth on Login

Problem: Your login endpoint returns 401 -- you cannot log in because the endpoint requires authentication.

Cause: If you have global auth middleware, the login endpoint needs the `@noauth` annotation to bypass it.

Fix: Add `@noauth` in a JSDoc comment above your login and register routes. Without it, users cannot authenticate because authentication is required to authenticate -- a catch-22.

6. Password Hash Column Too Short

Problem: Registration fails with a database error about the password hash being too long.

Cause: PBKDF2 hashes can be long. If your `password_hash` column is defined as `VARCHAR(50)`, it gets truncated.

Fix: Use `TEXT` for the password hash column, or at minimum `VARCHAR(255)`. Never constrain the hash length.

7. Token in URL Query Parameters

Problem: Tokens in URLs like `/api/profile?token=eyJ...` leak through browser history, server logs, and the Referer header.

Cause: Query parameters are visible in many places where headers are not.

Fix: Always send tokens in the `Authorization` header, never in the URL. The only exception is WebSocket connections, where the initial HTTP upgrade request cannot carry custom headers -- use a short-lived token for that case.

Sessions & Cookies

1. State in a Stateless World

JWT tokens handle APIs. Traditional web applications need more. A shopping cart that persists across pages. A flash message that appears once after a redirect. A "remember me" checkbox on the login form. These features run on sessions and cookies -- server-side state tied to a browser.

Picture an e-commerce site. A customer adds three items to their cart. Navigates to a product page. Comes back to the cart. Without sessions, the cart is empty every time. Sessions give the server memory. Each browser gets its own state, preserved across requests.

Sessions are auto-started. Every route handler receives `req.session` ready to use. No manual setup. No configuration required for the default file backend.

---

2. Session Configuration

The default backend is file-based sessions. They work out of the box with no additional dependencies.

To change the backend, set `TINA4_SESSION_BACKEND` in `.env`:

```
TINA4_SESSION_BACKEND=file
```

Available Backends

Backend	Value	Package Required	Best For
File	`file`	None	Development, single server
Redis	`redis`	`ioredis`	Production, multi-server
MongoDB	`mongodb`	`mongodb`	Production, document stores
Valkey	`valkey`	`iovalkey`	Production, Redis alternative
Database	`database`	None	Production, using existing DB

Redis Configuration

```
TINA4_SESSION_BACKEND=redis
TINA4_SESSION_REDIS_HOST=localhost
TINA4_SESSION_REDIS_PORT=6379
TINA4_SESSION_REDIS_PASSWORD=
```

Install the Redis driver:

```
npm install ioredis
```

MongoDB Configuration

```
TINA4_SESSION_BACKEND=mongodb
TINA4_SESSION_REDIS_HOST=localhost
TINA4_SESSION_REDIS_PORT=27017
TINA4_SESSION_MONGO_DB=tina4_sessions
```

Valkey Configuration

```
TINA4_SESSION_BACKEND=valkey
TINA4_SESSION_REDIS_HOST=localhost
TINA4_SESSION_REDIS_PORT=6379
```

Database Sessions

```
TINA4_SESSION_BACKEND=database
```

Stores sessions in the `tina4_session` table using your existing database connection (`TINA4_DATABASE_URL`). The table is auto-created on first use. Works with all 5 database engines (SQLite, PostgreSQL, MySQL, MSSQL, Firebird).

Session Lifetime

```
TINA4_SESSION_TTL=3600  # 1 hour in seconds (default)
```

Session Cookie

The session cookie is named `tina4_session`. It is `HttpOnly` and `SameSite=Lax` by default. Tina4 manages it -- you never need to set or read it yourself.

---

3. The Session API

Access session data through `req.session`. It is available in every route handler with zero configuration.

Full API Reference

Method	Description
`req.session.set(key, value)`	Store a value
`req.session.get(key, default)`	Retrieve a value (with optional default)
`req.session.delete(key)`	Remove a key
`req.session.has(key)`	Check if a key exists
`req.session.clear()`	Remove all session data
`req.session.destroy()`	Destroy the session entirely
`req.session.save()`	Persist session data (auto-called after response)
`req.session.regenerate()`	Generate a new session ID, preserve data
`req.session.flash(key, value)`	Set flash data (one-time read)
`req.session.flash(key)`	Read and remove flash data (dual-mode)
`req.session.getFlash(key)`	Explicit getter alias for flash(key)
`req.session.getSessionId()`	Get current session ID
`req.session.cookieHeader()`	Get Set-Cookie header value
`req.session.all()`	Get all session data as an object

---

4. Reading and Writing Session Data

Writing to the Session

```
import { Router } from "tina4-nodejs";

Router.post("/login-form", async (req, res) => {
    // After validating credentials...
    req.session.set("user_id", 42);
    req.session.set("user_name", "Alice");
    req.session.set("role", "admin");
    req.session.set("logged_in", true);

    return res.redirect("/dashboard");
});
```

Reading from the Session

```
import { Router } from "tina4-nodejs";

Router.get("/dashboard", async (req, res) => {
    if (!req.session.get("logged_in")) {
        return res.redirect("/login");
    }

    return res.render("dashboard.html", {
        user_name: req.session.get("user_name"),
        role: req.session.get("role")
    });
});
```

Deleting Session Data

```
// Delete a single key
req.session.delete("temp_data");

// Clear all session data (logout)
Router.post("/logout", async (req, res) => {
    req.session.clear();
    return res.redirect("/login");
});
```

Checking if a Key Exists

```
if (req.session.has("user_id")) {
    const userId = req.session.get("user_id");
} else {
    const userId = null;
}

// Or use .get() with a default
const userId = req.session.get("user_id", null);
```

Getting All Session Data

```
const allData = req.session.all();
```

Preferences Example

```
import { Router } from "tina4-nodejs";

Router.post("/api/preferences", async (req, res) => {
    const body = req.body;

    req.session.set("language", body.language ?? "en");
    req.session.set("theme", body.theme ?? "light");
    req.session.set("items_per_page", parseInt(body.items_per_page ?? "20", 10));

    return res.json({
        message: "Preferences saved",
        preferences: {
            language: req.session.get("language"),
            theme: req.session.get("theme"),
            items_per_page: req.session.get("items_per_page")
        }
    });
});

Router.get("/api/preferences", async (req, res) => {
    return res.json({
        language: req.session.get("language", "en"),
        theme: req.session.get("theme", "light"),
        items_per_page: req.session.get("items_per_page", 20)
    });
});
```

Visit Counter Example

```
import { Router } from "tina4-nodejs";

Router.get("/visit-counter", async (req, res) => {
    const count = (req.session.get("visit_count", 0)) + 1;
    req.session.set("visit_count", count);

    return res.json({
        visit_count: count,
        message: `You have visited this page ${count} time${count === 1 ? "" : "s"}`
    });
});
```

```
curl http://localhost:7148/visit-counter -c cookies.txt -b cookies.txt
```

```
{"visit_count":1,"message":"You have visited this page 1 time"}
```

---

5. Flash Messages

Flash messages are session values that exist for exactly one read. Set them before a redirect. Read them on the next request. They vanish after being read.

Setting a Flash Message

```
import { Router } from "tina4-nodejs";

Router.post("/api/contact", async (req, res) => {
    // Process the form...

    req.session.flash("message", "Your message has been sent!");
    req.session.flash("message_type", "success");

    return res.redirect("/contact");
});
```

Reading a Flash Message

Flash is dual-mode: `flash(key, value)` sets, `flash(key)` gets and removes. `getFlash()` is an explicit alias.

```
Router.get("/contact", async (req, res) => {
    const flashMessage = req.session.flash("message");       // get + auto-remove
    const flashType = req.session.flash("message_type") ?? "info";

    return res.render("contact.html", {
        flash_message: flashMessage,
        flash_type: flashType
    });
});
```

The `getFlash()` method reads the value and removes it in one step. The next time the user visits `/contact`, the flash message will be gone.

Flash Messages in Templates

```
{% if flash_message %}
    <div class="alert alert-{{ flash_type }}">
        {{ flash_message }}
    </div>
{% endif %}
```

---

6. Cookies

Cookies are small data fragments stored in the browser. Unlike sessions, cookies travel with every request and are visible to JavaScript (unless `httpOnly` is set).

Setting a Cookie

```
import { Router } from "tina4-nodejs";

Router.post("/preferences", async (req, res) => {
    const theme = req.body.theme ?? "light";
    const language = req.body.language ?? "en";

    return res
        .cookie("theme", theme, {
            maxAge: 365 * 24 * 60 * 60,  // 1 year in seconds
            path: "/",
            sameSite: "Lax"
        })
        .cookie("language", language, {
            maxAge: 365 * 24 * 60 * 60,
            path: "/",
            sameSite: "Lax"
        })
        .json({ message: "Preferences saved" });
});
```

Reading a Cookie

```
import { Router } from "tina4-nodejs";

Router.get("/", async (req, res) => {
    const theme = req.cookies.theme ?? "light";
    const language = req.cookies.language ?? "en";

    return res.render("home.html", {
        theme,
        language
    });
});
```

Deleting a Cookie

Set `maxAge` to 0:

```
Router.post("/clear-preferences", async (req, res) => {
    return res
        .cookie("theme", "", { maxAge: 0, path: "/" })
        .cookie("language", "", { maxAge: 0, path: "/" })
        .json({ message: "Preferences cleared" });
});
```

Cookie Options

Option	Type	Description
`maxAge`	number	Lifetime in seconds. 0 = delete. Omit = session cookie (deleted when browser closes).
`expires`	Date	Expiry date. `maxAge` is preferred -- it is simpler and more predictable.
`path`	string	URL path scope. `/` means the whole site. `/admin` means only under `/admin`.
`domain`	string	Domain scope. `.example.com` includes subdomains.
`secure`	boolean	Only send over HTTPS. Always `true` in production.
`httpOnly`	boolean	Not accessible via JavaScript. Use for session cookies.
`sameSite`	string	`"Strict"`, `"Lax"`, or `"None"`. Controls cross-site sending.

When to Use Cookies vs Sessions

Use Case	Use Cookies	Use Sessions
User preferences (theme, language)	Yes	No
Shopping cart	No	Yes
Authentication state	No (use JWT in header)	Yes (for form-based auth)
Flash messages	No	Yes
Tracking consent	Yes	No
Sensitive data (user ID, role)	No	Yes

The rule: sensitive data or data the client should not see goes in sessions. Non-sensitive data that should persist beyond session expiry goes in cookies.

---

7. Remember Me

The "remember me" pattern extends the session lifetime when the user checks a box on the login form:

```
import { Router } from "tina4-nodejs";

Router.post("/login", async (req, res) => {
    const { email, password, remember } = req.body;

    // Validate credentials...
    const user = await authenticateUser(email, password);
    if (!user) {
        req.session.flash("message", "Invalid email or password");
        req.session.flash("message_type", "error");
        return res.redirect("/login");
    }

    // Regenerate session ID (prevents session fixation)
    req.session.regenerate();

    // Set session data
    req.session.set("user_id", user.id);
    req.session.set("user_name", user.name);
    req.session.set("logged_in", true);

    // Handle "remember me"
    if (remember) {
        // Generate a long-lived token
        const Auth = require("tina4-nodejs").Auth;
        const rememberToken = Auth.getToken({ user_id: user.id });

        // Store it in a cookie that lasts 30 days
        return res
            .cookie("remember_token", rememberToken, {
                maxAge: 30 * 24 * 60 * 60,
                httpOnly: true,
                secure: true,
                path: "/",
                sameSite: "Lax"
            })
            .redirect("/dashboard");
    }

    return res.redirect("/dashboard");
});
```

Then, on every page load, check for the remember token if the session has expired:

```
async function rememberMeMiddleware(req, res, nextHandler) {
    if (!req.session.get("logged_in")) {
        const rememberToken = req.cookies.remember_token;

        if (rememberToken && Auth.validToken(rememberToken)) {
            const payload = Auth.getPayload(rememberToken);
            const db = new Database();
```

```
            const user = await db.fetchOne(
                "SELECT id, name FROM users WHERE id = ?",
                [payload.user_id]
            );

            if (user) {
                req.session.set("user_id", user.id);
                req.session.set("user_name", user.name);
                req.session.set("logged_in", true);
            }
        }
    }

    return await nextHandler(req, res);
}
```

The form:

```
<form method="POST" action="/login">
    {{ form_token() }}
    <div class="form-group">
        <label for="email">Email</label>
        <input type="email" id="email" name="email" class="form-control" required>
    </div>
    <div class="form-group">
        <label for="password">Password</label>
        <input type="password" id="password" name="password" class="form-control" required>
    </div>
    <div class="form-check">
        <input type="checkbox" id="remember" name="remember" value="1" class="form-check-input">
        <label for="remember" class="form-check-label">Remember me for 30 days</label>
    </div>
    <button type="submit" class="btn btn-primary">Login</button>
</form>
```

Without the checkbox, the session uses the default TTL (1 hour). With the checkbox, the remember token cookie persists for 30 days. The server controls the duration -- the client cannot extend it.

---

8. Session Security

Regenerate Session ID After Login

To prevent session fixation attacks, regenerate the session ID after login:

```
Router.post("/login-form", async (req, res) => {
    // After validating credentials...

    // Regenerate session ID (prevents session fixation)
    req.session.regenerate();

    req.session.set("user_id", user.id);
    req.session.set("logged_in", true);

    return res.redirect("/dashboard");
});
```

Destroy a Session

To destroy a session entirely (not just clear its data):

```
Router.post("/logout", async (req, res) => {
    req.session.destroy();
    return res.redirect("/login");
});
```

Secure Cookie Settings

The session cookie (`tina4_session`) is `HttpOnly` and `SameSite=Lax` by default. For production, ensure HTTPS:

```
TINA4_SESSION_SECURE=true     # Only send over HTTPS
TINA4_SESSION_HTTPONLY=true   # Not accessible via JavaScript (default)
TINA4_SESSION_SAMESITE=Lax   # Prevent CSRF via cross-site requests (default)
```

`TINA4_SESSION_SAMESITE` controls cross-site cookie behavior:

Value	Behavior
`Strict`	Never sent with cross-site requests. Safest. Breaks some flows (clicking links from email).
`Lax`	Sent with top-level navigations (clicking links) but not cross-site API calls. Good default.
`None`	Always sent. Requires `TINA4_SESSION_SECURE=true`. Only for cross-site cookie access.

Session Configuration Options

Variable	Default	Description
`TINA4_SESSION_BACKEND`	`file`	Storage backend: `file`, `redis`, `mongodb`, `valkey`, `database`
`TINA4_SESSION_TTL`	`3600`	Session lifetime in seconds
`TINA4_SESSION_REDIS_HOST`	`localhost`	Host for Redis/MongoDB/Valkey
`TINA4_SESSION_REDIS_PORT`	`6379`	Port for Redis/MongoDB/Valkey
`TINA4_SESSION_REDIS_PASSWORD`	(none)	Password for Redis/Valkey
`TINA4_SESSION_SECURE`	`false`	Cookie sent only over HTTPS
`TINA4_SESSION_HTTPONLY`	`true`	Cookie inaccessible to JavaScript
`TINA4_SESSION_SAMESITE`	`Lax`	Cross-site cookie policy

---

9. Exercise: Build a Shopping Cart

Build a shopping cart using sessions.

Requirements

- Create these endpoints:

Method	Path	Description
`GET`	`/cart`	View cart contents (rendered HTML page)
`POST`	`/cart/add`	Add an item to the cart
`POST`	`/cart/update`	Update item quantity
`POST`	`/cart/remove`	Remove an item from the cart
`POST`	`/cart/clear`	Clear the entire cart
`GET`	`/cart/api`	Get cart as JSON (for AJAX)

- The cart is stored in the session via `req.session.set("cart", [...])`
- Each cart item has: `product_id`, `name`, `price`, `quantity`

- Adding an existing product increments its quantity
- The cart page shows items, quantities, line totals, and a grand total
- Use flash messages for feedback ("Item added", "Cart cleared", etc.)

Use this product data for validation:

```
const PRODUCTS: Record<number, { name: string; price: number }> = {
    1: { name: "Wireless Keyboard", price: 79.99 },
    2: { name: "USB-C Hub", price: 49.99 },
    3: { name: "Monitor Stand", price: 129.99 },
    4: { name: "Mechanical Mouse", price: 59.99 },
    5: { name: "Desk Lamp", price: 39.99 }
};
```

Test with:

```
# Add items
curl -X POST http://localhost:7148/cart/add \
  -H "Content-Type: application/json" \
  -d '{"product_id": 1, "quantity": 2}' \
  -c cookies.txt -b cookies.txt

curl -X POST http://localhost:7148/cart/add \
  -H "Content-Type: application/json" \
  -d '{"product_id": 3, "quantity": 1}' \
  -c cookies.txt -b cookies.txt

# View cart
curl http://localhost:7148/cart/api -b cookies.txt

# Update quantity
curl -X POST http://localhost:7148/cart/update \
  -H "Content-Type: application/json" \
  -d '{"product_id": 1, "quantity": 3}' \
  -c cookies.txt -b cookies.txt

# Remove item
curl -X POST http://localhost:7148/cart/remove \
  -H "Content-Type: application/json" \
  -d '{"product_id": 3}' \
  -c cookies.txt -b cookies.txt

# Clear cart
curl -X POST http://localhost:7148/cart/clear -c cookies.txt -b cookies.txt
```

---

10. Solution

Create `src/routes/cart.ts`:

```
import { Router } from "tina4-nodejs";

const PRODUCTS: Record<number, { name: string; price: number }> = {
    1: { name: "Wireless Keyboard", price: 79.99 },
    2: { name: "USB-C Hub", price: 49.99 },
    3: { name: "Monitor Stand", price: 129.99 },
    4: { name: "Mechanical Mouse", price: 59.99 },
```

```
    5: { name: "Desk Lamp", price: 39.99 }
};


function getCart(session: any): any[] {
    return session.get("cart", []);
}


function saveCart(session: any, cart: any[]): void {
    session.set("cart", cart);
}


function calculateTotals(cart: any[]): { cart: any[]; grandTotal: number } {
    for (const item of cart) {
        item.line_total = Math.round(item.price * item.quantity * 100) / 100;
    }
    const grandTotal = Math.round(
        cart.reduce((sum, item) => sum + item.line_total, 0) * 100
    ) / 100;
    return { cart, grandTotal };
}


Router.get("/cart", async (req, res) => {
    const cart = getCart(req.session);
    const { grandTotal } = calculateTotals(cart);
    const flashMessage = req.session.getFlash("message");
    const flashType = req.session.getFlash("message_type") ?? "info";

    return res.render("cart.html", {
        cart,
        grand_total: grandTotal,
        item_count: cart.reduce((sum, item) => sum + item.quantity, 0),
        flash_message: flashMessage,
        flash_type: flashType
    });
});


Router.get("/cart/api", async (req, res) => {
    const cart = getCart(req.session);
    const { grandTotal } = calculateTotals(cart);

    return res.json({
        cart,
        grand_total: grandTotal,
        item_count: cart.reduce((sum, item) => sum + item.quantity, 0)
    });
});


Router.post("/cart/add", async (req, res) => {
    const body = req.body;
    const productId = parseInt(body.product_id, 10);
    const quantity = parseInt(body.quantity ?? "1", 10);

    if (!PRODUCTS[productId]) {
```

```
        return res.status(404).json({ error: "Product not found" });
    }

    if (quantity < 1) {
        return res.status(400).json({ error: "Quantity must be at least 1" });
    }

    const product = PRODUCTS[productId];
    const cart = getCart(req.session);

    // Check if product already in cart
    const existing = cart.find(item => item.product_id === productId);
    if (existing) {
        existing.quantity += quantity;
        saveCart(req.session, cart);
        req.session.flash("message", `Updated ${product.name} quantity`);
        req.session.flash("message_type", "success");
        return res.json({ message: `Updated ${product.name} quantity`, cart });
    }

    // Add new item
    cart.push({
        product_id: productId,
        name: product.name,
        price: product.price,
        quantity
    });
    saveCart(req.session, cart);

    req.session.flash("message", `Added ${product.name} to cart`);
    req.session.flash("message_type", "success");

    return res.status(201).json({ message: `Added ${product.name}`, cart });
});


Router.post("/cart/update", async (req, res) => {
    const body = req.body;
    const productId = parseInt(body.product_id, 10);
    const quantity = parseInt(body.quantity ?? "1", 10);

    const cart = getCart(req.session);
    const index = cart.findIndex(item => item.product_id === productId);

    if (index === -1) {
        return res.status(404).json({ error: "Product not in cart" });
    }

    if (quantity < 1) {
        const removed = cart[index];
        cart.splice(index, 1);
        req.session.flash("message", `Removed ${removed.name} from cart`);
    } else {
        cart[index].quantity = quantity;
        req.session.flash("message", `Updated ${cart[index].name} quantity to ${quantity}`);
    }

    req.session.flash("message_type", "success");
    saveCart(req.session, cart);
```

```
    return res.json({ message: "Cart updated", cart });
});


Router.post("/cart/remove", async (req, res) => {
    const productId = parseInt(req.body.product_id, 10);
    const cart = getCart(req.session);
    const index = cart.findIndex(item => item.product_id === productId);

    if (index === -1) {
        return res.status(404).json({ error: "Product not in cart" });
    }

    const removed = cart[index];
    cart.splice(index, 1);
    saveCart(req.session, cart);

    req.session.flash("message", `Removed ${removed.name} from cart`);
    req.session.flash("message_type", "success");

    return res.json({ message: `Removed ${removed.name}`, cart });
});


Router.post("/cart/clear", async (req, res) => {
    saveCart(req.session, []);
    req.session.flash("message", "Cart cleared");
    req.session.flash("message_type", "info");
    return res.json({ message: "Cart cleared", cart: [] });
});
```

Expected output for cart API after adding items:

```
{
  "cart": [
    {"product_id": 1, "name": "Wireless Keyboard", "price": 79.99, "quantity": 2, "line_total":
    {"product_id": 3, "name": "Monitor Stand", "price": 129.99, "quantity": 1, "line_total": 129
  ],
  "grand_total": 289.97,
  "item_count": 3
}
```

---

11. Gotchas

1. Session lost between requests

Problem: Data you stored in the session is gone on the next request.

Cause: The session cookie is not being sent back by the client. This happens when using curl without `-c cookies.txt -b cookies.txt`, or when the browser blocks cookies.

Fix: For curl testing, use `-c cookies.txt -b cookies.txt` to save and send cookies. In the browser, make sure cookies are not blocked for your site.

2. Session data is not JSON-serializable

Problem: Storing a JavaScript object with circular references or class instances in the session causes an error.

Cause: Sessions are serialized to JSON. Objects with circular references, `Map`, `Set`, `Date`, or custom class instances cannot be serialized directly.

Fix: Convert objects to plain values before storing. For dates: `req.session.set("login_time", new Date().toISOString())`. For maps: convert to a plain object with `Object.fromEntries(myMap)`.

3. Cookie not sent on cross-origin requests

Problem: Your frontend at `http://localhost:3000` calls your API at `http://localhost:7148`, but cookies are not included.

Cause: Browsers do not send cookies cross-origin by default. You need both CORS configuration and explicit `credentials: "include"` in fetch.

Fix: Set `CORS_CREDENTIALS=true` in `.env` and use `fetch(url, { credentials: "include" })` in your frontend JavaScript. Also set `TINA4_CORS_ORIGINS` to the specific frontend origin (not `*` -- wildcard does not work with credentials).

4. File-based sessions on multi-server deployments

Problem: A user is logged in on server A but logged out on server B.

Cause: File-based sessions are stored on the local filesystem. Each server has its own session files.

Fix: Switch to Redis, Valkey, MongoDB, or database-backed sessions so all servers share the same session store.

5. Session cookie overwritten by another app

Problem: Your session keeps getting reset even though you set it correctly.

Cause: Another application on the same domain uses the same session cookie name.

Fix: The session cookie name (`tina4_session`) is managed by Tina4. If you need to change it, set `TINA4_SESSION_NAME=myapp_session` in `.env`.

6. Secure cookies on localhost

Problem: Setting `secure: true` on a cookie means it never gets sent during development.

Cause: Secure cookies are only sent over HTTPS. `http://localhost` is not HTTPS.

Fix: Do not set `secure: true` during development. Use environment-based configuration: `secure: process.env.TINA4_DEBUG !== "true"`.

7. Flash messages shown twice

Problem: A flash message appears on the page, but when you refresh, it shows again.

Cause: You read the flash message but did not remove it. Using `req.session.get()` reads without deleting.

Fix: Use `req.session.getFlash("message")` to read flash data. It reads and deletes in one step. Do not use `req.session.get()` for flash messages.

8. Session data disappears after server restart

Problem: All sessions vanish when you restart the Node.js process.

Cause: File-based sessions may be stored in a temporary directory that gets cleared on restart, or in-memory state was lost.

Fix: Use Redis or database sessions for production. For file sessions, set `TINA4_SESSION_PATH` to a persistent directory.

9. Large session data causes slow requests

Problem: Pages load slower than expected. Session reads and writes take visible time.

Cause: You stored large objects in the session -- full user profiles, query results, or file contents.

Fix: Keep session data small. Store IDs, not entire objects. Look up the full data from the database when you need it.

Middleware & Security

1. The Pipeline Pattern

Every HTTP request passes through a series of gates before reaching your route handler. Rate limiter. Body parser. Auth check. Logger. These gates are middleware -- code that wraps your route handler and runs before, after, or both.

Picture a public API. Every request hits a rate limit check. Some endpoints require an API key. All responses need CORS headers. Errors need logging. Without middleware, that logic lives in every handler. Duplicated. Scattered. Fragile. With middleware, you write it once and attach it where it belongs.

Tina4 Node.js ships with built-in middleware (CORS, rate limiting, request logging, security headers) and lets you write your own. This chapter covers both.

---

2. What Middleware Is

Middleware is code that runs before or after your route handler. It sits in the HTTP pipeline between the incoming request and the response. Every request can pass through multiple middleware layers before reaching the handler.

Tina4 Node.js supports two styles of middleware:

Function-based middleware receives `req`, `res`, and `next`. Call `next()` to continue. Skip it to short-circuit.

```
function passthrough(req, res, next) {
    next();
}
```

```
async function blockEverything(req, res, next) {
    return res.status(503).json({ error: "Service unavailable" });
}
```

Class-based middleware uses naming conventions. Static methods whose names start with `before` run before the handler (via `MiddlewareRunner.runBefore`). Methods starting with `after` run after it (via `MiddlewareRunner.runAfter`). Each method receives `(req, res)` and returns `[req, res]`.

```
class MyMiddleware {
    static beforeCheck(req, res) {
        // Runs before the route handler
        return [req, res];
    }

    static afterCleanup(req, res) {
        // Runs after the route handler
        return [req, res];
    }
}
```

If a `before*` method sets the response status to >= 400, the handler is skipped. This is short-circuiting.

Register class-based middleware globally with `Router.use()`:

```
import { Router, CorsMiddleware, RateLimiterMiddleware, RequestLogger } from "tina4-nodejs";

Router.use(CorsMiddleware);
Router.use(RateLimiterMiddleware);
Router.use(RequestLogger);
```

---

3. Built-in CorsMiddleware

CORS (Cross-Origin Resource Sharing) controls which domains can call your API from a browser. When React at `http://localhost:3000` calls your Tina4 API at `http://localhost:7148`, the browser sends a preflight `OPTIONS` request first. Wrong headers and the browser blocks everything.

Tina4 provides both a function-based `cors()` middleware and a class-based `CorsMiddleware`. Configure via `.env`:

```
TINA4_CORS_ORIGINS=http://localhost:3000,https://myapp.com
TINA4_CORS_METHODS=GET,POST,PUT,PATCH,DELETE,OPTIONS
TINA4_CORS_HEADERS=Content-Type,Authorization,X-Request-ID
TINA4_CORS_MAX_AGE=86400
```

With these settings, only `localhost:3000` and `myapp.com` can make cross-origin requests to your API. The browser handles preflight `OPTIONS` requests on its own.

For development, allow all origins:

```
TINA4_CORS_ORIGINS=*
```

Apply using the function-based form on a single route:

```
import { Router, cors } from "tina4-nodejs";

Router.get("/api/products", async (req, res) => {
    return res.json({ products: [] });
}, [cors()]);
```

Or apply the class-based form globally via `Router.use()`:

```
Router.use(CorsMiddleware);
```

The CORS middleware is active by default when registered. You do not need to handle preflight requests yourself.

You can also use the `cors()` function programmatically to inspect or apply CORS headers in your own middleware:

```
import { cors } from "tina4-nodejs";

function customCors(req, res, next) {
    const origin = req.headers["origin"] ?? "";
```

```
    // Only allow CORS for specific paths
    if (req.url.startsWith("/api/public")) {
        cors()(req, res, next);
        return;
    }

    next();
}
```

Preflight `OPTIONS` requests return `204 No Content` with the correct CORS headers. The browser caches the preflight based on `TINA4_CORS_MAX_AGE`.

---

4. Built-in RateLimiterMiddleware

The rate limiter prevents a single client from flooding your API. It uses a sliding-window algorithm that tracks requests per IP in memory. Configure via `.env`:

```
TINA4_RATE_LIMIT=60
TINA4_RATE_WINDOW=60
```

This allows 60 requests per 60-second window per IP address. Apply it:

```
Router.use(RateLimiterMiddleware);
```

When a client exceeds the limit, they receive a `429 Too Many Requests` response with a `Retry-After` header.

Rate limit headers are included in every response:

```
X-RateLimit-Limit: 60
X-RateLimit-Remaining: 57
X-RateLimit-Reset: 1711113060
Retry-After: 42
```

You can use the `RateLimiterMiddleware` class directly for custom rate limiting logic:

```
import { RateLimiterMiddleware } from "tina4-nodejs";

function customRateLimit(req, res, next) {
    const clientIp = req.socket?.remoteAddress ?? "unknown";
    const result = RateLimiterMiddleware.check(clientIp);

    if (!result.allowed) {
        res({
            error: "Too many requests",
            retry_after: result.reset
        }, 429);
        return;
    }

    res.header("X-RateLimit-Limit", String(result.limit));
    res.header("X-RateLimit-Remaining", String(result.remaining));
    res.header("X-RateLimit-Reset", String(result.reset));

    next();
}
```

Like CORS, the rate limiter is active by default based on your `.env` configuration.

---

5. Built-in RequestLogger

The `RequestLogger` middleware logs every request with timing and coloured status codes. It uses two hooks:

- `beforeLog` stamps the start time before the handler runs
- `afterLog` calculates elapsed time and prints a coloured log line

Register it globally:

```
import { Router, RequestLogger } from "tina4-nodejs";

Router.use(RequestLogger);
```

The console output looks like:

```
  200 GET /api/users 12ms
  201 POST /api/products 45ms
  404 GET /api/missing 2ms
```

Green for 2xx, yellow for 3xx, red for 4xx and 5xx.

Built-in SecurityHeadersMiddleware

The `SecurityHeadersMiddleware` adds standard security headers to every response. Register it globally:

```
import { Router, SecurityHeadersMiddleware } from "tina4-nodejs";

Router.use(SecurityHeadersMiddleware);
```

It sets the following headers by default:

Header	Default Value
`X-Frame-Options`	`SAMEORIGIN`
`Content-Security-Policy`	`default-src 'self'`, plus `style-src` and `script-src` limited to `'self'` and a per-request nonce
`Strict-Transport-Security`	Not sent. With `TINA4_HSTS` set to a number of seconds: `max-age=<TINA4_HSTS>; includeSubDomains`, on HTTPS requests only
`Referrer-Policy`	`strict-origin-when-cross-origin`
`Permissions-Policy`	`camera=(), microphone=(), geolocation=()`
`X-Content-Type-Options`	`nosniff`

Override any header via environment variables in `.env`:

```
TINA4_FRAME_OPTIONS=SAMEORIGIN
TINA4_CSP=default-src 'self'; script-src 'self' https://cdn.example.com
TINA4_HSTS=63072000
TINA4_REFERRER_POLICY=no-referrer
TINA4_PERMISSIONS_POLICY=camera=(), microphone=(), geolocation=(self)
```

Combining All Four Built-In Middleware

A common production setup:

```
import { Router, CorsMiddleware, RateLimiterMiddleware, RequestLogger, SecurityHeadersMiddleware

Router.use(CorsMiddleware);
Router.use(RateLimiterMiddleware);
Router.use(RequestLogger);
Router.use(SecurityHeadersMiddleware);
```

Order matters. CORS handles `OPTIONS` preflight first. The rate limiter only counts real requests (not preflight). The logger measures total time including the other middleware. Security headers are added to every response.

---

6. Writing Custom Middleware

Function-Based Middleware

A middleware function takes three arguments: `req`, `res`, and `next`. It must call `next()` to continue the chain or return a response to short-circuit.

```
function myMiddleware(req, res, next) {
    // Code that runs BEFORE the route handler
    console.log("Before handler");

    // Call the next middleware or the route handler
    next();

    // Code that runs AFTER the route handler
    console.log("After handler");
}
```

Class-Based Middleware

Class-based middleware uses a naming convention: static methods prefixed with `before` run before the route handler, and methods prefixed with `after` run after it. Each method receives `(req, res)` and returns `[req, res]`.

```
class MyMiddleware {
    static beforeCheck(req, res) {
        // Runs before the route handler
        console.log("Before handler");
        return [req, res];
    }

    static afterCleanup(req, res) {
```

```
        // Runs after the route handler
        console.log("After handler");
        return [req, res];
    }
}
```

If a `before*` method sets the response status to >= 400, the route handler is skipped. This is short-circuiting.

Example: Request Timer (Class-Based)

```
class TimingMiddleware {
    static beforeStartTimer(req, res) {
        (req as any).startTime = Date.now();
        return [req, res];
    }

    static afterAddTiming(req, res) {
        const elapsed = Date.now() - ((req as any).startTime ?? Date.now());
        res.header("X-Response-Time", `${elapsed}ms`);
        return [req, res];
    }
}
```

Example: Request Logger (Function-Based)

```
function logMiddleware(req, res, next) {
    const timestamp = new Date().toISOString();
    console.log(`[${timestamp}] ${req.method} ${req.url}`);
    console.log(`  Headers: ${JSON.stringify(req.headers)}`);

    if (req.body) {
        console.log(`  Body: ${JSON.stringify(req.body)}`);
    }

    const start = Date.now();
    next();
    const duration = Date.now() - start;

    console.log(`  Completed in ${duration}ms`);
}
```

Example: Security Headers (Class-Based)

```
class CustomSecurityHeaders {
    static afterSecurity(req, res) {
        res.header("X-Content-Type-Options", "nosniff");
        res.header("X-Frame-Options", "DENY");
        res.header("X-XSS-Protection", "1; mode=block");
        res.header("Strict-Transport-Security", "max-age=31536000");
        return [req, res];
    }
}
```

Example: JSON Content-Type Enforcer (Function-Based)

```
function requireJson(req, res, next) {
    if (["POST", "PUT", "PATCH"].includes(req.method)) {
        const contentType = req.headers["content-type"] ?? "";

        if (!contentType.includes("application/json")) {
            res({
                error: "Content-Type must be application/json",
                received: contentType
            }, 415);
            return;
        }
    }

    next();
}
```

Example: Request ID (Class-Based)

```
import { randomUUID } from "crypto";

class RequestIdMiddleware {
    static beforeInjectId(req, res) {
        (req as any).requestId = randomUUID();
        return [req, res];
    }

    static afterAddHeader(req, res) {
        res.header("X-Request-ID", (req as any).requestId);
        return [req, res];
    }
}
```

Example: Input Sanitization (Class-Based)

```
class InputSanitizer {
    static beforeSanitize(req, res) {
        if (req.body && typeof req.body === "object") {
            req.body = InputSanitizer.sanitize(req.body);
        }
        return [req, res];
    }

    private static sanitize(data: Record<string, any>): Record<string, any> {
        const clean: Record<string, any> = {};
        for (const [key, value] of Object.entries(data)) {
            if (typeof value === "string") {
                clean[key] = value.replace(/[<>&"']/g, (c) =>
                    ({ "<": "&lt;", ">": "&gt;", "&": "&amp;", '"': "&quot;", "'": "&#39;" })[c]
                );
            } else if (typeof value === "object" && value !== null) {
                clean[key] = InputSanitizer.sanitize(value);
            } else {
                clean[key] = value;
            }
        }
        return clean;
    }
}
```

Example: IP Whitelist (Function-Based)

```
function ipWhitelist(req, res, next) {
    const allowedIps = (process.env.ALLOWED_IPS ?? "127.0.0.1").split(",");
    const clientIp = req.socket?.remoteAddress ?? "";

    if (!allowedIps.includes(clientIp)) {
        res({ error: "Access denied", your_ip: clientIp }, 403);
        return;
    }

    next();
}
```

---

7. Applying Middleware to Routes

Apply middleware to a single route by passing an array as the third argument. Both function-based and class-based middleware work:

```
import { Router } from "tina4-nodejs";

// Function-based middleware on a single route
Router.get("/api/data", async (req, res) => {
    return res.json({ data: [1, 2, 3] });
}, [logMiddleware]);

// Class-based middleware on a single route
Router.get("/api/stats", async (req, res) => {
    return res.json({ stats: {} });
}, [TimingMiddleware]);
```

Apply multiple middleware by listing them in the array:

```
Router.post("/api/items", async (req, res) => {
    return res.status(201).json({ item: req.body });
}, [RequestIdMiddleware, CustomSecurityHeaders, requireJson]);
```

You can mix function-based and class-based middleware freely. They run in the order you list them. In the example above: `RequestIdMiddleware` runs first (before and after hooks), then `CustomSecurityHeaders`, then `requireJson`, then the route handler.

---

8. Middleware on Route Groups

Apply middleware to all routes in a group:

```
import { Router } from "tina4-nodejs";

Router.group("/api/v1", (group) => {
    group.get("/users", async (req, res) => {
        return res.json({ users: [] });
    });

    group.post("/users", async (req, res) => {
        return res.status(201).json({ created: true });
```

```
    });

    group.get("/products", async (req, res) => {
        return res.json({ products: [] });
    });
}, [TimingMiddleware, CustomSecurityHeaders]);
```

Every route inside the group now has `TimingMiddleware` and `CustomSecurityHeaders` applied. You can still add route-specific middleware on top:

```
Router.group("/api/v1", (group) => {
    group.get("/public", async (req, res) => {
        // Only group middleware runs
        return res.json({ public: true });
    });

    group.post("/admin", async (req, res) => {
        // Group middleware + authMiddleware both run
        return res.json({ admin: true });
    }, [authMiddleware]);
}, [RequestIdMiddleware]);
```

Group middleware always runs before route-specific middleware. This means authentication checks at the group level cannot be bypassed by individual routes.

---

9. Execution Order

Stacked middleware forms a nested pipeline. Requests travel inward. Responses travel outward:

```
Request arrives
  → logMiddleware (before)
    → authMiddleware (before)
      → timerMiddleware (before)
        → route handler
      → timerMiddleware (after)
    → authMiddleware (after)
  → logMiddleware (after)
Response sent
```

Each middleware wraps around the next one. The outermost middleware runs first on the way in and last on the way out.

Here is a concrete example showing the order:

```
function middlewareA(req, res, next) {
    console.log("A: before");
    next();
    console.log("A: after");
}

function middlewareB(req, res, next) {
    console.log("B: before");
    next();
    console.log("B: after");
}

Router.get("/test", async (req, res) => {
```

```
    console.log("Handler");
    return res.json({ ok: true });
}, [middlewareA, middlewareB]);
```

When you request `/test`, the console shows:

```
A: before
B: before
Handler
B: after
A: after
```

---

10. Short-Circuiting

Skip `next()` and the chain stops cold. The route handler never runs. This is how blocking middleware works:

```
function maintenanceMode(req, res, next) {
    if (process.env.MAINTENANCE_MODE === "true") {
        // Allow health checks through
        if (req.url === "/health") {
            next();
            return;
        }
        res({
            error: "Service is under maintenance. Please try again later.",
            retry_after: 300
        }, 503);
        return;
    }

    next();
}
```

When `MAINTENANCE_MODE=true`, every request gets a 503 response without reaching any route handler. The health check endpoint still works -- a common pattern for load balancers.

Conditional Short-Circuit

```
function requireApiKey(req, res, next) {
    const apiKey = req.headers["x-api-key"] ?? "";

    if (!apiKey) {
        res({ error: "API key required" }, 401);
        return;
    }

    // Validate the key against a list or database
    const validKeys = ["key-abc-123", "key-def-456", "key-ghi-789"];
    if (!validKeys.includes(apiKey)) {
        res({ error: "Invalid API key" }, 403);
        return;
    }

    // Attach the key info to the request for the handler to use
    (req as any).apiKey = apiKey;

    next();
}
```

```
# No key -- 401
curl http://localhost:7148/api/data
```

```
{"error":"API key required"}
```

```
# Invalid key -- 403
curl http://localhost:7148/api/data -H "X-API-Key: wrong-key"
```

```
{"error":"Invalid API key"}
```

```
# Valid key -- 200
curl http://localhost:7148/api/data -H "X-API-Key: key-abc-123"
```

```
{"data":[1,2,3]}
```

---

11. Modifying Request and Response

Middleware can modify the request before it reaches the handler, and the response before it reaches the client.

Adding Data to the Request

```
function injectUserAgent(req, res, next) {
    const ua = req.headers["user-agent"] ?? "";

    (req as any).isMobile = ua.includes("Mobile") || ua.includes("Android") || ua.includes("iPho
    (req as any).isBot = ua.toLowerCase().includes("bot") || ua.toLowerCase().includes("spider")

    next();
}
```

Now the route handler can access `req.isMobile` and `req.isBot`:

```
Router.get("/api/content", async (req, res) => {
    const isMobile = (req as any).isMobile;
```

```
    if (isMobile) {
        return res.json({ layout: "compact", images: "low-res" });
    }

    return res.json({ layout: "full", images: "high-res" });
}, [injectUserAgent]);
```

Modifying the Response

```
function addSecurityHeaders(req, res, next) {
    next();

    // Add security headers to every response after the handler runs
    res.header("X-Content-Type-Options", "nosniff");
    res.header("X-Frame-Options", "DENY");
    res.header("X-XSS-Protection", "1; mode=block");
    res.header("Strict-Transport-Security", "max-age=31536000; includeSubDomains");
}
```

Adding a Request ID

```
function addRequestId(req, res, next) {
    const { randomUUID } = require("crypto");
    (req as any).requestId = randomUUID();
    res.header("X-Request-Id", (req as any).requestId);
    next();
}
```

The request ID follows the request through every layer. Log it in your handler, return it in error responses, and use it to trace issues across services.

---

12. Real-World Example: JWT Authentication Middleware

This class-based middleware verifies JWT tokens on protected routes. It uses the `before*` / `after*` convention.

```
import { Auth } from "tina4-nodejs";

class JwtAuthMiddleware {
    static beforeVerifyToken(req, res) {
        const authHeader = req.headers["authorization"] ?? "";

        if (!authHeader || !authHeader.startsWith("Bearer ")) {
            res(JSON.stringify({ error: "Authorization header required" }), 401);
            return [req, res];
        }

        const token = authHeader.slice(7);
        const secret = process.env.TINA4_SECRET || "tina4-default-secret";
        const payload = Auth.validToken(token, secret);

        if (!payload) {
            res(JSON.stringify({ error: "Invalid or expired token" }), 401);
            return [req, res];
        }

        // Attach the decoded payload to the request
```

```
        (req as any).user = payload;
        return [req, res];
    }
}
```

Apply it to a group of protected routes:

```
import { Router } from "tina4-nodejs";

Router.group("/api/protected", (group) => {
    group.get("/profile", async (req, res) => {
        return res.json({ user: (req as any).user });
    });

    group.post("/settings", async (req, res) => {
        const userId = (req as any).user.sub;
        return res.json({ updated: true, user_id: userId });
    });
}, [JwtAuthMiddleware]);
```

The middleware short-circuits with 401 if the token is missing or invalid. The route handler never runs. If the token is valid, the decoded payload is available as `req.user`.

---

13. Real-World Example: API Key Middleware with Database Lookup

```
import { Database } from "tina4-nodejs";

async function apiKeyMiddleware(req, res, next) {
    const apiKey = req.headers["x-api-key"] ?? "";

    if (!apiKey) {
        res({
            error: "API key required. Send it in the X-API-Key header."
        }, 401);
        return;
    }

    const db = new Database();
    const keyRecord = await db.fetchOne(
        "SELECT id, name, rate_limit, is_active FROM api_keys WHERE key_value = ?",
        [apiKey]
    );

    if (!keyRecord) {
        res({ error: "Invalid API key" }, 403);
        return;
    }

    if (!keyRecord.is_active) {
        res({ error: "API key has been deactivated" }, 403);
        return;
    }

    // Update last used timestamp
    await db.execute(
        "UPDATE api_keys SET last_used_at = ?, request_count = request_count + 1 WHERE id = ?",
        [new Date().toISOString(), keyRecord.id]
    );

    // Attach key info to request
    (req as any).apiKeyId = keyRecord.id;
    (req as any).apiKeyName = keyRecord.name;

    next();
}
```

The middleware checks the database on every request. Tina4's `Database` class uses connection pooling internally, so creating a new instance per request is safe. For high-traffic APIs, cache your key lookups in memory with a TTL instead of querying on every request.

---

14. Real-World Middleware Stack

```
import { Router } from "tina4-nodejs";

Router.group("/api/v1", (group) => {
    group.get("/products", async (req, res) => {
        return res.json({ products: [
            { id: 1, name: "Widget", price: 9.99 },
            { id: 2, name: "Gadget", price: 19.99 }
        ]});
    });

    group.post("/products", async (req, res) => {
        return res.status(201).json({
            id: 3,
            name: req.body.name ?? "Unknown",
            price: parseFloat(req.body.price ?? 0)
        });
    });
}, [addRequestId, logMiddleware, cors(), requireApiKey]);
```

Four middleware layers. Each does one job. The request ID comes first so every log line and error response includes it. The logger records the request. CORS handles browser preflight. The API key check gates access. The route handler never sees any of that work.

---

15. Exercise: Build an API Key Middleware System

Build a complete API key system with key management and usage tracking.

Requirements

- Create a migration for an `api_keys` table: `id`, `name`, `key_value` (unique), `is_active` (default true), `rate_limit` (default 100), `request_count` (default 0), `last_used_at`, `created_at`
- Build these endpoints:

Method	Path	Middleware	Description
`POST`	`/admin/api-keys`	Auth	Create a new API key (generate random key)
`GET`	`/admin/api-keys`	Auth	List all API keys with usage stats
`DELETE`	`/admin/api-keys/:id`	Auth	Deactivate an API key
`GET`	`/api/data`	API Key	Protected endpoint -- requires valid API key

`GET`	`/api/status`	API Key	Another protected endpoint

- The API key middleware should:
- Check `X-API-Key` header
- Validate against the database
- Reject deactivated keys
- Track usage (increment count, update last*used*at)
- Attach key info to the request

Test with:

```
# Create an API key (requires auth token from Chapter 8)
curl -X POST http://localhost:7148/admin/api-keys \
  -H "Authorization: Bearer YOUR_AUTH_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"name": "Mobile App"}'

# Use the API key
curl http://localhost:7148/api/data \
  -H "X-API-Key: THE_GENERATED_KEY"

# List keys with stats
curl http://localhost:7148/admin/api-keys \
  -H "Authorization: Bearer YOUR_AUTH_TOKEN"
```

---

16. Solution

Migration

Create `migrations/20260322170000_create_api_keys_table.sql`:

```
-- UP
CREATE TABLE api_keys (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    key_value TEXT NOT NULL UNIQUE,
    is_active INTEGER NOT NULL DEFAULT 1,
    rate_limit INTEGER NOT NULL DEFAULT 100,
    request_count INTEGER NOT NULL DEFAULT 0,
    last_used_at TEXT,
    created_at TEXT DEFAULT CURRENT_TIMESTAMP
);

CREATE UNIQUE INDEX idx_api_keys_key ON api_keys (key_value);

-- DOWN
DROP INDEX IF EXISTS idx_api_keys_key;
DROP TABLE IF EXISTS api_keys;
```

Routes

Create `src/routes/apiKeys.ts`:

```
import { Router, Auth, Database } from "tina4-nodejs";
import { randomBytes } from "crypto";

async function authMiddleware(req, res, next) {
    const authHeader = req.headers["authorization"] ?? "";
    if (!authHeader || !authHeader.startsWith("Bearer ")) {
        res({ error: "Authorization required" }, 401);
        return;
    }

    const token = authHeader.slice(7);
    const secret = process.env.TINA4_SECRET || "tina4-default-secret";
    const payload = Auth.validToken(token, secret);

    if (!payload) {
        res({ error: "Invalid or expired token" }, 401);
        return;
    }

    (req as any).user = payload;
    next();
}

async function apiKeyMiddleware(req, res, next) {
    const apiKey = req.headers["x-api-key"] ?? "";

    if (!apiKey) {
        res({ error: "API key required. Send in X-API-Key header." }, 401);
        return;
    }

    const db = new Database();
    const keyRecord = await db.fetchOne(
        "SELECT id, name, is_active FROM api_keys WHERE key_value = ?",
        [apiKey]
    );

    if (!keyRecord) {
        res({ error: "Invalid API key" }, 403);
        return;
    }

    if (!keyRecord.is_active) {
        res({ error: "API key has been deactivated" }, 403);
        return;
    }

    await db.execute(
        "UPDATE api_keys SET last_used_at = ?, request_count = request_count + 1 WHERE id = ?",
        [new Date().toISOString(), keyRecord.id]
    );

    (req as any).apiKeyId = keyRecord.id;
    (req as any).apiKeyName = keyRecord.name;

    next();
```

```
}

// Admin routes -- require auth token
Router.group("/admin", (group) => {
    group.post("/api-keys", async (req, res) => {
        const db = new Database();
        const name = req.body?.name ?? "Unnamed Key";
        const keyValue = `tk_${randomBytes(24).toString("hex")}`;

        await db.execute(
            "INSERT INTO api_keys (name, key_value) VALUES (?, ?)",
            [name, keyValue]
        );

        const key = await db.fetchOne(
            "SELECT * FROM api_keys WHERE key_value = ?",
            [keyValue]
        );

        return res.status(201).json({ message: "API key created", key });
    });

    group.get("/api-keys", async (req, res) => {
        const db = new Database();
        const keys = await db.fetch(
            "SELECT id, name, key_value, is_active, rate_limit, request_count, last_used_at, cre
        );
        return res.json({ keys, count: keys.length });
    });

    group.delete("/api-keys/:id", async (req, res) => {
        const db = new Database();
        const keyId = req.params.id;

        const existing = await db.fetchOne(
            "SELECT id FROM api_keys WHERE id = ?",
            [keyId]
        );

        if (!existing) {
            return res.status(404).json({ error: "API key not found" });
        }

        await db.execute(
            "UPDATE api_keys SET is_active = 0 WHERE id = ?",
            [keyId]
        );

        return res.json({ message: "API key deactivated" });
    });
}, [authMiddleware]);

// Public API routes -- require API key
Router.group("/api", (group) => {
    group.get("/data", async (req, res) => {
        return res.json({
            data: [1, 2, 3, 4, 5],
            api_key: (req as any).apiKeyName
        });
```

```
    });

    group.get("/status", async (req, res) => {
        return res.json({
            status: "operational",
            api_key: (req as any).apiKeyName
        });
    });
}, [apiKeyMiddleware]);
```

---

17. Gotchas

1. Forgetting to Call next()

Problem: The route handler never runs. The request hangs or times out.

Cause: You forgot to call `next()` after your middleware logic. Without `next()`, the chain stops and the handler never executes.

Fix: Always call `next()` to continue the chain. The `next()` function takes no arguments. If you intend to block the request, return a response instead of leaving the request hanging.

2. Middleware Modifies Response After It Is Sent

Problem: Headers or cookies you add in the "after" phase of middleware do not appear in the response.

Cause: The response was already finalized by the route handler. In Node.js, once the response body is sent, headers cannot be modified.

Fix: In the "after" phase, modify the result before it reaches the client. Some modifications need to happen in the "before" phase instead. For class-based middleware, the `after*` methods run before the response is finalized, so they can still add headers.

3. Middleware Applied to Wrong Routes

Problem: Your API key middleware runs on public routes that should not require a key.

Cause: The middleware is applied to the group, and the public route is inside that group.

Fix: Move public routes outside the group. Or create two groups -- one for public endpoints with no auth middleware, one for protected endpoints with it. For custom middleware, you can also check the path inside the middleware and skip the check for specific routes.

4. Middleware Execution Order Surprises

Problem: Your auth check runs after your logging middleware, but you wanted it to run first.

Cause: Middleware in the array `[a, b, c]` runs in left-to-right order: `a` wraps `b` wraps `c` wraps handler.

Fix: Put the middleware you want to run first at the leftmost position: `[authMiddleware, logMiddleware]`.

5. Error in Middleware Breaks the Chain

Problem: An unhandled exception in middleware causes a 500 error without reaching the route handler.

Cause: If middleware throws an exception before calling `next()`, no subsequent middleware or the handler runs.

Fix: Wrap risky middleware code in try/catch and return an appropriate error response instead of letting the exception propagate:

```
function safeMiddleware(req, res, next) {
    try {
        // risky operation
        const result = somethingThatMightFail();
        (req as any).result = result;
        next();
    } catch (err) {
        res({ error: "Internal middleware error" }, 500);
    }
}
```

6. Database Connections in Middleware

Problem: Opening a database connection in middleware that runs on every request causes connection pool exhaustion.

Cause: Each middleware call creates a new `Database()` instance.

Fix: Tina4's `Database` class uses connection pooling internally, so this is usually safe. But if you see issues under high traffic, cache your database lookups (like API keys) in memory with a TTL instead of querying on every request.

7. Modifying Request in Middleware Does Not Persist

Problem: You set `req.customField = "value"` in middleware, but the route handler does not see it.

Cause: TypeScript's type system does not know about your custom property. In some cases, the request object may be copied between middleware stages.

Fix: Use type assertion `(req as any).customField` consistently. If the property is not persisting, check that you are modifying the same request object that is passed to `next()`. Do not create a new request object.

8. CORS Preflight Returns 404

Problem: The browser sends an `OPTIONS` request and gets a 404, blocking the actual request.

Cause: No middleware handles the preflight `OPTIONS` request for that route.

Fix: Apply `CorsMiddleware` to the group or globally with `Router.use()`. It handles `OPTIONS` on its own. Make sure CORS middleware is registered before the rate limiter so preflight requests do not count against the rate limit.

9. Rate Limiter Counts Preflight Requests

Problem: Browsers burn through the rate limit with `OPTIONS` requests before making actual API calls.

Cause: The rate limiter runs before the CORS middleware, so it counts preflight requests.

Fix: Put `CorsMiddleware` before `RateLimiterMiddleware` in the registration order. CORS handles the preflight and returns early. The rate limiter only sees real requests.

10. Middleware File Not Auto-Loaded

Problem: You defined middleware in a file, but Tina4 does not recognize it.

Cause: The file is not inside the auto-loaded directory.

Fix: Put middleware files inside `src/routes/`. Tina4 auto-loads all files in that directory. If you place middleware in a separate folder, import it in a file inside `src/routes/`.

11. Short-Circuiting Skips Cleanup

Problem: Your timing middleware never logs the "after" phase because an earlier middleware short-circuited the request.

Cause: When middleware returns a response without calling `next()`, downstream middleware never runs -- including the "after" hooks of outer middleware.

Fix: Put timing and logging middleware at the outermost layer. They wrap everything else, so their "after" code runs regardless of what happens inside.

12. Middleware Must Be Passed as Function References

Problem: You passed a middleware name as a string and nothing happened.

Cause: The middleware array expects function or class references, not strings.

Fix: Pass middleware as references in an array: `[myMiddleware]`, not `["myMiddleware"]`. If you use a factory function like `cors()`, call it to get the actual middleware function.

---

18. Security

Every route you write is a door. Chapter 8 gave you locks. Chapter 9 gave you session keys. The first half of this chapter gave you guards. This section ties them together into a defence that works without thinking about it.

Tina4 ships secure by default. POST routes require authentication. CSRF tokens protect forms once you turn them on. Security headers harden every response. The framework does the boring security work so you focus on building features. But you need to understand what it does (and why) so you don't accidentally undo it.

---

1. Secure-by-Default Routing

Every POST, PUT, PATCH, and DELETE route requires a valid `Authorization: Bearer` token. No configuration needed. No export to remember. The framework enforces this before your handler runs.

```
// src/routes/api/orders/post.ts
import type { Tina4Request, Tina4Response } from "tina4-nodejs";

export default async function (req: Tina4Request, res: Tina4Response) {
    // This handler ONLY runs if the request carries a valid Bearer token.
    // Without one, the framework returns 401 before your code executes.
    return res.status(201).json({ created: true });
}
```

Test it without a token:

```
curl -X POST http://localhost:7148/api/orders \
  -H "Content-Type: application/json" \
  -d '{"product": "widget"}'
# 401 Unauthorized
```

Test it with a valid token:

```
curl -X POST http://localhost:7148/api/orders \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer eyJhbGciOiJIUzI1NiJ9..." \
  -d '{"product": "widget"}'
# 201 Created
```

GET routes are public by default. Anyone can read. Writing requires proof of identity.

Making a Write Route Public

Some endpoints need to accept unauthenticated writes: webhooks, registration forms, public contact forms. Export `meta` with `noAuth: true`:

```
// src/routes/api/webhooks/stripe/post.ts
import type { Tina4Request, Tina4Response } from "tina4-nodejs";

export const meta = { noAuth: true };

export default async function (req: Tina4Request, res: Tina4Response) {
    // No token required. Stripe can POST here freely.
    return res.json({ received: true });
}
```

Protecting a GET Route

Admin dashboards, user profiles, account settings: some pages need protection even though they only read data. The framework only enforces the Bearer-token gate on routes whose `secure` flag is set, and that flag is set by registering the route programmatically and chaining `.secure()` (there is no `meta`/`secured` export that file-discovery reads):

```
// src/routes/admin.ts
import { get } from "tina4-nodejs";

// .secure() flips the framework's auth gate on for this GET route.
get("/api/admin/users", async (req, res) => {
```

```
    // Requires a valid Bearer token, even though it's a GET.
    return res.json({ users: [] });
}).secure();
```

For a file-discovered `get.ts` route that has no `.secure()` hook, verify the token inside the handler instead:

```
// src/routes/api/admin/users/get.ts
import { validToken } from "tina4-nodejs";
import type { Tina4Request, Tina4Response } from "tina4-nodejs";

export default async function (req: Tina4Request, res: Tina4Response) {
    const auth = req.headers.authorization ?? "";
    const token = auth.startsWith("Bearer ") ? auth.slice(7) : "";
    if (!validToken(token)) {
        return res.status(401).json({ error: "Authentication required" });
    }
    return res.json({ users: [] });
}
```

The Rule

Method	Default	Override
GET, HEAD, OPTIONS	Public	chain `.secure()` on a programmatic route (or check the token in-handler)
POST, PUT, PATCH, DELETE	Auth required	chain `.noAuth()` on a programmatic route to open

Write routes are secured by default; reads are public unless you opt in.

---

2. CSRF Protection

Cross-Site Request Forgery tricks a user's browser into submitting a form to your server. The browser sends cookies automatically, including session cookies. Without CSRF protection, an attacker's page can submit forms as your logged-in user.

Tina4 blocks this with form tokens.

How It Works

- Your template renders a hidden token using `{{ form_token() }}`.
- The browser submits the token with the form data.
- The `CsrfMiddleware` validates the token before the route handler runs.
- Invalid or missing tokens receive a `403 Forbidden` response.

The Template

```
<form method="POST" action="/api/profile">
    {{ form_token() }}
    <input type="text" name="name" placeholder="Your name">
    <button type="submit">Save</button>
</form>
```

The `{{ form_token() }}` call generates a hidden input field containing a signed JWT. The token is bound to the current session; a token from one session cannot be used in another.

The Middleware

CSRF protection is off by default. Set `TINA4_CSRF=true` to turn it on; from then on every POST, PUT, PATCH, and DELETE request must include a valid form token. The middleware checks two places:

- **Request body** - `req.body.formToken`
- **Request header** - `X-Form-Token`

If the token is missing or invalid, the middleware returns 403 before your handler runs.

AJAX Requests

For JavaScript-driven forms, send the token as a header:

```
// frond.min.js handles this automatically via saveForm()
// For manual AJAX, extract the token from the hidden field:
const token = document.querySelector('input[name="formToken"]').value;

fetch("/api/profile", {
    method: "POST",
    headers: {
        "Content-Type": "application/json",
        "X-Form-Token": token
    },
    body: JSON.stringify({ name: "Alice" })
});
```

Tokens in Query Strings: Blocked

Tokens must never appear in URLs. Query strings are logged in server access logs, browser history, and referrer headers. A token in the URL is a token anyone can steal.

Tina4 rejects any request that carries `formToken` in the query string and logs a warning:

```
CSRF token found in query string - rejected for security.
Use POST body or X-Form-Token header instead.
```

Skipping CSRF Validation

Three scenarios skip CSRF checks automatically:

- **GET, HEAD, OPTIONS** - Safe methods don't modify state.
- **Routes with** `noAuth: true` - Public write endpoints don't need CSRF (they have no session to protect).

• **Requests with a valid Bearer token** - API clients authenticate with tokens, not cookies. CSRF only matters for cookie-based sessions.

Turning CSRF On

CSRF stays off until you turn it on. Turn it on for anything a browser can reach:

```
TINA4_CSRF=true
```

The cost is one hidden field per form. The protection is worth it. `TINA4_CSRF=false` turns it off again. Leave it off only for internal microservices behind a firewall, where no browser ever touches the API.

---

3. Session-Bound Tokens

A form token alone prevents cross-site forgery. But what if someone steals a token from a form? Session binding stops them.

When `{{ form_token() }}` generates a token, it embeds the current session ID in the JWT payload. The CSRF middleware checks that the session ID in the token matches the session ID of the request. A token stolen from one session cannot be replayed in another.

This happens automatically. No configuration. No extra code.

How Stolen Tokens Fail

• Attacker visits your site, gets a form token for session `abc-123`.

• Attacker sends that token from their own session `xyz-789`.

• The middleware compares: `abc-123 != xyz-789`, rejected with 403.

The token is cryptographically valid. But it belongs to the wrong session. The binding catches it.

---

4. Security Headers

Every response from Tina4 carries security headers. The `SecurityHeadersMiddleware` injects them before the response reaches the browser. No opt-in required.

Header	Default Value	Purpose
`X-Frame-Options`	`SAMEORIGIN`	Prevents clickjacking - your pages cannot be embedded in iframes on other domains.
`X-Content-Type-Options`	`nosniff`	Stops browsers from guessing content types. A script is a script, not HTML.
`Content-Security-Policy`	`default-src 'self'`	Controls which resources the browser loads. Blocks inline scripts from injected HTML.

`Referrer-Policy`	`strict-origin-when-cross-origin`	Limits referrer data sent to external sites. Protects internal URLs from leaking.
`X-XSS-Protection`	`0`	Disabled. Modern CSP replaces this legacy header. Keeping it on can introduce vulnerabilities.
`Permissions-Policy`	`camera=(), microphone=(), geolocation=()`	Disables browser APIs your app does not need.

HSTS: Enforcing HTTPS

Strict Transport Security tells the browser to always use HTTPS. Disabled by default (it breaks local development on HTTP). Enable it in production:

```
TINA4_HSTS=31536000
```

This sets a one-year HSTS policy with `includeSubDomains`. Once a browser sees this header, it refuses to connect over HTTP, even if the user types `http://`.

Customising Headers

Override any header via environment variables:

```
TINA4_FRAME_OPTIONS=DENY
TINA4_CSP=default-src 'self'; script-src 'self' https://cdn.example.com
TINA4_REFERRER_POLICY=no-referrer
TINA4_PERMISSIONS_POLICY=camera=(), microphone=(), geolocation=(), payment=()
```

---

5. SameSite Cookies

Session cookies control who can send them. The `SameSite` attribute tells the browser when to include the cookie in requests.

Value	Behaviour
`Strict`	Cookie sent only on same-site requests. Even clicking a link from email to your site won't include the cookie. The user must navigate directly.
`Lax`	Cookie sent on same-site requests and top-level navigations (clicking a link). Not sent on cross-site AJAX or form POSTs from other domains.

`None`	Cookie sent on all requests, including cross-site. Requires `Secure` flag (HTTPS only).

Tina4 defaults to `Lax`. This blocks cross-site form submissions (CSRF) while allowing normal link navigation. Users click a link to your site from an email, and they stay logged in. An attacker's page submits a hidden form, and the cookie is withheld.

For most applications, `Lax` is the right choice. Change it only if you understand the trade-offs.

---

6. Login Flow: Complete Example

Authentication, sessions, tokens, and security converge in the login flow. Here is a complete implementation.

The Login Route

```
// src/routes/api/login/post.ts
import type { Tina4Request, Tina4Response } from "tina4-nodejs";
import { getToken, checkPassword } from "tina4-nodejs";

export const meta = { noAuth: true };

export default async function (req: Tina4Request, res: Tina4Response) {
    const email: string = req.body.email || "";
    const password: string = req.body.password || "";

    if (!email || !password) {
        return res.status(400).json({ error: "Email and password required" });
    }

    // Look up user (replace with your database query)
    const user = await db.fetchOne(
        "SELECT id, email, password_hash, role FROM users WHERE email = ?",
        [email]
    );

    if (!user) {
        return res.status(401).json({ error: "Invalid credentials" });
    }

    // Verify password
    if (!checkPassword(password, user.password_hash)) {
        return res.status(401).json({ error: "Invalid credentials" });
    }

    // Generate token with user claims
    const secret = process.env.TINA4_SECRET || "tina4-default-secret";
    const token: string = getToken({
        sub: user.id,
        email: user.email,
        role: user.role,
    }, secret);

    // Store user in session
    req.session.set("user_id", user.id);
    req.session.set("email", user.email);
    req.session.set("role", user.role);

    return res.json({
        token,
        user: { id: user.id, email: user.email },
    });
}
```

The `meta.noAuth` flag opens this route to unauthenticated requests. The handler validates credentials and issues a token. The session stores the user identity for server-side lookups.

The Login Form

```
{% extends "base.twig" %}
{% block content %}
<div class="container mt-5" style="max-width: 400px;">
    <h2>Login</h2>
    <form id="loginForm">
        {{ form_token() }}
        <div class="mb-3">
            <label for="email">Email</label>
            <input type="email" name="email" id="email" class="form-control"
                   placeholder="you@example.com" required>
        </div>
        <div class="mb-3">
            <label for="password">Password</label>
            <input type="password" name="password" id="password" class="form-control"
                   placeholder="Your password" required>
        </div>
        <button type="button" class="btn btn-primary w-100"
                onclick="saveForm('loginForm', '/api/login', 'loginMsg', handleLogin)">
            Sign In
        </button>
        <div id="loginMsg" class="mt-3"></div>
    </form>
</div>

<script>
function handleLogin(result) {
    if (result.token) {
        localStorage.setItem("token", result.token);
        window.location.href = "/dashboard";
    }
}
</script>
{% endblock %}
```

Protected Pages: Checking the Session

```
// src/routes/dashboard/get.ts
import type { Tina4Request, Tina4Response } from "tina4-nodejs";

export default async function (req: Tina4Request, res: Tina4Response) {
    const userId: string | undefined = req.session.get("user_id");

    if (!userId) {
        return res.redirect("/login");
    }

    return res.render("dashboard.twig", {
        email: req.session.get("email"),
        role: req.session.get("role"),
    });
}
```

Logout: Destroying the Session

```
// src/routes/api/logout/post.ts
import type { Tina4Request, Tina4Response } from "tina4-nodejs";

export const meta = { noAuth: true };

export default async function (req: Tina4Request, res: Tina4Response) {
    req.session.destroy();
    return res.json({ logged_out: true });
}
```

---

7. Handling Expired Sessions

Sessions expire. Tokens expire. The user clicks a link and finds themselves staring at a broken page or a cryptic error. A good security implementation handles expiry gracefully.

The Pattern: Redirect to Login, Then Back

When a session expires mid-use, the user should:

- See a login page, not an error.
- Log in again.
- Land on the page they were trying to reach, not the home page.

```
// src/routes/account/settings/get.ts
import type { Tina4Request, Tina4Response } from "tina4-nodejs";

export default async function (req: Tina4Request, res: Tina4Response) {
    const userId: string | undefined = req.session.get("user_id");

    if (!userId) {
        // Remember where they wanted to go
        const returnUrl: string = encodeURIComponent(req.url);
        return res.redirect(`/login?redirect=${returnUrl}`);
    }

    return res.render("settings.twig", { user_id: userId });
}
```

The login handler reads the `redirect` parameter after successful authentication:

```
// src/routes/api/login/post.ts
import type { Tina4Request, Tina4Response } from "tina4-nodejs";
import { getToken, checkPassword } from "tina4-nodejs";

export const meta = { noAuth: true };

export default async function (req: Tina4Request, res: Tina4Response) {
    // ... validate credentials ...

    const redirectUrl: string = req.params.redirect || "/dashboard";

    return res.json({
        token,
        redirect: redirectUrl,
    });
```

```
}
```

The login form JavaScript redirects to the saved URL:

```
function handleLogin(result) {
    if (result.token) {
        localStorage.setItem("token", result.token);
        window.location.href = result.redirect || "/dashboard";
    }
}
```

Token Refresh

Tokens expire based on `TINA4_TOKEN_LIMIT` (default: 60 minutes). The `frond.min.js` frontend library handles token refresh automatically; every response includes a `FreshToken` header with a new token. The client stores it and uses it for the next request.

For custom AJAX code, read the header yourself:

```
const res = await fetch("/api/data", {
    headers: { "Authorization": "Bearer " + localStorage.getItem("token") }
});

const freshToken = res.headers.get("FreshToken");
if (freshToken) {
    localStorage.setItem("token", freshToken);
}
```

---

8. Rate Limiting

Brute-force login attempts. Credential stuffing. API abuse. Rate limiting stops all of them.

Tina4 includes a sliding-window rate limiter that tracks requests per IP address. It activates automatically.

```
TINA4_RATE_LIMIT=100
TINA4_RATE_WINDOW=60
```

One hundred requests per sixty seconds per IP. Exceed the limit and the server returns `429 Too Many Requests` with headers telling the client when to retry:

```
X-RateLimit-Limit: 100
X-RateLimit-Remaining: 0
X-RateLimit-Reset: 45
```

For login routes, consider a stricter limit:

```
// src/routes/api/login/post.ts
import type { Tina4Request, Tina4Response } from "tina4-nodejs";
import { getToken, checkPassword } from "tina4-nodejs";

export const meta = { noAuth: true };

// Simple in-memory rate limiter for login
const loginAttempts: Map<string, { count: number; resetAt: number }> = new Map();

export default async function (req: Tina4Request, res: Tina4Response) {
    const ip: string = req.ip;
```

```
    const now: number = Date.now();
    const windowMs: number = 60_000; // 60 seconds
    const maxAttempts: number = 5;

    // Check rate limit
    const entry = loginAttempts.get(ip);
    if (entry && now < entry.resetAt) {
        if (entry.count >= maxAttempts) {
            const retryAfter: number = Math.ceil((entry.resetAt - now) / 1000);
            return res.status(429).json({
                error: "Too many login attempts",
                retry_after: retryAfter,
            });
        }
        entry.count++;
    } else {
        loginAttempts.set(ip, { count: 1, resetAt: now + windowMs });
    }

    // ... login logic ...
}
```

---

9. CORS and Credentials

When your frontend runs on a different origin than your API (common in development), CORS controls whether the browser sends cookies and auth headers.

Tina4 handles CORS automatically. The relevant security settings:

```
TINA4_CORS_ORIGINS=*
TINA4_CORS_CREDENTIALS=true
```

Two rules to remember:

- `TINA4_CORS_ORIGINS=*` **with** `TINA4_CORS_CREDENTIALS=true` is invalid per the CORS spec. Tina4 handles this: when origin is `*`, the credentials header is not sent. But in production, list your actual origins.
- **Cookies need** `SameSite=None; Secure` for true cross-origin requests. If your API is on `api.example.com` and your frontend is on `app.example.com`, the default `Lax` cookie works because they share the same registrable domain. Different domains need `SameSite=None`.

Production CORS:

```
TINA4_CORS_ORIGINS=https://app.example.com,https://admin.example.com
TINA4_CORS_CREDENTIALS=true
```

---

10. Security Checklist

Before you deploy, verify:

- [ ] `TINA4_SECRET` is set to a long, random string, not the default.
- [ ] `TINA4_DEBUG=false` in production.

- [ ] `TINA4_HSTS=31536000` if serving over HTTPS.
- [ ] `TINA4_CORS_ORIGINS` lists your actual domains, not `*`.
- [ ] `TINA4_CSRF=true` for any browser-facing application (CSRF is off by default).
- [ ] Login route uses `noAuth: true` and validates credentials before issuing tokens.
- [ ] Session is regenerated after login (prevents session fixation).
- [ ] Passwords are hashed with `hashPassword()`, never stored in plain text.
- [ ] File uploads are validated and size-limited (`TINA4_MAX_UPLOAD_SIZE`).
- [ ] Rate limiting is active on login and registration routes.
- [ ] Expired sessions redirect to login with a return URL.

Gotchas

1. "My POST route returns 401 but I didn't add auth"

Cause: Tina4 requires authentication on all write routes by default.

Fix: Export `meta` with `noAuth: true` if the endpoint should be public. Otherwise, send a valid Bearer token with the request.

2. "CSRF validation fails on AJAX requests"

Cause: The form token is not included in the request.

Fix: Send the token as an `X-Form-Token` header. If using `frond.min.js`, call `saveForm()`; it handles tokens automatically.

3. "I disabled CSRF but forms still fail"

Cause: The route still requires Bearer auth (separate from CSRF). CSRF and auth are independent checks.

Fix: Either send a Bearer token or export `meta` with `noAuth: true` on the route.

4. "My Content-Security-Policy blocks inline scripts"

Cause: The default CSP is `default-src 'self'`, which blocks inline `<script>` tags and `onclick` handlers.

Fix: Move scripts to external `.js` files (the right approach) or relax the CSP:

```
TINA4_CSP=default-src 'self'; script-src 'self' 'unsafe-inline'
```

Prefer external scripts. Inline scripts are an XSS vector.

5. "User stays logged in after session expires"

Cause: The frontend stores a JWT in localStorage. The token is still valid even after the session is destroyed server-side.

Fix: Check the session on every page load. If the session is gone, redirect to login regardless of the token. Tokens authenticate API calls; sessions track server-side state. Both must be valid.

Exercise: Secure Contact Form

Build a public contact form that:

- Does not require login (`noAuth: true`).
- Validates CSRF tokens (form includes `{{ form_token() }}`).
- Rate-limits submissions to 3 per minute per IP.
- Stores messages in the database.
- Returns a success message.

Solution

```
// src/routes/contact/get.ts
import type { Tina4Request, Tina4Response } from "tina4-nodejs";

export default async function (req: Tina4Request, res: Tina4Response) {
    return res.render("contact.twig", { title: "Contact Us" });
}
```

```
// src/routes/api/contact/post.ts
import type { Tina4Request, Tina4Response } from "tina4-nodejs";

export const meta = { noAuth: true };

// Simple in-memory rate limiter for contact submissions
const submissions: Map<string, { count: number; resetAt: number }> = new Map();

export default async function (req: Tina4Request, res: Tina4Response) {
    // Rate limit: 3 submissions per 60 seconds per IP
    const ip: string = req.ip;
    const now: number = Date.now();
    const windowMs: number = 60_000;
    const maxSubmissions: number = 3;

    const entry = submissions.get(ip);
    if (entry && now < entry.resetAt) {
        if (entry.count >= maxSubmissions) {
            return res.status(429).json({ error: "Too many submissions. Try again later." });
        }
        entry.count++;
    } else {
        submissions.set(ip, { count: 1, resetAt: now + windowMs });
    }

    const name: string = (req.body.name || "").trim();
    const email: string = (req.body.email || "").trim();
    const message: string = (req.body.message || "").trim();

    if (!name || !email || !message) {
        return res.status(400).json({ error: "All fields are required" });
    }
```

```
    await db.insert("contact_messages", {
        name,
        email,
        message,
    });
    await db.commit();

    return res.json({ success: true, message: "Thank you for your message" });
}
```

```
{# src/templates/contact.twig #}
{% extends "base.twig" %}
{% block title %}Contact Us{% endblock %}
{% block content %}
<div class="container mt-5" style="max-width: 500px;">
    <h2>{{ title }}</h2>
    <form id="contactForm">
        {{ form_token() }}
        <div class="mb-3">
            <label for="name">Name</label>
            <input type="text" name="name" id="name" class="form-control"
                   placeholder="Jane Smith" required>
        </div>
        <div class="mb-3">
            <label for="email">Email</label>
            <input type="email" name="email" id="email" class="form-control"
                   placeholder="jane@example.com" required>
        </div>
        <div class="mb-3">
            <label for="message">Message</label>
            <textarea name="message" id="message" class="form-control" rows="4"
                      placeholder="How can we help?" required></textarea>
        </div>
        <button type="button" class="btn btn-primary"
                onclick="saveForm('contactForm', '/api/contact', 'contactMsg')">
            Send Message
        </button>
        <div id="contactMsg" class="mt-3"></div>
    </form>
</div>
{% endblock %}
```

The form is public. The CSRF token is present. The `noAuth: true` export opens the route. The middleware validates the token. The database stores the message. The user sees confirmation.

Five moving parts. Zero security holes. The framework handles the rest.

Caching

1. From 800ms to 3ms

Your product catalog page runs 12 database queries. 800 milliseconds to render. Every visitor triggers the same queries, the same template rendering, the same JSON serialization -- for data that changes once a day.

Add caching. The first request takes 800ms. The next 10,000 take 3ms each. A 266x improvement from one line of configuration.

Caching stores the result of expensive operations for reuse. Tina4 gives you three layers, each with its own job:

- **Request-scoped query cache** -- on by default, dedupes identical database reads within a single request.
- **Persistent database cache** -- opt-in, caches query results across requests for a few seconds.
- **Response cache** -- caches whole HTTP responses so the route handler never runs.

On top of those, a direct key/value API (`cacheGet`/`cacheSet`/...) caches anything you want.

> ***Node specifics.*** *Method names are camelCase. The key/value API, the response-cache middleware, and the database read path are* ***async*** *-- `await` them. Only `db.cacheStats()` and `db.cacheClear()` are synchronous.*

---

2. Layer 1: Request-Scoped Query Cache (On by Default)

The database wrapper caches every `SELECT` it runs during a single HTTP request. Run the same query twice in one handler and the second call returns the cached rows -- no second trip to the database.

This layer is **on by default**. You do not configure it. At the start of every request the framework clears it, so one request never sees another request's rows. Any write (`insert`, `update`, `delete`, `execute`) flushes it immediately.

```
import { get } from "tina4-nodejs";
import { Database } from "tina4-nodejs/orm";

get("/api/report", async (req, res) => {
    const db = await Database.getConnection();

    // First call hits the database.
    const totals = await db.fetchAll("SELECT category, COUNT(*) AS n FROM products GROUP BY cate

    // Same query later in the SAME request - served from the request cache, no DB hit.
    const totalsAgain = await db.fetchAll("SELECT category, COUNT(*) AS n FROM products GROUP BY

    return res.json({ totals, matches: JSON.stringify(totals) === JSON.stringify(totalsAgain) })
});
```

Control it with two environment variables:

```
# On by default. Set to false to turn the request-scoped cache off.
TINA4_AUTO_CACHING=true

# Safety TTL in seconds for non-request contexts (scripts, workers). Default: 5
TINA4_AUTO_CACHING_TTL=5
```

Inside an HTTP request the cache clears at the request boundary, so the TTL rarely matters. It only kicks in for long-running scripts or workers that never cross a request boundary.

This layer always runs in-process. It is the fastest cache there is -- a plain in-memory map, no serialization, no network.

---

3. Layer 2: Persistent Database Cache (Opt-In)

The request cache forgets everything between requests. The persistent cache does not. Turn it on and identical queries return cached rows across requests, until the entry expires.

```
TINA4_DB_CACHE=true
TINA4_DB_CACHE_TTL=30
```

`TINA4_DB_CACHE_TTL` defaults to **30** seconds. With the persistent cache enabled, identical queries return cached results instead of hitting the database:

```
import { get } from "tina4-nodejs";
import { Database } from "tina4-nodejs/orm";

get("/api/categories", async (req, res) => {
    const db = await Database.getConnection();

    // First call across all requests: runs the query.
    // Repeat calls within 30 seconds: returns the cached rows.
    const categories = await db.fetchAll("SELECT * FROM categories ORDER BY name");

    return res.json({ categories });
});
```

The cache key comes from the SQL and its parameters. Different queries or different parameters get different keys:

```
// Cached separately:
await db.fetchAll("SELECT * FROM products WHERE category = ?", ["Electronics"]);
await db.fetchAll("SELECT * FROM products WHERE category = ?", ["Fitness"]);
```

A write on any connection flushes the persistent cache, so a stale row never outlives an update.

Choosing the backend

By default the persistent cache lives in-process (same as request caching, just longer-lived). Point it at a shared store to share cached rows across instances:

```
TINA4_DB_CACHE_BACKEND=redis        # memory (default) | file | redis | valkey | memcached | mon
TINA4_DB_CACHE_URL=redis://localhost:6379
```

With `memory` (the default), each process keeps its own copy. With a network backend, every instance reads and writes the same cache, and a write on one instance invalidates the rest.

When to use it

- Read-heavy apps where the same queries run over and over
- Reference data that changes seldom (categories, countries, settings)
- Dashboard queries that aggregate large datasets

When not to use it

- Write-heavy apps where data changes on every request
- Queries with real-time requirements (inventory counts, live prices)
- Queries that must return the latest data at all times

Skipping the cache for one query

Pass `{ noCache: true }` as the **trailing options argument** to bypass both cache layers for a single call. It is a separate argument -- it never replaces the params array:

```
// fetchAll(sql, params, limit, offset, opts)
const fresh = await db.fetchAll("SELECT * FROM products WHERE category = ?", ["Electronics"], 50

// fetch(sql, params, limit, offset, opts) - returns the full DatabaseResult
const result = await db.fetch("SELECT * FROM products", [], 50, 0, { noCache: true });

// fetchOne(sql, params, opts)
const one = await db.fetchOne("SELECT * FROM products WHERE id = ?", [42], { noCache: true });
```

A `noCache` read runs straight against the database and leaves the hit/miss counters untouched.

---

4. Layer 3: Response Cache

The fastest cache skips your handler entirely. The response-cache middleware stores the complete response -- body, content type, status code -- and serves it on the next matching GET request without calling your route handler at all.

There are two ways to attach it. Both work.

String form in the middleware list

Pass `"ResponseCache:300"` as a middleware spec. The number after the colon is the TTL in seconds:

```
import { get } from "tina4-nodejs";

get("/api/products", async (req, res) => {
    // This handler runs 12 database queries and takes 800ms.
    // With the response cache, it runs once every 5 minutes.
    console.log("Handler called - should only appear once every 5 minutes");

    const products = [
        { id: 1, name: "Wireless Keyboard", price: 79.99 },
```

```
        { id: 2, name: "USB-C Hub", price: 49.99 },
        { id: 3, name: "Monitor Stand", price: 129.99 }
    ];

    return res.json({ products, generatedAt: new Date().toISOString() });
}, ["ResponseCache:300"]);
```

The middleware list is the third argument -- an array. Use `"ResponseCache"` on its own for the default TTL (`TINA4_CACHE_TTL`, 60 seconds), or `"ResponseCache:300"` to set it.

Function form via `.middleware(...)`

For the same effect with an explicit config object, chain `responseCache(...)`:

```
import { get, responseCache } from "tina4-nodejs";

get("/api/products", listProducts).middleware(responseCache({ ttl: 300 }));
```

What happens during the TTL

```
curl http://localhost:7148/api/products
```

```
{
  "products": [
    {"id": 1, "name": "Wireless Keyboard", "price": 79.99},
    {"id": 2, "name": "USB-C Hub", "price": 49.99},
    {"id": 3, "name": "Monitor Stand", "price": 129.99}
  ],
  "generatedAt": "2026-03-22T14:30:00.000Z"
}
```

Call it again within 5 minutes:

```
curl http://localhost:7148/api/products
```

```
{
  "products": [
    {"id": 1, "name": "Wireless Keyboard", "price": 79.99},
    {"id": 2, "name": "USB-C Hub", "price": 49.99},
    {"id": 3, "name": "Monitor Stand", "price": 129.99}
  ],
  "generatedAt": "2026-03-22T14:30:00.000Z"
}
```

The `generatedAt` timestamp is identical. The handler did not run -- the response came from cache.

Cache headers

The middleware sets two headers on every response:

```
X-Cache: HIT
X-Cache-TTL: 300
```

- `X-Cache` is `HIT` when the response came from cache, `MISS` when the handler ran.
- `X-Cache-TTL` reports the configured cache lifetime in seconds.

Caching with query parameters

The cache key is the method plus the full URL, including the query string. `/api/products?page=1` and `/api/products?page=2` cache separately:

```
import { get } from "tina4-nodejs";

get("/api/products", async (req, res) => {
    const page = parseInt(req.query.page ?? "1", 10);

    return res.json({
        page,
        products: [],
        generatedAt: new Date().toISOString()
    });
}, ["ResponseCache:300"]);
```

```
curl "http://localhost:7148/api/products?page=1"  # MISS, stores page=1
curl "http://localhost:7148/api/products?page=2"  # MISS, stores page=2
curl "http://localhost:7148/api/products?page=1"  # HIT
```

What not to cache

Only GET responses are cached -- the middleware passes other methods straight through. Beyond that, do not put the response cache on:

- **User-specific endpoints**: `/api/profile` returns different data per user, but the key is just the URL, so the first user's response goes to everyone.
- **Real-time data**: stock prices, live scores, chat messages.
- **Authenticated endpoints**: unless the cache is scoped per user (use the key/value API with a user-specific key instead).

```
// GOOD: public, rarely changing data
get("/api/categories", async (req, res) => {
    return res.json({ categories: [] });
}, ["ResponseCache:3600"]);

// BAD: user-specific data - do NOT cache
get("/api/profile", async (req, res) => {
    return res.json(req.user);
});
```

---

5. Backends

All three layers (and the key/value API below) share one backend set. Pick it with `TINA4_CACHE_BACKEND`:

Backend	Notes
`memory`	Default. In-process, fastest, lost on restart.
`file`	JSON files on disk. Survives restarts, no extra service.

`redis`	Shared across instances, sub-millisecond, built-in expiry.
`valkey`	Redis wire protocol -- same behaviour as redis.
`memcached`	Shared, unauthenticated.
`mongodb`	TTL collection. Needs the optional `mongodb` driver.
`database`	A `tina4_cache` table in your existing database.

Memory (default)

```
# This is the default - you do not need to set it.
TINA4_CACHE_BACKEND=memory
```

No disk I/O, no network calls. It resets when the server restarts. Ideal for development and single-server deployments where losing the cache on restart is fine.

Redis (and Valkey)

For cache that survives restarts and is shared across instances behind a load balancer, use Redis:

```
TINA4_CACHE_BACKEND=redis
TINA4_CACHE_URL=redis://localhost:6379
```

Credentials live in the URL (`redis://user:pass@host`, `redis://:pass@host`) or in separate env vars when they are not embedded:

```
TINA4_CACHE_USERNAME=myuser
TINA4_CACHE_PASSWORD=mypassword
```

Valkey speaks the same protocol -- set `TINA4_CACHE_BACKEND=valkey` and the rest is identical.

Why Redis:

- Cache survives server restarts.
- Shared across instances behind a load balancer.
- Sub-millisecond reads and writes.
- Built-in key expiry -- TTL cleanup is automatic.
- One instance can serve sessions, cache, and queues.

File

If you want persistence but cannot run Redis, use file-based caching:

```
TINA4_CACHE_BACKEND=file
TINA4_CACHE_DIR=/path/to/cache/directory
```

Each entry is a file on disk. Slower than memory or Redis, but survives restarts with zero extra infrastructure. Reach for it when you need persistence on limited hosting, or when entries are large and you would rather not hold them in memory.

Graceful fallback

If the configured backend's driver is missing, or the service is unreachable, or the credentials are wrong, the cache logs a warning and falls back to the **file** backend -- a real, persistent cache, never a silent no-op. Your code does not change; only the storage backend does.

---

6. Direct Key/Value API

For custom caching logic, use the key/value functions. They are **async** -- `await` every call. A miss returns `undefined`.

```
import { cacheGet, cacheSet, cacheDelete, clearCache, cacheStats } from "@tina4/core";
```

cacheSet

```
import { cacheSet } from "@tina4/core";

// Cache a value for 300 seconds.
await cacheSet("product:42", {
    id: 42,
    name: "Wireless Keyboard",
    price: 79.99,
    inStock: true
}, 300);

// Cache a string.
await cacheSet("exchangeRate:USD_EUR", "0.92", 3600);

// No TTL - uses the default (TINA4_CACHE_TTL, 60 seconds).
await cacheSet("app:config", { theme: "dark", lang: "en" });
```

cacheGet

```
import { cacheGet, cacheSet } from "@tina4/core";

const product = await cacheGet("product:42");
// Returns the cached value, or undefined if not found or expired.

if (product === undefined) {
    // Cache miss - fetch from the database, then store it.
    const fresh = await fetchProductFromDatabase(42);
    await cacheSet("product:42", fresh, 300);
}
```

cacheDelete

```
import { cacheDelete } from "@tina4/core";

await cacheDelete("product:42");

// Delete several keys.
await cacheDelete("product:43");
await cacheDelete("product:44");
```

clearCache

```
import { clearCache } from "@tina4/core";

// Wipe every entry in the active backend.
await clearCache();
```

Cache-aside pattern

The most common pattern is cache-aside (lazy loading). Note the `=== undefined` miss check -- and that every cache call is `await`ed:

```
import { get, cacheGet, cacheSet } from "@tina4/core";
import { Database } from "tina4-nodejs/orm";

get("/api/products/{id}", async (req, res) => {
    const id = parseInt(req.params.id, 10);
    const cacheKey = `product:${id}`;

    // 1. Try the cache first - await it.
    const cached = await cacheGet(cacheKey);
    if (cached !== undefined) {
        return res.json({ ...(cached as object), source: "cache" });
    }

    // 2. Miss - fetch from the database.
    const db = await Database.getConnection();
    const product = await db.fetchOne(
        "SELECT id, name, category, price, in_stock FROM products WHERE id = ?",
        [id]
    );

    if (product === null) {
        return res.status(404).json({ error: "Product not found" });
    }

    // 3. Store for next time.
    await cacheSet(cacheKey, product, 600);  // 10 minutes

    return res.json({ ...product, source: "database" });
});
```

First call (cache miss):

```
{ "id": 42, "name": "Wireless Keyboard", "category": "Electronics", "price": 79.99, "in_stock":
```

Second call (cache hit):

```
{ "id": 42, "name": "Wireless Keyboard", "category": "Electronics", "price": 79.99, "in_stock":
```

---

7. Cache Invalidation Strategies

Cache invalidation is the hard problem. Stale cache serves outdated data. Premature invalidation throws away performance gains. Three strategies handle this.

Strategy 1: Time-Based Expiry (TTL)

The simplest strategy. Set a TTL and let the cache expire on its own:

```
await cacheSet("products:featured", featuredProducts, 600);  // Expires in 10 minutes
```

Good for data where near-real-time accuracy is acceptable. A 10-minute delay in updating the featured products list is fine for most storefronts.

Strategy 2: Event-Based Invalidation

Clear the cache when the underlying data changes:

```
import { put, cacheDelete } from "@tina4/core";
import { Database } from "tina4-nodejs/orm";

put("/api/products/{id}", async (req, res) => {
    const id = parseInt(req.params.id, 10);
    const body = req.body;
    const db = await Database.getConnection();

    await db.execute(
        "UPDATE products SET name = ?, price = ? WHERE id = ?",
        [body.name, body.price, id]
    );

    // Invalidate the cache for this product.
    await cacheDelete(`product:${id}`);

    // Also invalidate any list caches that might include it.
    await cacheDelete("products:all");
    await cacheDelete("products:featured");

    const updated = await db.fetchOne("SELECT * FROM products WHERE id = ?", [id]);
    return res.json(updated);
});
```

The most accurate strategy -- the cache is fresh after every write. The downside: you must remember to invalidate every key that holds the affected data.

Strategy 3: Write-Through Cache

Update the cache at the same time as the database:

```
import { put, cacheSet } from "@tina4/core";
import { Database } from "tina4-nodejs/orm";

put("/api/products/{id}", async (req, res) => {
    const id = parseInt(req.params.id, 10);
    const body = req.body;
    const db = await Database.getConnection();
```

```
    await db.execute(
        "UPDATE products SET name = ?, price = ? WHERE id = ?",
        [body.name, body.price, id]
    );

    const updated = await db.fetchOne("SELECT * FROM products WHERE id = ?", [id]);

    // Write the new data to cache instead of deleting.
    await cacheSet(`product:${id}`, updated, 600);

    return res.json(updated);
});
```

The cache holds the latest data at all times. No cache miss after an update -- the next read comes from the already-warm cache.

Choosing a strategy

Strategy	Best For	Tradeoff
TTL	Read-heavy, tolerates staleness	Stale data for up to TTL duration
Event-based	Consistency matters	Every write must invalidate
Write-through	High traffic, frequent updates	Every write does cache work

Combine them. Use TTL as a safety net (entries expire even if invalidation misses). Use event-based invalidation for immediate consistency on critical data.

---

8. TTL Management

Choosing the right TTL depends on how often the data changes and how acceptable stale data is:

Data Type	Suggested TTL	Reasoning
Static config (categories, countries)	3600 (1 hour)	Changes rarely, stale data is harmless
Product catalog	300 (5 min)	Updates several times per day
User profile	60 (1 min)	Users expect changes to appear fast
Search results	120 (2 min)	Balance between freshness and performance
Dashboard stats	30 (30 sec)	Near-real-time but expensive to compute

Exchange rates	60 (1 min)	Updates often, slight delay is acceptable
Shopping cart	0 (no cache)	Must reflect current state at all times

Dynamic TTL

Adjust the TTL based on the data:

```
import { cacheGet, cacheSet } from "@tina4/core";

async function getCachedProduct(productId: number) {
    const cacheKey = `product:${productId}`;

    const cached = await cacheGet(cacheKey);
    if (cached !== undefined) {
        return cached;
    }

    const product = await fetchProductFromDatabase(productId);

    // Popular products: shorter TTL (more likely to change).
    // Inactive products: longer TTL (rarely change).
    const ttl = product.viewCount > 1000 ? 60 : 3600;
    await cacheSet(cacheKey, product, ttl);

    return product;
}
```

---

9. Cache Statistics

Two `cacheStats` calls report on two different things. Get the async/await right -- they differ.

Key/value backend stats (async)

`cacheStats()` from `@tina4/core` reports the backend behind the key/value API, the response cache, and the persistent DB cache. It is **async**:

```
import { get, cacheStats } from "@tina4/core";

get("/api/cache/stats", async (req, res) => {
    const stats = await cacheStats();
    return res.json(stats);
});
```

```
curl http://localhost:7148/api/cache/stats
```

```
{
  "hits": 15234,
  "misses": 891,
  "size": 42,
  "backend": "memory"
}
```

The shape is exactly `{ hits, misses, size, backend }` -- four fields, nothing more.

Database query-cache stats (sync)

`db.cacheStats()` reports the database query cache (layers 1 and 2). It is **synchronous** -- no `await`:

```
import { Database } from "tina4-nodejs/orm";

const db = await Database.getConnection();
const stats = db.cacheStats();
```

```
{
  "enabled": true,
  "mode": "request",
  "hits": 128,
  "misses": 12,
  "size": 9,
  "ttl": 5,
  "backend": "memory"
}
```

The shape is `{ enabled, mode, hits, misses, size, ttl, backend? }`. `mode` is one of:

- `"request"` -- request-scoped caching is active (the default).
- `"persistent"` -- `TINA4_DB_CACHE=true`, so entries survive across requests.
- `"off"` -- both layers are disabled.

`db.cacheClear()` flushes the query cache and resets its counters. It is synchronous too.

A key/value hit rate above 90% means your caching strategy works. Below 80% means TTLs are too short, the cache is too small, or you are caching data that is not accessed often enough to benefit.

---

10. Combining Cache Layers

For maximum performance, stack the layers. Each backstops the one above it:

```
import { get, cacheGet, cacheSet } from "@tina4/core";
import { Database } from "tina4-nodejs/orm";

get("/api/catalog", async (req, res) => {
    const page = parseInt(req.query.page ?? "1", 10);
    const cacheKey = `catalog:page:${page}`;

    // Layer A: application key/value cache - await it.
    const cached = await cacheGet(cacheKey);
    if (cached !== undefined) {
        return res.json({ ...(cached as object), cache: "application" });
    }

    // Layer B: database query (request + persistent caching apply here automatically).
    const db = await Database.getConnection();
    const limit = 20;
```

```
    const offset = (page - 1) * limit;

    const products = await db.fetchAll(
        `SELECT p.*, c.name AS category_name
         FROM products p
         JOIN categories c ON p.category_id = c.id
         WHERE p.active = 1
         ORDER BY p.created_at DESC
         LIMIT ? OFFSET ?`,
        [limit, offset]
    );

    const total = await db.fetchOne("SELECT COUNT(*) AS count FROM products WHERE active = 1");

    const catalog = {
        products,
        page,
        total: (total as { count: number }).count,
        pages: Math.ceil((total as { count: number }).count / limit),
        generatedAt: new Date().toISOString()
    };

    await cacheSet(cacheKey, catalog, 300);

    return res.json({ ...catalog, cache: "none" });
}, ["ResponseCache:60"]);
```

Three layers, in the order a request meets them:

- **Response cache** (60 seconds): the entire HTTP response is cached. The handler does not run.
- **Application key/value cache** (300 seconds): if the response cache expired but this is still fresh, skip the database work.
- **Database query cache**: individual query results dedupe within the request and (if `TINA4_DB_CACHE=true`) across requests.

The first visitor after a full expiry waits 800ms. Everyone else gets the response in under 5ms.

---

11. Exercise: Cache an Expensive Product Listing Endpoint

Build a product listing endpoint that caches at multiple levels.

Requirements

- Create a `GET /api/store/products` endpoint that:
- Accepts query parameters: `category`, `page`, `limit`
- Returns a list of products with pagination metadata
- Uses the key/value API (`cacheGet`/`cacheSet`) with a 5-minute TTL
- Includes a `source` field (`"cache"` or `"database"`)
- Create a `POST /api/store/products` endpoint that:

- Creates a new product
- Invalidates the relevant cache entries
- Create a `GET /api/store/cache-stats` endpoint that shows cache statistics

Test with

```
# First call - cache miss, slow
curl "http://localhost:7148/api/store/products?category=Electronics&page=1"

# Second call - cache hit, fast
curl "http://localhost:7148/api/store/products?category=Electronics&page=1"

# Different category - cache miss
curl "http://localhost:7148/api/store/products?category=Fitness&page=1"

# Create a product - should invalidate cache
curl -X POST http://localhost:7148/api/store/products \
  -H "Content-Type: application/json" \
  -d '{"name": "Smart Watch", "category": "Electronics", "price": 299.99}'

# Same query again - cache miss (invalidated by the POST)
curl "http://localhost:7148/api/store/products?category=Electronics&page=1"

# Check cache stats
curl http://localhost:7148/api/store/cache-stats
```

---

12. Solution

Create `src/routes/storeCached.ts`:

```
import { get, post, cacheGet, cacheSet, cacheDelete, cacheStats } from "@tina4/core";
import { createHash } from "crypto";

function getProductStore() {
    return [
        { id: 1, name: "Wireless Keyboard", category: "Electronics", price: 79.99, inStock: true
        { id: 2, name: "Yoga Mat", category: "Fitness", price: 29.99, inStock: true },
        { id: 3, name: "Coffee Grinder", category: "Kitchen", price: 49.99, inStock: false },
        { id: 4, name: "Standing Desk", category: "Electronics", price: 549.99, inStock: true },
        { id: 5, name: "Running Shoes", category: "Fitness", price: 119.99, inStock: true },
        { id: 6, name: "Bluetooth Speaker", category: "Electronics", price: 39.99, inStock: true
        { id: 7, name: "Resistance Bands", category: "Fitness", price: 14.99, inStock: true },
        { id: 8, name: "French Press", category: "Kitchen", price: 34.99, inStock: true },
    ];
}

function cacheKeyFor(category: string | null, page: number, limit: number): string {
    const keyData = JSON.stringify({ category, page, limit });
    return `store:products:${createHash("md5").update(keyData).digest("hex")}`;
}


get("/api/store/products", async (req, res) => {
    const category = req.query.category ?? null;
    const page = parseInt(req.query.page ?? "1", 10);
```

```
    const limit = parseInt(req.query.limit ?? "20", 10);

    const cacheKey = cacheKeyFor(category, page, limit);

    // Try cache first - await it. A miss is undefined.
    const cached = await cacheGet(cacheKey);
    if (cached !== undefined) {
        return res.json({ ...(cached as object), source: "cache" });
    }

    // Simulate an expensive database query.
    await new Promise(resolve => setTimeout(resolve, 100));  // 100ms delay

    let products = getProductStore();
    if (category !== null) {
        products = products.filter(
            p => p.category.toLowerCase() === String(category).toLowerCase()
        );
    }

    const total = products.length;
    const offset = (page - 1) * limit;
    products = products.slice(offset, offset + limit);

    const result = {
        products,
        page,
        limit,
        total,
        pages: Math.ceil(total / limit),
        generatedAt: new Date().toISOString()
    };

    // Cache for 5 minutes.
    await cacheSet(cacheKey, result, 300);

    return res.json({ ...result, source: "database" });
});


post("/api/store/products", async (req, res) => {
    const body = req.body;

    if (!body.name) {
        return res.status(400).json({ error: "Name is required" });
    }

    const product = {
        id: Math.floor(Math.random() * 9000) + 100,
        name: body.name,
        category: body.category ?? "General",
        price: parseFloat(body.price ?? "0"),
        inStock: true
    };

    // Invalidate every product-list cache we might have stored.
    const categories: (string | null)[] = ["Electronics", "Fitness", "Kitchen", "General", null]
    for (const cat of categories) {
        for (let p = 1; p <= 5; p++) {
```

```
            await cacheDelete(cacheKeyFor(cat, p, 20));
        }
    }

    return res.status(201).json({
        message: "Product created",
        product,
        cacheInvalidated: true
    });
});


get("/api/store/cache-stats", async (req, res) => {
    return res.json(await cacheStats());
});
```

First call (cache miss):

```
curl "http://localhost:7148/api/store/products?category=Electronics&page=1"
```


```
{
  "products": [
    {"id": 1, "name": "Wireless Keyboard", "category": "Electronics", "price": 79.99, "inStock":
    {"id": 4, "name": "Standing Desk", "category": "Electronics", "price": 549.99, "inStock": tr
    {"id": 6, "name": "Bluetooth Speaker", "category": "Electronics", "price": 39.99, "inStock":
  ],
  "page": 1,
  "limit": 20,
  "total": 3,
  "pages": 1,
  "generatedAt": "2026-03-22T14:30:00.000Z",
  "source": "database"
}
```


Second call (cache hit):


```
{
  "products": [
    {"id": 1, "name": "Wireless Keyboard", "category": "Electronics", "price": 79.99, "inStock":
    {"id": 4, "name": "Standing Desk", "category": "Electronics", "price": 549.99, "inStock": tr
    {"id": 6, "name": "Bluetooth Speaker", "category": "Electronics", "price": 39.99, "inStock":
  ],
  "page": 1,
  "limit": 20,
  "total": 3,
  "pages": 1,
  "generatedAt": "2026-03-22T14:30:00.000Z",
  "source": "cache"
}
```


Same `generatedAt`, but `source` changed from `"database"` to `"cache"`. The handler did not run.

---

13. Gotchas

1. Forgetting to await a cache call

Problem: `cacheGet` always looks like a miss, or `cacheSet` never seems to store anything.

Cause: The key/value API is async. `const v = cacheGet(key)` returns a Promise, not the value. Comparing a Promise with `=== undefined` is always false, so you treat every read as a hit -- of a Promise object.

Fix: `await` every key/value call: `const v = await cacheGet(key)`, `await cacheSet(...)`, `await cacheDelete(...)`, `await clearCache()`. The database read path is async too -- `await db.fetchAll(...)`. Only `db.cacheStats()` and `db.cacheClear()` are synchronous.

2. Caching authenticated responses

Problem: User A's profile is served to User B because the response was cached.

Cause: The response cache keys by method and URL alone. If `/api/profile` returns different data per user but every request hits the same URL, the first response is served to everyone.

Fix: Do not put the response cache on user-specific endpoints. Use the key/value API with a user-specific key instead: `` `await cacheSet(profile:${userId}, data, 300)` ``.

3. Memory cache lost on restart

Problem: After restarting the server, performance drops until the cache warms up.

Cause: The memory backend lives in the process. It is gone when the process restarts, so every request is a miss until data is cached again.

Fix: For production, use Redis (`TINA4_CACHE_BACKEND=redis`) or file (`TINA4_CACHE_BACKEND=file`). Both survive restarts. You can also run a warmup script that pre-populates the cache with your hottest data.

4. Stale data after a database update

Problem: You updated a product's price, but the API still returns the old price.

Cause: A key/value entry you set yourself still holds the old data and has not expired.

Fix: Invalidate or update on write. Use `` `await cacheDelete(product:${id}) after an update, or write-through with` await cacheSet(...)`. (The request and persistent DB caches flush themselves on any write -- this gotcha is about keys you manage by hand.)

5. Cache key collisions

Problem: Two different queries return the same cached data.

Cause: Your keys are not specific enough. Using `"products"` for both the full list and a filtered list collides.

Fix: Include every relevant parameter in the key: `"products:category:Electronics:page:1:limit:20"`, or hash the parameters: `

products:${createHash("md5").update(JSON.stringify(params)).digest("hex")}` .

6. Serialization overhead

Problem: Caching makes certain requests slower, not faster.

Cause: The cached object is large. Serializing and deserializing it costs more than recomputing it -- especially on a network backend, where every value is JSON over the wire.

Fix: Only cache data that is expensive to compute. If the original operation takes 5ms and serialization takes 10ms, caching is counterproductive. Profile before and after.

7. Forgetting to set a TTL

Problem: Cache entries never expire and the server's memory grows until it crashes.

Cause: You called `await cacheSet("key", value)` with no TTL, so it used the default. For data you expect to live a long time, that may be shorter than you think -- or, on a backend without eviction, longer.

Fix: Set an explicit TTL: `await cacheSet("key", value, 300)`. Even for data that "never changes," set a long one like 86400 (24 hours). It is a safety net against both stale data and unbounded growth. `TINA4_CACHE_MAX_ENTRIES` (default 1000) also caps the memory and file backends.

Queue System

1. Do Not Make the User Wait

Your app sends welcome emails on signup. Generates PDF invoices. Resizes uploaded images. Each task takes 2 to 30 seconds. Run them inside the HTTP request and the user stares at a spinner.

Queues move slow work to a background process. The user gets a response in milliseconds. The work still happens -- just not during the request.

Tina4 has a built-in queue system. Works out of the box with a file-based backend. No Redis. No RabbitMQ. No external services.

The providers share delivery semantics, but they do not pretend every broker supports every management operation:

Provider	Supported contract	Explicitly unavailable
File / lite	Every queue operation	None
MongoDB	Every queue operation	None
RabbitMQ	Push, consume, complete, fail/reject, stable non-destructive dead letters, broker size where available, close	Priority, delay, pop by ID, purge by status, failed-job listing, retry failed
Kafka	Delivery, acknowledge, fail, stable non-destructive dead letters; `size()` honestly returns `0`	Priority, delay, pop by ID, failed-job listing, retry failed

This capability contract is defined by ADR-0022, ADR-0023, and ADR-0024. An unsupported operation raises an error naming the backend and operation. It never succeeds as a no-op or returns a misleading empty result.

---

2. Why Queues Matter

Without queues: 6530ms response time. With queues: 33ms. Same work done. Different timing.

Queues also deliver retry logic, rate limiting, fault isolation, and horizontal scaling.

---

3. File Queue (Default)

The file-based backend is the default. No configuration needed.

Creating a Queue and Pushing a Job

```
import { Queue } from "tina4-nodejs";

const queue = new Queue({ topic: "emails" });

// Push a job
queue.push({
    to: "alice@example.com",
    subject: "Order Confirmation",
    body: "Your order #1234 has been confirmed."
});
```

The constructor takes a config object. Pass `topic`, and optionally `backend`, `path`, `maxRetries`, and `retryBackoff`:

```
const queue = new Queue({ topic: "emails", maxRetries: 3, retryBackoff: 30 });
```

A bare string as the first argument selects a backend, not a topic -- `new Queue("rabbitmq")` picks RabbitMQ. Use the config object to set a topic.

Convenience Method: produce

The `produce` method pushes to a specific topic without creating a separate Queue instance:

```
const queue = new Queue({ topic: "emails" });
queue.produce("invoices", { order_id: 101, format: "pdf" });
```

Push with Priority

Jobs default to priority 0 (normal). The queue pops the highest-priority job first. Jobs that share a priority pop oldest-first:

```
// Normal priority (default)
queue.push({ to: "alice@example.com", subject: "Newsletter" });

// High priority -- popped before normal jobs
// push(payload, delay, priority) -- delay 0, priority 10
queue.push({ to: "alice@example.com", subject: "Password Reset" }, 0, 10);
```

Queue Size

Check how many pending jobs are in the queue:

```
const count = queue.size();
```

Pass a status string to count jobs in a specific state. `size()` takes one argument -- the status. Valid statuses are `pending` (the default), `failed` (jobs still retrying), and `dead` (dead letters):

```
const dead = queue.size("dead");
```

---

4. Pushing from Route Handlers

```
import { Router, Queue } from "tina4-nodejs";

const queue = new Queue({ topic: "emails" });

Router.post("/api/register", async (req, res) => {
    const body = req.body;
    const userId = 42;

    queue.push({
        user_id: userId,
        to: body.email,
        name: body.name,
        subject: "Welcome!"
    });

    return res.status(201).json({
        message: "Registration successful. Welcome email will arrive shortly.",
        user_id: userId
    });
});
```

---

5. Consuming Jobs

The `consume` method is an **async generator** that yields jobs one at a time. Iterate it with `for await`, not a plain `for` loop -- a plain `for` loop throws because the generator is async. Call `complete` on success and `fail` on failure:

```
import { Queue } from "tina4-nodejs";

const queue = new Queue({ topic: "emails" });

for await (const job of queue.consume("emails")) {
    try {
        await sendEmail(job.payload.to, job.payload.subject, job.payload.body);
        job.complete();
    } catch (e) {
        job.fail(e.message);
    }
}
```

`consume` runs forever by default. When the queue drains it sleeps for `pollInterval` milliseconds (default 1000) and polls again, so a long-running worker keeps picking up new jobs without any outer loop.

Pass `pollInterval = 0` to drain the queue once and stop. This is the single-pass form -- handy in tests and one-shot scripts:

```
// consume(topic, id, pollInterval) -- 0 means single-pass drain
for await (const job of queue.consume("emails", undefined, 0)) {
    await sendEmail(job.payload.to, job.payload.subject, job.payload.body);
    job.complete();
}
```

Automatic Retry

`job.fail(reason)` does the retry for you. It records one failed attempt and re-enqueues the job until it has been attempted `maxRetries` times. After that, the job becomes a dead letter. You write a single try/catch -- no manual retry call:

```
// maxRetries defaults to 3, so this job runs at most 3 times before
// it is dead-lettered automatically.
const queue = new Queue({ topic: "emails", maxRetries: 3 });

for await (const job of queue.consume("emails", undefined, 0)) {
    try {
        await sendEmail(job.payload.to, job.payload.subject, job.payload.body);
        job.complete();
    } catch (e) {
        job.fail(e.message);   // re-enqueues automatically, dead-letters when exhausted
    }
}
```

Set `retryBackoff` (seconds) on the queue to delay each automatic re-enqueue. The default is 0 -- retry immediately, so the next poll picks the job straight up:

```
const queue = new Queue({ topic: "emails", maxRetries: 5, retryBackoff: 30 });
```

Manual Retry with Delay

`job.retry(delaySeconds)` re-queues a job yourself, ignoring the retry limit. Use it when you want explicit control over a single re-attempt rather than the automatic `fail` lifecycle:

```
for await (const job of queue.consume("emails", undefined, 0)) {
    try {
        await sendEmail(job.payload.to, job.payload.subject, job.payload.body);
        job.complete();
    } catch (e) {
        // Re-queue after 30 seconds, regardless of attempt count
        job.retry(30);
    }
}
```

Manual Pop

For more control, pop a single job:

```
const job = queue.pop();

if (job !== null) {
    try {
        await sendEmail(job.payload.to, job.payload.subject);
        job.complete();
    } catch (e) {
        job.fail(e.message);
    }
}
```

---

6. Job Lifecycle

```
push() -> PENDING -> pop()/consume() -> RESERVED -> job.complete() -> COMPLETED
                                                 -> job.fail()
                                                        |
                                                attempts < maxRetries
                                                        |
                                                    re-enqueued to PENDING
                                                        |
                                                attempts >= maxRetries
                                                        |
                                                    DEAD LETTER
```

`job.fail()` drives this automatically. Each call increments the attempt count exactly once. While attempts are below `maxRetries`, the job goes back to PENDING (after `retryBackoff` seconds, if set). Once attempts reach `maxRetries`, the job moves to the dead-letter store.

Job Methods

When you receive a job from `consume` or `pop`, you have these methods:

- `job.complete()` -- mark the job as done. Terminal -- the job was already removed from the queue on pop.
- `job.fail(reason)` -- record a failed attempt. Re-enqueues automatically until `maxRetries` is reached, then dead-letters.
- `job.reject(reason)` -- alias for `fail`.
- `job.retry(delaySeconds)` -- manual re-queue after a delay (in seconds), ignoring the retry limit. The job goes back to PENDING.

Job Properties

- `job.topic` -- the topic this job belongs to. Useful when consuming from multiple topics.
- `job.attempts` -- how many times this job has been attempted.
- `job.error` -- the last failure reason.

Always call `complete`, `fail`, or `retry` on every job. If you do not, the job stays reserved.

---

7. Retry and Dead Letters

Max Retries

The default `maxRetries` is 3. `job.fail()` re-enqueues the job until its attempt count reaches `maxRetries`, then dead-letters it. Set a different limit on the queue:

```
const queue = new Queue({ topic: "emails", maxRetries: 5 });
```

Failed vs Dead-Lettered Jobs

`failed()` returns jobs that failed at least once but still have retries left -- they are sitting in the pending queue, waiting for the next poll. `deadLetters()` returns jobs that exhausted their retries:

```
const retrying = queue.failed();       // failed once, still retrying
const exhausted = queue.deadLetters();  // out of retries
```

Dead Letters

A dead letter is a job that exceeded `maxRetries`. There is no magic dead letter queue -- you retrieve and handle them yourself:

```
const deadJobs = queue.deadLetters();

for (const job of deadJobs) {
    console.log(`Dead job: ${job.id}`);
    console.log(`  Payload: ${JSON.stringify(job.payload)}`);
    console.log(`  Error: ${job.error}`);
}
```

Reviving Jobs

Move a dead letter back to PENDING. `retry(jobId)` revives one job by ID; `retryFailed()` revives dead letters that are under a raised `maxRetries` limit:

```
// Revive a specific dead letter by ID
queue.retry(jobId);

// Re-queue dead letters that are under the (possibly raised) maxRetries limit
queue.retryFailed();
```

Purging Jobs

Remove jobs by status. `purge()` accepts `pending`, `failed`, and `dead`:

```
queue.purge("pending");
queue.purge("dead");
```

---

8. Switching Backends

Switching backends is a config change, not a code change. Two variables do the work. `TINA4_QUEUE_BACKEND` picks the backend. `TINA4_QUEUE_URL` points it at the broker:

```
TINA4_QUEUE_BACKEND=rabbitmq
TINA4_QUEUE_URL=amqp://guest:guest@localhost:5672/
```

`TINA4_QUEUE_URL` is the one connection variable. It is identical across every Tina4 framework -- Python, PHP, Ruby, and Node.js all read the same var, so a `.env` written for one runs unchanged on another. Each backend parses the URL into the settings it needs.

Per-backend variables (`TINA4_RABBITMQ_*`, `TINA4_KAFKA_*`, `TINA4_MONGO_*`) still work. They are **optional overrides**: set one and it wins over the matching field from `TINA4_QUEUE_URL`. The precedence for every field is: the specific per-backend variable, then the value from `TINA4_QUEUE_URL`, then the built-in default.

Default: File

```
# No config needed -- file is the default
# Optionally set a custom storage path (defaults to data/queue)
TINA4_QUEUE_PATH=data/queue
```

The file backend ignores `TINA4_QUEUE_URL` -- it stores jobs on disk, not over a broker connection.

RabbitMQ

Lead with `TINA4_QUEUE_URL` as an AMQP URL. Tina4 parses it into host, port, username, password, and vhost:

```
TINA4_QUEUE_BACKEND=rabbitmq
TINA4_QUEUE_URL=amqp://guest:guest@localhost:5672/   # host/port/user/pass/vhost
```

The URL format is `amqp://[user:pass@]host:port[/vhost]`. The vhost is the path segment after the port -- `amqp://localhost:5672/orders` sets the vhost to `/orders`.

The per-backend variables are optional overrides for individual fields:

```
TINA4_RABBITMQ_HOST=localhost      # override; default: localhost
TINA4_RABBITMQ_PORT=5672           # override; default: 5672
TINA4_RABBITMQ_USERNAME=guest      # override; default: guest
TINA4_RABBITMQ_PASSWORD=guest      # override; default: guest
TINA4_RABBITMQ_VHOST=/             # override; default: /
```

Kafka

`TINA4_QUEUE_URL` is the broker list. A leading `kafka://` is stripped if present; otherwise the value is used as-is:

```
TINA4_QUEUE_BACKEND=kafka
TINA4_QUEUE_URL=localhost:9092                    # or kafka://broker1:9092,broker2:9092
TINA4_KAFKA_GROUP_ID=tina4_consumer_group         # default: tina4_consumer_group
```

`TINA4_KAFKA_BROKERS` is the optional override -- set it and it wins over `TINA4_QUEUE_URL`:

```
TINA4_KAFKA_BROKERS=localhost:9092                # override; default: localhost:9092
```

MongoDB

`TINA4_QUEUE_URL` is the connection URI:

```
TINA4_QUEUE_BACKEND=mongodb
TINA4_QUEUE_URL=mongodb://user:pass@localhost:27017
TINA4_MONGO_DB=tina4               # default: tina4
TINA4_MONGO_COLLECTION=tina4_queue # default: tina4_queue
```

The overrides: `TINA4_MONGO_URI` is a full URI that wins over `TINA4_QUEUE_URL`. The individual field variables build a URI when no URI is set at all:

```
TINA4_MONGO_URI=mongodb://user:pass@localhost:27017  # override; wins over TINA4_QUEUE_URL
TINA4_MONGO_HOST=localhost         # used only when no URI is set; default: localhost
TINA4_MONGO_PORT=27017             # used only when no URI is set; default: 27017
TINA4_MONGO_USERNAME=              # optional
TINA4_MONGO_PASSWORD=              # optional
```

Install the MongoDB driver:

```
npm install mongodb
```

Your code stays identical. Same `queue.push()` and `queue.consume()` calls. The backend is an implementation detail.

---

9. Multiple Jobs from One Action

```
import { Router, Queue } from "tina4-nodejs";

const queue = new Queue({ topic: "emails" });

Router.post("/api/orders", async (req, res) => {
    const orderId = 101;

    queue.push({ order_id: orderId, to: req.body.email, subject: "Order Confirmation" });
    queue.produce("invoices", { order_id: orderId, format: "pdf" });
    queue.produce("inventory", { items: req.body.items });
    queue.produce("warehouse", { order_id: orderId, shipping_address: req.body.shipping_address

    return res.status(201).json({ message: "Order placed successfully", order_id: orderId });
});
```

---

10. Exercise: Build an Email Queue

Build a queue-based email system with failure handling.

Requirements

- Create these endpoints:

Method	Path	Description
`POST`	`/api/emails/send`	Queue an email for sending
`GET`	`/api/emails/queue`	Show pending email count
`GET`	`/api/emails/dead`	List dead letter jobs
`POST`	`/api/emails/retry`	Re-queue dead letters under a raised limit

- The email payload should include: `to` (required), `subject` (required), `body` (required)
- Create a consumer that processes the queue, simulating occasional failures

Test with:

```
curl -X POST http://localhost:7148/api/emails/send \
  -H "Content-Type: application/json" \
  -d '{"to": "alice@example.com", "subject": "Welcome!", "body": "Thanks for signing up."}'

curl http://localhost:7148/api/emails/queue

curl http://localhost:7148/api/emails/dead

curl -X POST http://localhost:7148/api/emails/retry
```

---

11. Solution

Create `src/routes/emailQueue.ts`:

```
import { Router, Queue } from "tina4-nodejs";

const queue = new Queue({ topic: "emails" });

/**
 * @noauth
 */
Router.post("/api/emails/send", async (req, res) => {
    const body = req.body;

    const errors: string[] = [];
    if (!body.to) errors.push("'to' is required");
    if (!body.subject) errors.push("'subject' is required");
    if (!body.body) errors.push("'body' is required");

    if (errors.length > 0) {
        return res.status(400).json({ errors });
    }

    const messageId = queue.push({
        to: body.to,
        subject: body.subject,
        body: body.body
    });

    return res.status(201).json({
        message: "Email queued for sending",
        message_id: messageId
    });
});

Router.get("/api/emails/queue", async (req, res) => {
    const count = queue.size();
    return res.json({ pending: count });
});

Router.get("/api/emails/dead", async (req, res) => {
    const deadJobs = queue.deadLetters();
    const items = deadJobs.map((job) => ({
        id: job.id,
        payload: job.payload,
        error: job.error
    }));
    return res.json({ dead_letters: items, count: items.length });
});

Router.post("/api/emails/retry", async (req, res) => {
    queue.retryFailed();
    return res.json({ message: "Dead letters re-queued for retry" });
});
```

Create a separate consumer file `src/workers/emailWorker.ts`:

```
import { Queue } from "tina4-nodejs";
```

```
const queue = new Queue({ topic: "emails" });

for await (const job of queue.consume("emails")) {
    const payload = job.payload;

    console.log(`Sending email to ${payload.to}...`);
    console.log(`  Subject: ${payload.subject}`);

    try {
        // Simulate sending (replace with real email logic)
        await new Promise((resolve) => setTimeout(resolve, 1000));

        // Simulate failure for a specific address
        if (payload.to === "bad@example.com") {
            throw new Error("SMTP connection refused");
        }

        console.log(`  Email sent to ${payload.to} successfully!`);
        job.complete();

    } catch (e) {
        console.log(`  Failed: ${e.message}`);
        job.fail(e.message);
    }
}
```

`job.fail()` re-enqueues the `bad@example.com` job automatically. After it has been attempted three times (the default `maxRetries`), `queue.deadLetters()` returns it. The `/api/emails/dead` endpoint shows it. You investigate, raise `maxRetries`, and call `/api/emails/retry` to re-queue.

---

12. Gotchas

1. Use `for await`, not `for`

Fix: `consume` is an async generator. Iterate it with `for await (const job of queue.consume(...))`. A plain `for` loop throws "is not iterable".

2. Always call complete or fail

Fix: Always call `job.complete()` on success and `job.fail(reason)` on failure. If you forget, the job stays reserved forever.

3. consume runs forever

Fix: The default `pollInterval` makes `consume` poll forever, even when the queue is empty -- that is what a worker wants. For a single-pass drain (tests, scripts), pass `pollInterval = 0`: `queue.consume(topic, undefined, 0)`.

4. Worker not picking up jobs

Fix: Make sure the consumer is running. Check that the topic in `queue.push()` matches the topic in `queue.consume()`.

5. Payload must be JSON-serializable

Fix: Only pass simple data types. Pass IDs, not full objects.

6. Dead letters pile up

Fix: Monitor `queue.deadLetters()` and set up alerts. Investigate root causes, then raise `maxRetries` and call `queue.retryFailed()`, or `queue.purge("dead")`.

7. File backend for production

Fix: For multiple workers, switch to RabbitMQ, Kafka, or MongoDB via `TINA4_QUEUE_BACKEND`.

Events System

1. Decouple Without Wiring Everything Together

Your user registers. You need to send a welcome email, create a default profile, log the signup, and update analytics. You could call all four functions from the signup handler. But now the signup handler knows about email, profiles, logging, and analytics. Change any of them and you're touching the handler.

Events decouple the action from the reaction. The signup handler emits `user.registered`. Anything that cares about that event subscribes to it. The handler knows nothing about what happens next.

Tina4 ships a built-in event bus. Zero external dependencies. Synchronous listeners, priority ordering, one-shot listeners, and cleanup.

---

2. Listening for Events

Register a listener with `Events.on()`:

```
import { Events } from "tina4-nodejs";

Events.on("user.registered", (payload) => {
    console.log(`New user: ${payload.name} (${payload.email})`);
});
```

The listener runs every time `user.registered` is emitted. The `payload` is whatever object was passed to `emit()`.

---

3. Emitting Events

Trigger all listeners for an event with `Events.emit()`:

```
import { Router, Events } from "tina4-nodejs";

Router.post("/api/register", async (req, res) => {
    const body = req.body;

    // ... create the user in the database ...
    const userId = 42;

    Events.emit("user.registered", {
        id: userId,
        name: body.name,
        email: body.email,
        registeredAt: new Date().toISOString()
    });

    return res.status(201).json({ message: "Registration complete", user_id: userId });
});
```

All registered listeners for `user.registered` run immediately, in priority order, before `emit()` returns.

---

4. One-Shot Listeners with once()

A listener registered with `Events.once()` fires exactly one time and then removes itself:

```
import { Events } from "tina4-nodejs";

// Fires the first time "server.ready" is emitted, then never again
Events.once("server.ready", () => {
    console.log("Server is up -- warming the cache");
    warmCache();
});
```

Useful for initialisation tasks, first-run setup, or integration tests that need to wait for a single event.

---

5. Priority

When multiple listeners are registered for the same event, priority controls the order they run. Higher priority runs first. Default priority is `0`.

```
import { Events } from "tina4-nodejs";

// Runs third (lowest priority)
Events.on("order.placed", (payload) => {
    console.log("Analytics: recording order");
}, 0);

// Runs first (highest priority)
Events.on("order.placed", (payload) => {
    console.log("Inventory: reserving stock");
}, 20);

// Runs second
Events.on("order.placed", (payload) => {
    console.log("Email: sending confirmation");
}, 10);

Events.emit("order.placed", { orderId: 101, total: 249.99 });
// Output:
// Inventory: reserving stock
// Email: sending confirmation
// Analytics: recording order
```

Priority is specified as the third argument to `on()`.

---

6. Removing Listeners with off()

Remove a specific listener by passing the same function reference used in `on()`:

```
import { Events } from "tina4-nodejs";

function onUserLogin(payload: { userId: number }) {
    console.log(`User ${payload.userId} logged in`);
}

// Register
Events.on("user.login", onUserLogin);

// Later, remove it
Events.off("user.login", onUserLogin);

// This emit does nothing -- the listener was removed
Events.emit("user.login", { userId: 99 });
```

Note: arrow functions assigned to variables work the same way as long as you pass the same reference.

---

7. Clearing All Listeners with clear()

Remove every listener for a given event (or all events):

```
import { Events } from "tina4-nodejs";

// Clear listeners for a specific event
Events.clear("user.registered");

// Clear all listeners for all events
Events.clear();
```

Useful in tests to reset state between test cases, or in long-running processes to tear down a subsystem cleanly.

---

8. Listing Active Listeners

Inspect registered listeners for debugging:

```
import { Events } from "tina4-nodejs";

const listeners = Events.listeners("order.placed");
console.log(`${listeners.length} listeners for order.placed`);
```

---

9. Organising Events in a Real Application

Keep all event subscriptions in one place so they are easy to audit. Create `src/events/index.ts`:

```
import { Events } from "tina4-nodejs";
import { sendWelcomeEmail } from "../services/email";
import { createDefaultProfile } from "../services/profile";
import { trackSignup } from "../services/analytics";
import { logUserEvent } from "../services/logging";

// user.registered
Events.on("user.registered", async (payload) => {
    await sendWelcomeEmail(payload.email, payload.name);
}, 10);

Events.on("user.registered", async (payload) => {
    await createDefaultProfile(payload.id);
}, 5);

Events.on("user.registered", (payload) => {
    trackSignup(payload.id, payload.registeredAt);
}, 1);

Events.on("user.registered", (payload) => {
    logUserEvent("signup", payload.id);
}, 0);
```

Then import this file in `app.ts`, the entry point at the project root, so listeners are registered before any requests arrive:

```
import "./src/events/index";
import { startServer } from "tina4-nodejs";

startServer();
```

---

10. Exercise: Order Processing Pipeline

Build an order processing pipeline using events.

Requirements

- Create a `POST /api/orders` endpoint that emits `order.placed` with order data
- Register three listeners for `order.placed`:
- Priority 20: validate and reserve inventory
- Priority 10: send confirmation email (simulated)
- Priority 0: record in analytics (simulated)
- Create a `GET /api/orders/events` endpoint that returns the current listener count for `order.placed`

Test with:

```
curl -X POST http://localhost:7148/api/orders \
  -H "Content-Type: application/json" \
  -d '{"items": [{"sku": "KB-001", "qty": 2}], "email": "alice@example.com"}'

curl http://localhost:7148/api/orders/events
```

---

11. Solution

Create `src/events/orders.ts`:

```
import { Events } from "tina4-nodejs";

Events.on("order.placed", (payload) => {
    console.log(`[Inventory] Reserving stock for order ${payload.orderId}`);
    for (const item of payload.items) {
        console.log(`  - SKU ${item.sku}: qty ${item.qty}`);
    }
}, 20);

Events.on("order.placed", (payload) => {
    console.log(`[Email] Sending confirmation to ${payload.email} for order ${payload.orderId}`)
}, 10);

Events.on("order.placed", (payload) => {
    console.log(`[Analytics] Recording order ${payload.orderId} - total $${payload.total}`);
}, 0);
```

Create `src/routes/orders.ts`:

```
import { Router, Events } from "tina4-nodejs";

let orderCounter = 1000;

Router.post("/api/orders", async (req, res) => {
    const body = req.body;

    if (!body.items || body.items.length === 0) {
        return res.status(400).json({ error: "Order must contain at least one item" });
    }

    const orderId = ++orderCounter;
    const total = body.items.reduce((sum: number, item: { qty: number; price?: number }) => {
        return sum + (item.qty * (item.price ?? 9.99));
    }, 0);

    Events.emit("order.placed", {
        orderId,
        items: body.items,
        email: body.email,
        total: parseFloat(total.toFixed(2)),
        placedAt: new Date().toISOString()
    });

    return res.status(201).json({
        message: "Order placed",
        order_id: orderId,
        total
    });
});

Router.get("/api/orders/events", async (req, res) => {
    const count = Events.listeners("order.placed").length;
    return res.json({ event: "order.placed", listener_count: count });
```

```
});
```

In `app.ts`, import the events file before starting the server:

```
import "./src/events/orders";
import { startServer } from "tina4-nodejs";

startServer();
```

---

12. Gotchas

1. Async listeners are not awaited

`Events.emit()` is synchronous. If a listener is `async`, the promise is not awaited. Long async work belongs in a queue, not an event listener.

Fix: For truly async fire-and-forget tasks, use `Queue`. For synchronous side effects, use events. If you must use async listeners, handle errors inside them -- unhandled rejections in event listeners are silent.

2. Same function reference required for off()

`Events.off("event", fn)` only works if `fn` is the exact same reference registered with `on()`.

Fix: Store named functions in variables. Arrow functions defined inline cannot be removed because each definition creates a new reference.

3. Listeners persist across requests

Events registered at module load time persist for the lifetime of the process. Registering listeners inside a route handler creates duplicates on every request.

Fix: Register listeners once, at startup, in a dedicated events file.

4. Forgetting to import the events file

Listeners registered in a file that is never imported do not exist.

Fix: Explicitly import your events file in `app.ts`. A missing import is a silent failure -- no error, no listeners, no side effects.

Localization

1. Your App Speaks One Language. Your Users Speak Many.

An e-commerce store. Half your traffic is German. Error messages in English. Product descriptions in English. Currency formatted American-style. Visitors leave.

Localization means serving content in the user's language and format. Tina4 provides an `I18n` class that loads JSON locale files, interpolates variables, falls back to a default locale when a key is missing, and switches locale per request.

---

2. Locale Files

Locale files are JSON files stored in a `locales/` directory in your project root. One file per language:

```
locales/
  en.json
  de.json
  fr.json
  es.json
```

`locales/en.json`:

```
{
  "welcome": "Welcome, {name}!",
  "errors": {
    "not_found": "The page you requested was not found.",
    "unauthorized": "You must be logged in to access this page.",
    "validation": "{field} is required."
  },
  "orders": {
    "placed": "Your order #{orderId} has been placed.",
    "total": "Total: {currency}{amount}"
  },
  "nav": {
    "home": "Home",
    "products": "Products",
    "cart": "Cart",
    "account": "Account"
  }
}
```

`locales/de.json`:

```
{
  "welcome": "Willkommen, {name}!",
  "errors": {
    "not_found": "Die angeforderte Seite wurde nicht gefunden.",
    "unauthorized": "Sie müssen angemeldet sein, um auf diese Seite zuzugreifen.",
    "validation": "{field} ist erforderlich."
  },
  "orders": {
    "placed": "Ihre Bestellung #{orderId} wurde aufgegeben.",
```

```
    "total": "Gesamt: {currency}{amount}"
  },
  "nav": {
    "home": "Startseite",
    "products": "Produkte",
    "cart": "Warenkorb",
    "account": "Konto"
  }
}
```

---

3. Creating an I18n Instance

```
import { I18n } from "tina4-nodejs";

// Loads all JSON files from ./locales/
// Default locale is "en"
const i18n = new I18n("en", "./locales");
```

Argument	Default	Description
`locale`	`TINA4_LOCALE`, then `"en"`	Default locale; also the fallback when a key is missing
`path`	`TINA4_LOCALE_DIR`, then `"src/locales"`	Directory containing locale JSON files

---

4. Translating Keys with t()

The `t()` method looks up a translation key in the current locale:

```
const i18n = new I18n("en", "./locales");

i18n.setLocale("en");
console.log(i18n.t("nav.home"));         // "Home"
console.log(i18n.t("nav.products"));     // "Products"

i18n.setLocale("de");
console.log(i18n.t("nav.home"));         // "Startseite"
console.log(i18n.t("nav.products"));     // "Produkte"
```

Keys use dot notation to access nested values. `"errors.not_found"` reaches `errors -> not_found` in the JSON file.

---

5. Interpolation

Pass variables to replace `{placeholder}` tokens:

```
i18n.setLocale("en");
console.log(i18n.t("welcome", { name: "Alice" }));
```

```
// "Welcome, Alice!"

console.log(i18n.t("errors.validation", { field: "Email" }));
// "Email is required."

console.log(i18n.t("orders.placed", { orderId: "10042" }));
// "Your order #10042 has been placed."

i18n.setLocale("de");
console.log(i18n.t("welcome", { name: "Alice" }));
// "Willkommen, Alice!"
```

Any key not found in the variables object is left as-is in the output string.

---

6. Locale Switching Per Request

Detect the user's preferred locale from the `Accept-Language` header, a query parameter, or a session value:

```
import { Router, I18n } from "tina4-nodejs";

const i18n = new I18n("en", "./locales");

Router.get("/api/greeting", async (req, res) => {
    // Priority: ?lang query param > Accept-Language header > default
    const lang = req.query.lang
        ?? (req.headers["accept-language"]?.split(",")[0]?.split("-")[0])
        ?? "en";

    i18n.setLocale(lang);

    return res.json({
        locale: lang,
        message: i18n.t("welcome", { name: req.query.name ?? "guest" })
    });
});
```

```
curl "http://localhost:7148/api/greeting?name=Alice&lang=de"
```

```
{
  "locale": "de",
  "message": "Willkommen, Alice!"
}
```

```
curl "http://localhost:7148/api/greeting?name=Alice&lang=en"
```

```
{
  "locale": "en",
  "message": "Welcome, Alice!"
}
```

---

7. Fallback Locale

When a key exists in the default locale but is missing from the requested locale, `I18n` falls back automatically rather than returning an empty string or throwing.

```
// locales/fr.json is missing the "orders" section
i18n.setLocale("fr");
console.log(i18n.t("orders.placed", { orderId: "202" }));
// Falls back to "en": "Your order #202 has been placed."
```

The fallback locale is configured once and applied globally. You never need to check whether a translation exists before calling `t()`.

---

8. Listing Available Locales

```
const locales = i18n.availableLocales();
console.log(locales);
// ["en", "de", "fr", "es"]
```

Use this to build a language picker in your UI or to validate the `lang` parameter on requests.

---

9. Using I18n in Templates

Pass the translation function to Frond templates:

```
import { Router, I18n } from "tina4-nodejs";

const i18n = new I18n("en", "./locales");

Router.get("/store", async (req, res) => {
    const lang = req.query.lang ?? "en";
    i18n.setLocale(lang as string);

    return res.render("store.frond", {
        t: (key: string, vars?: Record<string, string>) => i18n.t(key, vars),
        locale: lang,
        locales: i18n.availableLocales()
    });
});
```

In `templates/store.frond`:

```
<nav>
  <a href="/">{{t("nav.home")}}</a>
  <a href="/products">{{t("nav.products")}}</a>
  <a href="/cart">{{t("nav.cart")}}</a>
</nav>
```

---

10. Exercise: Multilingual API Errors

Build an API that returns validation error messages in the user's language.

Requirements

- Create locale files for `en` and `de` with an `errors` section covering `required`, `too_short`, and `invalid_email`

- Create a `POST /api/contact` endpoint that validates a `name` (required, min 2 chars) and `email` (required, valid format) field
- Return error messages in the locale specified by the `lang` query parameter (default: `en`)

Test with:

```
# English errors
curl -X POST "http://localhost:7148/api/contact?lang=en" \
  -H "Content-Type: application/json" \
  -d '{"name": "A", "email": "not-an-email"}'

# German errors
curl -X POST "http://localhost:7148/api/contact?lang=de" \
  -H "Content-Type: application/json" \
  -d '{"name": "A", "email": "not-an-email"}'
```

---

11. Solution

`locales/en.json`:

```
{
  "errors": {
    "required": "{field} is required.",
    "too_short": "{field} must be at least {min} characters.",
    "invalid_email": "{field} must be a valid email address."
  },
  "contact": {
    "success": "Thank you, {name}. We will be in touch."
  }
}
```

`locales/de.json`:

```
{
  "errors": {
    "required": "{field} ist erforderlich.",
    "too_short": "{field} muss mindestens {min} Zeichen lang sein.",
    "invalid_email": "{field} muss eine gültige E-Mail-Adresse sein."
  },
  "contact": {
    "success": "Danke, {name}. Wir werden uns bei Ihnen melden."
  }
}
```

`src/routes/contact.ts`:

```
import { Router, I18n } from "tina4-nodejs";

const i18n = new I18n("en", "./locales");

const EMAIL_RE = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;

Router.post("/api/contact", async (req, res) => {
    const lang = (req.query.lang ?? "en") as string;
    i18n.setLocale(lang);

    const body = req.body;
```

```
    const errors: string[] = [];

    if (!body.name) {
        errors.push(i18n.t("errors.required", { field: "Name" }));
    } else if (String(body.name).length < 2) {
        errors.push(i18n.t("errors.too_short", { field: "Name", min: "2" }));
    }

    if (!body.email) {
        errors.push(i18n.t("errors.required", { field: "Email" }));
    } else if (!EMAIL_RE.test(String(body.email))) {
        errors.push(i18n.t("errors.invalid_email", { field: "Email" }));
    }

    if (errors.length > 0) {
        return res.status(422).json({ locale: lang, errors });
    }

    return res.json({
        locale: lang,
        message: i18n.t("contact.success", { name: body.name })
    });
});
```

English error response:

```
{
  "locale": "en",
  "errors": [
    "Name must be at least 2 characters.",
    "Email must be a valid email address."
  ]
}
```

German error response:

```
{
  "locale": "de",
  "errors": [
    "Name muss mindestens 2 Zeichen lang sein.",
    "Email muss eine gültige E-Mail-Adresse sein."
  ]
}
```

---

12. Gotchas

1. Locale files must be valid JSON

A trailing comma or missing quote breaks the entire locale file silently. `I18n` will log a parse error and skip the file, falling back to the default locale for all keys.

Fix: Use a JSON linter or your editor's JSON validation before deploying. CI pipelines should run `JSON.parse()` against all locale files.

2. Missing keys return the key string

If a key does not exist in any locale, `t()` returns the key itself (e.g., `"orders.shipped"`). This makes missing translations visible in production.

Fix: Maintain the English locale as the complete reference. Other locales can be partial -- they fall back to English. Audit periodically with a script that compares all locale files against the English baseline.

3. setLocale is not thread-safe for concurrent requests

If multiple concurrent requests call `i18n.setLocale()` on the same instance, they interfere with each other.

Fix: Create a new `I18n` instance per request, or use `i18n.translate(key, vars, locale)` which accepts the locale as a parameter without mutating instance state.

4. Placeholder names are case-sensitive

`{Name}` and `{name}` are different placeholders. A mismatch silently leaves the placeholder unreplaced.

Fix: Use consistent lowercase placeholder names in all locale files.

Structured Logging

1. console.log Is Not a Logging Strategy

`console.log("something happened")` tells you nothing. No timestamp. No severity. No context. No way to filter by level in production. No way to ship to a log aggregator. When an incident happens at 2am, you need to know exactly what happened and when.

Structured logging emits machine-readable JSON. Every entry has a timestamp, log level, message, and optional context fields. You can filter, search, and aggregate without grep.

Tina4 provides a `Log` singleton with four severity levels. Zero configuration required. Control verbosity with one environment variable.

---

2. The Four Log Levels

```
import { Log } from "tina4-nodejs";

Log.debug("Cache lookup", { key: "product:42", hit: false });
Log.info("User registered", { userId: 99, email: "alice@example.com" });
Log.warn("Rate limit approaching", { ip: "203.0.113.5", requests: 95, limit: 100 });
Log.error("Payment failed", { orderId: 1042, reason: "Card declined" });
```

Each call emits a structured log line:

```
{"timestamp":"2026-04-02T08:12:01.234Z","level":"DEBUG","message":"Cache lookup","key":"product:
{"timestamp":"2026-04-02T08:12:01.235Z","level":"INFO","message":"User registered","userId":99,"
{"timestamp":"2026-04-02T08:12:01.236Z","level":"WARN","message":"Rate limit approaching","ip":"
{"timestamp":"2026-04-02T08:12:01.237Z","level":"ERROR","message":"Payment failed","orderId":104
```

Level	Use for
`debug`	Detailed diagnostic info, cache hits/misses, query plans
`info`	Normal application events: logins, signups, orders placed
`warn`	Unexpected but recoverable situations: retries, slow queries
`error`	Failures that need attention: payment errors, crashed workers

---

3. Controlling Verbosity with TINA4_LOG_LEVEL

Set the minimum level in `.env`:

```
TINA4_LOG_LEVEL=info
```

Only entries at or above the configured level are emitted:

	debug	info	warn	error
`debug`	shown	shown	shown	shown
`info`	silent	shown	shown	shown
`warn`	silent	silent	shown	shown
`error`	silent	silent	silent	shown

In development, use `debug`. In production, use `info` or `warn` to reduce log volume.

```
# .env.development
TINA4_LOG_LEVEL=debug

# .env.production
TINA4_LOG_LEVEL=warn
```

---

4. Adding Context Fields

The second argument to any log method is a plain object. Its fields are merged into the log entry:

```
import { Log } from "tina4-nodejs";

Log.info("HTTP request", {
    method: "POST",
    path: "/api/orders",
    status: 201,
    duration_ms: 47,
    user_id: 15
});
```

```
{"timestamp":"2026-04-02T08:15:00.000Z","level":"INFO","message":"HTTP request","method":"POST",
```

Any JSON-serializable value is valid: strings, numbers, booleans, arrays, nested objects.

---

5. Logging Errors

Pass an `Error` object alongside context:

```
import { Log } from "tina4-nodejs";

try {
    await processPayment(orderId, amount);
} catch (err) {
    Log.error("Payment processing failed", {
        orderId,
        amount,
        error: err instanceof Error ? err.message : String(err),
        stack: err instanceof Error ? err.stack : undefined
```

```
    });
}
```

```
{
  "timestamp": "2026-04-02T08:16:00.000Z",
  "level": "ERROR",
  "message": "Payment processing failed",
  "orderId": 1042,
  "amount": 249.99,
  "error": "Connection timeout after 5000ms",
  "stack": "Error: Connection timeout...\n    at PaymentGateway.charge ..."
}
```

---

6. Request-Scoped Logging

Add a request ID to every log entry in a request handler so you can trace all log lines from a single request:

```
import { Router, Log } from "tina4-nodejs";
import { randomUUID } from "crypto";

Router.post("/api/checkout", async (req, res) => {
    const requestId = randomUUID();
    const start = Date.now();

    Log.info("Checkout started", { requestId, user: req.user?.id });

    try {
        const orderId = await placeOrder(req.body, requestId);

        Log.info("Order placed", {
            requestId,
            orderId,
            duration_ms: Date.now() - start
        });

        return res.status(201).json({ order_id: orderId });

    } catch (err) {
        Log.error("Checkout failed", {
            requestId,
            error: err instanceof Error ? err.message : String(err),
            duration_ms: Date.now() - start
        });

        return res.status(500).json({ error: "Checkout failed" });
    }
});
```

Search your log aggregator for `requestId` to see every log line from that request, in order, with timings.

---

7. Performance Logging

Log slow operations to identify bottlenecks:

```
import { Log } from "tina4-nodejs";
import { Database } from "tina4-nodejs/orm";

async function fetchDashboardData(userId: number) {
    const t0 = Date.now();
    const db = await Database.getConnection();

    const data = await db.fetchAll(
        `SELECT o.id, o.total, o.status, COUNT(oi.id) as item_count
         FROM orders o
         JOIN order_items oi ON oi.order_id = o.id
         WHERE o.user_id = :userId
         GROUP BY o.id
         ORDER BY o.created_at DESC
         LIMIT 10`,
        { userId }
    );

    const duration = Date.now() - t0;

    if (duration > 500) {
        Log.warn("Slow dashboard query", { userId, duration_ms: duration, rows: data.length });
    } else {
        Log.debug("Dashboard query", { userId, duration_ms: duration, rows: data.length });
    }

    return data;
}
```

---

8. Exercise: Add Logging to an Existing API

Take the product listing endpoint from Chapter 11 and add structured logging at every meaningful point.

Requirements

- Log `info` when a request is received, including the route and query parameters
- Log `debug` for cache hits and misses with the cache key
- Log `warn` when a query takes longer than 200ms
- Log `error` when an exception is caught, with the error message and stack

Expected log output (debug level):

```
{"timestamp":"...","level":"INFO","message":"Products request","category":"Electronics","page":1
{"timestamp":"...","level":"DEBUG","message":"Cache miss","key":"store:products:a3f2..."}
{"timestamp":"...","level":"WARN","message":"Slow product query","duration_ms":312,"rows":3}
{"timestamp":"...","level":"INFO","message":"Products served","source":"database","count":3,"dur
```

---

9. Solution

```
import { Router, Log, cacheGet, cacheSet } from "tina4-nodejs";
import { createHash } from "crypto";

const PRODUCTS = [
    { id: 1, name: "Wireless Keyboard", category: "Electronics", price: 79.99 },
    { id: 2, name: "Yoga Mat", category: "Fitness", price: 29.99 },
    { id: 3, name: "Coffee Grinder", category: "Kitchen", price: 49.99 },
    { id: 4, name: "Standing Desk", category: "Electronics", price: 549.99 },
];

Router.get("/api/products/logged", async (req, res) => {
    const category = req.query.category ?? null;
    const page = parseInt(req.query.page ?? "1", 10);
    const start = Date.now();

    Log.info("Products request", { category, page });

    const keyData = JSON.stringify({ category, page });
    const cacheKey = `products:${createHash("md5").update(keyData).digest("hex")}`;

    try {
        const cached = await cacheGet(cacheKey);

        if (cached !== null) {
            Log.debug("Cache hit", { key: cacheKey });
            return res.json({ ...cached, source: "cache" });
        }

        Log.debug("Cache miss", { key: cacheKey });

        // Simulate database work
        const t0 = Date.now();
        await new Promise(resolve => setTimeout(resolve, 50));
        let products = PRODUCTS;

        if (category) {
            products = products.filter(
                p => p.category.toLowerCase() === String(category).toLowerCase()
            );
        }

        const queryDuration = Date.now() - t0;

        if (queryDuration > 200) {
            Log.warn("Slow product query", { duration_ms: queryDuration, rows: products.length }
        }

        const result = { products, page, total: products.length };
        await cacheSet(cacheKey, result, 300);

        Log.info("Products served", {
            source: "database",
            count: products.length,
            duration_ms: Date.now() - start
        });

        return res.json({ ...result, source: "database" });
```

```
    } catch (err) {
        Log.error("Products endpoint failed", {
            error: err instanceof Error ? err.message : String(err),
            stack: err instanceof Error ? err.stack : undefined,
            duration_ms: Date.now() - start
        });
        return res.status(500).json({ error: "Internal server error" });
    }
});
```

---

10. Gotchas

1. Logging sensitive data

`Log.info("Login", { email, password })` ships the password to your log aggregator.

Fix: Never log passwords, tokens, credit card numbers, or PII. Log user IDs and request IDs instead. Before shipping a log call to production, check every field.

2. Logging in hot paths adds latency

Calling `Log.debug()` on every database row in a loop adds up.

Fix: Log aggregates, not individual items. `Log.debug("Fetched rows", { count: rows.length })` is better than logging each row.

3. Circular references in context objects

`Log.info("Data", { obj })` throws if `obj` contains circular references, because JSON serialization fails.

Fix: Pass primitive values and simple objects. Use `JSON.stringify` with a replacer to handle circular references if you must log complex objects.

4. TINA4*LOG*LEVEL defaults to info

In development, you might expect to see debug output and see nothing.

Fix: Set `TINA4_LOG_LEVEL=debug` in your `.env` file. Check the value with `Log.debug("Log level check", {})` -- if it appears, debug logging is active.

Email with Messenger

1. Every App Sends Email

Signup confirmations. Password resets. Weekly digests. Invoices with PDF attachments. Every application needs email. Nobody enjoys building it.

SMTP configuration. Plain text fallbacks. Attachment encoding. Connection timeouts. The details pile up fast. Tina4's `Messenger` class absorbs all of it. Configure through `.env`. Create an instance. Send. Without an SMTP host, Messenger captures mail in the dev mailbox. An SMTP host enables real delivery, so staging needs an explicit safety setting.

---

2. Messenger Configuration via .env

All email configuration lives in `.env`:

```
TINA4_MAIL_HOST=smtp.example.com
TINA4_MAIL_PORT=587
TINA4_MAIL_USERNAME=your-email@example.com
TINA4_MAIL_PASSWORD=your-app-password
TINA4_MAIL_ENCRYPTION=tls
TINA4_MAIL_FROM=noreply@example.com
TINA4_MAIL_FROM_NAME=My Store
```

Variable	Description	Common Values
`TINA4_MAIL_HOST`	SMTP server hostname	`smtp.gmail.com`, `smtp.mailgun.org`, `smtp.sendgrid.net`
`TINA4_MAIL_PORT`	SMTP port	`587` (TLS), `465` (SSL), `25` (unencrypted)
`TINA4_MAIL_USERNAME`	Login username	Usually your email address
`TINA4_MAIL_PASSWORD`	Login password or app-specific password	App passwords for Gmail
`TINA4_MAIL_ENCRYPTION`	Encryption method	`tls` (recommended), `ssl`, `none`
`TINA4_MAIL_FROM`	Default "From" address	`noreply@yourdomain.com`
`TINA4_MAIL_FROM_NAME`	Default "From" display name	`My Store`, `Acme Corp`
`TINA4_MAIL_CAPTURE`	Disable SMTP and capture each message locally	`true` on staging when no external delivery is allowed
`TINA4_MAIL_REDIRECT_TO`	Replace every SMTP recipient with a safety list	`qa@example.com,product@example.com`

Staging delivery safety

`TINA4_DEBUG` controls developer tools. It does not disable SMTP delivery. Pick one explicit mail policy for staging:

```
# No message leaves the application. Inspect it in the dev mailbox.
TINA4_MAIL_CAPTURE=true
```

Capture wins even when an SMTP host exists. A missing SMTP host also captures mail. `TINA4_MAILBOX_DIR` only selects the dev mailbox directory; it does not turn capture on or off.

Use real SMTP without risking customer delivery by replacing all recipients:

```
TINA4_MAIL_REDIRECT_TO=qa@example.com,product@example.com
```

Tina4 trims each comma-separated address and drops blank entries. On the SMTP path, the safety list replaces `To`; Tina4 clears `Cc` and `Bcc` and stores all original recipients in `X-Tina4-Original-To`. An unset or empty list leaves delivery unchanged. If capture and redirect are both set, capture wins and nothing reaches SMTP.

Common Provider Configurations

Gmail:

```
TINA4_MAIL_HOST=smtp.gmail.com
TINA4_MAIL_PORT=587
TINA4_MAIL_USERNAME=your-email@gmail.com
TINA4_MAIL_PASSWORD=your-app-password
TINA4_MAIL_ENCRYPTION=tls
```

Gmail requires an "App Password" (not your regular password) when two-factor authentication is enabled.

Mailgun:

```
TINA4_MAIL_HOST=smtp.mailgun.org
TINA4_MAIL_PORT=587
TINA4_MAIL_USERNAME=postmaster@mg.yourdomain.com
TINA4_MAIL_PASSWORD=your-mailgun-smtp-password
TINA4_MAIL_ENCRYPTION=tls
```

SendGrid:

```
TINA4_MAIL_HOST=smtp.sendgrid.net
TINA4_MAIL_PORT=587
TINA4_MAIL_USERNAME=apikey
TINA4_MAIL_PASSWORD=your-sendgrid-api-key
TINA4_MAIL_ENCRYPTION=tls
```

---

3. Constructor Override Pattern

Different emails need different SMTP accounts. Transactional emails from one server. Marketing from another. Override the configuration in the constructor:

```
import { Messenger } from "tina4-nodejs";
```

```
// Uses .env defaults
const mailer = new Messenger();

// Override specific settings
const marketingMailer = new Messenger({
    host: "smtp.mailgun.org",
    port: 587,
    username: "marketing@mg.yourdomain.com",
    password: "marketing-smtp-password",
    encryption: "tls",
    fromAddress: "newsletter@yourdomain.com",
    fromName: "My Store Newsletter"
});
```

Constructor arguments take priority over `.env` values. Any argument you omit falls back to the environment variable.

---

4. Sending Plain Text Email

The simplest email:

```
import { Router, Messenger } from "tina4-nodejs";

Router.post("/api/contact", async (req, res) => {
    const body = req.body;
    const mailer = new Messenger();

    const result = await mailer.send({
        to: body.email,
        subject: "Contact Form Submission",
        body: `Name: ${body.name}\nEmail: ${body.email}\nMessage:\n${body.message}`
    });

    if (result.success) {
        return res.json({ message: "Email sent successfully" });
    }
    return res.status(500).json({ error: "Failed to send email", details: result.error });
});
```

```
curl -X POST http://localhost:7148/api/contact \
  -H "Content-Type: application/json" \
  -d '{"name": "Alice", "email": "alice@example.com", "message": "I love your products!"}'
```

```
{"message":"Email sent successfully"}
```

The `send()` method returns an object with three keys: `success` (boolean), `error` (string or null), and `messageId` (string or null).

The send() Method Signature

```
await mailer.send({
    to,                  // Recipient(s) -- string or array of strings
    subject,             // Email subject line
    body,                // Email body (plain text or HTML)
    html,                // If true, body is treated as HTML (default: false)
    text,                // Plain text alternative (when body is HTML)
    cc,                  // CC recipient(s) -- string or array
    bcc,                 // BCC recipient(s) -- string or array
    replyTo,             // Reply-To address
    attachments,         // Array of file paths or objects
    headers              // Additional email headers (object)
});
```

---

5. Sending HTML Email with Text Fallback

Most emails should carry HTML with a plain text fallback. Email clients that cannot render HTML display the text version instead:

```
import { Messenger } from "tina4-nodejs";

const mailer = new Messenger();

const htmlBody = `
<html>
<body style="font-family: Arial, sans-serif; max-width: 600px; margin: 0 auto;">
    <div style="background: #1a1a2e; color: white; padding: 20px; text-align: center;">
        <h1 style="margin: 0;">Welcome to My Store!</h1>
    </div>
    <div style="padding: 20px;">
        <p>Hi Alice,</p>
        <p>Thank you for creating your account. We are excited to have you!</p>
        <p>Here is what you can do next:</p>
        <ul>
            <li>Browse our <a href="https://mystore.com/products">product catalog</a></li>
            <li>Set up your <a href="https://mystore.com/profile">profile</a></li>
            <li>Check out our <a href="https://mystore.com/deals">current deals</a></li>
        </ul>
        <p>If you have any questions, reply to this email and we will get back to you within 24
        <p>Cheers,<br>The My Store Team</p>
    </div>
    <div style="background: #f5f5f5; padding: 12px; text-align: center; font-size: 12px; color:
        <p>You received this because you signed up at mystore.com</p>
    </div>
</body>
</html>
`;

const textBody = [
    "Hi Alice,",
    "",
    "Thank you for creating your account. We are excited to have you!",
    "",
    "Here is what you can do next:",
    "- Browse our product catalog: https://mystore.com/products",
    "- Set up your profile: https://mystore.com/profile",
```

```
    "- Check out our current deals: https://mystore.com/deals",
    "",
    "If you have any questions, reply to this email.",
    "",
    "Cheers,",
    "The My Store Team"
].join("\n");

const result = await mailer.send({
    to: "alice@example.com",
    subject: "Welcome to My Store!",
    body: htmlBody,
    html: true,
    text: textBody
});
```

Pass `html: true` to tell Messenger the body contains HTML. The `text` parameter provides the plain text alternative. Messenger builds a `multipart/alternative` message that carries both versions.

---

6. Adding Attachments

Attach files by providing their paths:

```
import { Messenger } from "tina4-nodejs";

const mailer = new Messenger();

const result = await mailer.send({
    to: "accounting@example.com",
    subject: "Monthly Invoice #1042",
    body: "<h2>Invoice #1042</h2><p>Please find the invoice attached.</p>",
    html: true,
    attachments: [
        "/path/to/invoices/invoice-1042.pdf",
        "/path/to/reports/monthly-summary.csv"
    ]
});
```

Messenger reads each file, determines its MIME type, and encodes it for email transmission. Each entry can be a file path string or an object for more control.

Attachments with Custom Names and Binary Content

```
const result = await mailer.send({
    to: "alice@example.com",
    subject: "Your Export",
    body: "<p>Here is your data export.</p>",
    html: true,
    attachments: [
        {
            filename: "my-store-export.csv",
            content: csvBuffer,
            mime: "text/csv"
        }
    ]
});
```

The object format accepts `filename` (display name), `content` (Buffer or raw bytes), and `mime` (MIME type string). The recipient sees the file named `my-store-export.csv` regardless of how it was generated.

---

7. CC, BCC, and Reply-To

```
import { Messenger } from "tina4-nodejs";

const mailer = new Messenger();

const result = await mailer.send({
    to: "alice@example.com",
    subject: "Team Meeting Notes",
    body: "<p>Here are the notes from today's meeting.</p>",
    html: true,
    cc: ["bob@example.com", "charlie@example.com"],
    bcc: ["manager@example.com"],
    replyTo: "alice@example.com"
});
```

- **cc**: Array of email addresses to carbon copy. All recipients see CC addresses.
- **bcc**: Array of email addresses to blind carbon copy. Recipients cannot see BCC addresses.
- **replyTo**: When the recipient clicks "Reply", this address fills the "To" field instead of the "From" address.

Both `cc` and `bcc` accept a single string or an array of strings.

---

8. Custom Headers

Messenger supports two methods for adding headers. The `addHeader()` method sets a default header on all emails sent by that instance. The `headers` property on `send()` sets headers for a single email.

Default Headers on an Instance

```
import { Messenger } from "tina4-nodejs";

const mailer = new Messenger();
mailer.addHeader("X-App-Name", "My Store");
mailer.addHeader("X-Environment", "production");

// Every email from this instance carries both headers
await mailer.send({ to: "alice@example.com", subject: "Test", body: "Hello" });
```

Per-Email Headers

```
const result = await mailer.send({
    to: "customer@example.com",
    subject: "Your Support Ticket #123",
    body: "We are looking into your issue.",
    replyTo: "support@mystore.com",
    headers: {
        "X-Ticket-Id": "123",
        "X-Priority": "1",
        "X-Mailer": "Tina4 Messenger"
    }
});
```

Per-email headers merge with default headers. If both define the same key, the per-email value wins.

Custom headers serve several purposes. Tracking headers like `X-Ticket-Id` let you correlate emails with support tickets. Priority headers influence some email clients' display. Bulk-sending headers like `Precedence: bulk` help mail servers classify newsletters.

Header	Purpose
`X-Priority`	Email priority (1=high, 3=normal, 5=low)
`X-Mailer`	Identifies the sending application
`List-Unsubscribe`	One-click unsubscribe link (required for bulk email)
`X-Entity-Ref-ID`	Prevents threading in some email clients
`References`	Controls email threading

---

9. Reading Inbox via IMAP

Messenger reads email through IMAP. Configure the IMAP server in `.env`:

```
TINA4_MAIL_IMAP_HOST=imap.example.com
TINA4_MAIL_IMAP_PORT=993
TINA4_MAIL_IMAP_USERNAME=support@example.com
TINA4_MAIL_IMAP_PASSWORD=your-imap-password
TINA4_MAIL_IMAP_ENCRYPTION=ssl
```

You can also override the IMAP host and port in the constructor:

```
const mailer = new Messenger({ imapHost: "imap.gmail.com", imapPort: 993 });
```

Listing Inbox Messages

The `getInbox()` method fetches message headers from the mailbox:

```
import { Router, Messenger } from "tina4-nodejs";

Router.get("/api/inbox", async (req, res) => {
    const mailer = new Messenger();

    const emails = await mailer.getInbox({ limit: 20, offset: 0 });

    const messages = emails.map(email => ({
        uid: email.uid,
        from: email.from,
        subject: email.subject,
        date: email.date,
        snippet: email.snippet,
        seen: email.seen
    }));

    return res.json({ messages, count: messages.length });
});
```

```
curl http://localhost:7148/api/inbox
```

```
{
  "messages": [
    {
      "uid": "12345",
      "from": "customer@example.com",
      "subject": "Order question",
      "date": "2026-03-22T10:30:00+00:00",
      "snippet": "Hi, I have a question about my recent order...",
      "seen": false
    }
  ],
  "count": 1
}
```

The `getInbox()` method returns messages newest-first. Each message contains `uid`, `subject`, `from`, `to`, `date`, `snippet` (first 150 characters of the body), and `seen` (boolean).

Reading a Specific Message

```
Router.get("/api/inbox/:uid", async (req, res) => {
    const mailer = new Messenger();
    const uid = req.params.uid;

    const email = await mailer.read(uid, { markRead: true });

    if (!email) {
        return res.status(404).json({ error: "Email not found" });
    }

    return res.json({
        uid: email.uid,
        from: email.from,
        to: email.to,
        cc: email.cc,
        subject: email.subject,
        date: email.date,
        bodyHtml: email.bodyHtml,
        bodyText: email.bodyText,
        attachments: (email.attachments ?? []).map(a => ({
            filename: a.filename,
            size: a.size,
            contentType: a.contentType
        }))
    });
});
```

The `read()` method fetches the full message including body and attachments. Pass `{ markRead: false }` to leave the message unread.

Searching Messages

```
Router.get("/api/inbox/search", async (req, res) => {
    const mailer = new Messenger();

    const results = await mailer.search({
        subject: req.query.q as string,
        sender: req.query.from as string,
        unseenOnly: req.query.unread === "true",
        limit: 20
    });

    return res.json({ messages: results, count: results.length });
});
```

The `search()` method accepts `subject`, `sender`, `since` (date string), `before`, and `unseenOnly` as filters. All filters combine with AND logic.

Other IMAP Operations

```
const mailer = new Messenger();

// Count unread messages
const count = await mailer.getUnreadCount();

// Mark a message as read
await mailer.markRead("12345");

// Mark a message as unread
await mailer.markUnread("12345");

// Delete a message
await mailer.deleteEmail("12345");

// List all mailbox folders
const folders = await mailer.getFolders();
// ["INBOX", "Sent", "Drafts", "Trash", "Spam"]
```

Read from a Specific Folder

```
const sent = await mailer.getInbox({ folder: "Sent", limit: 10 });
```

Testing IMAP Connectivity

```
const mailer = new Messenger();
const result = await mailer.testImapConnection();
if (result.success) {
    console.log("IMAP connection works");
} else {
    console.log(`IMAP failed: ${result.error}`);
}
```

---

10. Dev Mailbox Capture

Messenger captures outgoing mail when no SMTP host exists or when `TINA4_MAIL_CAPTURE=true`. `TINA4_DEBUG` only makes the dev dashboard available; it does not change delivery.

Navigate to `/__dev` and look for the "Mail" section. You see:

- Every email "sent" during the current session
- The To, CC, and BCC addresses
- The subject and body (both HTML and plain text)
- Attachments (viewable inline)
- The timestamp

Use `createMessenger()` to load the environment policy and capture without an SMTP host:

```
import { createMessenger } from "tina4-nodejs";

const mailer = createMessenger();
// No SMTP host or TINA4_MAIL_CAPTURE=true: captures locally
```

```
// SMTP host configured and capture off: sends through SMTP
```

Sending from a development environment

Configure an SMTP host and leave `TINA4_MAIL_CAPTURE` unset or false. The `TINA4_DEBUG` value does not matter to delivery. To test SMTP without reaching customers, set `TINA4_MAIL_REDIRECT_TO` to controlled inboxes.

---

11. Using Templates for Email Content

Hardcoded HTML in string literals is ugly and hard to maintain. Templates fix this.

Create `src/templates/emails/welcome.html`:

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="UTF-8">
</head>
<body style="font-family: Arial, sans-serif; max-width: 600px; margin: 0 auto; background: #f5f5
    <div style="background: #1a1a2e; color: white; padding: 24px; text-align: center; border-rad
        <h1 style="margin: 0; font-size: 24px;">Welcome, {{ name }}!</h1>
    </div>
    <div style="background: white; padding: 24px; border-radius: 0 0 8px 8px;">
        <p>Hi {{ name }},</p>
        <p>Your account has been created successfully. Here are your details:</p>

        <table style="width: 100%; border-collapse: collapse; margin: 16px 0;">
            <tr>
                <td style="padding: 8px; border-bottom: 1px solid #eee; font-weight: bold;">Emai
                <td style="padding: 8px; border-bottom: 1px solid #eee;">{{ email }}</td>
            </tr>
            <tr>
                <td style="padding: 8px; border-bottom: 1px solid #eee; font-weight: bold;">Acco
                <td style="padding: 8px; border-bottom: 1px solid #eee;">#{{ user_id }}</td>
            </tr>
            <tr>
                <td style="padding: 8px; border-bottom: 1px solid #eee; font-weight: bold;">Sign
                <td style="padding: 8px; border-bottom: 1px solid #eee;">{{ signed_up_at }}</td>
            </tr>
        </table>

        <p>Get started by exploring:</p>
        <ul>
            <li><a href="{{ base_url }}/products" style="color: #1a1a2e;">Our product catalog</a
            <li><a href="{{ base_url }}/profile" style="color: #1a1a2e;">Your profile settings</
        </ul>

        {% if promo_code %}
            <div style="background: #d4edda; padding: 16px; border-radius: 4px; margin: 16px 0;"
                <strong>Special offer!</strong> Use code <code>{{ promo_code }}</code> for 10% o
            </div>
        {% endif %}

        <p>Cheers,<br>The {{ app_name }} Team</p>
    </div>
```

```
    <div style="text-align: center; padding: 12px; color: #888; font-size: 12px;">
        <p>You received this because you signed up at {{ app_name }}.</p>
        <p><a href="{{ base_url }}/unsubscribe?token={{ unsubscribe_token }}" style="color: #888
    </div>
</body>
</html>
```

Rendering and Sending

```
import { Router, Messenger } from "tina4-nodejs";
import { Frond } from "tina4-nodejs/frond";
import crypto from "crypto";

Router.post("/api/register", async (req, res) => {
    const body = req.body;

    // Create user (database logic)
    const userId = 42;

    // Render the email template
    const emailData = {
        name: body.name,
        email: body.email,
        user_id: userId,
        signed_up_at: new Date().toLocaleDateString("en-US", { year: "numeric", month: "long", d
        base_url: process.env.APP_URL ?? "http://localhost:7148",
        app_name: "My Store",
        promo_code: "WELCOME10",
        unsubscribe_token: crypto.randomBytes(16).toString("hex")
    };

    const htmlBody = await Frond.render("emails/welcome.html", emailData);

    // Send the email
    const mailer = new Messenger();
    const result = await mailer.send({
        to: body.email,
        subject: `Welcome to My Store, ${body.name}!`,
        body: htmlBody,
        html: true,
        text: `Hi ${body.name},\n\nWelcome to My Store! Your account (#${userId}) has been creat
    });

    return res.status(201).json({
        message: "Registration successful",
        email_sent: result.success,
        user_id: userId
    });
});
```

```
curl -X POST http://localhost:7148/api/register \
  -H "Content-Type: application/json" \
  -d '{"name": "Alice", "email": "alice@example.com", "password": "securePass123"}'
```

```
{
  "message": "Registration successful",
  "email_sent": true,
  "user_id": 42
}
```

With `TINA4_MAIL_CAPTURE=true`, the email stays in the dev mailbox instead of reaching a real inbox. Enable `TINA4_DEBUG=true` to inspect the rendered HTML and verify the template values in the dashboard.

---

12. Sending Email via Queues

In production, never send email inside a route handler. The SMTP call blocks the response. Use the queue system from Chapter 12:

```
import { Router, Queue, Messenger } from "tina4-nodejs";
import { Frond } from "tina4-nodejs/frond";

const emailQueue = new Queue({ topic: "emails" });

// In the route handler, just queue the email
Router.post("/api/register", async (req, res) => {
    const userId = 42; // Simulated

    emailQueue.push({
        template: "emails/welcome.html",
        to: req.body.email,
        subject: `Welcome to My Store, ${req.body.name}!`,
        data: {
            name: req.body.name,
            email: req.body.email,
            user_id: userId,
            signed_up_at: "March 22, 2026",
            base_url: "http://localhost:7148",
            app_name: "My Store",
            promo_code: "WELCOME10"
        }
    });

    return res.status(201).json({ message: "Registration successful", user_id: userId });
});

// The consumer sends the actual email
emailQueue.process(async (job) => {
    const { template, to, subject, data } = job.payload as any;

    const frond = new Frond("src/templates");
    const htmlBody = frond.render(template, data);

    const mailer = new Messenger();
    const result = await mailer.send({ to, subject, body: htmlBody, html: true });

    if (!result.success) {
        console.log(`Email failed: ${result.message}`);
        job.fail(result.message);
        return;
    }

    console.log(`Email sent to ${to}`);
    job.complete();
});
```

The route handler returns in under 50 milliseconds. The queue worker sends the email on its own timeline. If the SMTP server is down, retries happen automatically.

---

13. Exercise: Build a Contact Form with Email Notification

Build a contact form that sends an email notification when submitted.

Requirements

- Create a `GET /contact` page that renders a contact form with fields: name, email, subject, and message
- Create a `POST /contact` endpoint that:
- Validates all fields are present
- Sends an email notification to the site admin (`admin@example.com`)
- The email should include all form fields, formatted in HTML
- Shows a flash message on success
- Redirects back to the contact page
- Create an email template at `src/templates/emails/contact-notification.html` that formats the contact submission

Test with:

```
# View the form
curl http://localhost:7148/contact

# Submit the form
curl -X POST http://localhost:7148/contact \
  -H "Content-Type: application/json" \
  -d '{"name": "Bob", "email": "bob@example.com", "subject": "Product inquiry", "message": "Do y

# Check the dev dashboard for the intercepted email
# Navigate to http://localhost:7148/__dev
```

---

14. Solution

Create `src/templates/emails/contact-notification.html`:

```
<!DOCTYPE html>
<html>
<head><meta charset="UTF-8"></head>
<body style="font-family: Arial, sans-serif; max-width: 600px; margin: 0 auto;">
    <div style="background: #1a1a2e; color: white; padding: 16px; text-align: center;">
        <h2 style="margin: 0;">New Contact Form Submission</h2>
    </div>
    <div style="padding: 20px; background: white;">
        <table style="width: 100%; border-collapse: collapse;">
            <tr>
                <td style="padding: 10px; border-bottom: 1px solid #eee; font-weight: bold; widt
```

```
                <td style="padding: 10px; border-bottom: 1px solid #eee;">{{ name }} ({{ email }
            </tr>
            <tr>
                <td style="padding: 10px; border-bottom: 1px solid #eee; font-weight: bold;">Sub
                <td style="padding: 10px; border-bottom: 1px solid #eee;">{{ subject }}</td>
            </tr>
            <tr>
                <td style="padding: 10px; border-bottom: 1px solid #eee; font-weight: bold;">Dat
                <td style="padding: 10px; border-bottom: 1px solid #eee;">{{ submitted_at }}</td
            </tr>
        </table>

        <div style="margin-top: 16px; padding: 16px; background: #f8f9fa; border-radius: 4px;">
            <h3 style="margin-top: 0;">Message</h3>
            <p style="white-space: pre-wrap;">{{ message }}</p>
        </div>

        <p style="margin-top: 16px; color: #888; font-size: 12px;">
            Reply directly to this email to respond to {{ name }} at {{ email }}.
        </p>
    </div>
</body>
</html>
```

Create `src/templates/contact.html`:

```
{% extends "base.html" %}

{% block title %}Contact Us{% endblock %}

{% block content %}
    <h1>Contact Us</h1>

    {% if flash %}
        <div style="padding: 12px; border-radius: 4px; margin-bottom: 16px;
            {% if flash.type == 'success' %}background: #d4edda; color: #155724;{% endif %}
            {% if flash.type == 'error' %}background: #f8d7da; color: #721c24;{% endif %}">
            {{ flash.message }}
        </div>
    {% endif %}

    <form method="POST" action="/contact" style="max-width: 500px;">
        <div style="margin-bottom: 12px;">
            <label for="name" style="display: block; margin-bottom: 4px; font-weight: bold;">Nam
            <input type="text" name="name" id="name" required
                   style="width: 100%; padding: 8px; border: 1px solid #ddd; border-radius: 4px;
        </div>

        <div style="margin-bottom: 12px;">
            <label for="email" style="display: block; margin-bottom: 4px; font-weight: bold;">Em
            <input type="email" name="email" id="email" required
                   style="width: 100%; padding: 8px; border: 1px solid #ddd; border-radius: 4px;
        </div>

        <div style="margin-bottom: 12px;">
            <label for="subject" style="display: block; margin-bottom: 4px; font-weight: bold;">
            <input type="text" name="subject" id="subject" required
                   style="width: 100%; padding: 8px; border: 1px solid #ddd; border-radius: 4px;
        </div>
```

```
        <div style="margin-bottom: 12px;">
            <label for="message" style="display: block; margin-bottom: 4px; font-weight: bold;">
            <textarea name="message" id="message" rows="6" required
                      style="width: 100%; padding: 8px; border: 1px solid #ddd; border-radius: 4
        </div>

        <button type="submit"
                style="padding: 10px 20px; background: #1a1a2e; color: white; border: none; bord
            Send Message
        </button>
    </form>
{% endblock %}
```

Create `src/routes/contact.ts`:

```
import { Router, Messenger } from "tina4-nodejs";
import { Frond } from "tina4-nodejs/frond";

Router.get("/contact", async (req, res) => {
    const flash = req.session._flash ?? null;
    delete req.session._flash;
    return res.render("contact.html", { flash });
});

Router.post("/contact", async (req, res) => {
    const body = req.body;

    // Validate
    const errors: string[] = [];
    if (!body.name) errors.push("Name is required");
    if (!body.email) errors.push("Email is required");
    if (!body.subject) errors.push("Subject is required");
    if (!body.message) errors.push("Message is required");

    if (errors.length > 0) {
        req.session._flash = { type: "error", message: `Please fill in all fields: ${errors.join
        return res.redirect("/contact");
    }

    // Render the email template
    const htmlBody = await Frond.render("emails/contact-notification.html", {
        name: body.name,
        email: body.email,
        subject: body.subject,
        message: body.message,
        submitted_at: new Date().toLocaleString()
    });

    // Send the email
    const mailer = new Messenger();
    const adminEmail = process.env.ADMIN_EMAIL ?? "admin@example.com";

    const result = await mailer.send({
        to: adminEmail,
        subject: `Contact Form: ${body.subject}`,
        body: htmlBody,
        html: true,
        replyTo: body.email,
        text: `Contact form submission from ${body.name} (${body.email}):\n\nSubject: ${body.sub
```

```
    });

    if (result.success) {
        req.session._flash = {
            type: "success",
            message: "Thank you for your message! We will get back to you shortly."
        };
    } else {
        req.session._flash = {
            type: "error",
            message: "Sorry, there was a problem sending your message. Please try again later."
        };
    }

    return res.redirect("/contact");
});
```

Testing:

- Open `http://localhost:7148/contact` in your browser
- Fill in the form and submit
- You should see a green "Thank you" flash message
- Open `http://localhost:7148/__dev` to see the intercepted email
- The email shows the sender details, subject, message, and formatted HTML

API test:

```
curl -X POST http://localhost:7148/contact \
  -H "Content-Type: application/json" \
  -d '{"name": "Bob", "email": "bob@example.com", "subject": "Product inquiry", "message": "Do y
  -c cookies.txt -b cookies.txt
```

The response is a `302` redirect to `/contact`. Follow the redirect to see the flash message:

```
curl http://localhost:7148/contact -b cookies.txt
```

The HTML response includes the success flash message.

---

15. Gotchas

1. Gmail Blocks "Less Secure" Apps

Problem: Sending via Gmail fails with "Authentication failed" or "Username and Password not accepted".

Cause: Gmail blocks SMTP access from apps that do not use OAuth2 by default. Your regular password will not work when two-factor authentication is enabled.

Fix: Generate an "App Password" in your Google Account settings (Security > 2-Step Verification > App Passwords). Use this 16-character password as `TINA4_MAIL_PASSWORD`. This is separate from your regular Google password.

2. Emails Go to Spam

Problem: Emails land in the recipient's spam folder.

Cause: Your sending domain lacks proper DNS records (SPF, DKIM, DMARC) or you send from a free email provider (Gmail, Yahoo).

Fix: Use a dedicated sending domain with proper DNS records. Set up SPF, DKIM, and DMARC records. Use a transactional email service -- Mailgun, SendGrid, or Amazon SES -- that handles email reputation for you.

3. HTML Email Looks Broken

Problem: The email looks fine in Gmail but broken in Outlook or Apple Mail.

Cause: Email clients have different HTML/CSS support. CSS flexbox, grid, and many modern properties do not work in email.

Fix: Use inline styles (not external stylesheets or `<style>` blocks). Use table-based layouts for complex designs. Test with an email preview tool. Keep it simple -- most transactional emails do not need elaborate designs.

4. Attachment File Not Found

Problem: `mailer.send()` returns an error about a missing file.

Cause: The attachment path is relative or incorrect. The file does not exist at the specified location.

Fix: Use absolute paths for attachments. Verify the file exists before calling `send()`: `if (!fs.existsSync(path)) { ... }`. If the file is generated dynamically (a PDF, for example), make sure the generation completes before sending.

5. Dev Mode Silently Intercepts Emails

Problem: You configured SMTP but no emails arrive. No errors either.

Cause: `TINA4_MAIL_CAPTURE=true` or a missing SMTP host sends mail to the dev mailbox. `TINA4_DEBUG` does not control delivery.

Fix: Check `/__dev` for captured mail. To use SMTP, configure `TINA4_MAIL_HOST` and remove `TINA4_MAIL_CAPTURE`. On staging, set `TINA4_MAIL_REDIRECT_TO` so tests cannot reach customer addresses.

6. Email Template Variables Not Substituted

Problem: The email body shows `{{ name }}` literally instead of the user's name.

Cause: You passed the raw template file content instead of rendering it through the template engine.

Fix: Use `Frond.render("emails/template.html", data)` to render the template with variables substituted. Do not read the file with `fs.readFileSync()` -- that gives you the raw template source.

---

7. Connection Timeout on Send

Problem: `mailer.send()` hangs for 30 seconds and then fails with a timeout error.

Cause: The SMTP server is unreachable from your network. The port may be blocked by a firewall, or the hostname may be wrong.

Fix: Test SMTP connectivity with `mailer.testConnection()`. Verify the hostname, port, and encryption settings. Check that your firewall allows outbound connections on the SMTP port. If you sit behind a corporate firewall, port 587 or 465 might be blocked -- ask your network administrator.

8. IMAP Host Not Configured

Problem: Calling `mailer.getInbox()` throws `MessengerError: IMAP host not configured`.

Cause: You did not set `TINA4_MAIL_IMAP_HOST` in `.env` or pass `imapHost` to the constructor.

Fix: Add `TINA4_MAIL_IMAP_HOST=imap.example.com` and `TINA4_MAIL_IMAP_PORT=993` to `.env`. The IMAP host is separate from the SMTP host -- many providers use different hostnames for sending and reading.

Frontend Integration

1. Beyond JSON APIs

Your API returns perfect JSON. Now someone has to build the interface.

Tina4 ships two frontend tools: **tina4css** (a utility CSS framework) and **frond.js** (a reactive JavaScript library). Both arrive with every project. No npm installs. No build tools. No webpack.

This chapter ends with a complete admin dashboard. Sidebar. Navigation. Cards. Tables. Modals. Dark mode. Progress bars. AJAX-driven user management. Zero npm dependencies.

---

2. What Ships with Tina4

Run `tina4 init nodejs`. Several files appear in your project:

```
src/public/
■■■ css/
■   ■■■ tina4.css        # The CSS framework
■■■ js/
■   ■■■ tina4.min.js     # Core AJAX utilities
■   ■■■ frond.min.js     # Template engine client-side helpers
■   ■■■ tina4js.min.js   # Reactive frontend framework (tina4-js)
■■■ scss/
    ■■■ tina4.scss       # SCSS source (optional, for customization)
```

Include them in any template:

```
<link rel="stylesheet" href="/css/tina4.css">
<script src="/js/tina4.min.js"></script>
<script src="/js/frond.min.js"></script>
```

Include only what you need. See section 7 for the full JavaScript API reference. No CDN. No package manager. No version conflicts.

---

3. The Grid System

tina4css uses a 12-column responsive grid. The class names match Bootstrap. Know Bootstrap? You know tina4css.

```
<div class="container">
    <div class="row">
        <div class="col-md-4">
            <p>One third</p>
        </div>
        <div class="col-md-4">
            <p>One third</p>
        </div>
        <div class="col-md-4">
            <p>One third</p>
        </div>
```

```
    </div>
</div>
```

Breakpoints:

Class Prefix	Screen Width	Typical Device
`col-`	All sizes	Phones and up
`col-sm-`	>= 576px	Large phones
`col-md-`	>= 768px	Tablets
`col-lg-`	>= 992px	Desktops
`col-xl-`	>= 1200px	Large desktops

Columns stack vertically on screens smaller than their breakpoint. A `col-md-6` element takes half the row on tablets and up, full width on phones.

---

4. Components

Navbar

```
<nav class="navbar navbar-dark bg-dark">
    <div class="container">
        <a class="navbar-brand" href="/">My Dashboard</a>
        <ul class="navbar-nav">
            <li class="nav-item"><a class="nav-link" href="/dashboard">Dashboard</a></li>
            <li class="nav-item"><a class="nav-link" href="/products">Products</a></li>
            <li class="nav-item"><a class="nav-link" href="/settings">Settings</a></li>
        </ul>
    </div>
</nav>
```

Use `navbar-light bg-light` for a light theme, or `navbar-dark bg-primary` for a colored background.

Cards

```
<div class="card">
    <div class="card-header">Monthly Revenue</div>
    <div class="card-body">
        <h2 class="card-title">$12,450</h2>
        <p class="card-text">Up 12% from last month</p>
    </div>
    <div class="card-footer text-muted">Updated 5 minutes ago</div>
</div>
```

Buttons

```
<button class="btn btn-primary">Save</button>
<button class="btn btn-secondary">Cancel</button>
<button class="btn btn-danger">Delete</button>
<button class="btn btn-success">Publish</button>
<button class="btn btn-outline-primary">Outlined</button>
<button class="btn btn-sm btn-primary">Small</button>
<button class="btn btn-lg btn-primary">Large</button>
```

Tables

```
<table class="table table-striped table-hover">
    <thead>
        <tr>
            <th>Name</th>
            <th>Category</th>
            <th>Price</th>
            <th>Status</th>
        </tr>
    </thead>
    <tbody>
        <tr>
            <td>Wireless Keyboard</td>
            <td>Electronics</td>
            <td>$79.99</td>
            <td><span class="badge bg-success">In Stock</span></td>
        </tr>
        <tr>
            <td>Standing Desk</td>
            <td>Furniture</td>
            <td>$549.99</td>
            <td><span class="badge bg-danger">Out of Stock</span></td>
        </tr>
    </tbody>
</table>
```

Table variants: `table-bordered`, `table-striped`, `table-hover`, `table-sm` (compact), `table-responsive` (wraps in a scrollable container on small screens). Mix and match.

Alerts

```
<div class="alert alert-success">Product created.</div>
<div class="alert alert-danger">Failed to save changes.</div>
<div class="alert alert-warning alert-dismissible">
    Your trial expires in 3 days.
    <button type="button" class="close" data-dismiss="alert">&times;</button>
</div>
<div class="alert alert-info">A new version is available.</div>
```

Modals

```
<button class="btn btn-primary" data-toggle="modal" data-target="#confirmModal">Delete Product</

<div class="modal" id="confirmModal">
    <div class="modal-dialog">
        <div class="modal-content">
            <div class="modal-header">
                <h5 class="modal-title">Confirm Delete</h5>
                <button class="btn-close" data-dismiss="modal"></button>
            </div>
            <div class="modal-body">
                <p>Are you sure you want to delete this product? This action cannot be undone.</
            </div>
            <div class="modal-footer">
                <button class="btn btn-secondary" data-dismiss="modal">Cancel</button>
                <button class="btn btn-danger">Delete</button>
            </div>
        </div>
    </div>
</div>
```

tina4css includes the JavaScript for modal toggling. No jQuery required.

Progress Bars

Progress bars visualize completion, upload status, or system health. The outer `progress` container holds a `progress-bar` fill element.

```
<div class="progress">
    <div class="progress-bar bg-success" style="width: 75%">75%</div>
</div>

<div class="progress">
    <div class="progress-bar bg-warning progress-bar-striped" style="width: 45%">
        45% - Uploading...
    </div>
</div>
```

Set the width with inline `style`. Add `bg-success`, `bg-info`, `bg-warning`, or `bg-danger` for color. Add `progress-bar-striped` for animated stripes.

Badges

```
<span class="badge bg-primary">Primary</span>
<span class="badge bg-success">Active</span>
<span class="badge bg-warning">Pending</span>
<span class="badge bg-danger">Overdue</span>
<span class="badge bg-info">Info</span>
<span class="badge bg-dark">Dark</span>
```

Forms

```
<form>
    <div class="form-group">
        <label for="name" class="form-label">Product Name</label>
        <input type="text" class="form-control" id="name" placeholder="Enter product name">
    </div>

    <div class="form-group">
        <label for="category" class="form-label">Category</label>
        <select class="form-control" id="category">
            <option value="">Select a category</option>
            <option value="electronics">Electronics</option>
            <option value="furniture">Furniture</option>
        </select>
    </div>

    <div class="form-group">
        <label for="description" class="form-label">Description</label>
        <textarea class="form-control" id="description" rows="4"></textarea>
    </div>

    <div class="form-check">
        <input type="checkbox" class="form-check-input" id="featured">
        <label class="form-check-label" for="featured">Featured product</label>
    </div>

    <button type="submit" class="btn btn-primary mt-3">Save Product</button>
</form>
```

Utility Classes

```
<p class="text-center">Centered</p>
<div class="mt-4">Margin top</div>
<div class="p-3">Padding</div>
<span class="text-muted">Gray text</span>
```

---

5. SCSS Customization

The default tina4css works out of the box. To customize colors, fonts, or spacing, edit the SCSS source.

Edit `src/public/scss/tina4.scss`:

```
// Override variables before importing the framework
$primary: #2d6a4f;
$secondary: #52b788;
$dark: #1b4332;
$font-family-base: 'Inter', sans-serif;
$border-radius: 8px;

// Import the framework
@import 'tina4-base';
```

Compile SCSS to CSS:

```
tina4 scss
```

```
Compiling SCSS...
  src/public/scss/tina4.scss -> src/public/css/tina4.css
Done (0.12s)
```

The compiled CSS replaces the default `tina4.css`. Your custom colors and fonts take effect across the entire application.

Live SCSS Compilation

During development, run SCSS compilation in watch mode:

```
tina4 scss --watch
```

Every save to a `.scss` file triggers a recompile. Combined with Tina4's live reload, changes appear in the browser within a second.

---

6. frond.js -- The JavaScript Helper

`frond.js` is a lightweight JavaScript library that ships with Tina4. It handles AJAX requests, form submission, JWT token management, and loading indicators. No jQuery. No Axios. No other dependency.

AJAX Requests

```
// GET request
frond.get("/api/products", function (data) {
    console.log("Products:", data);
});

// GET with error handling
frond.get("/api/products", function (data) {
    console.log(data);
}, function (error) {
    console.error("Failed:", error);
});

// POST request
frond.post("/api/products", {
    name: "New Product",
    price: 29.99
}, function (data) {
    console.log("Created:", data);
});

// PUT request
frond.put("/api/products/1", {
    name: "Updated Product"
}, function (data) {
    console.log("Updated:", data);
});

// DELETE request
frond.delete("/api/products/1", function (data) {
    console.log("Deleted:", data);
});
```

Form Submission via AJAX

```
<form id="product-form" data-frond-submit="/api/products" data-frond-method="POST">
    <input type="text" name="name" placeholder="Product name">
    <input type="number" name="price" placeholder="Price">
    <button type="submit">Create</button>
</form>

<script src="/js/frond.min.js"></script>
<script>
    frond.onFormSuccess("product-form", function (data) {
        alert("Product created: " + data.name);
    });

    frond.onFormError("product-form", function (error) {
        alert("Error: " + error.message);
    });
</script>
```

The `data-frond-submit` attribute tells frond.js to intercept the form submission and send it as an AJAX request. No page reload. frond.js serializes all form fields as JSON.

Token Management

frond.js manages JWT tokens. Store a token after login, and frond.js attaches it to every request.

```
// Store the token (usually after login)
frond.setToken("eyJhbGciOiJIUzI1NiIs...");

// All subsequent requests include:
// Authorization: Bearer eyJhbGciOiJIUzI1NiIs...
frond.get("/api/profile", function (data) {
    console.log("Profile:", data);
});

// Clear the token (logout)
frond.clearToken();
```

frond.js stores the token in `localStorage` and includes it as a `Bearer` token in the `Authorization` header on every request.

Loading Indicators

frond.js can show and hide a loading element during AJAX requests. Pass a CSS selector in the options object.

```
// Show a loading state while fetching
frond.get("/api/products", function (data) {
    renderProducts(data.products);
}, null, {
    loading: "#loadingSpinner"  // CSS selector for loading element
});
```

The element with id `loadingSpinner` appears while the request flies and disappears when it completes. Pair it with a spinner or "Loading..." text in your HTML:

```
<div id="loadingSpinner" class="text-center p-4" style="display: none;">
    Loading...
</div>
```

frond.js toggles `display: block` and `display: none` on the element. No extra CSS needed.

WebSocket (Covered in Chapter 23)

```
const ws = frond.ws("/ws/chat/general");
ws.on("message", function (data) {
    console.log("Message:", JSON.parse(data));
});
```

Connections drop. frond.js reconnects with exponential backoff. If the server restarts or the network blips, the client reconnects without intervention.

---

7. JavaScript API Reference

Tina4 ships three JavaScript files. Each serves a different purpose. Use them independently or together.

Including the Scripts

```
<script src="/js/tina4.min.js"></script>
<script src="/js/frond.min.js"></script>
<script src="/js/tina4js.min.js"></script>
```

All three live in `src/public/js/` and are served from `/js/`. Include only what you need.

7.1 tina4.min.js -- Core Utilities

Low-level helpers for AJAX page loading and form submission. Use this when you want simple dynamic page updates without a full framework.

`loadPage(url, targetId)`

Fetches HTML from `url` and injects it into the element with the given `id`.

```
<nav>
    <a href="#" onclick="loadPage('/dashboard', 'content')">Dashboard</a>
    <a href="#" onclick="loadPage('/settings', 'content')">Settings</a>
</nav>
<div id="content"><!-- pages load here --></div>

<script src="/js/tina4.min.js"></script>
```

`saveForm(formId, url, method)`

Serializes a form and submits it via AJAX. Prevents the default page reload.

```
<form id="product-form">
    <input type="text" name="name" placeholder="Product name">
    <input type="number" name="price" placeholder="Price">
    <button type="button" onclick="saveForm('product-form', '/api/products', 'POST')">
        Save
    </button>
</form>
```

`sendRequest(url, method, data, callback)`

Generic AJAX helper for any HTTP method. Returns the response to a callback function.

```
sendRequest("/api/products", "GET", null, function (response) {
    console.log("Products:", JSON.parse(response));
});

sendRequest("/api/products", "POST", { name: "Widget", price: 9.99 }, function (response) {
    console.log("Created:", JSON.parse(response));
});
```

7.2 frond.min.js -- Template Engine Client-Side Helpers

A companion to the Frond template engine. Handles AJAX form interception, WebSocket connections with auto-reconnect, JWT token refresh, and dynamic template loading.

AJAX Form Handling

Forms with `data-frond-submit` are intercepted. No page reload. No boilerplate.

```
<form id="login-form" data-frond-submit="/api/login" data-frond-method="POST">
    <input type="text" name="username" placeholder="Username">
    <input type="password" name="password" placeholder="Password">
    <button type="submit">Log In</button>
</form>

<script src="/js/frond.min.js"></script>
<script>
    frond.onFormSuccess("login-form", function (data) {
        frond.setToken(data.token);
        window.location.href = "/dashboard";
    });
</script>
```

WebSocket Auto-Reconnect

```
const ws = frond.ws("/ws/notifications");
ws.on("message", function (data) {
    const notification = JSON.parse(data);
    alert(notification.text);
});
// If the server restarts or the network blips, frond.js reconnects.
```

Token Refresh

JWT tokens stored via `frond.setToken()` are attached to every request. When a token expires, frond.js triggers a refresh before retrying the request.

```
frond.setToken("eyJhbGciOiJIUzI1NiIs...");
// All subsequent frond.get/post/put/delete calls include the token.
// When the token expires, frond.js calls the refresh endpoint.
```

Dynamic Template Loading

Load server-rendered Frond templates into any element without a full page reload.

```
frond.loadTemplate("/templates/user-card", { userId: 42 }, "user-panel");
```

```
// Fetches the rendered template and injects it into #user-panel.
```

7.3 tina4js.min.js -- Reactive Frontend Framework

A standalone reactive framework for building interactive client-side applications. Provides signals, computed values, effects, Web Components, client-side routing, and built-in fetch and WebSocket wrappers. This is the **tina4-js** project.

Reactive State: `signal()`, `computed()`, `effect()`

```
import { signal, computed, effect } from "/js/tina4js.min.js";

const count = signal(0);
const doubled = computed(() => count.value * 2);

effect(() => {
    console.log(`Count: ${count.value}, Doubled: ${doubled.value}`);
});

count.value = 5; // logs "Count: 5, Doubled: 10"
```

DOM Rendering: `html` Tagged Template

```
import { signal, html } from "/js/tina4js.min.js";

const name = signal("World");

const app = html`
    <div>
        <h1>Hello, ${name}!</h1>
        <input value="${name}" oninput="${(e) => name.value = e.target.value}" />
    </div>
`;

document.getElementById("app").append(app);
```

Web Components: `Tina4Element`

```
import { Tina4Element, signal, html } from "/js/tina4js.min.js";

class CounterButton extends Tina4Element {
    setup() {
        this.count = signal(0);
    }
    render() {
        return html`
            <button onclick="${() => this.count.value++}">
                Clicked ${this.count} times
            </button>
        `;
    }
}
customElements.define("counter-button", CounterButton);
```

Use it in HTML:

```
<counter-button></counter-button>
<script type="module" src="/js/counter-button.js"></script>
```

Fetch Wrapper: `api()`

```
import { api } from "/js/tina4js.min.js";

const products = await api("/api/products");            // GET
await api("/api/products", { method: "POST", body: { name: "Widget" } });
```

WebSocket Client: `ws()`

```
import { ws } from "/js/tina4js.min.js";

const socket = ws("/ws/chat");
socket.on("message", (data) => console.log(data));
socket.send({ text: "Hello" });
```

Client-Side Routing: `route()`, `navigate()`

```
import { route, navigate, html } from "/js/tina4js.min.js";

route("/", () => html`<h1>Home</h1>`);
route("/about", () => html`<h1>About</h1>`);

// Navigate programmatically
navigate("/about");
```

---

8. Building an Admin Dashboard

A complete admin dashboard. The kind of page that powers the backend of every web application.

Base Template

Create `src/templates/base.html`:

```
<!DOCTYPE html>
<html lang="en" data-theme="light">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>{% block title %}Dashboard{% endblock %}</title>
    <link rel="stylesheet" href="/css/tina4.css">
    <script>
        var t = localStorage.getItem("theme");
        if (t) document.documentElement.setAttribute("data-theme", t);
    </script>
</head>
<body>
    <nav class="navbar navbar-dark bg-dark">
        <div class="container-fluid">
            <a class="navbar-brand" href="/admin">TaskFlow Admin</a>
            <ul class="navbar-nav">
                <li class="nav-item"><a class="nav-link" href="/admin">Dashboard</a></li>
                <li class="nav-item"><a class="nav-link" href="/admin/products">Products</a></li
                <li class="nav-item"><a class="nav-link" href="/admin/users">Users</a></li>
                <li class="nav-item">
                    <button class="btn btn-sm btn-outline-light" onclick="toggleDarkMode()">Dark
                </li>
```

```
            </ul>
        </div>
    </nav>

    <div class="container-fluid mt-4">
        <div class="row">
            <div class="col-md-2">
                {% block sidebar %}
                <div class="list-group">
                    <a href="/admin" class="list-group-item list-group-item-action">Overview</a>
                    <a href="/admin/products" class="list-group-item list-group-item-action">Pro
                    <a href="/admin/orders" class="list-group-item list-group-item-action">Order
                    <a href="/admin/customers" class="list-group-item list-group-item-action">Cu
                    <a href="/admin/reports" class="list-group-item list-group-item-action">Repo
                    <a href="/admin/settings" class="list-group-item list-group-item-action">Set
                </div>
                {% endblock %}
            </div>
            <div class="col-md-10">
                {% block content %}{% endblock %}
            </div>
        </div>
    </div>

    <script src="/js/frond.min.js"></script>
    <script>
        function toggleDarkMode() {
            var html = document.documentElement;
            var current = html.getAttribute("data-theme");
            html.setAttribute("data-theme", current === "dark" ? "light" : "dark");
            localStorage.setItem("theme", current === "dark" ? "light" : "dark");
        }
    </script>
    {% block scripts %}{% endblock %}
</body>
</html>
```

Dashboard Page

Create `src/templates/dashboard.html`:

```
{% extends "base.html" %}

{% block title %}Dashboard - Admin{% endblock %}

{% block content %}
    <h1>Dashboard</h1>

    <div class="row mb-4">
        <div class="col-md-3">
            <div class="card">
                <div class="card-body">
                    <h6 class="card-subtitle text-muted">Total Products</h6>
                    <h2 class="card-title">{{ stats.total_products }}</h2>
                </div>
            </div>
        </div>
        <div class="col-md-3">
            <div class="card">
```

```
            <div class="card-body">
                <h6 class="card-subtitle text-muted">Total Orders</h6>
                <h2 class="card-title">{{ stats.total_orders }}</h2>
            </div>
        </div>
    </div>
    <div class="col-md-3">
        <div class="card">
            <div class="card-body">
                <h6 class="card-subtitle text-muted">Revenue</h6>
                <h2 class="card-title">${{ stats.revenue }}</h2>
            </div>
        </div>
    </div>
    <div class="col-md-3">
        <div class="card">
            <div class="card-body">
                <h6 class="card-subtitle text-muted">Active Users</h6>
                <h2 class="card-title">{{ stats.active_users }}</h2>
            </div>
        </div>
    </div>
</div>

<div class="row">
    <div class="col-md-8">
        <div class="card">
            <div class="card-header">Recent Orders</div>
            <div class="card-body">
                <table class="table table-striped">
                    <thead>
                        <tr>
                            <th>Order</th>
                            <th>Customer</th>
                            <th>Amount</th>
                            <th>Status</th>
                        </tr>
                    </thead>
                    <tbody>
                        {% for order in recent_orders %}
                        <tr>
                            <td>#{{ order.id }}</td>
                            <td>{{ order.customer }}</td>
                            <td>${{ order.amount }}</td>
                            <td>
                                <span class="badge bg-{{ order.badge }}">{{ order.status }}<
                            </td>
                        </tr>
                        {% endfor %}
                    </tbody>
                </table>
            </div>
        </div>
    </div>
    <div class="col-md-4">
        <div class="card">
            <div class="card-header">Quick Actions</div>
            <div class="card-body">
                <a href="/admin/products/new" class="btn btn-primary btn-block mb-2">Add Pro
```

```
                    <a href="/admin/orders" class="btn btn-outline-primary btn-block mb-2">View
                    <a href="/admin/reports" class="btn btn-outline-secondary btn-block">Generat
                </div>
            </div>

            <div class="card mt-3">
                <div class="card-header">System Health</div>
                <div class="card-body">
                    <p><strong>CPU:</strong></p>
                    <div class="progress mb-3">
                        <div class="progress-bar bg-success" style="width: {{ stats.cpu_usage }}
                            {{ stats.cpu_usage }}%
                        </div>
                    </div>
                    <p><strong>Memory:</strong></p>
                    <div class="progress mb-3">
                        <div class="progress-bar bg-info" style="width: {{ stats.memory_usage }}
                            {{ stats.memory_usage }}%
                        </div>
                    </div>
                    <p><strong>Disk:</strong></p>
                    <div class="progress">
                        <div class="progress-bar bg-warning" style="width: {{ stats.disk_usage }
                            {{ stats.disk_usage }}%
                        </div>
                    </div>
                </div>
            </div>
        </div>
    </div>
{% endblock %}
```

Dashboard Route

Create `src/routes/admin.ts`:

```
import { Router } from "tina4-nodejs";

Router.get("/admin", async (req, res) => {
    const stats = {
        total_products: 156,
        total_orders: 1243,
        revenue: "24,580",
        active_users: 89,
        cpu_usage: 42,
        memory_usage: 68,
        disk_usage: 55
    };

    const recent_orders = [
        { id: 1042, customer: "Alice Johnson", amount: "129.99", status: "Shipped", badge: "succ
        { id: 1041, customer: "Bob Smith", amount: "549.99", status: "Processing", badge: "warni
        { id: 1040, customer: "Carol White", amount: "79.99", status: "Delivered", badge: "info"
        { id: 1039, customer: "Dave Brown", amount: "34.99", status: "Cancelled", badge: "danger
    ];

    return res.render("dashboard.html", { stats, recent_orders });
});
```

Start the server and visit `http://localhost:7148/admin`. You see a sidebar, stat cards, a data table, quick action buttons, and system health progress bars. Zero npm dependencies.

---

9. Dark Mode

tina4css supports dark mode through a single attribute on the `<html>` element:

```
<!-- Light mode (default) -->
<html data-theme="light">

<!-- Dark mode -->
<html data-theme="dark">
```

The toggle function from the base template handles switching:

```
function toggleDarkMode() {
    var html = document.documentElement;
    var current = html.getAttribute("data-theme");
    html.setAttribute("data-theme", current === "dark" ? "light" : "dark");
    localStorage.setItem("theme", current === "dark" ? "light" : "dark");
}
```

Dark mode transforms every surface. Backgrounds darken. Text colors invert. Borders shift. Cards, tables, buttons -- all adjust contrast. Zero CSS rules required from you.

Respecting System Preference

Match the user's operating system dark mode setting:

```
var saved = localStorage.getItem("theme");
if (saved) {
    document.documentElement.setAttribute("data-theme", saved);
} else if (window.matchMedia("(prefers-color-scheme: dark)").matches) {
    document.documentElement.setAttribute("data-theme", "dark");
}
```

---

10. Responsive Design

tina4css is mobile-first. Components stack on small screens and expand on larger ones.

Responsive Sidebar

On mobile, the sidebar collapses into a toggle:

```
<button class="btn btn-dark d-md-none" onclick="toggleSidebar()">Menu</button>

<div id="sidebar" class="d-none d-md-block col-md-2">
    <!-- sidebar content -->
</div>

<script>
function toggleSidebar() {
    var sidebar = document.getElementById("sidebar");
    sidebar.classList.toggle("d-none");
```

```
}
</script>
```

Responsive Tables

Tables scroll horizontally on small screens:

```
<div class="table-responsive">
    <table class="table">
        <!-- table content -->
    </table>
</div>
```

Hiding Elements by Screen Size

```
<div class="d-none d-md-block">Only visible on tablet and up</div>
<div class="d-md-none">Only visible on mobile</div>
```

---

11. Building a Users Page with AJAX

A user management page. Data loads via AJAX using frond.js. No full page reloads. The table populates from the API. Users create, edit, and delete records through modals and inline actions.

The Template

Create `src/templates/admin/users.html`:

```
{% extends "base.html" %}

{% block title %}Users - Admin{% endblock %}

{% block content %}
    <div class="card">
        <div class="card-header d-flex justify-content-between align-items-center">
            <span>All Users</span>
            <button class="btn btn-sm btn-primary" data-toggle="modal" data-target="#addUserModa
                Add User
            </button>
        </div>
        <div class="card-body p-0">
            <div id="loadingSpinner" class="text-center p-4" style="display: none;">
                Loading...
            </div>
            <table class="table table-hover m-0" id="usersTable">
                <thead>
                    <tr>
                        <th>ID</th>
                        <th>Name</th>
                        <th>Email</th>
                        <th>Created</th>
                        <th>Actions</th>
                    </tr>
                </thead>
                <tbody id="usersBody">
                    <!-- Populated by JavaScript -->
                </tbody>
            </table>
```

```
        </div>
    </div>

    <!-- Add User Modal -->
    <div class="modal" id="addUserModal">
        <div class="modal-dialog">
            <div class="modal-content">
                <div class="modal-header">
                    <h5 class="modal-title">Add New User</h5>
                    <button type="button" class="btn-close" data-dismiss="modal"></button>
                </div>
                <div class="modal-body">
                    <form id="addUserForm">
                        <div class="form-group">
                            <label for="userName">Name</label>
                            <input type="text" class="form-control" name="name" id="userName" re
                        </div>
                        <div class="form-group">
                            <label for="userEmail">Email</label>
                            <input type="email" class="form-control" name="email" id="userEmail"
                        </div>
                    </form>
                </div>
                <div class="modal-footer">
                    <button class="btn btn-secondary" data-dismiss="modal">Cancel</button>
                    <button class="btn btn-primary" onclick="createUser()">Create User</button>
                </div>
            </div>
        </div>
    </div>

    <div id="alertArea" class="mt-3"></div>
{% endblock %}

{% block scripts %}
<script>
    function loadUsers() {
        frond.get("/api/users", function (data) {
            var tbody = document.getElementById("usersBody");
            tbody.innerHTML = "";

            if (data.data && data.data.length > 0) {
                data.data.forEach(function (user) {
                    var row = '<tr>'
                        + '<td>' + user.id + '</td>'
                        + '<td>' + user.name + '</td>'
                        + '<td>' + user.email + '</td>'
                        + '<td>' + user.created_at + '</td>'
                        + '<td>'
                        + '<button class="btn btn-sm btn-outline-primary" onclick="editUser(' +
                        + '<button class="btn btn-sm btn-outline-danger" onclick="deleteUser(' +
                        + '</td>'
                        + '</tr>';
                    tbody.innerHTML += row;
                });
            } else {
                tbody.innerHTML = '<tr><td colspan="5" class="text-center p-4">No users found</t
            }
        }, null, { loading: "#loadingSpinner" });
```

```
    }

    function createUser() {
        var name = document.getElementById("userName").value;
        var email = document.getElementById("userEmail").value;

        frond.post("/api/users", { name: name, email: email }, function (data) {
            document.getElementById("alertArea").innerHTML =
                '<div class="alert alert-success">User "' + data.name + '" created.</div>';
            loadUsers();
        }, function (error) {
            document.getElementById("alertArea").innerHTML =
                '<div class="alert alert-danger">Error creating user.</div>';
        });
    }

    function deleteUser(id) {
        if (confirm("Are you sure you want to delete this user?")) {
            frond.delete("/api/users/" + id, function () {
                loadUsers();
            });
        }
    }

    // Load users on page load
    loadUsers();
</script>
{% endblock %}
```

The Route

```
import { Router } from "tina4-nodejs";

Router.get("/admin/users", async (req, res) => {
    return res.render("admin/users.html", {});
});
```

This page loads users via an AJAX call to `/api/users` (provided by auto-CRUD on the User model from Chapter 6). The loading indicator appears while the request flies. The table populates when data arrives. Users add records through a modal form and delete with confirmation -- all without full page reloads.

---

12. Building a CRUD Interface

Create `src/templates/product-manager.html`:

```
{% extends "base.html" %}

{% block title %}Product Manager{% endblock %}

{% block content %}
    <h1>Product Manager</h1>

    <div id="app">
        <form id="add-form" class="card" style="padding: 16px; margin-bottom: 16px;">
            <h3>Add Product</h3>
```

```
            <div class="form-group">
                <label for="name">Name</label>
                <input type="text" id="name" class="form-control" required>
            </div>
            <div class="form-group">
                <label for="price">Price</label>
                <input type="number" id="price" class="form-control" step="0.01" required>
            </div>
            <button type="submit" class="btn btn-primary">Add Product</button>
        </form>

        <div id="product-list"></div>
    </div>

    <script src="/js/frond.js"></script>
    <script>
        async function loadProducts() {
            const data = await frond.get("/api/products");
            const list = document.getElementById("product-list");
            list.innerHTML = data.products.map(p =>
                `<div class="card" style="padding: 12px; margin-bottom: 8px;">
                    <strong>${p.name}</strong> - $${p.price.toFixed(2)}
                    <button class="btn btn-danger" style="float:right" onclick="deleteProduct(${
                </div>`
            ).join("");
        }

        document.getElementById("add-form").addEventListener("submit", async (e) => {
            e.preventDefault();
            const name = document.getElementById("name").value;
            const price = document.getElementById("price").value;
            await frond.post("/api/products", { name, price: parseFloat(price) });
            document.getElementById("name").value = "";
            document.getElementById("price").value = "";
            loadProducts();
        });

        async function deleteProduct(id) {
            await frond.del("/api/products/" + id);
            loadProducts();
        }

        loadProducts();
    </script>
{% endblock %}
```

---

13. Static File Serving

Files in `src/public/` are served directly:

```
src/public/images/logo.png → /images/logo.png
src/public/js/app.js       → /js/app.js
src/public/css/custom.css  → /css/custom.css
```

---

14. Integrating with React, Vue, or Svelte

Tina4 is the API backend. Point your frontend build tool's output to `src/public/`:

```
# Vue
VITE_OUTPUT_DIR=../my-tina4-project/src/public

# React (Create React App)
BUILD_PATH=../my-tina4-project/src/public
```

Or use CORS to run them on separate ports during development:

```
TINA4_CORS_ORIGINS=http://localhost:3000,http://localhost:5173
```

---

15. Exercise: Build an Admin Dashboard

Build an admin dashboard for a product management system.

Requirements

- Create a base template with:
- A dark navbar with the app name and navigation links
- A sidebar with menu items (Dashboard, Products, Orders, Settings)
- A main content area
- Dark mode toggle that persists across page loads
- Create a dashboard page at `GET /admin` with:
- Four stat cards (Products, Orders, Revenue, Users)
- A table showing recent orders with status badges
- Progress bars showing system health (CPU, Memory, Disk)
- Create a users page at `GET /admin/users` with:
- A table of users loaded via AJAX using frond.js
- An "Add User" button that opens a modal with a form
- AJAX form submission that refreshes the table without a page reload
- A loading indicator while data fetches
- Use tina4css classes throughout (no custom CSS needed)

Test by:

- Visit `http://localhost:7148/admin` -- see the dashboard with stats, orders, and progress bars
- Click "Dark Mode" -- the entire page switches to dark theme
- Refresh the page -- dark mode persists
- Resize the browser to mobile width -- the sidebar collapses

- Visit `http://localhost:7148/admin/users` -- see the user table loaded via AJAX

---

16. Solution

The base template, dashboard page, and users page are shown in sections 8 and 11. The product list page follows the same AJAX pattern from section 11, but for products instead of users.

For the users page, use auto-CRUD on the User model (`autoCrud = true`) so the API endpoints exist at `/api/users`. Load the table with `frond.get("/api/users", ...)` and handle form submission with `frond.post("/api/users", ...)`.

Add the product list route:

```
import { Router } from "tina4-nodejs";

Router.get("/admin/products", async (req, res) => {
    const search = req.query.search ?? "";
    const selectedCategory = req.query.category ?? "";

    let products = [
        { id: 1, name: "Wireless Keyboard", category: "Electronics", price: "79.99", inStock: tr
        { id: 2, name: "Standing Desk", category: "Furniture", price: "549.99", inStock: true },
        { id: 3, name: "Coffee Grinder", category: "Kitchen", price: "49.99", inStock: false },
        { id: 4, name: "Yoga Mat", category: "Fitness", price: "29.99", inStock: true },
        { id: 5, name: "USB-C Hub", category: "Electronics", price: "49.99", inStock: true },
    ];

    if (selectedCategory) {
        products = products.filter(p => p.category === selectedCategory);
    }

    if (search) {
        products = products.filter(p => p.name.toLowerCase().includes(search.toLowerCase()));
    }

    const categories = ["Electronics", "Furniture", "Kitchen", "Fitness"];

    return res.render("products.html", { products, categories, search, selected_category: select
});
```

---

17. Gotchas

1. CSS Not Loading

Problem: The page looks unstyled -- no colors, no layout, plain text.

Cause: The path to `tina4.css` is wrong. The file lives in `src/public/css/` but the link points elsewhere.

Fix: Tina4 serves everything in `src/public/` from the root path. Link to `/css/tina4.css` (not `/src/public/css/tina4.css`). Verify the file exists at `src/public/css/tina4.css`.

2. frond.js Functions Not Found

Problem: `frond.get is not a function` or `frond is not defined` in the browser console.

Cause: The `frond.min.js` script tag is missing or placed after the code that uses it.

Fix: Include `<script src="/js/frond.min.js"></script>` before any script that calls `frond.*`. Put it at the bottom of the body, before your custom scripts.

3. Dark Mode Flickers on Page Load

Problem: The page loads in light mode and then flashes to dark mode.

Cause: The dark mode JavaScript runs after the page renders. The browser paints the light theme first, then switches.

Fix: Add the theme detection script in the `<head>` (before the body renders):

```
<head>
    <script>
        var t = localStorage.getItem("theme");
        if (t) document.documentElement.setAttribute("data-theme", t);
    </script>
</head>
```

4. SCSS Not Compiling

Problem: You edited `tina4.scss` but the CSS did not change.

Cause: SCSS does not compile on its own. You need to run `tina4 scss` or use `--watch` mode.

Fix: Run `tina4 scss --watch` during development. For production builds, run `tina4 scss` as part of your build process.

5. Modal Does Not Open

Problem: Clicking the button does nothing -- the modal stays hidden.

Cause: The `data-toggle` and `data-target` attributes require frond.js to be loaded. Without it, no JavaScript handles the modal toggle.

Fix: Ensure `frond.min.js` is loaded. Verify the `data-target` matches the modal's `id` exactly (including the `#` prefix).

6. Grid Columns Do Not Stack on Mobile

Problem: Columns stay side-by-side on phone screens instead of stacking vertically.

Cause: You used `col-4` instead of `col-md-4`. The `col-4` class applies at all screen sizes.

Fix: Use responsive prefixes: `col-md-4` means "one-third on medium screens and up, full width on small screens."

7. Static Files Return 404

Problem: CSS, JS, or image files return 404 Not Found.

Cause: The files are not in the `src/public/` directory.

Fix: Static files must be in `src/public/`. The URL path maps to the file path within that directory. `/css/tina4.css` maps to `src/public/css/tina4.css`.

8. Form Data Not Reaching the Server

Problem: `frond.post()` sends the request but `req.body` is empty on the server.

Cause: frond.js sends JSON by default. Passing a `FormData` object or building the data object wrong prevents correct parsing.

Fix: Pass a plain JavaScript object to `frond.post()`. Do not use `new FormData()` -- frond.js handles serialization. Make sure your object keys match what the server expects.

9. Loading Indicator Never Disappears

Problem: The loading spinner shows but never hides after the AJAX request completes.

Cause: The CSS selector passed to the `loading` option does not match any element, or the element uses a CSS class to toggle visibility instead of inline `display`.

Fix: frond.js toggles `display: block` and `display: none`. Make sure the element exists and its initial style is `display: none`. Use an `id` selector: `{ loading: "#loadingSpinner" }`.

10. CORS Errors with Separate Frontend

Fix: Set `TINA4_CORS_ORIGINS` in `.env`.

11. Large Bundle Sizes

Fix: Use code splitting. Serve static assets from a CDN.

12. React Router Conflicts

Fix: Add a catch-all route that serves `index.html` for client-side routing.

---

18. HtmlElement: Programmatic HTML Builder

Build HTML in TypeScript without string concatenation:

```
import { HtmlElement, htmlElement, addHtmlHelpers } from "@tina4/core";

const el = new HtmlElement("div", { class: "card" }, ["Hello"]);
el.toString(); // '<div class="card">Hello</div>'

// Nesting
const card = new HtmlElement("div", { class: "card" }, [
    new HtmlElement("h2", {}, ["Title"]),
    new HtmlElement("p", {}, ["Content"]),
]);

// Helper functions
const h: Record<string, any> = {};
addHtmlHelpers(h);
```

```
const html = h._div({ class: "card" }, h._p("Hello"), h._a({ href: "/" }, "Home"));
```

Void tags (
, <img>, <input>) render without closing tags. Boolean attributes render as bare names.

Testing

1. Why Tests Matter More Than You Think

Friday afternoon. Your client reports a critical bug in production. You fix it -- one line. But did that fix break something else? 47 routes. 12 ORM models. 3 middleware functions. Clicking through every page takes an hour. Running the test suite takes 2 seconds.

```
npm test

Running tests...

  add
    + add([[5, 3]]) == 8
    + add([[null]]) raises Error

  isEven
    + isEven([[4]]) is truthy
    + isEven([[3]]) is falsy

  4 tests: 4 passed, 0 failed, 0 errors
```

Everything still works. Deploy with confidence. Weekend intact.

Tina4 ships an inline testing framework. No external packages. No Jest configuration. No setup ceremony.

---

2. Your First Test

Tina4's testing framework uses a decorator-style pattern. Attach test assertions directly to functions using `tests()`, `expectEqual()`, `expectRaises()`, `expectTrue()`, and `expectFalse()`. Then call `runAll()` to execute them.

Create `tests/basic.ts`:

```
import { tests, expectEqual, expectRaises, expectTrue, expectFalse, runAll } from "tina4-nodejs"

const add = tests(
    expectEqual([5, 3], 8),
    expectEqual([0, 0], 0),
    expectRaises(Error, [null]),
)(function add(a: number, b: number | null = null): number {
    if (b === null) throw new Error("b required");
    return a + b;
});

const isEven = tests(
    expectTrue([4]),
    expectFalse([3]),
)(function isEven(n: number): boolean {
    return n % 2 === 0;
});

runAll();
```

Run it:

```
tina4 test

  add
    + add([[5, 3]]) == 8
    + add([[0, 0]]) == 0
    + add([[null]]) raises Error

  isEven
    + isEven([[4]]) is truthy
    + isEven([[3]]) is falsy

  5 tests: 5 passed, 0 failed, 0 errors
```

How It Works

- The `tests()` function takes assertion objects and returns a decorator.
- The decorator wraps the function, registers it in the test registry, and returns the original function unchanged.
- The function works normally in production code -- tests only execute when you call `runAll()`.
- Named functions produce readable output. Anonymous functions show as "anonymous."

---

3. Assertion Functions

Function	Description
`expectEqual(args, expected)`	Call the function with `args` array, expect `expected` as the return value
`expectRaises(ErrorClass, args)`	Call the function with `args` array, expect it to throw `ErrorClass`
`expectTrue(args)`	Call the function with `args` array, expect a truthy return value
`expectFalse(args)`	Call the function with `args` array, expect a falsy return value

Each assertion specifies the arguments to pass and the expected outcome. The `args` parameter is always an array of arguments.

expectEqual

```
expectEqual([5, 3], 8)     // Call with 5, 3 -- expect 8
expectEqual(["hello"], 5)  // Call with "hello" -- expect 5
```

expectRaises

```
expectRaises(Error, [null])         // Expect Error when called with null
expectRaises(TypeError, ["bad"])    // Expect TypeError when called with "bad"
```

expectTrue / expectFalse

```
expectTrue([4])     // Expect a truthy return value
expectFalse([0])    // Expect a falsy return value
```

---

4. Testing Business Logic

```
import { tests, expectEqual, expectRaises, runAll } from "tina4-nodejs";

const calculateDiscount = tests(
    expectEqual([100, 10], 90),
    expectEqual([50, 0], 50),
    expectEqual([200, 50], 100),
    expectRaises(Error, [100, -5]),
    expectRaises(Error, [100, 101]),
)(function calculateDiscount(price: number, discountPercent: number): number {
    if (discountPercent < 0 || discountPercent > 100) {
        throw new Error("Discount must be between 0 and 100");
    }
    return price - (price * discountPercent / 100);
});

runAll();
```

The function works in production. The tests run only when you call `runAll()`.

---

5. Testing ORM Models

Test your models by writing functions that exercise create, read, update, and delete:

```
import { tests, expectTrue, expectEqual, runAll } from "tina4-nodejs";
import { Database } from "tina4-nodejs/orm";

const testCreateProduct = tests(
    expectTrue([]),
)(function testCreateProduct(): boolean {
    const product = new Product({
        name: "Test Widget",
        category: "Testing",
        price: 19.99,
    });
    product.save();
    return product.id !== undefined && product.id > 0;
});

const testLoadProduct = tests(
    expectTrue([]),
)(function testLoadProduct(): boolean {
    const product = new Product({ name: "Load Test", price: 29.99 });
    product.save();

    const loaded = Product.findById(product.id);
    return loaded !== null && loaded.name === "Load Test";
});
```

```
const testUpdateProduct = tests(
    expectTrue([]),
)(function testUpdateProduct(): boolean {
    const product = new Product({ name: "Update Test", price: 10 });
    product.save();

    product.name = "Updated Widget";
    product.price = 15;
    product.save();

    const reloaded = Product.findById(product.id);
    return reloaded !== null && reloaded.name === "Updated Widget" && reloaded.price === 15;
});

const testDeleteProduct = tests(
    expectTrue([]),
)(function testDeleteProduct(): boolean {
    const product = new Product({ name: "Delete Me", price: 5 });
    product.save();
    const id = product.id;

    product.delete();

    const gone = Product.findById(id);
    return gone === null;
});

runAll();
```

Test Database

Use a separate database for tests so development data stays safe:

```
TINA4_DATABASE_URL=sqlite:///data/test.db
```

---

6. Testing Routes

The `TestClient` lets you exercise your API endpoints end-to-end without starting a server. It builds a mock request, matches the route, runs the handler, and returns a `TestResponse`. Construct one instance and reuse it across test functions.

```
import { tests, expectTrue, runAll, TestClient } from "tina4-nodejs";

const client = new TestClient();

const testHealthEndpoint = tests(
    expectTrue([]),
)(async function testHealthEndpoint(): Promise<boolean> {
    const resp = await client.get("/health");
    const body = resp.json() as { status?: string } | null;
    return resp.status === 200 && body?.status === "ok";
});

const testCreateProduct = tests(
    expectTrue([]),
)(async function testCreateProduct(): Promise<boolean> {
```

```
    const resp = await client.post("/api/products", {
        json: {
            name: "Route Test Product",
            category: "Testing",
            price: 42,
        },
    });
    const body = resp.json() as { name?: string } | null;
    return resp.status === 201 && body?.name === "Route Test Product";
});

const testGetNotFound = tests(
    expectTrue([]),
)(async function testGetNotFound(): Promise<boolean> {
    const resp = await client.get("/api/products/99999");
    return resp.status === 404;
});

runAll();
```

TestClient Methods

```
const client = new TestClient();

// GET request
const resp1 = await client.get("/api/products");

// GET with query string
const resp2 = await client.get("/api/products?category=Electronics");

// POST with a JSON body - pass `json` inside the options object
const resp3 = await client.post("/api/products", { json: { name: "Widget", price: 9.99 } });

// PUT / PATCH / DELETE - same shape
const resp4 = await client.put("/api/products/1", { json: { name: "Updated" } });
const resp5 = await client.patch("/api/products/1", { json: { price: 12.99 } });
const resp6 = await client.delete("/api/products/1");

// Custom headers (e.g. Authorization)
const resp7 = await client.get("/api/profile", {
    headers: { authorization: "Bearer eyJhbGciOiJIUzI1NiIs..." },
});
```

TestResponse

```
resp.status        // HTTP status code
resp.body          // raw response body as a string
resp.text()        // alias for resp.body
resp.json()        // parses resp.body as JSON; returns null if not JSON
resp.headers       // response headers as a record (lowercased keys)
resp.contentType   // Content-Type header value
```

`resp.body` is always a string, so parse it with `resp.json()` (or `JSON.parse(resp.body)`) before reading properties.

---

7. Testing Authentication

```
import { tests, expectTrue, runAll, Auth } from "tina4-nodejs";

const secret = "test-secret";

const testTokenRoundTrip = tests(
    expectTrue([{ userId: 1, role: "admin" }]),
)(function testTokenRoundTrip(payload: Record<string, unknown>): boolean {
    const token = Auth.getToken(payload, secret);
    const decoded = Auth.validToken(token, secret);
    return decoded !== null && decoded.userId === payload.userId;
});

const testPasswordHash = tests(
    expectTrue(["securePass123"]),
    expectTrue(["another-password"]),
)(function testPasswordHash(password: string): boolean {
    const hash = Auth.hashPassword(password);
    return Auth.checkPassword(password, hash);
});

const testInvalidToken = tests(
    expectTrue([]),
)(function testInvalidToken(): boolean {
    const decoded = Auth.validToken("invalid.token.here", secret);
    return decoded === null;
});

runAll();
```

---

8. Resetting Tests Between Files

When running tests across multiple files, use `reset()` to clear the registry:

```
import { reset, tests, expectEqual, runAll } from "tina4-nodejs";

reset();

const multiply = tests(
    expectEqual([3, 4], 12),
    expectEqual([0, 5], 0),
)(function multiply(a: number, b: number): number {
    return a * b;
});

runAll();
```

Without resetting, tests from previously imported modules accumulate in the registry.

---

9. Runner Options

`runAll()` accepts an options object:

```
// Quiet mode -- no console output, returns results only
const results = runAll({ quiet: true });
console.log(`${results.passed} passed, ${results.failed} failed`);

// Fail fast -- stop on the first failure
runAll({ failfast: true });
```

The returned `TestResults` object:

Property	Type	Description
`passed`	number	Assertions that passed
`failed`	number	Assertions that failed
`errors`	number	Unexpected errors
`details`	array	Array of `{ name, status, message? }` objects

CI Integration

Use the return value for CI exit codes:

```
const results = runAll();
process.exit(results.failed === 0 ? 0 : 1);
```

---

10. Running Tests

Run All Tests

```
npm test
```

This runs `tests/run-all.ts` which discovers and executes all test files.

Run a Specific Test File

```
tina4 test tests/basic.ts
```

With npm Scripts

Add to `package.json`:

```
{
  "scripts": {
    "test": "npx tsx tests/run-all.ts",
    "test:products": "npx tsx tests/products.ts",
    "test:auth": "npx tsx tests/auth.ts"
  }
}
```

---

11. Testing Best Practices

Test One Thing Per Function

Each test function should verify one behavior. When it fails, you know exactly what broke.

```
// Good: each function tests one thing
const testCreateReturnsId = tests(
    expectTrue([]),
)(function testCreateReturnsId(): boolean {
    const product = new Product({ name: "Widget", price: 9.99 });
    product.save();
    return product.id > 0;
});

const testCreateSetsDefaults = tests(
    expectTrue([]),
)(function testCreateSetsDefaults(): boolean {
    const product = new Product({ name: "Widget", price: 9.99 });
    product.save();
    return product.category === "Uncategorized";
});
```

Use Named Functions for Readable Output

```
// Good: readable test names
const testLoginRejects = tests(expectTrue([]))(
    function testLoginRejectsInvalidPassword(): boolean { /* ... */ }
);

// Bad: anonymous function
const test = tests(expectTrue([]))(
    () => { /* shows as "anonymous" in output */ }
);
```

Isolate Tests

Each test should create its own data. Never depend on data from another test or from the development database.

```
// Good: creates its own data
async function testDeleteProduct(): Promise<boolean> {
    const product = new Product({ name: "Temporary", price: 1 });
    await product.save();
    await product.delete();
    return (await Product.findById(product.id)) === null;
}

// Bad: depends on external state
async function testDeleteProduct(): Promise<boolean> {
    const product = await Product.findById(1); // Assumes ID 1 exists
    await product.delete();
    return true;
}
```

Use Unique Values

Prevent unique constraint failures by generating unique data:

```
async function testCreateUser(): Promise<boolean> {
```

```
    const email = `test-${Date.now()}@example.com`;
    const user = new User({ name: "Test User", email });
    await user.save();
    return user.id > 0;
}
```

---

12. Failed Test Output

When a test fails, the output shows exactly what went wrong:

```
calculateDiscount
  + calculateDiscount([[100, 10]]) == 90
  - calculateDiscount([[200, 50]]) == 100
    Expected: 100
    Got: 150
  + calculateDiscount([[100, -5]]) raises Error

3 tests: 2 passed, 1 failed, 0 errors
```

The failure message shows the function name, the arguments, what was expected, and what was received. Find the line. Fix the logic. Run again.

---

13. Exercise: Write Tests for Utility Functions

Write inline tests for the following functions:

- A `slugify` function that converts `"Hello World"` to `"hello-world"`
- A `clamp` function that constrains a number between a min and max
- A `parsePrice` function that extracts a number from `"$19.99"` and throws on invalid input

---

14. Solution

```
import { tests, expectEqual, expectRaises, runAll } from "tina4-nodejs";

const slugify = tests(
    expectEqual(["Hello World"], "hello-world"),
    expectEqual(["  Multiple   Spaces  "], "multiple-spaces"),
    expectEqual(["UPPERCASE"], "uppercase"),
    expectEqual(["already-slugged"], "already-slugged"),
)(function slugify(input: string): string {
    return input.trim().toLowerCase().replace(/\s+/g, "-");
});

const clamp = tests(
    expectEqual([5, 0, 10], 5),
    expectEqual([-5, 0, 10], 0),
    expectEqual([15, 0, 10], 10),
    expectEqual([0, 0, 0], 0),
)(function clamp(value: number, min: number, max: number): number {
    return Math.min(Math.max(value, min), max);
});

const parsePrice = tests(
    expectEqual(["$19.99"], 19.99),
    expectEqual(["$0.50"], 0.5),
    expectRaises(Error, ["not-a-price"]),
    expectRaises(Error, [""]),
)(function parsePrice(input: string): number {
    const match = input.match(/\$?([\d.]+)/);
    if (!match) throw new Error("Invalid price format");
    const value = parseFloat(match[1]);
    if (isNaN(value)) throw new Error("Invalid price format");
    return value;
});

runAll();
```

---

15. Gotchas

1. The `tests()` Decorator Returns the Original Function

The wrapped function works identically in production. Tests only run when you call `runAll()`. You can use the decorated function in your application code without side effects.

2. Arguments Are Passed as an Array

`expectEqual([5, 3], 8)` means "call the function with arguments `5` and `3`, expect `8`." The first argument is always an array.

3. Named Functions Are Required for Readable Output

Anonymous functions show up as "anonymous" in test output. Use named function expressions: `function testCreateProduct() {}` not `() => {}`.

4. Call `reset()` Between Separate Test Files

Without resetting, tests from previously imported modules accumulate in the registry. Each test file should call `reset()` at the top.

5. `runAll()` Returns Results

Use the return value for CI integration. Check `results.failed === 0` to determine the exit code. Without this, your CI pipeline passes even when tests fail.

6. Database State Carries Between Tests

Problem: A test fails because a previous test left data in the database.

Fix: Each test should create its own data and clean up after itself. Use a separate test database. Reset it before each test run.

7. Async Functions Need Special Handling

Problem: You test an async function but the test finishes before the Promise resolves.

Fix: Wrap async operations in a synchronous test function that blocks on the result. Or structure your test to validate the return value of the resolved Promise.

Scaffolding

One command. Six files. A working CRUD feature with routes, templates, tests, and Swagger docs -- ready to run.

That is Tina4's scaffolding system. It generates the boilerplate you write by hand in every project: models, migrations, routes, forms, views, and tests. You describe what you want. The generators produce it.

The CRUD Generator

This is the generator most developers reach for first. It creates everything a feature needs in one shot.

```
tina4 generate crud Product --fields "name:string,price:float"
```

That single command creates six files:

#	File	Purpose
1	`src/orm/Product.ts`	ORM model with typed fields
2	`migrations/20260401_create_product.sql`	UP migration (CREATE TABLE)
3	`migrations/20260401_create_product.down.sql`	DOWN migration (DROP TABLE)
4	`src/routes/products.ts`	CRUD routes with Swagger annotations
5	`src/templates/products/form.html`	Form template with typed inputs and `form_token`
6	`src/templates/products/view.html`	List and detail templates
7	`test/products.test.ts`	Test stubs for all CRUD operations

What Each File Contains

The **model** maps the `product` table to a TypeScript class:

```
import { ORM } from "tina4-nodejs";

export class Product extends ORM {
  tableName = "product";

  id?: number;
```

```
  name?: string;
  price?: number;
  createdAt?: string;
  updatedAt?: string;
}
```

The **migration** creates the table:

```
-- migrations/20260401_create_product.sql
CREATE TABLE product (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name VARCHAR(255),
    price FLOAT,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    updated_at DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

The **down migration** reverses it:

```
-- migrations/20260401_create_product.down.sql
DROP TABLE IF EXISTS product;
```

The **routes** file wires up five endpoints with Swagger docs:

```
import { get, post, put, del } from "tina4-nodejs";
import { Product } from "../orm/Product";

get("/api/products", "List all products", async (request, response) => {
  const products = await new Product().select();
  return response(products);
});

get("/api/products/:id", "Get a product by ID", async (request, response) => {
  const product = new Product();
  product.id = request.params.id;
  await product.load();
  return response(product);
});

post("/api/products", "Create a product", async (request, response) => {
  const product = new Product(request.body);
  await product.save();
  return response(product, 201);
});

put("/api/products/:id", "Update a product", async (request, response) => {
  const product = new Product(request.body);
  product.id = request.params.id;
  await product.save();
  return response(product);
});

del("/api/products/:id", "Delete a product", async (request, response) => {
  const product = new Product();
  product.id = request.params.id;
  await product.delete();
  return response(null, 204);
});
```

The **form template** renders typed inputs with CSRF protection:

```
<form method="POST" action="/api/products">
    <input type="hidden" name="form_token" value="{{ form_token }}">
    <label>Name</label>
    <input type="text" name="name" required>
    <label>Price</label>
    <input type="number" step="0.01" name="price" required>
    <button type="submit">Save</button>
</form>
```

The **test file** stubs out CRUD assertions using the built-in test runner:

```
import { describe, it } from "node:test";
import assert from "node:assert";
import { App } from "tina4-nodejs";

const client = new App().testClient();

describe("Products API", () => {
  it("creates a product", async () => {
    const response = await client.post("/api/products", { name: "Widget", price: 9.99 });
    assert.strictEqual(response.statusCode, 201);
  });

  it("lists products", async () => {
    const response = await client.get("/api/products");
    assert.strictEqual(response.statusCode, 200);
  });

  it("gets a product", async () => {
    const response = await client.get("/api/products/1");
    assert.strictEqual(response.statusCode, 200);
  });

  it("updates a product", async () => {
    const response = await client.put("/api/products/1", { name: "Updated Widget" });
    assert.strictEqual(response.statusCode, 200);
  });

  it("deletes a product", async () => {
    const response = await client.delete("/api/products/1");
    assert.strictEqual(response.statusCode, 204);
  });
});
```

Run It

After generating, run the migration and start the server:

```
tina4 migrate
tina4 serve
```

Open Swagger UI at `http://localhost:7148/swagger` and test every endpoint. The scaffolded code works out of the box.

Individual Generators

The CRUD generator calls several smaller generators under the hood. You can call each one directly when you need a single piece.

Model

```
tina4 generate model Product --fields "name:string,price:float"
```

Creates three files: the ORM model (`src/orm/Product.ts`), the UP migration, and the DOWN migration. No routes, no templates, no tests.

Route

```
tina4 generate route products --model Product
```

Creates one file: `src/routes/products.ts` with CRUD endpoints and Swagger annotations. The model must exist first.

Migration

```
tina4 generate migration add_category_to_product
```

Creates two files: `migrations/20260401_add_category_to_product.sql` and `migrations/20260401_add_category_to_product.down.sql`. Both are empty stubs. You write the SQL.

Middleware

```
tina4 generate middleware AuthLog
```

Creates one file with before and after stubs:

```
import { middleware } from "tina4-nodejs";

middleware("AuthLogBefore", "before", async (request) => {
  console.log(`Request: ${request.method} ${request.url}`);
  return request;
});

middleware("AuthLogAfter", "after", async (request, response) => {
  console.log(`Response: ${response.statusCode}`);
  return response;
});
```

Test

```
tina4 generate test products --model Product
```

Creates one file: `test/products.test.ts` with CRUD stubs using the built-in test runner.

Form

```
tina4 generate form Product --fields "name:string,price:float"
```

Creates one file: `src/templates/products/form.html` with typed inputs and `form_token`.

View

```
tina4 generate view Product --fields "name:string,price:float"
```

Creates two templates: a list view and a detail view in `src/templates/products/`.

CRUD

```
tina4 generate crud Product --fields "name:string,price:float"
```

Shorthand for running all generators at once: model, migration, route, form, view, and test.

Auth

```
tina4 generate auth
```

Generates the full authentication scaffold: User model, migrations, login/register/logout routes, templates, and tests.

AutoCRUD

AutoCRUD automatically generates REST API endpoints from your ORM models:

- `GET /api/{table}` - List with pagination (`?limit=10&offset=0`)
- `GET /api/{table}/{id}` - Get single record
- `POST /api/{table}` - Create record
- `PUT /api/{table}/{id}` - Update record
- `DELETE /api/{table}/{id}` - Delete record

Usage

```
// AutoCRUD routes are auto-generated from discovered models
// Models in src/models/ get REST endpoints at /api/{tableName}
```

Place your ORM models in `src/models/` and the framework discovers and mounts all five standard endpoints automatically; no route files needed.

The Auth Generator

Authentication needs more than one file. The auth generator creates seven:

```
tina4 generate auth
```

#	File	Purpose
1	`src/orm/User.ts`	User model with hashed password field
2	`migrations/20260401_create_user.sql`	UP migration
3	`migrations/20260401_create_user.down.sql`	DOWN migration

4	`src/routes/auth.ts`	Login, register, logout routes
5	`src/templates/auth/login.html`	Login form
6	`src/templates/auth/register.html`	Registration form
7	`test/auth.test.ts`	Auth flow tests

The generated routes handle password hashing, JWT token creation, and session management. The templates include CSRF tokens. The tests cover registration, login, invalid credentials, and logout.

Run the migration, start the server, and you have working auth:

```
tina4 migrate
tina4 serve
```

Field Types

Generators accept these field types. Each type maps to a specific column type in migrations, input type in forms, and display format in views.

Field Type	Migration Column	Form Input	View Display
`string`	`VARCHAR(255)`	`<input type="text">`	Plain text
`int`	`INTEGER`	`<input type="number">`	Number
`float`	`FLOAT`	`<input type="number" step="0.01">`	Decimal
`bool`	`BOOLEAN`	`<input type="checkbox">`	Yes / No
`text`	`TEXT`	`<textarea>`	Paragraph
`datetime`	`DATETIME`	`<input type="datetime-local">`	Formatted date
`blob`	`BLOB`	`<input type="file">`	Download link

Table Naming Convention

Tina4 uses singular table names. The model name `Product` maps to the table `product`. The model name `OrderItem` maps to `order_item`. The generator handles the conversion.

Combining Generators

Sometimes you want a model with routes but no form. Run the individual generators one after another. Each adds one piece:

```
tina4 generate model Product --fields "name:string,price:float"
tina4 generate route products --model Product
```

`generate model` writes the migration by default. Pass `--no-migration` to leave it out. Add `generate form`, `generate view` or `generate test` for the other pieces.

The `generate crud` command builds the full set in one step.

Exercise: Scaffold a Blog

Build a blog with three resources using generators.

Step 1: Generate the auth system.

```
tina4 generate auth
```

Step 2: Scaffold the Post resource.

```
tina4 generate crud Post --fields "title:string,body:text,published:bool"
```

Step 3: Scaffold the Category resource.

```
tina4 generate crud Category --fields "name:string,description:text"
```

Step 4: Add a migration to link posts to categories.

```
tina4 generate migration add_category_id_to_post
```

Edit the migration to add the foreign key:

```
ALTER TABLE post ADD COLUMN category_id INTEGER REFERENCES category(id);
```

Step 5: Run all migrations and start the server.

```
tina4 migrate
tina4 serve
```

You now have a working blog with authentication, posts, categories, and Swagger documentation. Total commands: five. Total hand-written SQL: one line.

Gotchas

Generators Do Not Overwrite

If a file exists, the generator skips it and prints a warning. This protects your edits. To regenerate, delete the file first.

Run Migrate After Generate

The model generator creates migration files. Those files do nothing until you run `tina4 migrate`. Generate and migrate are separate steps by design.

File Naming Matters

The generator derives file names from the model name. `Product` becomes `Product.ts` for the model and `products.ts` for routes. Do not rename generated files unless you update all imports.

Field Changes Need New Migrations

Changing `--fields` and re-running the generator does not update existing migrations. Create a new migration with `generate migration` and write the ALTER TABLE by hand.

Singular Table Names

Tina4 uses singular table names: `product`, not `products`. The route paths use plural (`/api/products`), but the table stays singular. The generator handles this split.

npx Alternative

If `tina4nodejs` is not in your PATH, use `npx`:

```
npx tina4 generate crud Product --fields "name:string,price:float"
```

All generator commands work the same way through npx.

API Documentation with Swagger

1. The 47-Endpoint Problem

Your team builds 47 API endpoints. The frontend developer asks "what does this endpoint accept?" Again. And again.

Swagger kills that question. It generates interactive API documentation from annotations in your route files. The docs stay current because they live in the code. Change a handler. The documentation changes with it.

Tina4 builds a Swagger UI at `/swagger` from JSDoc comments on your routes. No build step. No extra tooling. No separate spec file to maintain.

---

2. What Swagger/OpenAPI Is

OpenAPI is a specification format for describing REST APIs. Swagger is the toolset that reads OpenAPI specs and renders interactive documentation. Tina4 scans your route files, reads JSDoc annotations, and builds the OpenAPI spec automatically. No manual spec writing.

The generated spec is standard OpenAPI 3.0. It works with every tool in the OpenAPI ecosystem: client SDK generators, testing tools, API gateways, and monitoring services.

---

3. Enabling Swagger

Swagger runs when `TINA4_DEBUG=true`. Navigate to:

```
http://localhost:7148/swagger
```

The interactive UI loads. Every annotated route appears. Click any endpoint to see its documentation. Click "Try it out" to send a real request.

The raw OpenAPI JSON spec lives at:

```
http://localhost:7148/swagger/json
```

Swagger in Production

By default, Swagger only shows in debug mode. To expose it in production:

```
TINA4_SWAGGER_ENABLED=true
```

This enables the Swagger UI even when `TINA4_DEBUG=false`. Useful for developer portals and partner integrations.

---

4. Adding Descriptions to Routes

Every JSDoc annotation starts with a summary line. The `@description` tag adds detail:

```
import { Router } from "tina4-nodejs";

/**
 * List all products
 * @description Returns a paginated list of all products in the catalog
 * @tags Products
 */
Router.get("/api/products", async (req, res) => {
    return res.json({ products: [] });
});
```

The first line ("List all products") becomes the endpoint summary in Swagger. The `@description` text appears when a user expands the endpoint.

Path Parameters

Use `@param` to document path parameters:

```
/**
 * Get a product by ID
 * @description Returns a single product with full details
 * @tags Products
 * @param int id The unique product identifier
 */
Router.get("/api/products/{id:int}", async (req, res) => {
    return res.json({ id: req.params.id, name: "Wireless Keyboard", price: 79.99 });
});
```

Query Parameters

Use `@query` to document query string parameters:

```
/**
 * Search products
 * @description Search the product catalog by name or category
 * @tags Products
 * @query string q Search query
 * @query string category Filter by category
 * @query int page Page number (default: 1)
 * @query int limit Items per page (default: 20)
 */
Router.get("/api/products/search", async (req, res) => {
    return res.json({ query: req.query.q, results: [], total: 0 });
});
```

Each `@query` tag specifies the type, name, and description. Swagger renders these as a form in the "Try it out" view.

---

5. Documenting Request Bodies

Use `@body` to describe what the endpoint accepts:

```
/**
```

```
 * Create a new product
 * @description Creates a product in the catalog
 * @tags Products
 * @body {"name": "string", "category": "string", "price": "float", "inStock": "bool"}
 * @response 201 {"id": "int", "name": "string", "category": "string", "price": "float"}
 * @response 400 {"error": "string"}
 */
Router.post("/api/products", async (req, res) => {
    if (!req.body.name) {
        return res.status(400).json({ error: "Name is required" });
    }
    return res.status(201).json({
        id: 1,
        name: req.body.name,
        category: req.body.category ?? "Uncategorized",
        price: parseFloat(req.body.price ?? 0),
    });
});
```

The `@body` tag defines the JSON schema. Keys are field names. Values are type strings. Swagger renders this as a schema in the request body section.

Supported Types in @body and @response

Type String	OpenAPI Type
`"string"`	string
`"int"`	integer
`"float"`	number (float)
`"number"`	number
`"bool"`	boolean
`"object"`	object
`"array"`	array

---

6. Documenting Responses

Use `@response` to describe each possible response:

```
/**
 * Update a product
 * @tags Products
 * @param int id Product ID
 * @body {"name": "string", "price": "float"}
 * @response 200 {"id": "int", "name": "string", "price": "float", "updatedAt": "string"}
 * @response 404 {"error": "string"}
 * @response 400 {"error": "string"}
 */
Router.put("/api/products/{id:int}", async (req, res) => {
    // ...
});
```

Each `@response` tag starts with the status code, followed by the JSON schema. Document every status code your endpoint can return. The frontend developer sees exactly what to expect for success and failure.

---

7. Tags for Grouping

Tags organize endpoints into sections in the Swagger UI:

```
/**
 * List all users
 * @tags Users
 */
Router.get("/api/users", async (req, res) => {
    return res.json({ users: [] });
});

/**
 * List all orders
 * @tags Orders
 */
Router.get("/api/orders", async (req, res) => {
    return res.json({ orders: [] });
});
```

All endpoints with `@tags Users` appear under the "Users" heading in Swagger. An endpoint can belong to multiple tags:

```
/**
 * List user orders
 * @tags Users, Orders
 */
```

Without tags, endpoints land in a "default" group. Always add tags. They make the documentation navigable.

---

8. Example Values

Static schemas tell the developer what type each field is. Examples show what real data looks like:

```
/**
 * Create a product
 * @tags Products
 * @body {"name": "string", "category": "string", "price": "float"}
 * @example request {"name": "Ergonomic Keyboard", "category": "Electronics", "price": 89.99}
 * @example response {"id": 42, "name": "Ergonomic Keyboard", "category": "Electronics", "price"
 */
Router.post("/api/products", async (req, res) => {
    return res.status(201).json({ id: 42, name: req.body.name, price: req.body.price });
});
```

`@example request` pre-fills the request body in "Try it out." The developer clicks the button and the form already has realistic data. No manual typing.

`@example response` shows what a successful response looks like. The Swagger UI displays it in the response section.

---

9. Authentication in Swagger

Public and Secured Routes

Mark routes as public or secured with `@noauth` and `@secured`:

```
/**
 * Login
 * @noauth
 * @tags Auth
 * @body {"email": "string", "password": "string"}
 * @response 200 {"token": "string"}
 */
Router.post("/api/auth/login", async (req, res) => {
    // ...
});

/**
 * Get user profile
 * @secured
 * @tags Users
 * @response 200 {"id": "int", "name": "string", "email": "string"}
 * @response 401 {"error": "string"}
 */
Router.get("/api/profile", async (req, res) => {
    // ...
}, [authMiddleware]);
```

`@noauth` tells Swagger this endpoint does not require authentication. `@secured` adds a lock icon and includes the Authorization header in the spec.

Authorize Button

The Swagger UI has an "Authorize" button at the top. Click it. Paste your JWT token. All subsequent "Try it out" requests include the `Authorization: Bearer <token>` header automatically.

Workflow

- Call the login endpoint in Swagger -- get a token in the response
- Click the "Authorize" button at the top
- Paste the token value
- All secured endpoints now send the token with every request

No curl. No Postman. Test the full auth flow from the browser.

---

10. Try-It-Out from the Swagger UI

The Swagger UI puts a "Try it out" button on every endpoint. Click it. The UI expands the endpoint into a form. Fill in path parameters, query parameters, and the request body. Click "Execute." The UI sends a real HTTP request to your running server and displays the response.

The response panel shows:

- The HTTP status code
- Response headers
- The response body (formatted JSON)
- The curl command equivalent

This replaces curl for development testing. See the request. See the response. Adjust and try again. All in the browser.

---

11. A Complete Documented API

Here is a full product API with complete Swagger annotations:

```
import { Router } from "tina4-nodejs";

/**
 * List all products
 * @description Returns a paginated list of products. Supports filtering by category and sorting
 * @tags Products
 * @query string category Filter by category name
 * @query string sort Sort field (name, price, created_at)
 * @query string order Sort direction (ASC or DESC)
 * @query int page Page number (default: 1)
 * @query int limit Items per page (default: 20)
 * @response 200 {"products": [{"id": "int", "name": "string", "category": "string", "price": "f
 * @example response {"products": [{"id": 1, "name": "Keyboard", "category": "Electronics", "pri
 */
Router.get("/api/products", async (req, res) => {
    // implementation
});

/**
 * Get a product by ID
 * @description Returns full product details including category and stock status
 * @tags Products
 * @param int id Product ID
 * @response 200 {"id": "int", "name": "string", "category": "string", "price": "float", "inStoc
 * @response 404 {"error": "string"}
 * @example response {"id": 1, "name": "Wireless Keyboard", "category": "Electronics", "price":
 */
Router.get("/api/products/{id:int}", async (req, res) => {
    // implementation
});

/**
 * Create a product
```

```
 * @description Adds a new product to the catalog
 * @secured
 * @tags Products
 * @body {"name": "string", "category": "string", "price": "float", "inStock": "bool"}
 * @response 201 {"id": "int", "name": "string", "category": "string", "price": "float"}
 * @response 400 {"error": "string"}
 * @response 401 {"error": "string"}
 * @example request {"name": "Ergonomic Mouse", "category": "Electronics", "price": 49.99, "inSt
 * @example response {"id": 42, "name": "Ergonomic Mouse", "category": "Electronics", "price": 4
 */
Router.post("/api/products", async (req, res) => {
    // implementation
}, [authMiddleware]);

/**
 * Update a product
 * @description Updates an existing product. Only include fields you want to change.
 * @secured
 * @tags Products
 * @param int id Product ID
 * @body {"name": "string", "category": "string", "price": "float", "inStock": "bool"}
 * @response 200 {"id": "int", "name": "string", "category": "string", "price": "float"}
 * @response 404 {"error": "string"}
 * @response 401 {"error": "string"}
 * @example request {"price": 69.99}
 * @example response {"id": 1, "name": "Wireless Keyboard", "category": "Electronics", "price":
 */
Router.put("/api/products/{id:int}", async (req, res) => {
    // implementation
}, [authMiddleware]);

/**
 * Delete a product
 * @description Removes a product from the catalog
 * @secured
 * @tags Products
 * @param int id Product ID
 * @response 204
 * @response 404 {"error": "string"}
 * @response 401 {"error": "string"}
 */
Router.delete("/api/products/{id:int}", async (req, res) => {
    // implementation
}, [authMiddleware]);
```

---

12. Hiding Routes

Use `@hidden` to exclude a route from Swagger:

```
/**
 * Internal health check
 * @hidden
 */
Router.get("/internal/ping", async (req, res) => {
    return res.json({ pong: true });
});
```

The route still works. It does not appear in the Swagger UI or the JSON spec.

---

13. Customizing the Swagger Info Block

Configure the API metadata through `.env`:

```
TINA4_SWAGGER_TITLE=My Store API
TINA4_SWAGGER_DESCRIPTION=API for managing products, orders, and users
TINA4_SWAGGER_VERSION=1.0.0
TINA4_SWAGGER_CONTACT_EMAIL=api@mystore.com
TINA4_SWAGGER_LICENSE=MIT
```

These values appear in the header of the Swagger UI. Set them once. Every endpoint inherits the context.

---

14. Generating Client SDKs

The OpenAPI spec at `/swagger/json` feeds into code generators. Generate a typed API client for your frontend:

```
npm install -g @openapitools/openapi-generator-cli

openapi-generator-cli generate \
  -i http://localhost:7148/swagger/json \
  -g typescript-fetch \
  -o ./frontend/api-client
```

This produces a fully typed TypeScript client. Every endpoint becomes a method. Every request body and response has type definitions. The frontend developer gets autocompletion and type checking from your Swagger annotations.

Available Generators

Generator	Output
`typescript-fetch`	TypeScript with Fetch API
`typescript-axios`	TypeScript with Axios
`javascript`	Plain JavaScript
`python`	Python client
`java`	Java client
`swift5`	iOS Swift client
`kotlin`	Android Kotlin client
`csharp`	C# .NET client

One spec. Any language. The API documentation becomes a contract between your backend and every frontend that consumes it.

---

15. Auto-CRUD Routes in Swagger

Models with `static autoCrud = true` generate Swagger documentation automatically. The generated routes include:

- Summary and description
- Path parameters
- Query parameters for the list endpoint (page, limit, sort, order, filters)
- Request body schema from the model's `fields` definition
- Response schemas

No JSDoc annotations needed for auto-CRUD routes. The ORM model definition drives the documentation.

---

16. Exercise: Document a Complete User API

Document a User API with the following endpoints. Include full Swagger annotations: `@param`, `@query`, `@body`, `@response`, `@example`, `@tags`, and `@secured` where appropriate.

Requirements

- `PUT /api/users/{id}` -- Update user profile
- `GET /api/users/{id}/orders` -- List a user's orders (with status filter and pagination)
- `POST /api/users/{id}/avatar` -- Upload a user avatar URL

---

17. Solution

```
import { Router } from "tina4-nodejs";

/**
 * Update a user
 * @description Updates an existing user's profile information. Only send fields you want to cha
 * @secured
 * @tags Users
 * @param int id User ID
 * @body {"name": "string", "email": "string", "role": "string"}
 * @response 200 {"id": "int", "name": "string", "email": "string", "role": "string", "updatedAt
 * @response 404 {"error": "string"}
 * @response 401 {"error": "string"}
 * @example request {"name": "Alice Smith", "email": "alice.smith@example.com"}
 * @example response {"id": 1, "name": "Alice Smith", "email": "alice.smith@example.com", "role"
 */
Router.put("/api/users/{id:int}", async (req, res) => {
    return res.json({ id: req.params.id, name: req.body.name ?? "Alice", updatedAt: new Date().t
});

/**
 * List user orders
 * @description Returns a paginated list of orders for a specific user. Filter by status to see
 * @secured
 * @tags Users, Orders
 * @param int id User ID
 * @query string status Filter by order status (pending, shipped, delivered)
 * @query int page Page number (default: 1)
 * @query int limit Items per page (default: 20)
 * @response 200 {"orders": [{"id": "int", "product": "string", "total": "float", "status": "str
 * @response 404 {"error": "string"}
 * @example response {"orders": [{"id": 101, "product": "Wireless Keyboard", "total": 79.99, "st
 */
Router.get("/api/users/{id:int}/orders", async (req, res) => {
    return res.json({ orders: [], total: 0, page: parseInt(req.query.page ?? "1", 10) });
});

/**
 * Upload user avatar
 * @description Sets or updates the avatar URL for a user. The URL must point to an accessible i
 * @secured
 * @tags Users
 * @param int id User ID
 * @body {"avatarUrl": "string"}
 * @response 200 {"id": "int", "avatarUrl": "string", "updatedAt": "string"}
 * @response 400 {"error": "string"}
 * @response 404 {"error": "string"}
 * @example request {"avatarUrl": "https://cdn.example.com/avatars/alice.jpg"}
 * @example response {"id": 1, "avatarUrl": "https://cdn.example.com/avatars/alice.jpg", "update
 */
Router.post("/api/users/{id:int}/avatar", async (req, res) => {
    if (!req.body.avatarUrl) {
        return res.status(400).json({ error: "avatarUrl is required" });
    }
    return res.json({ id: req.params.id, avatarUrl: req.body.avatarUrl, updatedAt: new Date().to
});
```

---

18. Gotchas

1. Annotations Must Be Directly Above the Route

Problem: Swagger does not pick up your annotations.

Cause: A blank line or other code sits between the JSDoc comment and the `Router.get(...)` call.

Fix: The `*/` closing must be on the line directly before the Router call. No blank lines between them.

2. Missing @tags Makes Endpoints Hard to Find

Problem: All endpoints land in a "default" group in the Swagger UI.

Fix: Add `@tags ResourceName` to every route doc-block. Group related endpoints together.

3. @body Must Be Valid JSON

Problem: Swagger shows an error parsing the body schema.

Fix: Every key and string value must be in double quotes. `{"name": "string"}` works. `{name: string}` fails.

4. Swagger Shows Routes You Did Not Annotate

Problem: Un-annotated routes appear in Swagger with no documentation.

Fix: By design -- Tina4 shows all registered routes. Add `@hidden` to hide a route from Swagger.

5. Response Examples Do Not Match Actual Responses

Problem: The Swagger example shows one format but the actual response has different fields.

Fix: Update annotations when you change the handler's response format. Examples are static text in your source code. They do not update automatically.

6. Swagger UI Not Available in Production

Problem: Navigating to `/swagger` returns a 404 in production.

Fix: Set `TINA4_SWAGGER_ENABLED=true` in `.env` to enable Swagger in production. Without it, Swagger only runs when `TINA4_DEBUG=true`.

7. SDK Generation Produces Incorrect Types

Problem: The generated TypeScript client has `any` types instead of proper types.

Fix: Use correct OpenAPI type strings in your annotations: `"string"`, `"int"`, `"float"`, `"bool"`. Avoid using TypeScript type syntax -- Swagger annotations use their own type vocabulary.

8. @secured Routes Not Sending Auth Token

Problem: "Try it out" sends requests without the Authorization header even though the route is marked `@secured`.

Fix: Click the "Authorize" button at the top of the Swagger UI. Paste your JWT token. All subsequent requests include the header. The token persists until you close the tab or click "Logout."

API Client

1. Calling External APIs Without the Boilerplate

Your application calls a payment gateway. A shipping provider. A weather service. A CRM. Every call needs the same setup: base URL, auth header, error handling, JSON parsing, timeout.

Tina4 provides an `Api` class, a small HTTP client over Node's built-in `node:http`/`node:https` that handles the repetitive parts. It covers GET, POST, PUT, PATCH, and DELETE with JSON serialization, auth headers set once on the instance, and a consistent response format, all with no external dependencies.

---

2. The Api Class

Import `Api` from `@tina4/core` and construct an instance:

```
import { Api } from "@tina4/core";

const api = new Api("https://api.example.com/v2");
```

`Api` is a ready-to-use HTTP client. Construct it once. Reuse the instance everywhere.

---

3. Configuring the Client

The constructor takes the base URL, an optional `Authorization` header value, and an optional timeout **in seconds** (default 30). Auth and custom headers can also be set after construction:

```
import { Api } from "@tina4/core";

// Positional form: (baseUrl, authHeader, timeoutSeconds)
const api = new Api("https://api.example.com/v2", "", 10 /* seconds */);

api.addHeaders({ "X-App-Version": "1.0.0" });
api.setBearerToken(process.env.TINA4_API_KEY ?? "");

// Or the options-bag form (recommended):
const api2 = new Api("https://api.example.com/v2", {
    bearerToken: process.env.TINA4_API_KEY,
    headers: { "X-App-Version": "1.0.0" },
    timeout: 10,
});
```

Constructor arg	Default	Description
`baseUrl`	`""`	Prepended to every request path

`authHeaderOrOptions`	`""`	An `Authorization` header value, or an options bag (`bearerToken`, `username`/`password`, `headers`, `timeout`, `verifySsl`)
`timeout`	`30`	Request timeout in **seconds**

Instance setters:

Method	Description
`addHeaders(headers)`	Merge headers sent with every request
`setBearerToken(token)`	Set `Authorization: Bearer <token>`
`setBasicAuth(user, pass)`	Set HTTP Basic auth
`setIgnoreSsl(true)`	Skip TLS verification (dev / self-signed certs only)

---

4. GET Requests

```
import { Api } from "@tina4/core";

const api = new Api("https://api.example.com");
const response = await api.get("/products");

if (response.error === null && response.http_code === 200) {
    const products = response.body as unknown[];
    console.log(`Fetched ${products.length} products`);
} else {
    console.error(`Error ${response.http_code}: ${response.error}`);
}
```

Pass query parameters as a flat object in the second argument:

```
const response = await api.get("/products", {
    category: "Electronics",
    page: "2",
    limit: "20",
});
// Requests: GET /products?category=Electronics&page=2&limit=20
```

Query values are strings (they go straight into the query string).

---

5. POST Requests

```
import { Api } from "@tina4/core";

const api = new Api("https://api.example.com");
const response = await api.post("/orders", {
    customerId: 15,
    items: [
        { sku: "KB-001", qty: 1, price: 79.99 },
        { sku: "HDMI-2M", qty: 2, price: 12.99 }
    ],
    shippingAddress: {
        line1: "123 Main St",
        city: "Springfield",
        country: "US"
    }
});

if (response.error === null && response.http_code === 201) {
    const order = response.body as { orderId: string };
    console.log(`Order created: ${order.orderId}`);
} else {
    console.error("Order failed:", response.error ?? response.http_code);
}
```

The body is serialized as JSON automatically. The `Content-Type: application/json` header is set for you (override it with the optional third argument).

---

6. PUT and PATCH Requests

```
import { Api } from "@tina4/core";

const api = new Api("https://api.example.com");

// Replace the entire resource
const putResponse = await api.put("/products/42", {
    name: "Wireless Keyboard Pro",
    price: 89.99,
    inStock: true,
    category: "Electronics"
});

// Update specific fields only
const patchResponse = await api.patch("/products/42", {
    price: 74.99
});

if (putResponse.error === null && putResponse.http_code === 200) {
    console.log("Product updated:", putResponse.body);
}
```

---

7. DELETE Requests

```
import { Api } from "@tina4/core";

const api = new Api("https://api.example.com");
const response = await api.delete("/products/42");

if (response.http_code === 200 || response.http_code === 204) {
    console.log("Product deleted");
} else if (response.http_code === 404) {
    console.log("Product not found");
} else {
    console.error("Delete failed:", response.error ?? response.http_code);
}
```

---

8. Response Format

Every method returns the same `ApiResult` shape:

```
interface ApiResult {
    http_code: number | null;           // HTTP status code, or null if the request never reach
    body: unknown;                       // Parsed JSON body, or the raw string if not JSON
    headers: Record<string, string>;     // Response headers
    error: string | null;                // Non-null on transport failure or timeout
}
```

`error` is non-null only on a transport-level failure (connection refused, DNS, timeout). An HTTP error *response* (e.g. 404, 500) still arrives with `error: null`; inspect `http_code` to branch on it.

```
const response = await api.get("/products/999");

if (response.error !== null) {
    // Never reached the server
    return res.status(502).json({ error: "Upstream unreachable" });
}

if (response.http_code === 404) {
    return res.status(404).json({ error: "Product not found upstream" });
}
if (response.http_code !== 200) {
    return res.status(502).json({ error: "Upstream service error" });
}

return res.json(response.body);
```

---

9. Per-Request Content Type and Generic Requests

`post`/`put`/`patch` take an optional third argument to override the content type, and `sendRequest()` lets you issue any method:

```
import { Api } from "@tina4/core";

const api = new Api("https://api.example.com");
```

```
// Send a non-JSON body
await api.post("/upload", "<xml/>", "application/xml");

// Any HTTP method
const options = await api.sendRequest("OPTIONS", "/users");
```

Headers configured with `addHeaders()` / `setBearerToken()` are sent on every request from that instance. For different auth, construct a second `Api` instance.

---

10. Using Api Inside Route Handlers

Proxy or transform external API calls inside your routes. Construct the client once at module load and reuse it:

```
import { get } from "@tina4/core";
import { Api } from "@tina4/core";

const weatherApi = new Api("https://api.openweathermap.org/data/2.5");
weatherApi.addHeaders({ "Accept": "application/json" });

get("/api/weather/{city}", async (req, res) => {
    const city = req.params.city;

    const response = await weatherApi.get("/weather", {
        q: city,
        appid: process.env.OPENWEATHER_API_KEY ?? "",
        units: "metric"
    });

    if (response.error !== null || response.http_code !== 200) {
        return res.status(502).json({
            error: `Weather service error: ${response.error ?? response.http_code}`
        });
    }

    const weather = response.body as {
        name: string;
        main: { temp: number; humidity: number };
        weather: { description: string }[];
    };

    return res.json({
        city: weather.name,
        temperature: weather.main.temp,
        humidity: weather.main.humidity,
        description: weather.weather[0]?.description ?? "unknown"
    });
});
```

---

11. Exercise: GitHub User Profile Proxy

Build a route that fetches a GitHub user profile via an `Api` client and returns a simplified version.

Requirements

- Construct an `Api` with `https://api.github.com` as the base URL and a `User-Agent` header (GitHub requires one)
- Create a `GET /api/github/{username}` route that fetches the user's public profile
- Return only: `login`, `name`, `public_repos`, `followers`, `following`, `bio`, and `avatar_url`
- Return `404` with a message if the GitHub user does not exist

Test with:

```
curl http://localhost:7148/api/github/torvalds
curl http://localhost:7148/api/github/this-user-does-not-exist-xyzabc
```

---

12. Solution

`src/services/github.ts`:

```
import { Api } from "@tina4/core";

export const githubApi = new Api("https://api.github.com", "", 8 /* seconds */);
githubApi.addHeaders({
    "User-Agent": "tina4-book-example/1.0",
    "Accept": "application/vnd.github+json"
});
```

`src/routes/github.ts`:

```
import { get } from "@tina4/core";
import { githubApi } from "../services/github";

interface GitHubUser {
    login: string;
    name: string | null;
    bio: string | null;
    public_repos: number;
    followers: number;
    following: number;
    avatar_url: string;
}

get("/api/github/{username}", async (req, res) => {
    const { username } = req.params;

    const response = await githubApi.get(`/users/${username}`);

    if (response.error !== null) {
        return res.status(502).json({ error: "GitHub API unreachable", detail: response.error })
    }
    if (response.http_code === 404) {
        return res.status(404).json({ error: `GitHub user '${username}' not found` });
    }
    if (response.http_code !== 200) {
        return res.status(502).json({ error: "GitHub API error", detail: response.http_code });
    }
```

```
    const user = response.body as GitHubUser;

    return res.json({
        login: user.login,
        name: user.name,
        bio: user.bio,
        public_repos: user.public_repos,
        followers: user.followers,
        following: user.following,
        avatar_url: user.avatar_url
    });
});
```

```
curl http://localhost:7148/api/github/torvalds
```

```
{
  "login": "torvalds",
  "name": "Linus Torvalds",
  "bio": null,
  "public_repos": 7,
  "followers": 234156,
  "following": 0,
  "avatar_url": "https://avatars.githubusercontent.com/u/1024025?v=4"
}
```

---

13. Gotchas

1. `http_code` vs `error`

`error` is set only when the request never reaches the server (timeout, connection refused). A `404` or `503` *response* arrives with `error: null`; branch on `http_code` for those.

Fix: Check `response.error` first for transport failures, then check `response.http_code` for HTTP-level branches.

2. Timeout is in seconds

The constructor's third argument is **seconds**, not milliseconds: `new Api(url, "", 10)` is a 10-second timeout. The default is 30 seconds.

Fix: Set an explicit timeout appropriate for your service SLA. For real-time endpoints, 5-10 seconds is usually the right upper bound.

3. Reuse the instance

Constructing a new `Api` for every request re-applies headers each time and adds noise.

Fix: Construct one `Api` per upstream service at module load, configure its headers once, and reuse it.

4. Sending secrets in URLs

Appending API keys as query parameters (e.g., `?apikey=secret`) logs the key to access logs.

Fix: Pass secrets in headers. Use `api.setBearerToken(token)` or `api.addHeaders({ "X-API-Key": key })` once on the instance.

GraphQL and SOAP

1. The Problem GraphQL Solves

Your mobile app needs a list of products. Each product carries a name, price, 20 image URLs, a full description, 15 review objects, and 8 fields you do not need. REST sends all of it -- 50KB of JSON when you needed 2KB. On a mobile connection, that waste stings.

Now your web dashboard needs the same products plus category, stock status, and supplier info. REST forces you to make three requests and stitch them together, or build a custom endpoint with query parameter parsing.

GraphQL ends both problems. The client asks for the fields it needs. The server returns those fields. One endpoint. One request. One response shaped to fit.

Tina4 includes a built-in GraphQL engine. No external packages. No Apollo Server. No graphql-yoga. Part of the framework.

---

2. GraphQL vs REST -- A Quick Comparison

Aspect	REST	GraphQL
Endpoints	One per resource (`/products`, `/users`)	One endpoint (`/graphql`)
Data shape	Server decides	Client decides
Over-fetching	Common (get all fields)	Never (get only requested fields)
Under-fetching	Common (multiple requests needed)	Never (get related data in one query)
Versioning	`/api/v1/`, `/api/v2/`	Schema evolves, no versioning needed
Caching	HTTP caching works naturally	Requires client-side cache management
Learning curve	Low	Moderate

REST is still great for simple APIs. GraphQL shines when clients have diverse data needs -- mobile apps, dashboards, third-party integrations.

---

3. Enabling GraphQL

GraphQL is available by default in Tina4. The engine serves requests at `/graphql`. If you want to change the endpoint, set it in `.env`:

```
TINA4_GRAPHQL_ENDPOINT=/graphql
```

To verify it is working, start the server and send a test query:

```
curl -X POST http://localhost:7148/graphql \
  -H "Content-Type: application/json" \
  -d '{"query": "{ __schema { queryType { name } } }"}'
```

```
{
  "data": {
    "__schema": {
      "queryType": {
        "name": "Query"
      }
    }
  }
}
```

If you see that response, GraphQL is running.

---

4. Defining a Schema

A GraphQL schema defines the types of data your API can return and the queries and mutations clients can execute.

Create `src/graphql/schema.graphql`:

```
type Product {
    id: Int!
    name: String!
    category: String!
    price: Float!
    inStock: Boolean!
    createdAt: String
}

type Query {
    products: [Product!]!
    product(id: Int!): Product
}
```

This schema says:

- A `Product` has six fields. The `!` means the field is non-nullable.
- The `Query` type has two operations: `products` returns a list of products, and `product(id)` returns a single product (or null if not found).

Tina4 auto-discovers `.graphql` files in `src/graphql/`. You do not need to register them.

---

5. Writing Resolvers

A resolver is the function that runs when a query or mutation is executed. Resolvers live in `src/graphql/` as TypeScript files.

Create `src/graphql/resolvers.ts`:

```
import { GraphQL } from "tina4-nodejs";
import { Product } from "../orm/Product";

// Resolve the "products" query
GraphQL.resolve("Query", "products", async (root, args) => {
    const product = new Product();
    const products = await product.select("*", "", {}, "name ASC");
    return products.map(p => p.toDict());
});

// Resolve the "product" query
GraphQL.resolve("Query", "product", async (root, args) => {
    const product = new Product();
    await product.load(args.id);
    return product.id ? product.toDict() : null;
});
```

`GraphQL.resolve()` takes three arguments:

- **Type name** -- The GraphQL type this resolver belongs to (`Query`, `Mutation`, or a custom type).
- **Field name** -- The field within the type this resolver handles.
- **Resolver function** -- Receives `root` (the parent value, if any) and `args` (the query arguments).

Testing the Queries

List all products:

```
curl -X POST http://localhost:7148/graphql \
  -H "Content-Type: application/json" \
  -d '{"query": "{ products { id name price inStock } }"}'
```

```
{
  "data": {
    "products": [
      {"id": 1, "name": "Wireless Keyboard", "price": 79.99, "inStock": true},
      {"id": 2, "name": "USB-C Hub", "price": 49.99, "inStock": true},
      {"id": 3, "name": "Standing Desk", "price": 549.99, "inStock": false}
    ]
  }
}
```

Notice: the response contains only the four fields we asked for (`id`, `name`, `price`, `inStock`), not `category` or `createdAt`. That is GraphQL in action.

Get a single product with all fields:

```
curl -X POST http://localhost:7148/graphql \
  -H "Content-Type: application/json" \
```

```
  -d '{"query": "{ product(id: 1) { id name category price inStock createdAt } }"}'
```

```
{
  "data": {
    "product": {
      "id": 1,
      "name": "Wireless Keyboard",
      "category": "Electronics",
      "price": 79.99,
      "inStock": true,
      "createdAt": "2026-03-22 14:30:00"
    }
  }
}
```

Request a product that does not exist:

```
curl -X POST http://localhost:7148/graphql \
  -H "Content-Type: application/json" \
  -d '{"query": "{ product(id: 999) { id name } }"}'
```

```
{
  "data": {
    "product": null
  }
}
```

---

6. Mutations

Mutations are how clients create, update, or delete data. They are defined in the schema alongside queries.

Update `src/graphql/schema.graphql`:

```
type Product {
    id: Int!
    name: String!
    category: String!
    price: Float!
    inStock: Boolean!
    createdAt: String
}

input ProductInput {
    name: String!
    category: String
    price: Float!
    inStock: Boolean
}

input ProductUpdateInput {
    name: String
    category: String
    price: Float
    inStock: Boolean
}

type DeleteResult {
```

```
    success: Boolean!
    message: String!
}

type Query {
    products: [Product!]!
    product(id: Int!): Product
    productsByCategory(category: String!): [Product!]!
}

type Mutation {
    createProduct(input: ProductInput!): Product!
    updateProduct(id: Int!, input: ProductUpdateInput!): Product
    deleteProduct(id: Int!): DeleteResult!
}
```

Key concepts:

- `input` **types** define the shape of data the client sends. They are like regular types but used for arguments.
- `Mutation` **type** lists all write operations. By convention, mutations are named with verbs: `createProduct`, `updateProduct`, `deleteProduct`.
- Fields in `ProductUpdateInput` are all optional (no `!`) because the client may only want to update one field.

Mutation Resolvers

Add to `src/graphql/resolvers.ts`:

```
GraphQL.resolve("Mutation", "createProduct", async (root, args) => {
    const input = args.input;
    const product = new Product();
    product.name = input.name;
    product.category = input.category ?? "Uncategorized";
    product.price = parseFloat(input.price);
    product.inStock = Boolean(input.inStock ?? true);
    await product.save();
    return product.toDict();
});

GraphQL.resolve("Mutation", "updateProduct", async (root, args) => {
    const product = new Product();
    await product.load(args.id);

    if (!product.id) return null;

    const input = args.input;
    if (input.name !== undefined) product.name = input.name;
    if (input.category !== undefined) product.category = input.category;
    if (input.price !== undefined) product.price = parseFloat(input.price);
    if (input.inStock !== undefined) product.inStock = Boolean(input.inStock);
    await product.save();

    return product.toDict();
});

GraphQL.resolve("Mutation", "deleteProduct", async (root, args) => {
```

```
    const product = new Product();
    await product.load(args.id);
    if (!product.id) return { success: false, message: "Product not found" };
    await product.delete();
    return { success: true, message: "Product deleted" };
});

GraphQL.resolve("Query", "productsByCategory", async (root, args) => {
    const product = new Product();
    const products = await product.select("*", "category = :category", { category: args.category
    return products.map(p => p.toDict());
});
```

Testing Mutations

Create a product:

```
curl -X POST http://localhost:7148/graphql \
  -H "Content-Type: application/json" \
  -d '{
    "query": "mutation { createProduct(input: { name: \"Desk Lamp\", category: \"Office\", price
  }'
```

```
{
  "data": {
    "createProduct": {
      "id": 4,
      "name": "Desk Lamp",
      "price": 39.99
    }
  }
}
```

Update a product:

```
curl -X POST http://localhost:7148/graphql \
  -H "Content-Type: application/json" \
  -d '{
    "query": "mutation { updateProduct(id: 4, input: { price: 44.99, inStock: false }) { id name
  }'
```

```
{
  "data": {
    "updateProduct": {
      "id": 4,
      "name": "Desk Lamp",
      "price": 44.99,
      "inStock": false
    }
  }
}
```

Delete a product:

```
curl -X POST http://localhost:7148/graphql \
  -H "Content-Type: application/json" \
  -d '{
    "query": "mutation { deleteProduct(id: 4) { success message } }"
  }'
```

```
{
```

```
  "data": {
    "deleteProduct": {
      "success": true,
      "message": "Product deleted"
    }
  }
}
```

---

7. Nested Types and Relationships

The real power of GraphQL: traversing relationships in a single query. Authors, posts, comments -- all in one request.

Update `src/graphql/schema.graphql`:

```
type User {
    id: Int!
    name: String!
    email: String!
    posts: [Post!]!
}

type Post {
    id: Int!
    title: String!
    body: String!
    published: Boolean!
    author: User!
    comments: [Comment!]!
    commentCount: Int!
}

type Comment {
    id: Int!
    authorName: String!
    body: String!
    post: Post!
}

type Query {
    posts: [Post!]!
    post(id: Int!): Post
    user(id: Int!): User
}

type Mutation {
    createPost(userId: Int!, title: String!, body: String!, published: Boolean): Post!
    addComment(postId: Int!, authorName: String!, body: String!): Comment!
}
```

Add resolvers for the nested types:

```
import { GraphQL } from "tina4-nodejs";
import { Post } from "../orm/Post";
import { User } from "../orm/User";
import { Comment } from "../orm/Comment";
```

```
GraphQL.resolve("Query", "posts", async (root, args) => {
    const post = new Post();
    const posts = await post.select("*", "published = :pub", { pub: 1 });
    return posts.map(p => p.toDict());
});

GraphQL.resolve("Query", "post", async (root, args) => {
    const post = new Post();
    await post.load(args.id);
    return post.id ? post.toDict() : null;
});

// Resolve Post.author (nested field)
GraphQL.resolve("Post", "author", async (post, args) => {
    const user = new User();
    await user.load(post.user_id);
    return user.toDict();
});

// Resolve Post.comments (nested field)
GraphQL.resolve("Post", "comments", async (post, args) => {
    const comment = new Comment();
    const comments = await comment.select("*", "post_id = :postId", { postId: post.id });
    return comments.map(c => c.toDict());
});

// Resolve Post.commentCount (computed field)
GraphQL.resolve("Post", "commentCount", async (post, args) => {
    const comment = new Comment();
    const result = await comment.select("count(*) as cnt", "post_id = :postId", { postId: post.i
    return parseInt(result[0]?.cnt ?? 0, 10);
});

// Resolve User.posts (nested field)
GraphQL.resolve("User", "posts", async (user, args) => {
    const post = new Post();
    const posts = await post.select("*", "user_id = :userId", { userId: user.id });
    return posts.map(p => p.toDict());
});
```

Now clients can query across relationships in a single request:

```
curl -X POST http://localhost:7148/graphql \
  -H "Content-Type: application/json" \
  -d '{
    "query": "{ posts { id title author { name email } comments { authorName body } commentCount
  }'
```

```
{
  "data": {
    "posts": [
      {
        "id": 1,
        "title": "My First Post",
        "author": {
          "name": "Alice",
          "email": "alice@example.com"
        },
        "comments": [
          {"authorName": "Bob", "body": "Great post!"}
```

```
        ],
        "commentCount": 1
      }
    ]
  }
}
```

One request. The fields the client needs. No over-fetching. No under-fetching.

---

8. Auto-Generating Schema from ORM Models

If your ORM models have `static autoCrud = true`, Tina4 can auto-generate GraphQL types and basic CRUD resolvers for them. Enable this in `.env`:

```
TINA4_GRAPHQL_AUTO_SCHEMA=true
```

With this setting, every ORM model with `static autoCrud = true` gets:

- A GraphQL type with fields matching the model properties
- A query to list all records (e.g., `products`)
- A query to get one by ID (e.g., `product(id: Int!)`)
- A mutation to create (e.g., `createProduct(input: ProductInput!)`)
- A mutation to update (e.g., `updateProduct(id: Int!, input: ProductUpdateInput!)`)
- A mutation to delete (e.g., `deleteProduct(id: Int!)`)

You can still define custom resolvers that override the auto-generated ones. Custom resolvers take precedence.

---

9. The GraphiQL Playground

When `TINA4_DEBUG=true`, Tina4 serves a GraphiQL interactive playground at:

```
http://localhost:7148/graphql/playground
```

GraphiQL gives you:

- A query editor with syntax highlighting and auto-completion
- A documentation explorer showing all types, queries, and mutations
- A results panel showing the response
- Query history

This is the fastest way to explore and test your GraphQL API during development. Type a query on the left, click the play button, and see the results on the right. The documentation explorer on the right sidebar shows every type, field, and argument available in your schema.

GraphiQL is only available when `TINA4_DEBUG=true`. In production, it is disabled.

---

10. Query Variables

For production use, clients should send query variables separately from the query string. This prevents injection and allows query caching.

```
curl -X POST http://localhost:7148/graphql \
  -H "Content-Type: application/json" \
  -d '{
    "query": "query GetProduct($id: Int!) { product(id: $id) { id name price } }",
    "variables": {"id": 1}
  }'
```

```
{
  "data": {
    "product": {
      "id": 1,
      "name": "Wireless Keyboard",
      "price": 79.99
    }
  }
}
```

The query uses `$id` as a placeholder, and the actual value comes from the `variables` object. This is safer and more efficient than string interpolation.

---

11. Exercise: Build a GraphQL API for a Blog

Build a complete GraphQL API for a blog with posts, authors, and comments.

Requirements

- **Schema** -- Define types for `User`, `Post`, and `Comment` with the following fields:
- `User`: id, name, email, posts (nested)
- `Post`: id, title, body, published, author (nested), comments (nested), commentCount (computed)
- `Comment`: id, authorName, body, createdAt
- **Queries**:
- `posts` -- List all published posts
- `post(id: Int!)` -- Get a single post by ID
- `user(id: Int!)` -- Get a user with their posts
- **Mutations**:
- `createPost(userId: Int!, title: String!, body: String!, published: Boolean)` -- Create a new post
- `addComment(postId: Int!, authorName: String!, body: String!)` -- Add a comment to a post

- Use the ORM models from Chapter 6 (User, Post, Comment).

Test with these queries:

```
# List published posts with author names
curl -X POST http://localhost:7148/graphql \
  -H "Content-Type: application/json" \
  -d '{"query": "{ posts { id title author { name } commentCount } }"}'

# Get a specific post with all comments
curl -X POST http://localhost:7148/graphql \
  -H "Content-Type: application/json" \
  -d '{"query": "{ post(id: 1) { title body author { name email } comments { authorName body } }

# Create a post
curl -X POST http://localhost:7148/graphql \
  -H "Content-Type: application/json" \
  -d '{"query": "mutation { createPost(userId: 1, title: \"GraphQL is great\", body: \"Here is w

# Add a comment
curl -X POST http://localhost:7148/graphql \
  -H "Content-Type: application/json" \
  -d '{"query": "mutation { addComment(postId: 1, authorName: \"Carol\", body: \"Nice article!\"
```

---

12. Solution

Schema

Create `src/graphql/blog-schema.graphql`:

```
type User {
    id: Int!
    name: String!
    email: String!
    posts: [Post!]!
}

type Post {
    id: Int!
    title: String!
    body: String!
    published: Boolean!
    author: User!
    comments: [Comment!]!
    commentCount: Int!
    createdAt: String
}

type Comment {
    id: Int!
    authorName: String!
    body: String!
    createdAt: String
}

type Query {
    posts: [Post!]!
```

```
    post(id: Int!): Post
    user(id: Int!): User
}

type Mutation {
    createPost(userId: Int!, title: String!, body: String!, published: Boolean): Post!
    addComment(postId: Int!, authorName: String!, body: String!): Comment!
}
```

Resolvers

Create `src/graphql/blog-resolvers.ts`:

```
import { GraphQL } from "tina4-nodejs";
import { Post } from "../orm/Post";
import { User } from "../orm/User";
import { Comment } from "../orm/Comment";

GraphQL.resolve("Query", "posts", async () => {
    const post = new Post();
    const posts = await post.select("*", "published = :pub", { pub: 1 }, "created_at DESC");
    return posts.map(p => p.toDict());
});

GraphQL.resolve("Query", "post", async (root, args) => {
    const post = new Post();
    await post.load(args.id);
    return post.id ? post.toDict() : null;
});

GraphQL.resolve("Query", "user", async (root, args) => {
    const user = new User();
    await user.load(args.id);
    return user.id ? user.toDict() : null;
});

GraphQL.resolve("Post", "author", async (post) => {
    const user = new User();
    await user.load(post.user_id);
    return user.toDict();
});

GraphQL.resolve("Post", "comments", async (post) => {
    const comment = new Comment();
    const comments = await comment.select("*", "post_id = :postId", { postId: post.id }, "create
    return comments.map(c => c.toDict());
});

GraphQL.resolve("Post", "commentCount", async (post) => {
    const comment = new Comment();
    const result = await comment.select("count(*) as cnt", "post_id = :postId", { postId: post.i
    return parseInt(result[0]?.cnt ?? 0, 10);
});

GraphQL.resolve("User", "posts", async (user) => {
    const post = new Post();
    const posts = await post.select("*", "user_id = :userId", { userId: user.id }, "created_at D
    return posts.map(p => p.toDict());
});
```

```
GraphQL.resolve("Mutation", "createPost", async (root, args) => {
    const user = new User();
    await user.load(args.userId);

    if (!user.id) {
        throw new Error("User not found");
    }

    const post = new Post();
    post.userId = args.userId;
    post.title = args.title;
    post.body = args.body;
    post.published = Boolean(args.published ?? false);
    await post.save();

    return post.toDict();
});

GraphQL.resolve("Mutation", "addComment", async (root, args) => {
    const post = new Post();
    await post.load(args.postId);

    if (!post.id) {
        throw new Error("Post not found");
    }

    const comment = new Comment();
    comment.postId = args.postId;
    comment.authorName = args.authorName;
    comment.body = args.body;
    await comment.save();

    return comment.toDict();
});
```

Expected output for `{ posts { id title author { name } commentCount } }`:

```
{
  "data": {
    "posts": [
      {
        "id": 1,
        "title": "My First Post",
        "author": {"name": "Alice"},
        "commentCount": 1
      },
      {
        "id": 2,
        "title": "GraphQL is great",
        "author": {"name": "Alice"},
        "commentCount": 0
      }
    ]
  }
}
```

---

13. Input Validation

Tina4's GraphQL engine validates every argument against its declared type before your resolver runs. You never need to check whether a required argument is present or whether a string arrived where an integer was expected -- the engine rejects the query before your code executes.

The built-in validator covers:

- **Non-null enforcement** -- Arguments marked with `!` must be present and non-null.
- **Scalar type checking** -- `Int`, `Float`, `String`, `Boolean`, and `ID` values are verified against their declared type.
- **Type coercion** -- When safe, the engine coerces compatible values automatically (e.g., an integer `42` passed to a `Float` field becomes `42.0`, or a string `"7"` passed to an `Int` field becomes `7`).
- **List item validation** -- For `[Int!]!` arguments, the engine checks that the value is a list, that no item is null, and that every item is an integer.

Example: Missing Required Argument

Suppose you define a query that requires an `id`:

```
import { GraphQL } from "tina4-nodejs";
import { User } from "../orm/User";

GraphQL.resolve("Query", "user", async (root, args) => {
    const user = new User();
    await user.load(args.id);
    return user.id ? user.toDict() : null;
});
```

With the schema declaring `user(id: ID!): User`, sending a query without the required argument:

```
{
  user {
    name
    email
  }
}
```

Returns an error before the resolver runs:

```
{
  "errors": [
    {
      "message": "Argument 'id' of type 'ID!' is required but not provided.",
      "locations": [{"line": 2, "column": 3}]
    }
  ]
}
```

Your resolver never executes. No undefined-check needed inside the function.

---

14. Field-Level Auth Directives

Tina4 supports three directives that control field-level access in your GraphQL schema:

Directive	Effect
`@auth`	Field resolves only when `ctx.user` is present (any authenticated user)
`@role(role: "admin")`	Field resolves only when `ctx.user.role` matches the specified role
`@guest`	Field resolves only when `ctx.user` is absent (unauthenticated visitors)

Passing Auth Context

Pass user information through the context parameter of `execute()`:

```
const result = await gql.execute(query, variables, {
    user: { id: 1, role: "admin" }
});
```

When no user is logged in, pass an empty context or omit the `user` key:

```
const result = await gql.execute(query, variables, {});
```

Schema Example

```
type User {
    id: ID!
    name: String!
    email: String! @auth
    role: String! @role(role: "admin")
}

type Query {
    me: User @auth
    publicStats: Stats @guest
    users: [User!]! @role(role: "admin")
}
```

Behavior on Failed Auth

When a directive check fails, the field resolves to `null` silently -- no error is added to the response. The rest of the query executes normally. This prevents leaking information about which fields exist behind authentication.

For example, if a non-admin user queries:

```
{
  users {
    name
    email
    role
  }
}
```

The response is:

```
{
  "data": {
    "users": null
  }
}
```

No error message. No hint that the field requires admin access. The field is simply excluded.

---

15. Gotchas

1. Schema File Not Found

Problem: Queries return errors about unknown types or fields.

Cause: The `.graphql` file is not in `src/graphql/`, or has a syntax error that prevents parsing.

Fix: Make sure schema files are in `src/graphql/` and end with `.graphql`. Check for syntax errors -- a missing `!` or unmatched brace will cause the entire schema to fail silently.

2. Resolver Not Called

Problem: A query returns `null` even though data exists.

Cause: The resolver is not registered, or the type/field names do not match the schema. GraphQL is case-sensitive.

Fix: Verify that `GraphQL.resolve("Query", "products", ...)` matches `type Query { products: ... }` in your schema. Check capitalization.

3. Nested Resolver Returns Wrong Data

Problem: `post.author` returns the entire post instead of the user.

Cause: The nested resolver receives the parent object as its first argument (`post`), not a fresh model. If you return the parent instead of loading the related record, you get the wrong data.

Fix: In a nested resolver, load the related record from the database using the foreign key from the parent:

```
GraphQL.resolve("Post", "author", async (post, args) => {
    const user = new User();
    await user.load(post.user_id);  // Use the foreign key from the parent
    return user.toDict();
});
```

4. Mutation Input Not Parsed

Problem: `args.input` is `undefined` inside a mutation resolver.

Cause: The mutation signature in the schema uses an `input` type, but the client query passes arguments directly instead of wrapping them in an `input` object.

Fix: Match the query to the schema. If the schema says `createProduct(input: ProductInput!)`, the client must send `createProduct(input: { name: "Widget", price: 9.99 })`, not `createProduct(name: "Widget", price: 9.99)`.

5. N+1 Query Problem

Problem: Loading 50 posts with their authors generates 51 database queries (1 for posts + 50 for authors).

Cause: Each nested resolver runs on its own. When you resolve `author` for each post, that is one query per post.

Fix: Use data loader patterns or batch loading. For simple cases, pre-load all related records in the parent resolver and pass them down. Check `ctx.__selections` to see which nested fields the client requested -- if `author` is not in the selection set, skip the join. For ORM queries, use the `include` parameter to eager-load relationships in a single query. For complex cases, consider using the REST API with eager loading instead of GraphQL for that particular endpoint.

6. GraphQL Playground Returns 404

Problem: `/graphql/playground` returns a 404 error.

Cause: `TINA4_DEBUG` is set to `false`. The playground is only available in debug mode.

Fix: Set `TINA4_DEBUG=true` in your `.env` file. The playground is disabled in production by design.

7. Type Mismatch Between Schema and Resolver

Problem: A field declared as `Int!` in the schema receives a string from the resolver, causing a type error.

Cause: JavaScript's loose typing means your resolver might return `"42"` (a string) for an `Int!` field. GraphQL rejects it.

Fix: The input validation layer (see section 13) now coerces compatible types automatically -- a string `"42"` passed as an argument to an `Int` parameter is converted before your resolver runs. However, resolver *return values* still need correct types. Cast values explicitly with `parseInt()`, `parseFloat()`, `Boolean()`:

```
return {
    id: parseInt(product.id, 10),
    price: parseFloat(product.price),
    inStock: Boolean(product.inStock)
};
```

The `toDict()` method handles this for model properties with type declarations, but computed or derived fields need manual casting.

---

14. SOAP / WSDL Services

What is SOAP?

SOAP (Simple Object Access Protocol) is an XML-based messaging protocol for exchanging structured data between systems. It predates REST and GraphQL, but remains common in enterprise integrations -- banking, healthcare, government, and ERP systems all rely on SOAP services.

A WSDL (Web Services Description Language) file describes a SOAP service: what operations are available, what parameters they accept, and what they return. Clients use the WSDL to auto-generate code for calling the service.

Tina4 includes a built-in SOAP 1.1 / WSDL 1.1 engine. Zero external dependencies -- XML parsing uses simple string matching.

Defining a SOAP Service

Create a class that extends `WSDLService`. Each method you want to expose gets the `@WSDLOp` decorator with `input` and `output` type maps.

```
import { WSDLService, WSDLOp } from "tina4-nodejs";

class Calculator extends WSDLService {
  serviceName = "Calculator";
  serviceUrl = "/api/calculator";

  @WSDLOp({
    description: "Add two numbers",
    input: { a: "int", b: "int" },
    output: { Result: "int" },
  })
  async Add(a: number, b: number): Promise<Record<string, unknown>> {
    return { Result: a + b };
  }

  @WSDLOp({
    description: "Multiply two numbers",
    input: { x: "float", y: "float" },
    output: { Product: "double" },
  })
  async Multiply(x: number, y: number): Promise<Record<string, unknown>> {
    return { Product: x * y };
  }
}
```

Key points:

- `serviceName` -- appears in the WSDL `<service>` element.
- `serviceUrl` -- the URL path for both WSDL and SOAP requests.
- `input` -- maps parameter names to type strings.
- `output` -- maps return field names to type strings.

• The method receives parameters in the order declared in `input` and returns a plain object matching `output`.

Registering the Service

Call `register()` with your router to wire up two routes:

```
const calc = new Calculator();
calc.register(router);
```

This creates:

Method	URL	Purpose
GET	`/api/calculator?wsdl`	Returns the auto-generated WSDL XML
POST	`/api/calculator`	Accepts SOAP XML requests

Auto-Generated WSDL

Fetch the WSDL document:

```
curl http://localhost:7148/api/calculator?wsdl
```

Tina4 generates a complete WSDL 1.1 document containing `<types>`, `<message>`, `<portType>`, `<binding>`, and `<service>` sections. The endpoint URL is inferred from the request's `Host` header.

Type Mappings

The `input` and `output` maps use short type names that are converted to XSD types:

Tina4 Type	XSD Type
`int`, `integer`	`xsd:int`
`float`	`xsd:float`
`double`, `number`, `numeric`	`xsd:double`
`string`	`xsd:string`
`bool`, `boolean`	`xsd:boolean`

Unknown types default to `xsd:string`.

Lifecycle Hooks

Override `onRequest` and `onResult` to add validation, logging, or transformation around every operation call.

```
import { WSDLService, WSDLOp } from "tina4-nodejs";

class AuditedService extends WSDLService {
  serviceName = "AuditedService";
  serviceUrl = "/api/audited";

  onRequest(request: any): void {
    console.log(`SOAP request from ${request.headers["x-forwarded-for"] ?? "unknown"}`);
  }

  onResult(result: Record<string, unknown>): Record<string, unknown> {
    result["Timestamp"] = new Date().toISOString();
    return result;
  }

  @WSDLOp({
    description: "Health check",
    input: {},
    output: { Status: "string", Timestamp: "string" },
  })
  async Ping(): Promise<Record<string, unknown>> {
    return { Status: "ok" };
  }
}
```

Testing with curl

Fetch the WSDL definition:

```
curl http://localhost:7148/api/calculator?wsdl
```

Send a SOAP request to the `Add` operation:

```
curl -X POST http://localhost:7148/api/calculator \
  -H "Content-Type: text/xml" \
  -d '<?xml version="1.0" encoding="UTF-8"?>
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Body>
    <Add>
      <a>5</a>
      <b>3</b>
    </Add>
  </soap:Body>
</soap:Envelope>'
```

Response:

```
<?xml version="1.0" encoding="UTF-8"?>
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Body>
    <AddResponse>
      <Result>8</Result>
    </AddResponse>
```

```
  </soap:Body>
</soap:Envelope>
```

If the operation name is wrong or the XML is malformed, the service returns a SOAP fault:

```
<soap:Fault>
  <faultcode>Client</faultcode>
  <faultstring>Unknown operation: Subtract</faultstring>
</soap:Fault>
```

A More Complete Example

A user lookup service with multiple operations:

```
import { WSDLService, WSDLOp } from "tina4-nodejs";

class UserService extends WSDLService {
  serviceName = "UserService";
  serviceUrl = "/api/users/soap";

  @WSDLOp({
    description: "Look up a user by ID",
    input: { userId: "int" },
    output: { Name: "string", Email: "string", Active: "boolean" },
  })
  async GetUser(userId: number): Promise<Record<string, unknown>> {
    // Replace with real database lookup
    return { Name: "Alice", Email: "alice@example.com", Active: true };
  }

  @WSDLOp({
    description: "Search users by name",
    input: { query: "string" },
    output: { Count: "int", Names: "string" },
  })
  async SearchUsers(query: string): Promise<Record<string, unknown>> {
    return { Count: 1, Names: "Alice" };
  }
}
```

SOAP Gotchas

- **Content-Type must be** `text/xml` -- SOAP requests are XML, not JSON.
- **Operation name must match** -- the element name inside `<Body>` must match your method name (case-sensitive).
- **Parameter order matters** -- values are extracted in the order declared in `input`.
- **Namespace prefixes are handled** -- Tina4 strips namespace prefixes when matching element names, so `<ns1:Add>` works the same as `<Add>`.

When to Use SOAP vs REST vs GraphQL

Scenario	Use
New API for web/mobile clients	REST or GraphQL
Integrating with legacy enterprise systems (SAP, banks, government)	SOAP
Clients require a machine-readable service contract	SOAP (WSDL)
Simple internal microservices	REST
Clients with diverse data needs	GraphQL

SOAP is rarely the right choice for new APIs, but when you need to expose a service that legacy systems can consume, Tina4 makes it straightforward -- define a class, annotate your types, and the framework handles the XML.

Real-time with WebSocket

1. The Refresh Button Problem

Your project management app needs live updates. Someone moves a card from "In Progress" to "Done." Everyone else on the team should see it. No page refresh. No polling. No waiting.

Traditional HTTP is request-response. The client asks. The server answers. The server cannot push data on its own. WebSocket breaks that wall. A persistent, bi-directional connection between browser and server. Either side sends messages at any time. The connection stays open until one side closes it.

Tina4 treats WebSocket the same way it treats routing. Define a WebSocket handler the same way you define an HTTP route. It runs on Node's built-in HTTP server -- no `ws` or `socket.io` required.

---

2. What WebSocket Is

HTTP works like this:

```
Client: "Give me /api/products"
Server: "Here are the products" (connection closes)
Client: "Give me /api/products" (new connection)
Server: "Here are the products" (connection closes)
```

WebSocket works like this:

```
Client: "I want to upgrade to WebSocket on /ws/chat"
Server: "Upgrade accepted. Connection open."
Client: "Hello everyone!"
Server: "Alice says: Hello everyone!"  (pushed to all clients)
Server: "Bob joined the chat"  (pushed to all clients, at any time)
Client: "Goodbye!"
Server: "Alice says: Goodbye!"
Client: (closes connection)
```

Key differences:

- **Persistent**: The connection stays open. No repeated handshakes.
- **Bi-directional**: The server pushes data without the client asking.
- **Low overhead**: After the initial handshake, messages are tiny. No HTTP headers per message.
- **Real-time**: Messages arrive within milliseconds.

---

3. Router.websocket() -- WebSocket as a Route

In Tina4, you define WebSocket handlers using `Router.websocket()`:

```
import { Router } from "tina4-nodejs";
```

```
Router.websocket("/ws/echo", (connection, event, data) => {
    if (event === "message") {
        connection.send(`Echo: ${data}`);
    }
});
```

This is the simplest WebSocket handler: it receives a message and sends it back with "Echo: " prepended. The callback receives three arguments:

	Type	Description		
	`WebSocketConnection`	The client connection object. Use it to send messages.		
	`"open" \	"message" \	"close"`	Which lifecycle
	`string`	The message text (only meaningful when `event === "message"`)		

Starting the Server

WebSocket runs alongside your HTTP server:

```
tina4 serve

  Tina4 Node.js v3.10.3
  HTTP server running at http://0.0.0.0:7148
  WebSocket server running at ws://0.0.0.0:7148
  Press Ctrl+C to stop
```

Both HTTP and WebSocket share the same port. The server detects the protocol upgrade request and routes it to the correct handler.

---

4. Connection Events

Every WebSocket connection goes through three lifecycle events:

Open

Fires when a client connects:

```
import { Router } from "tina4-nodejs";

Router.websocket("/ws/notifications", (connection, event, data) => {
    if (event === "open") {
        console.log(`Client connected: ${connection.id}`);
        connection.send(JSON.stringify({
            type: "welcome",
            message: "Connected to notifications",
            connection_id: connection.id
        }));
    }
});
```

Message

Fires when a client sends data:

```
import { Router } from "tina4-nodejs";

Router.websocket("/ws/notifications", (connection, event, data) => {
    if (event === "open") {
        connection.send(JSON.stringify({
            type: "welcome",
            connection_id: connection.id
        }));
    }

    if (event === "message") {
        const message = JSON.parse(data);
        console.log(`Received from ${connection.id}: ${data}`);

        if (message.type === "ping") {
            connection.send(JSON.stringify({
                type: "pong",
                timestamp: new Date().toISOString()
            }));
        }
    }
});
```

Close

Fires when a client disconnects:

```
import { Router } from "tina4-nodejs";

Router.websocket("/ws/notifications", (connection, event, data) => {
    if (event === "open") {
        console.log(`Client connected: ${connection.id}`);
    }

    if (event === "message") {
        console.log(`Message from ${connection.id}: ${data}`);
    }

    if (event === "close") {
        console.log(`Client disconnected: ${connection.id}`);
        // Clean up: remove from tracking, notify others, etc.
    }
});
```

A Complete Handler

Here is a handler that responds to all three events:

```
import { Router } from "tina4-nodejs";

Router.websocket("/ws/chat", (connection, event, data) => {
    switch (event) {
        case "open":
            console.log(`[Chat] New connection: ${connection.id}`);
            connection.send(JSON.stringify({
                type: "system",
```

```
                message: "Welcome to the chat!",
                your_id: connection.id
            }));
            break;

        case "message":
            const message = JSON.parse(data);
            console.log(`[Chat] ${connection.id}: ${message.text ?? data}`);
            connection.send(JSON.stringify({
                type: "message",
                from: connection.id,
                text: message.text ?? data,
                timestamp: new Date().toISOString()
            }));
            break;

        case "close":
            console.log(`[Chat] Disconnected: ${connection.id}`);
            break;
    }
});
```

---

5. Sending to a Single Client

`connection.send()` sends a message to the specific client that triggered the event:

```
import { Router } from "tina4-nodejs";

Router.websocket("/ws/private", (connection, event, data) => {
    if (event === "message") {
        const message = JSON.parse(data);
        const action = message.action ?? "";

        if (action === "get-time") {
            connection.send(JSON.stringify({
                type: "time",
                server_time: new Date().toISOString()
            }));
        }

        if (action === "get-status") {
            const memUsage = process.memoryUsage();
            connection.send(JSON.stringify({
                type: "status",
                uptime: Math.floor(process.uptime()),
                connections: 42,
                memory_mb: Math.round(memUsage.rss / 1024 / 1024 * 100) / 100
            }));
        }
    }
});
```

Only the client that sent the message receives the response. Other connected clients do not see it.

---

6. Broadcasting to All Clients

`connection.broadcast()` sends a message to every client connected to the same WebSocket path:

```
Router.websocket("/ws/announcements", (connection, event, data) => {
    if (event === "open") {
        connection.broadcast(JSON.stringify({
            type: "system",
            message: "A new user joined",
            online_count: connection.connectionCount()
        }));
    }

    if (event === "message") {
        const message = JSON.parse(data);
        connection.broadcast(JSON.stringify({
            type: "announcement",
            from: connection.id,
            text: message.text ?? "",
            timestamp: new Date().toISOString()
        }));
    }

    if (event === "close") {
        connection.broadcast(JSON.stringify({
            type: "system",
            message: "A user left",
            online_count: connection.connectionCount()
        }));
    }
});
```

When one client sends a message, every client connected to `/ws/announcements` receives it. This is the foundation for chat rooms, live dashboards, and collaborative editing.

Broadcast Excluding Sender

Sometimes you want to send to everyone except the client that triggered the event:

```
if (event === "message") {
    const message = JSON.parse(data);

    // Send to sender (confirmation)
    connection.send(JSON.stringify({
        type: "sent",
        text: message.text
    }));

    // Send to everyone else
    connection.broadcast(JSON.stringify({
        type: "message",
        from: message.username ?? "Anonymous",
        text: message.text,
        timestamp: new Date().toISOString()
    }), { excludeSelf: true });
}
```

The `{ excludeSelf: true }` option on `broadcast()` excludes the sender. The sender gets a "sent" confirmation. Everyone else gets the "message".

Sending JSON

Use `connection.sendJson()` to send an object as a JSON string without calling `JSON.stringify()` yourself:

```
Router.websocket("/ws/status", (connection, event, data) => {
    if (event === "open") {
        connection.sendJson({
            type: "welcome",
            connection_id: connection.id
        });
    }

    if (event === "message") {
        connection.sendJson({
            type: "ack",
            received: data
        });
    }
});
```

`sendJson()` serialises the data to JSON for you. It is equivalent to `connection.send(JSON.stringify(data))` but saves you the call.

Closing a Connection

Use `connection.close()` to close the connection from the server side:

```
Router.websocket("/ws/secure", (connection, event, data) => {
    if (event === "open") {
        const token = connection.params.token;
        if (!token || !validToken(token)) {
            connection.sendJson({ error: "Unauthorized" });
            connection.close();
            return;
        }

        connection.sendJson({ type: "welcome" });
    }
});
```

Connection Methods Summary

Method	Description
`connection.send(message)`	Send a string message to this connection only
`connection.sendJson(data)`	Send an object as JSON to this connection only
`connection.broadcast(message)`	Send to all connections on the same path
`connection.broadcast(message, { excludeSelf: true })`	Send to all except this connection
`connection.close()`	Close this connection from the server side
`connection.id`	Unique identifier for this connection
`connection.params`	Path parameters extracted from the URL
`connection.connectionCount()`	Number of active connections on this path

---

7. Path-Scoped Isolation

Different WebSocket paths are isolated. Clients connected to `/ws/chat/room-1` do not see messages from `/ws/chat/room-2`:

```
Router.websocket("/ws/chat/{room}", (connection, event, data) => {
    const room = connection.params.room;

    if (event === "open") {
        console.log(`[Room ${room}] New connection: ${connection.id}`);
        connection.broadcast(JSON.stringify({
            type: "system",
            message: `Someone joined room ${room}`,
            room,
            online: connection.connectionCount()
        }));
    }

    if (event === "message") {
        const message = JSON.parse(data);
        connection.broadcast(JSON.stringify({
            type: "message",
            room,
            from: message.username ?? "Anonymous",
            text: message.text ?? "",
            timestamp: new Date().toISOString()
        }));
    }

    if (event === "close") {
        connection.broadcast(JSON.stringify({
            type: "system",
            message: `Someone left room ${room}`,
```

```
            room,
            online: connection.connectionCount()
        }));
    }
});
```

Connect to different rooms:

```
ws://localhost:7148/ws/chat/general    -- general chat
ws://localhost:7148/ws/chat/random     -- random chat
ws://localhost:7148/ws/chat/dev-team   -- dev team chat
```

Broadcasting in `/ws/chat/general` reaches only clients connected to `/ws/chat/general`. The `dev-team` and `random` rooms are separate.

Chat rooms. Project-specific notifications. Per-user channels. No extra configuration. The URL path is the isolation boundary.

Path parameters use `{name}` curly-brace syntax, not `:name` colon syntax. Both `connection.params.room` and `connection.params["room"]` work.

---

8. Building a Live Chat

Here is a complete chat application with usernames, typing indicators, and online counts:

WebSocket Handler

Create `src/routes/chat-ws.ts`:

```
import { Router } from "tina4-nodejs";

const chatUsers: Record<string, { id: string; username: string; room: string; joinedAt: string }

Router.websocket("/ws/livechat/{room}", (connection, event, data) => {
    const room = connection.params.room;

    if (event === "open") {
        chatUsers[connection.id] = {
            id: connection.id,
            username: "Anonymous",
            room,
            joinedAt: new Date().toISOString()
        };

        connection.send(JSON.stringify({
            type: "welcome",
            message: `Connected to room: ${room}`,
            your_id: connection.id,
            online: connection.connectionCount()
        }));
    }

    if (event === "message") {
        const message = JSON.parse(data);
        const type = message.type ?? "message";

        if (type === "set-username") {
```

```
            const oldName = chatUsers[connection.id].username;
            chatUsers[connection.id].username = message.username;
            connection.broadcast(JSON.stringify({
                type: "system",
                message: `${oldName} is now known as ${message.username}`
            }));
        }

        if (type === "message") {
            const username = chatUsers[connection.id].username;
            connection.broadcast(JSON.stringify({
                type: "message",
                from: username,
                from_id: connection.id,
                text: message.text ?? "",
                timestamp: new Date().toISOString()
            }));
        }

        if (type === "typing") {
            const username = chatUsers[connection.id].username;
            connection.broadcast(JSON.stringify({
                type: "typing",
                username
            }), { excludeSelf: true });
        }
    }

    if (event === "close") {
        const username = chatUsers[connection.id]?.username ?? "Unknown";
        delete chatUsers[connection.id];

        connection.broadcast(JSON.stringify({
            type: "system",
            message: `${username} left the chat`,
            online: connection.connectionCount()
        }));
    }
});
```

---

9. Live Notifications

WebSocket is built for pushing notifications to users in real time:

```
import { Router } from "tina4-nodejs";

Router.websocket("/ws/notifications/{userId}", (connection, event, data) => {
    const userId = connection.params.userId;

    if (event === "open") {
        console.log(`[Notifications] User ${userId} connected`);
        connection.send(JSON.stringify({
            type: "connected",
            message: "Listening for notifications"
        }));
    }
```

```
    if (event === "message") {
        const message = JSON.parse(data);
        if (message.type === "mark-read") {
            console.log(`[Notifications] User ${userId} read notification ${message.id}`);
        }
    }
});

Router.post("/api/orders/{orderId}/ship", async (req, res) => {
    const orderId = req.params.orderId;
    const userId = req.body.user_id ?? 0;

    // Update order status in database...

    // Send real-time notification via WebSocket
    Router.pushToWebSocket(`/ws/notifications/${userId}`, JSON.stringify({
        type: "notification",
        title: "Order Shipped",
        message: `Your order #${orderId} has been shipped!`,
        action_url: `/orders/${orderId}`,
        timestamp: new Date().toISOString()
    }));

    return res.status(201).json({ message: "Order shipped, user notified" });
});
```

`Router.pushToWebSocket()` lets your HTTP handlers send messages to WebSocket clients. This bridges the gap between traditional request-response endpoints and real-time notifications.

Use Cases

- **Dashboard updates** -- New orders, user signups, system alerts
- **Notification feeds** -- Task assignments, comments, mentions
- **Live data** -- Stock tickers, sensor readings, log streams
- **Progress tracking** -- File upload progress, background job status

Scoped Notifications

Path parameters scope the broadcast. Only clients connected to a specific path receive the message:

```
// Only clients connected to /ws/project/42 receive this
Router.pushToWebSocket("/ws/project/42", JSON.stringify({
    type: "task_completed",
    task: "Design review"
}));
```

---

10. Connecting from JavaScript

Tina4 provides `frond.js`, a built-in JavaScript helper library that includes WebSocket support:

Basic Connection

```
<script src="/js/frond.js"></script>
<script>
    const ws = frond.ws("/ws/chat/general");

    ws.on("open", function () {
        console.log("Connected to chat");
    });

    ws.on("message", function (data) {
        const message = JSON.parse(data);
        console.log("Received:", message);

        if (message.type === "message") {
            addMessageToUI(message.from, message.text, message.timestamp);
        }

        if (message.type === "system") {
            addSystemMessage(message.message);
        }
    });

    ws.on("close", function () {
        console.log("Disconnected from chat");
    });

    function sendMessage(text) {
        ws.send(JSON.stringify({
            type: "message",
            text: text
        }));
    }

    function setUsername(name) {
        ws.send(JSON.stringify({
            type: "set-username",
            username: name
        }));
    }
</script>
```

Auto-Reconnect

`frond.js` reconnects when the connection drops:

```
const ws = frond.ws("/ws/notifications/42", {
    reconnect: true,           // Enable auto-reconnect (default: true)
    reconnectInterval: 3000,   // Retry every 3 seconds
    maxReconnectAttempts: 10   // Give up after 10 attempts
});

ws.on("reconnect", function (attempt) {
    console.log("Reconnecting... attempt " + attempt);
});

ws.on("reconnect_failed", function () {
    console.log("Failed to reconnect after maximum attempts");
});
```

Using Native WebSocket

If you prefer not to use `frond.js`, the native WebSocket API works too:

```
const ws = new WebSocket("ws://localhost:7148/ws/chat/general");

ws.onopen = function () {
    console.log("Connected");
    ws.send(JSON.stringify({ type: "set-username", username: "Alice" }));
};

ws.onmessage = function (event) {
    const message = JSON.parse(event.data);
    console.log("Received:", message);
};

ws.onclose = function () {
    console.log("Disconnected");
};

ws.onerror = function (error) {
    console.error("WebSocket error:", error);
};
```

`frond.js` gives you auto-reconnect, message buffering during reconnection, and a cleaner event API. Native WebSocket does not.

Manual Auto-Reconnect with Native WebSocket

If you use the native API and want reconnection, build it yourself:

```
function connectWebSocket(path) {
    const ws = new WebSocket(`ws://${location.host}${path}`);

    ws.addEventListener("close", () => {
        console.log("Disconnected. Reconnecting in 3s...");
        setTimeout(() => connectWebSocket(path), 3000);
    });

    ws.addEventListener("message", (event) => {
        const data = JSON.parse(event.data);
        console.log("Received:", data);
    });

    return ws;
}

const ws = connectWebSocket("/ws/chat/general");
```

This reconnects on every close. It does not limit retries or buffer messages sent during the reconnection window. For production use, `frond.js` handles these edge cases for you.

---

11. A Complete Chat Page

Here is a full chat page using templates and WebSocket:

Create `src/templates/chat.html`:

```
{% extends "base.html" %}

{% block title %}Chat - {{ room }}{% endblock %}

{% block content %}
    <h1>Chat Room: {{ room }}</h1>
    <p id="status">Connecting...</p>
    <p id="online">Online: 0</p>

    <div id="messages" style="border: 1px solid #ddd; height: 400px; overflow-y: scroll; padding
    </div>

    <div id="typing" style="height: 20px; color: #888; font-style: italic; margin-bottom: 4px;">

    <form id="chat-form" style="display: flex; gap: 8px;">
        <input type="text" id="message-input" placeholder="Type a message..."
               style="flex: 1; padding: 8px; border: 1px solid #ddd; border-radius: 4px;">
        <button type="submit" style="padding: 8px 16px; background: #333; color: white; border:
            Send
        </button>
    </form>

    <script src="/js/frond.js"></script>
    <script>
        const room = "{{ room }}";
        const ws = frond.ws("/ws/livechat/" + room);
        const messagesDiv = document.getElementById("messages");
        const statusEl = document.getElementById("status");
        const onlineEl = document.getElementById("online");
        const typingEl = document.getElementById("typing");

        let username = prompt("Enter your username:") || "Anonymous";

        ws.on("open", function () {
            statusEl.textContent = "Connected";
            ws.send(JSON.stringify({ type: "set-username", username: username }));
        });

        ws.on("message", function (data) {
            const msg = JSON.parse(data);

            if (msg.type === "message") {
                const div = document.createElement("div");
                div.style.marginBottom = "8px";
                div.innerHTML = "<strong>" + msg.from + ":</strong> " + msg.text;
                messagesDiv.appendChild(div);
                messagesDiv.scrollTop = messagesDiv.scrollHeight;
            }

            if (msg.type === "system") {
                const div = document.createElement("div");
                div.style.cssText = "color: #888; font-style: italic; margin-bottom: 8px;";
                div.textContent = msg.message;
                messagesDiv.appendChild(div);
                messagesDiv.scrollTop = messagesDiv.scrollHeight;
            }
```

```
            if (msg.type === "typing") {
                typingEl.textContent = msg.username + " is typing...";
                setTimeout(function () { typingEl.textContent = ""; }, 2000);
            }

            if (msg.online !== undefined) {
                onlineEl.textContent = "Online: " + msg.online;
            }
        });

        ws.on("close", function () {
            statusEl.textContent = "Disconnected. Reconnecting...";
        });

        document.getElementById("chat-form").addEventListener("submit", function (e) {
            e.preventDefault();
            const input = document.getElementById("message-input");
            if (input.value.trim()) {
                ws.send(JSON.stringify({ type: "message", text: input.value }));
                input.value = "";
            }
        });

        document.getElementById("message-input").addEventListener("input", function () {
            ws.send(JSON.stringify({ type: "typing" }));
        });
    </script>
{% endblock %}
```

Create the route to serve the page:

```
Router.get("/chat/{room}", async (req, res) => {
    const room = req.params.room;
    return res.render("chat.html", { room });
});
```

Visit `http://localhost:7148/chat/general` in two browser tabs. Type in one tab and watch the message appear in the other.

---

12. Exercise: Build a Real-Time Chat Room

Build a WebSocket chat room with the following features:

Requirements

- WebSocket endpoint at `/ws/room/{roomName}` that handles:
- `open`: Send welcome message with connection count
- `message` with type `set-name`: Set the user's display name
- `message` with type `chat`: Broadcast the message to all clients with the sender's name
- `close`: Broadcast that the user left
- HTTP endpoint at `GET /room/{roomName}` that serves an HTML chat page
- The chat page should:

- Prompt for a username on load
- Display messages in real time
- Show system messages (join/leave) in a different style
- Show the count of online users

Test by:

- Open `http://localhost:7148/room/test` in two browser tabs
- Set different usernames in each
- Send messages from both tabs and verify they appear in both
- Close one tab and verify the "user left" message appears in the other

---

13. Solution

Create `src/routes/chat-room.ts`:

```
import { Router } from "tina4-nodejs";

const roomUsers: Record<string, string> = {};

Router.websocket("/ws/room/{roomName}", (connection, event, data) => {
    const room = connection.params.roomName;
    const key = `${room}:${connection.id}`;

    if (event === "open") {
        roomUsers[key] = "Anonymous";

        connection.send(JSON.stringify({
            type: "system",
            message: `Welcome to room: ${room}. Set your name with: `
                + '{"type": "set-name", "name": "YourName"}',
            online: connection.connectionCount()
        }));

        connection.broadcast(JSON.stringify({
            type: "system",
            message: "A new user joined",
            online: connection.connectionCount()
        }), { excludeSelf: true });
    }

    if (event === "message") {
        const msg = JSON.parse(data);
        const msgType = msg.type ?? "chat";

        if (msgType === "set-name") {
            const oldName = roomUsers[key] ?? "Anonymous";
            const newName = msg.name ?? "Anonymous";
            roomUsers[key] = newName;

            connection.broadcast(JSON.stringify({
                type: "system",
```

```
                message: `${oldName} changed their name to ${newName}`
            }));
        }

        if (msgType === "chat") {
            const username = roomUsers[key] ?? "Anonymous";

            connection.broadcast(JSON.stringify({
                type: "chat",
                from: username,
                text: msg.text ?? "",
                timestamp: new Date().toTimeString().substring(0, 8)
            }));
        }
    }

    if (event === "close") {
        const username = roomUsers[key] ?? "Anonymous";
        delete roomUsers[key];

        connection.broadcast(JSON.stringify({
            type: "system",
            message: `${username} left the room`,
            online: connection.connectionCount()
        }));
    }
});

Router.get("/room/{roomName}", async (req, res) => {
    const room = req.params.roomName;
    return res.render("room.html", { room });
});
```

Create `src/templates/room.html`:

```
{% extends "base.html" %}

{% block title %}Room: {{ room }}{% endblock %}

{% block content %}
    <h1>Room: {{ room }}</h1>
    <p id="online" style="color: #666;">Online: 0</p>

    <div id="messages" style="border: 1px solid #ddd; height: 400px; overflow-y: auto; padding:
    </div>

    <form id="form" style="display: flex; gap: 8px;">
        <input type="text" id="input" placeholder="Type a message..."
               style="flex: 1; padding: 10px; border: 1px solid #ddd; border-radius: 4px; font-s
        <button type="submit"
                style="padding: 10px 20px; background: #1a1a2e; color: white; border: none; bord
            Send
        </button>
    </form>

    <script src="/js/frond.js"></script>
    <script>
        const room = "{{ room }}";
        const username = prompt("Choose a username:") || "Anonymous";
```

```
        const ws = frond.ws("/ws/room/" + room);
        const msgs = document.getElementById("messages");

        ws.on("open", function () {
            ws.send(JSON.stringify({ type: "set-name", name: username }));
        });

        ws.on("message", function (raw) {
            const msg = JSON.parse(raw);
            const div = document.createElement("div");
            div.style.marginBottom = "8px";

            if (msg.type === "chat") {
                div.innerHTML = "<strong>" + msg.from + "</strong> <span style='color:#999;font-
            } else if (msg.type === "system") {
                div.style.color = "#888";
                div.style.fontStyle = "italic";
                div.textContent = msg.message;
            }

            msgs.appendChild(div);
            msgs.scrollTop = msgs.scrollHeight;

            if (msg.online !== undefined) {
                document.getElementById("online").textContent = "Online: " + msg.online;
            }
        });

        document.getElementById("form").addEventListener("submit", function (e) {
            e.preventDefault();
            const input = document.getElementById("input");
            if (input.value.trim()) {
                ws.send(JSON.stringify({ type: "chat", text: input.value }));
                input.value = "";
            }
        });
    </script>
{% endblock %}
```

Open `http://localhost:7148/room/test` in two browser tabs. Set different usernames. Send messages from one tab and watch them appear in both. Close one tab and verify the "left the room" message appears.

---

14. Scaling with a Backplane

When you run a single server instance, `broadcast()` reaches every connected client. But in production you often run multiple instances behind a load balancer. Each instance only knows about its own connections. A message broadcast on instance A never reaches clients connected to instance B.

A backplane solves this. It relays WebSocket messages across all instances using the shared `tina4:ws` pub/sub channel. Tina4 supports Redis and NATS adapters. The file-free local path remains the default; a network backplane needs its broker and the matching client package.

Configuration

Set two environment variables in your `.env`:

```
TINA4_WS_BACKPLANE=redis
TINA4_WS_BACKPLANE_URL=redis://localhost:6379
```

For NATS:

```
TINA4_WS_BACKPLANE=nats
TINA4_WS_BACKPLANE_URL=nats://localhost:4222
```

Every `broadcast()` call delivers to local connections first, then publishes an envelope to the backplane. Sibling instances discard the sender's echo and relay the message to their own local connections. Your WebSocket routes do not change.

Requirements

Install only the client for the adapter your application selects:

```
npm install redis
npm install @nats-io/transport-node
```

`@nats-io/transport-node` is the maintained Node/Bun transport from the official NATS.js v3 package split; the legacy `nats` npm package has moved.

These are application dependencies, not Tina4 core dependencies. Tina4 imports them only when it starts the selected adapter. An explicit selection is a deployment contract: an unknown adapter, missing package, or unreachable broker fails startup with `Configured WebSocket backplane failed`. Tina4 never claims distributed delivery while running local-only.

When to Choose a Backplane

Deployment	Choice
One Tina4 process or container	Leave `TINA4_WS_BACKPLANE` unset
Multiple instances and Redis is already operated	`redis`
Multiple instances and NATS is already operated	`nats`
Unsure which broker to introduce	Redis is usually the simpler starting point

The signal is the process count, not traffic volume: once clients can land on different Tina4 instances, broadcasts need a shared backplane. Do not install NATS solely because Tina4 supports it; choose it when the deployment already uses NATS or needs NATS as its wider event bus.

Verify the setup with two Tina4 processes. Connect one WebSocket client to each process, broadcast from the first, and confirm the second client receives the frame. A single-process test proves only local delivery.

If `TINA4_WS_BACKPLANE` is not set, Tina4 broadcasts only to local connections. This is the correct default for a single instance.

---

15. Gotchas

1. WebSocket Needs a Persistent Server

Problem: WebSocket connections drop right away or do not work at all.

Cause: You are running behind a serverless platform or a proxy that closes idle connections. Traditional request-response servers do not support persistent connections. Each request is handled and the connection is closed.

Fix: Use `tina4 serve` which runs Tina4's built-in HTTP server that handles both HTTP and WebSocket. For production, you can use Nginx as a reverse proxy, but it must be configured for WebSocket proxying with `proxy_set_header Upgrade $http_upgrade` and `proxy_set_header Connection "upgrade"`.

2. Messages Are Strings, Not Objects

Problem: `data` in the message handler is a string, not a JavaScript object.

Cause: WebSocket transmits raw strings. If the client sends JSON, you receive the JSON string, not a parsed object.

Fix: Always `JSON.parse(data)` when you expect JSON messages. Always `JSON.stringify()` when you send structured data. WebSocket does not know about content types -- it is just bytes on a wire.

3. Connection Count Is Per-Path

Problem: `connection.connectionCount()` returns a lower number than expected.

Cause: Connection count is scoped to the WebSocket path. Clients on `/ws/chat/room-1` are counted separately from `/ws/chat/room-2`.

Fix: This is by design. Each path is an isolated group. If you need a global connection count across all paths, maintain a counter in a shared variable or use a module-level map.

4. Broadcasting Does Not Scale Across Servers

Problem: Users connected to different server instances do not see each other's messages.

Cause: `TINA4_WS_BACKPLANE` is unset, so `connection.broadcast()` correctly uses local-only mode. Multiple instances behind a load balancer each maintain their own connection sets.

Fix: Use a pub/sub backend like Redis to relay messages across server instances. Each server subscribes to a Redis channel, and broadcast messages are published to Redis so all servers receive them. See section 14 for configuration.

5. Large Messages Cause Disconnects

Problem: The connection drops when sending a large message.

Cause: WebSocket servers have a maximum message size. The default is usually around 1MB. If your message exceeds this, the server closes the connection.

Fix: Keep messages small. Under 64KB is a good target. For large data transfers, use HTTP endpoints instead of WebSocket. WebSocket is designed for small, frequent messages -- not bulk data transfer.

6. Memory Leak from Tracking Connected Users

Problem: The server's memory usage grows over time and the process crashes.

Cause: You store user data in an object (like `chatUsers`) but do not clean it up when users disconnect. If the `close` event handler does not remove the user from the object, the object grows without limit.

Fix: Always clean up in the `close` handler. Remove disconnected users from tracking objects: `delete chatUsers[connection.id]`. Test by connecting and disconnecting many times to verify memory stays stable.

7. No Authentication on WebSocket Connect

Problem: Anyone can connect to your WebSocket endpoint and see all messages.

Cause: The WebSocket upgrade request does not carry your JWT token in the `Authorization` header. Browsers do not support custom headers on WebSocket connections.

Fix: Pass the token as a query parameter: `ws://localhost:7148/ws/chat?token=eyJ...`. In your `open` handler, validate the token and disconnect if invalid. Use a short-lived token for WebSocket connections. You can also authenticate via an HTTP endpoint first, store the session, and check the session cookie during the WebSocket upgrade.

8. Route Params Use `{id}` Not `:id`

Problem: Your WebSocket path parameters do not resolve. `connection.params` is empty.

Cause: You used Express-style `:param` colon syntax instead of Tina4's `{param}` curly-brace syntax.

Fix: WebSocket path parameters follow the same `{param}` syntax as HTTP routes. Write `/ws/chat/{room}`, not `/ws/chat/:room`. Both `connection.params.room` and `connection.params["room"]` work.

Server-Sent Events (SSE)

1. The Polling Problem

Your monitoring dashboard needs live metrics. CPU usage. Memory. Active connections. The numbers change every few seconds.

The obvious solution: poll. A timer fires every three seconds. The browser sends a GET request. The server queries the database. The server responds. The browser updates the page. Repeat.

That works. It also wastes resources. Nineteen out of twenty polls return the same data. Each one opens a connection, sends headers, waits for a response, and closes the connection. The server does the same work whether the data changed or not.

WebSocket solves this. A persistent, bi-directional connection. The server pushes when data changes. But WebSocket is designed for two-way communication. Chat rooms. Collaborative editing. Live gaming. Your dashboard only needs one direction: server to client.

Server-Sent Events (SSE) is the middle ground. One-way streaming over plain HTTP. The server pushes. The client listens. The browser reconnects automatically if the connection drops. No upgrade handshake. No special protocol. Just HTTP with a twist.

---

2. What SSE Is

HTTP works like this:

```
Client: "Give me /api/metrics"
Server: "Here are the metrics" (connection closes)
Client: "Give me /api/metrics" (new connection, 3 seconds later)
Server: "Here are the metrics" (connection closes)
```

WebSocket works like this:

```
Client: "Upgrade to WebSocket on /ws/metrics"
Server: "Upgrade accepted. Connection open."
Client: "Subscribe to CPU metrics"
Server: "CPU: 42%"
Server: "CPU: 38%"  (pushed at any time)
Client: "Unsubscribe"
```

SSE works like this:

```
Client: "Give me /events (keep the connection open)"
Server: "data: CPU 42%"
Server: "data: CPU 38%"  (pushed, no client request)
Server: "data: CPU 45%"  (keeps flowing)
Client: (just listens)
```

The key differences:

Feature	HTTP Polling	WebSocket	SSE
Direction	Client to server	Both ways	Server to client

Connection	New each time	Persistent	Persistent
Protocol	HTTP	WebSocket (upgrade)	HTTP
Auto-reconnect	No	No (manual)	Yes (built-in)
Binary data	Yes	Yes	No (text only)
Browser support	Universal	Universal	Universal (except IE)
Complexity	Simple	Moderate	Simple

SSE uses plain HTTP. No protocol upgrade. No special server support. The server sets `Content-Type: text/event-stream` and keeps the connection open. The browser's `EventSource` API handles reconnection, event parsing, and last-event-ID tracking.

---

3. Your First Stream

Create `src/routes/events.py`:

```
import asyncio
from tina4_python import get

@get("/events")
async def stream_events(request, response):
    async def generate():
        for i in range(10):
            yield f"data: {{\"count\": {i}, \"message\": \"tick\"}}\n\n"
            await asyncio.sleep(1)
        yield "data: {\"done\": true}\n\n"

    return response.stream(generate())
```

Three things happen here:

- `generate()` is an async generator. Each `yield` produces one SSE message.
- `response.stream()` tells Tina4 to keep the connection open and send each chunk as it arrives.
- The framework sets `Content-Type: text/event-stream`, `Cache-Control: no-cache`, and `Connection: keep-alive` for you.

Start the server and test with curl:

```
curl -N http://localhost:7148/events
```

You see one message per second:

```
data: {"count": 0, "message": "tick"}

data: {"count": 1, "message": "tick"}

data: {"count": 2, "message": "tick"}
```

Each message arrives the moment the server yields it. No buffering. No waiting for the full response.

---

4. The SSE Protocol

SSE messages are plain text with a simple format. Each message is one or more field lines followed by a blank line.

The `data:` Field

The most common field. Contains the message payload.

```
data: Hello, world
```

Multiple `data:` lines in one message get joined with newlines:

```
data: line one
data: line two
```

The client receives this as `"line one\nline two"`.

The `event:` Field

Names the event type. Without it, the browser fires `onmessage`. With it, the browser fires a named event listener.

```
yield "event: metrics\ndata: {\"cpu\": 42}\n\n"
yield "event: alert\ndata: {\"level\": \"high\", \"message\": \"CPU spike\"}\n\n"
```

On the client:

```
source.addEventListener("metrics", (e) => {
    const data = JSON.parse(e.data);
    updateDashboard(data);
});

source.addEventListener("alert", (e) => {
    const data = JSON.parse(e.data);
    showAlert(data.message);
});
```

Named events let you multiplex different data types on a single connection.

The `id:` Field

Sets the last event ID. If the connection drops, the browser sends `Last-Event-ID` in the reconnection request. Your server can resume from where it left off.

```
yield f"id: {i}\ndata: {{\"count\": {i}}}\n\n"
```

The `retry:` Field

Tells the browser how many milliseconds to wait before reconnecting after a disconnect.

```
yield "retry: 5000\n\n"  # reconnect after 5 seconds
```

The Delimiter

Every SSE message ends with two newlines: `\n\n`. This tells the browser that the message is complete. A single `\n` separates fields within one message.

```
# One complete SSE message:
yield "event: update\nid: 42\ndata: {\"value\": 100}\n\n"

# Broken down:
# event: update\n     ← event type
# id: 42\n            ← event ID
# data: {"value": 100}\n  ← payload
# \n                   ← end of message (blank line)
```

---

5. Frontend: EventSource

The browser provides `EventSource` for consuming SSE streams. It handles connection management, reconnection, and message parsing.

Basic Usage

```
const source = new EventSource("/events");

source.onmessage = function (event) {
    const data = JSON.parse(event.data);
    console.log("Received:", data);
};

source.onerror = function () {
    console.log("Connection lost. Reconnecting...");
};

source.onopen = function () {
    console.log("Connected");
};
```

`EventSource` reconnects automatically when the connection drops. The `onerror` handler fires, the browser waits (default 3 seconds), then reconnects. Your server receives a new request with the `Last-Event-ID` header if you sent event IDs.

Named Events

```
const source = new EventSource("/events");

source.addEventListener("metrics", function (event) {
    updateChart(JSON.parse(event.data));
});

source.addEventListener("notification", function (event) {
    showToast(JSON.parse(event.data));
});

source.addEventListener("heartbeat", function (event) {
    // Connection is alive
});
```

Closing the Connection

```
source.close();
```

The browser stops reconnecting. The server receives a client disconnect.

With Authentication

`EventSource` does not support custom headers. Pass tokens as query parameters:

```
const token = getAuthToken();
const source = new EventSource(`/events?token=${token}`);
```

On the server:

```
@get("/events")
async def secure_events(request, response):
    token = request.query.get("token")
    payload = Auth.valid_token(token)
    if not payload:
        return response({"error": "Unauthorized"}, 401)

    async def generate():
        while True:
            yield f"data: {{\"user\": \"{payload['email']}\"}}\n\n"
            await asyncio.sleep(5)

    return response.stream(generate())
```

---

6. Real Examples

Live Dashboard

Stream database metrics every five seconds:

```
import asyncio
from tina4_python import get
from tina4_python.database import Database

@get("/events/dashboard")
async def dashboard_stream(request, response):
    async def generate():
        db = Database()
        while True:
            orders = db.fetch_one("SELECT count(*) as total FROM orders")
            revenue = db.fetch_one("SELECT sum(total) as revenue FROM orders WHERE date(created_

            yield f"data: {{\"orders\": {orders['total']}, \"revenue\": {revenue['revenue'] or 0
            await asyncio.sleep(5)

    return response.stream(generate())
```

The client updates the dashboard numbers without polling. One connection. One query every five seconds. No wasted requests.

Build Progress

Stream deployment status as it happens:

```
import asyncio
import json
from tina4_python import get

@get("/events/deploy/{build_id}")
async def deploy_stream(request, response):
    build_id = request.params["build_id"]

    async def generate():
        steps = [
            ("pull", "Pulling latest code..."),
            ("install", "Installing dependencies..."),
            ("test", "Running tests..."),
            ("build", "Building application..."),
            ("deploy", "Deploying to production..."),
            ("done", "Deployment complete"),
        ]

        for step, message in steps:
            yield f"event: progress\ndata: {json.dumps({'step': step, 'message': message, 'build
            await asyncio.sleep(2)  # simulate work

    return response.stream(generate())
```

The browser shows a progress bar that updates in real time. Each step arrives as it completes. The user watches the deployment unfold instead of staring at a spinner.

LLM Chat Streaming

Stream AI responses token by token. This is how ChatGPT works:

```
import asyncio
import json
import urllib.request
from tina4_python import get

@get("/events/chat")
async def chat_stream(request, response):
    prompt = request.query.get("q", "Hello")

    async def generate():
        # Call the LLM API with streaming enabled
        req_data = json.dumps({
            "model": "claude-sonnet-4-20250514",
            "max_tokens": 512,
            "stream": True,
            "messages": [{"role": "user", "content": prompt}],
        }).encode()

        req = urllib.request.Request(
            "https://api.anthropic.com/v1/messages",
            data=req_data,
            headers={
                "Content-Type": "application/json",
                "x-api-key": os.environ["ANTHROPIC_API_KEY"],
```

```
                "anthropic-version": "2023-06-01",
            },
        )

        with urllib.request.urlopen(req) as resp:
            for line in resp:
                line = line.decode().strip()
                if line.startswith("data: "):
                    chunk = json.loads(line[6:])
                    if chunk.get("type") == "content_block_delta":
                        text = chunk["delta"].get("text", "")
                        yield f"data: {json.dumps({'token': text})}\n\n"
                        await asyncio.sleep(0)

        yield "data: {\"done\": true}\n\n"

    return response.stream(generate())
```

The frontend appends each token as it arrives:

```
const source = new EventSource(`/events/chat?q=${encodeURIComponent(question)}`);
const output = document.getElementById("response");

source.onmessage = function (event) {
    const data = JSON.parse(event.data);
    if (data.done) {
        source.close();
        return;
    }
    output.textContent += data.token;
};
```

The response builds character by character. The user reads as the AI writes. No waiting for the full answer.

File Processing Progress

Upload a CSV, stream the processing status:

```
import asyncio
import json
from tina4_python import post

@post("/api/import")
async def import_csv(request, response):
    file = request.files.get("csv")
    if not file:
        return response({"error": "No file"}, 400)

    lines = file["content"].decode().strip().split("\n")
    total = len(lines) - 1  # minus header

    async def generate():
        for i, line in enumerate(lines[1:], 1):
            # Process each row
            cols = line.split(",")
            # ... insert into database ...
            yield f"data: {json.dumps({'processed': i, 'total': total, 'percent': round(i/total*
            await asyncio.sleep(0)  # yield control
```

```
        yield f"data: {json.dumps({'done': True, 'total': total})}\n\n"

    return response.stream(generate(), content_type="text/event-stream")
```

The browser shows a progress bar that fills as each row is processed. The user sees exactly where the import stands.

---

7. Custom Content Types

SSE defaults to `text/event-stream`, but `response.stream()` accepts any content type. Use this for newline-delimited JSON (NDJSON), chunked binary, or custom protocols.

NDJSON Streaming

```
import asyncio
import json
from tina4_python import get

@get("/api/export")
async def export_stream(request, response):
    async def generate():
        db = Database()
        result = db.fetch("SELECT * FROM products", limit=1000)
        for row in result.records:
            yield json.dumps(row) + "\n"
            await asyncio.sleep(0)

    return response.stream(generate(), content_type="application/x-ndjson")
```

Each line is a complete JSON object. The client reads line by line. No need to parse a giant array. Memory stays flat regardless of how many rows you export.

Consuming NDJSON on the Client

```
async function streamProducts() {
    const response = await fetch("/api/export");
    const reader = response.body.getReader();
    const decoder = new TextDecoder();
    let buffer = "";

    while (true) {
        const { done, value } = await reader.read();
        if (done) break;

        buffer += decoder.decode(value, { stream: true });
        const lines = buffer.split("\n");
        buffer = lines.pop(); // keep incomplete line

        for (const line of lines) {
            if (line.trim()) {
                const product = JSON.parse(line);
                addToTable(product);
            }
        }
    }
}
```

---

8. SSE vs WebSocket

Both keep a persistent connection. Both push data in real time. The choice depends on the direction of communication.

Use Case	Choose	Why
Live dashboard	SSE	Server pushes metrics. Client only listens.
Chat application	WebSocket	Both sides send messages.
Notification feed	SSE	Server pushes. Client marks as read via separate HTTP POST.
Collaborative editor	WebSocket	Both sides send cursor positions and edits.
Build/deploy progress	SSE	Server streams status. Client watches.
LLM token streaming	SSE	Server streams tokens. Client displays them.
Online gaming	WebSocket	Low-latency, bi-directional input and state.
Stock ticker	SSE	Server pushes prices. Client displays.
File upload progress	HTTP (native)	Browser tracks upload. Server tracks processing via SSE.

Rule of thumb: If the client only listens, use SSE. If the client sends data back on the same connection, use WebSocket.

SSE has one advantage WebSocket does not: automatic reconnection. The browser's `EventSource` reconnects without any code. WebSocket requires manual reconnection logic (or `frond.js` which handles it for you).

---

9. Production Considerations

Nginx Proxy Buffering

Nginx buffers responses by default. SSE messages accumulate in the buffer and arrive in batches instead of one at a time. Tina4 sets `X-Accel-Buffering: no` on streaming responses, which tells Nginx to disable buffering for that response.

If you configure Nginx manually:

```
location /events/ {
    proxy_pass http://localhost:7148;
    proxy_buffering off;
    proxy_cache off;
    proxy_set_header Connection '';
    proxy_http_version 1.1;
    chunked_transfer_encoding off;
}
```

Connection Limits

Each SSE connection is a persistent HTTP connection. Most servers limit concurrent connections. The default asyncio server handles thousands. In production:

- Monitor open connections
- Set timeouts on long-running streams
- Close streams when the client no longer needs them

```
@get("/events/metrics")
async def metrics_with_timeout(request, response):
    async def generate():
        for _ in range(3600):  # max 1 hour (one message per second)
            yield f"data: {{\"cpu\": {get_cpu()}}}\n\n"
            await asyncio.sleep(1)
        yield "event: timeout\ndata: {\"message\": \"Stream expired. Reconnect.\"}\n\n"

    return response.stream(generate())
```

Heartbeat Messages

Some proxies and load balancers close idle connections after a timeout (typically 30-60 seconds). Send a heartbeat comment to keep the connection alive:

```
async def generate():
    counter = 0
    while True:
        if has_new_data():
            yield f"data: {json.dumps(get_data())}\n\n"
        else:
            yield ": heartbeat\n\n"  # SSE comment (colon prefix) - ignored by EventSource
        counter += 1
        await asyncio.sleep(5)
```

Lines starting with `:` are SSE comments. The browser ignores them, but they keep the connection alive through proxies.

---

10. Exercise: Build a Live Metrics Dashboard

Build a dashboard that streams server metrics to the browser.

Requirements

- SSE endpoint at `GET /events/server-metrics` that streams every 3 seconds:
- Current timestamp
- A random CPU percentage (simulate with `random.randint(10, 90)`)
- A random memory percentage
- A random request count
- HTML page at `GET /dashboard` that:
- Connects to the SSE endpoint
- Displays the four metrics in cards
- Updates the values in real time (no page refresh)
- Shows a "Connected" / "Reconnecting..." status indicator

Test by:

- Open `http://localhost:7148/dashboard`
- Watch the numbers update every 3 seconds
- Stop the server and verify "Reconnecting..." appears
- Restart the server and verify it reconnects and resumes updating

---

11. Solution

Create `src/routes/server_metrics.py`:

```
import asyncio
import json
import random
from datetime import datetime, timezone
from tina4_python import get

@get("/events/server-metrics")
async def server_metrics_stream(request, response):
    async def generate():
        yield "retry: 3000\n\n"
        counter = 0
        while True:
            yield f"id: {counter}\ndata: {json.dumps({
                'timestamp': datetime.now(timezone.utc).isoformat(),
                'cpu': random.randint(10, 90),
                'memory': random.randint(30, 85),
                'requests': random.randint(100, 5000),
            })}\n\n"
            counter += 1
```

```
            await asyncio.sleep(3)

    return response.stream(generate())


@get("/dashboard")
async def dashboard_page(request, response):
    html = """
    <!DOCTYPE html>
    <html>
    <head><title>Live Dashboard</title></head>
    <body style="font-family: system-ui; max-width: 600px; margin: 40px auto; padding: 0 20px;">
        <h1>Server Metrics</h1>
        <p id="status" style="color: #888;">Connecting...</p>
        <div style="display: grid; grid-template-columns: 1fr 1fr; gap: 16px; margin-top: 20px;"
            <div style="background: #f5f5f5; padding: 20px; border-radius: 8px;">
                <div style="color: #888; font-size: 14px;">CPU</div>
                <div id="cpu" style="font-size: 32px; font-weight: 700;">--</div>
            </div>
            <div style="background: #f5f5f5; padding: 20px; border-radius: 8px;">
                <div style="color: #888; font-size: 14px;">Memory</div>
                <div id="memory" style="font-size: 32px; font-weight: 700;">--</div>
            </div>
            <div style="background: #f5f5f5; padding: 20px; border-radius: 8px;">
                <div style="color: #888; font-size: 14px;">Requests/min</div>
                <div id="requests" style="font-size: 32px; font-weight: 700;">--</div>
            </div>
            <div style="background: #f5f5f5; padding: 20px; border-radius: 8px;">
                <div style="color: #888; font-size: 14px;">Last Update</div>
                <div id="time" style="font-size: 16px; font-weight: 500;">--</div>
            </div>
        </div>
        <script>
            const source = new EventSource("/events/server-metrics");
            const status = document.getElementById("status");

            source.onopen = function () {
                status.textContent = "Connected";
                status.style.color = "#22c55e";
            };

            source.onmessage = function (event) {
                const data = JSON.parse(event.data);
                document.getElementById("cpu").textContent = data.cpu + "%";
                document.getElementById("memory").textContent = data.memory + "%";
                document.getElementById("requests").textContent = data.requests.toLocaleString()
                document.getElementById("time").textContent = new Date(data.timestamp).toLocaleT
            };

            source.onerror = function () {
                status.textContent = "Reconnecting...";
                status.style.color = "#ef4444";
            };
        </script>
    </body>
    </html>
    """
    return response(html)
```

Open `http://localhost:7148/dashboard`. The numbers update every three seconds. Stop the server. The status turns red: "Reconnecting..." Start it again. The status turns green. The numbers resume.

No polling. No WebSocket. No JavaScript timers. The browser handles everything.

---

12. What You Learned

- **SSE streams data from server to client** over a single HTTP connection. No upgrade. No special protocol.
- `response.stream(generator)` keeps the connection open and flushes each chunk as the generator yields it.
- **The SSE protocol** uses `data:`, `event:`, `id:`, and `retry:` fields. Messages end with `\n\n`.
- `EventSource` on the client handles connection, parsing, and automatic reconnection.
- **Named events** let you multiplex different data types on one stream.
- **Custom content types** let you stream NDJSON, binary, or any format.
- **SSE is for one-way server pushes.** Use WebSocket when the client needs to send data back on the same connection.
- **Heartbeat comments** keep connections alive through proxies. Tina4 sets `X-Accel-Buffering: no` to disable nginx buffering.

One direction. One connection. Real-time data. The server speaks. The client listens. The rest is infrastructure.

WSDL / SOAP

1. Legacy Services Are Still Services

SOAP is not new. It is also not gone. Banks, government agencies, ERP systems, and healthcare providers expose SOAP APIs. If you integrate with any of them, you need to speak SOAP.

WSDL (Web Services Description Language) is the schema for SOAP services. It defines operations, input types, and output types in XML. You call an operation, you get XML back, you parse it.

Tina4 provides a `WSDLService` base class that publishes your own SOAP services. You write a class with normal TypeScript methods. Tina4 generates the WSDL document and handles the XML envelope plumbing. It does not ship a SOAP client: to call an external SOAP service, you post the envelope yourself with `fetch`.

---

2. Consuming an External SOAP Service

Calling an Operation

```
const SERVICE_URL = "https://www.dataaccess.com/webservicesserver/NumberConversion.wso";

async function numberToWords(ubiNum: number): Promise<string> {
    const envelope = `<?xml version="1.0" encoding="utf-8"?>
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Body>
    <NumberToWords xmlns="http://www.dataaccess.com/webservicesserver/">
      <ubiNum>${ubiNum}</ubiNum>
    </NumberToWords>
  </soap:Body>
</soap:Envelope>`;

    const response = await fetch(SERVICE_URL, {
        method: "POST",
        headers: { "Content-Type": "text/xml; charset=utf-8" },
        body: envelope
    });
    const xml = await response.text();

    const fault = xml.match(/<faultstring>([^<]*)<\/faultstring>/);
    if (fault) {
        throw new Error(`SOAP fault: ${fault[1]}`);
    }
    return xml.match(/<m:NumberToWordsResult>([^<]*)</)?.[1] ?? "";
}

console.log(await numberToWords(42));
// "forty two "
```

You build the SOAP envelope, `POST` it with `Content-Type: text/xml`, and read the value you need out of the XML response. For richer client features (WSDL parsing, typed calls), use a dedicated npm package.

Handling Errors

```
try {
    console.log(await numberToWords(42));
} catch (err) {
    console.error((err as Error).message);
}
```

A network failure rejects the `fetch`. A SOAP fault comes back as XML with a `<faultstring>`, which `numberToWords` turns into an `Error`.

---

3. The @WSDLOperation Decorator

A SOAP service is a class that extends `WSDLService`. Mark each method you want to expose with the `@WSDLOperation` decorator and describe its input and output types:

```
import { WSDLService, WSDLOperation } from "tina4-nodejs";

export class PriceService extends WSDLService {
    serviceName = "PriceService";
    serviceUrl = "/soap/price";

    @WSDLOperation({
        description: "Returns the current price for a product by ID",
        input: { productId: "string" },
        output: { price: "double", currency: "string", available: "boolean" }
    })
    async GetProductPrice(productId: string) {
        const price = await lookupPrice(productId);
        return {
            price: price.amount,
            currency: price.currency,
            available: price.inStock
        };
    }
}
```

The operation name is the method name. `input` lists the parameters in the order the method takes them: each value is read from the child element of the same name and converted to its type. Each key of the object the method returns becomes an element of the response. The types are `string`, `int` / `integer`, `float`, `double` / `number` / `numeric` and `bool` / `boolean`; any other name is published as `xsd:string`.

---

4. Publishing a SOAP Service

Define the service in `src/app/CurrencyService.ts`:

```
// src/app/CurrencyService.ts
import { WSDLService, WSDLOperation } from "tina4-nodejs";

export class CurrencyService extends WSDLService {
    serviceName = "CurrencyService";
    serviceUrl = "/soap/currency";
```

```
    @WSDLOperation({
        input: { amount: "double", from: "string", to: "string" },
        output: { result: "double", rate: "double" }
    })
    async ConvertCurrency(amount: number, from: string, to: string) {
        // Simulate a conversion
        const rate = from === "USD" && to === "EUR" ? 0.92 : 1.0;
        return {
            result: parseFloat((amount * rate).toFixed(2)),
            rate
        };
    }

    @WSDLOperation({
        input: { from: "string", to: "string" },
        output: { rate: "double", timestamp: "string" }
    })
    async GetExchangeRate(from: string, to: string) {
        return {
            rate: from === "USD" && to === "EUR" ? 0.92 : 1.0,
            timestamp: new Date().toISOString()
        };
    }
}

export const currencyService = new CurrencyService();
```

Then mount it with two route files. `GET` serves the WSDL document, `POST` handles SOAP requests:

```
// src/routes/soap/currency/get.ts
import { currencyService } from "../../../app/CurrencyService";

export default async (req, res) => {
    return res.xml(currencyService.generateWSDL(`http://${req.headers.host}/soap/currency`));
};
```

```
// src/routes/soap/currency/post.ts
import { currencyService } from "../../../app/CurrencyService";

export const secure = false;

export default async (req, res) => {
    return res.xml(await currencyService.handle(req.body as string));
};
```

`export const secure = false` opens the `POST` route to SOAP clients, which do not send a bearer token.

Auto WSDL

The WSDL document is generated automatically from your operation decorators:

```
curl "http://localhost:7148/soap/currency?wsdl"
```

```
<?xml version="1.0" encoding="UTF-8"?>
<definitions name="CurrencyService"
  targetNamespace="urn:CurrencyService"
  xmlns:tns="urn:CurrencyService"
  xmlns:soap="http://schemas.xmlsoap.org/wsdl/soap/"
```

```
  xmlns:xsd="http://www.w3.org/2001/XMLSchema"
  xmlns="http://schemas.xmlsoap.org/wsdl/">

  <types>
    <xsd:schema targetNamespace="urn:CurrencyService">
      <xsd:element name="ConvertCurrency">
        <xsd:complexType>
          <xsd:sequence>
            <xsd:element name="amount" type="xsd:double"/>
            <xsd:element name="from" type="xsd:string"/>
            <xsd:element name="to" type="xsd:string"/>
          </xsd:sequence>
        </xsd:complexType>
      </xsd:element>
      ...
  </types>

  <message name="ConvertCurrencyInput">
    <part name="parameters" element="tns:ConvertCurrency"/>
  </message>
  ...

  <portType name="CurrencyServicePortType">
    <operation name="ConvertCurrency">
      <input message="tns:ConvertCurrencyInput"/>
      <output message="tns:ConvertCurrencyOutput"/>
    </operation>
    ...
  </portType>

  ...
</definitions>
```

The target namespace is `urn:` followed by `serviceName`.

---

5. Calling Your Published Service

Test with a SOAP client or curl:

```
curl -X POST http://localhost:7148/soap/currency \
  -H "Content-Type: text/xml" \
  -H "SOAPAction: ConvertCurrency" \
  -d '<?xml version="1.0" encoding="UTF-8"?>
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
               xmlns:tns="urn:CurrencyService">
  <soap:Body>
    <tns:ConvertCurrency>
      <amount>100</amount>
      <from>USD</from>
      <to>EUR</to>
    </tns:ConvertCurrency>
  </soap:Body>
</soap:Envelope>'
```

Response:

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
<soap:Body>
<ConvertCurrencyResponse>
<result>92</result>
<rate>0.92</rate>
</ConvertCurrencyResponse>
</soap:Body>
</soap:Envelope>
```

---

6. Exercise: Expose a Product Catalogue as a SOAP Service

Build a SOAP service with two operations: `GetProduct` and `ListProducts`.

Requirements

- `GetProduct` - takes `productId: string`, returns `id`, `name`, `price`, `inStock`
- `ListProducts` - takes `category: string`, returns the IDs of the matching products, one `<productIds>` element each
- Mount at `/soap/products` with auto WSDL at `/soap/products?wsdl`

Test with:

```
# Fetch the WSDL
curl "http://localhost:7148/soap/products?wsdl"

# Call GetProduct
curl -X POST http://localhost:7148/soap/products \
  -H "Content-Type: text/xml" \
  -H "SOAPAction: GetProduct" \
  -d '<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
        <soap:Body><GetProduct><productId>1</productId></GetProduct></soap:Body>
      </soap:Envelope>'
```

---

7. Solution

```
// src/app/ProductService.ts
import { WSDLService, WSDLOperation } from "tina4-nodejs";

const PRODUCTS = [
    { id: "1", name: "Wireless Keyboard", category: "Electronics", price: 79.99, inStock: true },
    { id: "2", name: "Yoga Mat", category: "Fitness", price: 29.99, inStock: true },
    { id: "3", name: "Coffee Grinder", category: "Kitchen", price: 49.99, inStock: false },
    { id: "4", name: "Standing Desk", category: "Electronics", price: 549.99, inStock: true },
];

export class ProductService extends WSDLService {
    serviceName = "ProductService";
    serviceUrl = "/soap/products";

    @WSDLOperation({
        input: { productId: "string" },
        output: { id: "string", name: "string", price: "double", inStock: "boolean" }
    })
    async GetProduct(productId: string) {
        const product = PRODUCTS.find(p => p.id === productId);
        if (!product) {
            throw new Error(`Product ${productId} not found`);
        }
        return { id: product.id, name: product.name, price: product.price, inStock: product.inSt
    }

    @WSDLOperation({
        input: { category: "string" },
        output: { productIds: "string" }
    })
    async ListProducts(category: string) {
        const filtered = category
            ? PRODUCTS.filter(p => p.category.toLowerCase() === category.toLowerCase())
            : PRODUCTS;
        return { productIds: filtered.map(p => p.id) };
    }
}

export const productService = new ProductService();
```

```
// src/routes/soap/products/get.ts
import { productService } from "../../../app/ProductService";

export default async (req, res) => {
    return res.xml(productService.generateWSDL(`http://${req.headers.host}/soap/products`));  //
};
```

```
// src/routes/soap/products/post.ts
import { productService } from "../../../app/ProductService";

export const secure = false;

export default async (req, res) => {
    return res.xml(await productService.handle(req.body as string));
};
```

An array value is returned as one element per item, so `ListProducts` answers with a `<productIds>` element for each match.

---

8. Gotchas

1. SOAP is XML -- whitespace matters in some parsers

Extra whitespace inside element values can cause remote parsers to reject your response.

Fix: `WSDLService` generates clean XML without extra whitespace. Do not manually construct SOAP envelopes.

2. Namespace mismatches break everything

SOAP parsers are strict about XML namespaces. A mismatch between the namespace in your request and the namespace declared in the WSDL causes a fault.

Fix: Always copy the namespace from the generated WSDL document into your client requests. Do not guess namespace URNs.

3. SOAP requests get a 401

Problem: The WSDL loads, but every SOAP call returns `401 Unauthorized`.

Cause: `POST` routes require a bearer token by default, and SOAP clients do not send one.

Fix: Add `export const secure = false;` to the `post.ts` route file.

4. Decimal types lose precision in JavaScript

SOAP `decimal` values are arbitrary precision. JavaScript `number` is IEEE 754 floating point. `0.1 + 0.2 !== 0.3`.

Fix: For financial values, use string representation in the SOAP layer and a decimal library internally. Return values as strings formatted to the required precision.

DI Container

1. Stop Passing Dependencies Through Every Function Call

Your database connection, email client, cache client, payment gateway, and logger are needed in dozens of places. You could import them all directly. You could pass them as function parameters. Both approaches make testing hard and create tight coupling.

Dependency injection (DI) separates what you need from how you get it. A container holds instances. You register once. You resolve anywhere. Tests swap implementations without touching production code.

Tina4's `Container` class is a lightweight DI container. Register classes or factories. Resolve by name. Singleton or transient. No reflection magic, no decorators required unless you want them.

---

2. The Container Class

```
import { Container } from "tina4-nodejs";
```

`Container` is a class you instantiate. You can have multiple containers for different subsystems, or use a single application-wide container.

---

3. Registering a Service

`register()` stores a factory function or class constructor under a name:

```
import { Container } from "tina4-nodejs";

const container = new Container();

// Register a factory function
container.register("logger", () => ({
    info: (msg: string) => console.log(`[INFO] ${msg}`),
    error: (msg: string) => console.error(`[ERROR] ${msg}`)
}));

// Register a class constructor
class EmailService {
    send(to: string, subject: string, body: string) {
        console.log(`Sending "${subject}" to ${to}`);
    }
}

container.register("email", () => new EmailService());
```

Every call to `container.get("email")` invokes the factory and returns a new instance by default.

---

4. Singleton Registration

Use `singleton()` to register a service that is only instantiated once. Every call to `get()` returns the same instance:

```
import { Container } from "tina4-nodejs";
import { Database } from "tina4-nodejs/orm";

const container = new Container();

// Database connections are expensive -- only create one
container.singleton("db", () => {
    return Database.getConnection();
});

// Both calls return the exact same connection object
const db1 = await container.get("db");
const db2 = await container.get("db");
console.log(db1 === db2);  // true
```

Use `singleton` for: database connections, cache clients, API clients, configuration objects. Use `register` (transient) for: request-scoped services, services with per-call state.

---

5. Resolving Services with get()

```
const logger = container.get<Logger>("logger");
logger.info("Application started");

const email = container.get<EmailService>("email");
email.send("alice@example.com", "Welcome!", "Thanks for signing up.");
```

Provide a type parameter for TypeScript type safety. The container returns `unknown` without one; with a type parameter it returns the typed instance.

---

6. Checking Registration with has()

```
if (container.has("payment")) {
    const payment = container.get("payment");
    // ...
} else {
    console.warn("Payment service not registered");
}
```

`has()` returns `true` if a name is registered, regardless of whether the service has been instantiated yet.

---

7. Resetting the Container

Remove all registrations, useful in tests:

```
// Remove a specific registration
container.unregister("email");

// Clear everything
container.reset();
```

After `reset()`, any call to `get()` throws until services are re-registered.

---

8. Services That Depend on Other Services

Factories receive the container as an argument, enabling nested resolution:

```
import { Container } from "tina4-nodejs";

const container = new Container();

container.singleton("config", () => ({
    smtpHost: process.env.TINA4_MAIL_HOST ?? "localhost",
    smtpPort: parseInt(process.env.TINA4_MAIL_PORT ?? "587")
}));

container.singleton("mailer", (c) => {
    const config = c.get<{ smtpHost: string; smtpPort: number }>("config");
    return new SMTPMailer(config.smtpHost, config.smtpPort);
});

container.singleton("notifier", (c) => {
    const mailer = c.get<SMTPMailer>("mailer");
    return new NotificationService(mailer);
});

// All dependencies resolved automatically
const notifier = container.get<NotificationService>("notifier");
```

---

9. Application-Wide Container Pattern

Expose a single container instance from a module so it can be imported anywhere:

`src/container.ts`:

```
import { Container } from "tina4-nodejs";
import { Database } from "tina4-nodejs/orm";
import { Messenger } from "tina4-nodejs";
import { Log } from "tina4-nodejs";

export const app = new Container();

const db = await Database.getConnection();
app.singleton("db", () => db);

app.singleton("mailer", () => new Messenger());

app.singleton("log", () => Log);

app.register("orderService", (c) => {
```

```
    return new OrderService(
        c.get("db"),
        c.get("mailer"),
        c.get("log")
    );
});
```

`src/routes/orders.ts`:

```
import { Router } from "tina4-nodejs";
import { app } from "../container";

Router.post("/api/orders", async (req, res) => {
    const orderService = app.get<OrderService>("orderService");
    const result = await orderService.place(req.body);
    return res.status(201).json(result);
});
```

---

10. Testing with the DI Container

Swap real services for test doubles without touching production code:

```
import { Container } from "tina4-nodejs";

// Production container
import { app } from "../src/container";

// In tests: override specific services
beforeEach(() => {
    app.unregister("mailer");
    app.singleton("mailer", () => ({
        send: async () => ({ success: true, messageId: "test-123" })
    }));
});

afterEach(() => {
    // Restore after each test
    app.unregister("mailer");
    app.singleton("mailer", () => new Messenger());
});
```

The test installs a no-op mailer. Route handlers use `app.get("mailer")`; they never know the difference.

---

11. Exercise: Build a Service Container for a Store API

Wire together a product store API using the DI container.

Requirements

- Register three singletons: `productRepo` (in-memory data), `priceService` (computes discounts), and `logger` (structured logging)
- Register a transient `productService` that depends on all three

- Create a `GET /api/store/{id}` route that uses `productService` to fetch a product with its discounted price

Test with:

```
curl http://localhost:7148/api/store/1
curl http://localhost:7148/api/store/99  # should return 404
```

---

12. Solution

`src/store/container.ts`:

```
import { Container, Log } from "tina4-nodejs";

export const store = new Container();

// Repository: source of truth
store.singleton("productRepo", () => ({
    findById: (id: number) => {
        const products = [
            { id: 1, name: "Wireless Keyboard", basePrice: 79.99, category: "Electronics" },
            { id: 2, name: "Yoga Mat", basePrice: 29.99, category: "Fitness" },
            { id: 3, name: "Standing Desk", basePrice: 549.99, category: "Electronics" }
        ];
        return products.find(p => p.id === id) ?? null;
    },
    findAll: () => []
}));

// Price service: applies category discounts
store.singleton("priceService", () => ({
    calculate: (basePrice: number, category: string) => {
        const discounts: Record<string, number> = { Electronics: 0.05, Fitness: 0.10 };
        const discount = discounts[category] ?? 0;
        return {
            original: basePrice,
            discount: parseFloat((basePrice * discount).toFixed(2)),
            final: parseFloat((basePrice * (1 - discount)).toFixed(2))
        };
    }
}));

// Logger singleton
store.singleton("logger", () => Log);

// Product service: transient, depends on repo and price service
store.register("productService", (c) => {
    const repo = c.get<ReturnType<typeof createProductRepo>>("productRepo");
    const pricing = c.get<ReturnType<typeof createPriceService>>("priceService");
    const log = c.get<typeof Log>("logger");

    return {
        getById: (id: number) => {
            log.debug("ProductService.getById", { id });
            const product = repo.findById(id);
            if (!product) return null;
```

```
            const price = pricing.calculate(product.basePrice, product.category);
            return { ...product, price };
        }
    };
});

// Type helpers (not real -- illustrative)
function createProductRepo() { return {} as any; }
function createPriceService() { return {} as any; }
```

`src/routes/store.ts`:

```
import { Router } from "tina4-nodejs";
import { store } from "../store/container";

Router.get("/api/store/{id:int}", async (req, res) => {
    const id = req.params.id;
    const productService = store.get<{ getById: (id: number) => unknown }>("productService");
    const product = productService.getById(id);

    if (!product) {
        return res.status(404).json({ error: `Product ${id} not found` });
    }

    return res.json(product);
});
```

---

13. Gotchas

1. Registering after first get()

If you call `get("db")` before `register("db", ...)`, the container throws. There is no lazy fallback.

Fix: Register all services at application startup before any code calls `get()`. Keep all registrations in one file (`src/container.ts`) that is imported first.

2. Circular dependencies

`serviceA` depends on `serviceB`, which depends on `serviceA`. The container will recurse infinitely and crash.

Fix: Circular dependencies indicate a design problem. Extract shared logic into a third service that neither depends on the other.

3. Singletons holding stale state

A singleton database connection created at startup may become invalid if the database server restarts. Every `get()` returns the same dead connection.

Fix: Implement reconnect logic inside the singleton service, not outside it. The container manages the lifecycle; the service manages its own health.

4. Over-registering transient services

Registering everything as transient creates a new instance on every `get()` call, including heavyweight services with connection pools.

Fix: Use `singleton()` for anything that is expensive to create or that holds persistent state. Use `register()` only for services that must be fresh per request.

Service Runner

1. Work That Never Stops

Some tasks run once per request. Some tasks run forever. A cache warmer that refreshes data every 60 seconds. A queue drainer that processes jobs in a loop.

These are background services -- long-running loops that live alongside your HTTP server. They start when the application starts. They stop when the application stops. If they crash, they restart.

Tina4's Service Runner manages background services. Define a service. Register it. The runner handles lifecycle, restart-on-crash, and clean shutdown.

---

2. Defining a Service

A service is a class that extends `Service` and implements a `run()` method:

```
import { Service } from "tina4-nodejs";

class CacheWarmer extends Service {
    name = "CacheWarmer";
    interval = 60_000;  // milliseconds between runs

    async run() {
        console.log("Warming cache...");
        await warmProductCache();
        await warmCategoryCache();
        console.log("Cache warm");
    }
}
```

The `run()` method is called once per interval. When it completes, the runner waits `interval` milliseconds before calling it again.

---

3. Registering and Starting Services

```
import { ServiceRunner } from "tina4-nodejs";
import { CacheWarmer } from "./services/CacheWarmer";
import { HealthMonitor } from "./services/HealthMonitor";
import { InvoiceScheduler } from "./services/InvoiceScheduler";

const runner = new ServiceRunner();

runner.register(new CacheWarmer());
runner.register(new HealthMonitor());
runner.register(new InvoiceScheduler());

runner.start();
```

`runner.start()` starts all registered services in parallel. Each service gets its own loop. They do not block each other.

---

4. Service Lifecycle

```
register() --> IDLE
start()     --> RUNNING --> run() --> wait(interval) --> run() --> ...
stop()      --> STOPPING --> (current run() finishes) --> STOPPED
crash       --> ERROR    --> wait(restartDelay)       --> RUNNING
```

Stopping a Service

```
// Stop a specific service by name
runner.stop("CacheWarmer");

// Stop all services
runner.stopAll();
```

A stopped service waits for its current `run()` call to complete before shutting down. It does not kill mid-execution.

Restart on Crash

If `run()` throws an uncaught exception, the runner logs the error and restarts the service after a configurable delay:

```
class UnreliableService extends Service {
    name = "UnreliableService";
    interval = 5_000;
    restartDelay = 10_000;  // Wait 10 seconds before restarting after a crash

    async run() {
        if (Math.random() < 0.2) {
            throw new Error("Random failure (20% chance)");
        }
        await doWork();
    }
}
```

The default `restartDelay` is 5 seconds. Set it higher for services that connect to external systems (avoid hammering a downed dependency).

---

5. Service Options

Option	Default	Description
`name`	class name	Identifier for logging and `stop()`
`interval`	`60_000`	Milliseconds between `run()` calls
`restartDelay`	`5_000`	Delay before restarting after a crash
`runOnStart`	`true`	Whether to call `run()` immediately on start
`maxRestarts`	`Infinity`	Stop restarting after this many crashes

```
class HourlyReport extends Service {
    name = "HourlyReport";
    interval = 3_600_000;  // 1 hour
    runOnStart = false;    // Wait a full hour before first run
    maxRestarts = 5;       // Give up after 5 consecutive crashes
    restartDelay = 30_000; // Wait 30 seconds between restarts

    async run() {
        await generateHourlyReport();
        await sendReportEmail();
    }
}
```

---

6. A Real-World Example: Queue Drainer

A background service that continuously processes a job queue:

```
import { Service, Queue } from "tina4-nodejs";

class EmailQueueDrainer extends Service {
    name = "EmailQueueDrainer";
    interval = 1_000;  // Check every second
    private queue = new Queue({ topic: "emails" });

    async run() {
        const job = this.queue.pop();

        if (job === null) {
            return;  // Nothing to process this tick
        }

        try {
            await sendEmail(job.payload.to, job.payload.subject, job.payload.body);
            job.complete();
        } catch (err) {
```

```
            const message = err instanceof Error ? err.message : String(err);
            job.retry(30);  // Retry after 30 seconds
            console.error(`Email job failed: ${message}`);
        }
    }
}
```

The service pops one job per second. Errors do not crash the service -- they call `job.retry()` and the service continues to the next tick.

---

7. Health Check Endpoint

Expose service status via an HTTP endpoint:

```
import { Router, ServiceRunner } from "tina4-nodejs";

const runner = new ServiceRunner();

Router.get("/api/health/services", async (req, res) => {
    const status = runner.status();
    return res.json(status);
});
```

```
{
  "services": [
    {
      "name": "CacheWarmer",
      "state": "RUNNING",
      "lastRun": "2026-04-02T08:00:00.000Z",
      "runs": 48,
      "errors": 0,
      "nextRun": "2026-04-02T08:01:00.000Z"
    },
    {
      "name": "EmailQueueDrainer",
      "state": "RUNNING",
      "lastRun": "2026-04-02T08:00:59.123Z",
      "runs": 2880,
      "errors": 3,
      "nextRun": "2026-04-02T08:01:00.123Z"
    }
  ]
}
```

---

8. Integrating with the Application

Register services in your main entry point so they start alongside the HTTP server:

```
import { Tina4 } from "tina4-nodejs";
import { ServiceRunner } from "tina4-nodejs";
import { CacheWarmer } from "./services/CacheWarmer";
import { EmailQueueDrainer } from "./services/EmailQueueDrainer";
import { HealthMonitor } from "./services/HealthMonitor";
import "./routes/index";
```

```
const runner = new ServiceRunner();
runner.register(new CacheWarmer());
runner.register(new EmailQueueDrainer());
runner.register(new HealthMonitor());

const app = new Tina4();

app.start(() => {
    // Start background services after the server is up
    runner.start();
    console.log("HTTP server and background services started");
});

// Clean shutdown: stop services before the process exits
process.on("SIGTERM", () => {
    runner.stopAll();
    process.exit(0);
});

process.on("SIGINT", () => {
    runner.stopAll();
    process.exit(0);
});
```

---

9. Exercise: Daily Report Service

Build a background service that generates a daily sales summary every 24 hours.

Requirements

- Create a `DailySalesReport` service with a 24-hour interval
- The service should compute: total orders, total revenue, and top-selling category (use in-memory mock data)
- Log the report with `Log.info()` and optionally store it
- Expose a `GET /api/reports/last` endpoint that returns the most recent report

Test by setting the interval to 5 seconds so you can see it run:

```
# Start the server and watch the console for report logs
# After 5 seconds:
curl http://localhost:7148/api/reports/last
```

---

10. Solution

`src/services/DailySalesReport.ts`:

```
import { Service, Log } from "tina4-nodejs";

interface SalesReport {
    generatedAt: string;
    totalOrders: number;
    totalRevenue: number;
```

```
    topCategory: string;
}

const MOCK_ORDERS = [
    { id: 1, category: "Electronics", total: 79.99 },
    { id: 2, category: "Fitness", total: 29.99 },
    { id: 3, category: "Electronics", total: 549.99 },
    { id: 4, category: "Kitchen", total: 49.99 },
    { id: 5, category: "Electronics", total: 39.99 },
    { id: 6, category: "Fitness", total: 14.99 }
];

export let lastReport: SalesReport | null = null;

export class DailySalesReport extends Service {
    name = "DailySalesReport";
    interval = 86_400_000;  // 24 hours (use 5000 for testing)

    async run() {
        const totalOrders = MOCK_ORDERS.length;
        const totalRevenue = MOCK_ORDERS.reduce((sum, o) => sum + o.total, 0);

        const categoryCounts: Record<string, number> = {};
        for (const order of MOCK_ORDERS) {
            categoryCounts[order.category] = (categoryCounts[order.category] ?? 0) + 1;
        }

        const topCategory = Object.entries(categoryCounts)
            .sort((a, b) => b[1] - a[1])[0]?.[0] ?? "None";

        lastReport = {
            generatedAt: new Date().toISOString(),
            totalOrders,
            totalRevenue: parseFloat(totalRevenue.toFixed(2)),
            topCategory
        };

        Log.info("Daily sales report generated", {
            totalOrders,
            totalRevenue: lastReport.totalRevenue,
            topCategory
        });
    }
}
```

src/routes/reports.ts:

```
import { Router } from "tina4-nodejs";
import { lastReport } from "../services/DailySalesReport";

Router.get("/api/reports/last", async (req, res) => {
    if (!lastReport) {
        return res.status(404).json({ error: "No report generated yet" });
    }
    return res.json(lastReport);
});
```

src/index.ts:

```
import { Tina4, ServiceRunner } from "tina4-nodejs";
```

```
import { DailySalesReport } from "./services/DailySalesReport";
import "./routes/reports";

const runner = new ServiceRunner();
runner.register(new DailySalesReport());

const app = new Tina4();
app.start(() => {
    runner.start();
});
```

---

11. Gotchas

1. Services run immediately by default

If `runOnStart` is `true` (the default), `run()` is called as soon as `runner.start()` is invoked. For services that depend on a warm database or a running HTTP server, this can fail.

Fix: Set `runOnStart = false` if the service needs the application to be fully up. Or start the runner inside the `app.start()` callback, which fires after the server is ready.

2. Uncaught exceptions outside run() crash the process

If an exception escapes your `run()` method without being caught, the runner catches it and restarts the service. But an unhandled promise rejection at the top level of your service class bypasses the runner's error handling entirely.

Fix: Wrap the entire body of `run()` in try/catch. Never let async errors escape.

3. interval is wall-clock delay between runs, not between start times

If `run()` takes 45 seconds and `interval` is 60 seconds, the service runs every 105 seconds -- not every 60. This is intentional: no overlapping runs.

Fix: If you need wall-clock scheduling (exactly at :00 each minute), compute the time until the next scheduled run and use that as a dynamic interval.

4. Services do not share state safely

Two services reading and writing the same global variable without synchronization can produce inconsistent data.

Fix: Use a shared service (e.g., a singleton from the DI container) as the data layer. Services interact through it, not directly through shared globals.

MCP Dev Tools

1. What is MCP?

The Model Context Protocol connects AI coding tools to your running application. Claude Code, Cursor, and other assistants speak this protocol. When they connect, they see your database, routes, templates, and files -- not as static text, but as live, queryable tools.

Tina4 ships a built-in MCP server. It starts automatically in dev mode. No configuration needed.

---

2. How It Works

Set `TINA4_DEBUG=true` in your `.env` file and start the server:

```
tina4 serve
```

The console prints:

```
MCP server available at http://localhost:7148/__dev/mcp
```

Tina4 writes the connection details to `.claude/settings.json` automatically. Claude Code discovers the server on its next restart. Cursor reads the same file.

Localhost-Only Security

The built-in MCP server activates only when two conditions hold:

- `TINA4_DEBUG=true`
- The server runs on `localhost`, `127.0.0.1`, or `0.0.0.0`

Deploy to a remote server and the MCP endpoint vanishes -- even with debug enabled. This prevents accidental exposure of database queries and file editing in production.

To enable on a staging server (rare, intentional), set:

```
TINA4_MCP_REMOTE=true
```

---

3. Connecting AI Tools

Claude Code

Tina4 auto-generates `.claude/settings.json` with the current Streamable HTTP transport:

```
{
  "mcpServers": {
    "tina4-dev": {
      "type": "http",
      "url": "http://localhost:7148/__dev/mcp"
    }
  }
}
```

Restart Claude Code. It connects and lists the tools.

Prefer the command line? Register the same server in one step:

```
claude mcp add --transport http tina4-dev http://localhost:7148/__dev/mcp
```

Cursor

Copy the same MCP config into Cursor's settings. The `type` and `url` are identical.

Manual Connection

Any modern MCP client talks to a single endpoint:

- **Streamable HTTP**: `POST http://localhost:7148/__dev/mcp` -- send JSON-RPC 2.0 and read the response inline. `initialize` returns an `Mcp-Session-Id` header; send it back on later calls. `DELETE` on the same URL ends the session.

Older clients that speak the 2024-11-05 HTTP+SSE transport keep working:

- **SSE handshake** (legacy): `GET http://localhost:7148/__dev/mcp/sse` -- returns the message endpoint URL
- **Message endpoint** (legacy): `POST http://localhost:7148/__dev/mcp/message` -- accepts JSON-RPC 2.0 messages

---

4. Built-in Tools

The MCP server exposes 48 tools organized by category. The exact list lives in the framework's `@tina4/core` MCP tools registration block -- that is authoritative.

Database

Tool	Description
`database_query`	Execute a read-only SQL query (SELECT)
`database_execute`	Execute arbitrary SQL (INSERT, UPDATE, DELETE, DDL)
`database_tables`	List all tables in the database
`database_columns`	Get column definitions for a specific table

Example -- an AI assistant queries your database directly:

```
Tool: database_query
Arguments: {"sql": "SELECT id, name, email FROM users WHERE active = 1"}
```

The `database_execute` tool handles write operations. It commits automatically after each statement.

Routes

Tool	Description
`route_list`	List all registered routes with methods and auth status
`route_test`	Call a route and return status, body, and headers
`swagger_spec`	Return the full OpenAPI 3.0.3 specification

The `route_test` tool lets an AI assistant verify endpoints without leaving the conversation:

```
Tool: route_test
Arguments: {"method": "GET", "path": "/api/users"}
```

Templates

Tool	Description
`template_render`	Render a Frond template string with provided data

Useful for testing template snippets:

```
Tool: template_render
Arguments: {"template": "Hello {{ name }}!", "data": "{\"name\": \"Alice\"}"}
```

Files

Tool	Description
`file_read`	Read a project file (relative to project root)
`file_write`	Write or update a project file (full overwrite)
`file_patch`	Targeted edit -- replace `old_string` with `new_string` in a file
`file_list`	List files in a directory
`asset_upload`	Upload a file to `src/public/`

File operations are sandboxed. Paths that escape the project directory are rejected. An AI assistant cannot read `/etc/passwd` or write outside your project.

Migrations

Tool	Description
`migration_status`	List pending and completed migrations
`migration_create`	Create a new migration file
`migration_run`	Run all pending migrations

Data and ORM

Tool	Description
`orm_describe`	List all ORM models with fields, types, and relationships
`seed_table`	Seed a table with fake data

Infrastructure

Tool	Description
`queue_status`	Queue size by status (pending, completed, failed)
`session_list`	Active sessions with data
`cache_stats`	Response cache hit/miss statistics

Debugging

Tool	Description
`log_tail`	Read recent log entries
`error_log`	Recent errors and exceptions with stack traces
`env_list`	Environment variables (sensitive values redacted)
`system_info`	Framework version, Node.js version, project info

The `env_list` tool redacts any variable containing "secret", "password", "token", "key", or "credential" in its name.

Project introspection

Tool	Description
`git_status`	Show current branch, modified/untracked files, and recent commits
`deps_list`	List the project's declared dependencies (from `package.json`)
`project_overview`	One-shot snapshot: dependencies, routes, models, tables, errors, git

Persistent code index

Tool	Description
`index_rebuild`	Refresh the persistent project index (lazy, mtime-based)
`index_search`	Find files by path, symbol, route, or summary -- use FIRST for "where is X"
`index_file`	Full index entry for one file: symbols, routes, imports
`index_overview`	Project shape: files by language, routes, models, recent edits

Framework documentation (prose)

Tool	Description
`docs_list`	List framework documentation files
`docs_search`	Search Tina4 framework docs for a query string -- use before guessing
`docs_section`	Return a full markdown section from a framework doc file

Live API RAG (reflection)

`api_*` tools query the live framework reflection -- actual class/method signatures, file/line locations, with a `framework` vs `user` source tag. Use these for "what's the signature of X?" questions; `docs_*` is for "explain queues conceptually".

Tool	Description
`api_search`	Live API search -- finds framework + user classes/methods by name/summary

`api_class`	Live class reflection -- returns full method list and signatures for an FQN
`api_method`	Live method reflection -- returns signature, params, return type, file, line

Plan management

The `plan_*` family lets an AI assistant cooperate with a markdown plan file in `plan/`. Tina4 uses these to keep a steady record of in-progress refactors and multi-session work.

Tool	Description
`plan_current`	The active plan: title, steps (done/not), next step, progress
`plan_list`	All plans in `plan/` with progress and which one is active
`plan_create`	Create a new markdown plan and make it active
`plan_switch_to`	Make a different plan the active one
`plan_complete_step`	Tick a step as done (call the moment the step finishes)
`plan_add_step`	Append a new unchecked step to the current plan
`plan_note`	Append a timestamped note/breadcrumb to the current plan
`plan_archive`	Move a finished plan to `plan/done/` and clear the current pointer
`plan_read`	Full structured view of any plan by filename
`plan_flesh`	Auto-generate concrete build steps via qwen and append them to a plan

---

5. Protocol Details

The MCP server speaks JSON-RPC 2.0 over the current Streamable HTTP transport (protocol versions `2025-06-18` and `2025-03-26`). It still answers the legacy `2024-11-05` HTTP+SSE transport so older clients keep working. The server negotiates: it echoes the version a client asks for when it can speak it, and otherwise falls back to the newest one.

Streamable HTTP (current)

One endpoint carries the whole conversation:

```
POST /__dev/mcp
Content-Type: application/json
```

`initialize` mints a session and returns it in a header:

```
Mcp-Session-Id: 0f9a...c31
```

Send that header on every later request. A request bearing an unknown session gets a `404`, the client's cue to initialize again. A notification (a message with no `id`) returns `202` with an empty body. Every other request answers inline with the JSON-RPC response as `application/json`. `GET /__dev/mcp` returns `405` with an `Allow: POST, DELETE` header; `DELETE /__dev/mcp` ends the session.

Legacy HTTP+SSE (still supported)

```
GET /__dev/mcp/sse
Content-Type: text/event-stream
```

Returns the message endpoint URL as a server-sent event:

```
event: endpoint
data: http://localhost:7148/__dev/mcp/message
```

Then POST JSON-RPC messages to that endpoint:

```
POST /__dev/mcp/message
Content-Type: application/json
```

Messages

Both transports carry the same JSON-RPC 2.0 requests:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "tools/list",
  "params": {}
}
```

Returns JSON-RPC 2.0 responses:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": {
    "tools": [
      {"name": "database_query", "description": "Execute a read-only SQL query", "inputSchema":
    ]
  }
}
```

Lifecycle

- Client sends `initialize` -- server responds with capabilities
- Client sends `notifications/initialized` -- server acknowledges

- Client calls `tools/list` -- server returns all registered tools with schemas
- Client calls `tools/call` with tool name and arguments -- server executes and returns results
- Client calls `resources/list` and `resources/read` for data resources

---

6. Troubleshooting

MCP server not starting:

- Check `TINA4_DEBUG=true` in `.env`
- Verify the server is running on localhost (not a remote host)

Claude Code not detecting the server:

- Restart Claude Code after starting the Tina4 server
- Check `.claude/settings.json` exists with the correct URL
- Verify the port matches your server's port

Tools returning errors:

- Database tools require `initDatabase()` in your server setup
- File tools are sandboxed to the project directory
- `database_execute` only works on localhost

Remote server -- MCP disabled:

- This is intentional. Set `TINA4_MCP_REMOTE=true` only on staging environments you trust
- Never enable MCP on production servers

Building Custom MCP Servers

1. Beyond Dev Tools

Chapter 28 covered the built-in MCP server that ships with Tina4. It exposes framework internals for AI-assisted development. This chapter goes further: you build your own MCP servers that expose your application's business logic.

A CRM system exposes customer lookup. An accounting system exposes invoice queries. A warehouse system exposes inventory checks. Any domain logic that an AI assistant should access becomes an MCP tool.

---

2. Creating an MCP Server

Import `McpServer` and create an instance on any path:

```
import { McpServer } from "@tina4/core";

const mcp = new McpServer("/api/my-tools", "My App Tools", "1.0.0");
```

The server registers HTTP endpoints at:

- `POST /api/my-tools` -- Streamable HTTP (the current transport): send JSON-RPC and read the response inline. `initialize` returns an `Mcp-Session-Id` header, `DELETE` ends the session.
- `POST /api/my-tools/message` -- legacy JSON-RPC message handler
- `GET /api/my-tools/sse` -- legacy SSE discovery handshake

Register it with the router in your server setup:

```
mcp.registerRoutes(router);
```

---

3. Registering Tools with mcpTool

The `mcpTool` function registers a handler as an MCP tool. Parameter metadata becomes the input schema:

```
import { McpServer, mcpTool, schemaFromParams } from "@tina4/core";

const mcp = new McpServer("/crm/mcp", "CRM Tools");

mcpTool("lookup_customer", "Find a customer by email", mcp, [
  { name: "email", type: "string" },
])((args) => {
  return db.fetchOne("SELECT * FROM customers WHERE email = ?", [args.email]);
});

mcpTool("recent_orders", "Get recent orders for a customer", mcp, [
  { name: "customer_id", type: "integer" },
```

```
  { name: "limit", type: "integer", default: 10 },
])((args) => {
  return db.fetch(
    "SELECT * FROM orders WHERE customer_id = ? ORDER BY created_at DESC",
    [args.customer_id], { limit: (args.limit as number) || 10 }
  );
});
```

The registration extracts:

- **Parameter names** from the params list
- **Types** from the type field (`"string"`, `"integer"`, `"boolean"`, etc.)
- **Required vs optional** -- parameters with defaults are optional
- **Description** from the description argument

An AI assistant sees these tools and their schemas:

```
{
  "name": "lookup_customer",
  "description": "Find a customer by email",
  "inputSchema": {
    "type": "object",
    "properties": {
      "email": {"type": "string"}
    },
    "required": ["email"]
  }
}
```

---

4. Registering Resources with mcpResource

Resources are read-only data endpoints. They expose reference data that AI assistants can browse:

```
import { mcpResource } from "@tina4/core";

mcpResource("crm://product-catalog", "All active products", "application/json", mcp)(
  () => db.fetch("SELECT id, name, price, category FROM products WHERE active = 1")
);

mcpResource("crm://tax-rates", "Current tax rates by region", "application/json", mcp)(
  () => db.fetch("SELECT region, rate FROM tax_rates")
);
```

Resources are accessed via `resources/list` and `resources/read` in the MCP protocol.

---

5. Class-Based MCP Services

Group related tools into a service class. Register each method as a tool:

```
import { McpServer, schemaFromParams } from "@tina4/core";

const mcp = new McpServer("/accounting/mcp", "Accounting Tools");
```

```
class AccountingService {
  constructor(private db: any) {}

  lookup(invoiceNo: string) {
    return this.db.fetchOne(
      "SELECT * FROM invoices WHERE invoice_no = ?", [invoiceNo]
    );
  }

  balances(minAmount = 0.0) {
    return this.db.fetch(
      "SELECT * FROM invoices WHERE paid = 0 AND total >= ?", [minAmount]
    );
  }

  summary(year: number, month: number) {
    return this.db.fetchOne(
      "SELECT SUM(total) as revenue, COUNT(*) as invoice_count " +
      "FROM invoices WHERE strftime('%Y', created_at) = ? " +
      "AND strftime('%m', created_at) = ?",
      [String(year), String(month).padStart(2, "0")]
    );
  }
}

const svc = new AccountingService(db);

mcp.registerTool("invoice_lookup",
  (args) => svc.lookup(args.invoice_no as string),
  "Find an invoice by number",
  schemaFromParams([{ name: "invoice_no", type: "string" }])
);

mcp.registerTool("outstanding_balances",
  (args) => svc.balances((args.min_amount as number) || 0),
  "List all unpaid invoices",
  schemaFromParams([{ name: "min_amount", type: "number", default: 0.0 }])
);

mcp.registerTool("monthly_summary",
  (args) => svc.summary(args.year as number, args.month as number),
  "Revenue summary for a month",
  schemaFromParams([{ name: "year", type: "integer" }, { name: "month", type: "integer" }])
);
```

---

6. Securing MCP Endpoints

By default, developer MCP servers are public. Add authentication using Tina4 middleware:

```
// Secure the MCP routes via middleware
import { secured } from "@tina4/core";

// Apply auth middleware before registering routes
mcp.registerRoutes(router); // then protect the path with middleware
```

Or check the bearer token inside individual tools:

```
import { Auth } from "@tina4/core";

mcp.registerTool("sensitive_data",
  (args) => {
    const payload = Auth.validToken(args.token as string, secret);
    if (!payload) return { error: "Unauthorized" };
    return db.fetch("SELECT * FROM sensitive_table");
  },
  "Access restricted data",
  schemaFromParams([{ name: "token", type: "string" }])
);
```

---

7. Testing MCP Tools

Test tool functions directly, or test via the MCP protocol:

```
// Test the tool function directly
async function testLookupCustomer() {
  const result = await db.fetchOne("SELECT * FROM customers WHERE email = ?", ["alice@example.co
  assert(result !== null);
  assert(result.email === "alice@example.com");
}

// Test via MCP protocol
function testMcpToolCall() {
  const resp = JSON.parse(mcp.handleMessage({
    jsonrpc: "2.0",
    id: 1,
    method: "tools/call",
    params: {
      name: "lookup_customer",
      arguments: { email: "alice@example.com" },
    },
  }));
  assert("result" in resp);
  const content = resp.result.content[0].text;
  assert(content.toLowerCase().includes("alice"));
}
```

---

8. Complete Example: CRM MCP Server

Here is a full working example -- a CRM system with customer, order, and product tools:

```
// src/routes/mcp/setup.ts
import { McpServer, mcpTool, mcpResource, schemaFromParams } from "@tina4/core";
import { initDatabase } from "@tina4/orm";

const db = await initDatabase({ url: "sqlite:///crm.db" });

// Create MCP server
const crmMcp = new McpServer("/crm/mcp", "CRM Assistant", "1.0.0");

// Tools
mcpTool("find_customer", "Search customers by name or email", crmMcp, [
```

```
  { name: "query", type: "string" },
])((args) => {
  const q = args.query as string;
  return db.fetch(
    "SELECT * FROM customers WHERE name LIKE ? OR email LIKE ?",
    [`%${q}%`, `%${q}%`]
  );
});

mcpTool("customer_orders", "Get all orders for a customer", crmMcp, [
  { name: "customer_id", type: "integer" },
])((args) => {
  return db.fetch(
    "SELECT o.*, GROUP_CONCAT(oi.product_name) as items " +
    "FROM orders o LEFT JOIN order_items oi ON o.id = oi.order_id " +
    "WHERE o.customer_id = ? GROUP BY o.id ORDER BY o.created_at DESC",
    [args.customer_id]
  );
});

mcpTool("create_note", "Add a note to a customer record", crmMcp, [
  { name: "customer_id", type: "integer" },
  { name: "note", type: "string" },
])(async (args) => {
  await db.execute("INSERT INTO customer_notes (customer_id, note) VALUES (?, ?)",
    [args.customer_id, args.note]);
  return { success: true };
});

// Resources
mcpResource("crm://products", "Product catalog", "application/json", crmMcp)(
  () => db.fetch("SELECT * FROM products WHERE active = 1")
);

mcpResource("crm://stats", "CRM statistics", "application/json", crmMcp)(() => {
  const customers = db.fetch("SELECT COUNT(*) as count FROM customers");
  const orders = db.fetch("SELECT COUNT(*) as count, SUM(total) as revenue FROM orders");
  return {
    customers: customers[0]?.count ?? 0,
    orders: orders[0]?.count ?? 0,
    revenue: orders[0]?.revenue ?? 0,
  };
});

// Register routes
crmMcp.registerRoutes(router);

export { crmMcp };
```

Connect Claude Code to `http://localhost:7148/crm/mcp` (Streamable HTTP) and ask:

> *"Find all customers named Smith and show their recent orders"*

The AI calls `find_customer` with `query: "Smith"`, then `customer_orders` for each result. No custom API needed. The MCP protocol handles it.

---

9. Best Practices

- **One server per domain** -- CRM tools on `/crm/mcp`, accounting on `/accounting/mcp`
- **Keep tools focused** -- one query per tool, not a Swiss-army-knife tool
- **Use param metadata** -- types and defaults become the schema. An AI assistant cannot call a tool correctly without knowing the parameter types
- **Return structured data** -- objects and arrays, not formatted strings. Let the AI format for the user
- **Secure production endpoints** -- use middleware for any MCP server that runs outside localhost
- **Test tools directly** -- call the TypeScript function in your test suite, not just through the MCP protocol

Dev Tools

1. Debugging at 2am

2am. Production monitoring pings you. A 500 error on checkout. You pull up the dev dashboard. The request inspector shows the failing request. The stack trace points to line 47 of `src/routes/checkout.ts` -- a null reference on the shipping address because the user skipped the form field. You add a null check. Push the fix. Back to sleep. Total time: 30 seconds.

Tina4's dev tools are not an afterthought. They ship with the framework from day one. When `TINA4_DEBUG=true`, you get a development dashboard, an error overlay with source code, live reload, a request inspector with replay, a SQL query runner, a queue monitor, and system info -- all without installing extra packages.

---

2. Enabling the Dev Dashboard

Set `TINA4_DEBUG=true` in your `.env`:

```
TINA4_DEBUG=true
```

Restart your server and navigate to:

```
http://localhost:7148/__dev
```

No token or additional environment variables needed. The dashboard runs only when debug mode is on. Set `TINA4_DEBUG=false` in production and the entire dashboard disappears.

---

3. Dashboard Overview

The dev dashboard has several sections. Navigation tabs run along the top.

System Overview

The landing page shows at a glance:

- **Framework version** -- The installed Tina4 Node.js version
- **Node.js version** -- The running Node.js version and loaded packages
- **Uptime** -- How long the server has been running
- **Memory usage** -- Current and peak memory consumption
- **Database status** -- Connection status, database engine, file size (for SQLite)
- **Environment** -- Current `.env` variables (sensitive values masked)
- **Project structure** -- Directory listing of your project with file counts

Check here first when something feels off. Is the database connected? Is the right Node.js version running? Are the environment variables loaded?

Request Inspector

- Recent HTTP requests with method, path, status, duration
- Click any request to see headers, body, database queries, template renders

Error Log

- Unhandled exceptions with stack traces
- Occurrence counts and timestamps

Queue Manager

- Queue status: pending, reserved, completed, failed, dead counts
- Recent jobs with status and duration

WebSocket Monitor

- Active WebSocket connections with metadata
- Message history

Routes

- All registered routes with methods, paths, middleware, auth status

Mail

- Intercepted emails with To, Subject, HTML body, attachments

---

4. The Dev Toolbar

Visit any HTML page in your application. A debug toolbar appears at the bottom of the page. Thin bar. Click it to expand.

The toolbar shows:

Field	What It Means
Request	HTTP method and URL of the current request
Status	HTTP response status code
Time	Total request processing time in milliseconds
DB	Number of database queries executed and total query time
Memory	Node.js memory used for this request
Template	The template rendered and how long rendering took
Session	Session ID and session data summary

Route	Which route handler matched this request

Click any section to expand it. Clicking "DB" shows every SQL query that ran during the request, with the query text, parameters, execution time, and row count.

Disabling the Toolbar

The toolbar hides when `TINA4_DEBUG=false`. You can also hide it for specific routes by returning a response with the `X-Debug-Toolbar: off` header:

```
import { Router } from "tina4-nodejs";

Router.get("/api/data", async (req, res) => {
    return res.header("X-Debug-Toolbar", "off").json({ data: "no toolbar" });
});
```

This is useful for API endpoints that return JSON -- the toolbar only matters for HTML pages.

---

5. Error Overlay

An unhandled exception occurs. Tina4 does not show a generic "500 Internal Server Error" page. It shows a detailed error overlay:

- **Exception type and message** -- What went wrong, in plain language
- **Stack trace** -- Every function call that led to the error, from entry point to crash
- **Source code** -- The TypeScript code around the line that threw the exception, with the failing line highlighted
- **Request data** -- HTTP method, URL, headers, body, and query parameters of the request that triggered the error
- **Environment** -- Relevant `.env` variables at the time of the error

Example Error

Write this handler:

```
Router.get("/api/users/{userId}", async (req, res) => {
    const userId = req.params.userId;
    const user = await getUserFromDb(userId);
    return res.json({ name: user.name, email: user.email });
});
```

If `getUserFromDb()` returns `null` for a missing user, the error overlay shows:

```
TypeError: Cannot read properties of null (reading 'name')

  File: src/routes/users.ts, line 4

  2 |     const userId = req.params.userId;
  3 |     const user = await getUserFromDb(userId);
> 4 |     return res.json({ name: user.name, email: user.email });
  5 | });
```

```
  Request: GET /api/users/999
  Headers: {"Accept": "application/json"}
```

The highlighted line makes the cause obvious. `user` is `null` and the code accesses `.name` on it.

Error Overlay in Production

When `TINA4_DEBUG=false`, the error overlay is disabled. Users see a clean error page. Full error details go to `logs/error.log` for later investigation. Custom error pages go in `src/templates/errors/`:

```
src/templates/errors/404.html
src/templates/errors/500.html
```

---

6. The Gallery

The gallery is a collection of ready-to-use code examples built into the dev dashboard. Each gallery item includes:

- A description of what it does
- The complete source code (routes, templates, models)
- A "Try It" button that installs the example into your project

Available gallery items:

- **JWT Authentication** -- Complete login/register flow with token management
- **CRUD API** -- Full REST API with ORM model
- **File Upload** -- Multipart form handling with file storage
- **WebSocket Chat** -- Real-time chat with rooms
- **Email Contact Form** -- Form with Messenger integration
- **Dashboard Template** -- Admin dashboard with tina4css

Click "Try It" on any gallery item. Tina4 creates the necessary files in your project. Modify them to fit your needs.

---

7. Live Reload

When `TINA4_DEBUG=true`, Tina4 watches your project files for changes and refreshes the browser automatically. Edit a route file. Save it. The browser refreshes with the new code. No manual restart required.

```
tina4 serve
```

```
  Tina4 Node.js v3.11.12
  HTTP server running at http://0.0.0.0:7148
  File watcher active - press Ctrl+C to stop
```

Live reload watches:

- `src/**` -- Routes, ORM, middleware, templates, SCSS
- `migrations/` -- SQL migrations
- `.env` -- Environment variables

How It Works

The `tina4` Rust CLI is the sole file watcher for the whole Tina4 stack. There is no framework-side watcher (`watcher.ts` using `fs.watch` was removed in 3.11.x).



- The CLI watches `src/`, `migrations/`, `.env` with the `notify` crate. Events are filtered to real source changes; metadata/access events, `__pycache__`, `.git`, `node_modules`, `vendor`, `dist`, `target`, `logs`, `.log`/`.db*`/`.swp` files are ignored. A real mtime check defeats overlayfs / polling-mode spurious events (Podman, distrobox).
- On a real change, the CLI POSTs `/__dev/api/reload` to the running Node server. The server keeps running, with no process restart for file edits.
- `DevAdmin` bumps its in-memory `_reloadMtime` counter and broadcasts `{type: "reload"}` over WebSocket at `/__dev_reload`. `GET /__dev/api/mtime` returns the counter for browsers using the polling fallback.
- The dev-toolbar script reloads the browser. SCSS/CSS changes are signalled as `type: "css"` and swap the stylesheet without a full reload.

The reload happens in under a second. Edit code. Switch to the browser. The changes are already there. The server process stays up throughout.

---

8. Request Inspector

The request inspector records every HTTP request to your application. For each request:

- **Timestamp** -- When the request arrived
- **Method** -- GET, POST, PUT, DELETE
- **URL** -- The full URL including query parameters
- **Status** -- The HTTP status code returned (color-coded: green for 2xx, yellow for 4xx, red for 5xx)
- **Time** -- How long the request took to process
- **Request ID** -- A unique identifier for correlating logs

Click on any request to see its full details.

Request Details Panel

- **Headers** -- All request headers (Accept, Content-Type, Authorization)

- **Body** -- The request body (for POST/PUT/PATCH), formatted as JSON if applicable
- **Query parameters** -- Parsed URL query parameters
- **Route match** -- Which route definition matched this request
- **Middleware** -- Which middleware ran and how long each took
- **Database queries** -- Every SQL query executed during this request, with timing
- **Template renders** -- Which templates rendered and how long each took
- **Response headers** -- The response headers sent back
- **Response body** -- The first 1000 characters of the response body

Filtering Requests

The inspector supports filtering:

- **By status**: Click the status code badges at the top (e.g., show only 5xx errors)
- **By method**: Filter by GET, POST, PUT, DELETE
- **By path**: Search for a URL pattern (e.g., `/api/` for API requests only)
- **By time range**: Show requests from the last 5 minutes, 1 hour, or all time

Request Replay

Click "Replay" on any request to re-send it. The inspector fires the same method, URL, headers, and body. You reproduce an error without constructing the curl command by hand.

This is the fastest path from "what happened?" to "I can see it happen again." Find a failing request. Hit Replay. Watch the error overlay show the stack trace. Fix the code. Replay again. Green status code. Done.

---

9. SQL Query Runner

The dev dashboard includes a SQL query runner. Execute queries against your database:

- **Execute queries** -- Run any SQL statement
- **See results** -- Formatted table with column headers and row numbers
- **View query timing** -- How long each query took
- **Browse tables** -- A sidebar lists all tables with their column definitions
- **Export results** -- Download as CSV

```
SELECT * FROM products WHERE category = 'Electronics' ORDER BY price DESC;
```

The results display in a table with column headers, row numbers, and data type indicators. Copy results, export as CSV, or run another query.

Safety

The query runner is read-write in development. You can run INSERT, UPDATE, and DELETE statements. Be careful -- there is no undo. In shared environments, consider a read-only database connection for the dev dashboard.

The query runner only works when `TINA4_DEBUG=true`. It is disabled in production.

---

10. Queue Monitor

If your application uses background job queues (Chapter 12 -- Queues), the dev dashboard includes a queue monitor. The monitor shows five categories:

- **Pending jobs** -- Jobs waiting to be processed
- **Active jobs** -- Jobs currently being processed
- **Completed jobs** -- Recently completed jobs with timing
- **Failed jobs** -- Jobs that threw exceptions, with error details
- **Dead-letter queue** -- Jobs that failed too many times

For each job, you see:

- The job function name
- The payload (serialized arguments)
- When it was enqueued
- When it started processing (if active)
- The error message and stack trace (if failed)
- How many times it has been retried

You can also take action:

- **Retry a failed job** -- Click "Retry" to move it back to the pending queue
- **Delete a job** -- Remove it from any queue
- **Pause/resume the queue** -- Stop processing without losing jobs

The queue monitor gives you a window into your background work. A job fails. You see the error. You fix the code. You hit Retry. The job processes. No log file hunting. No guesswork about what payload caused the failure.

---

11. System Info

The System Info tab shows detailed information about your environment:

- **Node.js Configuration** -- Version, installed packages, memory limit
- **Database Info** -- Engine, version, connection details, table sizes, index information

- **Server Info** -- OS, hostname, server software, document root
- **Tina4 Config** -- All loaded `.env` variables (sensitive values masked), auto-discovered routes, registered middleware, ORM models
- **Disk Usage** -- Size of your project directory, data directory, logs directory

This panel solves the "it works on my machine" problem. Compare the System Info output between two environments. Spot the difference. The wrong Node.js version. A missing package. A misconfigured environment variable. The answer is in the panel.

---

12. Logging

```
import { Log } from "tina4-nodejs";

Log.debug("Debug message");
Log.info("Info message");
Log.warning("Warning message");
Log.error("Error message");
```

Log levels are controlled by `TINA4_LOG_LEVEL` in `.env`:

Level	Shows
`ALL`	Everything
`DEBUG`	Debug and above
`INFO`	Info and above
`WARNING`	Warning and above
`ERROR`	Errors only
`NONE`	Nothing

Logs are written to `logs/app.log` and to stdout.

---

13. Health Check

```
http://localhost:7148/health
```

Returns system status: database connectivity, uptime, and version. Your monitoring tools and load balancers hit this endpoint.

---

14. Exercise: Debug a Failing Route

Your colleague wrote a route that fails. Use the dev tools to find and fix the bug.

Setup

Create `src/routes/buggy.ts`:

```
import { Router } from "tina4-nodejs";
import { Database } from "tina4-nodejs/orm";
import { cacheGet, cacheSet } from "tina4-nodejs";

Router.get("/api/buggy/users", async (req, res) => {
    const users = await cacheGet("all_users");
    if (users) {
        return res.json({ users, source: "cache" });
    }

    // Bug 1: This query has an error
    const db = await Database.getConnection();
    const result = await db.fetchAll("SELCT * FROM users ORDER BY name");

    await cacheSet("all_users", result, 300);
    return res.json({ users: result, source: "database" });
});

Router.post("/api/buggy/users", async (req, res) => {
    const body = req.body;

    // Bug 2: Missing validation
    const user = new User();
    user.name = body.name;
    user.email = body.email;
    await user.save();

    // Bug 3: Wrong status code
    return res.json({ message: "User created", user: user.toDict() });
});
```

Requirements

- Open the dev dashboard at `http://localhost:7148/__dev`
- Hit the `GET /api/buggy/users` endpoint and find Bug 1 using the error overlay
- Hit the `POST /api/buggy/users` endpoint without a body and find Bug 2 using the request inspector
- Fix all three bugs:
- Bug 1: Fix the SQL syntax error
- Bug 2: Add validation for required fields
- Bug 3: Return 201 instead of 200
- Use Request Replay to verify each fix

Solution

```
import { Router } from "tina4-nodejs";
import { Database } from "tina4-nodejs/orm";
import { cacheGet, cacheSet } from "tina4-nodejs";

Router.get("/api/buggy/users", async (req, res) => {
    const users = await cacheGet("all_users");
    if (users) {
        return res.json({ users, source: "cache" });
    }

    const db = await Database.getConnection();
    const result = await db.fetchAll("SELECT * FROM users ORDER BY name");  // Fixed: SELCT -> S

    await cacheSet("all_users", result, 300);
    return res.json({ users: result, source: "database" });
});

Router.post("/api/buggy/users", async (req, res) => {
    const body = req.body;

    // Fixed: Added validation
    if (!body.name || !body.email) {
        return res.status(400).json({ error: "Name and email are required" });
    }

    const user = new User();
    user.name = body.name;
    user.email = body.email;
    await user.save();

    return res.status(201).json({ message: "User created", user: user.toDict() });  // Fixed: 20
});
```

The dev tools made each bug visible. The error overlay showed the SQL syntax error with the exact line. The request inspector showed the missing body causing a null reference. Request Replay let you re-send the fixed requests without leaving the dashboard.

---

15. Gotchas

1. Dev Dashboard Accessible on Network

Problem: Anyone on your network can access the dev dashboard.

Cause: `TINA4_DEBUG=true` makes the dashboard available at `/__dev`.

Fix: In production, set `TINA4_DEBUG=false` to disable the dashboard. In shared development environments, restrict network access.

2. Live Reload Causes Connection Drops

Problem: WebSocket connections drop every time you save a file.

Cause: The server restarts on file change, which closes all active connections.

Fix: Use the file-based routing approach where only affected routes reload. For WebSocket development, clients with frond.js reconnect automatically after restart.

3. Error Overlay Shows in Production

Problem: Users see TypeScript stack traces when errors occur.

Cause: `TINA4_DEBUG=true` is set in the production environment.

Fix: Set `TINA4_DEBUG=false` in production. The error overlay is replaced by a clean error page. Full details go to `logs/error.log`.

4. Request Inspector Slows Down the Server

Problem: The server becomes slower as it runs longer.

Cause: The request inspector stores every request in memory. After thousands of requests, memory usage grows.

Fix: The inspector limits storage to the last 1000 requests. If you notice slowness, clear the inspector from the dashboard. In production, the inspector is disabled.

5. SQL Runner Executes Destructive Queries

Problem: Someone ran `DROP TABLE users` in the SQL query runner.

Cause: The SQL runner executes any valid SQL query. There is no confirmation step.

Fix: The SQL runner only works when `TINA4_DEBUG=true`. Never leave debug mode on in production. For sensitive development databases, use a read-only database connection for the SQL runner.

6. Template Changes Not Reflected

Problem: You edited a template but the page still shows the old version.

Cause: Template caching is enabled (`TINA4_TEMPLATE_CACHE_TTL=true`). The compiled template serves from cache.

Fix: In development, set `TINA4_TEMPLATE_CACHE_TTL=false` (this is the default when `TINA4_DEBUG=true`). If you enabled template caching manually, disable it for development.

7. Queue Monitor Shows No Jobs

Problem: The Queue Monitor tab is empty even though your application uses queues.

Cause: The queue system is not running. Jobs are enqueued but no worker processes them.

Fix: Ensure your queue consumers are running. The queue monitor reflects the state of the queue storage. If no consumer processes jobs, they sit in "Pending" and never move to "Active" or "Completed."

8. Debug Mode in Version Control

Fix: Add `.env` to `.gitignore`.

---

9. Logs Fill Up Disk

Fix: Set `TINA4_LOG_LEVEL=WARNING` in production.

10. Performance Overhead

Fix: Debug mode adds overhead. Always disable in production.

Tina4 CLI

1. Getting a New Developer Up to Speed

Monday morning. A new developer joins your team. You hand them the repo URL. By 10am they have a running project, a new database model, CRUD routes, a migration, and a deployment to staging. All from the command line. No documentation scavenger hunt. No boilerplate copy-paste.

The Tina4 CLI is a single Rust binary. It manages all four Tina4 frameworks (PHP, Python, Ruby, Node.js). The commands are identical across languages. Learn the CLI for Node.js. You know it for Python.

---

2. tina4 init -- Project Scaffolding

You saw this in Chapter 1. Now the details.

```
tina4 init my-project
```

```
Creating Tina4 project in ./my-project ...
  Detected language: Node.js (package.json)
  Created .env
  Created .env.example
  Created .gitignore
  Created src/routes/
  Created src/orm/
  Created migrations/
  Created src/seeds/
  Created src/templates/
  Created src/templates/errors/
  Created src/public/
  Created src/public/js/
  Created src/public/css/
  Created src/public/scss/
  Created src/public/images/
  Created src/public/icons/
  Created src/locales/
  Created data/
  Created logs/
  Created secrets/
  Created tests/

Project created! Next steps:
  cd my-project
  npm install
  tina4 serve
```

Language Detection

The CLI detects the language from existing files:

File Present	Language
`composer.json`	PHP

`pyproject.toml` or `requirements.txt`	Python
`Gemfile`	Ruby
`package.json`	Node.js

If no language-specific file exists, the CLI asks:

```
tina4 init my-project
```

```
No language detected. Which language?
  1. PHP
  2. Python
  3. Ruby
  4. Node.js
> 4

Creating Tina4 Node.js project in ./my-project ...
```

Explicit Language Selection

Skip the prompt by specifying the language:

```
tina4 init nodejs my-project
```

This creates a Node.js project with `package.json`, `app.ts`, and the full directory structure.

Init into an Existing Directory

Already have a project? Add Tina4 structure:

```
cd existing-project
tina4 init .
```

The CLI creates only files and directories that do not exist. It never overwrites.

---

3. tina4 serve -- Dev Server

```
tina4 serve
```

```
  Tina4 Node.js v3.10.3
  HTTP server running at http://0.0.0.0:7148
  WebSocket server running at ws://0.0.0.0:7148
  Live reload enabled
  Press Ctrl+C to stop
```

`tina4 serve` detects the language and starts the appropriate server. For Node.js, it spawns the Tina4 Node.js runtime with SCSS compilation, file watching, and live reload all wired in.

Options

```
tina4 serve --port 8080        # Custom port
tina4 serve -p 8080            # Same thing, short form
tina4 serve --host 127.0.0.1   # Bind to localhost only
tina4 serve --no-browser       # Don't open a browser tab on startup
tina4 serve --no-reload        # Don't reload the browser when files change
tina4 serve --dev              # Force the dev server even if a production server is installed
tina4 serve --production       # Production server, debug off, no live reload
```

Serving a Project by Name

You don't have to `cd` into a project first. Pass its folder name and the CLI will find it, change into it, and serve it:

```
tina4 serve my-app             # ./my-app, or my-app in your projects folder
tina4 serve my-app -p 7150     # Options still apply
```

The CLI looks for `./my-app` in the current folder first, then in the projects folder `tina4 setup` recorded. If neither has it, it stops with `Project 'my-app' not found` and names the folder it searched. Run a bare `tina4 serve` outside any project and it will look in the projects folder too: one project there gets served, and several get listed so you can pick one by name.

Bypassing the CLI

The framework refuses to start without the Rust CLI. For sandboxed environments (Docker images, CI runners) set `TINA4_OVERRIDE_CLIENT=true` in `.env` and only then run:

```
TINA4_OVERRIDE_CLIENT=true npx tsx app.ts
```

This bypasses SCSS compilation and live reload; use it only when the Rust CLI is genuinely unavailable.

---

4. tina4 generate model -- ORM Scaffolding

The `generate model` command creates an ORM model file and a matching migration. One command produces both.

```
tina4 generate model Product
```

```
Created src/models/Product.ts
Created migrations/20260322120000_create_product.sql
Created migrations/20260322120000_create_product.down.sql
Created tests/product_model.test.ts
```

The generated model:

```
import { BaseModel } from "tina4-nodejs/orm";

export class Product extends BaseModel {
    static tableName = "products";
    static primaryKey = "id";

    id!: number;
    createdAt!: string;
    updatedAt!: string;
```

```
}
```

The generated migration:

```
-- UP
CREATE TABLE products (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    created_at TEXT DEFAULT CURRENT_TIMESTAMP,
    updated_at TEXT DEFAULT CURRENT_TIMESTAMP
);

-- DOWN
DROP TABLE IF EXISTS products;
```

The model is ready to use. Add your fields and run migrations.

Adding Fields

Specify fields on the command line:

```
tina4 generate model Product --fields "name:string,price:float,category:string,inStock:bool"
```

The generated model includes all the fields:

```
import { BaseModel } from "tina4-nodejs/orm";

export class Product extends BaseModel {
    static tableName = "products";
    static primaryKey = "id";

    id!: number;
    name!: string;
    price!: number;
    category!: string;
    inStock!: boolean;
    createdAt!: string;
    updatedAt!: string;
}
```

And the migration:

```
-- UP
CREATE TABLE products (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL DEFAULT '',
    price REAL NOT NULL DEFAULT 0,
    category TEXT NOT NULL DEFAULT '',
    in_stock INTEGER NOT NULL DEFAULT 0,
    created_at TEXT DEFAULT CURRENT_TIMESTAMP,
    updated_at TEXT DEFAULT CURRENT_TIMESTAMP
);

-- DOWN
DROP TABLE IF EXISTS products;
```

Field Types

CLI Type	TypeScript Type	SQLite Column
`string`	`string`	`TEXT`
`int`	`number`	`INTEGER`
`float`	`number`	`REAL`
`bool`	`boolean`	`INTEGER`
`text`	`string`	`TEXT`
`date`	`string`	`TEXT`

Options

Flag	Description	Example
`--fields`	Comma-separated field definitions	`--fields "name:string,price:float"`
`--table-name`	Use this table name instead of the one derived from the class name	`--table-name products`
`--no-migration`	Skip migration generation	`--no-migration`

Apart from these and the general `--dry-run`, `--json` and `--help`, `generate model` stops with `Unknown option` on any other flag. To add CRUD routes, run `tina4 generate route` after it (section 8).

---

5. tina4 generate route -- CRUD Route Scaffolding

The `generate route` command creates a complete CRUD route file with all five REST endpoints. It reads the model's properties and builds routes with proper imports and response handling.

```
tina4 generate route products
```

```
Created src/routes/api/products/get.ts
Created src/routes/api/products/post.ts
Created src/routes/api/products/[id]/get.ts
Created src/routes/api/products/[id]/put.ts
Created src/routes/api/products/[id]/delete.ts
Created tests/products.test.ts
```

The generated route file:

```
import { Router } from "tina4-nodejs";

Router.get("/api/products", async (req, res) => {
```

```
    const page = parseInt(req.query.page ?? "1", 10);
    const perPage = parseInt(req.query.per_page ?? "20", 10);
    const offset = (page - 1) * perPage;

    const products = await Product.findAll({ limit: perPage, offset });
    const results = products.map(p => p.toDict());

    return res.json({
        data: results,
        page,
        per_page: perPage,
        count: results.length
    });
});

Router.get("/api/products/{productId:int}", async (req, res) => {
    const product = await Product.findById(req.params.productId);

    if (product === null) {
        return res.status(404).json({ error: "Product not found" });
    }

    return res.json(product.toDict());
});

Router.post("/api/products", async (req, res) => {
    const body = req.body;

    const product = new Product();
    product.name = body.name ?? "";
    product.price = parseFloat(body.price ?? "0");
    product.category = body.category ?? "";
    product.inStock = Boolean(body.inStock ?? false);
    await product.save();

    return res.status(201).json(product.toDict());
});

Router.put("/api/products/{productId:int}", async (req, res) => {
    const product = await Product.findById(req.params.productId);

    if (product === null) {
        return res.status(404).json({ error: "Product not found" });
    }

    const body = req.body;
    if (body.name !== undefined) product.name = body.name;
    if (body.price !== undefined) product.price = parseFloat(body.price);
    if (body.category !== undefined) product.category = body.category;
    if (body.inStock !== undefined) product.inStock = Boolean(body.inStock);
    await product.save();

    return res.json(product.toDict());
});

Router.delete("/api/products/{productId:int}", async (req, res) => {
    const product = await Product.findById(req.params.productId);

    if (product === null) {
```

```
        return res.status(404).json({ error: "Product not found" });
    }

    await product.delete();
    return res.status(204).end();
});
```

The generator reads the model's properties and creates routes with type casting and null checks. Customize the generated code immediately. It is regular TypeScript, not magic.

Options

Flag	Description	Example
`--prefix`	Custom route prefix (default: `/api`)	`--prefix /api/v2`
`--middleware`	Add middleware to all routes	`--middleware authMiddleware`

---

6. tina4 generate migration -- Migration Scaffolding

The `generate migration` command creates a timestamped migration file with `UP` and `DOWN` sections. The timestamp ensures migrations run in order.

```
tina4 generate migration add_category_to_products
```

```
Created migrations/20260322120500_add_category_to_products.sql
```

The generated file:

```
-- UP
-- Add your forward migration SQL here


-- DOWN
-- Add your rollback migration SQL here
```

Fill in the SQL:

```
-- UP
ALTER TABLE products ADD COLUMN category TEXT DEFAULT '';

-- DOWN
ALTER TABLE products DROP COLUMN category;
```

The timestamp prefix (`20260322120500`) ensures migrations run in order. Each migration runs once. The framework tracks which ones have been applied.

When to Generate a Migration

Generate a new migration when you need to change the database schema after the initial model migration. Adding a column. Creating an index. Renaming a table. Each change gets its own migration file with a unique timestamp.

---

7. tina4 generate middleware -- Middleware Scaffolding

The `generate middleware` command creates a middleware function with the correct signature and a placeholder for your logic.

```
tina4 generate middleware rateLimit
```

```
Created src/middleware/rateLimit.ts
```

The generated file:

```
import { Middleware } from "tina4-nodejs";

async function rateLimit(req: any, res: any, next: Function) {
    // Add your middleware logic here

    // Continue to the next middleware or route handler
    return next();

    // Or return early to block the request:
    // return res.status(429).json({ error: "Rate limit exceeded" });
}

export default rateLimit;
```

The middleware is a named function. Reference it in route definitions:

```
Router.get("/api/data", async (req, res) => {
    return res.json({ data: "protected" });
}, "rateLimit");
```

---

8. Generate a Model and Its Routes

`generate model` has no flag that adds routes. Run `generate route` after it:

```
tina4 generate model Product
tina4 generate route products
```

```
Created src/models/Product.ts
Created migrations/20260322120000_create_product.sql
Created migrations/20260322120000_create_product.down.sql
Created tests/product_model.test.ts
Created src/routes/api/products/get.ts
Created src/routes/api/products/post.ts
Created src/routes/api/products/[id]/get.ts
Created src/routes/api/products/[id]/put.ts
Created src/routes/api/products/[id]/delete.ts
Created tests/products.test.ts
```

Model, migration, CRUD routes, and a test for each. For admin pages over the model as well, use `tina4 generate crud` (see the Scaffolding chapter).

---

9. tina4 doctor -- Health Check

The `doctor` command checks your project for common issues:

```
tina4 doctor
```

```
Tina4 Doctor -- Checking your project...

  [OK] Node.js v20.11.1 detected
  [OK] npm package manager found
  [OK] tina4-nodejs package installed (v3.10.3)
  [OK] .env file exists
  [OK] Database connection: sqlite:///data/app.db
  [OK] Database is accessible
  [OK] src/routes/ directory exists (3 route files)
  [OK] src/orm/ directory exists (2 model files)
  [OK] src/templates/ directory exists (5 templates)
  [OK] src/public/ directory exists (static files served)
  [OK] tests/ directory exists (4 test files)
  [WARN] No migrations found in migrations/
  [OK] AI context: Claude Code detected, CLAUDE.md present
  [OK] .gitignore includes .env, data/, logs/

  12 checks passed, 1 warning, 0 errors
```

Doctor checks:

- Node.js version and package manager
- Tina4 package installation and version
- `.env` file existence and critical variables
- Database connectivity
- Directory structure
- Missing files or configurations
- AI tool context files
- Git configuration

The warnings give actionable advice. If your database is not configured, it tells you exactly what to add to `.env`.

---

10. tina4 test -- Running Tests

```
tina4 test
```

```
Running tests...

  ProductTest
    [PASS] test_create_product
    [PASS] test_load_product

  2 tests, 2 passed, 0 failed (0.12s)
```

This runs all tests in the `tests/` directory. See Chapter 18 for full testing documentation.

Test Options

```
tina4 test --filter ProductTest           # Specific test class
tina4 test --verbose                      # Show assertion details
```

Or use npm:

```
npm test
```

---

11. tina4 routes -- Route Listing

See all registered routes in your project:

```
tina4 routes
```

```
Registered Routes:

  Method  Path                        Handler                     Middleware
  ------  --------------------------  --------------------------  ----------
  GET     /health                     healthCheck                 -
  GET     /api/products               listProducts                ResponseCache:300
  GET     /api/products/{productId}   getProduct                  -
  POST    /api/products               createProduct               authMiddleware
  PUT     /api/products/{productId}   updateProduct               authMiddleware
  DELETE  /api/products/{productId}   deleteProduct               authMiddleware
  GET     /api/auth/login             -                           -
  POST    /api/auth/login             login                       -
  GET     /admin                      adminDashboard              -

  9 routes registered
```

This is useful for verifying that your routes are registered and for finding the handler function for a specific URL.

When debugging routing issues, check here first. If a route does not match, `tina4 routes` shows whether it was registered and what middleware is attached.

---

12. tina4 migrate -- Database Migrations

Run pending migrations:

```
tina4 migrate
```

```
Running migrations...
  [UP] 20260322000100_create_users_table.sql
  [UP] 20260322000200_create_products_table.sql
  [UP] 20260322000300_add_category_to_products.sql

  3 migrations applied
```

Rollback

```
tina4 migrate:rollback
```

```
Rolling back last migration...
  [DOWN] 20260322000300_add_category_to_products.sql
```

```
  1 migration rolled back
```

Migration Table Auto-Upgrade

If your project was created with an early v3 release, the `tina4_migration` tracking table may use the older two-column schema. Running `tina4 migrate` detects the old layout and adds the missing `migration_id`, `batch`, and `executed_at` columns, backfilling existing data. No manual intervention needed.

Status

```
tina4 migrate:status
```

```
Migration Status:

  Status    Migration
  --------  ----------------------------------------
  Applied   20260322000100_create_users_table.sql
  Applied   20260322000200_create_products_table.sql
  Applied   20260322000300_add_category_to_products.sql
  Pending   20260322000400_create_orders_table.sql

  3 applied, 1 pending
```

---

13. tina4 queue -- Queue Management

```
tina4 queue work                        # Start processing jobs
tina4 queue work --queue send-email     # Process specific queue
tina4 queue:dead                        # List dead letter jobs
tina4 queue:retry 42                    # Retry a dead job
tina4 queue:clear --older-than 7d       # Clear old dead jobs
```

---

14. tina4 build -- Production Build

```
tina4 build
```

Compiles TypeScript to JavaScript. Bundles for production. Optimizes:

```
Building for production...
  Compiled 47 TypeScript files
  Output: dist/
  Size: 245KB

Ready for deployment:
  node dist/app.js
```

---

15. Custom CLI Commands

Create custom seed scripts as standalone TypeScript files:

```
// scripts/seed-products.ts
import { Database } from "tina4-nodejs/orm";
```

```
async function seedProducts() {
    const db = await Database.getConnection();

    const products = [
        { name: "Keyboard", price: 79.99 },
        { name: "Mouse", price: 29.99 },
        { name: "Monitor", price: 399.99 }
    ];

    for (const p of products) {
        await db.execute(
            "INSERT INTO products (name, price) VALUES (:name, :price)",
            p
        );
    }

    console.log(`Seeded ${products.length} products`);
}

seedProducts();
```

Run it:

```
npx tsx scripts/seed-products.ts
```

---

16. Exercise: Scaffold a Feature in 5 Commands

Scaffold a complete "Customer" feature from scratch using only CLI commands.

Requirements

Starting from an existing Tina4 Node.js project, run 5 commands to create:

- A Customer ORM model with name, email, phone, and company fields
- A CRUD route file with all five REST endpoints
- A migration that creates the customers table
- Run the migration to create the table
- Run the doctor to verify everything

Expected Commands

```
# 1. Generate the model, its migration, and the CRUD routes
tina4 generate model Customer
tina4 generate route customers

# 2. Edit the model to add fields (manual step)
# Add fields to src/models/Customer.ts

# 3. Edit the migration to add columns (manual step)
# Add columns to the migration file

# 4. Run the migration
tina4 migrate

# 5. Run the doctor to verify everything is set up
tina4 doctor
```

Solution

Command 1: Generate the model, its migration, and the CRUD routes:

```
tina4 generate model Customer
tina4 generate route customers
```

Command 2: Edit `src/models/Customer.ts`:

```
import { BaseModel } from "tina4-nodejs/orm";

export class Customer extends BaseModel {
    static tableName = "customers";
    static primaryKey = "id";

    id!: number;
    name!: string;
    email!: string;
    phone!: string;
    company!: string;
    createdAt!: string;
}
```

Command 3: Edit the migration file:

```
-- UP
CREATE TABLE customers (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    email TEXT NOT NULL UNIQUE,
    phone TEXT,
    company TEXT,
    created_at TEXT DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_customers_email ON customers(email);

-- DOWN
DROP TABLE IF EXISTS customers;
```

Command 4: Run the migration:

```
tina4 migrate
```

```
Running migrations...
  [UP] 20260322120000_create_customers_table.sql

  1 migration applied
```

Command 5: Verify with doctor:

```
tina4 doctor
```

```
  [OK] src/orm/ directory exists (3 model files)
  [OK] src/routes/ directory exists (4 route files)
  [OK] Database connection: sqlite:///data/app.db
  ...
```

Now test the API:

```
curl -X POST http://localhost:7148/api/customers \
  -H "Content-Type: application/json" \
  -d '{"name": "Alice Corp", "email": "alice@corp.com", "phone": "+1-555-0100", "company": "Alic
```

```
{
  "id": 1,
  "name": "Alice Corp",
  "email": "alice@corp.com",
  "phone": "+1-555-0100",
  "company": "Alice Corp",
  "created_at": "2026-03-22 12:00:00"
}
```

From zero to a working CRUD API. Five commands. Under two minutes.

---

17. Gotchas

1. tina4 Command Not Found

Problem: Running `tina4` gives "command not found".

Cause: The Tina4 CLI is not installed or not in your PATH.

Fix: Install the CLI: `curl -fsSL https://tina4.com/install.sh | sh`. Verify with `tina4 --version`. If installed but not found, add the installation directory to your PATH:

```
echo 'export PATH="$HOME/.tina4/bin:$PATH"' >> ~/.zshrc
source ~/.zshrc
```

2. Wrong Language Detected

Problem: `tina4 init` creates a PHP project instead of Node.js.

Cause: A `composer.json` file exists in the directory from a previous project.

Fix: Use explicit language selection: `tina4 init nodejs my-project`. Or delete the conflicting language file before running `init`.

3. Generated Files Overwrite Existing Code

Problem: Running `tina4 generate route products` overwrites your custom route file.

Cause: The generate command creates files at fixed paths. If the file exists, it is overwritten.

Fix: The CLI warns you before overwriting. Check if the file exists first. If you need to regenerate, rename the existing file: `mv src/routes/products.ts src/routes/products_backup.ts`.

4. Migration Order Issues

Problem: A migration fails because it references a table that does not exist yet.

Cause: Migration files run in alphabetical (timestamp) order. If migration B depends on a table created by migration A, but A has a later timestamp, B runs first and fails.

Fix: Use consistent timestamps. The `tina4 generate migration` command uses the current timestamp. Generating migrations in order ensures correct execution order. If you need to fix ordering, rename the migration files to adjust their timestamps.

5. tina4 serve Uses Wrong Port

Problem: `tina4 serve` starts on port 7148 but you need port 8080.

Cause: The default port is 7148 unless overridden.

Fix: Set it in `.env`: `TINA4_PORT=8080`. Or pass it as a flag: `tina4 serve --port 8080`. The `.env` value takes precedence over the default. The command-line flag overrides everything.

6. Doctor Shows False Warnings

Problem: `tina4 doctor` warns about missing migrations, but your project does not use migrations.

Cause: Doctor checks for common conventions. It warns about missing migrations regardless of your approach.

Fix: These are warnings, not errors. Ignore warnings that do not apply to your project. Doctor is a guide, not a gatekeeper.

7. Model Name Must Be PascalCase

Problem: `tina4 generate model order_item` creates a model class named `order_item` which is not valid TypeScript convention.

Cause: The CLI uses the argument as-is for the class name.

Fix: Use PascalCase for model names: `tina4 generate model OrderItem`. The CLI converts it to snake_case for the table name (`order_items`).

8. Build Fails

Fix: Check TypeScript errors with `npx tsc --noEmit`.

9. Port Conflict

Fix: Use `tina4 serve --port 8080` or set `TINA4_PORT` in `.env`.

---

18. Documentation

```
tina4 docs      # Download framework-specific book chapters to .tina4-docs/
tina4 books     # Download the complete Tina4 book (all languages) to tina4-book/
```

`tina4 docs` detects your project language and downloads only the relevant chapters. The documentation is available in Markdown format, optimised for AI tools and local reference.

---

19. Test Port (Dual-Port Development)

When `TINA4_DEBUG=true`, Tina4 automatically starts a second HTTP server on `port + 1000`:

- **Main port** (e.g. 7148) - hot-reload enabled, for AI dev tools
- **Test port** (e.g. 8148) - stable, no hot-reload, for user testing

This prevents the browser from refreshing mid-test when AI tools edit files.

Setting	Effect
`TINA4_NO_AI_PORT=true`	Disables the test port entirely
`TINA4_NO_RELOAD=true`	Disables hot-reload on the main port too
`--no-reload`	CLI flag equivalent of TINA4*NO*RELOAD

Vibe Coding with AI

Why Tina4 is Built for AI-Assisted Development

Tell your AI assistant: "Add a product catalog with search, pagination, and category filtering." In most frameworks, the AI needs to know which packages to install, which config files to create, which naming conventions to follow, and how to wire everything together. It hallucinates half of it.

With Tina4, the AI reads one file -- `CLAUDE.md` -- and knows everything. Every method signature. Every import path. Every convention. It generates correct, runnable code on the first try because there is only one way to do things in Tina4.

This is not accidental. Tina4 was built from the ground up as the best framework for AI-assisted development.

This chapter explains coding assistants and the `tina4 ai` skills installer. To call a model from your application, see Chapter 39: AI Client.

The Zero-Dependency Advantage

When an AI generates code for Express, it might suggest `express-session` or `cookie-session` or `connect-redis`. All exist. All work differently. The AI guesses.

With Tina4, there is one cache. One queue. One ORM. One template engine. No ambiguity. No alternatives. No "it depends on which package you installed."

```
// There is only one way to cache in Tina4
const data = await cacheGet("products");

// There is only one way to queue in Tina4
const queue = new Queue({ topic: "emails" });
queue.push({ to: "user@test.com" });

// There is only one way to send email in Tina4
const mail = new Messenger();
await mail.send({ to: "user@test.com", subject: "Welcome", body: "<h1>Welcome!</h1>", html: true
```

Zero dependencies means zero confusion for AI.

CLAUDE.md -- The AI's Instruction Manual

Every Tina4 project includes a `CLAUDE.md` file that tells AI assistants exactly how the framework works:

- Every method signature with parameters and return types
- The project structure convention (`src/routes/`, `src/orm/`, `src/templates/`)
- The `.env` variable reference

- Code style rules (routes use `async (req, res) =>`, ORM uses `toDict()`)
- Database connection string formats
- Common gotchas and how to avoid them

When you open a Tina4 project in Claude Code, Cursor, or GitHub Copilot -- the AI reads this file first and immediately understands how to write correct Tina4 code.

Skills -- Deep Framework Knowledge

Beyond CLAUDE.md, Tina4 ships with three AI skill files:

tina4-maintainer

For framework maintenance -- porting features between languages, reviewing PRs, running benchmarks, checking parity.

tina4-developer

For application development -- creating routes, defining models, writing templates, setting up auth, configuring queues.

tina4-js

For frontend development -- tina4-js signals, Tina4Element components, reactive templates, WebSocket client.

The Convention Advantage

AI thrives on convention. Every Tina4 project follows the same structure. The AI never asks "where should I put this?"

```
src/
  routes/hello.ts       → AI knows: this is a route file
  orm/Product.ts        → AI knows: this is an ORM model
  templates/home.twig   → AI knows: this is a template
```

Tell the AI "add a User model" and it creates `src/orm/User.ts` with the correct class definition, the correct imports, and the correct property types. Every time.

Real-World Vibe Coding Session

Here is an actual conversation with Claude Code in a Tina4 Node.js project:

You: "Add a blog with posts and comments. Posts belong to users. Comments belong to posts. I want a REST API and a template page that lists posts."

Claude Code generates:

- `src/orm/Post.ts` -- Model with belongsTo User, hasMany Comments
- `src/orm/Comment.ts` -- Model with belongsTo Post
- `src/routes/api/posts.ts` -- Full CRUD API
- `src/routes/api/comments.ts` -- Nested comments API
- `src/routes/blog.ts` -- Template route for the blog page
- `src/templates/blog.twig` -- Page with tina4css cards for each post
- `migrations/20260322_create_posts_and_comments.sql` -- Database schema

All correct. All runnable. First try.

You: "Add pagination to the posts list"

Claude adds `req.query.page` handling and proper offset calculation -- because CLAUDE.md told it exactly how Tina4 pagination works.

You: "Cache the post list for 5 minutes"

Claude adds `cacheGet("posts_page_" + page)` with `cacheSet("posts_page_" + page, data, 300)` -- because it knows the cache API.

You: "Send an email when a new comment is posted"

Claude adds `const mail = new Messenger(); await mail.send(...)` -- because it knows Messenger reads from `.env`.

No research. No Stack Overflow. No "which package should I use?" Describe what you want. The AI builds it.

TypeScript-Specific Advantages

TypeScript makes AI coding even more reliable:

```
// The AI knows the exact types
import { Router, Auth, Queue, Messenger, cacheGet, cacheSet } from "tina4-nodejs";
import { BaseModel, Database } from "tina4-nodejs/orm";

// Type declarations guide the AI
export class Product extends BaseModel {
    static tableName = "products";
    static primaryKey = "id";
    static hasMany = [{ model: "Review", foreignKey: "product_id" }];

    id!: number;
    name!: string;
    price: number = 0;
    inStock: boolean = true;
}

// The AI generates type-safe route handlers
Router.get("/api/products/{id:int}", async (req, res) => {
    const product = new Product();
```

```
    await product.load(req.params.id);
    return product.id ? res.json(product.toDict()) : res.status(404).json({ error: "Not found" }
});
```

TypeScript's type system is a guardrail. It prevents the AI from generating code that fails at runtime. Property types, method signatures, and return types steer the AI toward correct code.

Supported AI Tools

Tina4 auto-detects and installs context for eight tools. The exact list lives in `@tina4/core` (the AI module):

Tool	Detection (context file)	Context installed
Claude Code	`CLAUDE.md`	CLAUDE.md + `.claude/` skills
Cursor	`.cursorules`	`.cursorules` (+ `.cursor/`)
GitHub Copilot	`.github/copilot-instructions.md`	`.github/copilot-instructions.md`
Windsurf	`.windsurfrules`	`.windsurfrules`
Aider	`CONVENTIONS.md`	`CONVENTIONS.md`
Cline	`.clinerules`	`.clinerules`
OpenAI Codex	`AGENTS.md`	`AGENTS.md`
Google Antigravity	`.antigravity/context.md`	`.antigravity/context.md`

Run `tina4 ai` to see which tools are detected and install context files via the menu.

The AI Chat in Dev Dashboard

The dev dashboard at `/__dev` includes an AI chat tab. By default it talks to a local **qwen2.5-coder** model served via Ollama, grounded by Tina4's built-in RAG index of the framework source. Nothing leaves your machine.

Configure it in `.env`:

```
TINA4_AI_URL=http://localhost:11434      # Ollama HTTP endpoint
TINA4_AI_MODEL=qwen2.5-coder             # Default coding model
TINA4_RAG_URL=http://localhost:11434     # RAG embedding endpoint (defaults to TINA4_AI_URL)
```

If you prefer a remote provider, point `TINA4_AI_URL` at any OpenAI-compatible endpoint and set `TINA4_AI_MODEL` accordingly.

The AI has full context of your Tina4 project. Ask it anything:

- "Why is my /api/users route returning 500?"
- "How do I add WebSocket support?"
- "Write a migration to add an avatar column to users"

The "Ask about this error" button on the error overlay sends the full stack trace and source code to the AI for instant debugging.

Tips for Effective Vibe Coding with Tina4

- **Be specific about what you want** -- "Add a product API with search by name and price range" works better than "add products"
- **Let the AI use tina4 generate** -- "Run tina4 generate model Product then add price and category fields"
- **Reference the .env** -- "Configure the queue to use RabbitMQ" -- the AI knows to update .env
- **Ask for tests** -- "Write tests for the Product API" -- the AI uses Tina4's inline testing
- **Review, don't rewrite** -- The AI's first attempt is usually 90% correct. Tweak the details.

Exercise

Open your Tina4 Node.js project in Claude Code (or your preferred AI tool) and try these prompts:

- "Add a Contact model with name, email, phone, and message fields"
- "Create a contact form page at /contact with a Twig template using tina4css"
- "Add an API endpoint that accepts the form submission and saves it to the database"
- "Send an email notification when a new contact is submitted"
- "Add rate limiting to the contact form -- max 3 submissions per minute"

Each prompt should generate correct, runnable TypeScript code on the first try. If it does not, check that CLAUDE.md is in your project root.

Gotchas

- **No CLAUDE.md?** Run `tina4 init` or `tina4 doctor` to generate it
- **AI suggests wrong import paths?** The CLAUDE.md might be outdated -- regenerate it
- **AI hallucinates a package?** Remind it: "Tina4 has zero dependencies, use only built-in features"
- **AI creates files in wrong location?** Tell it: "Follow the src/routes, src/orm, src/templates convention"

- **AI code doesn't run?** Check that routes return `res.json()` not just `console.log()` -- Tina4 convention matters
- **AI uses Express patterns?** Remind it: "This is Tina4, not Express. Import from tina4-nodejs."
- **AI forgets await?** All Tina4 database and ORM methods are async and need await

---

The Philosophy

Tina4 was not retrofitted for AI coding tools. It was designed for them from the ground up.

Convention over configuration means the AI knows where things go. Zero dependencies means the AI never chooses between packages. A single CLAUDE.md file means the AI has complete framework knowledge. Identical APIs across 4 languages means that knowledge transfers instantly.

You describe the intent. The AI writes the code. You review and refine. Tina4 is the ground the partnership stands on.

The Intelligent Native Application 4ramework. Built for AI. Built for you.

Environment Variables

> **■■ *BREAKING CHANGE: Tina4 v3.12.0***
>
> *Every framework env var now requires the `TINA4_` prefix. The legacy un-prefixed names (`DATABASE_URL`, `SECRET`, `SMTP_HOST`, `HOST_NAME`, etc.) no longer work. Setting them at startup makes the framework refuse to boot with a list of renames.*
>
> *Run `tina4 env --migrate` to rewrite your existing `.env` automatically, or rename manually using the table below. The runtime guard prints the same mapping if it detects legacy names.*
>
> ***Conventional names stay un-prefixed:*** `PORT`, `HOST`, `NODE_ENV`, `RACK_ENV`, `RUBY_ENV`, `ENVIRONMENT`. *These are runtime/PaaS conventions, not framework config.*

Tina4 Node.js is configured through environment variables, read from `.env` at the project root. Every variable has a sensible default; most projects set three or four values and leave the rest alone.

This chapter lists every variable the Node.js framework reads, grouped by subsystem. Start with the minimum-config examples at the end, then come back here when you need to tune something specific.

Core Server

Variable	Default	Description
`HOST`	`0.0.0.0`	Bind address. `0.0.0.0` listens on every interface. `127.0.0.1` restricts to localhost.
`TINA4_HOST`	`0.0.0.0`	Framework-prefixed bind address. Takes precedence over `HOST` when both are set.
`PORT`	`7148`	HTTP server port. The Rust CLI prefers `TINA4_PORT` but falls back to `PORT`.
`TINA4_PORT`	*(inherits `PORT`)*	Explicit Tina4-specific port override. Takes precedence over `PORT` when both are set.
`TINA4_HOST_NAME`	`localhost:7148`	Fully-qualified host used in generated absolute URLs (Swagger, OAuth redirects, emails).
`TINA4_DEBUG`	`false`	Master debug toggle. Enables Swagger UI, dev dashboard, live reload, template dump filter, error overlay. Never set to `true` in production.
`TINA4_SUPPRESS`	`false`	Suppresses the boot banner on startup. Useful for clean CI logs and embedded usage.
`TINA4_ENV_FILE`	`.env`	Path to the dotenv file loaded at startup. Relative paths resolve from the project root.
`TINA4_HEALTH_PATH`	`/__health`	Path for the built-in health endpoint. Legacy `/health` alias is also registered when this points at `/__health`.
`TINA4_NO_BROWSER`	`false`	Stops `tina4 serve` from opening your browser on every restart.

`TINA4_NO_RELOAD`	`false`	Disables the dev hot-reload signal from the Rust CLI. Use when you want a stable server for debugging.
`TINA4_PRODUCTION`	`false`	Forces production-mode startup (cluster mode, debug overlays disabled). Set automatically by `tina4 serve --production`.
`TINA4_ALLOW_LEGACY_ENV`	`false`	Bypass the v3.12 env-prefix guard. CI / migration scripts only, never enable in production.

Secrets and Authentication

Variable	Default	Description
`TINA4_SECRET`	*(empty)*	JWT signing secret. Must be long, random, and unique per environment. **Never commit.**
`TINA4_JWT_ALGORITHM`	`HS256`	JWT signing algorithm. Supports `HS256`, `HS384`, `HS512`.
`TINA4_TOKEN_LIMIT`	`60`	JWT token lifetime in minutes.
`TINA4_API_KEY`	*(empty)*	Static API key used as a fallback to JWT.

Database

Variable	Default	Description
`TINA4_DATABASE_URL`	*(required)*	Connection URL. Scheme selects the driver: `sqlite`, `postgres`, `mysql`.
`TINA4_DATABASE_USERNAME`	*(empty)*	Overrides the username embedded in `TINA4_DATABASE_URL`.
`TINA4_DATABASE_PASSWORD`	*(empty)*	Overrides the password embedded in `TINA4_DATABASE_URL`.
`TINA4_DATABASE_FIREBIRD_PATH`	*(empty)*	Overrides the database path/alias parsed from `TINA4_DATABASE_URL` for Firebird. Useful for Windows backslash paths and split-config setups.
`TINA4_AUTOCOMMIT`	`true`	Standalone writes auto-commit on their own connection (durable + visible across a pool); explicit transactions stay atomic. Set `false` for strict manual-commit mode.
`TINA4_DB_CACHE`	`false`	Enables in-memory query-result caching for read queries.
`TINA4_DB_CACHE_TTL`	`30`	Query cache TTL in seconds when `TINA4_DB_CACHE=true`.
`TINA4_DB_POOL`	`0`	Connection-pool size override. Applied to the database adapter's `pool` config when not set explicitly in code. `0` leaves the adapter default in place.
`TINA4_ORM_PLURAL_TABLE_NAMES`	`true`	When `true`, the ORM pluralises class names into table names (`User` → `users`).

CORS

Variable	Default	Description
`TINA4_CORS_ORIGINS`	*(empty)*	Comma-separated allowed origins. Empty denies every cross-origin request; set the origins your front end is served from. `*` allows any origin and has to be set explicitly.
`TINA4_CORS_METHODS`	`GET,POST,PUT,PATCH,DELETE,OPTIONS`	Comma-separated list of allowed methods on preflight responses.
`TINA4_CORS_HEADERS`	`Content-Type,Authorization`	Comma-separated list of allowed request headers on preflight responses.
`TINA4_CORS_MAX_AGE`	`86400`	Preflight cache lifetime in seconds.
`TINA4_CORS_CREDENTIALS`	`true`	Sets `Access-Control-Allow-Credentials`.

Security Headers

Variable	Default	Description
`TINA4_CSP`	`default-src 'self'`	`Content-Security-Policy` header.
`TINA4_CSRF`	`false`	CSRF token validation on POST/PUT/PATCH/DELETE.
`TINA4_HSTS`	*(empty/off)*	`Strict-Transport-Security` max-age in seconds. Set `31536000` in production with HTTPS.
`TINA4_FRAME_OPTIONS`	`SAMEORIGIN`	`X-Frame-Options` header.
`TINA4_REFERRER_POLICY`	`strict-origin-when-cross-origin`	`Referrer-Policy` header.
`TINA4_PERMISSIONS_POLICY`	`camera=(), microphone=(), geolocation=()`	`Permissions-Policy` header.

Rate Limiting

Variable	Default	Description
`TINA4_RATE_LIMIT`	`100`	Maximum requests per window per IP. Set `0` to disable.
`TINA4_RATE_WINDOW`	`60`	Rate-limit window in seconds.

Sessions

Variable	Default	Description
`TINA4_SESSION_BACKEND`	`file`	Storage backend. Options: `file`, `redis`, `valkey`, `mongo`, `database`.
`TINA4_SESSION_NAME`	`tina4_session`	Session cookie name.
`TINA4_SESSION_TTL`	`1800`	Session expiry in seconds (30 minutes).
`TINA4_SESSION_SAMESITE`	`Lax`	SameSite cookie attribute. Options: `Strict`, `Lax`, `None`.
`TINA4_SESSION_HTTPONLY`	`true`	Emit the `HttpOnly` flag on the session cookie.
`TINA4_SESSION_SECURE`	`false`	Emit the `Secure` flag on the session cookie. Enable behind HTTPS.
`TINA4_SESSION_PATH`	`data/sessions`	Filesystem path for the file backend.

Redis/Valkey session backend

Variable	Default	Description
`TINA4_SESSION_REDIS_URL`	*(none)*	Full `redis://` URL. Overrides host/port when set.
`TINA4_SESSION_REDIS_HOST`	`127.0.0.1`	Redis host.
`TINA4_SESSION_REDIS_PORT`	`6379`	Redis port.
`TINA4_SESSION_REDIS_PASSWORD`	*(none)*	Redis auth password.
`TINA4_SESSION_REDIS_DB`	`0`	Redis database number.
`TINA4_SESSION_REDIS_PREFIX`	`tina4:session:`	Key prefix for stored sessions.
`TINA4_SESSION_VALKEY_HOST`	`127.0.0.1`	Valkey host.
`TINA4_SESSION_VALKEY_PORT`	`6379`	Valkey port.
`TINA4_SESSION_VALKEY_PASSWORD`	*(none)*	Valkey auth password.
`TINA4_SESSION_VALKEY_DB`	`0`	Valkey database number.
`TINA4_SESSION_VALKEY_PREFIX`	`tina4:session:`	Key prefix for stored sessions.

MongoDB session backend

Variable	Default	Description
`TINA4_SESSION_MONGO_URI`	*(none)*	Full MongoDB connection string. Overrides host/port when set.
`TINA4_SESSION_MONGO_HOST`	`127.0.0.1`	MongoDB host.
`TINA4_SESSION_MONGO_PORT`	`27017`	MongoDB port.
`TINA4_SESSION_MONGO_USERNAME`	*(none)*	MongoDB username.
`TINA4_SESSION_MONGO_PASSWORD`	*(none)*	MongoDB password.
`TINA4_SESSION_MONGO_DB`	`tina4_sessions`	MongoDB database name.
`TINA4_SESSION_MONGO_COLLECTION`	`sessions`	MongoDB collection name.

Cache

Variable	Default	Description
`TINA4_CACHE_BACKEND`	`memory`	Response cache backend. Options: `memory`, `file`, `redis`, `valkey`, `memcached`, `mongodb`, `database`. Falls back to `file` if the configured backend is unreachable.
`TINA4_CACHE_DIR`	`data/cache`	Cache directory for the file backend.
`TINA4_CACHE_TTL`	`60`	Default cache TTL in seconds.
`TINA4_CACHE_MAX_ENTRIES`	`1000`	Maximum cache entries.
`TINA4_CACHE_URL`	`redis://localhost:6379`	Connection URL for remote cache backends. For `database`, falls back to `TINA4_DATABASE_URL` when unset.
`TINA4_CACHE_USERNAME`	*(none)*	Username for the cache backend. May also be embedded in `TINA4_CACHE_URL`.
`TINA4_CACHE_PASSWORD`	*(none)*	Password for the cache backend. May also be embedded in `TINA4_CACHE_URL` (e.g. `redis://:pass@host`). Memcached is unauthenticated.

Queues

Variable	Default	Description
`TINA4_QUEUE_BACKEND`	`file`	Queue backend. Options: `file`, `rabbitmq`, `kafka`, `mongodb`.
`TINA4_QUEUE_PATH`	`data/queue`	Filesystem path for the file backend.
`TINA4_QUEUE_URL`	*(none)*	Connection URL for remote backends (rabbitmq/kafka).

Kafka queue backend

Variable	Default	Description
`TINA4_KAFKA_BROKERS`	`localhost:9092`	Comma-separated broker list.
`TINA4_KAFKA_GROUP_ID`	`tina4_consumer_group`	Kafka consumer group ID.

RabbitMQ queue backend

Variable	Default	Description
`TINA4_RABBITMQ_HOST`	`localhost`	RabbitMQ host.
`TINA4_RABBITMQ_PORT`	`5672`	RabbitMQ port.
`TINA4_RABBITMQ_USERNAME`	`guest`	RabbitMQ username.
`TINA4_RABBITMQ_PASSWORD`	`guest`	RabbitMQ password.
`TINA4_RABBITMQ_VHOST`	`/`	RabbitMQ virtual host.

MongoDB queue backend

Variable	Default	Description
`TINA4_MONGO_URI`	*(none)*	Full MongoDB connection string. Overrides host/port when set.
`TINA4_MONGO_HOST`	`localhost`	MongoDB host.
`TINA4_MONGO_PORT`	`27017`	MongoDB port.
`TINA4_MONGO_USERNAME`	*(none)*	MongoDB username.
`TINA4_MONGO_PASSWORD`	*(none)*	MongoDB password.
`TINA4_MONGO_DB`	`tina4`	MongoDB database name.
`TINA4_MONGO_COLLECTION`	`tina4_queue`	MongoDB collection name for jobs.

WebSocket

Variable	Default	Description
`TINA4_WS_PORT`	`8080`	Default WebSocket server port when one isn't passed to the constructor.
`TINA4_WS_BACKPLANE`	*(none)*	Backplane type for multi-instance broadcasts. Options: `redis`, `nats`.
`TINA4_WS_BACKPLANE_URL`	`redis://localhost:6379` (redis) / `nats://localhost:4222` (nats)	Connection URL for the chosen backplane.

Email

Variable	Default	Description
`TINA4_MAIL_HOST`	*(none)*	SMTP server hostname.
`TINA4_MAIL_PORT`	`587`	SMTP server port.
`TINA4_MAIL_USERNAME`	*(none)*	SMTP authentication username.
`TINA4_MAIL_PASSWORD`	*(none)*	SMTP authentication password.
`TINA4_MAIL_FROM`	*(none)*	Default sender email address.
`TINA4_MAIL_FROM_NAME`	*(none)*	Default sender display name.
`TINA4_MAIL_ENCRYPTION`	`tls`	Connection encryption. Options: `tls`, `ssl`, `none`.
`TINA4_MAIL_IMAP_HOST`	*(none)*	IMAP server for inbound mail.
`TINA4_MAIL_IMAP_PORT`	`993`	IMAP server port.
`TINA4_MAIL_IMAP_ENCRYPTION`	`tls`	IMAP transport security. Options: `tls`, `starttls`, `none`. Node also accepts `ssl` as an alias for `tls` for back-compat.
`TINA4_MAIL_CAPTURE`	`false`	Force local DevMailbox capture and skip SMTP, even when an SMTP host exists. A missing host also captures.
`TINA4_MAIL_REDIRECT_TO`	*(none)*	Comma-separated safety recipients for real SMTP delivery. Replaces To, Cc, and Bcc.
`TINA4_MAILBOX_DIR`	`data/mailbox`	Storage directory for captured mail. This setting does not enable or disable capture.

> `TINA4_DEBUG` *does not suppress email. On staging, set* `TINA4_MAIL_CAPTURE=true` *to keep every message local, or set* `TINA4_MAIL_REDIRECT_TO=qa@example.com,product@example.com` *to exercise SMTP without reaching original recipients. Capture takes precedence when both are set.*

> `TINA4_MAIL_HOST`, `TINA4_MAIL_PORT`, `TINA4_MAIL_USERNAME`, `TINA4_MAIL_PASSWORD`, `TINA4_MAIL_FROM`, `TINA4_MAIL_FROM_NAME`, `TINA4_MAIL_IMAP_HOST`, `TINA4_MAIL_IMAP_PORT`, `TINA4_MAIL_IMAP_USERNAME`, `TINA4_MAIL_IMAP_PASSWORD` *are accepted as legacy aliases. New projects should use the* `TINA4_MAIL_*` *names.*

Logging

Logs default to stdout. Set `TINA4_LOG_OUTPUT=file` plus `TINA4_LOG_FILE=app.log` to write to disk; the framework rotates at `TINA4_LOG_ROTATE_SIZE` bytes and keeps `TINA4_LOG_ROTATE_KEEP` backups using stdlib `fs` calls.

Variable	Default	Description
`TINA4_LOG_LEVEL`	`INFO`	Console severity threshold. Options: `ALL`, `DEBUG`, `INFO`, `WARNING`, `ERROR`, `CRITICAL`, `NONE`.
`TINA4_LOG_FILE_LEVEL`	`ALL`	Independent severity threshold for the file sink.
`TINA4_LOG_STRICT`	`false`	Raise when a configured log sink fails instead of continuing with the remaining sinks. Accepts only `true` or `false`.
`TINA4_LOG_MAX_SIZE`	`10`	Per-file log size limit in megabytes. Rotated when exceeded.
`TINA4_LOG_KEEP`	`5`	Number of rotated log files to retain.

File logging

Variable	Default	Description
`TINA4_LOG_FILE`	*(empty = stdout only)*	Explicit log file path. Absolute paths are used as-is; relative paths resolve against `TINA4_LOG_DIR`.
`TINA4_LOG_DIR`	`logs`	Directory for log files when `TINA4_LOG_FILE` is relative or unset.
`TINA4_LOG_FORMAT`	`text`	File log format. Options: `text`, `json`.
`TINA4_LOG_OUTPUT`	`stdout`	Where to write logs. Options: `stdout`, `file`, `both`.
`TINA4_LOG_CRITICAL`	`false`	Enable the `CRITICAL` level shortcut. Off by default to keep the log surface compact.
`TINA4_LOG_ROTATE_SIZE`	`10485760`	Rotate the file when it reaches this many bytes (10 MB). `0` disables rotation. Node uses an atomic `fs.renameSync` shift on each write.
`TINA4_LOG_ROTATE_KEEP`	`5`	Number of rotated historical files to retain.

Localisation

Variable	Default	Description
`TINA4_LOCALE`	`en`	Default locale for the `I18n` module.
`TINA4_LOCALE_DIR`	`src/locales`	Directory containing locale JSON files.

Uploads

Variable	Default	Description
`TINA4_MAX_UPLOAD_SIZE`	`10485760`	Maximum multipart upload size in bytes (10 MB).

Services (background tasks)

Variable	Default	Description
`TINA4_SERVICE_DIR`	`src/services`	Directory scanned for service classes.
`TINA4_SERVICE_SLEEP`	`5`	Seconds the service runner sleeps between empty polls.

Routing

Variable	Default	Description
`TINA4_TEMPLATE_ROUTING`	`on`	Auto-route any request without a matching handler to a same-named Frond template. Set to `off`, `false`, `0`, `no`, or `disabled` for explicit-only routing.
`TINA4_TRAILING_SLASH_REDIRECT`	`false`	When `true`, requests with a trailing slash are 301-redirected to the canonical no-trailing-slash form.

Templates

Variable	Default	Description
`TINA4_TEMPLATE_CACHE_TTL`	`0`	Frond template cache TTL in seconds. `0` (the default) caches permanently for the process lifetime.

GraphQL

Variable	Default	Description
`TINA4_GRAPHQL_AUTO_SCHEMA`	`true`	Auto-build a schema from registered ORM models when the dev server starts. Set `false` to require an explicit schema.
`TINA4_GRAPHQL_ENDPOINT`	`/graphql`	Path the GraphQL handler is mounted on.

Application AI Client and Developer AI Tools

The app-facing `Ai` client and the developer dashboard share the `TINA4_AI_*` namespace. The client supports local OpenAI-compatible servers, OpenAI, and Anthropic. See Chapter 39: AI Client for code examples.

Variable	Default	Description
`TINA4_AI_PROVIDER`	`local`	Provider used by the app-facing client: `local`, `openai`, or `anthropic`.
`TINA4_AI_URL`	Provider-specific	Base URL or full endpoint. Local defaults to `http://localhost:11437`; OpenAI and Anthropic use their public API bases.
`TINA4_AI_MODEL`	Provider-specific	Default model. Local uses `llama3.2`, OpenAI uses `gpt-4o-mini`, and Anthropic uses `claude-3-5-haiku-latest`.
`TINA4_AI_KEY`	*(none)*	Required before an OpenAI or Anthropic request. Local requests need no key.
`TINA4_AI_TIMEOUT`	`60`	Total request deadline in seconds, including retries and response reads.

`TINA4_AI_CONNECT_TIMEOUT`	`10`	Connection-establishment deadline in seconds.
`TINA4_AI_MAX_RETRIES`	`2`	Retries before output for connection failures, HTTP 429, and HTTP 5xx.
`TINA4_EMBED_URL`	*(inherits* `TINA4_AI_URL`*)*	Optional embedding base URL or full endpoint.
`TINA4_RAG_URL`	`http://localhost:11438`	RAG service used by the developer dashboard.
`TINA4_RAG_TOPK`	`4`	Number of RAG matches returned per query.
`TINA4_VISION_URL`	`http://localhost:11437/api/chat`	Vision endpoint used by developer tools, not the app-facing AI client.
`TINA4_IMAGE_URL`	`http://localhost:11437/api/generate`	Image endpoint used by developer tools, not the app-facing AI client.
`TINA4_SUPERVISOR_URL`	`http://localhost:9999`	Rust supervisor URL used by developer tools.
`TINA4_MCP_REMOTE`	`false`	Allows MCP to bind beyond localhost. Do not enable it on a public production interface.
`TINA4_NO_AI_PORT`	`false`	Disables the developer AI port listener.
`TINA4_OVERRIDE_CLIENT`	`false`	Lets the framework start without the Rust client in containers and CI.

MCP

Variable	Default	Description
`TINA4_MCP`	*(inherits `TINA4_DEBUG`)*	Enables the MCP server. Defaults to `true` when `TINA4_DEBUG=true`, `false` otherwise.
`TINA4_MCP_PORT`	*(main port + 2000)*	Port the MCP server listens on. Defaults to the framework HTTP port plus 2000.

Swagger / OpenAPI

Variable	Default	Description
`TINA4_SWAGGER_TITLE`	`Tina4 API`	OpenAPI spec title.
`TINA4_SWAGGER_DESCRIPTION`	`Auto-generated API documentation`	OpenAPI spec description.
`TINA4_SWAGGER_VERSION`	`0.0.1`	OpenAPI spec version.
`TINA4_SWAGGER_CONTACT_EMAIL`	*(empty)*	Contact email written into the generated OpenAPI `info.contact` block.
`TINA4_SWAGGER_LICENSE`	*(empty)*	License name written into the generated OpenAPI `info.license` block.
`TINA4_SWAGGER_ENABLED`	*(inherits `TINA4_DEBUG`)*	Mount the Swagger UI. Defaults to enabled when `TINA4_DEBUG=true`. Force `true`/`false` to override.

Configuration Recipes

The tables above list every knob. These are the setups most apps actually reach for, ready to paste into `.env`. Each block sets only what the feature needs. Everything else keeps its default.

PostgreSQL in production

One URL points the ORM, the migrations, and the query builder at Postgres. Credentials can ride in the URL or sit in their own variables, which keeps the password out of your shell history.

```
TINA4_DATABASE_URL=postgresql://localhost:5432/myapp
TINA4_DATABASE_USERNAME=myapp
TINA4_DATABASE_PASSWORD=changeme
TINA4_DB_POOL=4
```

`TINA4_DB_POOL=4` opens four connections and rotates across them. Leave it at `0` for a single connection on a small app.

A shared cache on Redis

The response cache and the cross-request query cache both speak to the same Redis. Point them at it and every instance shares one cache, invalidated globally on every write.

```
TINA4_CACHE_BACKEND=redis
TINA4_CACHE_URL=redis://localhost:6379
TINA4_DB_CACHE=true
TINA4_DB_CACHE_BACKEND=redis
TINA4_DB_CACHE_URL=redis://localhost:6379
```

If Redis is down or the driver is missing, the cache logs a warning and falls back to the file backend. It never silently stops caching.

Sessions that survive more than one box

The file backend is fine for a single server. Move sessions to Redis the moment you run more than one instance, so a user stays logged in whichever instance answers the next request.

```
TINA4_SESSION_BACKEND=redis
TINA4_SESSION_REDIS_HOST=localhost
TINA4_SESSION_REDIS_PORT=6379
TINA4_SESSION_SECURE=true
TINA4_SESSION_SAMESITE=Strict
```

`TINA4_SESSION_SECURE=true` keeps the cookie off plain HTTP. Turn it on once you have TLS.

A queue on RabbitMQ or Kafka

One URL is enough for RabbitMQ; the per-field variables only exist for split configs. The queue API stays identical whichever backend you pick.

```
TINA4_QUEUE_BACKEND=rabbitmq
TINA4_QUEUE_URL=amqp://guest:guest@localhost:5672/
```

Kafka reads a broker list instead:

```
TINA4_QUEUE_BACKEND=kafka
TINA4_KAFKA_BROKERS=localhost:9092
TINA4_KAFKA_GROUP_ID=myapp_workers
```

WebSocket broadcasts across instances

A single server broadcasts in memory. Add a backplane and a message sent on one instance reaches clients connected to every other instance.

```
TINA4_WS_BACKPLANE=redis
TINA4_WS_BACKPLANE_URL=redis://localhost:6379
TINA4_WS_ALLOWED_ORIGINS=https://myapp.com
```

Set the origin allow-list in production. Empty allows every origin, which is fine in dev and risky on the public internet.

Locked-down production headers

The defaults are already safe. These four tighten the screws for a public site on HTTPS.

```
TINA4_CORS_ORIGINS=https://myapp.com,https://www.myapp.com
TINA4_HSTS=31536000
TINA4_FRAME_OPTIONS=DENY
TINA4_SESSION_SECURE=true
```

Never pair `TINA4_CORS_CREDENTIALS=true` with `TINA4_CORS_ORIGINS=*`. Name your real origins instead.

The dev dashboard AI, kept local

The dashboard AI talks to a local model through Ollama by default, so nothing leaves your machine. Point the URLs elsewhere only when you run the hosted Tina4 AI services.

```
TINA4_AI_URL=http://localhost:11437/api/chat
TINA4_AI_MODEL=qwen2.5-coder:14b
TINA4_RAG_URL=http://localhost:11438
```

Minimal `.env` for Development

```
TINA4_DEBUG=true
TINA4_LOG_LEVEL=DEBUG
TINA4_NO_BROWSER=true
```

Debug mode lights up the Swagger UI, the dev dashboard, detailed error pages, and live reload. Keeping the browser flag on stops a new tab opening every time you save a file.

Minimal `.env` for Production

```
TINA4_SECRET=your-long-random-secret-here
TINA4_DATABASE_URL=postgresql://user:password@db-host:5432/myapp
TINA4_CORS_ORIGINS=https://myapp.com,https://www.myapp.com
TINA4_HSTS=31536000
TINA4_MAIL_HOST=smtp.example.com
TINA4_MAIL_PORT=587
TINA4_MAIL_USERNAME=noreply@myapp.com
TINA4_MAIL_PASSWORD=your-smtp-password
TINA4_MAIL_FROM=noreply@myapp.com
```

No `TINA4_DEBUG`. It defaults to `false`, which is what you want in production. Set a real secret, a real database, locked-down CORS origins, HSTS, and SMTP credentials if you send email. Everything else has a production-appropriate default.

Feature Module Settings

These settings are read by focused framework modules rather than the server bootstrap. They remain part of the public configuration surface.

Variable	Default	Description
`TINA4_ENV`	`development`	Runtime environment name. Set `production` to disable development-only behaviour.
`TINA4_FEEDBACK_DEV_USER`	*(empty)*	Local-development identity for testing the feedback widget without authentication. Never use this as production authentication.
`TINA4_MCP_URL`	`https://mcp.tina4.com`	Grounding service URL shown and used by the developer admin.
`TINA4_SESSION_MEMCACHED_HOST`	`127.0.0.1`	Memcached host for the session backend.
`TINA4_SESSION_MEMCACHED_PORT`	`11211`	Memcached port for the session backend.
`TINA4_SESSION_MEMCACHED_PREFIX`	`tina4:session:`	Key prefix for Memcached sessions. Give each application a distinct prefix when sharing a server.
`TINA4_SSE_HEARTBEAT`	`15`	Seconds between SSE comment heartbeats. Set `0` or a negative value to disable them.
`TINA4_SWAGGER_CONTACT_TEAM`	*(empty)*	Contact name placed in the generated OpenAPI `info.contact` object.
`TINA4_SWAGGER_CONTACT_URL`	*(empty)*	Contact URL placed in the generated OpenAPI `info.contact` object.
`TINA4_TRUSTED_PROXIES`	*(empty)*	Comma-separated IP addresses and CIDR ranges allowed to supply forwarding headers. Empty trusts no proxy.

Deployment

1. From Development to Production

The app works on `localhost:7148`. Now it needs to run around the clock on a real server. Handle thousands of concurrent users. Survive restarts. Hold steady on memory. The gap between "works on my machine" and "works in production" is where projects stumble.

This chapter covers everything for a production deployment: environment configuration, build process, Docker packaging, health checks, graceful shutdown, SSL/TLS, scaling, and monitoring.

When you run `tina4 init`, the framework generates a production-ready `Dockerfile` and `.dockerignore` in your project root. The Dockerfile uses a multi-stage build: the first stage installs npm dependencies and the second stage copies only the runtime artifacts into a slim image. You do not need to write a Dockerfile from scratch -- the generated one is a solid starting point.

---

2. Production .env Configuration

Development defaults optimize for debugging. Production defaults optimize for performance and security. The first deployment step: configure `.env` for production.

Create a production `.env`:

```
# Core
TINA4_DEBUG=false
TINA4_LOG_LEVEL=WARNING
TINA4_PORT=7148

# Database
TINA4_DATABASE_URL=sqlite:///data/app.db

# Security
TINA4_CORS_ORIGINS=https://yourdomain.com
TINA4_SECRET=your-long-random-secret-at-least-32-characters
TINA4_RATE_LIMIT=120

# Performance
TINA4_TEMPLATE_CACHE_TTL=true
```

Key Differences from Development

Setting	Development	Production	Why
`TINA4_DEBUG`	`true`	`false`	Hides stack traces, disables toolbar
`TINA4_LOG_LEVEL`	`ALL`	`WARNING`	Reduces log noise
`TINA4_CORS_ORIGINS`	Your dev front end (for example `http://localhost:3000`), or empty	Your domain	Only listed origins can call the API from a browser

Sensitive Values

Production secrets never go into version control. The `.env` file is gitignored by default. For deployment, use environment variables from your hosting platform, CI/CD secrets, or a secrets manager.

```
# Docker: pass env vars at runtime
docker run -e TINA4_SECRET=your-secret -e TINA4_DATABASE_URL=sqlite:///data/app.db my-app

# Fly.io: set secrets
fly secrets set TINA4_SECRET=your-secret

# Railway: use the dashboard or CLI
railway variables set TINA4_SECRET=your-secret
```

---

3. Building for Production

```
tina4 build
```

```
Building for production...
  Compiled 47 TypeScript files
  Output: dist/
  Size: 245KB
```

Run in production:

```
node dist/app.js
```

---

4. Docker Deployment

Docker is the most portable deployment path. Your app runs the same way on your laptop, in CI, and on the production server.

Dockerfile

Create `Dockerfile`:

```
FROM node:20-alpine
```

```
WORKDIR /app

# Copy dependency files first (for better layer caching)
COPY package*.json ./
RUN npm ci --only=production

# Copy application code
COPY dist/ ./dist/
COPY src/templates/ ./src/templates/
COPY src/public/ ./src/public/
COPY migrations/ ./migrations/

# Create directories for data and logs
RUN mkdir -p data logs

ENV TINA4_DEBUG=false
ENV TINA4_PORT=7148

EXPOSE 7148

# Health check
HEALTHCHECK --interval=30s --timeout=5s --retries=3 \
  CMD wget -qO- http://localhost:7148/health || exit 1

CMD ["node", "dist/app.js"]
```

.dockerignore

Create `.dockerignore`:

```
.git
.env
node_modules
.claude
data/*.db
logs/*.log
src/
!src/templates/
!src/public/
!migrations/
```

Building and Running

```
# Build the image
tina4 build
docker build -t my-tina4-app .

# Run the container
docker run -d \
  --name my-app \
  -p 7148:7148 \
  -e TINA4_SECRET=your-production-secret \
  -e TINA4_DATABASE_URL=sqlite:///data/app.db \
  -v $(pwd)/data:/app/data \
  -v $(pwd)/logs:/app/logs \
  my-tina4-app
```

Docker Compose

For a complete setup with supporting services, use Docker Compose.

Create `docker-compose.yml`:

```
services:
  app:
    build: .
    ports:
      - "7148:7148"
    environment:
      - TINA4_DEBUG=false
      - TINA4_LOG_LEVEL=WARNING
      - TINA4_SECRET=${TINA4_SECRET}
      - TINA4_DATABASE_URL=sqlite:///data/app.db
      - TINA4_CACHE_BACKEND=redis
      - TINA4_CACHE_URL=redis://redis:6379
    volumes:
      - app-data:/app/data
      - app-logs:/app/logs
    depends_on:
      - redis
    restart: unless-stopped
    healthcheck:
      test: ["CMD", "wget", "-qO-", "http://localhost:7148/health"]
      interval: 30s
      timeout: 5s
      retries: 3

  redis:
    image: redis:7-alpine
    ports:
      - "6379:6379"
    volumes:
      - redis-data:/data
    restart: unless-stopped

volumes:
  app-data:
  app-logs:
  redis-data:
```

```
# Start everything
docker compose up -d

# View logs
docker compose logs -f app

# Stop everything
docker compose down
```

---

5. One Event Loop, and How to Get More

Node serves every request on ONE event loop. That single fact decides how you write handlers and how you deploy.

What "slow" means here

A handler that awaits gives the loop back while it waits, so it blocks nobody. A handler that computes holds the loop, and every other request waits behind it. Measured on a Tina4 Node server, timing a trivial route while a second route was busy:

The other route was	Trivial route answered in
idle	0.032s
awaiting a 2 second timer	0.030s
running a 2 second busy loop	1.575s

Both slow routes took two seconds of wall clock. Only one of them cost anything to anybody else. You cannot tell them apart by timing the slow route, which is why this catches people out.

The server watches for it and says so. When a handler holds the loop past a threshold, you get a warning naming the duration:

```
Event loop blocked for 1204ms. Node serves every request on one loop, so a
handler doing CPU-bound work or synchronous I/O stalls all the others for that
long. Move the work to Tina4's queue, or await it.
```

The threshold is 250ms. Tune it or turn it off:

```
TINA4_LOOP_LAG_WARN_MS=500   # warn at half a second instead
TINA4_LOOP_LAG_WARN_MS=0     # silence it
```

The three fixes

- Push the work to the queue and return. Image resizing, PDF generation,

report building and bulk imports all belong there.

- Await your I/O. `readFileSync` blocks the loop; `await readFile` does not.

`JSON.parse` on a 40MB payload blocks it too.

- Run more processes, so a blocked worker stalls only its own share.

Cluster mode

Cluster mode forks one worker per CPU core. The primary binds the port once and hands the socket to the workers, so they share it. The `--production` flag turns it on:

```
tina4 serve --production --no-browser
```

That is what the generated Dockerfile already runs, so a `tina4 deploy docker` image clusters with no extra configuration. The flag works by setting `TINA4_PRODUCTION=true`, which you can also set yourself when you start the app some other way:

```
TINA4_PRODUCTION=true node app.js
```

The banner tells you it took:

```
  Server:    http://localhost:7148 (cluster, 8 workers)
```

The worker count is the CPU core count. There is no setting to change it, and that is on purpose: the number that matters is how many cores the box has, and the box already knows. On a single-core host the server stays one process, because there is nothing to spread the work across.

A worker that dies with a non-zero exit code gets replaced, so the pool does not quietly shrink. A worker that exits cleanly is treated as intentional and is not respawned.

What clustering costs you

Workers are separate processes and share no memory. Anything you keep in a variable lives in one worker and is invisible to the other seven:

- In-memory cache entries. Set `TINA4_CACHE_BACKEND` to Redis or Memcached so all workers see the same cache.
- In-memory sessions. Point `TINA4_SESSION_BACKEND` at the database, Redis or Mongo.
- Rate-limit counters held in a module-level object. Same answer: shared store.
- WebSocket connections. A client is connected to ONE worker. Broadcasting from another worker will not reach it without a shared backplane.

If your app keeps state in memory and you switch clustering on, it will not break loudly. It will serve one request in eight from a worker that has never heard of the thing you cached. Move the state to a shared store first.

6. Health Checks

Production deployments need a health check endpoint. Load balancers, container orchestrators, and monitoring tools all rely on it to verify the app is running.

Create a health check route:

```
import { Router } from "tina4-nodejs";
import { Database } from "tina4-nodejs/orm";

Router.get("/health", async (req, res) => {
    let dbOk = false;

    try {
        const db = await Database.getConnection();
        await db.fetchOne("SELECT 1");
        dbOk = true;
    } catch (e) {
        // Database is down
    }

    const status = dbOk ? "ok" : "degraded";
    const statusCode = dbOk ? 200 : 503;

    return res.status(statusCode).json({
        status,
```

```
        version: "1.0.0",
        database: dbOk ? "connected" : "disconnected",
        timestamp: new Date().toISOString()
    });
});
```

This endpoint:

- Returns `200` when everything is healthy
- Returns `503` when the database is down (so the load balancer stops routing traffic)
- Includes version information for deployment tracking
- Runs fast (no heavy queries, no authentication)

Broken File Check

Tina4 watches for a `.broken` file in production. When the file exists, the health check returns `503`. A signal to the load balancer: stop sending traffic.

```
touch .broken          # Health check returns 503
# Deploy new code...
rm .broken             # Health check returns 200
```

---

7. Graceful Shutdown

Deploy a new version. The old process must stop. Graceful shutdown finishes active requests before terminating.

Tina4 handles this when it receives a `SIGTERM` signal (the standard shutdown signal from Docker, Kubernetes, and systemd):

- Stop accepting new connections
- Wait for active requests to complete (up to 30 seconds)
- Close database connections
- Flush logs
- Exit

Configuring Shutdown Timeout

```
TINA4_SHUTDOWN_TIMEOUT=30
```

If active requests do not finish within the timeout, the server terminates them. Set this based on your longest expected request time. For most applications, 30 seconds is generous.

Docker Stop Grace Period

Docker sends `SIGTERM`, waits for the grace period, then sends `SIGKILL`. Match the Docker grace period to your shutdown timeout:

```
services:
  app:
    stop_grace_period: 30s
```

---

8. Log Rotation

In production, logs grow without limit unless rotated. Tina4 writes logs to `logs/app.log` and `logs/error.log`.

Using logrotate (Linux)

Create `/etc/logrotate.d/tina4`:

```
/app/logs/*.log {
    daily
    rotate 14
    compress
    delaycompress
    missingok
    notifempty
    create 0640 www-data www-data
    postrotate
        kill -USR1 $(cat /app/tina4.pid) 2>/dev/null || true
    endscript
}
```

This rotates logs daily, keeps 14 days of history, and compresses old logs. The `USR1` signal tells Tina4 to reopen its log files after rotation.

Docker Logging

Docker captures stdout/stderr automatically. Configure log rotation in the Docker daemon or compose file:

```
services:
  app:
    logging:
      driver: json-file
      options:
        max-size: "10m"
        max-file: "5"
```

This keeps at most 50MB of logs (5 files of 10MB each).

---

9. Nginx Reverse Proxy

In production, Tina4 runs behind Nginx. Nginx handles:

- SSL/TLS termination (HTTPS)
- Static file serving (faster than Node.js)
- Request buffering
- Rate limiting
- WebSocket proxying

Create `/etc/nginx/sites-available/my-app`:

```
server {
    listen 80;
    server_name yourdomain.com;
    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl;
    server_name yourdomain.com;

    ssl_certificate /etc/letsencrypt/live/yourdomain.com/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/yourdomain.com/privkey.pem;

    # Static files (served by Nginx, faster than Node.js)
    location /css/ {
        alias /app/src/public/css/;
        expires 30d;
        add_header Cache-Control "public, immutable";
    }

    location /js/ {
        alias /app/src/public/js/;
        expires 30d;
        add_header Cache-Control "public, immutable";
    }

    location /images/ {
        alias /app/src/public/images/;
        expires 30d;
        add_header Cache-Control "public, immutable";
    }

    # WebSocket upgrade
    location /ws/ {
        proxy_pass http://127.0.0.1:7148;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_read_timeout 86400;
    }

    # Application
    location / {
        proxy_pass http://127.0.0.1:7148;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

---

10. SSL/TLS with Let's Encrypt

HTTPS is non-negotiable in production. Let's Encrypt provides free SSL certificates with automatic renewal.

With Nginx and Certbot

Install Certbot:

```
sudo apt install certbot python3-certbot-nginx
```

Obtain a certificate:

```
sudo certbot --nginx -d yourdomain.com
```

Certbot modifies your Nginx configuration to include SSL settings and sets up automatic renewal. Verify auto-renewal works:

```
sudo certbot renew --dry-run
```

Certbot renews certificates 30 days before expiry. The renewal runs via a systemd timer or cron job that Certbot creates during installation.

With Docker (Using Traefik as Reverse Proxy)

Traefik handles SSL termination and automatic certificate provisioning. Add it to your Docker Compose setup:

```
services:
  reverse-proxy:
    image: traefik:v3.0
    command:
      - "--providers.docker=true"
      - "--entrypoints.web.address=:80"
      - "--entrypoints.websecure.address=:443"
      - "--certificatesresolvers.letsencrypt.acme.tlschallenge=true"
      - "--certificatesresolvers.letsencrypt.acme.email=you@example.com"
      - "--certificatesresolvers.letsencrypt.acme.storage=/letsencrypt/acme.json"
    ports:
      - "80:80"
      - "443:443"
    volumes:
      - /var/run/docker.sock:/var/run/docker.sock:ro
      - letsencrypt:/letsencrypt

  app:
    build: .
    labels:
      - "traefik.http.routers.app.rule=Host(`yourdomain.com`)"
      - "traefik.http.routers.app.tls=true"
      - "traefik.http.routers.app.tls.certresolver=letsencrypt"
    environment:
      - TINA4_DEBUG=false
      - TINA4_SECRET=${TINA4_SECRET}

volumes:
  letsencrypt:
```

Traefik detects your app container through Docker labels, provisions a certificate from Let's Encrypt, and handles all HTTPS traffic. No Nginx configuration needed.

Certificate Monitoring

Certificates expire. Even with auto-renewal, things go wrong -- DNS changes, firewall rules blocking port 80 for ACME challenges, or a crashed renewal service. Set up monitoring:

```
# Check certificate expiry manually
echo | openssl s_client -connect yourdomain.com:443 2>/dev/null | openssl x509 -noout -dates

# Verify Certbot timer is active
sudo systemctl list-timers | grep certbot
```

Use an external monitoring service (Uptime Robot, Better Uptime) that checks certificate expiry and alerts you 14 days before it expires.

---

11. Process Management with systemd

Create `/etc/systemd/system/tina4-app.service`:

```
[Unit]
Description=Tina4 Node.js Application
After=network.target

[Service]
Type=simple
User=www-data
WorkingDirectory=/var/www/my-app
ExecStart=/usr/bin/node /var/www/my-app/dist/app.js
Restart=always
RestartSec=5
Environment=NODE_ENV=production
EnvironmentFile=/var/www/my-app/.env.production

[Install]
WantedBy=multi-user.target
```

```
sudo systemctl enable tina4-app
sudo systemctl start tina4-app
sudo systemctl status tina4-app
```

---

12. Queue Workers in Production

```
[Unit]
Description=Tina4 Queue Worker
After=network.target

[Service]
Type=simple
User=www-data
WorkingDirectory=/var/www/my-app
ExecStart=/usr/local/bin/tina4 queue work
Restart=always
RestartSec=5
EnvironmentFile=/var/www/my-app/.env.production

[Install]
WantedBy=multi-user.target
```

---

13. Zero-Downtime Deployment

```
#!/bin/bash
set -e

cd /var/www/my-app

# Signal load balancer to stop sending traffic
touch .broken
sleep 5

# Pull latest code
git pull origin main

# Install dependencies and build
npm ci --only=production
tina4 build

# Run migrations
tina4 migrate

# Restart the service
sudo systemctl restart tina4-app

# Wait for health check
sleep 2
curl -f http://localhost:7148/health

# Re-enable traffic
rm .broken

echo "Deployment complete"
```

---

14. Scaling

A single server handles many applications. When traffic outgrows one server, you scale.

Vertical Scaling

Use cluster mode with more workers:

Load Balancing with Nginx

When you run multiple Tina4 instances, Nginx distributes traffic across them:

```
upstream tina4_backend {
    server 127.0.0.1:7148;
    server 127.0.0.1:7248;
    server 127.0.0.1:7348;
    server 127.0.0.1:7448;
}

server {
    listen 80;
    server_name yourdomain.com;

    location / {
        proxy_pass http://tina4_backend;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

Start four instances on different ports. Node's framework default is `7148`; use any free ports for the additional instances. Set `TINA4_OVERRIDE_CLIENT=true` so Node can run the build directly without going through `tina4 serve`:

```
TINA4_OVERRIDE_CLIENT=true TINA4_PORT=7148 node dist/app.js &
TINA4_OVERRIDE_CLIENT=true TINA4_PORT=7248 node dist/app.js &
TINA4_OVERRIDE_CLIENT=true TINA4_PORT=7348 node dist/app.js &
TINA4_OVERRIDE_CLIENT=true TINA4_PORT=7448 node dist/app.js &
```

Nginx distributes requests in round-robin order by default. If a backend goes down, Nginx routes traffic to the remaining instances.

Docker Scaling

With Docker Compose, scale horizontally with a single command:

```
docker compose up -d --scale app=4
```

Horizontal Scaling

Run multiple instances behind a load balancer. Use Redis for shared sessions, cache, and queues:

```
TINA4_SESSION_HANDLER=redis
TINA4_CACHE_BACKEND=redis
TINA4_QUEUE_BACKEND=rabbitmq
# Or use MongoDB for queues:
# TINA4_QUEUE_BACKEND=mongodb
# TINA4_MONGO_URI=mongodb://user:pass@mongo.internal:27017/tina4
```

Scaling Considerations

Scaling introduces shared-state problems. When four instances serve requests, each must agree on the state of the world.

Sessions: Store sessions in Redis, not in-memory. Otherwise, a user who logs in on instance 1 appears logged out on instance 2.

Database: SQLite handles one writer at a time. Under high load with multiple instances, switch to PostgreSQL or MySQL. If you must use SQLite, enable WAL mode.

File uploads: Store uploaded files in shared storage (S3, a mounted volume) -- not the local filesystem of a single container.

Cache: Use Redis as the cache backend so all instances share the same cache.

---

15. Monitoring

Your app runs in production. You need to know when it breaks, slows down, or runs out of resources.

Log Aggregation

Switch to JSON-formatted logs for production. Structured logs feed into aggregation services:

```
TINA4_LOG_FORMAT=json
```

Example JSON log entry:

```
{
  "timestamp": "2026-03-22T14:30:00.123Z",
  "level": "WARNING",
  "message": "Rate limit exceeded",
  "request_id": "req-abc123",
  "ip": "203.0.113.42",
  "path": "/api/products",
  "method": "GET"
}
```

Services that ingest JSON logs:

Service	Strengths
Grafana Loki	Open source, pairs with Grafana dashboards
Elastic Stack (ELK)	Full-text search across logs
Datadog	Managed service, correlates logs with metrics
AWS CloudWatch	Native for AWS deployments

Docker makes log aggregation straightforward. Configure the logging driver to send container output to your chosen service:

```
services:
```

```
  app:
    logging:
      driver: "fluentd"
      options:
        fluentd-address: "localhost:24224"
        tag: "tina4.app"
```

Uptime Monitoring

Point an external monitoring service at your health endpoint:

```
https://yourdomain.com/health
```

Services like Uptime Robot, Pingdom, or Better Uptime ping this endpoint every 30-60 seconds. When it stops responding or returns a non-200 status, you receive an alert via email, SMS, or Slack.

The health endpoint from section 6 serves double duty: container orchestrators use it for restart decisions, and uptime monitors use it for alerting.

Application Performance Monitoring (APM)

Uptime monitoring tells you the app is running. APM tells you how well it performs. APM agents track:

- Request latency (average, p95, p99)
- Database query performance (slow queries, connection pool usage)
- Error rates (which endpoints fail, how often)
- Memory and CPU usage over time

Since Tina4 Node.js runs on standard Node.js, any Node.js APM agent works:

- **Datadog APM**: `npm install dd-trace` and `require('dd-trace').init()` at the top of your app
- **New Relic**: `npm install newrelic` and `require('newrelic')` at the top of your app
- **Elastic APM**: `npm install elastic-apm-node` and configure in your app startup

A basic monitoring stack for a small team: Uptime Robot for availability alerts (free tier covers it), JSON logs shipped to Grafana Loki for debugging, and `docker stats` for resource usage. Add APM when your application serves enough traffic to warrant the cost.

---

16. Exercise: Docker Deploy

Deploy a Tina4 Node.js application using Docker.

Requirements

- Create a `Dockerfile` that:
- Uses `node:20-alpine` as the base image
- Installs production dependencies with `npm ci`

- Copies the built application code
- Exposes port 7148
- Includes a health check
- Runs the app with `node dist/app.js`
- Create a `docker-compose.yml` that:
- Builds and runs the app
- Starts a Redis container for caching
- Mounts volumes for data persistence
- Sets environment variables for production
- Create a `/health` endpoint that checks database connectivity
- Build, run, and verify:

```
# Build
tina4 build
docker compose build

# Start
docker compose up -d

# Test health
curl http://localhost:7148/health

# Test the app
curl http://localhost:7148/api/products

# View logs
docker compose logs -f app

# Stop
docker compose down
```

Solution

The Dockerfile and docker-compose.yml are shown in section 4. The health check route is shown in section 6. Combine them in your project, then:

```
tina4 build
docker compose up -d --build
```

```
[+] Building 12.3s
[+] Running 2/2
  Container redis    Started
  Container my-app   Started
```

```
curl http://localhost:7148/health
```

```
{
  "status": "ok",
  "version": "1.0.0",
  "database": "connected",
  "timestamp": "2026-03-22T14:30:00.000Z"
}
```

The app runs in production mode with Redis caching, persistent data volumes, automatic restarts, and health monitoring.

HTTP Caching, Compression, and ETags

Tina4 applies gzip to eligible dynamic responses and then computes a strong ETag from the final post-compression bytes. The identity and gzip representations therefore have different ETags, and `Vary: Accept-Encoding` keeps intermediary caches from mixing them. A matching `If-None-Match` returns `304 Not Modified` with an empty body while preserving the validators.

Static files use a weak validator derived from file size and modification time. Streaming responses bypass dynamic compression and ETag generation because their complete representation is not buffered. Test both identity and gzip requests when validating cache behavior.

17. Gotchas

1. .env Not Loaded in Docker

Problem: Environment variables from `.env` are not available in the container.

Cause: Docker does not read `.env` files automatically. The `.env` file belongs in `.dockerignore` (never ship secrets in the image).

Fix: Pass environment variables via `docker run -e`, the `environment` section in `docker-compose.yml`, or an `env_file` directive. For secrets, use Docker secrets or your platform's secret management.

2. SQLite Database Lost on Container Restart

Problem: All data disappears when the container restarts.

Cause: The SQLite database file sits inside the container. When the container is recreated, the file is gone.

Fix: Mount a volume for the data directory: `-v $(pwd)/data:/app/data`. In Docker Compose, use a named volume: `volumes: [app-data:/app/data]`.

3. WebSocket Connections Drop Behind Nginx

Problem: WebSocket connections fail or drop immediately behind Nginx.

Cause: Nginx does not proxy WebSocket by default. It treats the upgrade request as a regular HTTP request.

Fix: Add WebSocket proxy headers in your Nginx config:

```
proxy_http_version 1.1;
proxy_set_header Upgrade $http_upgrade;
proxy_set_header Connection "upgrade";
proxy_read_timeout 86400;
```

4. Static Files Slow

Problem: CSS, JS, and images load slowly in production.

Cause: Node.js serves static files. Every static file request goes through the Node.js process.

Fix: Serve static files from Nginx (see the Nginx config in section 9). Nginx serves static files from memory-mapped files, which is orders of magnitude faster than Node.js.

5. Memory Usage Grows Over Time

Problem: The container's memory usage climbs until it crashes with OOM (out of memory).

Cause: Memory leaks in the application -- unclosed database connections, growing caches without TTL, accumulating request data in global variables.

Fix: Set TTLs on all cache entries. Close database connections properly. Avoid storing request data in module-level variables. Use `docker stats` to monitor memory usage. Set memory limits in Docker Compose: `deploy: {resources: {limits: {memory: 512M}}}`.

6. Container Starts Before Database Is Ready

Problem: The app crashes on startup because the database is not ready.

Cause: Docker Compose starts services in parallel. The app container starts before the database container finishes initializing.

Fix: For SQLite, this is not an issue (the file is created automatically). For PostgreSQL or MySQL, use a startup script that waits for the database, or use Docker Compose healthcheck on the database service with `depends_on: {db: {condition: service_healthy}}`.

7. SSL Certificate Not Renewing

Problem: Your HTTPS certificate expires and the site goes down.

Cause: The auto-renewal process (Certbot or Traefik) failed. Common reasons: DNS changes, firewall blocking port 80 for ACME challenges, or the renewal service crashed.

Fix: Monitor certificate expiry with an external service. Check renewal logs and verify the renewal timer is active:

```
sudo certbot renew --dry-run
sudo systemctl list-timers | grep certbot
```

8. Scaled Instances Have Different State

Problem: Users see inconsistent data across requests when running multiple app instances.

Cause: In-memory sessions and cache are not shared between instances. A user who logs in on instance 1 appears logged out when the load balancer routes the next request to instance 2.

Fix: Store sessions in Redis. Use Redis as the cache backend. Store uploaded files in shared storage (S3 or a mounted volume). All instances must read from and write to the same data stores.

9. Debug Mode in Production

Fix: Set `TINA4_DEBUG=false`.

10. .env in Version Control

Fix: Add to `.gitignore`.

11. Missing Migrations

Fix: Always run `tina4 migrate` during deployment.

12. Port Conflicts

Fix: Ensure only one process listens on port 7148.

Building a Complete App

1. Putting It All Together

Twenty chapters of building blocks. Routing. Templates. Databases. ORM. Authentication. Middleware. Queues. WebSocket. Caching. Frontend. GraphQL. Testing. Dev tools. CLI scaffolding. Deployment. Now all of it works together in one application.

We are building **TaskFlow** -- a task management system with:

- User registration and JWT authentication
- Task creation, assignment, and tracking
- A dashboard with real-time updates
- Email notifications when tasks are assigned
- Caching for dashboard performance
- A full test suite
- Docker deployment

Not a toy project. A production-ready application. Every major Tina4 feature in one codebase.

---

2. Planning the App

Models

Model	Table	Fields
User	users	id, name, email, passwordHash, role, createdAt
Task	tasks	id, title, description, status, priority, createdBy, assignedTo, dueDate, completedAt, createdAt, updatedAt

Relationships

- A User has many Tasks (created by them)
- A User has many Tasks (assigned to them)
- A Task belongs to a User (creator)
- A Task belongs to a User (assignee)

Routes

Method	Path	Description	Auth
POST	/api/auth/register	Register a new user	public
POST	/api/auth/login	Login, get JWT	public
GET	/api/profile	Get current user profile	secured
GET	/api/tasks	List tasks (with filters)	secured
GET	/api/tasks/{id}	Get a single task	secured
POST	/api/tasks	Create a task	secured
PUT	/api/tasks/{id}	Update a task	secured
DELETE	/api/tasks/{id}	Delete a task	secured
GET	/api/dashboard/stats	Dashboard statistics	secured
GET	/admin	Dashboard HTML page	public

Templates

- `base.html` -- Base layout with sidebar and topbar
- `dashboard.html` -- Dashboard with stats, task list, quick actions
- `login.html` -- Login page
- `register.html` -- Registration page

---

3. Step 1: Init Project and Set Up Database

```
tina4 init taskflow
cd taskflow
npm install
```

Update `.env`:

```
TINA4_DEBUG=true
TINA4_SECRET=taskflow-dev-secret-change-in-production
TINA4_TOKEN_LIMIT=86400
```

Create Migrations

Create `migrations/20260322000100_create_users_table.sql`:

```
-- UP
CREATE TABLE users (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    email TEXT NOT NULL UNIQUE,
```

```
    password_hash TEXT NOT NULL,
    role TEXT NOT NULL DEFAULT 'user',
    created_at TEXT DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_users_email ON users(email);

-- DOWN
DROP TABLE IF EXISTS users;
```

Create `migrations/20260322000200_create_tasks_table.sql`:

```
-- UP
CREATE TABLE tasks (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    title TEXT NOT NULL,
    description TEXT DEFAULT '',
    status TEXT NOT NULL DEFAULT 'todo',
    priority TEXT NOT NULL DEFAULT 'medium',
    created_by INTEGER NOT NULL,
    assigned_to INTEGER,
    due_date TEXT,
    completed_at TEXT,
    created_at TEXT DEFAULT CURRENT_TIMESTAMP,
    updated_at TEXT DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (created_by) REFERENCES users(id),
    FOREIGN KEY (assigned_to) REFERENCES users(id)
);

CREATE INDEX idx_tasks_status ON tasks(status);
CREATE INDEX idx_tasks_assigned_to ON tasks(assigned_to);
CREATE INDEX idx_tasks_created_by ON tasks(created_by);

-- DOWN
DROP TABLE IF EXISTS tasks;
```

Run migrations:

```
tina4 migrate
```

```
Running migrations...
  [UP] 20260322000100_create_users_table.sql
  [UP] 20260322000200_create_tasks_table.sql

  2 migrations applied
```

Verify the database:

```
curl http://localhost:7148/health
```

```
{"status": "ok", "database": "connected"}
```

---

4. Step 2: Define Models

Create `src/orm/User.ts`:

```
import { BaseModel, Auth } from "tina4-nodejs";

export class User extends BaseModel {
```

```
    static tableName = "users";
    static primaryKey = "id";
    static hasMany = [
        { model: "Task", foreignKey: "created_by", as: "createdTasks" },
        { model: "Task", foreignKey: "assigned_to", as: "assignedTasks" }
    ];

    id!: number;
    name!: string;
    email!: string;
    passwordHash!: string;
    role: string = "user";
    createdAt!: string;

    async setPassword(password: string): Promise<void> {
        this.passwordHash = await Auth.hashPassword(password);
    }

    async verifyPassword(password: string): Promise<boolean> {
        if (!this.passwordHash) return false;
        return Auth.checkPassword(password, this.passwordHash);
    }

    safeDict(): Record<string, any> {
        const data = this.toDict();
        delete data.passwordHash;
        delete data.password_hash;
        return data;
    }
}
```

The `setPassword` method hashes the plain-text password before storage. `verifyPassword` compares a candidate password against the stored hash. `safeDict` strips the hash from any response -- never expose password data to the client.

Create `src/orm/Task.ts`:

```
import { BaseModel } from "tina4-nodejs/orm";

export class Task extends BaseModel {
    static tableName = "tasks";
    static primaryKey = "id";
    static belongsTo = [
        { model: "User", foreignKey: "created_by", as: "creator" },
        { model: "User", foreignKey: "assigned_to", as: "assignee" }
    ];

    static STATUSES = ["todo", "in_progress", "done", "cancelled"];
    static PRIORITIES = ["low", "medium", "high", "urgent"];

    id!: number;
    title!: string;
    description: string = "";
    status: string = "todo";
    priority: string = "medium";
    createdBy!: number;
    assignedTo: number | null = null;
    dueDate: string | null = null;
    completedAt: string | null = null;
```

```
    createdAt!: string;
    updatedAt!: string;
}
```

The static `STATUSES` and `PRIORITIES` arrays serve as validation lists. Any route that accepts a status or priority value checks it against these arrays before writing to the database.

---

5. Step 3: Auth Middleware

Create `src/routes/middleware.ts`:

```
import { Auth } from "tina4-nodejs";

export function authMiddleware(req, res, next) {
    const authHeader = req.headers["authorization"] ?? "";

    if (!authHeader || !authHeader.startsWith("Bearer ")) {
        res({ error: "Authorization required" }, 401);
        return;
    }

    const token = authHeader.substring(7);

    const secret = process.env.TINA4_SECRET || "tina4-default-secret";
    const payload = Auth.validToken(token, secret);
    if (payload === null) {
        res({ error: "Invalid or expired token" }, 401);
        return;
    }

    req.user = payload;
    req.userId = payload.user_id;
    next();
}
```

Verify the middleware blocks unauthenticated requests:

```
curl http://localhost:7148/api/tasks
```

```
{"error": "Authorization required"}
```

The middleware stands guard. No token, no access.

---

6. Step 4: Auth Routes

Create `src/routes/auth.ts`:

```
import { Router, Auth } from "tina4-nodejs";
import { Database } from "tina4-nodejs/orm";
import { authMiddleware } from "./middleware";

/**
 * Register a new user
 * @noauth
 * @tags Auth
```

```
 * @body {"name": "string", "email": "string", "password": "string"}
 * @response 201 {"message": "string", "user": {"id": "int", "name": "string", "email": "string"
 */
Router.post("/api/auth/register", async (req, res) => {
    const { name, email, password } = req.body;

    if (!name || !email || !password) {
        return res.status(400).json({ error: "Name, email, and password are required" });
    }

    if (password.length < 8) {
        return res.status(400).json({ error: "Password must be at least 8 characters" });
    }

    const db = await Database.getConnection();
    const existing = await db.fetchOne("SELECT id FROM users WHERE email = :email", { email });

    if (existing) {
        return res.status(409).json({ error: "Email already registered" });
    }

    const hash = await Auth.hashPassword(password);
    await db.execute(
        "INSERT INTO users (name, email, password_hash) VALUES (:name, :email, :hash)",
        { name, email, hash }
    );

    const user = await db.fetchOne("SELECT id, name, email, role FROM users WHERE id = last_inse

    return res.status(201).json({ message: "Registration successful", user });
});

/**
 * Login and get JWT token
 * @noauth
 * @tags Auth
 * @body {"email": "string", "password": "string"}
 * @response 200 {"token": "string", "user": {"id": "int", "name": "string", "email": "string"}}
 */
Router.post("/api/auth/login", async (req, res) => {
    const { email, password } = req.body;

    if (!email || !password) {
        return res.status(400).json({ error: "Email and password are required" });
    }

    const db = await Database.getConnection();
    const user = await db.fetchOne(
        "SELECT id, name, email, password_hash, role FROM users WHERE email = :email",
        { email }
    );

    if (!user || !(await Auth.checkPassword(password, user.password_hash))) {
        return res.status(401).json({ error: "Invalid email or password" });
    }

    const secret = process.env.TINA4_SECRET || "tina4-default-secret";
    const token = Auth.getToken({
        user_id: user.id,
```

```
        email: user.email,
        name: user.name,
        role: user.role
    }, secret);

    return res.json({
        message: "Login successful",
        token,
        user: { id: user.id, name: user.name, email: user.email, role: user.role }
    });
});

/**
 * Get current user profile
 * @tags Auth
 * @response 200 {"id": "int", "name": "string", "email": "string", "role": "string", "created_a
 */
Router.get("/api/profile", async (req, res) => {
    const db = await Database.getConnection();
    const user = await db.fetchOne(
        "SELECT id, name, email, role, created_at FROM users WHERE id = :id",
        { id: req.user.user_id }
    );
    return res.json(user);
}, [authMiddleware]);
```

Test Registration and Login

```
# Start the server
tina4 serve
```

```
# Register a user
curl -X POST http://localhost:7148/api/auth/register \
  -H "Content-Type: application/json" \
  -d '{"name": "Alice Johnson", "email": "alice@example.com", "password": "securePass123"}'
```

```
{
  "message": "Registration successful",
  "user": {"id": 1, "name": "Alice Johnson", "email": "alice@example.com", "role": "user"}
}
```

```
# Login
curl -X POST http://localhost:7148/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email": "alice@example.com", "password": "securePass123"}'
```

```
{
  "message": "Login successful",
  "token": "eyJhbGciOiJIUzI1NiIs...",
  "user": {"id": 1, "name": "Alice Johnson", "email": "alice@example.com", "role": "user"}
}
```

Authentication works. Save the token for the next steps.

```
export TOKEN="eyJhbGciOiJIUzI1NiIs..."
```

---

7. Step 5: Task Routes

Create `src/routes/tasks.ts`:

```
import { Router, Queue } from "tina4-nodejs";
import { Database } from "tina4-nodejs/orm";
import { cacheDelete } from "tina4-nodejs";
import { authMiddleware } from "./middleware";
import { Task } from "../orm/Task";
import { pushTaskUpdate } from "./ws-tasks";

/**
 * List tasks with filters
 * @tags Tasks
 * @query string status Filter by status (todo, in_progress, done)
 * @query string priority Filter by priority (low, medium, high)
 * @query string assigned Filter by assigned user ID
 * @query int page Page number
 * @query int limit Results per page
 */
Router.get("/api/tasks", async (req, res) => {
    const db = await Database.getConnection();
    const userId = req.user.user_id;
    const page = parseInt(req.query.page ?? "1", 10);
    const limit = parseInt(req.query.limit ?? "20", 10);
    const offset = (page - 1) * limit;

    const conditions = ["(created_by = :userId OR assigned_to = :userId)"];
    const params: Record<string, any> = { userId };

    if (req.query.status) {
        conditions.push("status = :status");
        params.status = req.query.status;
    }

    if (req.query.priority) {
        conditions.push("priority = :priority");
        params.priority = req.query.priority;
    }

    if (req.query.assigned) {
        conditions.push("assigned_to = :assignedTo");
        params.assignedTo = parseInt(req.query.assigned, 10);
    }

    const sql = `SELECT t.*,
        creator.name as creator_name,
        assignee.name as assignee_name
        FROM tasks t
        LEFT JOIN users creator ON t.created_by = creator.id
        LEFT JOIN users assignee ON t.assigned_to = assignee.id
        WHERE ${conditions.join(" AND ")}
        ORDER BY
            CASE t.priority WHEN 'high' THEN 1 WHEN 'medium' THEN 2 WHEN 'low' THEN 3 END,
            t.created_at DESC
        LIMIT :limit OFFSET :offset`;

    params.limit = limit;
    params.offset = offset;
```

```
    const tasks = await db.fetch(sql, params);

    return res.json({ tasks, page, limit, count: tasks.length });
}, [authMiddleware]);

/**
 * Get a single task
 * @tags Tasks
 * @param int id Task ID
 */
Router.get("/api/tasks/{id:int}", async (req, res) => {
    const db = await Database.getConnection();
    const task = await db.fetchOne(
        `SELECT t.*, creator.name as creator_name, assignee.name as assignee_name
         FROM tasks t
         LEFT JOIN users creator ON t.created_by = creator.id
         LEFT JOIN users assignee ON t.assigned_to = assignee.id
         WHERE t.id = :id`,
        { id: req.params.id }
    );

    if (!task) {
        return res.status(404).json({ error: "Task not found" });
    }

    return res.json(task);
}, [authMiddleware]);

/**
 * Create a task
 * @tags Tasks
 * @body {"title": "string", "description": "string", "priority": "string", "assigned_to": "int"
 */
Router.post("/api/tasks", async (req, res) => {
    const db = await Database.getConnection();
    const { title, description, priority, assigned_to, due_date } = req.body;

    if (!title) {
        return res.status(400).json({ error: "Title is required" });
    }

    if (priority && !Task.PRIORITIES.includes(priority)) {
        return res.status(400).json({ error: `Invalid priority. Must be one of: ${Task.PRIORITIE
    }

    await db.execute(
        `INSERT INTO tasks (title, description, status, priority, created_by, assigned_to, due_d
         VALUES (:title, :description, 'todo', :priority, :createdBy, :assignedTo, :dueDate)`,
        {
            title,
            description: description ?? "",
            priority: priority ?? "medium",
            createdBy: req.user.user_id,
            assignedTo: assigned_to ?? null,
            dueDate: due_date ?? null
        }
    );

    const task = await db.fetchOne("SELECT * FROM tasks WHERE id = last_insert_rowid()");
```

```
    // Invalidate dashboard cache
    cacheDelete("dashboard:stats");

    // Queue notification if assigned to someone
    if (assigned_to) {
        const notifyQueue = new Queue({ topic: "task-notifications" });
        notifyQueue.push({
            type: "assigned",
            task_id: task.id,
            task_title: title,
            assigned_to,
            assigned_by: req.user.name
        });
    }

    // Push real-time update
    pushTaskUpdate("created", task);

    return res.status(201).json(task);
}, [authMiddleware]);

/**
 * Update a task
 * @tags Tasks
 */
Router.put("/api/tasks/{id:int}", async (req, res) => {
    const db = await Database.getConnection();
    const id = req.params.id;

    const existing = await db.fetchOne("SELECT * FROM tasks WHERE id = :id", { id });
    if (!existing) {
        return res.status(404).json({ error: "Task not found" });
    }

    const { title, description, status, priority, assigned_to, due_date } = req.body;

    if (status && !Task.STATUSES.includes(status)) {
        return res.status(400).json({ error: `Invalid status. Must be one of: ${Task.STATUSES.jo
    }

    if (priority && !Task.PRIORITIES.includes(priority)) {
        return res.status(400).json({ error: `Invalid priority. Must be one of: ${Task.PRIORITIE
    }

    const completedAt = status === "done" && existing.status !== "done"
        ? new Date().toISOString()
        : existing.completed_at;

    // Detect reassignment for notification
    const oldAssignee = existing.assigned_to;
    const newAssignee = assigned_to ?? existing.assigned_to;

    await db.execute(
        `UPDATE tasks SET title = :title, description = :description, status = :status,
         priority = :priority, assigned_to = :assignedTo, due_date = :dueDate,
         completed_at = :completedAt, updated_at = CURRENT_TIMESTAMP
         WHERE id = :id`,
        {
            title: title ?? existing.title,
```

```
            description: description ?? existing.description,
            status: status ?? existing.status,
            priority: priority ?? existing.priority,
            assignedTo: newAssignee,
            dueDate: due_date ?? existing.due_date,
            completedAt,
            id
        }
    );

    const task = await db.fetchOne("SELECT * FROM tasks WHERE id = :id", { id });

    // Invalidate dashboard cache
    cacheDelete("dashboard:stats");

    // Notify new assignee if reassigned
    if (newAssignee && newAssignee !== oldAssignee) {
        const notifyQueue = new Queue({ topic: "task-notifications" });
        notifyQueue.push({
            type: "assigned",
            task_id: task.id,
            task_title: task.title,
            assigned_to: newAssignee,
            assigned_by: req.user.name
        });
    }

    // Push real-time update
    pushTaskUpdate("updated", task);

    return res.json(task);
}, [authMiddleware]);

/**
 * Delete a task
 * @tags Tasks
 */
Router.delete("/api/tasks/{id:int}", async (req, res) => {
    const db = await Database.getConnection();
    const id = req.params.id;

    const existing = await db.fetchOne("SELECT * FROM tasks WHERE id = :id", { id });
    if (!existing) {
        return res.status(404).json({ error: "Task not found" });
    }

    await db.execute("DELETE FROM tasks WHERE id = :id", { id });

    // Invalidate dashboard cache
    cacheDelete("dashboard:stats");

    // Push real-time update
    pushTaskUpdate("deleted", { id });

    return res.status(204).json(null);
}, [authMiddleware]);
```

Test the Task API

```
# Set your token from the login step
TOKEN="eyJhbGciOiJIUzI1NiIs..."

# Create a task
curl -X POST http://localhost:7148/api/tasks \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer $TOKEN" \
  -d '{"title": "Design database schema", "priority": "high"}'
```

```
{
  "id": 1,
  "title": "Design database schema",
  "priority": "high",
  "status": "todo",
  "created_by": 1,
  "created_at": "2026-03-22 10:00:00"
}
```

```
# Create more tasks
curl -X POST http://localhost:7148/api/tasks \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer $TOKEN" \
  -d '{"title": "Build API endpoints", "priority": "high"}'

curl -X POST http://localhost:7148/api/tasks \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer $TOKEN" \
  -d '{"title": "Write documentation", "priority": "medium", "due_date": "2026-04-01"}'

# List all tasks
curl http://localhost:7148/api/tasks \
  -H "Authorization: Bearer $TOKEN"

# Update a task status
curl -X PUT http://localhost:7148/api/tasks/1 \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer $TOKEN" \
  -d '{"status": "done"}'

# Delete a task
curl -X DELETE http://localhost:7148/api/tasks/3 \
  -H "Authorization: Bearer $TOKEN"
```

Every endpoint responds. Create. Read. Update. Delete. The CRUD cycle works.

---

8. Step 6: Dashboard Stats with Caching

Create `src/routes/dashboard.ts`:

```
import { Router, cacheGet, cacheSet } from "tina4-nodejs";
import { Database } from "tina4-nodejs/orm";
import { authMiddleware } from "./middleware";

/**
 * Dashboard statistics
 * @tags Dashboard
```

```
 * @response 200 {"total_tasks": "int", "by_status": "object", "overdue_tasks": "int", "total_us
 */
Router.get("/api/dashboard/stats", async (req, res) => {
    const userId = req.user.user_id;
    const cacheKey = `dashboard:${userId}`;

    const cached = await cacheGet(cacheKey);
    if (cached) {
        return res.json({ ...cached, source: "cache" });
    }

    const db = await Database.getConnection();

    const total = await db.fetchOne(
        "SELECT COUNT(*) as count FROM tasks WHERE created_by = :userId OR assigned_to = :userId
        { userId }
    );

    const todo = await db.fetchOne(
        "SELECT COUNT(*) as count FROM tasks WHERE (created_by = :userId OR assigned_to = :userI
        { userId }
    );

    const inProgress = await db.fetchOne(
        "SELECT COUNT(*) as count FROM tasks WHERE (created_by = :userId OR assigned_to = :userI
        { userId }
    );

    const done = await db.fetchOne(
        "SELECT COUNT(*) as count FROM tasks WHERE (created_by = :userId OR assigned_to = :userI
        { userId }
    );

    const overdue = await db.fetchOne(
        "SELECT COUNT(*) as count FROM tasks WHERE (created_by = :userId OR assigned_to = :userI
        { userId }
    );

    const users = await db.fetchOne("SELECT COUNT(*) as count FROM users");

    const recentTasks = await db.fetch(
        `SELECT t.*, u.name as assignee_name FROM tasks t
         LEFT JOIN users u ON t.assigned_to = u.id
         WHERE t.created_by = :userId OR t.assigned_to = :userId
         ORDER BY t.created_at DESC LIMIT 10`,
        { userId }
    );

    const stats = {
        total_tasks: total.count,
        todo: todo.count,
        in_progress: inProgress.count,
        done: done.count,
        overdue_tasks: overdue.count,
        total_users: users.count,
        recent_tasks: recentTasks
    };

    await cacheSet(cacheKey, stats, 30);
```

```
    return res.json({ ...stats, source: "database" });
}, [authMiddleware]);

/**
 * Dashboard HTML page
 * @noauth
 * @tags Dashboard
 */
Router.get("/admin", async (req, res) => {
    return res.render("dashboard.html", {});
});
```

Verify the stats endpoint:

```
curl http://localhost:7148/api/dashboard/stats \
  -H "Authorization: Bearer $TOKEN"
```

```
{
  "total_tasks": 2,
  "todo": 1,
  "in_progress": 0,
  "done": 1,
  "overdue_tasks": 0,
  "total_users": 1,
  "recent_tasks": [...],
  "source": "database"
}
```

Hit it again. The second response comes from cache:

```
curl http://localhost:7148/api/dashboard/stats \
  -H "Authorization: Bearer $TOKEN"
```

```
{
  "total_tasks": 2,
  "todo": 1,
  "done": 1,
  "source": "cache"
}
```

The cache holds for 30 seconds. Creating, updating, or deleting a task invalidates it.

---

9. Step 7: WebSocket for Real-Time Updates

Create `src/routes/ws-tasks.ts`:

```
import { Router } from "tina4-nodejs";

Router.websocket("/ws/tasks", async (connection, event, data) => {
    if (event === "open") {
        connection.send(JSON.stringify({
            type: "connected",
            message: "Listening for task updates"
        }));
    }

    if (event === "message") {
        const message = JSON.parse(data);
```

```
        if (message.type === "ping") {
            connection.send(JSON.stringify({ type: "pong" }));
        }
    }
});

/**
 * Push a task update to all connected WebSocket clients.
 */
export function pushTaskUpdate(action: string, task: any): void {
    Router.pushToWebSocket("/ws/tasks", JSON.stringify({
        type: "task_update",
        action,
        task
    }));
}
```

Any user creates, updates, or deletes a task. All connected dashboard users see the change.

---

10. Step 8: Email Notifications via Queue

Create `src/routes/queue-consumers.ts`:

```
import { Queue, Messenger } from "tina4-nodejs";
import { Database } from "tina4-nodejs/orm";

const notifyQueue = new Queue({ topic: "task-notifications" });

notifyQueue.process(async (job) => {
    const { type, task_title, task_id, assigned_to, assigned_by } = job.payload as any;

    const db = await Database.getConnection();
    const user = await db.fetchOne("SELECT name, email FROM users WHERE id = :id", { id: assigne

    if (!user) {
        job.complete();
        return;
    }

    console.log(`[Notification] Task "${task_title}" assigned to ${user.name} by ${assigned_by}`

    const mailer = new Messenger();
    await mailer.send({
        to: user.email,
        subject: `New Task Assigned: ${task_title}`,
        body: `<h2>New Task Assigned</h2>
               <p>Hi ${user.name},</p>
               <p><strong>${assigned_by}</strong> assigned you a new task:</p>
               <div style="border:1px solid #ddd; padding:16px; margin:16px 0;">
                   <h3>${task_title}</h3>
                   <p><a href="http://localhost:7148/admin#task-${task_id}">View Task</a></p>
               </div>`,
        html: true
    });

    job.complete();
});
```

Create the email template at `src/templates/emails/task-assigned.html`:

```
<!DOCTYPE html>
<html>
<head><meta charset="UTF-8"></head>
<body>
    <div class="container">
        <h2>New Task Assigned</h2>
        <p>Hi {{ assignee_name }},</p>
        <p><strong>{{ creator_name }}</strong> assigned you a new task:</p>

        <div class="card">
            <div class="card-body">
                <h3>{{ task_title }}</h3>
                <p>{{ task_description }}</p>
                <table class="table">
                    <tr><td><strong>Priority:</strong></td><td>{{ task_priority }}</td></tr>
                    <tr><td><strong>Due:</strong></td><td>{{ task_due_date }}</td></tr>
                </table>
            </div>
        </div>

        <p><a href="{{ task_url }}">View Task</a></p>
    </div>
</body>
</html>
```

The queue consumer runs in the background. The task route pushes a job. The consumer picks it up, looks up the assignee, and sends the email. No blocking. No delay in the API response.

---

11. Step 9: Dashboard Template

Create `src/templates/dashboard.html`:

```
{% extends "base.html" %}

{% block title %}TaskFlow Dashboard{% endblock %}

{% block content %}
<h1>Dashboard</h1>

<div id="stats" class="row mb-4">
    <div class="col-md-2"><div class="card"><div class="card-body">
        <h6 class="text-muted">Total</h6><h2 id="stat-total">--</h2>
    </div></div></div>
    <div class="col-md-2"><div class="card"><div class="card-body">
        <h6 class="text-muted">To Do</h6><h2 id="stat-todo">--</h2>
    </div></div></div>
    <div class="col-md-2"><div class="card"><div class="card-body">
        <h6 class="text-muted">In Progress</h6><h2 id="stat-progress">--</h2>
    </div></div></div>
    <div class="col-md-2"><div class="card"><div class="card-body">
        <h6 class="text-muted">Done</h6><h2 id="stat-done">--</h2>
    </div></div></div>
    <div class="col-md-2"><div class="card"><div class="card-body">
        <h6 class="text-muted">Overdue</h6><h2 id="stat-overdue" class="text-danger">--</h2>
```

```
    </div></div></div>
    <div class="col-md-2"><div class="card"><div class="card-body">
        <h6 class="text-muted">Users</h6><h2 id="stat-users">--</h2>
    </div></div></div>
</div>

<div class="row">
    <div class="col-md-8">
        <div class="card">
            <div class="card-header d-flex justify-content-between">
                <span>Recent Tasks</span>
                <button class="btn btn-sm btn-primary" data-toggle="modal" data-target="#newTask
                    New Task
                </button>
            </div>
            <div class="card-body">
                <table class="table table-striped" id="task-table">
                    <thead>
                        <tr><th>Title</th><th>Assignee</th><th>Priority</th><th>Status</th></tr>
                    </thead>
                    <tbody id="task-list"></tbody>
                </table>
            </div>
        </div>
    </div>
    <div class="col-md-4">
        <div class="card">
            <div class="card-header">Live Updates</div>
            <div class="card-body" id="live-updates">
                <p class="text-muted">Connecting...</p>
            </div>
        </div>
    </div>
</div>

<script src="/js/frond.min.js"></script>
<script>
    var token = localStorage.getItem("token");
    if (token) frond.setToken(token);

    // Load dashboard stats
    frond.get("/api/dashboard/stats", function (data) {
        document.getElementById("stat-total").textContent = data.total_tasks;
        document.getElementById("stat-todo").textContent = data.todo;
        document.getElementById("stat-progress").textContent = data.in_progress;
        document.getElementById("stat-done").textContent = data.done;
        document.getElementById("stat-overdue").textContent = data.overdue_tasks;
        document.getElementById("stat-users").textContent = data.total_users;

        var tbody = document.getElementById("task-list");
        tbody.innerHTML = "";
        (data.recent_tasks || []).forEach(function (task) {
            var tr = document.createElement("tr");
            var badgeColor = {todo: "secondary", in_progress: "warning",
                              done: "success", cancelled: "danger"};
            tr.innerHTML = "<td>" + task.title + "</td>" +
                "<td>" + (task.assignee_name || "Unassigned") + "</td>" +
                "<td>" + task.priority + "</td>" +
                "<td><span class='badge bg-" + (badgeColor[task.status] || "secondary") +
```

```
                "'>" + task.status + "</span></td>";
            tbody.appendChild(tr);
        });
    });

    // WebSocket for live updates
    var ws = frond.ws("/ws/tasks");
    var updates = document.getElementById("live-updates");

    ws.on("open", function () {
        updates.innerHTML = "<p class='text-success'>Connected - listening for updates</p>";
    });

    ws.on("message", function (raw) {
        var msg = JSON.parse(raw);
        if (msg.type === "task_update") {
            var div = document.createElement("div");
            div.innerHTML = "<strong>" + msg.action + ":</strong> " + msg.task.title;
            updates.prepend(div);

            // Refresh stats
            frond.get("/api/dashboard/stats", function () {});
        }
    });
</script>
{% endblock %}
```

Verify the dashboard:

```
curl http://localhost:7148/admin
```

The server returns the rendered HTML. Open `http://localhost:7148/admin` in your browser to see the full dashboard with stats, task list, and live update panel.

---

12. Step 10: Tests

Create `tests/TaskFlowTest.ts`:

```
import { tests, expectEqual, expectTrue, runAll, Auth } from "tina4-nodejs";
import { TestClient } from "tina4-nodejs/test";

const client = new TestClient();

function randomEmail(): string {
    const hex = Math.random().toString(16).substring(2, 10);
    return `test-${hex}@example.com`;
}

async function registerAndLogin(): Promise<string> {
    const email = randomEmail();
    await client.post("/api/auth/register", {
        name: "Test User",
        email,
        password: "TestPassword123!"
    });
    const loginResp = await client.post("/api/auth/login", {
        email,
```

```
        password: "TestPassword123!"
    });
    return loginResp.body.token;
}

// Test registration
const testRegister = tests(
    expectEqual([201])
)(async function testRegister(): Promise<number> {
    const resp = await client.post("/api/auth/register", {
        name: "Alice",
        email: randomEmail(),
        password: "SecurePass123!"
    });
    return resp.statusCode;
});

// Test login returns a token
const testLogin = tests(
    expectTrue([true])
)(async function testLogin(): Promise<boolean> {
    const email = randomEmail();
    await client.post("/api/auth/register", {
        name: "Bob",
        email,
        password: "SecurePass123!"
    });
    const resp = await client.post("/api/auth/login", {
        email,
        password: "SecurePass123!"
    });
    return resp.statusCode === 200 && resp.body.token !== undefined;
});

// Test task creation
const testCreateTask = tests(
    expectTrue([true])
)(async function testCreateTask(): Promise<boolean> {
    const token = await registerAndLogin();
    const resp = await client.post("/api/tasks", {
        title: "Test Task",
        description: "A test task",
        priority: "high"
    }, { headers: { Authorization: `Bearer ${token}` } });
    return resp.statusCode === 201 && resp.body.title === "Test Task" && resp.body.priority ===
});

// Test listing tasks
const testListTasks = tests(
    expectTrue([true])
)(async function testListTasks(): Promise<boolean> {
    const token = await registerAndLogin();
    await client.post("/api/tasks", { title: "List Task 1" }, { headers: { Authorization: `Beare
    await client.post("/api/tasks", { title: "List Task 2" }, { headers: { Authorization: `Beare
    const resp = await client.get("/api/tasks", { headers: { Authorization: `Bearer ${token}` }
    return resp.statusCode === 200 && resp.body.tasks.length >= 2;
});

// Test updating task status
```

```
const testUpdateTaskStatus = tests(
    expectTrue([true])
)(async function testUpdateTaskStatus(): Promise<boolean> {
    const token = await registerAndLogin();
    const createResp = await client.post("/api/tasks", { title: "Status Task" },
        { headers: { Authorization: `Bearer ${token}` } });
    const taskId = createResp.body.id;
    const resp = await client.put(`/api/tasks/${taskId}`, { status: "done" },
        { headers: { Authorization: `Bearer ${token}` } });
    return resp.statusCode === 200 && resp.body.status === "done" && resp.body.completed_at !==
});

// Test deleting a task
const testDeleteTask = tests(
    expectEqual([204])
)(async function testDeleteTask(): Promise<number> {
    const token = await registerAndLogin();
    const createResp = await client.post("/api/tasks", { title: "Delete Me" },
        { headers: { Authorization: `Bearer ${token}` } });
    const taskId = createResp.body.id;
    const resp = await client.delete(`/api/tasks/${taskId}`,
        { headers: { Authorization: `Bearer ${token}` } });
    return resp.statusCode;
});

// Test dashboard stats
const testDashboardStats = tests(
    expectTrue([true])
)(async function testDashboardStats(): Promise<boolean> {
    const token = await registerAndLogin();
    await client.post("/api/tasks", { title: "Stats Task" },
        { headers: { Authorization: `Bearer ${token}` } });
    const resp = await client.get("/api/dashboard/stats",
        { headers: { Authorization: `Bearer ${token}` } });
    return resp.statusCode === 200 && resp.body.total_tasks >= 1;
});

// Test unauthorized access
const testUnauthorizedAccess = tests(
    expectEqual([401])
)(async function testUnauthorizedAccess(): Promise<number> {
    const resp = await client.get("/api/tasks");
    return resp.statusCode;
});

// Test that token creation and validation round-trips correctly
const testTokenRoundTrip = tests(
    expectTrue([{ userId: 1, role: "admin" }]),
)(function testTokenRoundTrip(payload: Record<string, unknown>): boolean {
    const secret = process.env.TINA4_SECRET || "test-secret";
    const token = Auth.getToken(payload, secret);
    const decoded = Auth.validToken(token, secret);
    return decoded !== null && decoded.userId === payload.userId;
});

// Test that password hashing and checking works
const testPasswordHash = tests(
    expectTrue(["securePass123"]),
)(function testPasswordHash(password: string): boolean {
```

```
    const hash = Auth.hashPassword(password);
    return Auth.checkPassword(password, hash);
});

runAll();
```

Run the tests:

```
npm test
```

```
Running tests...

  TaskFlowTest
    [PASS] testRegister
    [PASS] testLogin
    [PASS] testCreateTask
    [PASS] testListTasks
    [PASS] testUpdateTaskStatus
    [PASS] testDeleteTask
    [PASS] testDashboardStats
    [PASS] testUnauthorizedAccess
    [PASS] testTokenRoundTrip
    [PASS] testPasswordHash

  10 tests, 10 passed, 0 failed (1.12s)
```

All green. The application works.

---

13. Step 11: Docker Deployment

Create `Dockerfile`:

```
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production
COPY dist/ ./dist/
COPY src/templates/ ./src/templates/
COPY src/public/ ./src/public/
COPY migrations/ ./migrations/
RUN mkdir -p data logs
ENV TINA4_DEBUG=false
EXPOSE 7148

HEALTHCHECK --interval=30s --timeout=5s --retries=3 \
  CMD curl -f http://localhost:7148/health || exit 1

CMD ["node", "dist/app.js"]
```

Create `docker-compose.yml`:

```
services:
  app:
    build: .
    ports:
      - "7148:7148"
    environment:
      - TINA4_DEBUG=false
```

```
      - TINA4_SECRET=${JWT_SECRET:-change-me-in-production}
      - TINA4_CACHE_BACKEND=redis
      - TINA4_CACHE_URL=redis://redis:6379
    volumes:
      - taskflow-data:/app/data
      - taskflow-logs:/app/logs
    depends_on:
      - redis
    restart: unless-stopped

  redis:
    image: redis:7-alpine
    restart: unless-stopped

volumes:
  taskflow-data:
  taskflow-logs:
```

Build and deploy:

```
tina4 build
docker compose up -d --build
```

Verify:

```
curl http://localhost:7148/health
```

```
{"status": "ok", "version": "1.0.0", "database": "connected"}
```

TaskFlow runs in production. Authenticated APIs. Real-time WebSocket updates. Email notifications. Cached dashboard stats. Tested. Dockerized.

---

14. The Complete Project Structure

```
taskflow/
■■■ .env
■■■ .env.example
■■■ .gitignore
■■■ package.json
■■■ package-lock.json
■■■ tsconfig.json
■■■ app.ts
■■■ Dockerfile
■■■ docker-compose.yml
■■■ src/
■   ■■■ routes/
■   ■   ■■■ auth.ts               # Registration, login
■   ■   ■■■ tasks.ts              # Task CRUD
■   ■   ■■■ dashboard.ts          # Dashboard stats + page
■   ■   ■■■ middleware.ts         # Auth middleware
■   ■   ■■■ queue-consumers.ts    # Email notification consumer
■   ■   ■■■ ws-tasks.ts           # WebSocket event handlers
■   ■■■ orm/
■   ■   ■■■ User.ts               # User model with auth methods
■   ■   ■■■ Task.ts               # Task model with relationships
■   ■■■ migrations/
■   ■   ■■■ 20260322000100_create_users_table.sql
■   ■   ■■■ 20260322000200_create_tasks_table.sql
■   ■■■ templates/
■   ■   ■■■ base.html             # Base layout
■   ■   ■■■ dashboard.html        # Dashboard page
■   ■   ■■■ emails/
■   ■   ■   ■■■ task-assigned.html # Assignment notification
■   ■   ■■■ errors/
■   ■       ■■■ 404.html
■   ■       ■■■ 500.html
■   ■■■ public/
■   ■   ■■■ css/
■   ■   ■   ■■■ tina4.css
■   ■   ■■■ js/
■   ■       ■■■ tina4.min.js
■   ■       ■■■ frond.min.js
■   ■■■ locales/
■       ■■■ en.json
■■■ data/
■   ■■■ app.db
■■■ dist/                        # Production build output
■■■ logs/
■■■ secrets/
■■■ tests/
    ■■■ TaskFlowTest.ts
```

Every file has a purpose. Every directory follows the convention. A new developer looks at this structure and knows where to find things.

---

15. What You Built

This chapter used every major concept from the book:

Feature	Chapter
Route registration (`Router.get`, `Router.post`, `Router.put`, `Router.delete`)	Chapter 2
Request/response handling	Chapter 3
Twig templates	Chapter 4
Database queries	Chapter 5
ORM models (User, Task)	Chapter 6
JWT authentication	Chapter 8
Auth middleware	Chapter 10
Email notifications (Messenger)	Chapter 16
Cache (dashboard stats)	Chapter 11
Frontend (tina4css dashboard)	Chapter 17
WebSocket (live updates)	Chapter 23
Testing (full test suite)	Chapter 18
Docker deployment	Chapter 34

---

16. What to Build Next

TaskFlow is a solid foundation. Here are ideas for extending it.

Features:

- **Task comments** -- Add a Comment model with a `taskId` foreign key. Display comments on the task detail page.
- **File attachments** -- Let users upload files to tasks. Store them in `data/uploads/` and serve them via a route.
- **Team management** -- Add a Team model. Users belong to teams. Tasks are scoped to teams.
- **Task labels/tags** -- Many-to-many relationship between tasks and labels for categorization.
- **Due date reminders** -- Use the queue system to schedule reminder emails 24 hours before a task's due date.
- **Activity log** -- Record every change to a task (who changed what, when) for audit trails.
- **Search** -- Full-text search across task titles and descriptions.
- **Calendar view** -- Render tasks on a calendar based on their due dates.

- **Mobile API** -- The API already works for mobile apps. Add push notification support via Firebase Cloud Messaging.

Technical improvements:

- **Rate limiting per user** -- Replace the global rate limiter with per-user limits.
- **Database upgrade** -- Switch from SQLite to PostgreSQL for better concurrency.
- **CI/CD pipeline** -- Add GitHub Actions to run tests on every push.
- **API documentation** -- Generate OpenAPI/Swagger docs from your route annotations.
- **Internationalization** -- Add `src/locales/` files for multiple languages.

---

17. Closing Thoughts -- The Tina4 Philosophy

You built a complete application. User auth. CRUD. Real-time updates. Email. Caching. Tests. Deployment. Your project has one dependency: `tina4-nodejs`.

No separate ORM package. No template engine package. No authentication library. No WebSocket server. No caching library. No testing framework. No CLI tool. No CSS framework. No JavaScript helpers. All built in.

One framework. Zero extra dependencies. Everything you need.

The same patterns work in PHP, Python, and Ruby. Same project structure. Same CLI commands. Same `.env` variables. Same template syntax. Learn Tina4 once. Use it everywhere.

Build things. Ship them. Keep it simple.

Release Notes

v3.13.138 (2026-09-24) - Safer requests, templates, and database operations

Licensing from this release. Tina4 is available under MPL-2.0, with separate commercial terms available from Code Infinity. Copyright Code Infinity. Previously published releases retain their original licences.

This release brings together the framework hardening work, database correctness fixes, measured-time guidance for the AI skills, and verifiable release artifacts.

Request and template boundaries. Frond escapes output by default and preserves only explicitly trusted output. Review templates that intentionally render HTML when upgrading; never mark untrusted input as safe. Header validation, request-body limits, security headers, CORS handling, model-field validation, and diagnostic redaction follow the shared contracts. Explicit Content-Type headers survive response conversion and binary bodies preserve bytes.

Credentials and development access. API clients retain credentials only for their configured origin, including uploads, downloads and streams. Use a separate client when authenticating to another service. Forwarded host and protocol require a trusted proxy. Development reads and writes enforce the raw-peer, Host and browser-origin boundary; remote non-MCP development access requires the dedicated token. The documented MCP transport API-key fallback is unchanged. File access checks resolved paths so symlinks cannot expose secrets or escape the project.

Database and service correctness. Statements that return rows retain those rows, filtered counts and write results agree across adapters, SQL placeholders preserve literal text, and ORM fields mapped to database columns read back correctly. Database operations and explicit transactions reserve their connection; a fully occupied pool fails clearly instead of sharing an in-use connection. Session transports reuse MongoDB connections and handle replica-set replies. Mail transports verify certificates and require STARTTLS when configured. Optional driver errors identify the needed install, and unconfirmed broker writes fail clearly.

Skills and release evidence. The maintainer skill estimates work from measured agent durations. Release packages carry checksums, an SPDX software inventory, and build provenance; release Actions are pinned to commits. The updated PowerShell skills installer is digitally signed. The ISO programme remains in progress; this release makes no certification or conformance claim.

The accompanying CLI 3.8.91 verifies update checksums before replacing an installed binary, refuses missing checksum manifests, and corrects generated Node.js package imports. It also includes the browser-opening and CI behavior fixes. Tina4 JS 1.7.2 ships the merged client hardening and verified browser bundle. Refresh the AI skills with `curl -fsSL https://tina4.com/install-skills.sh | sh` or the documented PowerShell installer.

v3.13.105 (2026-08-19) - Cross-API invariants, honest logs, portable tests

A bug release with teeth. Five audit bugs closed in the queue and ORM cache seams — the places where two spellings of the same intent quietly disagreed. Route inspection stops booting the app. The startup log stops lying about `@noauth`. Hot reload reaches the modules that captured the changed one. Firebird's migration ledger tolerates any case the driver hands back. And every framework developer skill gained a spine: it announces every step before taking it, and it warns with ■ when it is out of date.

Queue and ORM audit bugs — five fixed

- `Model.clear_cache()` **invalidates the DB layer too.** When both

`TINA4_AUTO_CACHING` and `TINA4_DB_CACHE` are enabled, a manual `clear_cache()` used to leave the DB-layer cache holding stale rows. It now cascades to `db.cache_clear()` on this model's bound connection.

- `Queue.retry()` **revives every dead letter.** The no-arg form used a

generator inside `any(...)`, which short-circuits after the first success. Now it materialises the loop so all N dead letters revive, not just one.

- `Job.retry()` **removes the dead-letter file.** Iterating `dead_letters()`

and calling `.retry()` on each used to leave the failed directory carrying every revived job. The next `dead_letters()` call re-reported them and consumers processed each job twice.

- **MongoDB** `retry_job(id)` **finds the dead letter.** The old filter looked

for `{_id: job_id, topic: self._topic, status: "failed"}` — three reasons that combination could never match a real dead-letter doc (fresh `_id`, `.dead_letter` topic namespace, `dead` status). The new filter reads the dead-letter namespace by `data.id`, deletes the record, and upserts the original back to pending.

- **MongoDB** `purge(status)` **returns a count.** Used to return `None` and

honour only the `pending` status. Now returns `deleted_count`, honours every status, and scopes the delete so `purge("pending")` no longer nukes completed or reserved docs under the same topic.

Route inspection scans, never boots

`tina4 routes` walks canonical route files without executing `app.py` or starting the server. The prior implementation booted the app on the same port and terminated whichever process was holding it — a nasty surprise for anyone running `tina4 routes` in a live shell. Closes tina4-python #104.

Startup log tells the truth about @noauth / @secured

Python decorators apply bottom-up, so `@post()` logged the route as `auth=required` a microsecond before the outer `@noauth()` flipped the flag. `@noauth()` and `@secured()` now emit a corrective `Route auth updated:` line whenever they change the auth default on a route another decorator already logged. Closes tina4-python #103, where a public login endpoint appeared to log its own security bug. Python-only — PHP, Ruby and Node do not log an auth field at registration time.

Hot reload reaches every module that imports the changed one

A route file that did `from src.orm.Todo import Todo` kept its stale `Todo` reference after the model changed — the API then lied about the schema on the next request. `_auto_discover` now cascades a re-import to every module that captured a binding from the changed one, recursive with a visited set, bounded to the discovery scope, fail-loud. Closes tina4-python #102.

Firebird migration ledger is case-agnostic

`tina4_migration` reads and writes work regardless of the case the Firebird driver returns for the `migration_name` column. Uses the atomic sequence pattern the other engines already have.

Test-harness fixes

- **SIGTERM port probe checks both interfaces.** The graceful-shutdown test

probed only `0.0.0.0` with `SO_REUSEADDR`, which succeeds on macOS while the framework's default `127.0.0.1` listener holds the port — a silent false negative on every Mac dev machine. Now probes both interfaces and refuses to conclude the port is free until both agree.

- **MySQL connect-timeout assertion tolerates driver cleanup time.** The

assertion compared the outer clock to the wrapper's rendered message. mysql-connector adds around 0.7 seconds of cleanup after aborting, so the two stopwatches legitimately differed. The assertion now parses the reported elapsed via regex and checks it against the configured bound, not against the outer clock.

Skills grow a spine

- **Announce before you act.** Every framework developer skill now instructs

itself to say what it is about to do — one line before each file write, migration, or dependency install — so the developer can stop the work before it spends their afternoon. Same rhythm across Python, PHP, Ruby and Node.

- ■ **stale-skill detection.** Every framework developer skill also checks

whether a newer skill is available (compared against the latest published skill version at `https://tina4.com/skills/<name>/version`, never against the project's framework version) and prepends ■ to every reply if it is behind. Your framework version stays your call; the skill just refuses to pretend its advice is current.

Upgrade notes

- No API changes. All fixes preserve existing signatures.
- Consumers iterating `dead_letters()` and calling `job.retry()` on each — now safe. Previously it produced duplicates on the next `dead_letters()` call.
- `Model.clear_cache()` under `TINA4_DB_CACHE=true` now also flushes the DB-layer cache. This is the expected behaviour.
- Lab full-suite verified: 25,263 tests across all four frameworks, zero failures, zero skips.

v3.13.104 (2026-08-17) - OIDC SSO and PostGIS land

Two big features, uniform across all four frameworks: a provider-neutral OIDC handoff for SSO logins, and a shared `Point` contract for PostGIS geospatial queries. Both wire through the same small API in Python, PHP, Ruby and Node.

OIDC / OpenID Connect — provider-neutral, session handoff

`Sso.from_issuer(...)` discovers a real OIDC provider (Keycloak, Auth0, Okta, anything speaking the standard) via its `.well-known/openid-configuration` endpoint and hands a validated identity into an existing Tina4 `Session`. The `login` / `callback` / `refresh` / `logout` cycle runs against the real provider — no mock. PKCE with S256 by default. Reserved SSO session keys (`Sso.PENDING_KEY`, `Sso.SESSION_KEY`) never leak through `session.all()`.

Opt in with `TINA4_SSO_ISSUER` plus the client id, secret and redirect URI.

PostGIS spatial support

`PointField` on an ORM model creates the right column for whichever engine you are on — PostGIS `geometry(Point, SRID)` on PostgreSQL, JSON on the others. `select_distance`, `intersects` and `bbox` predicates read the same way across engines but use each engine's native functions where available. GeoJSON in, GeoJSON out. Refuses hostile input (garbage geometry, injected column names, NaN or Inf ordinates) before any SQL runs.

Opt in by declaring a `PointField` on a model. Existing models unchanged.

Swagger is SSO-aware

The Swagger generator recognises OIDC-secured routes and emits the right `security` scheme in the OpenAPI 3.0.3 spec.

v3.13.102 (2026-08-15) - Unified client and CI hardening

Small release. Framework developer skills teach the unified `tina4 init` / `tina4 serve` workflow consistently across the four languages, and CI now verifies the downloaded native CLI asset before running the metrics handoff.

v3.13.103 (2026-08-16) - Metrics you can trust, releases that cannot lie

This release restores one public version across Python, PHP, Ruby, and Node.js. Version 3.13.102 reached PyPI and Packagist, but its RubyGems and npm packages still contained 3.13.101. Skip 3.13.102 and upgrade to 3.13.103.

What changed

• Dev-admin hands metrics work to the signed Tina4 client 3.8.76. Frameworks do

not keep a second metrics analyser or silently fall back to one.

• `has_referencing_test` means that test source refers to the file. It does not

claim that the test ran, passed, or covered the file.

• Frond expression parsing and evaluation are split into smaller internal steps,

while public APIs and the shared 84-case expression corpus remain unchanged.

• Tina4 skills lead with `tina4 init` and `tina4 serve`, and keep scaffolding

prominent in every language.

• Release workflows now reject a tag that differs from the source package

version before publishing.

No runtime packages were added. Language extensions remain extensions, not framework dependencies.

+## v3.13.101 (2026-08-14) - One AI client, four languages

One class. Three methods. No provider SDK. Tina4 applications can now call local models, OpenAI, and Anthropic through the same small API: `Ai.chat`, `Ai.complete`, and `Ai.embed`.

What ships

- The native `tina4 metrics` CLI now owns every code-health calculation. Framework-level metrics commands and quick census implementations are gone.
- Dev-admin keeps its metrics tab through two thin native-JSON adapters: full analysis and file detail.
- `Ai.chat` returns one `ChatResponse` with text, model, token usage, finish reason, and the raw provider response.
- `Ai.complete` sends one user message and returns its text.
- `Ai.embed` accepts one string or a list and preserves that input shape in its output.
- `{ stream: true }` returns an async generator of ordered OpenAI-compatible or Anthropic text deltas.

Failure is data

Hosted providers refuse to send a request without `TINA4_AI_KEY`. Connection and total-request deadlines stay separate. Retries cover connection failures, HTTP 429, and HTTP 5xx responses, but stop after the first streamed delta. Errors never include the key, prompt, or provider response body.

Upgrade notes

This release adds a new API. Vision, image generation, tools, agents, and conversation state remain outside this release. The client uses Node's built-in HTTP, HTTPS, and JSON modules; your dependency tree does not grow.

v3.13.100 (2026-08-14) - Frond keeps the whole page

This fast-follow release fixes template inheritance and puts a ceiling on Frond's caches. The template engine now rejects a second `{% extends %}` tag, preserves nested root blocks, and evicts stale compiled state instead of letting a long-running process grow without limit.

Template inheritance

- A template may declare one parent. A second `{% extends %}` now raises a clear error instead of changing the parent without warning (`831bd57`)
- The final block-substitution pass now tracks nesting depth. A block nested inside another root block no longer causes the outer content to disappear (`5a57300`)

Bounded caches

- Template, fragment, and expression caches now have size bounds and TTL sweeps. Long-running workers release stale entries while rendered output stays unchanged (`d1e6b42`)

Frond extension scope

- Class calls to `addFilter`, `addGlobal`, and `addTest` remain process-global. Instance calls now affect that renderer only, so one request or test cannot seed every Frond instance created later (`e10b830`)

AI skill installer

- Skill downloads retry transport failures and transient HTTP statuses. Permanent 4xx responses fail after one request, while a short GitHub raw-content outage gets another attempt (`b859bcf`, `4b75fe7`)

Release consistency

- The root package, five published workspaces, npm lockfile, and AI-facing guide now agree on `3.13.100`. A 15-case release guard checks every surface (`70ffce8`)

Upgrade notes

No valid template syntax changes. Code that relied on an instance extension leaking globally must register on `Frond` itself. A template with two `{% extends %}` tags was already contradictory; it now fails at the line that needs fixing. Cache eviction may trigger a reparse, but it does not change the rendered bytes.

The Frond AOT compiler is not part of this release. Parser and compiler parity for Ruby and Node.js remains planned for `3.13.101`.

v3.13.99 (2026-08-13) - Stability, parity, and secure by default

The biggest step yet on the road to **3.14 stable**. This release closes out Phases 1 through 5 of the v3 parity audit: roughly thirty breaking changes, and every one of them is a security fix, a parity fix, or a correctness fix, never a change made for its own sake. Two conformance grids, logging and database adapters, are now fully proven against real services in all four frameworks, and a small batch of genuinely new bugs gets fixed alongside them. Read "Possible breaking" before you upgrade: Node carries a real `Database.executeMany()` contract change and a `Log.warn` removal this time.

Security

Security now defaults on instead of requiring opt-in.

- Security headers emit by default, including `Content-Security-Policy: default-src 'self'`, which blocks inline scripts and third-party CDNs. Relax the policy with `TINA4_CSP`; HSTS emits only on HTTPS, when `TINA4_HSTS` is set (`aa73dbf`)
- The CSRF `403` body is unified to `{error, code, message, status}`, where Node used to send `{error: "CSRF_INVALID"}`. `TINA4_CSRF=true` now actually attaches the CSRF middleware, where it used to be inert, and a blank `TINA4_SECRET` fails closed now instead of minting a forgeable public-default token (`cc6642a`)
- The dev server binds `127.0.0.1` by default; set `TINA4_HOST=0.0.0.0` to expose it. A cross-origin `/__dev` mutation is refused, and `.env` is never served through the file endpoints (`97d4f22`)
- A symlink whose real path escapes the public directory is refused, dotfiles (`.env`, `.git`) 404 instead of serving, and `TINA4_PUBLIC_DIR` is honoured now (`77592ad`)
- `{% include %}`, `{% extends %}`, and `{% import %}` in a Frond template are confined to the templates directory. A path that escapes it, with `..`, an absolute path, or a symlink, now raises (`22e6057`)
- `tina4 serve` on a busy port no longer kills whatever holds it. It reclaims only a port held by an identifiable Tina4 dev server, refuses a foreign holder, and honours `TINA4_NO_TAKEOVER` / `--no-kill` (`ae7332d`)
- A hostile inbound `X-Request-ID` (CRLF, illegal characters, an over-long value) is sanitized to a fresh id instead of echoed back raw, closing a response-header and log-injection path (`0828904`)

• The dead `renderProductionError` function is removed; nothing called it. The dev error overlay now redacts `Authorization`, `Cookie`, and `Set-Cookie` headers plus secret-looking body fields, and caps the rendered stack at 50 frames (`f9afed3`)

• A repeated multipart file field now yields a list instead of silently dropping every upload but the last. The new safe-save helper rejects `..` and absolute filenames (`5a9bdbe`)

Data integrity

Footguns that used to fail silently now fail loud, or stop failing at all.

• An unparseable or unsupported MongoDB WHERE clause now raises instead of silently matching every document. A DELETE or UPDATE with no WHERE is rejected outright. Write an explicit WHERE, or call `truncate()` for the whole collection (`ad62c2d`)

• `truncate()` on a Mongo collection now actually empties it. It used to report success while leaving every document in place (`5d2afea`)

• MSSQL pagination converges on `OFFSET`/`FETCH` in all four frameworks; Node no longer uses `TOP` for page one (`2de3fa4`)

• Firebird write results are correct now: `db.insert()`/`db.update()`/`db.delete()` return real `affectedRows`/`lastId` values instead of missing or zero ones. `node-firebird` moves from a devDependency to an optionalDependency of `@tina4/orm`, so it installs only when you actually use Firebird (`47d3d4e`)

• `handle.stop()` on a background task now returns a boolean instead of `void` (`fe6e069`)

ORM and validation

Six fixes bring the ORM's write path, validation, and generated schema into agreement across all four frameworks, plus two genuine additions: declarative relationships now work in Node at all.

• **Declarative relationships now function and lazy-load.** Before 3.13.99, a field declared with `type: "foreignKey"` never attached its accessors; only the imperative `post.belongsTo(Author, "author_id")` form worked. Both now work (`ad51731`)

• `toDict()` now includes an imperatively-loaded relation that used to be silently omitted whenever the table name differed from the lowercased model name. The imperative `hasMany` default row cap changes from a silent 100 to the whole result set (`4126c4a`)

• `toDict()`/`toJson()` now log a warning when a declared relation is skipped because it was not eager-loaded, instead of dropping it silently (`5b5cdcc`)

• AutoCrud PUT now validates the request body, where it used to validate only on create. An update with a type, length, pattern, or required violation now gets a `422`, and the `isUpdate` partial-update mode no longer demands unrelated fields. The regex validation message becomes `"does not match the required format"` (`bf0ea6a`)

• `createTable()` now injects the `is_deleted` column for a soft-delete model that does not declare it itself (`f33ed9b`)

• A soft-deleted child is no longer returned through relationship traversal, lazy or eager (`ad51731`)

• A REST list `?page` below 1 now clamps to page 1, instead of handing a negative offset to the driver; an oversized `?limit`/`?per_page` is capped at 100 (`feaf660`)

• The write body on an AutoCrud create or update is now allow-listed: `is_deleted` is never client-writable, and a client-supplied primary key is stripped on both create and update, closing a mass-assignment hole and an IDOR-shaped redirect (`1c4f7cd`)

Request model - the big one

`req.params` is route-params-only now, in all four frameworks, closing a param-pollution surface in the other three languages where the query string or body could shadow a route parameter. Client input lives only in `req.query` and `req.body`. A malformed JSON body returns the raw string it failed to parse, and an empty body returns each language's native null (`c3acf40`).

Migrations

• `rollback` is fail-safe now: a missing `.down.sql` file or a failed down script raises and leaves the `tina4_migration` ledger row in place, instead of deleting the tracking row and leaving the schema applied but untracked

• The `tina4 migrate` CLI runs the same ORM `migrate()` now (transactional, a robust `;` split, Firebird/MSSQL idempotency skips), replacing its own naive `split(";")` re-implementation. A mid-file failure now rolls back on a transactional-DDL engine (`9e79da6`)

HTTP

• Your responses now gzip-compress when eligible (body over 1024 bytes, `Accept-Encoding: gzip`, a compressible content type); a cacheable 200 gets a strong ETag and a matching `If-None-Match` gets a 304. The static-file ETag format is unified to `W/"<size>-<mtime>"` on all four frameworks, so a cache revalidates once instead of on every deploy (`ce7e672`)

• A `403` now returns a real negotiated body (HTML or JSON), where it used to return a bare empty body. `404` carries a `request_id` too (`a03d752`)

• The OpenAPI spec now includes routes registered or hot-reloaded after boot, where it used to freeze at a boot-time snapshot. The spec converges on one shape across languages: a secured operation documents a `401`, and `summary`/`tags` always populate. The Swagger UI CDN default moves to jsdelivr, off unpkg (`cc8c739`)

• Python, PHP, and Ruby's route-group prefix join is normalized to match your existing behaviour, so a route no longer mis-registers on a bare concatenation or an uncollapsed double slash (`94c6cb0`)

Dev tooling

• `TestClient` now dispatches through the real request pipeline, where it used to short-circuit around it. A route middleware that used to run on a would-be-401 request no longer runs, and session auth is genuinely exercised now; a test leaning on the old low-fidelity client may see a different outcome, because that outcome is now correct (`32a468c`)

• `sessionAutoStart` now uses `appendHeader`, so a route that also sets a cookie no longer has it clobbered. This was a real bug on the live server too, not only in tests (`32a468c`)

• The banner and health check report the real framework version now, instead of `0.0.0` in a relocated or published layout (`925a3a0`)

• The inline `tests()` descriptor builders are renamed: `assertEqual`/`assertRaises`/`assertTrue`/`assertFalse` become `expectEqual`, `expectRaises`, `expectTrue`, `expectFalse`. `tina4 test` now discovers and runs the inline surface with a real exit code (`e4202b5`)

Proven in all four

Two conformance grids close out fully proven, tested against real services in every framework, no mocks:

• **Logging.** A real per-language runner now exercises the shared logger contract end to end. `Log.warn` **is removed - use** `Log.warning`**.** Node also gains `TINA4_LOG_FILE_LEVEL`, an independent level for the file sink from the console's `TINA4_LOG_LEVEL` (additive, defaults to `ALL`), and environment variables now resolve once at startup instead of being re-read on every call (`15c6941`)

• **Database adapters** (ADR-0044). A real runner against SQLite, PostgreSQL, MySQL, MSSQL, and Firebird proved every adapter implements the same interface; the interface and six adapters were missing `getDatabaseType`/`autocommit`, caught by the typecheck. `Database.executeMany()` **used to loop per row, returning one result per row; it now delegates once through** `adapterExecuteMany()` **and returns a single aggregate** `DatabaseResult`**, matching** `insert`**/**`update`**/**`delete`**.** The rewiring surfaced two more real bugs: `CachedDatabaseAdapter` had no `executeManyAsync` passthrough, so a standalone `executeMany` against any network adapter threw, and `SQLiteAdapter`'s `executeMany` issued an unguarded raw `BEGIN` (`fc52fe8`)

Bug fixes

• A route path containing a literal parenthesis, like `/products/(sale)`, now matches correctly everywhere. Python and Ruby were already correct; `router.ts`'s `compilePattern()` compiled the literal characters as regex syntax instead of escaping them (`57f375d`)

Possible breaking

Read these before you upgrade. Every entry here is a security, parity, or correctness fix, none is a change made for its own sake.

• `req.params` **is route-params-only.** Read query-string values from `req.query` and body values from `req.body`. This is the single largest behaviour change in the release.

• `Log.warn` **is removed.** Use `Log.warning` at every call site.

• `Database.executeMany()` **now returns one aggregate** `DatabaseResult`, not an array of one result per row. Update any code that iterated the old per-row array.

• **Security headers, including CSP, emit by default.** An app depending on inline scripts or a third-party CDN needs `TINA4_CSP` to relax the policy.

• **The CSRF** `403` **body shape changed** from `{error: "CSRF_INVALID"}` to `{error, code, message, status}`.

• `TINA4_CSRF=true` **now actually attaches the CSRF middleware.** If you set it and never noticed CSRF enforcement, it enforces now.

• **A blank** `TINA4_SECRET` **fails closed.** Set a real secret; the framework no longer falls back to a guessable public default.

• **The dev server binds** `127.0.0.1` **by default.** Set `TINA4_HOST=0.0.0.0` to expose it on your network.

• **An unparseable Mongo WHERE now raises instead of matching everything.** Add an explicit WHERE, or call `truncate()`.

• `handle.stop()` **returns a boolean**, not `void`.

• `node-firebird` **is now an optionalDependency**, not a devDependency, of `@tina4/orm`.

• **AutoCrud PUT now validates the body.** An update payload that previously skipped validation may now fail with a `422`.

• **The imperative** `hasMany` **no longer caps at 100 rows**, and a declared relation Node used to silently drop from `toDict()` now appears (or logs a warning if still not eager-loaded).

• `createTable()` **adds** `is_deleted` **for a soft-delete model automatically.** A model that already declares the column is unaffected.

• `?limit`**/**`?per_page` **caps at 100.** A client can no longer request the whole table in one page.

• **AutoCrud never accepts** `is_deleted` **or a client-supplied primary key in the write body.**

• **The static-file** `ETag` **format changed**, so every cache revalidates once on upgrade.

• **A** `403` **now returns a real body**, where it used to return nothing.

• `TestClient` **dispatches through the real pipeline now.** A test that depended on the old short-circuit may see a different, correct, outcome.

• **The inline testing descriptors are renamed** `assert*` to `expect*`, and `tina4 test` now actually runs them.

Coming in 3.13.100

Frond's compiler, extensibility, auto-escaping, sandboxing, and caching work (features 48 through 60) is deferred to the 3.13.100 fast-follow, alongside a refreshed Carbonah benchmark harness and a configurable database column-name casing option.

v3.13.98 (2026-08-11) - Skills discipline

A maintenance release. No API changed, and no behaviour changed in your app.

The bundled AI coding skills - the ones that install with `curl -fsSL https://tina4.com/install-skills.sh | sh` - moved onto one shared plan discipline. A plan file has one format (Scope, Tests, Bugs, Commits), and a checkbox is ticked only when the work is verified green at HEAD, never on a claim. That keeps an agent's plan file honest, so it always reflects what actually shipped.

The framework package also shed internal dead weight: duplicate design-system source files that were never compiled or served at runtime. Nothing your code imports or renders changed.

Nothing to upgrade. If you use the Tina4 AI skills, re-run the installer.

v3.13.97 (2026-08-07) - Behaviour corrections

A small bug-fix release on the road to **3.14 stable**. No new surface, no renames. Two behaviours come back into line with the Python reference, and two correct ones get locked so they stay that way. `flash(key, null)` reads and clears instead of storing a null. A RabbitMQ queue refuses an operation it cannot honour instead of draining itself. Read "Possible breaking" before you upgrade.

Sessions

- `flash(key, null)` reads and clears. `null` is the get sentinel, so passing it returns the stored value and removes it, the same as the one-argument call. Node used to store `null` under the key. The other three frameworks already read and cleared; Node matches them now (`89e0a02`)

Queues

- `clear()` and `purge()` on a RabbitMQ backend raise a named refusal. A broker delivers messages to consumers and cannot address them by status, so neither call can be honoured. Both drained the live queue and destroyed its messages before; they raise now, rather than throw away data you meant to keep (`586b092`)

Proven

- `forceDelete()` is locked by a regression test. Node was already correct; the test keeps it that way (`6b8cc2e`)
- `distinct()` dedups values by equality, so two equal dates collapse to one row. A parity test now locks it across all four frameworks (`93cc61d`)

Possible breaking

- `clear()` **and** `purge()` **raise on a RabbitMQ queue.** They drained the live queue and destroyed every message before. Catch the refusal, or guard the call. A database-backed queue still clears and purges as before.
- `flash(key, null)` **no longer stores null.** It reads and clears now, because `null` is the get sentinel. Code that passed `null` to park a null under the key finds the key cleared instead.

v3.13.96 (2026-08-07) - One paginate envelope, one message shape

Another step toward **3.14 stable**, and the theme is still parity: the same call means the same thing in Python, PHP, Ruby and Node. This release settles two subsystems that had drifted into four shapes. Pagination returns one envelope of seven keys with a true total. The Messenger IMAP path returns one message shape, addresses mail by a real IMAP UID, and hands back attachments you can write straight to disk. Some of these are behaviour changes, so read "Possible breaking" at the end before you upgrade.

Pagination

- `toPaginate()` takes no arguments and derives every field from the query that ran; pass one and it throws (`c1b8ae6`)
- The envelope is exactly seven snake_case keys: `records`, `total`, `page`, `per_page`, `total_pages`, `limit`, `offset` (`c1b8ae6`)
- Node carried the widest envelope of the four, thirteen keys. The camelCase `perPage` and `totalPages`, plus `data`, `count`, `has_next` and `has_prev`, are all dropped (`c1b8ae6`)
- `.count` and the envelope `total` are a true `COUNT(*)` on both read paths. `QueryBuilder.get()` left `count` at the row count and now returns the true total, matching `db.fetch()` (`c1b8ae6`)
- The AutoCrud REST list endpoint returns the same seven keys (`28dcb49`)

```
// Fetch the page you want, then describe it. No arguments.
const result = await db.fetch("SELECT * FROM orders", [], 20, 40);   // 20 per page, page 3
return res.json(result.toPaginate());
// { records: [...], total: 250, page: 3, per_page: 20,
//   total_pages: 13, limit: 20, offset: 40 }
```

Messenger

- `send()` carries `{success, message, id}` on both paths. On failure `id` is present as `null` rather than omitted, so a caller reads one shape from both branches (`b1d16b4`)
- `inbox()` items are exactly `{uid, subject, from, to, date, snippet, seen}`, `date` is ISO-8601, and `snippet` is real decoded body text where it was always empty (`b1d16b4`)
- `read()` returns `date` as ISO-8601 and gains an `attachments` array (`b1d16b4`)
- IMAP credentials are separate from SMTP, with an `imapEncryption` constructor option (`b1d16b4`)
- `deleteMessage()` is renamed `delete()`, the one cross-framework name; `deleteMessage` stays a deprecated alias for one release. New: `markUnread()` and `sendTemplate()` (`b1d16b4`)
- The `uid` from an IMAP read is a real IMAP UID, so it still addresses the right message after another client expunges the mailbox (`c1b28fd`)
- `read()` folds each attachment's decoded bytes into its attachment item, so you write it straight to disk (`6fbf263`)

Swagger

- `components.schemas` is keyed by the model class name (`Item`), not the table name (`items`), so a generated client gets `class Item` (`c9e0ec9`)
- `info.version` defaults to `1.0.0` and `info.description` to an empty string (`c9e0ec9`)
- A model write documents only `200`; the unconditional `422` and inferred `201` are dropped from the document, and `401` still appears on a secured route (`c9e0ec9`)
- `operationId` keeps leading underscores, so `/__health` and `/health` stay distinct (`c9e0ec9`)

Migrations

- `code` is the canonical kind for a code migration, the same word in all four frameworks. An unknown kind throws and names the valid ones (`fddb3e7`)

Server

- An oversized request body answers `413`, not `500` (`d3e14e2`)
- `TINA4_PORT` is the port variable and wins; bare `PORT` still works but is deprecated and goes in 3.14 (`a33930f`)
- Cluster mode stopped killing its own workers, and the server warns when the event loop is blocked (`84a2a2b`)

Build and skills

- The framework SCSS compilers are gone; the `tina4` Rust CLI owns SCSS now (`68fc0e9`)
- tina4-css is pinned to one artefact and one URL (`6f45ae3`)
- The skills point at the ADRs and make `tina4 metrics`, Carbonah and mcp.tina4.com the lean, green, grounded workflow (`11bfae6`)

Proven in all four

The Messenger, Swagger and pagination behaviours are locked by machine-checked contract suites that run against real services in every framework: a live GreenMail server for the mailbox, a real 250-row SQLite table for pagination. No mocks.

Possible breaking

Read these before you upgrade. Each affects an app that relied on the old shape:

- `.count` **and** `toPaginate().total` **are the true total.** Node's `QueryBuilder.get()` left `count` at the row count; it is now a `COUNT(*)` for the filter. Read `records.length` for the page size.
- `toPaginate()` **takes no arguments.** The old `toPaginate(page, perPage)` form throws. Fetch the page, then call `toPaginate()`.
- **The paginate envelope is seven keys.** The dropped Node keys `data`, `count`, `perPage`, `totalPages`, `has_next` and `has_prev` are gone; move to the seven canonical keys.

• **A Messenger `uid` is a real IMAP UID.** Discard any `uid` stored by an older version and re-read it; the old values were never stable across an expunge.

• `send()` **and** `read()` **changed keys.** `send()` returns `id` as `null` on failure where it was omitted; `read()` dates are ISO-8601. `deleteMessage()` is now `delete()`, the alias is deprecated.

• **Swagger defaults moved.** `info.version` is `1.0.0`, `info.description` is empty, schemas are keyed by class name. `TINA4_SWAGGER_VERSION` and `TINA4_SWAGGER_DESCRIPTION` still override.

• **An unknown migration kind throws** instead of being ignored.

• **The framework no longer compiles SCSS.** Use the `tina4` Rust CLI.

v3.13.95 (2026-08-06) - Preparing for the 3.14 stable release

One of several releases still to come before **3.14 stable**. The theme is parity: the same call now means the same thing in Python, PHP, Ruby and Node. Method shapes and inputs are unchanged - this is bug fixes, optimizations, and a much harder test suite behind them. See "Possible breaking" at the end for the handful of behaviour corrections that could affect an app relying on the old, wrong result.

Queues

• The dev dashboard queue panel lists the store it counts (`b07a9d9`)

• The AMQP vhost is read as the path segment (`815fad9`)

Logging

• An explicit argument beats the environment, which beats the default (`9af3c52`). `configure()` no longer writes `process.env`, so it cannot clobber the value for child processes

• `Log.reset()` returns a long-lived process to environment-driven resolution (`d04edfc`)

Databases

• Every connect is bounded by `TINA4_DATABASE_CONNECT_TIMEOUT` (`72f7074`)

• Where a driver has its own connect timer, it wins and its message is translated (`e196c00`)

• `count()` returns the true total, not the last page's row count (`cb831c3`)

• `toPaginate` reports the page it is on (`8ba6c82`)

• Case-insensitive primary-key match; Firebird `tableExists` matches both spellings (`9f31a35`)

• Firebird: a column name is folded back only when Firebird folded it (`ac8528d`)

• `node-firebird` bumped past a 13% authentication failure, and pinned (`688cf53`)

Sessions

• The Mongo session database default is `tina4`, matching the others (`d068c40`)

- An environment URI no longer overrides an explicitly configured Mongo host and port (`696731c`)
- The session key prefix contract is locked in (`3e5a30e`)
- Firebird's DDL branch is measured, not inferred (`08ed08e`)

Document store

- `find()` is deferred, and there is one sort contract (`abfc1a9`, `3163b3e`)

Local by default, production by environment variable

The document store runs on a local SQLite file with no configuration and no services, and becomes a real MongoDB collection when you set one environment variable. The call sites are identical either way. The same shape holds across the framework: SQLite is the default database, the queue is file-backed until you point it at a broker, and the cache is in-memory until you give it a URL.

Bulk insert

Pass a list of rows to `insert()` and the framework builds one prepared statement and runs it inside a single transaction on a single connection, instead of a round trip per row. Same call in all four languages.

```
$db->insert("orders", [
    ["customer_id" => 1, "total" => 9.99],
    ["customer_id" => 2, "total" => 14.50],
    ["customer_id" => 3, "total" => 22.00],
]);
```

Tooling

The `tina4` CLI reached 3.8.67 over this cycle:

- `tina4 init` now wires the project up rather than printing instructions for it
- `tina4 deploy docker` gained `--runtime cli|fpm|swoole` for PHP
- The skills installer worked on macOS but installed nothing on Debian/Ubuntu. Fixed
- Skills install for Codex as well as Claude
- `tina4 metrics` gained cross-file duplication detection, Rust support, and now refuses a file it cannot parse instead of scoring it
- `tina4 serve` no longer opens a duplicate browser tab

The documentation site you are reading runs on **tina4press**, our own zero-Vue static site generator built on tina4-js. 271 pages, no framework runtime on the page.

Possible breaking

Method shapes and inputs did not change. These are behaviour corrections, so they only affect an application that depended on the previous, incorrect result:

- `count()` **in Ruby and Node** returned the number of rows the last page

produced. It now returns the true total. Code that treated it as a page size will see a different number.

- **The AMQP vhost** was read as `"/"` plus the path segment, so

`amqp://host/production` looked for `//production`. If you worked around this by naming your vhost with a leading slash, remove it.

- **Configuration precedence.** An explicit argument now beats the environment.

If you both set `TINA4_LOG_DIR` and passed a directory to `configure()`, the code now wins. Setting only one is unaffected. Ruby deployments that read `tina4.log` from the project root should read `logs/tina4.log`.

- **The Mongo session database default in Node** is now `tina4`, matching the

other three. Set it explicitly if you relied on the old default.

- **An unknown connection scheme, or an unknown** `TINA4_SESSION_BACKEND`**,**

raises instead of silently falling back to SQLite or to file sessions.

- **A missing MongoDB driver** raises rather than falling through.

v3.13.94 (2026-07-29) - A write with no filter is an error

`update()` and `delete()` with no WHERE clause used to be accepted. Two of the four frameworks then wrote every row in the table. The other two changed nothing and said they had succeeded. Neither answer is a write, and neither told you which one you got.

An unfiltered write now refuses to run and names what is missing. `truncate()` is the explicit spelling for the case where you really did mean every row.

Breaking. Read the migration note below before upgrading.

The write path (BREAKING)

A write needs a target. `update(table, data)` with no filter now takes the primary key out of `data` and uses it as the WHERE clause. With neither a filter nor the complete primary key, it raises and names the columns it wanted. `delete(table)` with no filter raises too. `truncate(table)` is the whole-table spelling, and it always was.

A failed write raises rather than returning a falsy result the caller never inspected.

Primary keys are read as a LIST, because a key can span several columns and every one of them belongs in the WHERE. Match a composite key on its first column alone and you hit every row that shares that value: the same data-loss shape the fix exists to prevent. Two of the four SQLite drivers had a second bug underneath that one. `PRAGMA table_info` returns `pk` as the 1-BASED POSITION within the key, not a boolean, so a composite key reports `pk=1, pk=2`. Code that tested `pk == 1` saw a key one column wide and truncated the rest.

Migration. Two changes to make, both mechanical:

- An `update()` that relied on the primary key travelling inside `data` still

works, and now works on composite keys. An `update()` with neither a filter nor a full primary key was already broken. Add the filter.

- Replace a deliberate delete-everything with `truncate(table)`.

Nothing else in the write path moved. A filtered write behaves as it did before.

The Frond sandbox revokes capability, it does not skip a step

`sandbox()` filtered the tag and filter lists but let a denied name fall through to the default path, so a template could still reach an escape filter it was never granted. A denied filter cannot confer safety now, and the tag gate collapsed to one check at dispatch instead of several that could disagree.

The shared render corpus grew from 72 cases to 84, covering `|safe` and `|escape`. It is one committed fixture with one answer key, byte-identical across all four frameworks, so a divergence is a failing test rather than a discovery six months later.

Messenger: one name, one signature

The dev branch and the send branch had drifted into two shapes of the same method, and capture keyed off debug mode rather than whether a transport was available. One signature behind one name now, and capture follows availability.

Base images that boot

No CI had ever built or run a base image. Two of the four had therefore never served a request at all, and the two that did boot were each wrong in their own way. `docker run` is a supported entry point, so all four are now gated in CI on Docker Hub, on `/health`, and on the version they actually serve.

The `FROM` to `COPY` recipe for adding your own database driver is documented in all four image headers, and in one case that documented line did not work until this release. An install instruction nobody runs is a claim, not a feature.

Frond, measured against the engines it replaces

The benchmark page carried seven categories and none of them timed a template. That absence flattered us, because template rendering is the one axis where Frond competes head-on with the engine it replaced. It is a headline category now, and the numbers are not kind.

Every engine renders the same page: a 20-row product list with a loop, an index, an even/odd class, an uppercase filter, two-decimal money, and a conditional footer. Output is compared byte for byte and proven identical before any timing counts, so nothing here is a strawman.

Frond loses in all four languages, on its own compiled path rather than the interpreter fallback. The competition compiles a template to code the host runtime optimises. Frond walks a tree and calls back into engine primitives per hole. What Frond buys is the zero in the dependency column, and one template language that renders the same across four runtimes.

The per-language multiples are in each framework's own note below. Publishing them costs us the comparison and buys the reader a real number.

Also in this release

MQTT test infrastructure locks its password file to `0600` and the broker's own user, so a run cannot leave a world-readable credential behind.

Node.js specifics

Node's write failure was the quiet one. It built `UPDATE t SET ... WHERE` with an empty clause, caught the SQL error, and resolved to `{ success: false, affectedRows: 0 }`. A caller who did not inspect the result believed the write had landed. Destructive in two of the others, silent here, and invisible in both. `sqlite getColumns` also read `r.pk === 1`, the same 1-based position bug Ruby had, so composite keys arrived one column wide.

`pop()` no longer reports a real failure as an empty queue. A dead broker, an authorization failure and a genuinely idle topic were all `null`, so a consumer polled an "idle" queue forever and logged nothing.

Session handlers report the backend's cause. Each runs its command in a short `node -e` child, and every handler threw `execFileSync`'s own message, which embeds the entire generated script: a refused Valkey connection produced a 4302-character error, MongoDB 8264, with `connect ECONNREFUSED` nowhere inside. The children were already writing the reason to stderr. Nothing read it.

The `redis` driver's default reconnect strategy meant a refused connection never rejected, so the child hung until its timeout and reported "timed out" when the truth was "connection refused". Disabled for a child that runs one command, that is a better message and five seconds less latency per request while Redis is down.

`@tina4/*` was unimportable under plain node, because the exports map pointed at TypeScript source. The packages ship real `.d.ts` files now instead of pointing types at source, which is the second time that defect has been fixed and the reason a build now runs before the tests. The base image shipped every optional driver, because `packages/orm` declares `mongodb`, `mysql2`, `pg` and `tedious` as optionalDependencies and npm installs those by default. `tedious` pulls in `@azure/identity`, which pulls in the Azure and AWS SDK surface: 61 of 282 lockfile entries were driver-related, in an image whose default database is the built-in `node:sqlite`. SQLite only now, 288 MB down to 181 MB, matching what the other three already did.

The documented `RUN npm install pg` did not work before this release, which is the half that mattered. `/app` was a workspace root, so any npm operation in a derived image re-resolved the whole monorepo: a 495 MB image with `pg` still unloadable. npm also owns `node_modules` and pruned `@tina4/*` out of it, so `import "@tina4/core"` threw `ERR_MODULE_NOT_FOUND`. The framework lives in `/opt/tina4/framework` now and the runtime manifest declares it.

`@tina4/orm` and `@tina4/swagger` were thirteen versions behind the rest of the workspace. They are in lockstep again.

Frond: 8.90x slower than Nunjucks and 7.64x slower than EJS on identical output. Both compile a template into a real JS function and let V8 optimise it.

v3.13.92 (2026-07-27) - One algorithm, from the env var to the middleware

`getToken` read `TINA4_JWT_ALGORITHM`. `validToken` read it. `authMiddleware` ignored it and passed the string `"HS256"` instead.

That mismatch was harmless while HS256 was the only algorithm Node could sign. This release adds HS384 and HS512, which turns the hardcoded default into a live breakage: tokens minted HS512, middleware verifying HS256, every valid token refused. It is fixed in the same change that made it reachable. Three reported issues, one shared contract, all four frameworks.

The header names the algorithm that signed

The digest was hardcoded to `sha256` while the header carried whatever `TINA4_JWT_ALGORITHM` said. An app configured for HS512 emitted a token claiming HS512 over a 32-byte HMAC-SHA256 signature, and any verifier that reads the header and computes what it names got different bytes and rejected the token.

The digest now comes from a lookup keyed by the resolved algorithm, so the `alg` in the header is always the one that produced the signature. HS256, HS384 and HS512 all sign and verify through `node:crypto`, with nothing new installed.

The assertion that catches this measures the signature's byte length: 32, 48, 64. Comparing one algorithm's token against another's proves nothing, because the header is part of the signing input, so two tokens differ either way.

`RS256` is kept. Node ships `node:crypto`, so RSA signing costs nothing here, and the same holds for PHP with ext-openssl. Python and Ruby would need a third-party package, and the core takes no dependencies, so `RS256` is a documented PHP and Node extra rather than part of the cross-framework contract. The other two are not getting it.

authMiddleware stops overriding your configuration

`authMiddleware(secret?, algorithm = "HS256")` shadowed the environment for every route it guarded. Nothing broke while HS256 was the only option, and with HS384 and HS512 now supported it would have broken every app that moved off HS256.

The parameter no longer carries a default, so an unset argument resolves through `TINA4_JWT_ALGORITHM` exactly as `getToken` and `validToken` do. Pass one explicitly and it still wins.

authenticateRequest honours the overrides it accepts

`authenticateRequest(headers, secret?, algorithm?)` took both arguments and dropped them. The body called `validToken(token)` with no arguments, so a caller asking to verify against a particular secret silently got the environment's instead, and `algorithm` defaulted to the literal `"HS256"`, shadowing `TINA4_JWT_ALGORITHM` the same way `authMiddleware` did.

Both are now forwarded. Python, PHP and Ruby already honoured them, so this was the last framework where the same call gave a different answer.

The test covering this used to assert the old behaviour - that the secret parameter was ignored and the environment always governed. That documented a defect as a contract and is why the divergence survived as long as it did. It now asserts the override works, with a negative case alongside it so the positive one cannot pass by accident.

nbf is enforced, with 60 seconds of leeway (nodejs#39)

A token stamped not-valid-until-noon was accepted at nine. `validToken` checked `exp` and walked straight past `nbf`.

It now refuses a post-dated token until its `nbf` arrives, and accepts one up to `JWT_LEEWAY_SECONDS` early. RFC 7519 allows a small leeway, and without one a token minted on a host a second ahead is rejected for nothing at all. An `nbf` that is present but not a number is rejected rather than read as no constraint.

A token carrying no `nbf` is unconstrained, which is what keeps this non-breaking for every token already in circulation. `getToken` does not stamp one, matching Python and PHP. Ruby used to, and no longer does, so all four now issue the same claims.

The header cannot choose the verifier

`validToken` now rejects a token whose header names any algorithm other than the expected one, `alg: "none"` included, before it spends a single HMAC.

Node was not exploitable here. `verifySignature` always used the configured algorithm and never read the header to pick one, so a forged header changed the signing input and broke the signature anyway. What the check buys is parity with Ruby, which always had it, an exit before the wasted work, and a lock-in test: a future refactor that starts trusting `header.alg`, which is the classic algorithm substitution mistake, now fails four tests. That mistake has its own CVE list. Now it has a test here.

A misconfigured algorithm throws (Breaking)

`validToken` resolved the algorithm inside its own `try`, so a typo in `TINA4_JWT_ALGORITHM` was caught and returned `null`. Every request got a 401 and no explanation. `getToken` threw on the same value, so the two paths disagreed about what a misconfiguration means.

The fail-closed argument does not survive contact with that: `getToken` already threw, so a typo surfaced at login regardless. Swallowing it in `validToken` never avoided the error, it only made the rejection silent. A misconfigured algorithm is a deployment error, not a bad token, and both paths now say so with a message naming the variable and the supported set.

A malformed or forged token still returns `null`, exactly as before. Only the configuration error escapes.

Migration: set `TINA4_JWT_ALGORITHM` to `HS256`, `HS384`, `HS512` or `RS256`, or leave it unset for the HS256 default. A value that used to become a silent 401 now throws and tells you why.

The tests that hold it

Sixty-nine real assertions, a positive and a negative case per issue, no mocks and no doubles. Signatures are checked against an independently computed `node:crypto` HMAC and against the digest's raw byte length. Each check ran against the old code first and was watched to fail: forcing `sha256` for every HMAC algorithm fails exactly the five digest tests, disabling the `nbf` check fails seven, and a header-trusting verifier fails the four pinning tests. Full suite on macOS with Node 25: 6,005 passing across 191 files, typecheck green.

v3.13.91 (2026-07-27) - The offenders list finally points at code worth fixing

`tina4 metrics` ranked `public/js/frond.js` as the worst code in the framework, at cyclomatic complexity 191. Second place went to `register_dev_tools` at 139. Neither number was real. The first belongs to a file wrapped in one anonymous function. The second belongs to a registrar that declares twenty small handlers inside itself.

Both scores were the same arithmetic mistake, and it had been in every framework since the metrics module shipped.

A function is no longer charged for the functions inside it

Complexity was measured across a function's whole span. A branch inside a nested function landed on that function and on every function wrapping it. Count the same branch twice, three times, four, once per level of nesting.

The deeper the nesting, the worse the lie:

```
def outer(a):
    def inner1(x):
        if x: return 1
        if x > 2: return 2
        return 3
    def inner2(y):
        if y: return 1
        if y > 2: return 2
        return 3
    return inner1(a) + inner2(a)
```

`outer` branches on nothing. It scored 5. It now scores 1, and each inner keeps its own 3. The branches moved to where they belong. None went missing.

Python and the Rust engine walk a real syntax tree, so they stop descending at a nested function or class. PHP, Ruby and Node scan text, where that surgery would be three risky rewrites. They apply an identity instead:

```
own(F) = raw(F) - sum over direct children C of (raw(C) - 1)
```

A raw score is 1 plus every decision in the span, so `raw - 1` is the total decision count of a whole subtree. Subtract that for each direct child and what remains is the function's own work. It needs only the line and LOC each extractor already reports.

Closures, blocks, lambdas and arrow functions stay exactly where they are. None of them appear in the function list, so nothing subtracts them, and their decisions remain with the function that contains them.

Breaking: every complexity number drops, and so does every file total. A `tina4 metrics --fail-on` gate that failed on an inflated score may now pass. Run the gate once before you trust the old threshold, and lower it if the inflation was holding your ceiling up.

Ruby complexity was double what it should have been

In the tree-sitter Ruby grammar, `if`, `unless`, `while`, `when` and `rescue` name two things: the construct, and the keyword token inside it. The engine matched on the name alone, so it counted both.

`return 1 if y` scored 3. It has one branch.

Every Ruby number the engine produced came out at roughly double. The fix ignores anonymous nodes, which is what a bare keyword token is. The engine and `metrics.rb` now agree on the same file, method for method.

LOC means one thing again

Every framework counted file LOC as code lines, skipping blanks and comments. Every framework counted function LOC as a raw line span. One word, two units, sitting side by side in the same payload.

The dashboard sized its bubbles in one unit and printed its function table in the other. A function documented with care looked bigger than a function with no comments at all.

Each language now has one definition of a code line, shared by both levels, so they cannot drift apart again. `Swagger.generate` reports 167 lines in Python and 167 in the Rust engine, which is the first time two implementations have been asked the same question and given the same answer.

Fixing it in PHP surfaced a third bug. A bare `}` is a token with no line number, so the scan for a function's last line landed on the whitespace before it. Every PHP function was one line short. That line is back.

Nothing but the dashboard reads function LOC. Violations and the maintainability index both use file LOC, which was right all along, so no threshold moves.

The dev dashboard reads the coupling it is given

The bubble chart now uses the numbers the engine resolves. Files that everything depends on drift to the centre. A ring around each bubble runs blue for stable and amber for unstable. Hovering prints the real figures behind them.

The chart draws none of this unless the coupling actually resolved. The older metrics modules never mapped an import to a file, so they reported zero dependents for everything and an instability of exactly 1.0. Drawing that would paint every ring at maximum instability, which is a confident lie, so the chart leaves those channels dark instead.

Thirty-three regression tests hold all of it in place across the four frameworks, and seven more in the engine. Each was run against the old code first and watched to fail. A test that has never failed has never proved anything.

v3.13.90 (2026-07-27) - A scaffolded model and its migration finally agree

`tina4 generate crud Todo` wrote a model that declared a `name` column and a migration that never created it. The first write died:

```
table todo has no column named name
```

This sits on the path https://tina4.com/llms.txt hands to AI assistants, so the documented way to stand up a REST API returned a 500 on its first POST.

One default, defined once (php#186, ruby#33, nodejs#38)

The default field set lived inside the model template. Other generators each carried their own copy, and the migration builder had none, so it built the table from an empty field list: `id` and `created_at`, nothing else. The default now lives in a single constant that flows into the model, the migration, the form, the view and the co-emitted test alike.

Ruby hid a second trap. An empty array is truthy in Ruby, so `fields_override || parse_fields(...)` short-circuited to the empty array and the fallback never fired. That check now tests for content, not truthiness.

Python shipped this fix in 3.13.89 and missed one generator: the form kept its own inline copy of the default. The output matches while the default happens to be `name`, and diverges the moment it changes, at which point the form renders an input for a column the table does not have. Fixed, with a source invariant that fails if any generator restates the literal again.

A failed write no longer reports success

The generated create and update routes serialised their result without checking it. `create()` and `save()` report failure by return value; they do not raise. In Node a failed insert therefore returned HTTP 201 carrying unsaved data. In Ruby it surfaced as a NoMethodError on `false` and buried the real cause. Both now check the result and return 400 with a clear message. PHP already checked, and a test pins that behaviour in place.

Every generator test in all four frameworks passed `--fields` explicitly, which is why one bug survived in four languages. Each framework now exercises the no-fields path and asserts that every field the model declares reaches the migration, ending with a real write against a real database.

v3.13.89 (2026-07-27) - A typo'd tag no longer renders what it was hiding

Two Frond bugs, both present identically in all four frameworks since v3, both compatibility gaps against Twig and Jinja2. One of them was quietly showing people content that was supposed to be gated.

An unknown tag raises instead of leaking its body (Breaking, Security)

Look at this template and decide what a visitor sees:

```
{% iff user.is_admin %}
    <a href="/admin">Admin console</a>
{% endiff %}
```

Everyone. `iff` is a typo, and Frond did not recognise it, so the tag rendered nothing and its body rendered as ordinary content. The admin link went to every visitor. Worse than a broken page: the template reads as guarded, so a reviewer scrolling past sees a check that is not there.

The same shape hid behind any misspelling. `{% ifff %}`, `{% unles %}`, `{% i %}`, a tag copied from another engine. Each one silently removed its own condition.

An unrecognised tag now raises, naming it and listing what Frond does know:

```
Frond: unknown tag "iff" -- known tags are: autoescape, block, cache, extends,
for, from, if, import, include, live, macro, raw, set, spaceless
```

Frond has no plugin system for tags, so an unrecognised name is always a typo, never an extension. Twig and Jinja2 both raise on one. Frond now does too.

Two things deliberately still render nothing. A stray terminator, an `{% endif %}` with no `{% if %}`, and an empty `{% %}`. Neither has a body, so neither can expose anything, and both have always been silent.

What to do: run your templates once. A typo'd tag now fails on the page that contains it, which is where you want to find it.

set works as a block (Fixed)

```
{% set badge %}
    <span class="badge">{{ order.status|upper }}</span>
{% endset %}

<td>{{ badge }}</td>
<td class="mobile-only">{{ badge }}</td>
```

That is core syntax in both Twig and Jinja2, and until now it did the wrong thing in all four frameworks: the body printed inline where the block stood, and the variable was never bound. `{% set g %}Hello{% endset %}[{{ g }}]` rendered `Hello[]` instead of `[Hello]`.

It captures now. The captured value is marked safe, so markup you wrote in the body renders as markup rather than as escaped angle brackets, matching both reference engines. A value interpolated into the body is still escaped on the way in, which is the right place for it: the escaping happens once, where the untrusted value enters.

Blocks nest, and a loop inside the body renders into the capture rather than to the page:

```
{% set rows %}{% for i in items %}<li>{{ i }}</li>{% endfor %}{% endset %}
<ul>{{ rows }}</ul>
```

One rule decides which form you get: an `=` anywhere in the tag means assignment. So `{% set label = "a = b" %}` stays an assignment and is never read as a block. The inline form is untouched.

The expression corpus grew to 82

Three entries cover the block form: a plain capture, a capture that must not be double-escaped, and an inline `set` whose value contains an `=`. The unknown-tag rule is locked by named tests in each framework rather than the corpus, because the corpus compares rendered output and this one raises.

v3.13.88 (2026-07-27) - json_encode gives you JSON again

3.13.87 HTML-escaped the `json_encode` filter in all four frameworks. That was wrong, and this release reverts it.

Entity-encoded JSON is not JSON. `{"a":1}` inside a `<script>` block is a SyntaxError, and a script block is most of what the filter is for. One change broke all four languages at once.

The same pass fixed a bug that had been there far longer: a value JSON cannot represent used to escape as itself, or as nothing at all.

json_encode emits JSON, not entities (Breaking, reverts 3.13.87)

`{{ product | json_encode }}` renders `{"id":1,"name":"Widget"}` again. Put it straight into a script block:

```
<script>
    const product = {{ product | json_encode }};
</script>
```

The output is still safe on the page. Frond escapes the four characters that can break out of HTML, as JSON unicode escapes rather than entities. A `</script>` inside a string arrives as `\u003c/script\u003e` and cannot close the block early. A single quote becomes `\u0027`, so the value also drops into a single-quoted attribute untouched:

```
<div data-product='{{ product | json_encode }}'>
```

Use single quotes there. JSON carries its own double quotes, so a double-quoted attribute needs `| e` on top. This is the model Jinja2 uses for `tojson`, and it is why the result is marked safe.

`| raw` after `json_encode` is now a no-op. If you added one for 3.13.87 you can delete it, and nothing breaks if you leave it.

A value JSON cannot represent becomes null (Fixed)

Reported as tina4-php#184 by justin-k-bruce, who hit it in production: two infinite sort keys in a 657-row grid blanked the whole screen.

JSON has no Infinity and no NaN. All four frameworks handled that differently and all four were wrong. Python wrote a bare `Infinity`, which no parser reads. PHP's `json_encode` returned false, which arrived as an empty string, so the payload vanished with no error anywhere. Ruby fell back to Ruby inspect output. Only Node.js already did the right thing.

Every framework now writes `null`, which is what `JSON.stringify` has always produced and the only answer the grammar allows:

```
{{ readings | json_encode }}
```

```
{"low":1.5,"high":null}
```

The rule behind it is wider than one value type. This filter never returns an empty string and never returns a token that would fail to parse. A payload always arrives, in the worst case as `null`. An empty script assignment is at least a loud error; a silently wrong one is not.

Two smaller differences went with it. Forward slashes stay unescaped, so `a/b` no longer comes back as `a\/b` from PHP. Non-ASCII text stays raw, so `cafe` with an accent is itself rather than a `\u00e9` escape from Python.

to_json and tojson are the same filter

All three names now share one serializer. They cannot drift apart again.

The `to_json` indent argument is gone. PHP's pretty printer has a fixed four-space indent and cannot honour an arbitrary width, so supporting the argument in three frameworks and not the fourth broke byte parity on the one filter whose entire job is a wire format.

The expression corpus grew to 79

The cross-framework expression fixture added seven entries: the json_encode escape shape, a non-finite value, a NaN nested in an object, the `to_json` alias, a filter pipe feeding a `~` concatenation, that same expression inside a ternary, and `number_format` with locale separators.

The last three close tina4-php#170 and tina4-php#171. Both were reported against 3.13.68 and both were already fixed by the 3.13.87 expression work; they are locked in now so they cannot come back.

What to change in your templates

Grep for `json_encode`. Three cases:

- Inside a `<script>` block: nothing to do. It works again.
- Inside a single-quoted attribute: nothing to do.
- Inside a double-quoted attribute: add `| e`.

v3.13.87 (2026-07-27) - Frond expressions agree in all four frameworks

Frond has always been described as one template language with four implementations. That was an assumption. Nobody had ever measured it.

So we measured it. Seventy-two expressions covering filters, operators, ternaries, null coalescing, concatenation, comparisons, math precedence, missing values, dotted paths, arrays, hashes, escaping and filter chains, rendered through Python, PHP, Ruby and Node.js against one identical dataset. Eleven of the seventy-two disagreed.

All eleven are fixed. Every framework now renders all seventy-two identically.

The corpus is no longer a one-off script. It ships as a test fixture in all four repositories, byte for byte identical, with a single shared answer key. If one framework drifts, its own suite turns red while the other three stay green, and the failure names the expression. Changing the contract on purpose now means changing the answer key in four repositories in the same change, which is the point.

Booleans print as true or false

Node.js already printed `true` and `false`, so your templates keep working.

The note is here because the contract is now shared rather than coincidental. Python printed `True` and `False`, PHP printed `1` and an empty string, and Ruby printed two different things depending on where the value came from. All three were changed to match Node.js, and the corpus fixture holds all four to it.

{% import "file" as alias %} now works (New)

The alias import form rendered as nothing at all. The tag parsed, the macros were never registered, and `{{ forms.button("Save") }}` produced an empty string with no error and no warning. Only `{% from "file" import name %}` worked.

Both forms now work and behave identically. The macros bind to a plain object rather than a class instance, so no argument is silently consumed as a receiver.

A second macro bug went with it: a parameter declared with a default value, as in `{% macro greet(name, greeting = "Hello") %}`, was mis-parsed. Defaulted parameters now bind correctly.

The not operator works in output (Fixed)

`{{ not user.active }}` rendered as nothing.

Every logical operator was matched with spaces on both sides, so a leading `not` with nothing to its left matched none of them, fell through to variable lookup, and was resolved as a variable literally named "not user.active". Finding no such variable, it rendered empty.

`{% if not x %}` and `{{ x and not y }}` always worked, so the operator itself was never broken. Only the standalone output expression was lost, and before booleans printed lowercase a dropped expression and a false value looked identical.

A leading `not` now routes to the same evaluator `{% if %}` uses, so a condition means the same thing in a condition and in an output expression.

json_encode escaping

Node.js has always escaped `{{ data | json_encode }}`, and still does. Use `{{ data | json_encode | raw }}` inside a `<script>` block.

The note records that PHP returned raw JSON from this filter and was brought in line.

The gate that keeps this true

Every one of these bugs survived because each implementation looked correct on its own. Reading the code four times would not have found them. Rendering the same expression through all four and diffing the bytes found eleven in an afternoon.

That diff is now a test. `frond_expression_corpus.txt` and `frond_expression_expected.txt` live in every framework's test directory as identical bytes, and every suite renders the corpus and compares against the shared answer key. Alongside it sit named regression tests for each bug above, each carrying the negative case that was failing before the fix.

One more thing worth naming, because it is a pattern rather than an incident. Every boolean bug was a falsy guard: `|| ""`, `a[k] || a[k.to_sym]`, `true ? '1' : ''`. In a template engine, "absent" and "false" are different things, and code that conflates them prints nothing where it should print something.

v3.13.86 (2026-07-25) - The package imports under plain Node, not just tsx

`import "tina4-nodejs"` and its subpaths (`/orm`, `/swagger`, `/frond`) now resolve to the built JavaScript in `dist/`, so an installed app runs them under plain Node with no TypeScript loader. The `exports` map used to point at `.ts` source, and `exports` resolves ahead of `main`, so a consumer not running `tsx` got a file Node could not execute. The map is now conditional: `types` still reads the TypeScript source for editors and `tsc`, while `import` and `default` load the compiled bundle. Fixes nodejs#32.

v3.13.86 (2026-07-25) - Write-result field names now match the family

`db.insert`, `db.update`, and `db.delete` return a write result object, unchanged. What changed is the names of its two data fields, renamed to match the Python master, PHP, and Ruby: `rowsAffected` is now `affectedRows`, and `lastInsertId` is now `lastId`. A write now exposes the same field names in every Tina4 language, and `lastId` also agrees with Node's own `getLastId()` method.

Breaking: read `result.affectedRows` and `result.lastId`; the old `result.rowsAffected` / `result.lastInsertId` are gone. The `success` and `error` fields are unchanged, and the adapter-level `lastInsertId()` method keeps its name (only the result field was renamed).

v3.13.85 (2026-07-24) - The dev-admin bundle ships once

Every install carried the dev-admin dashboard twice. `tina4-dev-admin.js` and `tina4-dev-admin.min.js` sat side by side in the framework's public assets, byte-for-byte identical, and nothing referenced the first one. The route that serves the dashboard, the SPA shell that loads it, and the asset resolver that finds it all name the `.min.js`.

The unminified copy is gone. That is 0.92MB removed from every install, in Python, Ruby and Node.js. PHP had already dropped its copy and is unchanged.

Nothing about the dashboard changes. The file that ships is the same file that was always being served, and the two were identical anyway, so the only difference is what you download.

Why the surviving file is still called `.min.js`

It is not minified, and it never was. The two files had the same SHA-256, so the `.min` suffix described an intention rather than a fact.

Renaming it would break every project that references the asset by path, for no benefit, so the name stays. If the bundle is ever genuinely minified the name will finally be honest; until then it is simply the name of the bundle.

A gate, so the duplicate cannot come back

Nothing compared the two files, which is how 0.92MB per package went unnoticed. All four frameworks now test the shipped assets directly: the `.min.js` is present, the unminified duplicate is absent, exactly one `tina4-dev-admin*.js` exists (so a differently-named copy cannot slip through), and the surviving file is the real bundle rather than a stub.

The Ruby half of this was quietly broken in a way worth naming. Two specs read the unminified file behind a `skip ... unless File.exist?` guard. Deleting the file would have turned both into silent skips: a green suite with two dead assertions, and no signal that the asset had vanished. They now read the shipped bundle with no skip guard, so a missing asset fails loudly. A skip that hides a missing file is not a passing test.

v3.13.84 (2026-07-24) - Every generated Dockerfile actually starts

`tina4 deploy docker` wrote Dockerfiles that could not run. Building each one for real found that of the eight Dockerfile generators in the stack (four templates in the `tina4` CLI, plus one inside each framework's own CLI), exactly one was correct.

The Python image is the clearest case. Its `CMD` was:

```
CMD ["python", "-m", "tina4_python.cli", "serve", "--production"]
```

`tina4_python/cli.py` used to be a module, and `python -m` is valid for a module. The v3 restructure replaced it with the package `tina4_python/cli/`, which has no `__main__.py`, so `-m` cannot execute it. Containers died on startup with "'tina4_python.cli' is a package and cannot be directly executed". The code moved; the hard-coded string in a different repo did not, and nothing tested the generated file.

Every generator now names an entry point that exists, and asks for production mode:

language	CMD
Python	`tina4python serve --production`

PHP	`php vendor/bin/tina4php serve --host 0.0.0.0 --port 7145 --production`
Ruby	`bundle exec tina4ruby serve --production`
Node.js	`npx tina4nodejs serve --production`

All four were verified by scaffolding a project, running `tina4 deploy docker`, building the image, and starting a container.

`serve` no longer kills PID 1 (all four frameworks)

Before starting, the CLI reclaims the port from a stale dev server by reading `lsof -ti` and signalling what it finds. It never validated that output. Where `lsof` exists but prints a different shape, a non-numeric field coerced to 0 or 1, and signalling PID 0 sends the signal to every process in the caller's own process group.

In a container the server is PID 1, so it killed itself. The Node image logged `Killed existing process on port 7148 (PID: 1 ...)` and exited 143. The PHP image logged the same attempt and survived by luck.

Port reclaiming is now skipped entirely inside a container, where there is no stale sibling to reclaim from. Outside one, only all-digit PIDs are accepted, and PID 0, PID 1 and the current process are never signalled. The message reports only what was really killed.

Node.js: the package ships built JavaScript

`tina4-nodejs` published TypeScript sources only, and its `bin` was a shell script running `npx tsx src/bin.ts`. Since `tsx` is a devDependency and the production stage installs with `npm ci --omit=dev`, every container had to fetch tsx from the network at startup, and failed without one.

The package now ships `dist/` and `bin` points at `packages/cli/dist/bin.js`, a self-contained bundle whose only imports are Node built-ins. `prepublishOnly` builds it so a tarball can never go out without it, and `prepare` builds it after a plain `npm install` in a checkout, so a fresh clone still works immediately. Verified by running the container with `--network none`: it starts and never mentions tsx.

Node.js: the CLI reported a version that had not shipped

`tina4nodejs commands --json` is the self-describing manifest the `tina4` CLI reads to learn this framework's command surface. Its version comes from the nearest package.json, which is `packages/cli/package.json`. That file was bumped every release through 3.13.81 and then missed, so the manifest kept announcing 3.13.81 while 3.13.82 and 3.13.83 shipped. Nothing compared the two files.

Both are back in step, and a test now asserts that `packages/cli` and `packages/core` carry the root version. Miss a bump again and the suite fails instead of the CLI quietly misreporting itself.

A gate, so this cannot rot again

Nothing tested a generated artifact. The CLI's deploy tests covered argument parsing and nothing else, and the docs truth-check inspects prose, not template payloads.

The CLI now carries five contract tests over the Dockerfile templates that fail on exactly the mistakes above: no `python -m` on a package, no dev-only runner such as tsx in a production CMD, every CMD names a published entry point, every CMD requests production, and every template sets `TINA4_OVERRIDE_CLIENT`.

The port-reclaim rule is pinned down too, in all four frameworks. Deciding which PIDs to signal now lives in one pure function per language (`selectable_pids`, `tina4SelectablePids`, `selectablePids`), so the safety rule is testable without touching a real process: 43 tests assert that a junk token, PID 0, PID 1, the current process and the current process group are each refused, and that a genuine stale sibling is still reclaimed. One of them feeds in the exact line from the container log that started this.

Also in the CLI (tina4 v3.8.58)

The four bundled Dockerfile templates carry the corrected commands, and the Ruby template gains its build toolchain. Each framework's own `docker` generator was corrected to match, so the two paths agree. The PHP generator now detects whether a project has `bin/tina4php` or `vendor/bin/tina4php` and writes whichever exists, because composer does not link a root package's own bin into `vendor/bin`.

v3.13.83 (2026-07-24) - Swagger UI stops serving itself in production

Security. With swagger disabled, `/swagger/openapi.json` returned 404 and `/swagger` returned 200. The gate was real. The static files walked around it.

The framework ships the Swagger UI as files inside its own public directory. Static serving runs before route matching, and it resolves a directory to its index, so `/swagger` became `swagger/index.html` and never reached the gated route. A production server kept serving the whole UI on four paths:

```
/swagger                       200
/swagger/                      200
/swagger/index.html            200
/swagger/oauth2-redirect.html  200
```

The tell was the mismatch. The spec route obeyed `TINA4_SWAGGER_ENABLED`; the UI ignored it. Static serving now checks the swagger gate before it resolves an index, so all four paths return 404 when swagger is off and serve as before when it is on. Fixed in all four frameworks, each with a lock-in test that fails against the old code. (python#97)

The banner advertises only what answers (python#99)

Every boot printed both dev links, whatever the environment:

```
Swagger:   http://localhost:7148/swagger
Dashboard: http://localhost:7148/__dev
```

In production both URLs 404. That misled two readers at once. An operator scanning a production log believed a dev surface was exposed. A developer clicking the link landed on a dead page.

The banner now prints a row only when that surface answers. Each framework builds those rows through one pure function of the port and two booleans, so the contract is unit tested instead of inferred from stdout: `banner_surface_lines` in Python, `App::bannerSurfaceLines` in PHP, `Tina4.banner_surface_lines` in Ruby, `bannerSurfaceLines` in Node.

Node prints a banner from two places, the single-server path and the cluster path. Cluster mode is the production path, so a `Dashboard:` line there was always wrong; that site now asks for the dashboard row explicitly and gets nothing. A test reads the source to keep it that way.

The CLI runs no watcher in production (tina4 v3.8.57)

`tina4 serve --production` started the file watcher and the SCSS watcher anyway. Both burn CPU, both contend with the single server process, and neither has anything to do: production compiles SCSS once at boot and never re-imports code. `--production` now starts neither. The CLI stays to supervise the server process and shut its tree down cleanly on Ctrl-C.

A stale CA no longer turns a missing certificate into six failures (python#98)

The MQTT TLS tests trusted whatever CA file happened to sit in the shared temp directory. Once that file went stale, the tests did not skip. They failed, six of them, in all four frameworks, pointing at TLS code that was correct. Each suite now verifies that the CA actually validates the broker certificate before it treats the TLS environment as present, and skips when it does not.

v3.13.82 (2026-07-23) - MQTT 3.1.1: talk to any broker, no dependency

Tina4 now speaks MQTT. The new `Mqtt` client publishes and subscribes to any MQTT 3.1.1 broker - Mosquitto, EMQX, HiveMQ, AWS IoT - and adds nothing to your dependency tree. It is built on `node:net` and `node:tls` alone, and it is the same client in all four frameworks. Node has no blocking socket read, so it is async by design: `connect`, `publish`, `subscribe`, and `receive` return promises, and `consume` is an async generator.

```
import { Mqtt } from "tina4-nodejs";

const mqtt = new Mqtt();   // reads TINA4_MQTT_URL, default mqtt://127.0.0.1:1883
await mqtt.connect();
await mqtt.publish("fleet/meter-42/telemetry", '{"kwh":12.5}', 1);

for await (const message of mqtt.consume("fleet/+/telemetry", 1)) {
    if (message.isDuplicate()) continue;
    store(message.topic, message.payload);
}
```

- **QoS 0 and QoS 1, retained messages, and a Last Will.** `publish`, `subscribe`, and `consume` mirror the Queue you already know. `consume` acknowledges a message only after your loop body has processed it, so a body that throws leaves the message for redelivery - at-least-once, which is what QoS 1 is for.

• **QoS 2 is refused, loudly.** Asking for exactly-once raises an error that names the limit and the fix - a QoS 1 consumer keyed on device id and timestamp - rather than silently downgrading to at-least-once and double-processing forever.

• **TLS with a per-connection trust store.** An `mqtts://` connection verifies the broker against the CA you supply and no other. A self-signed certificate is rejected unless you provide its CA, and a CA loaded for one client never leaks into the next.

• **Username and password auth**, over plain or TLS connections, from the URL or from explicit arguments.

• **No mocks.** Every test runs against a real Mosquitto broker - the anonymous, authenticated, and TLS listeners - so the wire protocol is verified for real, not simulated.

This release also:

• **Counts 98 built-in features.** MQTT is the newest. 97 features are identical across all four languages; Ruby adds ERB as a native second template engine for 98.

• **Breaking:** `sqlTranslation` **is renamed to** `sqlTranslator` (module `sqlTranslator.ts`) so the name matches across all four frameworks. Update your imports.

• **Fixes two dev endpoints** that a bare `require()` inside the ESM package silently broke.

• **Stops and deregisters a single background task.** The handle returned by `background()` has a `stop()` that ends just that task and removes it from the registry.

• **Dispatches the in-process test client through the real front-controller tail**, so a test sees the same routing, middleware, and 404 path a real request does.

• **Ships the compiled Tina4 CSS assets** and honours SCSS `!default` instead of leaking it into the output.

• **Reaches** `doctor`**,** `setup`**, and** `deploy` from `tina4nodejs` by delegating to the shared Rust CLI.

v3.13.81 (2026-07-21) - `tina4 test` exit code, locked in

Node needed no fix here. `tina4 test` already propagated a non-zero exit when a test failed, in both the single-file and the auto-discover paths. This release adds a real lock-in test so the contract can never drift: it spawns the actual CLI and asserts a non-zero exit on failure, including one failing test among passing ones. Parity with the python#96 fix.

This release re-aligns all four frameworks on one version. Python, Ruby, and Node skip 3.13.80, a PHP-only patch that shipped the `\Tina4\Test` base class; PHP moves from 3.13.80 to 3.13.81. Everyone is back on 3.13.81.

v3.13.79 (2026-07-19) - Session cookies get Secure behind a proxy, and a renamed cookie is read back

If you run Node behind a TLS-terminating proxy, the session cookie shipped without `Secure` over the very deployments that were encrypted. This release fixes that and reads a renamed cookie back.

- **Security:** `Secure` **is now proxy-aware.** `buildSessionCookie()` set `Secure` only from an explicit `TINA4_SESSION_SECURE` and ignored `x-forwarded-proto`, so HTTPS behind a TLS-terminating proxy shipped the session cookie without `Secure`. `Secure` now reads the scheme through `isSecureScheme()` (the `x-forwarded-proto` first hop, else the native scheme), still honours `TINA4_SESSION_SECURE`, and `SameSite=None` forces `Secure`.
- **Plain HTTP is unchanged.** Without a proxy header and without TLS, the cookie stays non-Secure.
- `TINA4_SESSION_NAME` **is now read back.** The name resolves through one function (`sessionCookieName()`) on both the write and the read side, and the incoming cookie is matched on an exact `name=` prefix; the default is byte-identical.

Reported by justin-k-bruce (nodejs#34). Real wire tests read the actual `Set-Cookie` and replay a renamed cookie.

v3.13.77 (2026-07-16) - A slow background task no longer runs on top of itself

- `background()` **never overlaps a task with itself.** The timer used `setInterval`, which fires on a fixed schedule and does not wait for an async callback, so a run slower than the interval had a second copy start alongside it. The timer is now re-armed only after each run settles, making the interval the gap between runs. Found by cross-checking the Python report against all four frameworks, not by a Node report.
- **Stopping a task mid-run really stops it.** `stop()` and `stopAllBackgroundTasks()` cleared the timer but a run already in flight would schedule another when it finished. Both paths now mark the task stopped and the in-flight run checks that before re-arming.

PHP and Ruby already never overlapped, so all four frameworks now behave the same way.

v3.13.76 (2026-07-16) - Migrations apply again on a database created before 3.13.55

If your database was created by Tina4 v3 3.13.54 or earlier, every new migration failed and none could ever be applied. This release fixes that. Node never created that column, so this only reached apps pointed at a database whose tracking table came from tina4-python; the same hardening now applies.

- **The bookkeeping insert now writes the columns your table actually has.** The 3.13.55 rename added `migration_name` and left the old `migration_id` column in place, calling it harmless. It was harmless on reads and anything but harmless on writes: that column is `NOT NULL`, the insert never filled it, so recording a migration raised a not-null violation, the migration rolled back, and the database was stuck. The runner now builds its insert from the table's real columns and fills any legacy one it finds. No schema change, no `ALTER`, every engine.
- **Fresh databases are untouched.** A table with no legacy column still gets exactly the six canonical columns. That is the case CI and every new project exercised, which is why this only ever bit long-lived staging and production databases.

Thanks to justin-k-bruce, who reported it against 3.13.75 with a full compatibility matrix and a working patch. Real-database regression tests now cover a pending migration on a legacy table in all four frameworks.

v3.13.75 (2026-07-14) - Static assets revalidate, so a deploy reaches users without a hard refresh

The built-in static file handler (everything under public/) now lets a browser cache an asset but forces it to revalidate on every use. A redeployed CSS or JS file reaches the browser on the next page load, with no manual hard refresh - and an unchanged file costs a cheap 304 Not Modified, not a full re-download.

- **Cache-Control and validators on every static response.** Each asset carries `Cache-Control: no-cache, must-revalidate`, an `ETag`, and a `Last-Modified`. Before, a static asset carried no cache headers at all - the worst case of the four.
- **Conditional requests get a 304.** The handler answers `If-None-Match` and `If-Modified-Since` with a `304 Not Modified` and no body, so a revalidation is a small round trip rather than a re-download. Real-file, real-request tests lock the behaviour in.

This lands identically across Python, PHP, Ruby, and Node.js. It closes the class of "I already reported this" where a browser kept serving a fixed-but-cached front-end asset.

v3.13.74 (2026-07-13) - The dev dashboard connection tester works again

The dev dashboard "Test connection" panel now reports the real table count and server version. On Node.js the handler called the async `db.getTables()` and `db.execute()` without awaiting them, so `tableCount` was always 0 (an unresolved Promise is not an array) and the version always fell back to a bare label. This release awaits `db.getTables()` and reads the version through `db.fetchOne(...)`, matching Python and Ruby.

- **A real node:sqlite test drives the endpoint end to end.** It opens a live database with two tables and asserts the panel returns success, a table count of two or more, and a real `SQLite <version>` string. No mocks, and the version assertion cannot pass against the old empty output.

The same panel was broken in different ways in Python and PHP and is fixed there too; Ruby was already correct and gained the same lock-in test.

v3.13.73 (2026-07-13) - A failed migration re-applies cleanly

This release makes a previously-failed migration run again on the next migrate, at full parity across the four frameworks.

- **A leftover `passed = 0` row no longer wedges the next run.** When a migration succeeds, the runner deletes any existing bookkeeping row for that migration name before it writes the fresh `passed = 1` row. A migration that failed earlier - whether you recorded it with `recordMigration(name, batch, 0)` or carried it over from a v2 table - re-applies cleanly instead of colliding on the unique `migration_name`. The `tina4_migration` table holds at

most one row per migration, and the latest run wins. The delete-then-insert path is identical on every engine, so there is no dialect-specific behaviour to reason about.

• **The v2 upgrade tells you what happens next.** When the v2 to v3 upgrade finds `passed = 0` rows, it logs that those migrations re-apply on the next migrate, instead of asking you to clear them by hand.

• **Gallery ORM imports resolve from a consumer app (#32).** The database gallery examples now import from the published entry points, `tina4-nodejs` and `tina4-nodejs/orm`, instead of the internal `@tina4/*` workspace names, and they `await initDatabase()` before use. A copied gallery file now runs in a fresh project without an import error.

v3.13.72 (2026-07-12) - Frond fixes, a sandbox hardening, and a malformed-path guard

This release sharpens the Frond template engine, guards the Node worker against a malformed request path, locks in a database error contract, and brings the dev dashboard to parity across the four frameworks.

• `number_format` **reads all three arguments.** The filter now honours the full Twig signature, `number_format(decimals, decimalPoint, thousandsSep)`:

```twig
{{ 1234.5 | number_format(2, ',', '.') }} {# renders 1.234,50 #}
```

The one-argument form is unchanged, so every existing template behaves as before. (php#170)

• **The filter pipe binds tighter than concat.** `|` now groups before `~`:

```twig
{{ amount|number_format(2) ~ ' EUR' }} {# (amount|number_format(2)) ~ ' EUR' -> 1,234.50 EUR #}
```

The rule holds at any nesting depth, including both branches of a ternary. On Node it also clears a latent case where a pipe inside parentheses rendered empty. (php#171)

• **The sandbox allow-list covers every filter path (Security).** A filter applied inside a `~` concatenation or a ternary condition now respects the `{% sandbox %}` filter allow-list. A filter you did not allow-list no longer runs its code in sandbox mode.

• **A malformed request path returns 404 instead of crashing (Security).** A path like `//` (or `///`, `/\`) used to crash the Node worker in production through an unguarded `new URL()` call. The worker now guards the parse and answers with a normal 404. Python, PHP, and Ruby were already safe.

• **Database errors still fail loud (python#57).** `execute()` and `fetch()` raise on failure and record the message on `getError()` rather than returning `false` or an empty result. This shipped in 3.13.38; this release adds a real-PostgreSQL regression test across all four frameworks so it can never slip back to a silent failure.

• **Dev dashboard parity (`TINA4_DEBUG`).** The dev-admin dependency installer (`deps/install`), the grounding-token proxy, and the Migrate, Test, and Seed run-chips now match across all four frameworks. This is development-only; nothing changes in production.

v3.13.71 (2026-07-11) - AI skills: sharper tina4_code guidance

A skills-and-docs release; no change to the Node.js package. The bundled Tina4 AI skills now state WHY `tina4_code` is deprecated: in a boot-and-verify gate (scaffold the output, boot it, run it) `tina4_code` failed where a strong model grounded with `tina4_context` passed, so the tools point to grounding plus a strong model over the self-hosted coder. The recommendation is unchanged - ground with `tina4_context` and write the code yourself; only the rationale is sharper. Running `curl -fsSL https://tina4.com/install-skills.sh | sh` now installs these updated skills by default.

v3.13.70 (2026-07-11) - Installed-package imports, column defaults on INSERT, a Firebird charset override, and stacked Swagger metadata

Installed-package imports resolve (#32)

Importing `tina4-nodejs/orm`, `tina4-nodejs/frond`, or `tina4-nodejs/swagger` from an application that installed the package now works, and `response.render()` renders correctly from an installed app. The published packages referenced their siblings by the internal `@tina4/*` workspace names, which only resolve inside this monorepo, so a consumer install failed to find them. Those references now resolve to the real subpaths, so an installed app imports the subpath entry points and renders templates the same way the in-repo examples do.

ORM honours column defaults on INSERT (#165)

An INSERT now omits any column you never set on the model, so the database applies that column's own DEFAULT. Previously `save()` serialised every declared column, sending an explicit NULL for the ones you left alone; a `NOT NULL DEFAULT <x>` column then failed the insert, because a DB default applies only when the column is omitted, not when NULL is passed. The rule is now precise: a column you never touch is omitted and the database fills it in; a column you explicitly set to `null` is written as NULL; a column with a non-null ORM default is still written. When every insertable column is unset, the row inserts with the engine's all-defaults form. UPDATE is unchanged. Shipped across all four frameworks, verified with real-SQLite positive and negative tests.

Firebird connection charset override (#160)

The Firebird adapter no longer hardcodes the connection charset to UTF8. You can override it, in precedence order, with a `?charset=` query on the connection URL (`firebird://host:port/path?charset=NONE`), an explicit `charset` option on `connect()`, or the `TINA4_DATABASE_CHARSET` environment variable. The default stays UTF8, so nothing changes unless you ask for it. This fixes double-encoded UTF-8 bytes read from a legacy NONE database. Shipped across all four frameworks.

Stacked Swagger metadata all survives (#59)

Stacking summary, description, and tags on one route now keeps every value in the generated OpenAPI spec, whatever the order. This was a Node-visible drift where all but the annotation nearest the route method were dropped; it is fixed here and in PHP (Python and Ruby were already correct), and all four now carry a lock-in test so the behaviour cannot drift again.

v3.13.69 (2026-07-10) - The Api client grows up: uploads, downloads, and safe redirects

The built-in HTTP `Api` client gained four zero-dependency capabilities, shipped across all four frameworks. All are opt-in and non-breaking.

- **Multipart upload.** `api.upload()` POSTs a `multipart/form-data` body from a file on disk (`filePath`) OR from in-memory bytes (`fileBytes` plus `filename`), with optional extra text fields, so you never need a temp file. The part Content-Type is guessed from the filename. A missing file or no source returns a clean error result and never throws.
- **Streaming download.** `api.download()` writes a GET body straight to disk in 64KB chunks, so a multi-megabyte file never lands in memory whole. It returns the status, headers, and the on-disk `path` (there is no `body` field), and writes nothing on an error status.
- **Transport seam.** An injectable transport lets application developers unit-test code that calls an `Api` without a live server. Tina4's own suite never injects a fake (the no-mock rule stands); every framework test hits a real local server.
- **Opt-in cookie jar.** Pass `{ cookies: true }` for a per-client in-memory jar: the client parses `Set-Cookie` (leading `name=value`, last write wins) and replays the accumulated `Cookie` header on later requests.

Bare `node:http` and `node:https` do not follow redirects, so the client now follows them itself (bounded to 10 hops): 301/302/303 on a body-bearing method become GET, and 307/308 preserve method and body. On a cross-origin hop (a different scheme, host, or port) it strips the `Authorization` and `Cookie` headers, so neither a bearer token nor a session cookie can leak to a host you did not authenticate to. Same-origin redirects keep them.

Also shipping

- **AI coder rule-path skill.** The AI coding-assistant scaffolder writes each tool's rule and context files to the correct path, across all four frameworks.

v3.13.68 (2026-07-10) - Parity release

A version bump to keep all four frameworks on the same number. No functional changes to the Node.js package since 3.13.67; the accompanying change is a fix to the Tina4 maintainer AI skill so it links plan and source files by a path that resolves when clicked.

v3.13.67 (2026-07-10) - The MCP table browser, locked down

The `database_tables` dev-tool now has a real behavioural test. A sibling bug in the PHP framework (#164) fataled the same tool: it called a method that does not exist instead of listing tables, and the test never caught it because it only checked the tool was registered, never that it ran. This framework already listed tables correctly, but carried the same blind spot in its own tests. The new test invokes the real handler against a real SQLite database and asserts a table list comes back, so this class of drift is caught here too. Shipped across all four frameworks.

v3.13.66 (2026-07-10) - A self-describing CLI, and generators that ship their tests

The command line grew up. The `tina4` client no longer keeps its own copy of what each framework can do. It asks the framework, then forwards the request. A command you add to the framework shows up in the client automatically; one you remove simply stops being offered. The whole class of client-versus-framework drift is gone.

The self-describing CLI

- `commands` / `commands --json`. Every framework CLI now prints its own command

table from a single source. The `tina4` client reads that manifest to render an accurate `tina4 --help`, caches it by a cheap fingerprint of the resolved CLI, and refreshes with `--refresh`. Dispatch never depends on the manifest, so a discovery miss shortens the help listing and never breaks a command.

- **Pass-through dispatch.** The client keeps its own conductor commands (serve, scss,

setup, init, deploy, agent, doctor, install, update, build) and forwards everything else to the framework verbatim. It carries no per-command flag knowledge, so it can never fall out of parity.

- `queue` **is now a top-level command** in all four frameworks: `queue work`,

`queue stats`, `queue retry`, `queue clear`, wired to the real queue. Run a worker straight from the CLI instead of only scaffolding one.

- `build` **builds the deployable Docker image** (`docker build`), replacing the old

library-packaging behaviour. `build` produces the image; `deploy` ships it.

- **Ruby gains** `migrate:create`, matching the other three.

Generators now ship a test with the code

Every code-producing `generate` subcommand writes a real, passing test next to the code it scaffolds. `generate model Product` also gives you a `Product` test that talks to a real database. The tests use real collaborators (real SQLite, a real test client, a real queue), never mocks, and pass the moment they are generated.

Writing those tests exposed real bugs in the scaffolds that no string-matching test would have caught: the generated `auth` current-user endpoint rejected valid tokens in Python and PHP, and the generated migration created then immediately dropped its table in Ruby and PHP. All four are fixed and locked in by the new tests.

Databases

- **PHP silent PDO fallback.** SQLite and PostgreSQL adapters prefer the native

extension and fall back to the matching PDO driver when it is missing. The developer gets a working database either way, with identical behaviour (native types, raw-byte BLOBs, last insert id, transactions, fail-loud errors).

- **ORM `where()` takes an order.** `Model.where(...)` now accepts `order_by` /

`orderBy` (and Ruby also gains `limit`/`offset`), matching `find()`, `all()`, and the query builder.

Fixes

- The Frond browser helper now applies a 30 second request timeout by default.
- SQLite datetime adapters no longer emit a deprecation warning on Python 3.12+.
- Node route handlers accept a `void` return in TypeScript without a type error.

Breaking

- **Frond `request()` now times out after 30 seconds by default.** A request that

used to hang forever now fails after 30 seconds and calls `onError`. Pass `timeout: 0` to restore the old unbounded behaviour, or a millisecond value to set your own.

- `build` **changed target.** It now builds a Docker image rather than packaging the

framework as a library. Projects that relied on the old behaviour should call their packaging tool directly.

- `generate` **writes an extra test file per scaffold.** If you script generation and

assert on the exact set of created files, expect one more file (the co-emitted test).

v3.13.56 (2026-07-08) - Skills that own up when they drift

Every AI skill now tells the assistant how to report itself when it is wrong. A skill is documentation, and documentation drifts. When a skill still describes a method, default, or column the framework no longer has, an assistant writes confident code against an API that is gone. This release closes that loop.

Every skill, and every project context file the AI installer writes (CLAUDE.md, .cursorules, .github/copilot-instructions.md, .windsurfrules, CONVENTIONS.md, .clinerules, AGENTS.md), now carries one instruction: if Tina4 behaves differently from what the skill describes, that is a bug in the skill. Tell the developer, then report it at https://tina4.com/report-a-skill. The report lands as an issue on the documentation repository, gets fixed at the source, and ships to everyone.

The skills themselves are corrected too. The ORM soft-delete guidance now names the real `is_deleted` column (it wrongly said `deleted_at`), the tina4-js signal-persistence reference ships alongside the skill, and the per-framework skill copies are back in sync with the canonical set.

The web framework runtime does not change in this release. Update your installed skills with `curl -fsSL https://tina4.com/install-skills.sh | sh` (re-run to refresh in place).

v3.13.55 (2026-07-07) - One migration tracking schema on every engine

The `tina4_migration` bookkeeping table now has the same shape on every framework and every engine. Before this release the four frameworks each named and typed the tracking table a little differently. A project that moved between them, or a tool that read the table directly, met a different schema each time.

The canonical table is six columns: an auto-increment `id`, a `migration_name` (unique, the migration file stem), a `description`, a `batch`, an `executed_at` timestamp, and a `passed` flag. The auto-increment and the column types follow the engine: `AUTOINCREMENT` on SQLite, `SERIAL` on PostgreSQL, `AUTO_INCREMENT` on MySQL, `IDENTITY(1,1)` on SQL Server, and a generator on Firebird.

Existing installs upgrade in place, and no applied migration re-runs. The runner detects the old name column (`migration_id` in Python, `migration` in PHP, `name` in Node; Ruby already used `migration_name`), adds `migration_name`, copies the values across, and backfills the new columns. The old column stays where it is, ignored. A migration already marked applied stays applied.

No new third-party dependencies. Shipped across all four frameworks.

v3.13.54 (2026-07-07) - Migrations honour the SET TERM directive

A Firebird trigger or stored procedure now survives the migration splitter. Those bodies end their inner statements with a semicolon, the same character the runner uses to separate one statement from the next. Run under the default terminator, a trigger body split apart on its own punctuation and the migration failed.

A `SET TERM` line fixes it. Wrap the block in the universal isql idiom and the runner switches its active terminator, so the whole body travels as one statement:

```
SET TERM ^ ;
CREATE OR ALTER TRIGGER t_bi FOR t ACTIVE BEFORE INSERT AS
BEGIN
  IF (NEW.id IS NULL) THEN NEW.id = GEN_ID(GEN_T, 1);
END^
SET TERM ; ^
```

The runner consumes each `SET TERM` line instead of sending it to the engine, restores the previous terminator when the block ends, and handles a multi-character terminator such as `!!`. A migration with no `SET TERM` splits on the semicolon exactly as before. Shipped across all four frameworks.

PHP and Ruby also repair the Firebird v2 to v3 upgrade. Firebird returns column names in upper case, so the migration tracker read a null migration name and treated every applied

migration as pending, re-running the lot. The tracking-table reads now normalise to one key shape. PHP additionally records a row on an upgraded table with its original 14-character id and detects the Firebird dialect through the Database facade.

Thanks to justin-k-bruce for the contribution.

v3.13.53 (2026-07-06) - JSONField: JSON document columns

Store a JSON document in a column. A model field can now hold a whole object or array. The framework encodes it to JSON when it writes and decodes it back to a native object when it reads, so the attribute is always live data, never a raw string.

```
class Event extends BaseModel {
    payload: { type: "json" },   // object or array
```

The column type follows the engine. PostgreSQL gets native `JSONB`, MySQL native `JSON`, SQL Server `NVARCHAR(MAX)`, Firebird a text `BLOB`, and SQLite `TEXT` (queryable through JSON1). One field declaration, the right column on every database.

A value that cannot be encoded to JSON does not slip through. `save()` fails loud: it rolls back, returns `false`, and records the cause, so a half-written row never reaches the table. A mutable default is copied per instance, so two records never share and mutate the same object.

No new third-party dependencies.

v3.13.52 (2026-07-04) - Frond live blocks, pgsql:// URL scheme, SCSS colour functions

Frond live blocks. A page can now carry a region that keeps itself current. Wrap the region in `{% live %}` and Frond paints it on the server with the first request, then refreshes it over the transport you name.

```
{% live "prices" poll 5 %}
  <ul>{% for row in rows %}<li>{{ row.name }}: {{ row.price }}</li>{% endfor %}</ul>
{% endlive %}
```

The block renders server-side on first paint, so a crawler and a client with no JavaScript both see real content. After that it refreshes on its own. `poll N` re-fetches every N seconds. `sse` streams updates over Server-Sent Events. `ws "/path"` rides a WebSocket route you already own. A data provider feeds every refresh: `@live_source` in Python, `Frond::liveSource` in PHP, `Frond.live_source` in Ruby, `Frond.liveSource` in Node. The provider re-runs with the live request, so a block that reads the signed-in user reads it again on every refresh, and an authenticated block cannot serve one user another user's data. For poll and SSE, Tina4 mounts one always-on endpoint, `GET /__frond/live/{name}`. For a WebSocket block, `push_live(name, data)` re-renders the block and broadcasts the fresh HTML to every client on that path. Nested live blocks are rejected, and a block's optional `src` attribute is same-origin only.

One client script drives it, `frond.js`, byte-identical across Python, PHP, Ruby, and Node. No build step. No framework on the page.

pgsql:// is a Postgres URL scheme again (#58). A connection string like `pgsql://user:pass@host/db` was rejected. v3 registered only `postgresql://` and `postgres://` and dropped the older spelling, but `pgsql` is the scheme PDO, Laravel, and Doctrine all use, so real config files carried it and Tina4 refused to start. `pgsql://` now resolves to the PostgreSQL driver in all four frameworks, next to the two existing spellings. Same driver, three accepted names.

SCSS colour functions evaluate at compile time. `rgba(#3498db, 0.5)` used to pass through to the stylesheet as literal text, and the browser dropped the whole rule because `rgba()` cannot take a hex string. The built-in SCSS compiler now evaluates the colour functions: `rgba(#hex, a)` and `rgb(#hex)` expand to real channel values, `mix(c1, c2, weight)` blends two colours, and `lighten()` and `darken()` shift a colour through HSL. The output is byte-identical across all four compilers, down to the same integer rounding, so a shared stylesheet renders the same colour whichever framework served it.

No new third-party dependencies.

v3.13.51 (2026-07-03) - MCP Streamable HTTP transport, Firebird fixes

The built-in dev MCP server now speaks the current MCP Streamable HTTP transport, the one Claude Code and today's MCP clients expect. It still answers the older 2024-11-05 HTTP+SSE transport, so nothing that already worked stops working.

One endpoint carries the whole session. A client POSTs JSON-RPC to `/__dev/mcp` and reads the response inline. `initialize` mints a session and returns it in an `Mcp-Session-Id` header; the client sends that header back on every later call. A request with an unknown session gets a `404`, its cue to initialize again. A notification returns `202`. `GET` on the endpoint returns `405` with `Allow: POST, DELETE`, and `DELETE` ends the session. The server negotiates the protocol version: it echoes the version a client asks for when it can speak it, otherwise it picks the newest one it knows.

Tina4 writes the connection details to `.claude/settings.json` for you, now with `"type": "http"` and the bare `/__dev/mcp` URL. Prefer the command line? Run `claude mcp add --transport http tina4-dev http://localhost:7148/__dev/mcp`. The change lands uniformly across Python, PHP, Ruby, and Node. Python and Node keep a full persistent legacy SSE stream; PHP and Ruby serve the current transport plus a one-shot legacy handshake, with no long-lived connection required.

Why the transport slipped past us. Our MCP tests spoke our own JSON-RPC shape over the endpoints we built, never the wire a real client speaks. They stayed green while a real Claude Code client could not connect. Every framework now ships a no-mock transport test that drives the real session lifecycle, and each was verified end to end against a live server booted through the tina4 CLI.

Firebird (PHP). Three reported issues are fixed. The migration runner and the ORM now call `execute()` rather than the old `exec()` (#120). Parameterized DML no longer throws a type error when it fetches the last insert id (#121). NULL parameters bind correctly again, rewritten to a literal `NULL` because the ibase driver cannot bind a PHP null (#123). The `exec()` method stays

as a deprecated alias for `execute()`.

Migration recording on upgraded schemas (PHP). The fail-loud `execute()` surfaced a quiet bug. On a database whose `tina4_migration` table had been upgraded in place from the old v2 layout, a new migration never recorded itself, so it re-ran on every boot. The old `exec()` had swallowed the constraint error. The runner now supplies the legacy `migration_id` column when it is present, so a migration records once and stays recorded.

No new third-party dependencies.

v3.13.50 (2026-07-02) - Path route params match INTEGER primary keys on SQLite (Ruby fix)

A route path parameter like `{id}`, matched against a real HTTP request, must find an INTEGER primary-key row. On Ruby it did not. Rack delivers the request path as ASCII-8BIT, so an untyped `{id}` capture reached the SQL bind as a binary string, and the sqlite3 gem bound it as a BLOB. SQLite gives a BLOB no numeric affinity, so `WHERE id = ?` never matched an INTEGER column - `GET /api/users/{id}` returned 404 for a row that plainly existed (and `GET /api/users` listed it). The router now relabels path captures as UTF-8 so they bind as TEXT, which SQLite coerces to the column's integer affinity, and the row matches. Typed `{id:int}` params were never affected - they cast to an Integer. The SQLite driver is left alone on purpose: coercing every binary string there would corrupt genuine BLOB writes, so the encoding is fixed at the source (the router).

Python, PHP, and Node were confirmed unaffected - their string path params already bind as TEXT - and each gains a real regression test: real router extraction feeding a real SQLite integer-primary-key lookup, no mocks, so the contract cannot silently drift. No new third-party dependencies.

v3.13.47 (2026-06-25) - Open-issue batch: migration comment splitting, global middleware, SCSS interpolation

Three reported issues, fixed and locked in with tests against the real thing.

Migration statement splitter (#54). A migration whose SQL carried a `;` inside a `-- ...` line comment fragmented into broken pieces, because the runner split on the `;` delimiter before it stripped comments. A `CREATE TABLE` with a trailing `-- drop then re-add; old way` comment raised "incomplete input" on SQLite. The splitter is now a single-pass, quote- and comment-aware scanner: it strips `--` line and `/* */` block comments, copies single- and double-quoted string literals verbatim (honouring the `''`/`""` escape), keeps `$$`/`//` stored-procedure blocks intact, and splits on the delimiter only outside all of that. A `;` or `--` inside a comment or a string literal can no longer split or corrupt a statement. Named regression tests plus an end-to-end migrate against a real temp SQLite database lock it in, with no mocks.

Global middleware lock-in (#55). Global class-based middleware registered with `Router.use(...)` already ran on every route in Node, with `global-middleware.test.ts` covering it. This release is the Python-master dispatch fix plus matching lock-in tests across the family.

SCSS `#{}` interpolation (#116). The SCSS compiler did not support interpolation, so `calc(100% - #{$gap})` left the `#{...}` in the output and corrupted the CSS around it. The compiler now resolves `#{ ... }` before variable substitution and nesting: a `$variable` inside the braces resolves to its value and anything else inlines verbatim, so `calc(100% - #{$gap})` becomes `calc(100% - 20px)` and `.icon-#{$name}` becomes `.icon-home`. Shipped across all four frameworks for parity.

No new third-party dependencies.

v3.13.46 (2026-06-24) - Atomic batch insert across MySQL, MSSQL and PostgreSQL

A batch insert that hits a bad row must roll the whole batch back, not leave the rows before it committed. The MySQL, MSSQL and PostgreSQL adapters ran executeManyAsync as a row-by-row loop with no transaction, so a failure mid-batch left a partial write - despite the documented "wrapped in a transaction" contract that only SQLite honoured. Each adapter now wraps the batch in one transaction and rolls back on any error, joining a caller's explicit transaction when already inside one. Two MSSQL fixes ride along: transactions use the native tedious begin, commit and rollback calls (a raw BEGIN through sp_executesql failed SQL Server's transaction-count check, which had also broken explicit transactions), and a single-object insert now reports one affected row instead of two. The batch-insert tests run the full contract against every live engine, no mocks. No new third-party dependencies.

v3.13.45 (2026-06-24) - Real-service test hardening

The MongoDB queue tests now isolate the Mongo connection environment, so a host-level `TINA4_MONGO_URI` can no longer bleed into the two tests that build a connection from explicit host and port values and flip their expected result. The suite runs clean against the provisioned services. No framework runtime change. No new third-party dependencies.

v3.13.44 (2026-06-24) - Real-service bug-fix sweep (no mocks)

Standing up live infrastructure - PostgreSQL, MongoDB, Redis, Valkey, Memcached, RabbitMQ, Kafka - and running the suites against the real services surfaced a batch of bugs that mock-based and skipped tests had hidden. This release fixes them across the family and makes the no-mock rule absolute: a test that touches a dependency exercises the real service, never a stand-in. **Migrations on PostgreSQL/MySQL/MSSQL:** the migration runner built its bookkeeping table (`tina4_migration`) with SQLite-only DDL (`id INTEGER PRIMARY KEY AUTOINCREMENT`) for every non-Firebird engine, so it failed on PostgreSQL with a syntax error and never applied a single file. Engine detection now unwraps the `CachedDatabaseAdapter` that `initDatabase` returns, so PostgreSQL is no longer mis-detected as SQLite - the source of the `AUTOINCREMENT` failure - and the tracking-table id is engine-aware (`SERIAL` on PostgreSQL, `AUTO_INCREMENT` on MySQL, `IDENTITY` on MSSQL, `AUTOINCREMENT` on SQLite). `initDatabase` now sets the database type, fixing a "tina4_sequences does not exist" error on PostgreSQL. The existing migration tests only covered SQLite, which is why this shipped; a gated live-PostgreSQL migration test now guards the engine-aware path. **ORM:** a UUID primary-key insert returns its id. **RabbitMQ:** the hand-rolled backend negotiates `TuneOk` and three frame-buffer bugs are fixed, so it works against a real broker for the first time. No new third-party runtime dependencies. All four suites pass against live services.

A parity sweep shipped in the same release. Session storage now works against a live server. The MongoDB and Valkey/Redis session handlers bridged their synchronous interface to async sockets through a child script that read the reply on the socket `end` event, which a live Redis or Mongo connection never sends, so every read, write, and destroy waited out its timeout and surfaced as a transport failure. Both handlers round-trip for real now: the Redis/Valkey transport parses RESP incrementally as bytes arrive, and the MongoDB transport uses the `mongodb` driver when it is installed with a corrected raw OP_MSG fallback (the command name comes first, the BSON is little-endian, and the response is decoded properly). The server honours the `autoCrud` opt-in flag, so CRUD routes generate only for models that set it. `rollback()` returns the down-migration file it ran. The MCP `migration_status` and `migration_run` tools call the real migration API. The GraphQL executor awaits async resolvers, a deliberate Node-only divergence from the synchronous Python, PHP, and Ruby resolvers. The smoke tests were rewritten to assert real behaviour.

MySQL and MSSQL join the provisioned test services (#262). Both engines now run live round-trip tests by default, gated on reachability the same way the other services are. The non-skippable real-service gate fails on a MySQL or MSSQL skip under `TINA4_REQUIRE_SERVICES`, so a missing engine in CI breaks the build instead of passing quiet. CI gained a MySQL 8 container and a SQL Server 2022 container. Running the suites against these two engines for the first time surfaced adapter bugs that no prior test could reach.

One adapter fix landed in Node. Boolean binding is fixed in the SQLite adapter: a raw boolean now binds as `1`/`0` at the parameter boundary instead of crashing or stringifying. Two adapter behaviours were already correct and earned regression tests to keep them that way. `getLastId()` after an insert returns the new auto-increment or `IDENTITY` value and survives a later `SELECT`. The MSSQL row-count probe handles a query that ends in `ORDER BY`, where the Python master needed a fix and Node did not.

The no-mock rule reached further this release (#250). The messenger SMTP and IMAP tests now talk to a real GreenMail mail server, the WebSocket backplane tests run against a real Redis backplane, and the HTTP-client tests hit a real loopback HTTP server. Every in-test double in those paths is gone. The dev mailbox gained a non-ASCII round-trip regression test for parity with the family: write a message with an accented name, read it back, confirm the bytes survive. Node reads the message files with an explicit UTF-8 decode, so it never carried the decode bug that bit Ruby, but the test now guards the contract.

v3.13.43 (2026-06-22) - Queue: MongoDB no longer re-delivers completed jobs

Node's MongoDB queue ran a split-brain lifecycle: `push`/`pop` went to MongoDB, but `complete()`/`fail()` routed to the local file backend, and the MongoDB backend had no acknowledge operation at all. A job popped from MongoDB was reserved but never acknowledged on `complete()`, so the visibility-timeout reclaim re-delivered every completed job after the window elapsed. This release gives the MongoDB backend the full lifecycle (`complete`/`fail`/`retry`/`deadLetters`/`retryFailed`/`failed`/`purge`) and routes the queue's lifecycle to the active backend: `complete()` now acks the MongoDB reservation, `fail()` requeues under `maxRetries` or dead-letters at the limit, and `pop` carries the job's topic so the ack lands on the right documents. Also fixes the options-object form of `consume({ topic })`, which ignored its `topic` and drained the construction-time queue (the string-argument form was already correct). RabbitMQ (no-ack on get) and Kafka (offset-based) auto-acknowledge or delegate redelivery to the broker, so they are unchanged. Verified end-to-end against a live MongoDB: complete then wait past the window yields no redelivery; topic isolation and fail-to-dead-letter hold. Full suite: 4,669 passing.

v3.13.42 (2026-06-22) - Swagger configurability for external and public APIs

Closes four gaps that pushed teams to hand-roll their own OpenAPI spec instead of using the built-in generator. **Configurable security schemes:** the built-in `bearerAuth` scheme honours `TINA4_SWAGGER_BEARER_FORMAT` (default `JWT`; set `opaque` for `sk_live_`-style keys), and setting `TINA4_SWAGGER_API_KEY_NAME` emits an `apiKeyAuth` scheme (`TINA4_SWAGGER_API_KEY_IN` is `header`/`query`/`cookie`). Register any scheme - including an `oauth2` flow with scopes - programmatically with `addSecurityScheme(name, definition)` (`resetRegistry()` clears it), both exported from `@tina4/swagger`. **Per-route security:** a route declares its own requirement through its `meta` - `security` as a scheme name plus a `scopes` array, a `{name: [scopes]}` map, a list of maps for an OR requirement, or `"public"` to mark a write route open (emits `security: []`). A secured route with no explicit declaration falls back to `TINA4_SWAGGER_DEFAULT_SCHEME` (default `bearerAuth`). Scopes stay spec-valid: only `oauth2`/`openIdConnect` schemes carry them, every other type gets `[]`, so the output validates against 3.0 and 3.1. **Path filtering:** `TINA4_SWAGGER_INCLUDE` documents only routes whose path starts with one of its comma-separated prefixes; `TINA4_SWAGGER_EXCLUDE` drops matching prefixes; framework internals (`/swagger`, `/__dev`) are always excluded. **OpenAPI 3.1 opt-in:** `TINA4_SWAGGER_OPENAPI` (default `3.0.3`) emits `3.1.0` when set to `3.1`/`3.1.0`. **Reusable component schemas:** register a shared shape with `addSchema(name, schema)` and reference it from a route's `requestSchema` / `responseSchemas` meta, extending the ORM-model `$ref` mechanism to arbitrary schemas. Identical behaviour and tests across all four frameworks. Zero new third-party dependencies. Full suite: 4,639 passing across 128 files.

v3.13.41 (2026-06-22) - Queue reservation/visibility timeout (at-least-once delivery)

A targeted fix for silent job loss in multi-replica / rolling-deploy setups. When a consumer reserved a queue message and then died before acknowledging - a crash, an OOM kill, a Kubernetes pod eviction - the message was stranded forever: never re-delivered, never retried, never dead-lettered. The file backend deleted the job on pop (lost outright); the MongoDB backend flipped the document to `reserved` without advancing `availableAt` and never re-evaluated it. Now a popped job is held as a reservation with `availableAt = now + visibilityTimeout` (plus a `reservedAt` stamp). If the consumer does not acknowledge in time, the next pop reclaims the abandoned reservation: it increments `attempts` and re-enqueues the job, or dead-letters it once it has hit `maxRetries`. A dead consumer can no longer strand a job - standard at-least-once delivery, the contract SQS and RabbitMQ already provide. The window is configurable via `TINA4_QUEUE_VISIBILITY_TIMEOUT` (default 300 seconds; `<= 0` disables the reclaim) or the per-queue `visibilityTimeout` option; `complete()`/`fail()`/`retry()` clear the reservation. RabbitMQ and Kafka are unchanged - the broker already owns redelivery there. Regression tests lock the behaviour in across all four frameworks (file backend: reclaim after the timeout, no reclaim before it, dead-letter past maxRetries, complete/fail clear the reservation, env override, disable-at-zero; MongoDB: dequeue advances `availableAt` and the reclaim requeues or dead-letters). Zero new third-party dependencies. Full suite: 4,618 passing across 127 files.

---

v3.13.40 (2026-06-22) - MCP security hardening + Swagger/OpenAPI overhaul

A coordinated cross-framework release with two themes: MCP transport security and a full Swagger/OpenAPI sweep. **MCP security:** the built-in dev MCP server now authorises every request on the raw socket peer rather than a configured host name, closing a remote-reach surface where a debug box bound to `0.0.0.0` exposed the file and database tools to unauthenticated callers. The gate is two layers - a host-independent capability check (`TINA4_MCP`, else `TINA4_DEBUG`) and a per-request authorisation: loopback always passes, while a remote caller needs `TINA4_MCP_REMOTE=true` AND a token matching `TINA4_MCP_TOKEN` (fallback `TINA4_API_KEY`), sent as `Authorization: Bearer`, `X-MCP-Token`, or `X-Api-Key`. Every MCP surface returns 404 to a disallowed caller, the SQL tool is read-only (SELECT/WITH, no stacked statements), and the file tools are sandboxed to the project root. **Swagger:** the production on/off switch is wired for real - set `TINA4_SWAGGER_ENABLED=false` to disable `/swagger` and `/swagger/openapi.json` in any environment, or `true` to expose them in production (it falls back to `TINA4_DEBUG` when unset). Secured routes now carry a `bearerAuth` security requirement, so the documentation no longer presents protected endpoints as public. ORM models become reusable `components.schemas` referenced by `$ref` across all four frameworks. The spec is valid where it was not: wildcard and splat routes emit proper `{name}` path parameters, `operationId` values are de-duplicated, and WebSocket routes no longer leak an invalid method. **Swagger configuration:** `TINA4_SWAGGER_SERVERS` (comma-separated) drives a multi-server block, `TINA4_SWAGGER_UI_CDN` points the UI assets at a self-hosted mirror for air-gapped use, and the generator adds typed-parameter formats, enums, top-level tags, and multipart request bodies. **SqliteDocStore (new):** a pymongo-style document store with a zero-config SQLite fallback. `getCollection(name)` returns a real Mongo collection when a Mongo URI is set (`TINA4_MONGO_URI`, then `TINA4_SESSION_MONGO_URI`, then the legacy `TINA4_SESSION_MONGO_URL`), otherwise a SQLite-backed collection over a local file (`TINA4_DOC_STORE_PATH`, default `data/tina4_docstore.db`) - the call sites are identical, only the backend differs (sync in the serverless path, a Promise on the real-Mongo path). It pushes filters down to JSON1 `json_extract` (equality, `$in`, `$nin`, `$gt`/`$gte`/`$lt`/`$lte`, `$ne`, `$exists`, `$regex`, `$or`, `$and`, dotted nested keys), supports `$set`/`$unset`/`$inc`/replace/upsert with lazy `sort`/`limit`/`skip`/projection cursors, ships a zero-dependency 12-byte ObjectId, and round-trips Dates and ObjectIds so values stay queryable. Develop against the local store and switch to MongoDB in production by setting one env var. **Developer experience:** the blank-`TINA4_SECRET` warning now explains why it fired - the run was not detected as development - and gives both fixes (set `TINA4_SECRET`, or set `TINA4_DEBUG=true` to auto-generate one into `.env.local`); the legacy-env strict check now hints the names may come from a `.env` baked into a Docker image and points at `tina4 env --migrate`. **Session Mongo env parity:** the session and DocStore Mongo URI is canonical as `TINA4_SESSION_MONGO_URI` across all four frameworks, with `TINA4_SESSION_MONGO_URL` kept as a back-compat legacy alias (Python and PHP historically read `_URL`). **Tests:** new DB contract tests (execute raises on a bad statement, read-after-write, generator monotonicity, transaction bracketing) and a queue isolation contract (a job in one topic never leaks into another, and a queue on a fresh storage path starts empty). **Breaking:** the Swagger and MCP production gates are now enforced - if you relied on `/swagger` being reachable in production, set `TINA4_SWAGGER_ENABLED=true`, and a

remote MCP caller now needs `TINA4_MCP_REMOTE=true` and a token. Zero new third-party dependencies. Full suite: 4,577 passing across 127 files.

v3.13.39 (2026-06-21) - Auto-migration, unified critical log level, fail-loud ORM, per-route WebSocket auth

A cross-framework parity sweep that hardens the data layer and tightens a few safe-by-default behaviours. **Migrations:** pending migrations can now run on startup, gated by `TINA4_AUTO_MIGRATE` and off by default so existing apps are untouched. A footgun and clear-bug pass adds stop-at-failure, MSSQL bootstrap, numeric-aware ordering (so `10_` sorts after `2_`, not after `1_`), `CREATE TABLE` idempotency, a URL-safe `//` delimiter, and smart/curly-quote normalization in migration SQL before it runs. **ORM:** `save()`, `create()`, `QueryBuilder`, and the Mongo path now fail loud instead of swallowing errors - `save()` validates first and returns `false` with the reason on `getError()`, `create()` resolves to `false` when the write fails, and the silent fallbacks are gone. **Logging:** `critical` is now a first-class top-level severity, a new `Log.isEnabled(level)` lets callers skip building an expensive log payload, and logs default to stdout-only in production and container environments to avoid file bloat. **WebSockets:** a route can require authentication on the upgrade itself, and user WebSocket routes are now wired into the integrated server. **Security:** the `/__dev/mcp` endpoint enforces its localhost guard and honors `TINA4_MCP_REMOTE`, and the built-in `Api` client strips the `Authorization` header on a cross-origin redirect and gains opt-in retry with backoff. **Env:** `TINA4_CORS_CREDENTIALS` now defaults to `false` (was `true`), with a uniform `.env.example` and AI hosts defaulting to localhost. **Tooling:** the `tina4 metrics` coverage detection now counts full-package-path imports, short (3-character) class names, and dynamic and inline-type imports, the complexity counter strips string literals before counting, and Kafka TLS/SASL config reaches the producer and consumer. Breaking: `critical` is now a top-level severity and the previous toggle is removed; `TINA4_CORS_CREDENTIALS` now defaults to `false`. Full suite: 4,459 passing.

v3.13.38 (2026-06-19) - Coordinated security & robustness release

A large bundled release closing a cross-framework hardening sweep. **WebSockets:** the Redis/NATS backplane is now wired for real - local-first delivery, then a published envelope on the shared `tina4:ws` channel, relayed with an origin guard (no own-echo, no cluster loop) - plus an origin allow-list (`TINA4_WS_ALLOWED_ORIGINS`), an idle reaper (`TINA4_WS_IDLE_TIMEOUT`), slow-client backpressure drop (`TINA4_WS_MAX_BACKLOG`), and SSE hardening (heartbeat + mid-stream error + client disconnect). **Sessions:** the external handlers now throw a transport error instead of silently dropping data, with a log-loud-and-degrade boundary (`TINA4_SESSION_STRICT` to re-raise). **GraphQL/WSDL:** a SOAP `<!DOCTYPE>` is rejected before parsing, non-numeric SOAP int/float params now fault instead of silently yielding `NaN`, a recursion-depth guard (`TINA4_GRAPHQL_MAX_DEPTH`, default 50) catches deep queries **and** circular fragments, resolver/SOAP faults are masked in production (full detail only under `TINA4_DEBUG`), and GraphQL fragment spreads + inline fragments now parse and resolve (the parser used to error on `...`). **Tooling:** a new `tina4 metrics` command reports the top-N code-health offenders with `--top/--json/--fail-on/--path`, the coverage test-detection is now precise (a real import / defined-class reference, not a name-substring scan), `@tina4/orm` now has **zero hard dependencies** (`pg`/`mongodb` are optional), and the repo type-checks clean under tsc (`npm run typecheck`). Zero new third-party dependencies. Full suite: 4,244 passing.

v3.13.37 (2026-06-18) - Dev-admin editor: TypeScript + Ruby highlighting fixed

The dev-admin file-read endpoint returned `{path, content, bytes}` with **no** `language` **field**, so the dashboard editor highlighted nothing - including `.ts`. It now returns a `language` (canonical extension map matching the Python master, plus no-extension `Dockerfile`), and the rebuilt editor bundle adds the Ruby/Rust/Go/Java/SCSS grammars. `.ts` and `.rb` now highlight correctly. Dev-mode tooling only. Full suite: 3,980 passing.

v3.13.36 (2026-06-18) - Instant WebSocket dev-reload + dev-admin file browser fix

Dev-reload is now a WebSocket push, matching Python. `tina4 serve` POSTs `/__dev/api/reload`; the server re-imports changed routes in-process (no respawn, same PID) and broadcasts `{type, file, mtime}` over the `/__dev_reload` WebSocket upgrade route (debug-only, never mounted on the stable AI port). The injected client is WebSocket-primary and only polls `/__dev/api/mtime` when the socket drops. **Also fixed:** the dev-admin file browser returned `type` instead of `is_dir`, so folders never rendered in the dashboard tree - `/__dev/api/files` now returns `is_dir`, `has_children`, real per-entry `git_status` and the repo `branch`, full parity with Python/PHP. Full suite: 3,957 passing.

v3.13.35 (2026-06-17) - Live MCP endpoint + working DB tools for AI agents

The built-in MCP server is now reachable and its database tools actually work. It was fully built but never mounted; ~10 dev tools used a bare `require` that's undefined under ESM (so they errored); and the DB tools read a `globalThis.__tina4_db` that nothing set - and called the async `Database` methods synchronously. Now: `DevAdmin.register()` mounts `/__dev/mcp` (JSON-RPC) + `/__dev/mcp/sse` in debug mode; `initDatabase()` exposes the Database on `globalThis.__tina4_db`; the tool dispatch is async end to end; and the require-based tools resolve correctly. An AI agent (Claude Desktop/Code) gets live access (real DB queries, file I/O, routes, docs) scoped to the running project. New regression tests; full suite 3,909 passing.

v3.13.34 (2026-06-17) - Scaffolder fix + dual-port reload correction

`npx tina4nodejs init` scaffolded an unresolvable `"tina4-nodejs": "^0.0.1"` dependency and dev/serve scripts that invoked the Rust CLI - fixed to `^3.0.0` and `npx tina4nodejs serve` so a pure-npm project installs and runs. Corrected the AI dual-port dev mode, which was inverted vs Python: the **main port now hot-reloads** (dev toolbar + `/__dev_reload` injected) and **port+1000 is the stable AI port** - previously reversed, so the `tina4` client's reload POST (which targets the base port) never reached the browser. Full suite: 3,858 passing.

v3.13.33 (2026-06-17) - Queues: priority pop + auto dead-lettering + TINA4*QUEUE*URL parity (Warning: behavioural change)

Behavioural change. `job.fail(reason)` now re-enqueues (incrementing `attempts` exactly once - a double-increment bug is fixed) until `attempts >= maxRetries`, then dead-letters - a `for await` consume loop retries automatically. `pop`/`consume` are now priority-ordered (was FIFO); new additive `retryBackoff`. **Config parity:** the broker backends now read `TINA4_QUEUE_URL` like Python/PHP/Ruby (per-backend `TINA4_RABBITMQ_*`/`KAFKA_*`/`MONGO_*` vars remain as overrides). Only the file backend changed for lifecycle. Queue chapter rewritten to match. Full suite: 3,858 passing.

v3.13.32 (2026-06-17) - Caching: per-query bypass + string-middleware + X-Cache-TTL (chapter rewritten)

Added a per-query bypass - `await db.fetchAll(sql, params, limit, offset, { noCache: true })` (also `fetch`/`fetchOne`) skips lookup + store; the option is a trailing arg, not the params array. The `"ResponseCache:300"` string-middleware form now works (parity with Python/Ruby), and `responseCache` now also sets `X-Cache-TTL` alongside `X-Cache`. The caching chapter was rewritten to match code - correct async/await on every cache-aside example, real `cacheStats()` shapes, all seven backends + file fallback, the three cache layers - dropping earlier aspirational claims. Full suite: 3,801 passing.

v3.13.31 (2026-06-17) - Version alignment (no functional change in Node)

Cross-framework version alignment with the Ruby request/response parity release. Node's request/response surface (parsed `req.body`, `req.query`, case-insensitive headers, `req.files[...].content` as a Buffer, `res.json`/`redirect`/`file`/`stream`) was already in parity - no behavioural change here. Full suite: 3,775 passing.

v3.13.30 (2026-06-16) - Typed route params coerce + JWT expiry now in minutes (Warning: two breaking changes)

Two behavioural changes. (1) Typed path params now arrive coerced: `{id:int}` -> `number`, `{price:float}` -> `number` (other types and untyped params stay strings; matching unchanged) - previously the value was the string `"42"`. (2) `getToken` / `refreshToken` `expiresIn` is now in **minutes** (default 60), not seconds - matching Python/PHP/Ruby and Node's own docs; callers passing a seconds value (e.g. `3600`) must divide by 60. Both bring Node into cross-framework parity. Also fixed a stale `hashPassword` iteration-count docstring and a `refreshToken` signature drift in the guide. Full suite: 3,775 passing.

v3.13.29 (2026-06-16) - Live API search ranks qualified queries + resolves the public import path

Parity with the Python master fix for the `api_*` live-reflection tools. (Node's `Frond.addFilter`/`addGlobal`/`addTest` are normal class methods the reflector already sees - the metaprogramming gap that hit Python/PHP doesn't apply.)

- **Class-qualified ranking.** `api_search("Frond.addTest")` now ranks `Frond.addTest` first - the owning class, fqn segments, and an exact `Class.method` match are scored.
- **Natural-name lookups.** `api_class`/`api_method` resolve the published import path (`@tina4/orm.Database`) and a bare class name, not just the stored fqn.

The bundled AI skills now tell assistants to query `api_*` before guessing. Full suite: 3,756 passing.

v3.13.27 (2026-06-16) - Frond template-engine parity fixes

A 50-case cross-engine audit (every Frond tag, filter, and test rendered through all four frameworks with identical templates) surfaced two places where Node's output diverged from the Twig/Jinja standard. Both are now fixed to match:

- `{{ "%.2f" | format(value) }}` is now a real printf - it handles precision/width/flags (`%.2f` -> `3.14`) instead of only `%s`/`%d`, and it resolves a *variable* argument to its value. Unquoted filter arguments are now treated as variable references (a `VarRef` resolved at apply-time); quoted literals stay literal, numbers/bools/null are coerced.
- `nl2br` escapes its input, inserts `<br />`, and is marked safe (it was emitting an un-safe `<br>` that the auto-escaper then escaped).

Behavioural note: these change rendered output for the affected filters - correctness fixes toward the documented Twig/Jinja behaviour. Full suite: 3,752 passing.

v3.13.26 (2026-06-16) - pooling fix: standalone writes auto-commit; explicit transactions stay atomic

Behavioural default change. A standalone write - `execute`/`insert`/`update`/`delete` made **outside** an explicit transaction - now **auto-commits on its own connection before returning** (`autoCommit` default flipped to *on*). Previously autocommit was off by default, which broke connection pooling: a standalone write stayed uncommitted on one pooled connection while the next read round-robined to a different connection and saw nothing.

Explicit transactions stay atomic. The per-statement commit branches now also check whether a transaction adapter is pinned to the current async context (`AsyncLocalStorage`) and suppress the commit inside `startTransaction()` ... `commit()`/`rollback()`, so a `rollback()` still discards everything. Set `TINA4_AUTOCOMMIT=false` for strict manual-commit mode.

Verified live on PostgreSQL: standalone write visible from a separate connection, explicit rollback discards, explicit commit persists, and pooled standalone writes visible across every round-robin connection. Full suite: 3,748 passing.

v3.13.25 (2026-06-16) - Node.js: distributed responseCache + persistent DB cache (async completion)

Node.js only. Completes the async cache work so Node reaches full parity with Python, PHP, and Ruby on the *automatic* cache paths. Previously (v3.13.24) Node's `responseCache` middleware and persistent DB query cache ran in-process (per-instance) because the middleware runner and `db.fetch()` were synchronous; distributed caching needed the explicit KV API.

Now the middleware runner (`MiddlewareRunner.runBefore`/`runAfter`) is **async**, so the `responseCache` middleware routes GET-response caching through the unified async backend - cached responses **distribute across instances** via `redis`/`valkey`/`memcached`/`mongodb` (selected by `TINA4_CACHE_BACKEND`). The **persistent DB query cache** (`TINA4_DB_CACHE=true`) routes through the async `fetchAsync` path to the same backend (`TINA4_DB_CACHE_BACKEND` + `TINA4_DB_CACHE_URL`), so multiple instances share one DB-query cache with global write-invalidation. The previous in-process-only restriction and its warning are gone.

The default backend remains `memory` (in-process - behaviour unchanged for apps that don't opt into a network backend); the request-scoped auto cache (`TINA4_AUTO_CACHING`) stays in-process by design (ephemeral, fastest). All network I/O is native async (no child processes).

Full suite: 3,804 passing.

v3.13.24 (2026-06-15) - unified cache backends across response, KV, and persistent DB cache

The response/KV cache now supports **seven backends**, selected by `TINA4_CACHE_BACKEND`: `memory` (default), `file`, `redis`, `valkey`, `memcached`, `mongodb`, and `database`. `TINA4_CACHE_URL` carries the connection string for `redis`/`valkey`/`memcached`/`mongodb`, or a SQL URL for the `database` backend (which falls back to `TINA4_DATABASE_URL`). Credentials can be embedded in the URL (`redis://user:pass@host`, `redis://:pass@host`, `mongodb://user:pass@host`) or supplied via `TINA4_CACHE_USERNAME` / `TINA4_CACHE_PASSWORD` (mirroring `TINA4_DATABASE_USERNAME`/`_PASSWORD`); memcached is unauthenticated. The usual `TINA4_CACHE_TTL` (60), `TINA4_CACHE_MAX_ENTRIES` (1000), and `TINA4_CACHE_DIR` (`data/cache`) still apply.

Graceful fallback: if a configured backend's driver is missing or the service/credentials are unreachable or wrong, the cache logs a warning and falls back to the **file** backend - a real persistent cache, never a silent no-op.

The **persistent DB query cache** (`TINA4_DB_CACHE=true`) now routes through the same backend set via `TINA4_DB_CACHE_BACKEND` + `TINA4_DB_CACHE_URL`. `db.cacheStats()` reports `mode`, and the KV `cacheStats()` reports a `backend` field.

Node characteristic (by design): Node's KV API is **async** - `await cacheGet(...)`, `await cacheSet(...)`, etc. - matching Node's async-everywhere idiom, and all seven backends use native async clients (no child processes). Because Node's middleware runner and `db.fetch()` are synchronous, the `responseCache` **middleware and the persistent DB query cache run in-process (per-instance) in Node**; for distributed, cross-instance caching in Node, use the async KV API (`await cacheGet`/`cacheSet`). The other three frameworks route those auto-paths through the configured backend (distributed). A full async middleware/DB pipeline is a future-major item.

Full suite: 3,787 passing.

v3.13.23 (2026-06-15) - request-scoped DB query cache, on by default (+ cache fixes)

A new **request-scoped query cache** protects your database from rapid repeat reads. Within a single request, identical `SELECT`s and ORM reads are deduped automatically - the DB is hit once and subsequent identical reads are served from memory. The cache is **cleared at the start of every request** (so it never serves stale rows across requests) and **flushed on any write** (insert/update/delete/execute). For non-request contexts (scripts, workers) a short safety TTL applies.

It is **on by default** via `TINA4_AUTO_CACHING=true` (off-switch `TINA4_AUTO_CACHING=false`); the in-request TTL is `TINA4_AUTO_CACHING_TTL` (default 5 seconds). The existing `TINA4_DB_CACHE` (default `false`) remains the separate *persistent* cross-request cache (TTL `TINA4_DB_CACHE_TTL`, default 30s) and is not cleared per request. `db.cacheStats()` now reports a `mode` field: `"request"` (default), `"persistent"`, or `"off"`.

Also fixed (Node): `cacheStats()` now reflects the real KV backend (it was wrongly reading the response-cache middleware store). And the DB query cache - previously dead code, where `db.cacheStats()` hardcoded `size: 0`, `db.cacheClear()` was a no-op, and the cache wrapper was never applied - now actually caches `db.fetch()` **and** ORM reads, with real `db.cacheStats()` / `db.cacheClear()`.

Full suite: 3,708 passing.

v3.13.21 (2026-06-15) - docs: `render()` corrections + version re-sync

Documentation consistency pass - no behavior change. The `res.template(...)` reference in `llms.txt` and a stale `server.ts` comment are corrected to `res.render(...)` - the real method; `template` is only the route-level binding (`export const template`), not a response method. Version re-synced to 3.13.21 with the other frameworks (this release also carries a Python-side JWT-secret security hardening).

Full suite: 3,684 passing.

v3.13.20 (2026-06-15) - Node.js: global class middleware (`Router.use`) now runs

Node.js only. Class-based middleware registered globally with `Router.use(SomeMiddleware)` was never executed - only per-route `.middleware(fn)` and the built-in CORS / logger / rate-limiter chain ran. The documented pattern (register a `beforeX`/`afterX` class once and have it apply to every route) silently did nothing.

`startServer` now runs every globally-registered class middleware around each route handler: `beforeX` hooks run **before** the handler (they can set response headers, mutate the request, or short-circuit by setting a status >= 400), and `afterX` hooks run **after** it. This brings Node to parity with Python, PHP, and Ruby, whose `Router.use` class middleware already ran.

```
class PoweredBy {
  static beforePoweredBy(req, res) {
    res.header("X-Powered-By", "Tina4");
    return [req, res];
  }
}
Router.use(PoweredBy);   // now applies to every response
```

Note: in Node the response is flushed by the handler, so set response headers in `beforeX` (they persist through the handler's write); `afterX` is for logging / post-processing (header changes after the body is sent are no-ops). Full suite: 3,684 passing.

v3.13.19 (2026-06-15) - return domain objects, construct from JSON, and one database binder

Three ergonomic improvements surfaced by the live side-by-side review of the book's own examples across all four frameworks.

`response(...)` serializes domain objects

Return an ORM model, an array of models, or a query result straight from a route - Tina4 serializes it to JSON. No more hand-rolled `toDict()` / `toJson()`:

```
get("/api/users", async (req, res) => {
  res.json(await User.all());        // array of models -> JSON array
});
```

A single model becomes a JSON object; an array of models or a `DatabaseResult` becomes a JSON array. Plain objects, arrays and strings behave exactly as before - purely additive.

Construct a model from a JSON object string

```
new User('{"name": "Alice"}');     // JSON object string -> one record
new User({ name: "Alice" });       // still works
```

Passing an **array** to a single-record constructor now throws a clear `TypeError` (previously it silently produced an empty model). To build many records, map over the list.

One database binder: `bindDatabase` (+ named connections)

Node gains a public `bindDatabase(adapter, name?)`. This is **not a breaking change** - `initDatabase()` (which auto-binds the `.env` default) and the internal `setAdapter()` are unchanged.

```
// Most apps: nothing to do - initDatabase() auto-binds the .env default at boot.

bindDatabase(adapter);                      // set/override the default explicitly

// Register a NAMED connection and point a model at it:
bindDatabase(await createAdapterFromUrl("postgres://u:p@.../analytics"), "analytics");

class Visit extends BaseModel {
  static _db = "analytics";          // uses the analytics connection
}
```

`bindDatabase(adapter, "...")` registers a named connection; a model selects it with `static _db = "..."`. A mistyped/missing named connection now throws a clear error instead of silently falling back to the default.

Full suite: 3,679 passing. Shipped with parity across all four frameworks (where the binder is named `bind_database` in Python/Ruby and `bindDatabase` in PHP/Node).

v3.13.18 (2026-06-15) - ORM eager-load + include + aggregate fixes

Found by the live side-by-side validation against PostgreSQL. (No v3.13.17 - that was a PHP/Ruby release; Node goes 3.13.16 -> 3.13.18.)

- **Eager load (`include`) silently returned no relations** in standalone use - `_eagerLoad` processed foreign keys only on the parent model, but the `hasMany` registry entry is registered by the *child* model's `_processForeignKeys()`, which is never called outside server boot. It now processes all registered models' FKs, so `Model.findById(id, ["Related"])` populates relations as documented.

- `include` **keys are now resilient** - matched case-insensitively against the model name, its singular/plural key, or the related table name; an include name that matches nothing emits a `Log.warn` instead of silently doing nothing.
- **Aggregate columns return numbers** - `SUM()`/`AVG()` came back as strings (node-postgres returns `int8`/`numeric` as strings). The PostgreSQL adapter now registers type parsers (`int8`, `numeric` -> number) so aggregates match Python/Ruby/PHP. (Values beyond `Number.MAX_SAFE_INTEGER` lose precision - documented; cast to `::text` when exactness is needed.)

Full suite: 3,653 passing.

v3.13.16 (2026-06-15) - Warning: Async database API (BREAKING) + `createTable` on PostgreSQL + result indexing

Found by the live documentation-verification pass - running the book's own samples against a real PostgreSQL database. The entire documented `Database`/`BaseModel`/`QueryBuilder` API was unusable on PostgreSQL (and MySQL/MSSQL/Firebird/MongoDB): every call threw `Use fetchAsync() for PostgreSQL.`

Warning: Breaking: the database / ORM / QueryBuilder API is now uniformly async

The Node DB layer was sync-first (built around synchronous `node:sqlite`). The async adapters implemented only `*Async` methods and made the sync methods throw - so the documented API worked **only on SQLite**. The public API is now uniformly **async** (returns Promises) and works identically across every engine - the cross-engine parity the docs always promised.

```
// before (worked only on SQLite):
const rows = db.fetch("SELECT * FROM users");
const user = User.find(1);
const list = QueryBuilder.fromTable("users").get();

// now (all engines, incl. PostgreSQL):
const rows = await db.fetch("SELECT * FROM users");
const user = await User.find(1);
const list = await QueryBuilder.fromTable("users").get();
```

Migration - add `await` to: `db.fetch / fetchOne / fetchAll / execute / executeMany / insert / update / delete / startTransaction / commit / rollback / tableExists / getTables / getColumns / getNextId`; all `BaseModel` operations (`save / find / findById / all / where / count / createTable / delete / ...`); and `QueryBuilder.get / first / count / exists`. Pure builders and serializers stay synchronous (`toSql`, `toMongo`, `toDict`, `toJson`) - so relationships must be eager-loaded (via `include`) before a synchronous `toDict`/`toJson`.

`createTable` engine-aware on PostgreSQL

Now emits `TIMESTAMP` for datetime, native `BOOLEAN` for boolean, and `SERIAL` for auto-increment on PostgreSQL; a failed `CREATE` no longer reports success.

`result[0]` index access

The book documents `const firstUser = result[0];` `DatabaseResult` now supports integer index access (alongside iteration, `length`, and `.at()`).

Verified against PostgreSQL 16: `db.fetch/fetchOne/execute`, `BaseModel.createTable + save + findById + all + count`, and `QueryBuilder.get/count/exists` all work. New PG-backed test; 10 SQLite test files updated to `await` (the intended breaking change). Full suite: 3,644 passing across 97 files.

v3.13.14 (2026-06-13) - Logs reach stdout in containers + per-request logging + schema-qualified tables (#48)

Cross-framework release (all four). Deployed Docker containers were getting no application logs. In production Node's logger gated console output behind `!Log.isProduction()` (which is `!TINA4_DEBUG`), so a deployed app - where `TINA4_DEBUG` is off - printed nothing to stdout, writing only to `logs/tina4.log` inside the container. `docker logs` reads PID 1 stdout, so it was empty. A follow-on report - the dev server going silent after startup - surfaced a second gap in the other frameworks: requests weren't logged.

Per-request logging - now gated, routed through Log

Node already logged every request, but via a bare `console.log` with a status-first format, and **always on** (even in production). v3.13.14 aligns it with the family:

```
2026-06-12T10:15:03.221Z [INFO   ] GET /api/users -> 200 (12.3ms)
```

- Routed through the Tina4 `Log` (so prod -> JSON, dev -> human) instead of `console.log`.
- Gated by `TINA4_LOG_REQUESTS`: on by default in dev (`TINA4_DEBUG`), **off by default in production** (was always-on) so prod doesn't pay the per-request cost unless you opt in with `TINA4_LOG_REQUESTS=true`.
- Standard line format `METHOD /path -> STATUS (Nms)`, identical across all four frameworks (was `STATUS METHOD url ms`).

What changed (stdout)

- **Console output is no longer gated on `isProduction()`.** Logs go to stdout in production too (subject to `TINA4_LOG_OUTPUT` and level).
- **Production emits structured JSON** to both stdout and the file (parity with Python/Ruby - Node previously wrote *text* to the file in production unless `TINA4_LOG_FORMAT=json`). Dev keeps the coloured human-readable line.
- **Default log level is `INFO`** (was `DEBUG`).

```
// In a container (TINA4_DEBUG off), default config:
Log.info("worker started");
// pre-v3.13.14: console suppressed in production -> docker logs empty
// v3.13.14:    {"timestamp":"...","level":"INFO","message":"worker started"} on stdout
```

> *Node cluster workers (production auto-cluster) inherit the primary's stdio by default, so worker logs already propagate to the container's stdout - no change needed there.*

Why it spanned all four

The same logging-in-containers gap showed up in every framework:

Framework	Pre-v3.13.14 cause	Fix
Python	`not _is_production` gate suppressed stdout; default ERROR	stdout always on (flushed); default INFO
PHP	`$stdout = $development` (file-only in prod); no `TINA4_LOG_LEVEL` read	stdout default on + `fflush`; reads `TINA4_LOG_LEVEL`; default INFO
Ruby	stdout written but never flushed (block-buffered on non-TTY); default ALL	`$stdout.sync = true`; default INFO; accepts plain + bracket names
Node	`!isProduction()` gate suppressed console; default DEBUG	console always on; production emits JSON; default INFO

The Rust `tina4` CLI was already correct (inherits child stdio).

Schema-qualified tables (#48) + a PostgreSQL `fetch()` regression

Issue #48 - *"Database Table Does Not Exist"* on PostgreSQL. A model whose table lives in a non-default schema (`gift_cards.gift_card`, MSSQL `dbo.widget`, MySQL `otherdb.table`, SQLite ATTACH `extra.widget`) was invisible to the framework's introspection. `tableExists`, `getTables`, and `getColumns` hardcoded the default namespace (`public`) and matched the whole dotted string as one flat name - so plain reads worked, but `createTable`, migrations, and auto-CRUD were blind to the table and reported it missing.

A shared `SQLTranslator.splitSchema()` helper drives schema-awareness in every affected adapter:

- **PostgreSQL** - `tableExists` uses `to_regclass()` (honours schema + `search_path`); `getColumns` filters by `table_schema`; `getTables` lists every non-system schema and returns non-`public` tables schema-qualified.
- **MySQL** - schema = database; a qualified name checks that catalog, a bare name defaults to `DATABASE()` (`DESCRIBE` back-quotes each part).
- **MSSQL** - honours `dbo.table`; a bare name matches in any schema.
- **SQLite** - honours an ATTACH alias (`extra.widget`) for both `tableExists` and `getColumns`.
- **Firebird** - N/A (no schemas).

Verified against a live PostgreSQL 16 container: `tableExists('gift_cards.gift_card') -> true`, `getTables -> ['gift_cards.gift_card', 'gift_cards.transaction']`, `getColumns -> 12 columns` - identical results across all four frameworks.

> ***PHP also fixed a v3.13.12 regression found while cross-checking #48.*** *Its* `PostgresAdapter` *referenced* `stripTrailingSemicolons()` *(added in v3.13.12) and the new* `splitSchema()` *but never mixed in* `SqlNormalizerTrait` *- so* ***every PostgreSQL*** `fetch` **/** `fetchOne` **/** `getColumns` ***fatalled****. It shipped silently because the PostgreSQL test suite skips without a live server. Fixed and pinned by server-free reflection guards.*

Tests

- Node: 3,628 passed (+16 net - production JSON stdout; request-log gating, format, and Log routing; #48 schema split + SQLite ATTACH introspection)
- Family: Python 2,829 | PHP 2,394 | Ruby 2,999 | Node 3,628 - **11,850 total, zero regressions.** (PHP also fixed #119, a `cli-server` boot crash, and the PG `fetch` regression above.)

---

v3.13.12 (2026-06-11) - SQL safety + implicit ORM binding + `fetchAll` correctness

Three high-impact fixes that close out long-standing footguns. All three ship with full parity across all four frameworks.

`fetchAll` actually fetches ALL rows now (no silent 100-row truncation)

Pre-v3.13.12 the Python/PHP/Ruby conveniences silently truncated at 100 rows. Node already had the correct semantics (the `limit` parameter is optional and `undefined` skips LIMIT injection at the adapter layer), but this release locks the contract in with explicit tests:

```
// 150 rows in the table
db.fetchAll("SELECT * FROM rows");           // -> 150 rows (always did, now tested)
db.fetchAll("SELECT * FROM rows", undefined, 10);   // -> 10 rows (explicit cap)
db.fetchAll("SELECT * FROM rows", undefined, 5, 20); // -> 5 rows starting at offset 20
```

`db.fetch()` (the paginated sibling that returns a `DatabaseResult` with count metadata) keeps its 100-row default at the HTTP query-builder layer - pagination is its job. Only the low-level `db.fetchAll()` convenience returns everything.

For very large tables, prefer `db.fetch()` (returns a `DatabaseResult` with count) or pass an explicit `limit` to `db.fetchAll()`.

Trailing `;` is now stripped from user SQL in `fetch()` / `fetchOne()`

The framework appends `LIMIT n OFFSET m` to the user-supplied query (and wraps it in `SELECT COUNT(*) FROM (...) AS subq` for the count probe). When the user's query already ended with a `;`, both rewrites broke:

```
db.fetch("SELECT * FROM users;")
// pre-v3.13.12: syntax error near "LIMIT" - the appended LIMIT followed a ;
// v3.13.12:    works - trailing ; is stripped before LIMIT is appended
```

The strip is conservative: only trailing whitespace + semicolons are removed (any number of them, including `;;`), nothing inside the statement is touched. Parameters and quoting are

unchanged - the existing parameter-binding defense against injection still does all the heavy lifting.

```
import { stripTrailingSemicolons } from "@tina4/orm";

stripTrailingSemicolons("SELECT 1; ");       // "SELECT 1"
stripTrailingSemicolons("SELECT 1;;  ");     // "SELECT 1"
stripTrailingSemicolons("SELECT ';' AS x;"); // "SELECT ';' AS x"  (string literal preserved)
```

Applied at the top of `Database.fetch()` and `Database.fetchOne()`.

Implicit ORM binding from `TINA4_DATABASE_URL`

Node already auto-discovered `TINA4_DATABASE_URL` via `initDatabase()` on the env-driven path - this release simply documents and pins it as parity behaviour. An explicit `initDatabase({ url, ... })` call still takes precedence and can be used to bind a second database.

Cross-framework parity

Fix	Python	PHP	Ruby	Node
`fetch_all`/`fetchAll` returns ALL rows by default	[x] `limit=0` default	[x] `$limit = 0` default	[x] `limit: nil` default	[x] already correct (`limit?` undefined)
Strip trailing `;` from fetch SQL	[x] shared helper on `DatabaseAdapter`	[x] `SqlNormalizerTrait` on 5 adapters	[x] `Tina4::Database.strip_trailing_semicolons`	[x] exported `stripTrailingSemicolons`
Implicit ORM binding from env	[x] already worked	[x] already worked	[x] **fixed** (wired `auto_discover_db`)	[x] already worked

Tests

- Python: 2,811 passed (+24 new)
- PHP: 2,316 passed (+13 new)
- Ruby: 2,980 passed (+18 new)
- Node: 3,612 passed across 95 files (+16 new)

11,719 tests across the family, +71 new for v3.13.12, zero regressions.

---

v3.13.11 (2026-06-11) - ORM correctness pass

Mirrors Python's ORM correctness pass. One Node-side change plus regression-pinning tests.

#50.2 - `save()` correctly INSERTs natural (non-auto-increment) PKs

Pre-v3.13.11 the BaseModel `save()` decided INSERT vs UPDATE purely on `pkValue != null`. For models with a user-supplied PK (e.g. `gift_card_number = "GC-100"` set before the first save), this always picked UPDATE - matched zero rows - and silently returned success without inserting anything.

v3.13.11 checks `ModelClass.exists(pkValue)` for non-auto-increment PKs:

```
class GiftCard extends BaseModel {
  static tableName = "gift_cards";
  static fields = {
    gift_card_number: { type: "string", primaryKey: true, maxLength: 50 },
    owner: { type: "string", maxLength: 120 },
  };
}

const gc = new GiftCard();
gc.gift_card_number = "GC-100";
gc.owner = "alice@example.com";
gc.save();                          // -> INSERT (pre-v3.13.11: silent UPDATE no-op)
GiftCard.find({ gift_card_number: "GC-100" });  // -> returns the row
```

Auto-increment PKs are unchanged: `pk == null -> INSERT`, `pk != null -> UPDATE`. The fix also stops the engine-assigned `lastInsertRowid` from overwriting a natural PK that the caller already set.

#50.1 - callable defaults (N/A in Node)

The Python/Ruby auto-default-application pattern doesn't exist in Node's `BaseModel`. The constructor only copies provided data; field defaults are metadata used by `createTable()` for the DDL `DEFAULT` clause, not auto-applied at construction. If a user wants a callable default, they set the field manually before `save()`. Worth noting for parity but no source change needed.

BooleanField engine-aware DDL (already correct in Node)

Node's per-adapter `fieldTypeTo*()` functions already mapped boolean to each engine's native type: PG -> `BOOLEAN`, MySQL -> `TINYINT(1)`, MSSQL -> `BIT`, Firebird -> `SMALLINT`, SQLite -> `INTEGER`. Pinned with a regression test on SQLite.

PG error-visibility fixes (Python only)

`node-postgres` uses libpq in autocommit mode - the InFailedSqlTransaction cascade that Python's psycopg2 produces never happens. No Node changes needed.

Tests

3,596 passed across 94 files (+10 new - `test/ormV3_13_11.test.ts`). No regressions.

---

v3.13.9 (2026-06-10)

Non-destructive AI installer - `installSelected()` / `installAll()` no longer clobber the user's `CLAUDE.md`. They write (or refresh) a marker-bracketed Tina4 skill block and leave the rest of the file alone.

The bug

Pre-v3.13.9 the installer wrote a full developer guide to `CLAUDE.md` (and to `.cursorules` / `.github/copilot-instructions.md` / `.windsurfrules` / `CONVENTIONS.md` / `.clinerules` / `AGENTS.md` / `.antigravity/context.md`) on every run, clobbering whatever the user had put there. Comment in the old code: *"Always overwrite -- user chose to install"* - but they didn't choose to lose their notes.

The fix

A marker-bracketed skill block - HTML comments for `.md` files, `#`-prefixed line comments for rule files:

```
<!-- tina4-skills:start -->
## Tina4 Skills

- **tina4-maintainer** - Read `.claude/skills/tina4-maintainer/SKILL.md` for framework-level cha
- **tina4-developer** - Read `.claude/skills/tina4-developer/SKILL.md` before building features.
- **tina4-js** - Read `.claude/skills/tina4-js/SKILL.md` for frontend work.
<!-- tina4-skills:end -->
```

Four behaviours:

- **Fresh install** -> write the framework guide plus the skill block.
- **Marker refresh** (idempotent) -> file exists with our markers -> replace only the bracketed block.
- **One-time migration** -> file starts with the pre-v3.13.9 framework header -> replace the old dump with the new framework guide + skill block.
- **Preserve user content** -> file exists with the user's own content (no markers, no old header) -> append the skill block to the end, leave everything else verbatim.

The helpers (`markersFor`, `skillBlock`, `hasMarkers`, `replaceMarkerBlock`, `looksLikeOldFrameworkInstall`, `writeOrMerge`) are exported from `packages/core/src/ai.ts` so external tooling can compose them.

Same algorithm in Python / PHP / Ruby

Identical four-branch logic, identical marker syntax, identical canonical action verbs in the log output. Skill content stays consistent across the family.

Tests

46 new assertions in `test/aiInstaller.test.ts`. All four branches plus marker detection, block replacement, idempotency, old-header detection, and rule-file vs markdown-file behaviour.

3,586 passed across 93 files - no regressions.

What you'll see when you re-install

```
[OK] Migrated (replaced old framework dump in) CLAUDE.md   <- first run after upgrade
[OK] Refreshed skill block in CLAUDE.md                     <- every subsequent run
[OK] Appended skill block to CLAUDE.md                      <- user-curated file
```

---

v3.13.7 (2026-06-10)

Two changes from the 24rent app-platform team (PLATFORM-2159) - one observability hook, one production-safety fix. Both ship across **all four frameworks** with identical event payload shape.

NEW: `tina4.request.error` event

When the dispatch catch fires for a thrown route exception, the server now emits `tina4.request.error` **before** rendering the 500 page. Listeners receive `{ exception, request }` and can ship the failure to CloudWatch / Sentry / Slack - even though the framework caught it.

```
import { Events, Log } from "@tina4/core";

Events.on("tina4.request.error", (payload: any) => {
  const err: Error = payload.exception;
  const req = payload.request;
  Log.error(`Route error: ${err.name}: ${err.message}`, {
    method: req?.method,
    path: req?.path,
  });
  // ...or POST to your centralised logging pipeline
});
```

- **Fires for caught route throwables.** Does NOT fire for 404s - those aren't server errors.
- **Listener errors are swallowed + warning-logged** so a broken listener can't break the 500 render.
- **Listeners fire in priority order** (higher priority first, matching `Events.on(event, cb, priority)`).
- **Identical event name + payload across Python / PHP / Ruby** - only the per-language syntax differs.

The dispatch catch also now calls `Log.error` with the exception name, message, method, and path. Previously route exceptions hit `console.error` (raw stderr); they now flow through the framework's structured logger so they reach the same sinks as everything else.

FIX: Stack trace removed from production 500 body (CWE-209)

Before v3.13.7, an unhandled route exception in Node would (in some configurations) render the raw `String(err)` into the 500 response body - exception name, message, and depending on the renderer, the stack - when `TINA4_DEBUG` was truthy. That's CWE-209 / OWASP A05: information disclosure.

The framework's own `packages/core/templates/errors/500.twig` now guards the trace block with `{% if error_message %}`. When `TINA4_DEBUG=false`, the dispatcher passes an empty `error_message` and the trace block doesn't render. The trace stays in `Log.error` (server-side) and reaches observability via the new event.

When `TINA4_DEBUG=true`, the rich `renderErrorOverlay()` page is unchanged.

Tests

14 new assertions in `test/routerErrorEvent.test.ts`: event payload shape, listener priority order, no traceback markers in prod body, request_id still surfaces, listener-error safety, multiple-listener fanout.

- 3,540 tests passing across 92 files, no regressions.

Background

Reported by DevProx on the 24rent platform - they centralise observability by scraping structured JSON lines from stderr -> CloudWatch -> a Slack notifier. Route-level exceptions weren't surfacing because the framework caught them silently. The event hook fixes that without forcing any team's logging convention; the trace-leak fix is independently a security concern.

---

v3.13.6 (2026-06-09)

Parity bump alongside Python's #46 / #47 fixes, plus a Node-side polish on driver install hints.

Better driver install hints (#47)

Missing-driver errors across all six adapters (PostgreSQL, MySQL, MSSQL, Firebird, ODBC, MongoDB) now suggest every common Node package manager instead of only `npm`:

```
PostgreSQL adapter requires the "pg" package. Install one of:
    npm install pg
    yarn add pg
    pnpm add pg
    bun add pg
```

Useful for monorepos and Bun/Yarn-first projects where the npm command is the wrong recommendation.

#46 - PostgreSQL transaction cascade (no fix needed)

The cascade behaviour that prompted Python's #46 fix is psycopg2-specific (DB-API 2.0 mandates an implicit transaction on first statement). `node-postgres` runs in libpq autocommit by default - each query is its own transaction, so a failed query does not poison subsequent ones. The async PostgreSQL adapter already returns the error in its result object:

```
const result = await db.executeAsync("SELECT * FROM does_not_exist");
result.success;  // false
result.error;    // 'relation "does_not_exist" does not exist'
```

Verified - no source change needed.

Tests

3,526 passing across 91 files.

---

v3.13.5 (2026-06-05)

Frond static-facade parity across PHP, Ruby, Node.js. Closes the last documented v3 parity gap (tina4-python task #32). Python's `Frond.add_filter` / `add_global` / `add_test` have worked as classmethods since v3.13.0 - now PHP / Ruby / Node match.

What changes

Filters, globals, and tests registered at app-startup persist across `new Frond()` instances. Every framework now supports the same pattern:

```
// PHP
\Tina4\Frond::addFilter("money", fn($v) => number_format((float)$v, 2));
\Tina4\Frond::addGlobal("APP_NAME", "My App");
\Tina4\Frond::addTest("positive", fn($v) => $v > 0);
```

```
# Ruby
Tina4::Frond.add_filter("money") { |v| "%.2f" % v.to_f }
Tina4::Frond.add_global("APP_NAME", "My App")
Tina4::Frond.add_test("positive") { |v| v > 0 }
```

```
// Node.js
Frond.addFilter("money", (v) => Number(v).toFixed(2));
Frond.addGlobal("APP_NAME", "My App");
Frond.addTest("positive", (v) => Number(v) > 0);
```

```
# Python - already shipped in v3.13.0
Frond.add_filter("money", lambda v: f"{float(v):.2f}")
Frond.add_global("APP_NAME", "My App")
Frond.add_test("positive", lambda v: v > 0)
```

In every framework, registering at the class level updates a static registry. The next `new Frond()` drains that registry into its own filter/global/test maps automatically. No need to thread a single `Frond` instance through the application - register at startup, render everywhere.

Instance form still works

Existing per-instance registration continues to work, and now propagates to the class registry too - so the lifecycle is symmetric:

```
$frond = new \Tina4\Frond();
$frond->addFilter("currency", $fn);
// Future `new Frond()` instances also see "currency"
```

`clearRegistry()` for test fixtures

Every framework exposes a class-level method to wipe user-registered filters/globals/tests without touching the built-ins (upper, lower, length, defined, even, ...). Useful in test setup/teardown to prevent state leaks between specs.

```
\Tina4\Frond::clearRegistry();
```

```
Tina4::Frond.clear_registry
```

```
Frond.clearRegistry();
```

```
Frond.clear_registry()
```

Implementation notes per framework

Framework	Mechanism
Python	`_ClassOrInstanceMethod` descriptor - one method, dual-callable via `__get__`
PHP	`__call` + `__callStatic` magic-method pair - PHP can't have same-name static and instance methods
Ruby	Same-name class method and instance method - Ruby naturally allows this
Node.js	TypeScript class supports same-name `static foo()` and `foo()` instance methods - distinct lookup spaces

Test count

Framework	Before	After	New
Python	2,741	2,741	0 (already covered)
PHP	2,858	2,871	+13
Ruby	2,907	2,928	+21
Node.js	3,508	3,526	+18
Total	**12,014**	**12,066**	**+52**

Upgrade

Drop-in patch. No breaking changes. Existing instance-form code (`$frond->addFilter(...)` / `frond.add_filter` / `frond.addFilter`) keeps working unchanged. The new static form is purely additive.

v3.13.4 (2026-06-04)

Three middleware/header bug fixes across all four frameworks, plus Python chapter 10 + 18 docs rewrites. Reported in tina4-book#140 and tina4-book#141 by MichaelC8E.

PY-10-02 - `@middleware()` no longer silently disables auth (SECURITY)

Before: Applying `@middleware(...)` to a POST/PUT/PATCH/DELETE route silently flipped `auth_required = false`, removing the framework's built-in Bearer-token gate. A developer adding custom logging or rate-limiting middleware to an admin endpoint would, with no warning, open it to unauthenticated callers.

After: Middleware is purely additive. Write routes stay Bearer-token-gated by default. Use `@noauth()` to open a write route, `@secured()` to lock a read route. Same rule across all four frameworks.

This is a **behaviour change** - if your code relied on the old auto-disable to handle auth in custom middleware, add `@noauth()` (and have your middleware enforce auth on its own).

PY-10-03 - `request.headers` is now case-insensitive

Before: `request.headers["Content-Type"]` returned `None`/`undefined`/`nil`. The dict was lowercase-only; mixed-case lookups silently failed. Six chapter 10 examples (`Content-Type`, `X-API-Key`, `Authorization`, `User-Agent`) were broken.

After: HTTP headers are case-insensitive per RFC 7230 Section 3.2. Same is true in every framework:

Framework	Implementation
Python	`CaseInsensitiveDict` (dict subclass, normalises string keys to lowercase on read/write)
PHP	`Tina4\CaseInsensitiveArray` (ArrayAccess + IteratorAggregate + Countable)
Ruby	`Tina4::CaseInsensitiveHash < Hash` (overrides `[]`, `[]=`, `key?`, `delete`, etc.)
Node	Proxy wrapper around `http.IncomingHttpHeaders`

`request.headers.get("Content-Type")`, `request.headers.get("content-type")`, and `request.headers.get("CONTENT-TYPE")` all return the same value. Existing lowercase code keeps working unchanged.

PY-10-01 - Function-based middleware now runs

Before: Chapter 10 taught Express-style `async def mw(req, resp, next_handler)` in 8+ examples, but the Python framework's dispatcher only looked for class-based `before_*`/`after_*` methods. Function-style middleware was silently inert - body never executed. PHP and Ruby had similar gaps (closures ran but no `next` continuation).

After: Express-style continuation chain is implemented across the family. Python adds `_is_function_middleware()` + `_invoke_handler_with_middleware()`. PHP wraps

closures with `array_reverse` continuation. Ruby uses lambdas + `reverse_each`. Node already had `next()` continuation - added a regression test to keep it green.

```
@middleware(my_mw)
@post("/api/orders")
async def create_order(req, resp):
    ...

async def my_mw(req, resp, next_handler):
    if not authorised(req):
        return resp.json({"error": "forbidden"}, 403)
    result = await next_handler(req, resp)   # continue the chain
    return result
```

First-declared middleware is the outermost layer; calling `next_handler` descends to the next layer (or the route handler if last). Omitting the `next_handler` call short-circuits the chain.

Python chapter rewrites - book + docs

- **Chapter 18 (Testing)** - Fixed PY-18-04 (test runner output now shows real pytest output, not the fictional `[PASS] test_addition` format), PY-18-07a (added missing `from src.orm.Product import Product` import), PY-18-08 (`resp.status_code` -> `resp.status` across 14+ call sites, positional body `self.post(path, dict)` -> keyword `self.post(path, json=dict)`).
- **Chapter 10 (Middleware)** - Added two callouts: headers are case-insensitive in v3.13.4+; `@middleware()` is purely additive (does not change auth_required). Existing mixed-case header examples now work against v3.13.4.

Test count

Framework	Before	After	New
Python	2,725	2,741	+16
PHP	2,844	2,858	+14
Ruby	2,887	2,906	+19
Node.js	3,477	3,508	+31
Total	**11,933**	**12,013**	**+80**

Upgrade

PY-10-02 is a behaviour change with a security implication. Audit routes that use `@middleware()` on POST/PUT/PATCH/DELETE: if you rely on custom middleware to handle auth, add `@noauth()` above `@middleware()` (and make sure your middleware enforces auth). Otherwise, no action - your write routes were always supposed to require Bearer tokens.

PY-10-01 and PY-10-03 are purely additive - no breaking changes.

v3.13.3 (2026-06-03)

Two reporter-driven ergonomic additions, shipped across all four frameworks with full parity per `feedback_parity`.

`Env` typed env-var helpers (tina4-python#43)

Reading env vars by hand gets old fast: every boolean flag becomes a `os.getenv("TINA4_DEBUG", "false").lower() in ("1", "true", "on", "yes")` incantation. Every numeric tuning knob needs a try/except around `int()`. `Env` centralises it:

```
from tina4_python import Env

debug   = Env.bool("TINA4_DEBUG", default=False)
workers = Env.int("WORKERS", default=4)
rate    = Env.float("RATE_LIMIT", default=10.0)
region  = Env.str("AWS_REGION", default="us-east-1")
```

Same API across all four frameworks:

- **Python** - `from tina4_python import Env`
- **PHP** - `Tina4\Env::bool / int / float / str`
- **Ruby** - `Tina4::Env.bool / int / float / str`
- **Node.js** - `import { Env } from "@tina4/core"`

Truthy tokens (case-insensitive after `strip`/`trim`): `1`, `true`, `on`, `yes`, `y`, `t`. Falsy: `0`, `false`, `off`, `no`, `n`, `f`, empty string. Anything else returns the `default` - never raises. `int`/`float` parse failures log a warning via `Log` and fall back to default.

Function-name in log lines (tina4-python#41)

Opt-in via `TINA4_LOG_FUNC=true`. When enabled, the calling function name is injected into every log line so a `tail -f` gives you free context:

```
2026-06-03T14:22:18.341Z [INFO   ] [super_trooper] Hello from inside the function
```

Or in JSON mode:

```
{"timestamp":"...","level":"INFO","function":"super_trooper","message":"Hello..."}
```

Default off - zero overhead unless opted in. When on, ~5% per-call cost from the stack walk.

Per-framework implementation:

- **Python** - `inspect.currentframe()` walk past Log's own frames
- **PHP** - `debug_backtrace(DEBUG_BACKTRACE_IGNORE_ARGS)` + `{closure}` filter
- **Ruby** - `caller_locations(2, 16)` + block-noise regex
- **Node.js** - `new Error().stack` regex parse + anonymous filter

Anonymous frames (`<lambda>`, `<module>`, `{closure}`, anonymous IIFEs) are filtered as noise - showing `[<lambda>]` would be uglier than nothing.

Test count

Framework	Before	After	New
Python	2,675	2,725	+50
PHP	2,780	2,844	+64
Ruby	2,839	2,887	+49 (+1 pre-existing rack_app fail unchanged)
Node.js	3,420	3,477	+57
Total	**11,714**	**11,933**	**+220**

Upgrade

Drop-in patch. No breaking changes. Two new exports (`Env`, plus one new env var `TINA4_LOG_FUNC`). Existing logs and existing code keep working unchanged.

v3.13.2 (2026-06-03)

Bug-fix patch - three field reports, fixed with full cross-framework parity audit per `feedback_crosscheck_bugs`.

SCSS calc() with mixed units (tina4-python#42, tina4-php#116, tina4-nodejs#1)

The SCSS math evaluator silently folded mixed-unit arithmetic by keeping operand 1's unit and dropping operand 2's, producing wrong CSS:

- `max-height: calc(100vh - 170px)` -> `calc(-70vh)` (negative, layout-breaking)
- `width: 100% - 20px` -> `80%` (pixel term silently lost)
- `padding: 1rem + 4px` -> `5rem`

Fixed in Python, PHP, and Node - the evaluator now captures both operand units, only folds when units match (or one side is unitless for `*`/`/`), and masks `calc(...)` ranges so the browser computes them as intended. Ruby unaffected (delegates to libsass).

Router.group docs taught a crashing pattern (tina4-python#44)

The Python book and docs site showed `Router.group("/api/v1", lambda: [...])` with a zero-arg lambda. Source intentionally passes a `RouteGroup` instance to the callback, so users hit `TypeError: <lambda>() takes 0 positional arguments but 1 was given`. Docs rewritten to `lambda group: [group.get(...), group.post(...)]` matching the real contract (Node has always taught this correctly; PHP and Ruby use ambient state, no group arg needed).

DATABASE*URL -> TINA4*DATABASE_URL drift (tina4-python#45)

Three real bugs:

• **Python ORM error message** told users to "set DATABASE_URL in .env" - but the v3.12 boot guard rejects that bare name. Users following the error message hit a hard stop.

• **Python dev-admin `.env` writer** stripped the `TINA4_` prefix when updating existing rows, actively corrupting the config every time the user saved a new connection through the dashboard.

• **Node `Database.fromEnv()`** defaulted to `"DATABASE_URL"` as the env-var key, missing the project's actual connection. The ORM error message had the same drift.

All fixed. PHP and Ruby audited - already correct in both.

Test count

Framework	Before	After	New
Python	2,665	2,675	+10
PHP	2,774	2,780	+6
Ruby	2,839	2,839	0 (parity bump only)
Node.js	3,406	3,420	+14
Total	**11,684**	**11,714**	**+30**

Upgrade

Drop-in patch. No breaking changes. No new public API.

v3.13.1 (2026-06-02)

Cross-framework parity patch. Closes the remaining audit-flagged docs-vs-code gaps that didn't make 3.13.0 - the documentation claimed APIs across PHP / Ruby / Node that only Python had. This release ships those APIs everywhere and rewrites the PHP chapters that referenced fictional symbols.

Convenience parity additions (Groups A / B)

Three highest-impact cross-framework methods every documentation set already claimed existed. PHP / Ruby / Node now match Python:

• `db.fetchAll(sql, params)` / `db.fetch_all` / `$db->fetchAll` - returns the records list directly. Symmetric with `fetch_one`. For the 80% case where you don't need the `DatabaseResult` metadata.

• `Database.getConnection(url)` / `.get_connection` / `::getConnection` - classmethod factory matching SQLAlchemy's `engine.connect()`. Falls back to in-memory SQLite when no URL or env resolves.

• `Api(bearerToken=, username=, password=, headers=, verifySsl=)` **ergonomic kwargs** - three setter calls collapse to one constructor. Bearer wins over basic-auth when both are passed. `verifySsl=False` is the positive form of `ignoreSsl=true`.

Decorator-style GraphQL resolvers across the family

Python `@GraphQL.resolve` shipped in 3.13.0. This release adds:

- **PHP** - `GraphQL::resolve("Type", "field", $callable)` static method + class-level resolver registry that `new GraphQL()` drains into its schema.
- **Ruby** - `Tina4::GraphQL.resolve("Type", "field") { |root, args, ctx| ... }` with block-based registration.
- **Node.js** - `GraphQL.resolve(typeName, fieldName, resolver)` matching the cross-framework shape.

All four frameworks now support the FastAPI / Strawberry / Ariadne pattern where resolvers register at module-import time before any `GraphQL` instance is constructed, and where post-startup registrations land in the active default singleton via `setDefault(gql)` / `Tina4::GraphQL.default_instance = gql`.

Class-based service pattern across the family

`class FooWorker extends Service { run() { ... } }` - chapter 27 / equivalent docs have long taught this pattern. Until 3.13.1, only the runner was real:

- **PHP** - new `Tina4\Service` abstract base class + `ServiceRunner::registerService($name, $service)` static helper.
- **Ruby** - new `Tina4::Service` class + `Tina4::ServiceRunner.register_service(name, service)`.
- **Node.js** - new `Tina4Service` abstract class + `ServiceRunner.registerService(name, service)`.
- **Python** - new `tina4_python.service.Service` base + `ServiceRunner.register_service(name, service)` (this release closes the gap; Python had only the function-style runner before).

All four ship `run()` (abstract), `stop()`, and `should_stop()` / `shouldStop()` helpers backed by an internal flag. Function-style services using bare callables continue to work alongside the new class-based pattern.

PHP chapter rewrites (`docs/php/` and `book-2-php/`)

The 3.13.0 audit found that the PHP testing-chapter disaster was the tip of a larger pattern - multiple PHP chapters taught APIs that didn't exist. 3.13.1 rewrites all seven of them:

- **Chapter 15 - Logging** - primary surface now `Tina4\Log::info()/warning()/error()` instead of the legacy `Tina4\Debug::message()` shim (still works).
- **Chapter 18 - Testing** - `$response->statusCode` -> `$response->status` across 23 occurrences; CLI section updated (`tina4 test` runs the suite; `vendor/bin/phpunit` for targeted runs).
- **Chapter 19 - Scaffolding** - v2 `Tina4\Get::add()` / `Post::add()` / `Put::add()` / `Delete::add()` syntax replaced with `Tina4\Router::get/post/put/delete`; fictional `->description()` chain replaced with real `->swagger([...])`.

• **Chapter 22 - GraphQL** - chapter's decorator pattern (`GraphQL::resolve("Type", "field", $fn)`) now matches real source (built this release).

• **Chapter 25 - WSDL** - `@wsdl_operation` docblock replaced with `#[WSDLOperation([...])]` PHP attribute; methods now return associative arrays matching the response-shape spec; `Router::soap()` -> `Router::any()` + manual `(new Service($request))->handle()`.

• **Chapter 27 - ServiceRunner** - `new ServiceRunner()` + `->add()` instance API replaced with `ServiceRunner::registerService()` + `ServiceRunner::start()` static API. The `Tina4\Service` base class the chapter teaches now exists.

• **Chapter 34 - Deployment** - un-prefixed env vars (`SECRET`, `CORS_ORIGINS`, `SMTP_USER`, `JWT_SECRET`, `API_KEY`, `SWAGGER_TITLE`) replaced with `TINA4_`-prefixed forms. The v3.12 boot guard rejects the legacy names with `exit(2)`.

Test count

Net new across the family this release:

Framework	Before	After	New
Python	2,654	2,665	+11
PHP	2,749	2,774	+25
Ruby	2,827	2,839	+12
Node.js	3,384	3,406	+22
Total	**11,614**	**11,684**	**+70**

Upgrade

Drop-in patch - no breaking changes. Existing source-code patterns from 3.13.0 continue to work; the new methods are additive. Documentation rewrites in chapters 15 / 18 / 19 / 22 / 25 / 27 / 34 redirect copy-paste examples to the real APIs the framework actually ships.

v3.13.0 (2026-06-01)

The docs-vs-code parity release. A cross-framework audit of 381 markdown files surfaced 146 hallucinations, signature drifts, and stale references across Python, PHP, Ruby, Node, and tina4-js. 3.13.0 closes the chapter-18 disaster pattern - where documentation taught a class-based API that didn't exist - by shipping the missing pieces, renaming the misnamed pieces, and rewriting the aspirational chapters.

The headline fire: `Test` class with HTTP helpers

Every framework's chapter 18 has long shown integration tests like:

```
class UserApiTest(Test):           # tina4_python.test.Test
    def test_health(self):
        resp = self.get("/health")
        assert_equal(resp.status, 200)
```

Until 3.13.0, only `Test` (the bare assertion base) existed - calling `self.get(...)` crashed with `AttributeError`. The HTTP test client lived in a separate `TestClient` class that the docs never mentioned.

This release mixes `TestClient.get / post / put / patch / delete` into the `Test` base across every framework:

- **Python** - `tina4_python.test.Test` (extends `unittest.TestCase`, pytest auto-discovers)
- **PHP** - `Tina4\Test` (extends `PHPUnit\Framework\TestCase`)
- **Ruby** - `Tina4::Test` (zero-dep; built-in `run_all` runner)
- **Node.js** - `Tina4Test` (zero-dep; built-in `Tina4Test.runAll()` runner)

Plus positional assertions on every framework - `assertEqual(actual, expected, message)`, `assertNotEqual`, `assertTrue`, `assertFalse`, `assertNull`/`assertNullValue`, `assertNotNull`/`assertNotNullValue`, `assertRaises` - matching the documented `(actual, expected, message)` shape.

`Auth.valid_token` now returns the payload, not a bare bool - BREAKING

The most common silent-fail pattern caught by the audit. Every framework's docs claimed `valid_token` returned the decoded JWT payload; every framework's source returned `bool` and forced a second `get_payload` call.

Framework	Before	After	
Python	`Auth.valid_token(token) -> bool`	`Auth.valid_token(token) -> dict \	None`
PHP	`Auth::validToken(token) -> bool`	`Auth::validToken(token) -> array \	null`
Ruby	`Auth.valid_token(token) -> Boolean`	`Auth.valid_token(token) -> Hash \	nil`
Node	`validToken(token) -> boolean`	`validToken(token) -> Record<string, unknown> \	null`

Matches PyJWT / firebase-jwt-ruby / firebase/php-jwt / jsonwebtoken conventions. Truthy/falsy contract preserved - existing `if (validToken(t))` callers keep working because a non-null object is truthy and null is falsy.

Python-specific groups (mirrored to PHP/Ruby/Node in follow-up patches)

The Python framework is the reference per `feedback_python_master`. Six groups landed in tina4-python:

- **Group A - ergonomic additions**: `Database.get_connection()`, `db.fetch_all()`, `db.pool`, `DatabaseResult.columns`, `Job.error`, `Queue.produce(delay_until=datetime)`, module-level `migrate(db)`/`rollback(db)`/`status(db)`, module-level `i18n.t()`, dict-style

`session[key]`, `WebSocketConnection.connection_count`. All zero-risk additions - no signatures changed.

• **Group B - signature expansions**: `Api(bearer_token=, username=, password=, headers=, verify_ssl=)` kwargs, `Model.find(pk)` int overload (Active Record convention), `@description(summary, detail=, params=, query=)`, `@tags(str | list)`, `@example_response(status_code, data)`, `response.render(template, data, status_code)`, `response.cookie(name, value, options_dict)`, `response(data, headers={})`, `@get(path, description=, middleware=["ResponseCache:300"])` with string-form middleware parser.

• **Group C - mixins + decorators**: the Test HTTP mixin (covered above), `Frond.add_filter / add_global / add_test` callable as classmethod OR instance method via a `_ClassOrInstanceMethod` descriptor, `@GraphQL.resolve("Type", "field")` decorator with class-level registry - chapter 22's pattern now works as documented.

• **Group D - return-type changes (BREAKING)**: `Container.reset()` now clears singleton cache only (factories survive); new `Container.reset_all()` for the old wipe-everything behaviour. `queue.dead_letters()` returns `list[Job]` with `.error` populated, not `list[dict]`. `Model.where(..., with_count=True)` returns `(list, int)` tuple for pagination UIs.

• **Group E - renames (BREAKING)**: `ai.install_all()` -> `ai.install_context()`; new `ai.detect_ai()`, `ai.detect_ai_names()`, `ai.status_report()`. `queue.consume(id=)` -> `queue.consume(job_id=)`. `Api.send_request()` -> `Api.send()`. `I18n(locale=, path=)` preferred over `I18n(locale_dir=, default_locale=)` (legacy kept). `TINA4_TOKEN_EXPIRES_IN` preferred over `TINA4_TOKEN_LIMIT` for JWT expiry (both honoured; new wins; constructor arg overrides both).

• **Group F - top-level re-exports + scaffolder**: `from tina4_python import Api, WSDL, wsdl_operation, GraphQL, AutoCrud, Messenger, on, emit, once, off, tests` now resolve. `Model.select()` with no args defaults to `SELECT * FROM <table>` so the CRUD-list scaffolder template's emitted code actually runs.

PHP-specific: `Tina4\Debug` shim

Chapter 15 of the PHP logging docs taught `Tina4\Debug::message($msg, TINA4_LOG_INFO, [...])`. Neither the class nor the constants existed. Real logger is `Tina4\Log`.

This release ships a `Tina4\Debug` compatibility shim that forwards to `Tina4\Log`, plus defines the `TINA4_LOG_*` level constants - so the chapter's code samples run as-written. For new code, prefer `\Tina4\Log::info()` etc.

Documentation sweep

Aside from the source-side changes, the audit caught hundreds of stale references in docs site + book + AI skills + CLAUDE.md files. All fixed in this release:

• ~80 occurrences of `from tina4 import` -> `from tina4_python import` (the Python package is `tina4_python`, not `tina4`)

- `from tina4_python.router` -> `from tina4_python.core.router`
- `TINA4_SESSION_HANDLER` -> `TINA4_SESSION_BACKEND` (matches the env var the framework actually reads)
- `DATABASE_NAME=` -> `TINA4_DATABASE_URL=` (legacy un-prefixed names get rejected by the v3.12 boot guard)
- `@cached(True, max_age=N)` -> `@cached(max_age=N)` (bogus first arg)
- `Template.render()` -> `response.render()` (Template class doesn't exist; renamed to Frond)
- `Debug.error()` -> `Log.error()` in Python (Debug class doesn't exist)
- `Producer` / `Consumer` (removed in v3.2.0) -> `Queue.push / consume`
- `Email` -> `Messenger`, `event.fire / @listener` -> `emit / @on`, `gql` singleton -> `GraphQL()` + `@GraphQL.resolve`
- **Security fix**: `Auth.check_password(hash, password)` -> `(password, hash)` in skill ref - the bcrypt comparison was returning False every time due to reversed args (silent-failure auth)
- `request.files['content']` is **raw bytes** - drop `base64.b64decode()` from upload examples
- Deployment chapter env vars all `TINA4_`-prefixed (un-prefixed names brick boot under v3.12 guard)

Aspirational chapters rewritten

Two Python chapters were built on APIs that didn't exist:

- **Chapter 22 (GraphQL)**: rewritten around the new `@GraphQL.resolve("Type", "field")` decorator (the FastAPI/Strawberry pattern). The previous `gql.schema.add_query("name", {dict})` form still works but is no longer the primary documented path.
- **Chapter 25 (WSDL)**: rewritten around the real subclass pattern (`class Calculator(WSDL): @wsdl_operation({"Result": int}) def Add(self, ...): ...; Calculator(request).handle()`). The previous `WSDL(service_name=, namespace=, endpoint=)` constructor + `handle_wsdl` / `handle_request` API was entirely fictional.

tina4-js doc drift caught

The cross-framework audit's synthesis pass dropped tina4-js findings; the raw agent transcripts had 23 real findings:

- Every import in CLAUDE.md and `docs/js/09-graphql.md` used `"tina4-js"` (with hyphen). The npm package is named `tina4js`. Fixed.
- `pwa({...})` was treated as callable; real API is `pwa.register({...})`. `PWAConfig.icon` is a single string, not an `icons: [...]` array.
- `static props = { label: { type: String, default: "..." } }` - the `{ type, default }` wrapper is fictional. Real shape is `static props = { label: String }`.

- `router.navigate('/users/42')` - `navigate` is a top-level export, not a method on `router`.
- Chapter 14's `<slot>` inside a `static shadow = false` component - slots are a Shadow DOM feature. Chapter 14 contradicted chapter 4. Switched to `shadow = true`.

Test counts

Net new across the family:

Framework	Before	After	New
Python	2,537	2,654	+117
PHP	2,742	2,749	+20
Ruby	2,800	2,816	+16 (1 pre-existing unrelated rack_app failure)
Node.js	3,366	3,384	+18
Total	**11,445**	**11,603**	**+171**

Upgrade

`Auth.validToken` is the breakage to know about - your `if Auth::validToken($t)` style code keeps working unchanged because non-null arrays are truthy and null is falsy. If you do `=== true` / `=== false` strict comparisons, switch to `!== null` / `=== null`.

Python: `ai.install_all()` -> `ai.install_context()`, `queue.consume(id=)` -> `consume(job_id=)`, `Api.send_request()` -> `Api.send()`, `Container.reset()` semantic change (use `reset_all()` for old behaviour).

Everything else is additive - new properties, new kwargs, new convenience methods that match what the docs have promised for years.

v3.12.14 (2026-06-01)

Two independent fixes ship together as 3.12.14. **Python** - the `tina4_python.test` class-based xUnit testing surface that the chapter 18 documentation has always promised but never actually existed. Reports came in of developers copy-pasting `from tina4_python.test import Test, assert_equal, assert_true` straight out of the book and getting `ModuleNotFoundError`. The fix was to build the module to match the documentation, not the other way around. **PHP** - `:named` placeholder translation for the four non-PDO adapters where `ORM::save()` was silently failing.

Python - `tina4_python.test` xUnit testing surface

The testing chapter taught a `Test` base class with positional assertions:

```
from tina4_python.test import Test, assert_equal, assert_true

class BasicTest(Test):
```

```
    def test_addition(self):
        assert_equal(2 + 2, 4, "Basic addition should work")

    def test_string_contains(self):
        greeting = "Hello, World!"
        assert_true("World" in greeting, "Greeting should contain 'World'")
```

The module did not exist. Every developer who followed the chapter hit an immediate import error. The other surface, `tina4_python.Testing` with the inline `@tests` decorator, has always existed - but the two are for different purposes and the docs only documented one of them.

The fix ships the missing module - `tina4_python/test/__init__.py` - with the `Test` base class (inherits `unittest.TestCase`, so pytest discovers any subclass regardless of class-name convention) and 13 positional assertions. The signatures are uniform: `(actual, expected, message)`. The 2-arg legacy `(value, message)` form keeps working - a type-based dispatch detects which shape the caller used. `assert_raises` accepts three forms: docs form (`callable, exception, message`), context-manager form (`with assert_raises(X):`), and unittest order (`exception, callable`). Lifecycle hooks come in both flavours - snake_case `set_up`/`tear_down` (the Tina4 idiom) and camelCase `setUp`/`tearDown` (for users coming from unittest) - without double-calling when a subclass uses either one.

```
# 13 assertions, all uniform (actual, expected, message)
assert_equal(actual, expected, message="")
assert_not_equal(actual, expected, message="")
assert_true(actual, expected=True, message="")
assert_false(actual, expected=False, message="")
assert_none(actual, expected=None, message="")
assert_not_none(actual, expected="not None", message="")
assert_in(item, container, message="")
assert_not_in(item, container, message="")
assert_is_instance(value, expected_type, message="")
assert_greater(actual, expected, message="")
assert_less(actual, expected, message="")
assert_almost_equal(actual, expected, places=7, message="")
assert_raises(callable, exception_class, message="")
```

51 new tests in `tests/test_test_module.py` pin the contract: BasicTest from the chapter runs verbatim, every assertion fails when it should and passes when it should, both 2-arg and 3-arg shapes work, snake_case and camelCase lifecycle hooks fire once each (never both). Full Python suite: 2,537 passing.

`tina4 test` continues to run pytest (`subprocess.run([sys.executable, "-m", "pytest", "tests/"] + args)`).

Cross-framework parity check (testing)

PHP / Ruby / Node testing chapters already teach native conventions correctly - PHPUnit, RSpec, and `node:test` respectively. No fake API to fix. The Python-specific gap was that Tina4 had two testing surfaces (`tina4_python.Testing` for inline `@tests` decorator, `tina4_python.test` for class-based suites) and only one of the two existed. The other three frameworks defer to a single native runner each, so the same trap doesn't apply.

PHP - :named placeholder translation across non-PDO adapters

The ORM's `save()` emits `:named` placeholders because PDO would accept them. Four of the five PHP database adapters do not use PDO. `MySQLAdapter` (mysqli), `MSSQLAdapter` (sqlsrv), `FirebirdAdapter` (ibase/fbird), and `PostgresAdapter` (pgsql) all bind positionally. Every INSERT/UPDATE through `save()` against those four engines failed silently. Reads worked because read paths typically use `?` or no params.

A single helper, `SqlTranslation::namedToPositional($sql, $params)`, translates `:name` to `?` and reorders `$params` to match the SQL order. Wired into the four affected adapters at the top of their prepare/execute paths. The helper skips string literals and SQL comments, so a literal `:colon` inside a value stays as a value. Duplicate names bind once per occurrence, so `WHERE id = :id AND parent_id = :id` works as expected.

`SQLite3Adapter` is untouched. ext-sqlite3 natively accepts `:name` via `SQLite3Stmt::bindValue`. The other four did not, and now do.

15 unit tests pin the helper in `tests/SqlTranslationNamedToPositionalTest.php`: order preservation, duplicate names, quoted strings, line and block comments, unknown placeholders, null values, and the `0`-as-value case. Full PHP suite: 2,290 passing.

Cross-framework parity check (`:named` placeholders)

Python (`mysql-connector-python` uses `%s`), Ruby (`mysql2` uses `?`), and Node (`mysql2` uses `?`) build their INSERT/UPDATE SQL with positional placeholders from the ORM down. No `:named` ever emitted. Audited the MySQL adapter and `save()` path in each before shipping; confirmed clean. PHP-only fix.

Upgrade

Drop in for both Python and PHP. No `.env` changes, no API changes.

Python users who followed chapter 18 and hit `ModuleNotFoundError` - bump to `3.12.14`, the `from tina4_python.test import Test, assert_equal, ...` import now resolves. Existing tests written against `tina4_python.Testing` (the inline `@tests` decorator) continue to work - that surface was not touched.

PHP users - `:named` and `?` both work, and the framework picks the right form for whichever driver is underneath. Existing ORM `save()` calls start succeeding on MariaDB/MySQL, PostgreSQL, MSSQL, and Firebird.

Ruby and Node users - no framework change shipped in 3.12.14. Stay on `3.12.13` or bump to `3.12.14` for version alignment. Both are functionally identical.

v3.12.13 (2026-05-29)

Consolidated parity release. PHP ran ahead through two independent patch releases (3.12.11-3.12.12) while Python / Ruby / Node stayed at 3.12.10. This release realigns all four frameworks on **3.12.13** and ships the cross-framework dev-admin parity sweep - five tiers of work that bring PHP, Ruby and Node up to Python's AI-assisted development surface.

Cross-framework dev-admin parity sweep (Tier 1-5)

The Python framework had pulled ahead on a series of dev-admin features driven by real frustration with the AI coder loop ("Applying a small patch went and messed up my whole file", "Says it is creating files but then doesn't", repeated import-error spirals). This release ports the full set to PHP, Ruby, and Node - same intent, language-idiomatic implementations.

Tier 1 - MCP defensive write layer. `file_write` and `file_patch` now refuse prose-as-filenames (the LLM occasionally emits `## FILE: I'll implement Step 1 by creating the database migration` and the parser used to write a zero-byte file with that sentence as its filename), normalise bare top-level `routes/` / `orm/` / `templates/` / `seeds/` / `controllers/` / `middleware/` paths to their canonical `src/<dir>/` form (auto-discovery only scans `src/`, so a file at `templates/foo.twig` was dead weight), back up existing files to `.tina4/backups/<flat-path>.<ISO-ts>.bak` before overwrite, and refuse suspicious truncations (>200B file -> <30% size = almost always a truncated LLM response). Every attempt logs to `.tina4/agent.log` with a structured category (`write.ok` / `write.refused` / `write.path_normalized` / `write.import_failed`) - the supervisor reads that file on every turn so it sees what broke last time and can self-correct without asking the developer "what's the error?".

Tier 2 - Post-write syntax verification. PHP shells out to `php -l`, Ruby to `ruby -c`, Node to `node --check` (and single-file `tsc --noEmit --allowJs --skipLibCheck` for `.ts`). On parse error the tool result gets an `import_error` field AND a `write.import_failed` log entry surfaces in the next supervisor turn's failure context. Catches hallucinated framework APIs (`CharField` doesn't exist in `tina4_python.orm.fields` - should be `StrField`; `auto_now_add` keyword on `Field.__init__()`) at write time instead of letting them propagate to a runtime 500 the user only discovers by hitting the URL.

Tier 3 - `/__dev/api/threads` + `/__dev/api/chat` proxy. The SPA now talks to the Rust supervisor agent the same way regardless of framework. `_supervisor_base_url()` matches Python's 4-step ladder (`TINA4_SUPERVISOR_URL` -> `TINA4_AGENT_PORT` -> `PORT+2000` -> `9145`). `active_file` rides through `/chat` POST verbatim so deictic phrases ("fix this", "explain this") bind to the editor's open file without the supervisor asking. The Node port forwards SSE chunks as they arrive; PHP and Ruby buffer (functional - EventSource parses fine - but feels less snappy until a future round of Rack/PHP-FPM streaming work).

Tier 4 - Customer feedback widget. A floating bubble for end-users of a shipped Tina4 app, gated by `TINA4_ENABLE_FEEDBACK=true` AND a non-empty `TINA4_FEEDBACK_WHITELIST`. The framework's response middleware injects `<script src="/__feedback/widget.js" data-tina4-feedback></script>` immediately before the LAST `</body>` tag on text/html responses, ONLY for whitelisted users, NEVER on `/__dev` or `/__feedback` paths (no double-bubble UX on the developer dashboard). One conversational turn at a time POSTs to `/__feedback/api/turn` -> server-side identity stamp from the verified JWT (clients cannot fake `sender`) -> forward to the Rust agent's intake-only agent (zero tools, JSON-only output). Finalised tickets land in the dev admin sidebar with `kind:"feedback"`. Rate-limited at 5 turns/hour per user.

Tier 5 - Stale-source overlay badge + `list_plans()` merge. The error overlay now stamps `captured_at` on render and tags each stack frame whose source file has been modified since:

"FILE MODIFIED @ HH:MM:SS UTC - source may not match what failed". Stops the user from chasing ghosts when the AI coder rewrote the file between the error and the page reload. `list_plans()` reads from BOTH `plan/` (user-curated canonical) AND `.tina4/plans/` (AI-planner output), dedupes by filename with `plan/` winning on collision, sorts newest-first, and returns a `path` field so the SPA can open the right file regardless of source dir.

Test counts. Per-framework deltas across the sweep:

Framework	Before -> After (full suite)
Python	2453 -> 2453 (canonical - no new tests, just released)
PHP	2235 -> 2714 (+479)
Ruby	2747 -> 2800 (+53)
Node	3263 -> 3368 (+105)

PHP's larger delta reflects new tests + the 3.12.11 + 3.12.12 lineage rolling forward.

Why all four frameworks at once. Per the cross-framework parity rule: a feature that exists in only one framework is technical debt. The Python-only Tier 1-5 surface had been accumulating for two weeks while the UX was settling. With it settled, this release closes the gap in one coordinated sweep.

Folded-in from PHP 3.12.11 - file upload regression (`tina4-book#139`)

`WebSocket::parseHttpHeaders()` previously split the entire raw HTTP request on `\r\n` and iterated every line for a `:` to fill the headers map. Multipart body parts have their own `Content-Type`, `Content-Disposition`, and `Content-Transfer-Encoding` headers - those lines matched the parser and overwrote the real request `Content-Type: multipart/form-data; boundary=...` with whatever the last body part's content type was (typically `application/pdf`, `image/png`). Downstream `str_contains($contentType, 'multipart/form-data')` then failed, the multipart branch was skipped, `$parsedFiles` was never set, and `$request->files` came out empty. Every file upload through the stream-socket server was silently lost - the body landed in `$request->body` as a raw multipart string with no way to parse it.

Fix. Stop the parser at the first `\r\n\r\n` (RFC 9112 Section 2.2 boundary between headers and body) before splitting into lines. One logical change in `Tina4/WebSocket.php`. 9 regression tests in `tests/BookIssue139Test.php` cover single-part, multi-part, and mixed-header cases.

Cross-framework parity check. Python (`http.server`), Ruby (`webrick`/`puma`), and Node (built-in `http` module) all delegate header parsing to upstream stdlib HTTP parsers that already split headers from body correctly. PHP was the only framework with a hand-rolled HTTP parser in this code path. No port needed.

Folded-in from PHP 3.12.12 + Python 3.12.13 - v2 `tina4_migration` auto-upgrade (#115)

Projects upgrading from tina4 ^2.x to ^3.x carried a v2-shaped `tina4_migration` table that v3's `ensureMigrationsTable()` left untouched (the `CREATE TABLE IF NOT EXISTS` short-circuited). The v3 reader then selected columns that didn't exist, fell into the "never seen this migration, run it" branch, and re-applied already-applied migrations - typically failing on duplicate-column / table-already-exists errors when the SQL was non-idempotent. The AirOffices ~190-migration codebase tripped on this in March 2026 and needed a manual SQL backfill at the time.

Framework	v2 schema	v3 schema
PHP	`migration_id VARCHAR(14)`, `description`, `content BLOB`, `passed`	`id INT PK`, `migration`, `batch`, `applied_at`
Python	`description` as identifier, `content`, `passed`	`migration_id`, `migration_name`, `executed_at`

Fix. `ensureMigrationsTable()` (PHP) and `_ensure_tracking_table()` (Python) now detect a v2-shaped table (v2 columns present, v3 columns absent) and call an in-place upgrade that ALTERs in the v3 columns alongside the v2 ones, then backfills v3 fields from the v2 data. v2 columns are kept in place so a manual rollback path stays open - they're simply ignored by v3 readers. The match is by file stem: a v2 row's identifier is matched against `migrations/` files by basename (Python uses `000001_create_users.sql` -> stem `000001_create_users` -> v2 description `create_users`).

Cross-framework parity check. Ruby and Node never shipped a v2 migration table with the trapping shape - their v2 lineages used a different column layout that v3's tracker tolerated. Nothing to port.

Folded-in from PHP 3.12.11 - request URL parity

`$request->url` now returns the full absolute URL (`https://host:port/path?query`) instead of just the path. `$request->queryString` (raw query bytes) added for parity with `request.query_string` on the other frameworks. Drop-in - old code that read `$request->path` (untouched) keeps working.

Upgrade

Drop in. No `.env` changes, no API changes.

For projects upgrading from v2.x: the v2 `tina4_migration` auto-upgrade runs once on first boot against v3 - back up your migrations table beforehand if you're paranoid. The upgrade is non-destructive (v2 columns are kept alongside the new v3 ones).

For projects using the dev admin AI coder loop: the new MCP defensive layer will silently rewrite `## FILE: routes/foo.py` to `## FILE: src/routes/foo.py` and log a

`write.path_normalized` entry. If you were relying on the old behaviour (writes landing wherever the LLM emitted them), this will move some files. Run `tail -n 50 .tina4/agent.log | grep path_normalized` after upgrading to see what got rewritten.

For shipping apps that want the customer feedback widget: set `TINA4_ENABLE_FEEDBACK=true` AND `TINA4_FEEDBACK_WHITELIST=alice@example.com,bob@example.com` in `.env`. The widget appears only for those users on non-`/__dev` pages.

v3.12.10 (2026-05-14)

Version-alignment release. PHP ran ahead through three independent patch releases (3.12.7-3.12.9) while Python / Ruby / Node stayed at 3.12.6. This release realigns all four frameworks on **3.12.10** and ships the ORM `save()` fix.

PHP - `ORM->save()` no longer swallows write failures (#114)

`ORM->save()` called `update()`/`insert()` but ignored their `bool` return - it only caught exceptions. The PHP adapter's `exec()` returns `false` on a bad statement instead of throwing, so a failed `UPDATE` (commonly: one referencing a public model property with no matching DB column, since `getDbData()` includes every public property) slipped through. The empty transaction got committed and `save()` returned `$this` - the documented success signal. Callers relying on the `save(): static|false` contract believed the row persisted when nothing changed. **Silent data loss** - no exception, no log.

Fix. `save()` now captures the `bool` return of `update()`/`insert()`, rolls back, and returns `false` on a falsy result.

```
$ok = $this->_exists || ... ? $this->update() : $this->insert();
if ($ok === false) { $this->_db->rollback(); return false; }
$this->_db->commit();
```

Cross-framework parity check. Python, Ruby and Node don't have this exact failure mode - they build the write payload from declared fields only (not all public properties), and their DB adapters raise on bad SQL, which the existing `try/except` already catches. PHP was the outlier on both counts. 3 regression tests in `tests/Issue114Test.php`; PHP suite 2235 -> 2238 passing.

Also in the PHP 3.12.7-3.12.9 patch line

These shipped to PHP between 3.12.6 and this release; folded into the consolidated 3.12.10 line:

- **3.12.7** - `Request` now normalises caller-provided header keys to lowercase. Some upstream entry points (Apache+PHP-FPM custom mappings, certain proxies, hand-written test fixtures) hand headers in with original case. The constructor only looks them up by lowercase key, so without normalisation `multipart/form-data` content-type detection silently missed and the body fell through as raw bytes - a follow-up to the #135 fix.
- **3.12.8 / 3.12.9** - Router gained RFC 9110 HTTP method conformance: proper `HEAD` and `OPTIONS` handling, `405 Method Not Allowed` with an `Allow` header listing the methods a route does support.

Python / Ruby / Node

Version-only bump 3.12.6 -> 3.12.10 to realign with PHP. No behavioural changes in these three since 3.12.6.

Upgrade

Drop in. No `.env` changes, no API changes. PHP users on 3.12.9 get the `save()` fix; everyone else gets a version-number realignment.

v3.12.6 (2026-05-06)

Python-only fix release. PHP / Ruby / Node ship the same version stamp for parity but carry no behavioural changes.

Python - psycopg2 `%` substitution no longer trips PL/pgSQL function bodies (#40)

A migration containing a PL/pgSQL function with literal `%` characters in a `RAISE EXCEPTION` (or `format()`) call used to fail with the misleading:

> *RuntimeError: Migration failed: list index out of range*

The error message gave no hint that the `%` chars were the problem. The user-facing failure looked like a tina4 internal bug - actually psycopg2's argument-substitution system tripping on the literal percent signs.

Root cause. `PostgreSQLAdapter.execute(sql, params)` always called `cursor.execute(sql, params or [])`. psycopg2 interprets `%` as parameter placeholders WHENEVER the `params` arg is supplied - even an empty list `[]`. So a function body containing `RAISE EXCEPTION 'thing % conflicts with %', a, b` (perfectly valid PL/pgSQL) blew up because psycopg2 thought `%` was a placeholder and there were no values to substitute.

Fix. New `PostgreSQLAdapter._safe_execute(cursor, sql, params)` helper routes empty/None params through `cursor.execute(sql)` (no second arg), which makes psycopg2 skip the substitution pass entirely. Literal `%` chars flow through untouched. Applied at every `cursor.execute(...)` call site in the adapter (5 spots across `execute`, `fetch`, `fetch_one`).

Tests. 5 new unit tests in `tests/test_postgres_percent_substitution.py` pin the helper's branching. 3 live-Postgres regression tests in `tests/test_postgres_plpgsql_percent.py` exercise a real CREATE FUNCTION + trigger flow with literal `%` in the body - skipped automatically when no Postgres is reachable. Full suite: 2453 passing (was 2448).

Cross-framework parity check. PHP (`pg_query` vs `pg_query_params`) and Ruby (`exec` vs `exec_params`) already branch on params presence so they don't have this bug. Node uses `$1` placeholders not `%`, so the same class of bug doesn't apply.

Long-standing tina4-js #37 confirmed fixed

`frond.form.submit` not following 3xx redirects - fixed in frond v2.1.2 back on April 11, 2026 (`xhr.responseURL` comparison + `window.location.href` navigation). All four framework `public/js/frond.min.js` copies carry the fix. The original issue stayed open because the reporter never confirmed against the patched build.

Upgrade

Drop in. No `.env` changes, no API changes.

v3.12.5 (2026-05-06)

PHP-only bug fix release. Python / Ruby / Node ship the same version stamp for parity but carry no behavioural changes.

PHP - multipart bodies with file uploads now parse correctly (#135)

Two stacked bugs in `Tina4\Request::__construct` made `$request->body` come through as the raw multipart bytes (~11 KB blobs starting with `------WebKitFormBoundary...`) whenever the request included a file upload:

- The constructor called `$this->parseBody()` BEFORE initialising `$this->files`. Inside parseBody's multipart branch, the line `$this->files = array_merge($this->files, $parsed['files'])` read an uninitialised typed property - fatal `Error`.
- After fixing the init order, that same line tried to mutate the `readonly` `$files` property - another fatal `Error`.

Both errors got swallowed by the upstream error handler and the route handler received the raw multipart payload instead of the parsed associative array. Routes that worked fine for ordinary form posts broke the moment a file field came along.

Fix. Move `$this->files` initialisation AFTER `parseBody()` runs. parseBody stashes extracted multipart files on a new private mutable `$multipartFiles`; the constructor merges them into the readonly `$files` in a single assignment that respects the readonly contract.

4 new regression tests in `tests/Issue135Test.php` pin the constructor's contract. Full PHP suite: 2235 passing (was 2231).

Upgrade

Drop in. No `.env` changes, no API changes, no other framework changes.

v3.12.4 (2026-05-06)

Documentation-truth release. The `audit-truth.py` CI gate (introduced post-3.12.3) flagged 39 env vars referenced in docs that no framework actually read. This release closes that gap: 25 of them now exist in code, the other 14 are deleted from docs (11 hallucinations + 6 clustering vars deferred to tina4#2). Both audit gates (CLI drift + env-var drift) are now strict in CI.

25 new env vars across all 4 frameworks

Server: `TINA4_HOST`, `TINA4_SUPPRESS`, `TINA4_ENV_FILE`. Health: `TINA4_HEALTH_PATH` (default `/__health`, with `/health` kept as a legacy alias), `TINA4_TRAILING_SLASH_REDIRECT`. Sessions: `TINA4_SESSION_HTTPONLY`, `TINA4_SESSION_NAME`, `TINA4_SESSION_SECURE`. Templates: `TINA4_TEMPLATE_CACHE_TTL` (`0` = permanent). GraphQL: `TINA4_GRAPHQL_AUTO_SCHEMA`, `TINA4_GRAPHQL_ENDPOINT`. Mail: `TINA4_MAIL_IMAP_ENCRYPTION` (`tls`/`starttls`/`none`). MCP: `TINA4_MCP`, `TINA4_MCP_PORT`. Swagger: `TINA4_SWAGGER_ENABLED`, `TINA4_SWAGGER_CONTACT_EMAIL`, `TINA4_SWAGGER_LICENSE`. Database: `TINA4_DB_POOL` (env override on the existing `Database(url, pool=N)` constructor argument).

Logging - env-driven file output + rotation

Six new vars give you full control over logging without touching code:

Var	Default	What it does
`TINA4_LOG_FILE`	*(empty - stdout only)*	Path to a log file. Empty leaves you on stdout.
`TINA4_LOG_DIR`	`logs`	Directory for log files (joined with `_LOG_FILE` if relative).
`TINA4_LOG_FORMAT`	`text`	`text` or `json`. JSON mode emits one structured record per line.
`TINA4_LOG_OUTPUT`	`stdout`	`stdout`, `file`, or `both`. Strict - `stdout` means stdout only.
`TINA4_LOG_CRITICAL`	`false`	Enables a `Log.critical()` level above `error`. Off = no-op.
`TINA4_LOG_ROTATE_SIZE`	`10485760` (10 MB)	Rotate when the file exceeds this many bytes. `0` disables rotation.
`TINA4_LOG_ROTATE_KEEP`	`5`	Number of rotated files to retain (`app.log.1` ... `app.log.N`). Older ones are deleted.

Implementation uses each language's stdlib - Python's `logging.handlers.RotatingFileHandler`, Ruby's `Logger.new(path, shift_age, shift_size)`, and a roll-your-own atomic-rename pattern in PHP and Node. Zero new dependencies in any framework.

Documentation-truth CI gate now strict on both axes

The `audit-truth.py` script now blocks merges to `main` of `tina4-documentation` whenever a doc references a `tina4 <command>` or `TINA4_*` env var that doesn't exist in source. Previously CLI drift was strict; env drift was warn-only. Today both are strict.

Tests added

- Python: +53 tests in `tests/test_env_vars.py` (2395 -> 2448)
- PHP: +59 tests in `tests/EnvVarTest.php` (2172 -> 2231)
- Ruby: +51 examples in `spec/env_vars_spec.rb` (2696 -> 2747)
- Node: +59 tests in `test/envVars.test.ts` (3204 -> 3263)

Cross-framework total: 10,689 tests passing, +222 from 3.12.3.

Upgrade path

Drop in. No breaking changes - every new env var is opt-in with a sensible default. If you were setting any of the 17 deleted vars in your `.env`, the boot guard will warn (then ignore) - clean them out at your leisure.

v3.12.3 (2026-05-05)

Cross-framework parity sweep. Two minor breaking changes in the Ruby and PHP public API that bring all four frameworks onto the same shape.

Breaking changes (Ruby + PHP only)

Ruby Container - predicate now uses `?` suffix.

```
# before (3.12.2 and earlier)
Tina4::Container.has(:mailer)        # outdated

# after (3.12.3)
Tina4::Container.has?(:mailer)       # idiomatic Ruby predicate
```

This brings Ruby in line with Python (`has()`), PHP (`has()`), and Node (`has()`) while still respecting Ruby's `?`-suffix idiom for predicates returning bool. The pre-existing `resolve` -> `get` rename happened earlier; only the predicate was lagging.

ResponseCache public surface - middleware-only across all four frameworks.

The cache has always been middleware. Two of the four frameworks (PHP, Ruby) historically exposed lookup/store as public methods, which let users couple to internals. The public API is now consistent across all four: use the middleware on a route, and read stats with module-level helpers.

```
# Ruby - module-level helpers (parity with Python)
Tina4.cache_stats   # -> { hits:, misses:, size:, backend:, keys: }
Tina4.clear_cache   # flush all entries

# PHP - static methods on the class
\Tina4\Middleware\ResponseCache::cacheStats();
\Tina4\Middleware\ResponseCache::clearCache();
```

Internal methods that used to be public (`get`, `lookup`, `store`, `cache_response`) are now private. Tests that needed them retain access via `_internal*` test seams marked `@internal`.

Doc parity - CLAUDE.md and book chapter 33

- **CLAUDE.md**: every framework's "Key Method Stubs" section now covers the same surface area Python documents - Queue, QueryBuilder, Frond, Api, Background Tasks, ResponseCache, etc. PHP added 4 sections; Ruby added 5; Node added 13.
- **Book chapter 33**: env var tables are now grounded in source. Each framework's chapter 33 lists every `TINA4_*` var its source actually reads. Found and fixed several gaps - Ruby was missing `TINA4_CACHE_*`, `TINA4_QUEUE_*`, `TINA4_KAFKA_*`, `TINA4_RABBITMQ_*`, `TINA4_MONGO_*`, `TINA4_WS_BACKPLANE`, and the entire `TINA4_SESSION_VALKEY_*` block.

Other fixes

- **Ruby** `lib/tina4/ai.rb` - subprocess output is now force-encoded to UTF-8 before `String#strip`, fixing `Encoding::CompatibilityError` that crashed 4 ai specs on systems with non-ASCII pip output.
- **Node** `test/serverParity.test.ts` - sets `TINA4_OVERRIDE_CLIENT=true` so `start()` actually runs, plus emits the `N passed, M failed` summary line the runner expects. The test was effectively a no-op before; now it's recorded properly.

Genuine gaps surfaced by the parity audit (follow-up, not blocking 3.12.3)

The chapter 33 audit flagged env vars Python documents that no other framework actually reads - Ruby/PHP/Node lack `TINA4_OPEN_BROWSER`, `TINA4_DEV_POLL_INTERVAL`, `TINA4_PUBLIC_DIR`, `TINA4_TOKEN_EXPIRES_IN` alias, plus a few framework-specific gaps (Ruby has no Mongo session backend; Node `TINA4_CSRF` defaults to `false` vs Python's `true`). Tracked for a future patch.

Upgrade path

Symptom	Fix
Ruby: `NoMethodError: undefined method 'has' for Tina4::Container`	Replace `has(:key)` with `has?(:key)`
PHP: `BadMethodCallException` calling `$cache->lookup(...)`	Use the middleware: `[ResponseCache::class, 'beforeCache']` / `[..., 'afterCache']`. Or call `_internalLookup` if you really need direct access (test code only - `@internal`).
Ruby: `NoMethodError: undefined method 'get' for ResponseCache instance`	Use `Tina4.cache_stats` / `Tina4.clear_cache` for stats. Lookup goes through the middleware.

No `.env` changes from 3.12.2.

v3.12.2 (2026-05-05)

Quality-of-life patch. Two related portability fixes - no breaking changes from 3.12.1.

Firebird URL auto-detect

Firebird is the awkward one in the stack. Every other engine has a server-side database name (`postgres://host:port/dbname`), but Firebird wants either an absolute file path on the server, a Windows drive-letter path, or an alias. The classic URI form needs a double slash to keep the leading `/` of an absolute path through the URL parser - unintuitive to anyone used to the way postgres / mysql / mssql encode the database name.

The framework now accepts five equivalent forms and normalises all of them transparently:

URL path you write	Resolved Firebird identifier
`//abs/path/db.fdb` (classic double-slash)	`/abs/path/db.fdb`
`/abs/path/db.fdb` (single-slash, intuitive)	`/abs/path/db.fdb`
`/C:/Data/db.fdb` (Windows drive letter)	`C:/Data/db.fdb`
`/C%3A/Data/db.fdb` (URL-encoded colon)	`C:/Data/db.fdb`
`/employee` (Firebird alias)	`employee`

For ops setups that keep server URL and DB location in separate config layers - or for Windows backslash paths that fight URL encoding - set `TINA4_DATABASE_FIREBIRD_PATH`. The env override wins over whatever path is in the URL.

```
TINA4_DATABASE_FIREBIRD_PATH=C:\firebird\data\app.fdb
TINA4_DATABASE_URL=firebird://SYSDBA:masterkey@localhost:3050/ignored
```

Shipped to all 4 frameworks. 11 regression tests per framework (8 unit + 3 live).

Bug fix specific to PHP - `mysqli` localhost+port quirk

PHP's `mysqli` has a long-standing quirk where `host == "localhost"` triggers a Unix socket lookup and IGNORES the port argument entirely. Connecting to `mysql://...:53306` against a Docker container fails with "No such file or directory" - `mysqli` is hunting for `/tmp/mysql.sock` instead of opening a TCP connection. `MySQLAdapter::rewriteHostForTcp()` now rewrites `localhost` to `127.0.0.1` when a non-zero port is specified, forcing the TCP code path. Bare `mysql:///db` (no port) is preserved so existing socket-based setups keep working.

Other fixes

- **chore(python):** `pyproject.toml` had drifted to `3.10.41` while `__init__.py` read `3.12.1`. Synced both to 3.12.2 so `uv build` and runtime introspection now agree.
- **chore(claude.md, all 4):** stale framework version banners in `CLAUDE.md` headers updated.

No `.env` changes from 3.12.1, no migration needed. Existing 3.12.1 installs upgrade by changing one version number.

---

v3.12.1 (2026-05-04)

CI-only patch - no framework code changes from 3.12.0.

- **fix(ci, all 4):** every `publish.yml` workflow now declares `permissions: contents: write` on the publish job. Without this, `softprops/action-gh-release` 403'd against the default `GITHUB_TOKEN` on repos whose default Workflow permissions setting was read-only (Ruby and Node hit this every release; PHP and Python worked by luck of repo settings). The explicit declaration makes the workflow self-sufficient.
- **chore(ci):** bumped `softprops/action-gh-release` from `@v1` (unmaintained) to `@v2`.

No `.env` changes, no API changes, no migration needed. Existing 3.12.0 installs can upgrade without touching anything else.

The version-bump itself is the test: a successful 3.12.1 release proves the workflow fix works on Ruby and Node where 3.12.0 needed manual `gh release create`.

v3.12.0 (2026-05-04)

> ***Warning: Breaking change - read before upgrading.*** *Every framework env var now uses the `TINA4_` prefix. Existing `.env` files set with `DATABASE_URL`, `SECRET`, `SMTP_HOST`, `HOST_NAME`, etc. will cause the framework to refuse to boot. Run `tina4 env --migrate` to rewrite, or follow the rename table below.*

Why this release

Tina4's env vars had grown inconsistent. Some had the `TINA4_` prefix (`TINA4_DEBUG`, `TINA4_LOCALE`, `TINA4_CACHE_BACKEND`), others didn't (`DATABASE_URL`, `SECRET`, `SMTP_HOST`). Newcomers had to guess which convention applied to which feature. Existing tools and PaaS dashboards collided with un-prefixed names like `SECRET` and `API_KEY` that other libraries also read. Documentation drifted - 91 env-var names appeared in the docs that didn't exist in any framework, and 22 framework-specific env vars in the code didn't match the names users were told to set.

This release closes all three gaps with a single hard rename. No deprecation period, no fallback chain. The framework refuses to boot if it detects a legacy name in the environment, prints a list of every var to rename, and tells you which command to run.

What changed

- **22 env vars renamed** to `TINA4_*` form. See the migration table below.
- `tina4 env --migrate` **CLI** added to all four frameworks. Reads your `.env`, rewrites it in place, leaves a `.env.bak` backup, prints a diff. Idempotent.
- **Boot-time guard** scans `os.environ` (or the language equivalent) for the 22 legacy names. If any are present, prints the rename map and exits with code 2. Bypass with `TINA4_ALLOW_LEGACY_ENV=true` for migration scripts that need both names set during transition.
- **All 4 framework books rewritten.** Chapter 33 (Environment Variables) is now a clean canonical list - every var prefixed, descriptions current, legacy names removed.

• **Doc-vs-code drift closed.** Of the 91 stale env vars previously documented, 61 were renames (corrected), 32 were never implemented (removed). The `audit-links.py` CI gate stays at 0 broken links / 0 broken anchors.

• **Frond bundle** rebuilt at v2.1.3 - `frond.min.js` footer now shows the version explicitly so users can verify what they have.

Bug fixes shipped alongside the rename

• **#38 PostgreSQL UUID-PK transaction abort** - the post-INSERT `lastval()` probe is now wrapped in a SAVEPOINT, so UUID-PK INSERTs no longer poison the outer transaction with `InFailedSqlTransaction`. Live regression test against PostgreSQL 16. (Affects all 4 frameworks where the PG adapter does this probe.)

• **#39 Landing page + template auto-routing**:

• Auto-routing now scans `src/templates/pages/` only. Partials, layouts, base.twig, errors/, components/, and `_*` files never auto-serve from a URL.

• `TINA4_TEMPLATE_ROUTING=off` kills the feature entirely.

• `src/public/index.html` auto-serves at `/` (and `/foo/` serves `src/public/foo/index.html`) - SPA hosting Just Works.

• The framework landing page only renders when `TINA4_DEBUG=true`. Production never shows it; framework version, dev-admin link, and gallery don't leak to real users.

• The malformed `HTTP/1.1 404 OK` status line is fixed - every status code now uses its canonical RFC 7231/9110 reason phrase.

• **#37 frond.form.submit redirect handling** - verified shipped at v2.1.x; `xhr.responseURL` change triggers `window.location` navigation correctly.

• **#36 Session file handler** - re-verified safeguards (lazy save, WebSocket skip, probabilistic GC, new-and-empty skip) all still in place.

Migration - every renamed var

Legacy name	New name
`DATABASE_URL`	`TINA4_DATABASE_URL`
`DATABASE_USERNAME`	`TINA4_DATABASE_USERNAME`
`DATABASE_PASSWORD`	`TINA4_DATABASE_PASSWORD`
`DB_URL`	`TINA4_DATABASE_URL` (alias dropped)
`SECRET`	`TINA4_SECRET`
`API_KEY`	`TINA4_API_KEY`
`JWT_ALGORITHM`	`TINA4_JWT_ALGORITHM`
`SMTP_HOST`	`TINA4_MAIL_HOST`
`SMTP_PORT`	`TINA4_MAIL_PORT`
`SMTP_USERNAME`	`TINA4_MAIL_USERNAME`
`SMTP_PASSWORD`	`TINA4_MAIL_PASSWORD`
`SMTP_FROM`	`TINA4_MAIL_FROM`
`SMTP_FROM_NAME`	`TINA4_MAIL_FROM_NAME`
`IMAP_HOST`	`TINA4_MAIL_IMAP_HOST`
`IMAP_PORT`	`TINA4_MAIL_IMAP_PORT`
`IMAP_USER`	`TINA4_MAIL_IMAP_USERNAME`
`IMAP_PASS`	`TINA4_MAIL_IMAP_PASSWORD`
`HOST_NAME`	`TINA4_HOST_NAME`
`SWAGGER_TITLE`	`TINA4_SWAGGER_TITLE`
`SWAGGER_DESCRIPTION`	`TINA4_SWAGGER_DESCRIPTION`
`SWAGGER_VERSION`	`TINA4_SWAGGER_VERSION`
`ORM_PLURAL_TABLE_NAMES`	`TINA4_ORM_PLURAL_TABLE_NAMES`

Names that stay un-prefixed (not framework config)

`PORT`, `HOST`, `NODE_ENV`, `RACK_ENV`, `RUBY_ENV`, `ENVIRONMENT` - these are runtime / PaaS conventions, not framework config. Heroku, Railway, Vercel, and friends set them; we keep reading them.

How to upgrade

- **Backup your** `.env`**:** `cp .env .env.bak.pre-v3.12`

- **Run the migration:** `tina4 env --migrate` - rewrites your `.env` in place.
- **Update PaaS dashboards:** Heroku, Railway, Vercel, Render, Fly.io etc - rename the same vars in your provider's env-var UI.
- **Restart your app.** The boot guard verifies nothing legacy remains.

If your app uses `SECRET`, `DATABASE_URL`, or any other listed name in places besides `.env` (e.g. your CI pipeline's `env:` blocks), update those too - the boot guard checks `os.environ`, not just `.env`.

Parity

All 4 frameworks aligned at **3.12.0**:

- tina4-python 3.11.32 -> 3.12.0
- tina4-php 3.11.32 -> 3.12.0
- tina4-ruby 3.11.32 -> 3.12.0
- tina4-nodejs 3.11.32 -> 3.12.0

Coordinated release across PyPI, Packagist, RubyGems, npm.

v3.11.32 (2026-04-25)

Critical fix - pool + transactions are now actually atomic. Plus a coordinated parity release that aligns all four frameworks at the same version after months of drift.

Before this release, creating a `Database` with `pool > 0` silently broke transactions. The pool's round-robin checkout rotated to a different adapter on every call - so `start_transaction()` pinned its flag on adapter A, the executes autocommitted on adapters B and C, and the final `commit()` / `rollback()` landed on adapter D, which had nothing to commit. Result: `rollback()` was a no-op, writes leaked through, and no error or log surfaced the problem.

The fix pins one adapter to the calling context for the lifetime of a transaction. Each language uses its own primitive:

- **Python** - `threading.local()` on the `Database` instance
- **Ruby** - `Thread.current[:tina4_pinned_adapter_<obj_id>]`
- **Node.js** - `AsyncLocalStorage` from `node:async_hooks` (async-safe across overlapping awaits)
- **PHP** - per-instance property (PHP-FPM is one process per request; threading.local is unnecessary)

While pinned, every database call routes to the same adapter. `commit()` and `rollback()` release the pin so subsequent calls round-robin again.

- **fix (database / all 4):** adapter pinning across transaction scope in `Database._get_adapter()` (and language equivalents). Every backend is affected - SQLite, PostgreSQL, MySQL, MSSQL, Firebird. Firebird exposed it loudest because of its honest "commit-empty-txn is a real no-op" semantics; the others mostly hid the bug behind eager autocommits but still lost rollback atomicity.

• **tests (all 4):** new regression suite - three INSERTs followed by `rollback()` under `pool=4` now leaves zero rows (was leaking three). Three INSERTs followed by `commit()` persists exactly three. Pin-release after commit/rollback verified. `pool=0` regression test added so single-connection mode stays unaffected.

• **parity / version alignment:** all 4 frameworks bumped to 3.11.32 - closes the cross-framework version drift that had built up (PHP at 3.11.31, Python at 3.11.24, Ruby and Node at 3.11.19). A single coordinated release across all four registries: PyPI, Packagist, RubyGems, npm.

No migration needed. Code using `pool=0` (the default for every adapter except where explicitly raised) is unaffected. Code using `pool>0` will now actually honour transactions instead of silently dropping them.

If you've been seeing intermittent "writes vanished" or "rollback didn't help" reports on a pooled `Database`, this release is the cause and the cure.

v3.11.13 (2026-04-16)

Issue-driven release. Everything reported in the open tina4-book issues either was fixed in this version or is already fixed in 3.11.12; this release consolidates the remaining bits and corrects documentation drift.

• **feat (router / all 4):** Explicit typed-parameter system shared across Python, PHP, Ruby, Node. Adds `alpha`, `alnum`, `slug`, `uuid`, and explicit `string` types in addition to the existing `int`/`integer`, `float`/`number`, `path`/`.*`. **Unknown type names now throw at registration** - `{name:str}`, `{id:inetger}`, etc. raise with a clear message listing the valid types instead of silently falling through to the default matcher. Fixes tina4-book#125. +45 new tests across the four suites.

• **fix (gallery / python+php+ruby):** Gallery Try-It / View buttons now open the deployed example in a new tab (`window.open(url, '_blank')`) instead of navigating away from the gallery home. Fixes tina4-book#115.

• **fix (ruby gemspec):** `sqlite3` promoted from `add_development_dependency` to `add_dependency`. Matches the "zero-config SQLite on first run" promise. Fixes tina4-book#100.

• **docs (tina4-book):** PHP Chapter 2 updated - correct port (7145), `->noAuth()` on write-method examples, and an explicit callout explaining the secure-by-default policy for POST/PUT/PATCH/DELETE. Addresses tina4-book#87, #94, #123.

• **docs (tina4-book):** Python `@template` decorator ordering corrected (must sit BELOW the route decorator) in book chapters 04 and 10; Python `request->query` vs `request->params` distinction in PHP chapter 1.

• **tests (python):** Session-handler tests updated to reflect the real default TTL of 3600s (were stale at 1800s).

• **verified already fixed in earlier 3.11.x releases** - closed comments posted on all of these:

• #79 dotted numeric index (`{{ items.0.name }}`)

• #80 `truncate` filter

- #82 `{{ parent() }}` / `{{ super() }}` across all 4 frameworks
- #83 Ruby dashboard - WEBrick is runtime dep
- #89 `load_dotenv` rename, `DatabaseResult` methods, SQLite WAL locking
- #91 Ruby `request.params` symbol + string keys via `IndifferentHash`
- #93 Ruby `/docs/*` and bare `/*` wildcard routes
- #97 Frond ternary operator
- **parity:** All 4 frameworks bumped to 3.11.13.

v3.11.12 (2026-04-16)

Breaking: `sqlite:///X` URLs are now relative to the project root (cwd), matching the documented convention. For absolute paths use four slashes (`sqlite:////abs/path.db`) or a Windows drive letter (`sqlite:///C:/Users/app.db`).

Before this release, `TINA4_DATABASE_URL=sqlite:///data/app.db` was interpreted differently by every framework. Python/Node/Ruby tried to open `/data/app.db` (absolute) which crashed on macOS with `OSError: [Errno 30] Read-only file system: '/data'`. PHP did the same under the hood. All four frameworks now agree: three slashes = relative, four slashes = absolute.

- **fix (all 4):** `sqlite:///X` resolves under cwd; parent directory auto-created only when inside cwd. Absolute paths are trusted and never mkdir'd at root.
- **fix (python):** `_ensure_folders` no longer creates a bogus `src/migrations/` directory. The migration runner always looks at `migrations/` at the project root - there is only one correct location.
- **parity (php, ruby, node):** Same `sqlite:///X` parsing as Python. Dedicated `resolve_path` / `resolveSqlitePath` helpers in each framework so adapters consistently handle `:memory:`, `./` forms, Windows drive letters.
- **tests:** 9 new Python tests in `TestSQLiteConnectionPath` + `TestProjectFolders`. 4 new PHP tests in `DatabaseUrlTest` covering relative/absolute/Windows/bruce-regression. 6 new Ruby specs in `database_drivers_spec.rb :: SqliteDriver.resolve_path`. Node URL tests expanded in `database.test.ts` with the full relative/absolute/Windows/:memory: matrix.
- **parity:** All 4 frameworks bumped to 3.11.12.

Migration note: If your `.env` has `TINA4_DATABASE_URL=sqlite:///data/app.db`, it will now create `./data/app.db` in the project root (which is what most users actually want). If you genuinely want an absolute path, change to `sqlite:////data/app.db` (four slashes).

v3.11.11 (2026-04-16)

• **fix (python ORM):** `Field.validate` no longer re-coerces values that are already the correct type. Previously, any PostgreSQL/MSSQL read of a row containing a `DateTimeField` crashed because `datetime(datetime_instance)` raises `TypeError`. The fix accepts native driver types (`datetime`, `bytes`, `int`, `bool`, `float`, `str`) without re-wrapping, and parses ISO-8601 strings into `datetime` for SQLite. See `tina4-python/plan/orm-field-validate-native-types.md`.

• **fix (python ORM):** `BooleanField` vs `IntegerField` ordering handled explicitly. `BooleanField(1)` still coerces to `True`, `IntegerField(True)` still coerces to `1`; no regression for either direction (bool is a subclass of int in Python).

• **tests (python):** 10 new `TestFieldsNativeTypes` cases covering datetime/int/bool/float/bytes/string/ForeignKey round-trips.

• **tests (parity):** Regression-guard "datetime round-trip on read path" tests added to PHP (`ORMV3Test`), Ruby (`orm_spec`) and Node.js (`orm.test.ts`) so an equivalent bug can't creep in there later.

• **parity:** All 4 frameworks bumped to 3.11.11.

v3.11.10 (2026-04-15)

• **fix (php):** Hot-reload loop - DevAdmin's polling fallback used `mt=0` as the baseline, so the first poll after every page load triggered `location.reload()`, which reset `mt=0` again. Loop now initialises the baseline on the first poll.

• **fix (php):** Reload sentinel removed - PHP was the only framework recursively walking `src/` and touching `src/.reload_sentinel` on every reload POST. The sentinel lived inside the Rust CLI's watched tree and fed back into the watcher, triggering a second loop. Replaced with the same in-memory counter used by Python/Ruby/Node.

• **fix (php):** Polling no longer starts more than once when the WebSocket reconnect retry budget is exhausted (added a `pollStarted` guard).

• **feat (parity):** `GET /__dev/api/queue/topics` and `GET /__dev/api/queue/dead-letters` added to PHP, Ruby and Node (previously only in Python). PHP queue endpoints now read from the real `Tina4\Queue` backend instead of returning stubs.

• **feat (devadmin):** Refreshed `tina4-dev-admin.js` bundle (87.8 KB) across all 4 frameworks - adds the topic selector dropdown, inline payload expand/copy, and corrected version display.

• **tests:** 4-way parity tests for hot-reload: mtime starts at 0, POST /__dev/api/reload bumps the counter, no sentinel file is written to disk, mtime is monotonic across successive reloads. Mirrored in `tina4-php/tests/DevAdminTest.php`, `tina4-python/tests/test_dev_admin.py`, `tina4-ruby/spec/dev_admin_spec.rb`, `tina4-nodejs/test/devAdmin.test.ts`.

• **parity:** All 4 frameworks bumped to 3.11.10.

---

v3.11.9 (2026-04-15)

Catch-up release covering v3.11.0 -> v3.11.9 across all 4 frameworks.

• **feat (websocket):** Full WebSocket parity across Python/PHP/Node/Ruby - `get_client_rooms()` / `getClientRooms()`, `route()` usable as decorator or direct handler registration, matching room/broadcast semantics, plus new parity tests on all 4.

• **feat (graphql):** Input validation and field-level `@auth` directives with context threading.

• **feat (graphql):** Auto-discovery of schemas; removed legacy DevAdmin HTML/JS in favour of the new UI.

• **feat (devadmin - Python):** Queue tab with topic selector, dead-letter listing and replay endpoints, inline payload expand/copy, version display.

• **feat (cli):** Rust CLI now owns file watching - frameworks receive `POST /__dev/api/reload` and internal watchers are disabled when launched by the Rust CLI (`--managed`).

• **fix (cli):** `parseFlags` / `parse_flags` / `parseCliArgs` no longer swallow `host:port` or positional args after boolean flags.

• **fix (scss):** SCSS recompilation loop fixed; output path corrected to `src/public/css/` to match CLI and static serving.

• **fix (frond - Python):** Numeric dotted index for lists (`items.0.name`) now resolves correctly.

• **fix (router - Ruby):** Bare `/*` wildcard capture exposed under `"*"` key for parity.

• **fix (orm - PHP):** Three data-sync bugs fixed: `load()` double-fill, `getPrimaryKeyValue`, `save()` ID sync.

• **fix (graphql):** `from_orm` / `fromOrm` list resolver used `select(skip=)` instead of `all(offset=)`.

• **fix (metrics):** Windows backslash paths normalised to forward slashes.

• **fix (app - PHP):** No longer crashes on notices/deprecations in loaded files; `run()` now prints the banner when starting the server directly.

• **chore:** Example demo store ships with the repo; Windows-friendly setup; `.env.example` and setup scripts added.

• **parity:** All 4 frameworks bumped to 3.11.9. PHP aligned to the 3.x tag scheme on `v3`.

v3.10.99 (2026-04-12)

• **breaking:** `autoMap` now defaults to `true` - ORM models automatically map between camelCase properties and snake_case DB columns. Set `static autoMap = false;` on your model to restore the old behaviour.

• **feat:** `toDict(include, case)` parameter - pass `'snake'` as second arg to get snake_case keys matching DB columns, or `'camel'` (default) for camelCase.

• **feat:** Frond `replace` filter now accepts object args - `{{ v|replace({"T": " ", "-": "/"}) }}` for multiple substitutions in one call.

• **tests:** 13 new parity tests covering `toDict(case)`, `autoMap` default, `replace` filter (object + positional), and `ServiceRunner` registration. 268 tests passing.

• **parity:** All features shipped identically across Python, PHP, Ruby, Node.js.

v3.10.97 (2026-04-11)

• **fix:** frond.form.submit redirect handling - XHR follows 3xx redirects transparently; fixed by detecting `xhr.responseURL` mismatch and navigating instead.

• **dep:** Updated frond.min.js to v2.1.2.

• **parity:** All 4 frameworks bumped to 3.10.97.

v3.10.93 (2026-04-11)

• **fix:** Frond bracket depth tracking in `findOutsideQuotes()` and `splitOutsideQuotes()` - expressions like `arr[i % 2]` no longer treated as top-level arithmetic.

• **fix:** Frond subscript expression evaluation - bracket content uses `evalExpr()` instead of direct context lookup, enabling `arr[loop.index0 % 2]`.

• **fix:** Frond slice with variable bounds - `items[start:end]` evaluates bounds through `evalExpr()`.

• **docs:** Developer skills updated - Metrics Dashboard guidance, Frond Template Parity rules, `@noauth` security warnings.

• **parity:** All Frond fixes applied identically across Python, PHP, Ruby, Node.js. 2,831 tests passing (268 Frond).

v3.10.92 (2026-04-10)

• **feat:** Add `DevAdmin` methods - `capture()` (5-param), `clearAll()`, `health()`, `unresolvedCount()`, `reset()`, `register()`.

• **feat:** Add `Server.start()` and `Server.stop()` for cross-framework parity.

• **feat:** Add `DatabaseResult.size()` method.

• **feat:** Add `DevReload.start()` and `DevReload.stop()`.

• **feat:** Add `ScssCompiler.compileScss()` method.

• **fix:** `autoCrud.ts` - fix spread syntax on non-iterable, add id in POST response, correct response format to `{data, meta}`, change validation status from 400 to 422.

• **parity:** 44/44 cross-framework features green. 2,752 tests passing.

v3.10.91 (2026-04-10)

• **feat:** Add parity methods - `GraphQLType.parse()`, `CorsMiddleware.isPreflight()`, `RateLimiterMiddleware.check()`.

• **breaking:** Rename `from()` -> `fromTable()`, remove `template()` alias - align with Python canonical names.

v3.10.90 (2026-04-09)

• **docs:** Chapter 4 (Templates) - new "Dumping Values for Debugging" section covering both `{{ x|dump }}` and `{{ dump(x) }}` forms, the v3.10.88 `inspectValue()` inspector (circular refs, BigInt, Map/Set, Error, Date, class instances), and the `TINA4_DEBUG=true` production gate. Filter table entry updated to reference the new section.

• **docs:** `plan/parity/parity-template.md` updated with a cross-framework dump helper comparison table and marks dump parity as confirmed across all 4 frameworks at v3.10.89.

• **chore:** Version sync release - brings all 4 frameworks to the same patch version (3.10.90) so downstream users can upgrade PHP/Python/Ruby/Node.js in lockstep without hunting version mismatches.

v3.10.89 (2026-04-09)

• **feat:** `{{ dump(value) }}` global function form added to Frond alongside the existing `{{ value|dump }}` filter. Both call a single `renderDump()` helper (which delegates to the v3.10.88 `inspectValue()` inspector) and produce identical output.

• **security:** Dump is now **gated on** `TINA4_DEBUG=true`. In production (env var unset or `false`) both the filter and function silently return an empty `SafeString`. This prevents accidental leaks of internal state, object shapes, and sensitive values into rendered HTML when a developer leaves a `{{ dump(x) }}` call in a template.

• **test:** 4 new tests in `frond.test.ts` covering `dump()`/`|dump` parity, debug-mode circular ref handling, production silencing for both forms.

v3.10.88 (2026-04-09)

• **fix:** `{{ value|dump }}` filter now handles complex objects safely. The previous implementation used `JSON.stringify` which crashed on circular references and BigInt, silently dropped functions/Symbols/`undefined`, and serialised `Map`/`Set`/`Error`/class instances as empty `{}`. Replaced with an `inspectValue()` inspector that matches PHP's `var_dump`, Python's `repr`, and Ruby's `inspect`:

• Circular references: `[Circular]`

• BigInt: `123n`

• Date: `Date(2026-04-09T13:00:00.000Z)`

• Map / Set: `Map(2) { "a" => 1, "b" => 2 }` / `Set(3) { 1, 2, 3 }`

• Error: `Error("boom")`

• Class instances: `User { name: "Alice", age: 30 }` (class name preserved)

• Functions: `[Function: name]`

• Depth-capped at 8 levels to prevent runaway graphs

• **test:** 11 new edge-case assertions in `frond.test.ts` (frond.test now 254 passing).

v3.10.87 (2026-04-09)

• **fix:** Dev toolbar no longer vanishes after a hot-reload. The CLI watcher used to call `server.router.clear()` on every file change - including template/CSS/JS asset edits - which left a brief window of 404 responses that bypass the dev toolbar injection. The watcher now reports whether a `.ts/.tsx/.js/.jsx` source file changed; router re-discovery only runs on code changes, and asset edits pass through without touching the router. Matches the PHP v3.10.87 fix.

v3.10.86 (2026-04-09)

• **feat:** `foreignKey` field type on `BaseModel` auto-wires both sides of a foreign key relationship. Declaring `user_id: { type: "foreignKey", references: "User" }` injects a `belongsTo` entry on the declaring model and a `hasMany` entry on the referenced model via a module-level FK registry. New static methods `_processForeignKeys()` and `_applyFkRegistry()` are called lazily before relationship resolution. Optional `relatedName` overrides the has-many key.

• **feat:** Cross-framework parity - same FK auto-wiring semantics now available in Python (`ForeignKeyField`), PHP (`$foreignKeys`), and Ruby (`foreign_key_field`)

• **docs:** Chapter 6 (ORM) updated with a new "foreignKey Field Type - Auto-Wired Relationships" section

v3.10.85 (2026-04-09)

• Version bump for parity with Python and PHP releases

v3.10.84 (2026-04-09)

• **fix:** Router/middleware was setting `request.user` / `request.auth` / auth payload to `true` (boolean) instead of the actual JWT payload after `validToken()` was changed to return bool - any code reading `request.user.sub` etc. would have failed silently or crashed

• **fix:** CSRF middleware was not correctly rejecting invalid tokens (null check on bool result always passed)

• **add:** Headless routing auth payload integration tests to prevent regression

v3.10.83 (2026-04-08)

• **feat:** WebSocket rooms - `joinRoom`, `leaveRoom`, `broadcastToRoom`, `getRoomConnections`, `roomCount`, `getClientRooms`

• **feat:** Queue signature parity - instance-scoped `push`/`pop`/`retry`, no topic params on public methods

• **feat:** Auth alias cleanup - removed `createToken`/`validateToken`, canonical `getToken`/`validToken`

v3.10.70 (2026-04-06)

- **New:** SSE (Server-Sent Events) support via `response.stream()` - pass a generator, framework handles chunked transfer encoding, keep-alive, and `text/event-stream` content type
- **New:** Chapter 24 added to documentation: Server-Sent Events
- Feature count: 45 (was 44)
- Full parity across Python, PHP, Ruby, Node.js

Tina4 Node.js follows semantic versioning. The major version (3) marks the initial Node.js launch - Tina4 Node.js is new in the v3 line, alongside Tina4 Ruby. The minor version tracks feature additions. The patch version tracks fixes, template engine corrections, and cross-framework parity updates.

This chapter covers every release from v3.0.0 through v3.10.x. Each section groups releases by minor version, lists features added, bugs fixed, and breaking changes with migration code where relevant.

---

v3.10.68 (2026-04-03) - Full Parity Release

- **100% API parity** across Python, PHP, Ruby, Node.js - 30+ issues fixed
- **ORM:** save() returns self/false, arrays not tuples, toDict/toAssoc, scope registers method, where()/all() on Node, count() on PHP
- **Auth:** expires*in minutes, PBKDF2 260k, env TINA4*SECRET fallback, API key fallback
- **Session:** dual-mode flash(), get_flash, cookieHeader, getSessionId
- **Database:** execute() bool/DatabaseResult, get/*last*id/get_error, getColumns, cacheStats
- **Request/Response:** files dict, query, cookies, contentType, xml(), callable
- **Queue:** consume() poll_interval
- **WebSocket:** event naming, connection properties
- **GraphQL:** schema_sdl() + introspect() on all 4
- **Events:** emitAsync() on all 4
- **i18n:** zero-dep YAML support

v3.10.67 (2026-04-03)

- **load() returns boolean** - `model.load(sql, params)` calls selectOne internally, populates the instance, returns `true`/`false`. Use `findById()` for PK lookups
- **api.upload()** added to tina4-js - sends FormData with Bearer token auth for multipart file uploads
- **ORM CLAUDE.md rewrite** - all method stubs now match actual API signatures
- **File upload docs** - `req.files` format documented in CLAUDE.md

v3.10.66 (2026-04-03)

• **Metrics file detail fix** - clicking bubbles in framework scanning mode now resolves paths correctly via scan root tracking

v3.10.65 (2026-04-03)

• **Metrics 3-stage test detection** - filename, path, and content matching

• **Metrics framework mode** - scans framework source with correct relative paths

• **tina4 console** - interactive REPL with framework loaded

• **tina4 env** - interactive environment configuration

• **Brand** - "TINA4 - The Intelligent Native Application 4ramework"

• **Quick references** - 36 sections, DotEnv API documented

• **37 chapters** - 7 new (Events, Localization, Logging, API Client, WSDL/SOAP, DI Container, Service Runner)

• **MongoDB + ODBC adapters** across all 4 frameworks

• **Pagination standardized** - limit/offset primary, merged dual-key response

• **Port kill-and-take-over** on startup

---

v3.10.60 (2026-04-03)

• **tina4 console** - interactive Node REPL with framework loaded (db, Router, Database, Log)

• **tina4 env** - interactive environment configuration

• **Brand update** - "TINA4 - The Intelligent Native Application 4ramework"

• **Dynamic version** - reads from package.json at runtime

• **Port kill-and-take-over** - default port always reclaimed

• **findAvailablePort** - checks 0.0.0.0 not 127.0.0.1

• **MongoDB adapter** (mongodb npm), **ODBC adapter** (odbc npm)

• **Pagination standardized** - limit/offset primary, merged dual-key response

• **Metrics dependency lines** - basename fix for correct rendering

• **autoMap uppercase** - snakeToCamel lowercases first

---

v3.10.57 (2026-04-02)

• **MongoDB adapter** - `initDatabase({ url: "mongodb://host:port/db" })`, requires `npm install mongodb`

- **ODBC adapter** - `initDatabase({ url: "odbc:///DSN=MyDSN" })`, requires `npm install odbc`
- **Pagination standardized** - limit/offset primary, merged dual-key toPaginate() response
- **Test port at +1000** - user testing port (e.g. 8148) stable, no hot-reload
- **Dynamic version** - read from package.json, no hardcoded constant
- **Metrics dependency lines** - fixed basename parsing
- **autoMap uppercase columns** - snakeToCamel lowercases first
- **ORM TINA4*DATABASE*URL discovery** - auto-connect from env for SQLite
- **108 features at 100% parity**, 2,646 tests

---

v3.10.54 (2026-04-02)

- **Auto AI dev port** - second HTTP server on port+1 with no-reload when TINA4_DEBUG=true
- **TINA4*NO*RELOAD** env var + --no-reload CLI flag
- **SQLite transaction safety** - commit/rollback/startTransaction guarded
- **autoMap uppercase columns** - snakeToCamel lowercases first
- **ORM TINA4*DATABASE*URL discovery** - auto-connect from env for SQLite
- **QueryBuilder docs** - added to ORM chapter

---

v3.10.48 - April 2, 2026

Bug Fixes

Cluster mode requires `TINA4_PRODUCTION=true` - Worker forking no longer auto-triggers when debug is off. Set `TINA4_PRODUCTION=true` env var or use `tina4 serve --production` to enable cluster mode.

---

v3.10.46 - April 1, 2026

Test Coverage

CSRF middleware expanded to 32 tests matching Python reference. Node.js now at 2,546 tests with full parity across all 49 core areas.

---

v3.10.45 - April 1, 2026

Notes

Version bump for parity with PHP CLI serve fix. No Node.js-specific changes.

---

v3.10.44 - April 1, 2026

New Features

Database tab redesign - Split-screen layout with tables navigation on the left and query editor + results on the right. Click-to-select table highlighting.

Copy CSV / Copy JSON - Copy query results to clipboard in CSV or JSON format.

Paste data - Modal for pasting JSON arrays or CSV/tab-separated data. Auto-generates INSERT statements targeting the selected table, or prompts for a new table name with CREATE TABLE generation. SQL input passes through unchanged.

Multi-statement execution - Query runner handles batched SQL statements in a transaction.

Database badge on load - Table count shows immediately without clicking the Database tab.

Star wiggle animation - Empty star (*) on the landing page with delayed wiggle animation at random intervals.

Bug Fixes

Default port - Node.js default port set to 7148 (PHP=7145, Python=7146, Ruby=7147, Node=7148).

SQLite LIMIT fix - Prevents double-LIMIT errors in the database browser.

browseTable quote escaping - Fixed table name click handlers.

Server handler dispatch regex - Fixed a regex that required whitespace after `async` in handler functions. Transpiled auto-CRUD handlers producing `async(req,res)=>` were called with zero arguments, causing crashes.

Cluster mode in tests - Server-based tests now set `TINA4_DEBUG=true` to prevent cluster mode forking, which was causing ECONNREFUSED errors.

Test Coverage

Massive test expansion - 718 new tests added across Auth (+52), ORM (+30), FakeData (+48), Cache (+23), DevMailbox (+32), Static (+21), Queue (+20), Frond (+57), CLI scaffolding (55), Metrics (69), plus v3.10.44 feature tests and server test fixes. 2,530 tests passing, 0 failures.

---

v3.10.40 - April 1, 2026

Bug Fixes

Dev overlay version check - Fixed misleading "You are up to date" message when running a version ahead of what's published on npm. The overlay now shows a purple "ahead of npm" message. Also added a breaking changes warning (red banner with changelog link) when a major or minor version update is available.

---

v3.10.39 - April 1, 2026

New Features

`Database.getColumns(tableName)` - Returns `[{name, type, nullable, default, primaryKey}]` for each column. Uses `PRAGMA table_info` for SQLite and `information_schema.columns` for PostgreSQL/MySQL/MSSQL.

`Database.executeMany(sql, paramSets)` - Execute a SQL statement with multiple parameter arrays in a single transaction for atomicity and performance.

`BaseModel.create<T>(data)` - Static factory method: instantiates, saves, and returns the new record.

`BaseModel.find()` **and** `BaseModel.load()` - aliases for `findById()` (parity with Python, PHP, Ruby).

`seed` **CLI command** - `tina4nodejs seed` scans `src/seeds/*.ts` and executes them via `tsx`.

`Router.allRoutes()` - alias for `getRoutes()`.

---

v3.10.38 - April 1, 2026

Code Metrics & Bubble Chart

The dev dashboard (`/__dev`) now includes a **Code Metrics** tab with a PHPMetrics-style bubble chart visualization. Files appear as animated bubbles sized by LOC and colored by maintainability index. Click any bubble to drill down into per-function cyclomatic complexity.

The metrics engine uses regex-based TypeScript/JavaScript analysis for zero-dependency static analysis covering cyclomatic complexity, Halstead volume, maintainability index, coupling, and violation detection. File analysis is sorted worst-first. Results are cached for 60 seconds.

AI Context Installer

`npx tina4nodejs ai` now presents a simple numbered menu instead of auto-detection. Select tools by number, comma-separated or `all`. Already-installed tools show green. Generated context includes the full skills table.

Dashboard Improvements

Full-width layout, sticky header/tabs, full-screen overlay. Fixed `/__dev/` trailing slash returning 404.

Cleanup

Removed `demo/` directory. Removed old `plan/` spec documents, replaced with `PARITY.md` and `TESTS.md`. Central parity matrix added to tina4-book.

---

v3.10.x - Previous Releases (March 28-31, 2026)

The v3.10 line focused on ORM refinements, Frond template engine fixes, and cross-framework parity. Thirty-two patch releases landed in four days.

Features

autoMap for ORM field mapping (v3.10.1)

The ORM gained automatic translation between JavaScript camelCase and database snake_case. Set `autoMap = true` on a model and the framework handles the rest.

```
import { BaseModel } from "tina4-nodejs";

class User extends BaseModel {
  static tableName = "users";
  static autoMap = true;
  static fields = {
    id: { type: "integer" as const, primaryKey: true },
    firstName: { type: "string" as const },   // maps to first_name
    lastName: { type: "string" as const },     // maps to last_name
    createdAt: { type: "datetime" as const },  // maps to created_at
  };
}
```

Explicit `fieldMapping` entries take precedence over auto-generated ones. The two utilities `snakeToCamel()` and `camelToSnake()` are exported for direct use.

WSDL lifecycle hooks and dotted function names (v3.10.6)

WSDL services gained `beforeCall` and `afterCall` hooks. The Frond template engine learned to resolve dotted function names like `{{ utils.format(value) }}`.

ORM auto-commit on write operations (v3.10.13)

The ORM now commits after every `save()` and `delete()` call. Before this change, writes on SQLite would silently succeed in memory but never persist to disk unless you called `commit()` yourself.

get*next*id() for ID pre-generation (v3.10.14)

Models gained a `getNextId()` method. It queries the database engine for the next auto-increment value before the insert happens. Useful when you need the ID for a related record before you save the parent.

```
const nextId = await User.getNextId();
// Use nextId in a related record before saving the User
```

Template filters: to*json*, *tojson*, *j*sescape (v3.10.16)

Three new Frond filters for passing data from templates to JavaScript:

```
<script>
  const config = {{ settings|to_json }};
  const message = "{{ userInput|js_escape }}";
</script>
```

formTokenValue() in Frond templates (v3.10.23)

Templates gained a `formTokenValue()` function that generates a unique CSRF token per form. Each token carries a nonce in the JWT payload, so two forms on the same page get distinct tokens (v3.10.22).

```
<form method="POST" action="/submit">
  <input type="hidden" name="formToken" value="{{ formTokenValue() }}">
  <button type="submit">Send</button>
</form>
```

Arithmetic in set and expressions (v3.10.31)

The Frond engine learned arithmetic. `{% set total = price * quantity %}` and `{{ width + padding }}` now work as expected.

MCP server (v3.10.32)

Tina4 Node.js ships a built-in MCP (Model Context Protocol) server. AI coding tools can connect to your running application and inspect routes, models, and database schema.

Bug Fixes

Frond dict[variable_key] access (v3.10.11)

Variable keys in dictionary access were ignored. The engine treated `dict[myVar]` as a literal string lookup instead of resolving `myVar` first.

```
{# Before fix - broken: always looked up the literal string "myVar" #}
{% set key = "name" %}
{{ user[key] }}  {# returned undefined #}

{# After fix - works: resolves key to "name", then looks up user["name"] #}
{% set key = "name" %}
{{ user[key] }}  {# returns the user's name #}
```

Frond |replace filter backslash escaping (v3.10.15)

The `|replace` filter mangled backslashes. A replacement string containing `\n` would insert a literal newline instead of the two characters `\n`.

Frond variable resolution (v3.10.17)

Nested variable lookups in certain template constructs returned `undefined`. The engine now walks the scope chain correctly.

Frond inline-if with quoted strings (v3.10.19)

Inline conditionals broke when the true/false branches contained quoted strings with spaces. The parser split on whitespace inside the quotes.

Filters in if conditions (v3.10.21)

Filters inside `{% if %}` conditions were silently ignored. The condition evaluated the raw value instead of the filtered one.

```
{# Before fix - broken: |length filter ignored, condition tested the array itself #}
{% if items|length > 0 %}

{# After fix - works: |length runs first, condition compares the number #}
{% if items|length > 0 %}
```

Stale templates in dev mode (v3.10.24)

The dev server cached compiled templates and ignored file changes. Editing a template required a server restart. The fix reads the filesystem on every request in development mode, while production mode keeps the cache.

ORM save/delete transaction safety (v3.10.25)

SQLite threw "cannot commit - no transaction is active" when the ORM called `commit()` outside an explicit transaction. The ORM now wraps every `save()` and `delete()` in a `startTransaction()`/`commit()`/`rollback()` block.

```
// Before fix - threw on SQLite:
const user = new User({ firstName: "Alice" });
await user.save(); // Error: cannot commit - no transaction is active

// After fix - works on all database engines:
const user = new User({ firstName: "Alice" });
await user.save(); // Transaction handled internally
```

Frond macro HTML escaping (v3.10.27)

Macro output was HTML-escaped when used inside `{{ }}` expressions. A macro that generated `<div>` would render as `<div>`. Nested macros double-escaped. Macro output is now treated as safe HTML, matching standard Twig behaviour.

js*escape and to*json auto-escaping (v3.10.17-19)

The `js_escape` and `to_json` filters produced output that Frond then HTML-escaped. A JSON string like `{"key":"value"}` became `{"key":"value"}`. These filters now wrap their output in SafeString to bypass auto-escaping.

Firebird-Specific

Migration runner fixes (v3.10.10)

The migration runner generated SQLite-style `AUTOINCREMENT` and `TEXT` types for Firebird. Firebird needs generators and `VARCHAR`. The runner now emits the correct DDL and generates IDs from a `GEN_TINA4_MIGRATION_ID` sequence.

---

v3.9.x - QueryBuilder, Sessions, Path Injection (March 26-27, 2026)

The v3.9 line delivered three features that changed how developers write routes and query data.

Features

QueryBuilder (v3.9.0)

A fluent SQL builder that integrates with the ORM. Chain methods to build queries without writing raw SQL.

```
import { User } from "./orm/User.js";

// ORM integration
const admins = await User.query()
  .where("role = ?", ["admin"])
  .orderBy("name")
  .limit(10)
  .get();

// Standalone usage
import { QueryBuilder } from "tina4-nodejs";

const results = await QueryBuilder.from("orders")
  .where("total > ?", [100])
  .leftJoin("customers", "orders.customer_id = customers.id")
  .orderBy("total", "DESC")
  .limit(20)
  .get();

// Utility methods
const exists = await User.query().where("email = ?", [email]).exists();
const total = await User.query().where("active = ?", [true]).count();
const first = await User.query().where("id = ?", [1]).first();
```

The builder supports `select`, `where`, `orWhere`, `join`, `leftJoin`, `groupBy`, `having`, `orderBy`, `limit`, `first`, `count`, `exists`, and `toSql`.

Path parameter injection (v3.9.0)

Route handlers receive path parameters as named function arguments. The framework inspects the handler's parameter names and injects matching values.

```
import { Router } from "tina4-nodejs";

// The framework injects id as a typed argument
Router.get("/users/{id:int}", async (id, request, response) => {
  const user = await User.find(id);
  response.json(user);
});
```

Auto-start sessions (v3.9.0)

Every route handler receives a session object on `request.session` with zero configuration. No middleware to register. No setup code.

```
Router.post("/login", async (request, response) => {
```

```
  request.session.set("userId", 42);
  request.session.flash("message", "Welcome back.");
  response.redirect("/dashboard");
});

Router.get("/dashboard", async (request, response) => {
  const userId = request.session.get("userId");
  const flash = request.session.getFlash("message");
  response.render("dashboard", { userId, flash });
});
```

The session API: `get`, `set`, `delete`, `has`, `clear`, `destroy`, `save`, `regenerate`, `flash`, `getFlash`, `all`.

CSRF middleware and secure-by-default (v3.9.1)

POST, PUT, PATCH, and DELETE routes require authentication by default. The framework ships a `CsrfMiddleware` that validates session-bound form tokens.

```
// Routes that modify data are protected out of the box
Router.post("/api/orders", async (request, response) => {
  // Request must include a valid form token or auth header
  // Otherwise the framework returns 403
});
```

Queue parity (v3.9.1)

The queue system gained priority-based push, `size(status)` to count jobs by state, `job.retry()`, and `job.topic` for filtering.

NoSQL QueryBuilder and WebSocket backplane (v3.9.2)

The QueryBuilder gained MongoDB support. WebSocket servers gained a backplane for broadcasting across multiple server instances.

SameSite=Lax cookie default (v3.9.2)

Session cookies now set `SameSite=Lax` by default. This prevents CSRF attacks from cross-origin form submissions without breaking same-site navigation.

Breaking Changes

Secure-by-default for mutation routes (v3.9.1)

All POST, PUT, PATCH, and DELETE routes now require authentication. If your application has public mutation endpoints, mark them as open:

```
// BEFORE (v3.8.x) - all routes were open by default
Router.post("/api/feedback", async (request, response) => {
  // Anyone could call this
});

// AFTER (v3.9.x) - opt out of auth for public endpoints
Router.post("/api/feedback", async (request, response) => {
  // Now requires auth unless you explicitly open it
}).secure(false);
```

session.delete() replaces session.unset() (v3.9.0)

The session method was renamed for cross-framework parity.

```
// BEFORE (v3.8.x)
request.session.unset("userId");

// AFTER (v3.9.x)
request.session.delete("userId");
```

Bug Fixes

ESM compatibility (v3.9.4)

Internal `require()` calls broke on Node 22 with ESM-only configurations. All internal imports now use the ESM `import()` function.

Zero dependencies achieved (v3.9.3)

The `better-sqlite3` native module was the last remaining npm dependency. This release replaced it with Node's built-in `node:sqlite` module. `npm install` no longer needs a C++ compiler or `node-gyp`.

---

v3.8.x - Template Engine, Typed Params, Security (March 25-26, 2026)

The v3.8 line replaced the template engine, added typed route parameters, and introduced production-grade security middleware.

Features

Typed route parameters (v3.8.0)

Route paths gained type annotations. The framework validates and converts parameters before your handler runs.

```
Router.get("/products/{id:int}", async (request, response) => {
  // id is guaranteed to be an integer
  // /products/abc returns 404 automatically
});

Router.get("/prices/{amount:float}", async (request, response) => {
  // amount is a floating-point number
});

Router.get("/docs/{path:path}", async (request, response) => {
  // path captures everything including slashes: /docs/api/v2/users
});
```

Template fallback (v3.8.0)

Requesting `/hello` now serves `src/templates/hello.twig` or `src/templates/hello.html` when no route matches. No explicit route registration needed for static pages.

Connection pooling (v3.8.1)

Set `TINA4_DB_POOL=5` in your `.env` file and the framework creates five database connections. Requests distribute across them in a round-robin pattern.

SecurityHeadersMiddleware (v3.8.1)

A built-in middleware that sets `X-Frame-Options`, `Strict-Transport-Security`, `Content-Security-Policy`, and `X-Content-Type-Options` on every response.

Validator class (v3.8.1)

Input validation with structured error responses:

```
import { Validator } from "tina4-nodejs";

Router.post("/api/users", async (request, response) => {
  const errors = Validator.validate(request.body, {
    email: ["required", "email"],
    name: ["required", "minLength:2"],
    age: ["integer", "min:18"],
  });

  if (errors.length > 0) {
    return response.json({ errors }, 422);
  }
});
```

Upload size limit and Docker support (v3.8.1)

Set `TINA4_MAX_UPLOAD_SIZE=10mb` to cap file uploads. The `tina4 init` scaffolding now includes a multi-stage Alpine Dockerfile.

base64encode and base64decode Frond filters (v3.8.0)

```
{{ sensitiveId|base64encode }}
{{ encodedValue|base64decode }}
```

Production template caching (v3.8.0)

Template lookups are cached in production mode. In development mode, the framework reads the filesystem on every request so changes appear without a restart.

Breaking Changes

Frond replaces Twig dependency (v3.8.4)

The `@tina4/twig` npm package was removed. The framework now uses its built-in Frond engine for all template rendering. Frond supports the same Twig syntax, but if your templates relied on Twig-specific extensions, you need to rewrite them as Frond filters.

```
// BEFORE (v3.7.x) - Twig as a separate dependency
// package.json included "@tina4/twig": "^1.x"
// Templates used Twig-specific extensions

// AFTER (v3.8.x) - Frond is built in, zero dependencies
// Remove @tina4/twig from package.json
// Templates use Frond filters (same Twig syntax, built-in engine)
```

Groundwork for zero dependencies (v3.8.4)

v3.8.4 began migrating from `better-sqlite3` to Node's built-in `node:sqlite` module. The migration completed in v3.9.3.

---

v3.7.x - Template Auto-Serve, Firebird Migrations (March 25, 2026)

A focused release. Two features, no breaking changes.

Features

Template auto-serve at / (v3.7.0)

Place `index.html` or `index.twig` in `src/templates/` and the framework serves it at `/`. User-registered `GET /` routes take priority. When neither exists, the Tina4 landing page appears.

Firebird idempotent migrations (v3.7.0)

`ALTER TABLE ADD` statements on Firebird now check `RDB$RELATION_FIELDS` before executing. If the column exists, the migration logs "already applied" and moves on. Other databases and statement types are unaffected.

---

v3.6.x - Architectural Parity (March 25, 2026)

Features

src/orm/ as primary model directory (v3.6.0)

Models now live in `src/orm/` by default, matching the convention across all Tina4 frameworks. The framework still scans `src/models/` as a fallback.

Bug Fixes

Outdated API references (v3.6.0)

Internal references to deprecated function names (`createToken` instead of `getToken`, `validateToken` instead of `validToken`) and route parameter syntax were updated.

---

v3.5.x - Bundled Frontend, Middleware (March 25, 2026)

Features

Bundled tina4js.min.js (v3.5.0)

The reactive frontend library ships inside the framework. A 13.6 KB file gives your templates reactive signals, client-side routing, and API calls with zero additional installs.

session.clear() (v3.5.0)

Wipe all session data without destroying the session itself. The session ID and cookie persist.

```
// clear() removes data but keeps the session alive
request.session.clear();

// destroy() ends the session entirely
request.session.destroy();
```

Standardized middleware classes (v3.5.0)

Middleware follows a naming convention: `before*` classes run before the handler, `after*` classes run after. Three built-in middleware classes ship with the framework.

---

v3.4.x - Database, Auth, WebSocket, Uploads (March 24, 2026)

The v3.4 line added production-grade features across several subsystems.

Features

Database class wrapper (v3.4.0)

A constructor-based pattern for database connections, replacing bare function calls:

```
import { Database } from "tina4-nodejs";

const db = new Database("sqlite://data.db");
const result = await db.fetch("SELECT * FROM users WHERE active = ?", [true]);
```

DatabaseResult with columnInfo() (v3.4.0)

Query results return a `DatabaseResult` object. Call `columnInfo()` to inspect column names, types, and sizes without a separate schema query.

```
const result = await db.fetch("SELECT * FROM users LIMIT 1");
const columns = result.columnInfo();
// [{ name: "id", type: "INTEGER" }, { name: "email", type: "VARCHAR" }, ...]
```

Auth class wrapper (v3.4.0)

Authentication functions grouped into a class with `getToken()` and `validToken()` as the primary API. The old names `createToken` and `validateToken` remain as aliases.

```
import { Auth } from "tina4-nodejs";

const token = Auth.getToken({ userId: 42, role: "admin" });
const payload = Auth.validToken(token);
```

Redis session handler (v3.4.0)

Sessions can now persist to Redis for multi-server deployments. Set `TINA4_SESSION_HANDLER=redis` in your `.env` file.

Path-scoped WebSocket broadcast (v3.4.0)

Broadcast messages to WebSocket clients subscribed to a specific path:

```
Router.websocket("/chat/{room}", (connection, request) => {
```

```
  connection.onMessage((message) => {
    connection.broadcast(message); // Only reaches clients on the same path
  });
});
```

File uploads with raw Buffer and data_uri (v3.4.0)

Uploaded files include a raw `Buffer` and a `data_uri` template filter for embedding images directly in HTML.

Breaking Changes

WebSocket handler signature changed to 3 arguments (v3.4.0)

WebSocket handlers now receive `(connection, request, params)` instead of `(connection, request)`.

```
// BEFORE (v3.3.x)
Router.websocket("/ws", (connection, request) => {
  // No access to path params
});

// AFTER (v3.4.x)
Router.websocket("/ws/{room}", (connection, request, params) => {
  const room = params.room;
});
```

Auth function rename (v3.4.0)

`getToken()` and `validToken()` are now the primary function names. The old names `createToken` and `validateToken` continue to work as aliases but are deprecated.

```
// BEFORE (v3.3.x)
const token = Auth.createToken({ userId: 42 });
const valid = Auth.validateToken(token);

// AFTER (v3.4.0)
const token = Auth.getToken({ userId: 42 });
const valid = Auth.validToken(token);
```

Queue job file extension (v3.4.0)

Queue jobs on the file backend use `.queue-data` instead of `.json`. Existing `.json` job files need renaming or the queue treats them as new.

File upload format (v3.4.0)

Upload objects now use `{ type, content }` where `content` is base64-encoded. Legacy property names (`filename`, `data`) remain as aliases.

---

v3.3.x - Queue API, Field Mapping, Route Chaining (March 24, 2026)

Features

Queue API (v3.3.0)

Produce jobs, consume them with an async generator, and manage their lifecycle:

```
import { produce, consume, Job } from "tina4-nodejs";

// Produce a job
await produce("email-queue", { to: "user@example.com", subject: "Welcome" });

// Consume jobs
for await (const job of consume("email-queue")) {
  try {
    await sendEmail(job.data);
    job.complete();
  } catch (error) {
    job.fail(error.message);
  }
}
```

Switch between SQLite, RabbitMQ, Kafka, and MongoDB backends with a single `.env` variable: `TINA4_QUEUE_BACKEND=rabbitmq`.

ORM fieldMapping (v3.3.0)

Map JavaScript property names to database column names explicitly:

```
class User extends BaseModel {
  static fieldMapping = {
    firstName: "first_name",
    lastName: "last_name",
  };
}
```

Route chaining with .secure() and .cache() (v3.3.0)

Routes return a `RouteRef` that supports chainable modifiers:

```
Router.get("/api/products", handler)
  .secure()
  .cache(300); // Cache for 5 minutes
```

MongoDB queue backend (v3.3.0)

The queue system gained MongoDB as a backend. Set `TINA4_QUEUE_BACKEND=mongodb` in your `.env` file.

Database session handler (v3.3.0)

Sessions can persist to any supported database. Set `TINA4_SESSION_HANDLER=database`.

Dev admin improvements (v3.3.0)

Routes in the dev admin panel are now clickable links that open in a new tab. The error overlay shows full request details.

---

v3.2.x - Flexible Route Handlers (March 22, 2026)

Features

Zero-param and single-param route handlers (v3.2.0)

Route handlers accept multiple signatures. The framework inspects the function's parameter names and injects the right objects.

```
// All of these work:
Router.get("/health", () => ({ status: "ok" }));
Router.get("/health", (response) => response.json({ status: "ok" }));
Router.get("/health", (request, response) => response.json({ status: "ok" }));
```

Name your single parameter `request` or `req` and the framework passes the request object. Name it anything else and it receives the response.

---

v3.1.x - Response Parity, Routing API (March 21-22, 2026)

Features

Explicit routing methods (v3.1.0)

`Router.get()`, `Router.post()`, `Router.put()`, `Router.delete()`, and `Router.websocket()` replaced generic registration. Each method reads like what it does.

```
import { Router } from "tina4-nodejs";

Router.get("/users", listUsers);
Router.post("/users", createUser);
Router.put("/users/{id:int}", updateUser);
Router.delete("/users/{id:int}", deleteUser);
```

response.file() and response.render() (v3.1.0)

Two new response methods for serving files and rendering templates:

```
Router.get("/download", async (request, response) => {
  response.file("reports/quarterly.pdf");
});

Router.get("/dashboard", async (request, response) => {
  response.render("dashboard", { user: currentUser });
});
```

FetchResult with toPaginate() (v3.1.0)

Database queries return a `FetchResult` object with built-in pagination:

```
const result = await db.fetch("SELECT * FROM products", [], 20, 0);
const paginated = result.toPaginate();
```

```
// { data: [...], total: 156, page: 1, perPage: 20, totalPages: 8 }
```

ORM relationships (v3.1.0)

`hasMany`, `hasOne`, and `belongsTo` with eager loading:

```
class User extends BaseModel {
  static relationships = {
    posts: { type: "hasMany", model: "Post", foreignKey: "user_id" },
    profile: { type: "hasOne", model: "Profile", foreignKey: "user_id" },
  };
}

const user = await User.find(1, { include: ["posts", "profile"] });
```

Unified Cache, Messenger, and Queue (v3.1.0)

Switch between memory, Redis, and file-based caching with a single environment variable. The messenger and queue systems follow the same pattern. No code changes needed.

```
# .env

## v3.10.70 (2026-04-06)

- **New:** SSE (Server-Sent Events) support via `response.stream()` - pass a generator, framewor
- **New:** Chapter 24 added to documentation: Server-Sent Events
- Feature count: 45 (was 44)
- Full parity across Python, PHP, Ruby, Node.js

TINA4_CACHE_BACKEND=redis
TINA4_QUEUE_BACKEND=sqlite
```

tina4 generate command (v3.1.0)

Scaffold models, routes, migrations, and middleware from the command line:

```
tina4 generate model User
tina4 generate route api/products
tina4 generate migration add_email_to_users
tina4 generate middleware AuthCheck
```

Frond pre-compilation (v3.1.0)

The template engine caches compiled tokens. File rendering runs 2.8x faster than v3.0.0.

---

v3.0.0 - Initial Release (March 21, 2026)

The initial Node.js release. No Express. No Fastify. No dependencies.

Features

- **Native node:http** - The server uses Node's built-in HTTP module. Zero framework overhead.
- **TypeScript-first** - Strict mode, ESM only. No separate build step.
- **Database adapters** - SQLite, PostgreSQL, MySQL, MSSQL, and Firebird. Same API across all five.

- **File-based routing** - `src/routes/api/users/[id]/get.ts` maps to `GET /api/users/:id`.
- **Auto-CRUD** - Generate full REST endpoints from a model definition.
- **DevAdmin dashboard** - A built-in developer panel with route inspection and database tools.
- **AI integration** - Auto-detect and configure context for seven AI coding tools.
- **1,311 tests** across 43 test files.
- **Configurable port and host** - Default port 7148, binds to 0.0.0.0 for Docker.

```
import { startServer } from "tina4-nodejs";

startServer({ port: 7148 });
```

One import. One function call. The server starts and your application is live.

Pre-Release (rc.2-rc.5)

Four release candidates preceded v3.0.0. They stabilized the scaffolding, fixed the init command, added the error overlay, refined the landing page, and established the benchmark suite. If you started a project on a release candidate, upgrade to v3.0.0 and run `tina4 init` to regenerate your scaffolding files.

Version Timeline

Version	Date	Headline
v3.0.0	March 21	Initial release - zero dependencies, TypeScript-first
v3.1.0	March 21	Response parity, ORM relationships, unified cache/queue
v3.2.0	March 22	Flexible route handler signatures
v3.3.0	March 24	Queue API, field mapping, route chaining
v3.4.0	March 24	Database class, auth wrapper, Redis sessions, WebSocket broadcast
v3.5.0	March 25	Bundled frontend, standardized middleware
v3.6.0	March 25	src/orm/ as primary model directory
v3.7.0	March 25	Template auto-serve, Firebird idempotent migrations
v3.8.0	March 25	Typed route params, template fallback, Frond replaces Twig
v3.9.0	March 26	QueryBuilder, path injection, auto-start sessions
v3.10.0	March 28	Cached Frond instances, autoMap, ORM transactions, template fixes
v3.10.32	March 31	MCP server, arithmetic expressions, current stable

Forty-two releases in eleven days. Each one a step closer to the framework the code deserves.

Feature Catalog

Tina4 3.14 has **140 numbered catalog entries**. The number describes the framework family's implementation and audit inventory. It does not mean that 140 features have reached four-language parity, and it does not mean that every entry lives inside each backend package.

This chapter is the map. The earlier chapters explain the public APIs available to Node.js applications. The numbered audit packets define the parity work and the clean-room formula for another language.

Read the status correctly

A feature moves through four distinct states:

- **Catalogued** means the feature has a stable number, name, owner, and audit packet.
- **Shipped** means code exists in one or more released components.
- **Audited** means the team has measured all relevant implementations and recorded contradictions and decisions.
- **Contract-proven** means a shared fixture passes in every applicable implementation with real dependencies.

Do not collapse those states into one green tick. At version 3.13.101, the contract ledger reports **55 fixtures and 282 proven invariants, with 0 owed and 0 broken inside those fixture-covered contracts**. That result proves the named contracts. It does not certify the remaining catalog entries.

Where the features live

Range	Owner	Meaning
1-109, 126-132, 135-140	Backend frameworks	Runtime, developer, testing, and application-facing capabilities. Availability and maturity still require per-feature evidence.
110-125 and 134	Shared Rust CLI	One language-neutral client used with all four backends. These are not four separate backend implementations.
133	Verification contract	Carbonah benchmark and report shape, not an application runtime API.
Provider entries	Selected integration	Each selectable provider has its own number because another language must implement and test it separately. Providers can require an external service, driver, language extension, or extra package.

Feature 63 includes browser helpers and the separate tina4-js frontend package. The backend books explain their integration, but tina4-js is not embedded four times.

Current parity boundary

Evidence	Python	PHP	Ruby	Node.js
Catalog membership	Inventory source	Audit in progress	Audit in progress	Audit in progress
Fixture-covered contracts	Proven	Proven	Proven	Proven
Entire 140-entry catalog	Not yet proven	Not yet proven	Not yet proven	Not yet proven

The initial catalog followed the Python module inventory, but Python is not a permanent master. Tina4 promotes the best implementation after audit and captures that decision in an ADR and shared fixture.

Known missing feature surfaces remain tracked in PHP issue 185 and Node.js issue 37. An empty issue list in another repository is not parity evidence. The fixture ledger is the evidence.

Dependency boundary

The core, Frond, and Swagger packages declare no required third-party packages. The root and ORM packages declare optional driver or SDK packages for selected providers. The CLI depends on Tina4 workspace packages, which are part of the Tina4 distribution. Node.js 22 or newer supplies `node:sqlite` for the default database path.

"Core dependency" and "provider dependency" are different facts. A local SQLite application can stay small while PostgreSQL, RabbitMQ, Kafka, Redis, MongoDB, S3, or hosted AI still requires the driver or service that speaks that protocol.

Numbered catalog

Foundation

#	Catalog entry
1	DotEnv and typed environment
2	Structured logger

Database and providers

#	Catalog entry
3	Database adapter interface
4	Database URL parser
5	Database facade and safe writes
6	Query builder
7	SQL translator
8	SQLite provider
9	PostgreSQL provider
10	MySQL provider
11	MSSQL provider
12	Firebird provider
13	ODBC provider
14	MongoDB SQL-translation provider
15	Migrations
16	Race-safe database sequences

ORM and data layer

#	Catalog entry
17	ORM base class
18	ORM fields and column mapping
19	Input and request validation
20	Soft delete
21	Declarative ORM relationships
22	Imperative ORM relationships
23	ORM scopes
24	Paginated database and ORM results
25	ORM result caching
26	ORM instance loading
27	Automatic CRUD from models
28	Seeder and fake data

HTTP core

#	Catalog entry
29	HTTP request model
30	HTTP response model and representation types
31	Router and dispatch
32	Route groups
33	Middleware pipeline

HTTP policies

#	Catalog entry
34	CORS middleware
35	Rate limiting
36	Security headers middleware
37	CSRF protection

HTTP runtime

#	Catalog entry
38	Health and readiness endpoints
39	Graceful shutdown
40	HTTP compression and ETag
41	Static assets and cache revalidation
42	Configurable error pages
43	Request ID tracking
44	File upload contract
45	Swagger and OpenAPI
46	Default landing page
47	In-process background tasks

Frond template engine

#	Catalog entry
48	Frond lexer
49	Frond parser
50	Frond compiler
51	Frond runtime
52	Frond filters
53	Frond tags
54	Frond expression tests
55	Frond functions
56	Frond extensibility API
57	Frond auto-escaping
58	Frond sandboxing
59	Frond template caching
60	Frond fragment caching

Frontend assets

#	Catalog entry
61	SCSS compiler
62	Tina4 CSS
63	Frond and Tina4 browser helpers

Authentication

#	Catalog entry
64	JWT and request authentication

Sessions and providers

#	Catalog entry
65	Session lifecycle
66	File session provider
67	Redis session provider
68	Valkey session provider
69	MongoDB session provider
70	Database session provider
71	Memcached session provider

Cache and providers

#	Catalog entry
72	Cache interface and provider selection
73	Memory cache provider
74	File cache provider
75	Redis cache provider
76	Valkey cache provider
77	Memcached cache provider
78	MongoDB cache provider
79	Database cache provider
80	HTTP response cache

Integrations and storage

#	Catalog entry
81	Standard-library HTTP API client
82	GraphQL
83	WebSocket protocol and server
84	Redis WebSocket backplane
85	NATS WebSocket backplane
86	WSDL and SOAP
87	Localization and i18n
88	Email and messenger
89	Queue lifecycle
90	Lite queue provider
91	RabbitMQ queue provider
92	Kafka queue provider
93	MongoDB queue provider
94	MQTT client
95	Document store interface
96	SQLite document store provider
97	MongoDB document store provider
98	Realtime collaboration
99	Local realtime attachment storage
100	S3 realtime attachment storage
101	Model Context Protocol server
102	Local source and documentation context index
103	Live framework and application API index
135	App-facing LLM client
137	GIS spatial points and queries
138	RBAC (role/permission guards)

139	Graph databases (Ultipa, Neo4j, Memgraph, ArangoDB)
140	Provider-neutral Web Push notifications

Enterprise authentication

#	Catalog entry
136	Configuration-first OpenID Connect SSO

Application runtime

#	Catalog entry
104	Event and listener system
105	Dependency injection container
106	Service runner
107	HTML element builder
108	AI coding-tool integration
109	Lazy feature loading and preload manifest

CLI

#	Catalog entry
110	CLI project initialization
111	CLI development server
112	CLI migrations
113	CLI seeding
114	CLI test runner
115	CLI route inspection
116	CLI interactive console
117	CLI environment management
118	CLI queue management
119	CLI container build
120	CLI code generation
121	CLI code metrics
122	CLI command discovery and help
123	CLI doctor
124	CLI guided setup
125	CLI deployment
134	CLI AI-skills installation and refresh

Developer runtime

#	Catalog entry
126	Development error overlay
127	Development admin dashboard
128	Dual development and test ports
129	Development port takeover
130	Dynamic framework version

Testing and verification tools

#	Catalog entry
131	In-process HTTP test client
132	Inline testing API
133	Carbonah benchmark contract

What parity means

Parity applies to observable contracts: response shapes, status codes, environment keys, error messages, file formats, and security rules. Host-language code remains idiomatic. A Python method can use snake_case while PHP and Node.js use camelCase, but all three must return the same public data.

The catalog gives another language a checklist. The audit packet supplies the decisions. The fixture supplies the proof. A name alone proves nothing; a green contract earns the claim.

Real-time Collaboration (WebRTC)

1. The Media Server You Do Not Have to Run

You want a call button. Two people click it and they are talking: audio, video, a shared screen. Next to the call sits a chat panel and a place to drop a file. This is the Slack shape, the Teams shape, and the usual advice is heavy. Stand up a media server. Wire in TURN. Operate all of it.

You do not need any of that to start. Modern browsers already speak WebRTC to each other directly, peer to peer. What they cannot do is *find* each other. One browser has to hand its connection offer to the other and get an answer back. That hand-off is called **signalling**, and it is a tiny amount of text relayed through a server. The media itself, the actual audio and video, never touches your server. It flows browser to browser.

Tina4's realtime module is that relay, plus the two things every collaboration tool needs around a call: persistent **chat** and permissioned **file** transfer. It shipped in 3.13.57, carries zero core dependencies, and mounts with one line. Media is peer to peer, a **mesh**. Tina4 carries no media and never parses your SDP. It forwards the handshake and nothing more.

```
import { realtime } from "tina4-nodejs/orm";

await realtime({ features: ["calls", "chat", "files"] });
```

Mesh is peer to peer: every participant connects to every other participant. Perfect for 1:1 and small-group sessions. A large room wants an SFU, and that is not what ships here (section 11).

---

2. What You Get, and How It Differs from Raw WebSocket

The WebSocket chapter gave you the raw primitive: `Router.websocket(path, handler)`, a `(connection, event, data)` handler, a room manager. That is the tool you reach for when you build your *own* protocol. The realtime module is the opposite end. It is a *pre-built* protocol for calls, chat, and files, assembled from that same primitive underneath.

The module gives you three surfaces, and you enable only the ones you want:

- **calls** - a WebRTC signalling relay (mesh, peer to peer) plus a self-describing ICE-config endpoint. The relay forwards offer, answer, and ICE frames between peers in a room. It is public: no login to join a room.
- **chat** - persistent channels and messages backed by framework-owned ORM models, a secured chat WebSocket with live presence, typing, and read receipts, and a history endpoint for catch-up on reconnect.
- **files** - permissioned upload and download through a pluggable storage backend (local filesystem by default, S3-compatible optional).

The distinction in one line: with `Router.websocket` you write the handler and invent the message protocol; with `realtime()` the protocol is written for you and the browser discovers every path from one config endpoint. Under the hood the realtime WebSockets *are* Tina4 WebSockets. Same `(connection, event, data)` handler convention, same room manager.

You are handed the handlers pre-written.

The whole surface exports from the ORM package. Import what you need from `tina4-nodejs/orm`:

```
import {
  realtime,           // the mount
  iceServers,         // build the ICE/TURN list from env
  selectStorage,      // resolve a storage backend
  storageKey,         // opaque, collision-free file keys
  LocalStorage,       // default filesystem store
  S3Storage,          // opt-in S3-compatible store
  type RealtimeOptions,
  type StorageBackend,
  // framework-owned ORM models (Realtime-prefixed so they never collide with yours)
  RealtimeWorkspace,
  RealtimeChannel,
  RealtimeChannelMember,
  RealtimeMessage,
  RealtimeAttachment,
} from "tina4-nodejs/orm";
```

Do the backend here. Do the browser side with the plain browser Web APIs shown in section 10, or with the tina4-js `rtc` module. The examples in this chapter use plain browser APIs so they run anywhere.

---

3. Mounting the Module: `realtime()`

Call `realtime()` once in `app.ts`, **before** `startServer()`. It creates the chat tables (when needed), registers the routes, and returns the resolved path map, the same map the config endpoint serves. So you can log it or assert against it.

```
// app.ts
import { startServer } from "tina4-nodejs";
import { initDatabase, realtime } from "tina4-nodejs/orm";

await initDatabase(process.env.TINA4_DATABASE_URL!);   // bind the DB FIRST (section 11)

await realtime();                                          // calls only (default)
await realtime({ features: ["calls", "chat"] });           // add persistent chat
await realtime({ prefix: "/api/collab", features: ["calls", "chat", "files"] });

startServer();
```

`realtime()` is `async` in Node, so always `await` it. Route registration itself is synchronous, but Node's ORM creates the `tina4_rt_*` tables asynchronously, so the mount returns a `Promise`. Awaiting it guarantees the tables exist before the first chat, history, or file request lands. The Python master's `realtime()` is synchronous. Node's is not, and this is a Node-specific gotcha.

The signature is an options object:

```
async function realtime(options?: RealtimeOptions): Promise<Record<string, string>>

interface RealtimeOptions {
```

```
  prefix?: string;
  authorize?: (identity: string, channelId: number) => boolean | Promise<boolean>;
  storage?: StorageBackend;
  features?: string[];
}
```

	Meaning	
`x`	Mounts the whole surface under `/<prefix>` (default: root). Leading and trailing slashes are stripped, so `"/api/collab/"` becomes `/api/collab`.	
`rize`	Channel-membership guard, `(identity, channelId) => boolean \`	Promise<boolean> `(sync or as` `Used by` chat `and` files`. Defaults` RealtimeChannelMember `check.` i is the **string** user id from the JWT.
`ge`	A `StorageBackend` for the `files` feature. Defaults to the env-selected store (`local`).	
`res`	Array of `"calls"`, `"chat"`, `"files"`. Defaults to `["calls"]`.	

There is no `media` option. The Node port is mesh only (section 11).

The returned map holds the **base** paths. The config endpoint body adds the `{room}` / `{channel}` / `{id}` template tokens the client fills in.

```
await realtime();
// -> { backend: "mesh", config: "/api/rtc/config", signalling: "/ws/rtc" }

await realtime({ features: ["calls", "chat"] });
// -> { backend, config, signalling: "/ws/rtc", chat: "/ws/chat", messages: "/api/channels" }

await realtime({ features: ["files"] });
// -> { backend, config, files: "/api/files" }
```

`config` is added by any enabled feature. `calls` sets it outright; `chat` and `files` add it with `??=`. So even a chat-only or files-only mount still exposes `/api/rtc/config`.

What each feature wires

Feature	Route registered	Auth
any	`GET {p}/api/rtc/config`	**public**, no `.secure()`
`calls`	`WS {p}/ws/rtc/{room}`	**public**, unauthenticated
`chat`	`WS {p}/ws/chat/{channel}`	**secured**, `Router.websocket(..., { secured: true })`, valid JWT on upgrade
`chat`	`GET {p}/api/channels/{id}/messages`	`.secure()`
`files`	`POST {p}/api/files`	auth-required (Tina4 secures write routes by default)
`files`	`GET {p}/api/files/{key}`	`.secure()`

When `chat` or `files` is enabled, the framework runs `ensureChatTables()` at mount time to create the `tina4_rt_*` tables. A missing database logs an error but does not crash boot (section 11).

---

4. The Config Bootstrap: `GET {p}/api/rtc/config`

The client never hardcodes a URL. It fetches this one public endpoint, and everything else comes back in the response: the signalling path, the ICE servers, the chat, messages, and files paths. Move `prefix` on the server and the client follows automatically.

The body is feature-gated. Only keys for enabled features appear, and this is where the template tokens live.

```
{
  "backend": "mesh",
  "iceServers": [ /* result of iceServers() */ ],   // calls
  "signalling": "/ws/rtc/{room}",                    // calls
  "chat": "/ws/chat/{channel}",                      // chat
  "messages": "/api/channels/{id}/messages",         // chat
  "files": "/api/files"                              // files
}
```

The `{room}`, `{channel}`, and `{id}` are literal template tokens. The client substitutes the real room name, channel id, or message id before connecting.

```
const cfg = await fetch("/api/rtc/config").then(r => r.json());

const pc = new RTCPeerConnection({ iceServers: cfg.iceServers });
const signalling = new WebSocket(
  location.origin.replace(/^http/, "ws") + cfg.signalling.replace("{room}", roomId)
);
```

This endpoint is public, and it returns your ICE and TURN configuration including freshly-minted ephemeral TURN credentials (section 5). That is intentional. The browser needs those before it can authenticate anywhere, which is exactly why the credentials are short-lived by design. Be aware anyone can read it.

---

5. ICE and TURN Servers

Before two browsers connect, each gathers candidate addresses using **ICE**. A **STUN** server tells a browser its public address. A **TURN** server relays media when a direct path is impossible: strict NAT, symmetric firewalls. The module builds this list from environment variables via `iceServers()`, which is exported so you can inspect or reuse it.

`iceServers()` always includes a STUN entry. It adds a TURN entry only when both `TINA4_RTC_TURN_URL` and `TINA4_RTC_TURN_SECRET` are set, using coturn's `use-auth-secret` scheme with time-limited credentials:

```
username   = String(Math.floor(Date.now() / 1000) + ttl)
credential = base64( HMAC_SHA1(secret, username) )
```

built with Node's `node:crypto createHmac`. The credential expires after `ttl` seconds, so a leaked config from `/api/rtc/config` is only briefly useful.

```
// no TURN env set - STUN only:
[ { "urls": ["stun:stun.l.google.com:19302"] } ]

// TINA4_RTC_TURN_URL + TINA4_RTC_TURN_SECRET set:
[ { "urls": ["stun:stun.l.google.com:19302"] },
  { "urls": ["turn:turn.example.com:3478"], "username": "1783546725", "credential": "ie7Mm...=="
```

Environment variables

```
# .env
TINA4_RTC_STUN_URLS=stun:stun.l.google.com:19302
TINA4_RTC_TURN_URL=turn:turn.example.com:3478   # enables TURN when paired with the secret
TINA4_RTC_TURN_SECRET=your-coturn-shared-secret
TINA4_RTC_TURN_TTL=3600                          # ephemeral credential lifetime (seconds)
```

Variable	Default	Effect
`TINA4_RTC_STUN_URLS`	`stun:stun.l.google.com:19302`	Comma-separated STUN URLs.
`TINA4_RTC_TURN_URL`	-	Comma-separated TURN URLs; enables TURN when set together with the secret.
`TINA4_RTC_TURN_SECRET`	-	coturn `use-auth-secret` shared secret (drives the ephemeral credentials).
`TINA4_RTC_TURN_TTL`	`3600`	Ephemeral TURN credential lifetime, in seconds.

`TINA4_RTC_BACKEND` is **not read** in Node. The backend is always `mesh` (section 11).

---

6. Signalling: The Mesh Relay

The `calls` feature registers the signalling handler at `WS {p}/ws/rtc/{room}`, and it is public. Anyone can join any room. It follows the framework's WebSocket handler convention:

```
(connection, event, data) => {
  // connection : the WebSocketConnection
  // event      : "open" | "message" | "close"
  // data       : the string frame on "message"
};
```

The mesh relay behaviour is small and deliberate:

- `room = connection.params.room ?? ""`. An empty room is a no-op, and the handler returns.
- On `event === "open"` it calls `connection.joinRoom("rtc:" + room)`.
- On `event === "message"` it calls `connection.broadcastToRoom("rtc:" + room, data, true)`, relaying the **raw** frame to the other peers (`excludeSelf = true`).

Tina4 never inspects the SDP. Peers put a `to` field in their own frames and filter for themselves. Rooms are namespaced `rtc:<room>` so signalling rooms never collide with chat channels (`chat:<id>`), which share the same WebSocket manager.

The `WebSocketConnection` surface the relay uses is camelCase in Node:

```
connection.params                                  // { room: "..." }
connection.auth                                    // verified JWT payload, or null on a public
connection.joinRoom(name)
connection.broadcastToRoom(name, message, excludeSelf)
connection.getRoomConnections(key)                 // live connections in a room (for presence)
connection.sendJson(obj)
connection.close()
```

A minimal browser peer, framework-agnostic:

```
const ws = new WebSocket(signallingUrl);           // ".../ws/rtc/room-42"
const pc = new RTCPeerConnection({ iceServers: cfg.iceServers });

pc.onicecandidate = (e) => {
  if (e.candidate) ws.send(JSON.stringify({ to: peerId, type: "ice", candidate: e.candidate }));
};

ws.onmessage = async (ev) => {
  const msg = JSON.parse(ev.data);
  if (msg.to !== myId) return;                     // Tina4 relays to everyone; filter yourself
  if (msg.type === "offer") {
    await pc.setRemoteDescription(msg.sdp);
    const answer = await pc.createAnswer();
    await pc.setLocalDescription(answer);
    ws.send(JSON.stringify({ to: msg.from, type: "answer", sdp: answer }));
  } else if (msg.type === "answer") {
    await pc.setRemoteDescription(msg.sdp);
  } else if (msg.type === "ice") {
```

```
    await pc.addIceCandidate(msg.candidate);
  }
};
```

Because signalling is public, the room name is your only barrier. If a call must be private, gate access at your app layer: issue unguessable room ids, or check membership before you hand the client a room name.

For a private call, do not mount the built-in `calls` feature and assume its `authorize` callback protects signalling: that callback applies only to chat and files. Register an application-owned signalling route with `Router.websocket(path, handler, { secured: true })`, then verify the authenticated identity in `connection.auth` belongs to the requested room on open and on every relayed frame. Browser clients carry the JWT with `new WebSocket(url, ["bearer", token])`.

The current `tina4-js` `rtc.call()` helper opens the built-in public signalling socket and has no signalling-token option. Use a small application WebSocket client for the protected route (or add token support to that client API in a separately versioned change). Unguessable room names reduce discovery; they are not authorization.

---

7. Chat: Secured Channels, Presence, History

Chat is the opposite of signalling: secured. It registers as `Router.websocket(path, chatHandler, { secured: true })`, so a **valid JWT is required on the WebSocket upgrade**. The router rejects an unauthenticated upgrade (401) before your handler ever runs.

- Channels are addressed by **integer id**: `connection.params.channel` must match `/^\d+$/`. A non-integer `{channel}` makes the handler return silently. The socket opens and does nothing (section 11).
- Identity comes from the verified token: `connection.auth` (section 9).
- The room key is `chat:<channelId>`.

Every inbound frame is JSON; every broadcast is a `JSON.stringify(...)` string. The handler is `async`, since it awaits the membership check and message persistence on each frame.

Event / message	Server behaviour
`open`	Authorize. **Fail:** send `{ type: "error", error: "not a member of this channel" }` then `close()`. **OK:** `joinRoom`, send the caller the roster `{ type: "presence", event: "roster", users: [...] }`, then broadcast `{ type: "presence", event: "join", user_id }` (exclude self).
`close`	Broadcast `{ type: "presence", event: "leave", user_id }` (exclude self).

message `typing`	Broadcast `{ type: "typing", user_id }` (exclude self).
message `read`	Advance the member's read cursor (`last_read_at = now`), broadcast `{ type: "read", user_id, at: <iso> }` (exclude self).
message `message`	Trim `body`; if empty, **ignore**. Persist a `RealtimeMessage` row; on success broadcast `{ type: "message", message: <saved> }` to **everyone including the sender**, so the sender's optimistic message reconciles with its server `id` and `created_at`.

`type` defaults to `"message"` when absent. Unknown types are ignored. The roster is the sorted set of distinct identities currently in the room, read from each live connection's `auth`. Authorization is re-checked on **every inbound frame**, not just on join. Membership can be revoked mid-session, and the server never trusts an identity carried in the payload.

The saved-message JSON shape, also what history returns:

```
{ "id": 12, "channel_id": 3, "user_id": "7", "body": "hi",
  "thread_id": null, "created_at": "2026-07-08T10:00:00Z" }
```

`thread_id` is `null` for a top-level message, or the parent message's id for a threaded reply.

A minimal browser chat client:

```
const chat = new WebSocket(chatUrl + "?token=" + jwt);   // upgrade must carry a valid JWT

chat.onmessage = (ev) => {
  const m = JSON.parse(ev.data);
  switch (m.type) {
    case "presence": /* m.event: "roster" | "join" | "leave" */ break;
    case "typing":   showTyping(m.user_id); break;
    case "read":     advanceReadReceipt(m.user_id, m.at); break;
    case "message":  appendMessage(m.message); break;   // includes the server id + created_at
    case "error":    console.warn(m.error); break;
  }
};

// send a message
chat.send(JSON.stringify({ type: "message", body: "hello" }));
// or a threaded reply
chat.send(JSON.stringify({ type: "message", body: "re:", thread_id: 12 }));
// typing indicator / read receipt
chat.send(JSON.stringify({ type: "typing" }));
chat.send(JSON.stringify({ type: "read" }));
```

Chat history: `GET {p}/api/channels/{id}/messages`

The catch-up-on-reconnect endpoint, marked `.secure()`.

- Identity comes from `req.user`, the verified JWT payload the router attached on the secured route.
- Invalid channel id returns `400 { "error": "invalid channel id" }`; not authorized returns `403 { "error": "forbidden" }`.
- Query params: `before` (return messages with `id < before`) and `limit` (default **50**, clamped to **1-200**).
- Returns messages **newest-first**, the standard infinite-scroll-backwards shape. Each item uses the saved-message JSON shape above.

```
curl -H "Authorization: Bearer $JWT" \
  "http://localhost:7148/api/channels/3/messages?limit=50"

# older page: pass the smallest id you already have as `before`
curl -H "Authorization: Bearer $JWT" \
  "http://localhost:7148/api/channels/3/messages?before=12&limit=50"
```

---

8. Files: Upload and Download

Enable by adding `"files"` to `features`. Uploads flow through a `StorageBackend`: the `storage` option, or the env-selected store (default `LocalStorage`).

`POST {p}/api/files` - upload (auth-required)

- Multipart: a file field named `file` (`req.files.file`), plus a form field `channel_id` (required integer, read from body, query, or params).
- Missing or invalid `channel_id` returns `400 { "error": "channel_id is required" }`; not a channel member returns `403 { "error": "forbidden" }`; no file returns `400 { "error": "no file uploaded (field 'file')" }`.
- Stores the blob under an opaque, collision-free `storageKey` (16 random bytes hex plus a sanitized extension, **never** a user-controlled path), inserts a `RealtimeAttachment` row (metadata only), and responds `201`:

```
{ "id": 4, "key": "<storageKey>", "filename": "report.pdf", "mime": "application/pdf",
  "size": 20481, "url": "<direct url OR {files}/{key}>" }
```

`url` is `store.url(key)` when the backend exposes a direct URL (for example an S3 presigned link), otherwise the app download route `{files}/{key}`.

```
const fd = new FormData();
fd.append("file", fileInput.files[0]);
fd.append("channel_id", "3");

const att = await fetch("/api/files", {
  method: "POST",
  headers: { Authorization: `Bearer ${jwt}` },      // POST is auth-required
  body: fd,
}).then(r => r.json());
// att.url is either a direct (presigned) URL or /api/files/<key>
```

`GET {p}/api/files/{key}` - download (`.secure()`)

- Looks up the `RealtimeAttachment` by `storage_key`; missing returns `404 { "error": "not found" }`.
- Authorizes against the attachment's `channel_id`; a non-member gets `403`.
- If the backend has a direct URL, it returns a `302` redirect (`res.redirect(url, 302)`). Otherwise it **streams the bytes** (`200`) with `Content-Disposition: inline; filename="..."` and the attachment's `mime` (default `application/octet-stream`). Missing bytes return `404`.

Storage backends

`selectStorage(storage?)` resolves from the `storage` argument or `TINA4_STORAGE_BACKEND` (`local` default, or `s3`). An `s3` backend that cannot be built (the `@aws-sdk/client-s3` driver missing, or config incomplete) **falls back to** `LocalStorage` with a warning: a real store, never a silent no-op.

Node uses `@aws-sdk/client-s3` (plus `@aws-sdk/s3-request-presigner`), not boto3. They are optional peer dependencies, loaded lazily. Install them only when you set `TINA4_STORAGE_BACKEND=s3`.

```
# .env - local (default)
TINA4_STORAGE_BACKEND=local
TINA4_STORAGE_DIR=data/rt_storage

# .env - S3-compatible (MinIO, AWS, ...)
TINA4_STORAGE_BACKEND=s3
TINA4_STORAGE_URL=https://s3.example.com
TINA4_STORAGE_KEY=...
TINA4_STORAGE_SECRET=...
TINA4_STORAGE_BUCKET=my-bucket
TINA4_STORAGE_REGION=us-east-1
```

Variable	Default	Effect
`TINA4_STORAGE_BACKEND`	`local`	`local` or `s3`.
`TINA4_STORAGE_DIR`	`data/rt_storage`	Local filesystem directory.
`TINA4_STORAGE_URL`	-	S3 endpoint URL (S3-compatible / MinIO); `forcePathStyle: true`.
`TINA4_STORAGE_KEY` / `TINA4_STORAGE_SECRET`	-	S3 credentials.
`TINA4_STORAGE_BUCKET`	-	S3 bucket. Required for S3; missing means the constructor throws and selection falls back to local.
`TINA4_STORAGE_REGION`	`us-east-1`	S3 region.

`LocalStorage` resolves every key inside its root and rejects path traversal; its `url()` returns `null`, so files serve through the permissioned download route. `S3Storage.url()` returns a presigned GET URL (default TTL 3600s), so clients fetch large blobs straight from object storage and the download becomes a 302 redirect. You can also pass an instance explicitly:

```
import { realtime, S3Storage } from "tina4-nodejs/orm";

await realtime({
  features: ["chat", "files"],
  storage: new S3Storage({ bucket: "collab-files", region: "eu-west-1" }),
});
```

---

9. Auth, Identity, and the Data Model

Identity is always taken from the **verified token**, never from a message payload. Two pieces make membership work: how an identity is extracted, and how membership is checked.

Extracting identity, `identityOf(auth)`. It pulls a stable **string** user id from a verified JWT payload, trying the claims `user_id`, then `sub`, then `id`, and returns `null` if none are present. Because identities round-trip as strings, an integer id, a UUID, or an email all work. WebSocket identity comes from `connection.auth`, the payload the router attached on the secured upgrade. HTTP identity comes from `req.user` inside each handler. This is the Node and PHP convention: the router validates the JWT on the secured or auth-required route and attaches the payload for you.

Checking membership. The secure default requires channel membership: `RealtimeChannelMember.count("channel_id = ? AND user_id = ?", [channelId, identity]) > 0`. Any exception is logged and denies (`false`). A custom `authorize` overrides it, `(identity, channelId) => boolean | Promise<boolean>` (sync or async; a promise is awaited). Use it to open public channels to any authenticated user. It short-circuits to `false` when `identity` is `null`, so an unauthenticated caller is always denied. It runs on every inbound chat frame, so keep it cheap.

```
await realtime({
  features: ["calls", "chat"],
  // any authenticated user may read/write any channel
  authorize: (_identity, _channelId) => true,
});
```

The data model

Framework-owned `BaseModel` classes, all carrying the `tina4_rt_` table prefix so they never collide with your own tables. They are created on demand at mount via `ensureChatTables()`, in dependency order: `Workspace, Channel, ChannelMember, Message, Attachment`.

Model (public alias)	Table	Key fields
`RealtimeWorkspace`	`tina4_rt_workspaces`	`id`, `name`, `created_at`

`RealtimeChannel`	`tina4_rt_channels`	`id`, `workspace_id`, `name`, `kind` (`public` / `private` / `dm`, default `public`), `created_at`
`RealtimeChannelMember`	`tina4_rt_channel_members`	`id`, `channel_id`, `user_id` (string), `role` (default `member`), `last_read_at` (read cursor)
`RealtimeMessage`	`tina4_rt_messages`	`id`, `channel_id`, `user_id` (string), `body` (text), `thread_id` (nullable parent id), `created_at`, `edited_at` (nullable)
`RealtimeAttachment`	`tina4_rt_attachments`	`id`, `channel_id`, `message_id` (nullable), `storage_key`, `filename`, `mime`, `size`, `thumb_key` (nullable)

`user_id` is a **string** everywhere, so any JWT identity shape fits. The mount seeds no data. Create channels and memberships with these models (or your own admin flow) **before** clients connect:

```
import { RealtimeChannel, RealtimeChannelMember } from "tina4-nodejs/orm";

const general = new RealtimeChannel({ workspace_id: 1, name: "general", kind: "public" });
await general.save();

await new RealtimeChannelMember({
  channel_id: (general as { id: number }).id,
  user_id: "7",
  role: "member",
}).save();
```

Now user `7` can open `WS /ws/chat/<channel.id>` and pass the history and file checks.

---

10. A Complete Example

A runnable server with all three features, a seeded channel, and a custom authorize guard.

Backend - `app.ts`

Bind the database, mount the surface before `startServer()`, and seed one channel so chat has somewhere to land.

```
// app.ts
import { startServer } from "tina4-nodejs";
import {
```

```
  initDatabase,
  realtime,
  RealtimeChannel,
  RealtimeChannelMember,
} from "tina4-nodejs/orm";

// 1. Bind a database FIRST - the chat/files tables are created at mount.
await initDatabase(process.env.TINA4_DATABASE_URL ?? "sqlite://./data/app.db");

// 2. Mount the realtime surface (await it - it is async in Node).
const paths = await realtime({
  prefix: "/api/collab",
  features: ["calls", "chat", "files"],
  // Members-only channels: return true here to open every channel instead.
  authorize: async (identity, channelId) =>
    (await RealtimeChannelMember.count(
      "channel_id = ? AND user_id = ?",
      [channelId, identity],
    )) > 0,
});

console.log(paths);
// {
//   backend: "mesh",
//   config:     "/api/collab/api/rtc/config",
//   signalling: "/api/collab/ws/rtc",
//   chat:       "/api/collab/ws/chat",
//   messages:   "/api/collab/api/channels",
//   files:      "/api/collab/api/files"
// }

// 3. Seed a channel + membership so a client can connect.
const existing = await RealtimeChannel.where("name = ?", ["general"], 1);
if (existing.length === 0) {
  const general = new RealtimeChannel({ workspace_id: 1, name: "general", kind: "public" });
  await general.save();
  await new RealtimeChannelMember({
    channel_id: (general as { id: number }).id,
    user_id: "7",
    role: "member",
  }).save();
}

// 4. Boot.
startServer();
```

```
# run it
tina4 serve
```

Browser - bootstrap from the config endpoint

The client fetches `/api/collab/api/rtc/config` first and never hardcodes a path.

```
<script type="module">
  const base = "/api/collab";
  const jwt = localStorage.getItem("token");        // minted by your login route

  // Discover everything from the server - never hardcode paths.
  const cfg = await fetch(`${base}/api/rtc/config`).then(r => r.json());
  const wsOrigin = location.origin.replace(/^http/, "ws");
```

```
  // --- calls: mesh WebRTC signalling (public, no token) ---
  const signalling = new WebSocket(wsOrigin + cfg.signalling.replace("{room}", "room-42"));
  const pc = new RTCPeerConnection({ iceServers: cfg.iceServers });
  const local = await navigator.mediaDevices.getUserMedia({ audio: true, video: true });
  local.getTracks().forEach(t => pc.addTrack(t, local));
  // ...exchange offer/answer/ICE over `signalling`, filtering by a `to` field...

  // --- chat: secured WebSocket (JWT on the upgrade) ---
  const chat = new WebSocket(wsOrigin + cfg.chat.replace("{channel}", "1") + `?token=${jwt}`);
  chat.onmessage = (ev) => console.log(JSON.parse(ev.data));   // presence/typing/read/message
  chat.onopen = () => chat.send(JSON.stringify({ type: "message", body: "hello" }));

  // --- history: catch up on reconnect ---
  const history = await fetch(
    cfg.messages.replace("{id}", "1") + "?limit=50",
    { headers: { Authorization: `Bearer ${jwt}` } },
  ).then(r => r.json());

  // --- files: upload to the channel ---
  async function upload(file) {
    const fd = new FormData();
    fd.append("file", file);
    fd.append("channel_id", "1");
    return fetch(cfg.files, {
      method: "POST",
      headers: { Authorization: `Bearer ${jwt}` },
      body: fd,
    }).then(r => r.json());   // -> { id, key, filename, mime, size, url }
  }
</script>
```

The signalling socket is public: anyone with the room name joins. The chat socket and the history and file endpoints require a valid JWT and channel membership. Two browser tabs, both members of channel `1` and joined to `room-42`, see each other's video peer to peer and each other's messages through the relay.

---

11. Footguns and Hard Rules

- **Bind a database BEFORE** `realtime({ features: ["chat" | "files"] })`**.** `ensureChatTables()` runs at mount, but a failure (no DB bound) is caught, logged as an ERROR, and boot continues. `realtime` still returns the full path map and registers every route, and the failure only resurfaces at query time. Call `initDatabase(url)` or `bindDatabase(db)` first, then `await realtime(...)`.
- `realtime()` **is async, so always** `await` **it.** Skipping the `await` risks the first chat, history, or file request racing table creation.
- **The signalling WebSocket (**`/ws/rtc/{room}`**) is PUBLIC.** It is not secured, so anyone can join any room and receive the relayed frames. Only the **chat** WebSocket is JWT-secured. Gate call access at the app layer if you need it.
- **The config endpoint (**`/api/rtc/config`**) is PUBLIC** and returns your ICE and TURN config, including freshly-minted ephemeral TURN credentials. This is by design, which is exactly why the TURN credentials are time-limited (~~`TINA4_RTC_TURN_TTL`~~).

• **The WebSocket handler signature is** `(connection, event, data)`**.** `event` is `"open"` / `"message"` / `"close"`, and `data` is the string frame on `"message"`. This matches the Python master. The **PHP** port fires `($connection, $data, $event)`, so the order differs there, not here.

• **Channels are addressed by integer id.** A non-integer `{channel}` makes the chat handler return silently, with no error frame. The client sees a socket that opens and does nothing. Pass the numeric channel id, never its name.

• **Chat authorization is re-checked on every frame,** and identity is always taken from the verified token (`connection.auth` / `req.user`), never from the message payload. A custom `authorize` must be cheap, since it runs on every inbound message.

• **A message with an empty or whitespace** `body` **is silently dropped.** No persist, no broadcast. `read`, `typing`, and unknown types never persist anything.

• **The backend is hardcoded** `mesh` **in Node.** There is no `media` option and `TINA4_RTC_BACKEND` is ignored. The Python master's `media=` parameter and `mint_join` SFU token do **not** exist in the Node port. Only mesh (peer to peer, zero core dependency) ships. An SFU or LiveKit backend is a future drop-in, not a current option, and because signalling paths would not change, the client keeps working.

Where to Go Next

• **The WebSocket chapter** - the raw `Router.websocket()` primitive these handlers are built on, and the connection API (`joinRoom`, `broadcastToRoom`, `sendJson`).

• **The ORM and Authentication chapters** - you seed workspaces, channels, and members with the ORM, and the JWT that secures chat is the same one from the Authentication chapter.

• **The tina4-js** `rtc` **module** - the browser side, whose `rtcConfig()` helper wraps `GET /api/rtc/config` so the client and server never drift on paths.

You started this chapter wanting a call button. You end it with calls, chat, presence, read receipts, history, and file transfer, mounted in one line, carrying no media, running no server you did not already have. The hard part of real-time was never the video. It was finding the other browser, and Tina4 relays that in a few lines of text.

AI Client

One class. Three methods. No provider SDK. The Tina4 AI client sends chat, completion, embedding, and streaming requests through one provider-neutral API.

This chapter covers the application-facing `Ai` class. The `tina4 ai` command serves a different purpose: it installs Tina4 skills and context for coding assistants.

Configure a provider

The client supports `local`, `openai`, and `anthropic`. Local mode targets an OpenAI-compatible endpoint and needs no API key.

```
TINA4_AI_PROVIDER=local
TINA4_AI_URL=http://localhost:11437
TINA4_AI_MODEL=llama3.2
TINA4_AI_TIMEOUT=60
TINA4_AI_CONNECT_TIMEOUT=10
TINA4_AI_MAX_RETRIES=2
```

Use the hosted OpenAI service by changing three values:

```
TINA4_AI_PROVIDER=openai
TINA4_AI_URL=https://api.openai.com/v1
TINA4_AI_MODEL=gpt-4o-mini
TINA4_AI_KEY=your-api-key
```

Anthropic uses the same keys:

```
TINA4_AI_PROVIDER=anthropic
TINA4_AI_URL=https://api.anthropic.com/v1
TINA4_AI_MODEL=claude-3-5-haiku-latest
TINA4_AI_KEY=your-api-key
```

An explicit method option wins over the environment. The environment wins over the built-in default. This lets one request use another model without changing the rest of the process.

Complete one prompt

`Ai.complete` sends one user message and returns the response text.

```
import { Ai } from "tina4-nodejs";

const answer = await Ai.complete("Explain why this query needs an index");
console.log(answer);
```

Pass options when one call needs a different model or provider:

```
const answer = await Ai.complete("Summarize this incident report", {
  provider: "openai",
  model: "gpt-4o-mini",
  temperature: 0.2,
  maxTokens: 300,
  timeout: 20,
});
```

Hold a chat

`Ai.chat` accepts a non-empty list of messages. Each message needs a `system`, `user`, or `assistant` role and string content.

```
import { Ai } from "tina4-nodejs";

const response = await Ai.chat([
  { role: "system", content: "Answer as a concise database engineer." },
  { role: "user", content: "When should I use a composite index?" },
]);

console.log(response.text);
console.log(response.model);
console.log(response.usage.totalTokens);
console.log(response.finishReason);
```

The `ChatResponse` object carries five fields:

Field	Meaning
`text`	Normalized response text
`model`	Model reported by the provider
`usage`	`promptTokens`, `completionTokens`, and `totalTokens`
`finishReason`	Provider finish reason, or `null`
`raw`	Original provider response

Use `raw` only when you need provider-specific metadata. Keep application logic on the normalized fields so a provider change does not spread through your code.

Stream text

Set `stream: true` to receive ordered text deltas. The async generator yields text only and ignores provider metadata events.

```
import { Ai } from "tina4-nodejs";

const chunks = Ai.chat(
  [{ role: "user", content: "Write a short release announcement." }],
  { stream: true },
);

for await (const chunk of chunks) {
  process.stdout.write(chunk);
}
```

Tina4 may retry before the first delta arrives. It never retries after yielding text because that could duplicate content the caller has already displayed.

Create embeddings

`Ai.embed` preserves the input shape. One string returns one vector. A list returns one vector per input, in the same order.

```
import { Ai } from "tina4-nodejs";

const vector = await Ai.embed("Tina4 keeps application code small");

const vectors = await Ai.embed([
  "The router finds a matching handler",
  "The ORM maps a row to a model",
]);
```

Set `TINA4_EMBED_URL` when the embedding service uses a different base URL. Anthropic does not expose embeddings through this contract, so `{ provider: "anthropic" }` raises `AiConfigError`.

Handle failures

All client failures inherit from `AiError`:

Error	Meaning
`AiConfigError`	Missing key, invalid provider, bad option, or unsupported capability
`AiHTTPError`	Provider returned a failing HTTP status or the transport failed
`AiTimeoutError`	Connection or total request deadline expired
`AiParseError`	A successful response did not match the provider contract

```
import { Ai, AiError } from "tina4-nodejs";

try {
  console.log(await Ai.complete("Create a migration plan"));
} catch (error) {
  if (error instanceof AiError) {
    console.error(`AI request failed: ${error.message}`);
  }
}
```

Hosted providers fail before sending when `TINA4_AI_KEY` is missing. Error text never includes the key, prompt, or provider response body. A failure stays a failure; Tina4 never turns it into an empty answer.

Timeouts and retries

`TINA4_AI_CONNECT_TIMEOUT` bounds connection setup. `TINA4_AI_TIMEOUT` bounds the whole request, including retries and response reads. `TINA4_AI_MAX_RETRIES` applies only to connection failures, HTTP 429, and HTTP 5xx responses.

Other HTTP 4xx responses and malformed successful responses run once and fail. The client carries transient trouble for a bounded distance, then hands the error back to your code.

OpenID Connect SSO

Tina4 completes one provider-neutral OpenID Connect flow. Configure an issuer and Tina4 discovers the provider, runs Authorization Code with PKCE, regenerates the Session, and sends the normalized identity through the existing secured route gate. The browser keeps the normal Tina4 Session cookie.

Configure SSO

```
TINA4_SSO_ISSUER=https://identity.example.com/realms/my-app
TINA4_SSO_CLIENT_ID=my-app
TINA4_SSO_CLIENT_SECRET=replace-me
TINA4_SSO_REDIRECT_URI=https://app.example.com/auth/callback
TINA4_SSO_SCOPES=["openid", "profile", "email"]
TINA4_SSO_VERIFY=introspection
TINA4_SSO_POST_LOGOUT_REDIRECT_URI=https://app.example.com/
```

SSO stays off until the issuer, client id, and redirect URI exist. Production issuer and callback URLs require HTTPS; loopback HTTP remains available for development. When configured, Tina4 mounts `GET /auth/login`, `GET /auth/callback`, and `POST /auth/logout`. A route collision stops startup with an error.

Use the identity

Secured routes use the same gate as local Tina4 authentication. The normalized identity is available as `request.user`. Provider tokens never enter it, logs, error pages, Swagger, or `session.all()`.

```
import { Sso } from "tina4-nodejs";

if (Sso.configured()) {
  const sso = await Sso.fromIssuer();
  const metadata = await sso.discover();
  const identity = sso.identity(request);
}
```

Method	Result
`Sso.configured()`	Reports whether required environment values exist.
`Sso.fromIssuer()`	Creates the client and discovers the issuer.
`discover(force = false)`	Returns validated metadata. Force refreshes it.
`login(request, returnTo = "/")`	Stores one-use state and returns the authorization URL.
`callback(request, query)`	Validates the callback and regenerates Session.

`identity(request)`	Returns the normalized identity or `null`.
`refresh(request)`	Replaces credentials and identity atomically.
`logout(request, returnTo = "/")`	Destroys Session and returns the safe logout target.

The return path must be local and absolute. Tina4 rejects schemes, hosts, protocol-relative URLs, backslashes, and control characters.

Claims and provider recipes

```
TINA4_SSO_CLAIM_MAP={"username":"preferred_username","roles":"realm_access.roles","groups":"grou
```

The runtime does not name providers. Keycloak uses a realm issuer such as `https://sso.example.com/realms/my-realm`. Microsoft Entra ID uses its tenant-specific v2 issuer. Register the exact callback URI and map claims only when roles or groups use different names.

Version 3.13.104 supports introspection without another npm package and needs a client secret. Selecting `jwks` fails during configuration until the application installs a supported cryptography capability. State, nonce, code, and verifier are one-use. Logout always destroys the local Session first.

When configured, Swagger publishes the issuer discovery URL as an `openIdConnect` scheme.

GIS and PostGIS

Tina4 stores geographic points in PostGIS, measures distance in metres, and returns map-ready GeoJSON. Coordinates always use longitude first. GPS may supply coordinates, but GIS is the feature.

Define a point field

Enable PostGIS with `CREATE EXTENSION IF NOT EXISTS postgis;`.

```
import { BaseModel, Point } from "tina4-nodejs/orm";

class ChargePoint extends BaseModel {
  static tableName = "charge_point";
  static fields = {
    location: { type: "point", srid: 4326, spatialIndex: true },
  };
}

site.location = new Point(18.4241, -33.9249);
await site.save();
```

Tina4 creates `geography(Point,4326)` and an idempotent GiST index. Unsupported database engines and PostgreSQL without PostGIS fail clearly. `Point.parse()` accepts a Point, number pair, WKT or EWKT, GeoJSON Point or Feature, and WKB or EWKB. It rejects invalid ranges, non-finite numbers, non-point geometry, and conflicting SRIDs before SQL runs.

Query and return GeoJSON

```
const nearby = await ChargePoint.query()
  .withinDistance("location", [18.42, -33.92], 5_000)
  .selectDistance("location", [18.42, -33.92], "metres")
  .orderByDistance("location", [18.42, -33.92])
  .get();

const feature = site.toFeature();
const collection = BaseModel.featureCollection(nearby);
```

The query builder also provides `intersects()` for bound WKT, EWKT, or GeoJSON geometry and `bbox()` for a validated bounding box. Spatial values use bound parameters. Column and alias names are validated as identifiers.

`toFeature()` moves the chosen Point into `geometry` and keeps other selected fields in `properties`. A null point creates `"geometry": null`. `featureCollection()` preserves query order. Supply a geometry field for models with several Point fields and an include list to limit properties.

Version 3.13.104 persists Point fields only. It does not persist lines, polygons, rasters, tiles, geocoding results, or routes. `intersects()` may accept a polygon as query geometry. PostGIS is the only storage provider.

IoT and MQTT

Tina4 includes a zero-dependency MQTT 3.1.1 client for telemetry, device state, asset tracking, and EV charging. It connects to an external broker such as Mosquitto, EMQX, HiveMQ, or AWS IoT. Tina4 is not a broker.

Configure, connect, and publish

```
TINA4_MQTT_URL=mqtt://127.0.0.1:1883
TINA4_MQTT_CLIENT_ID=warehouse-api
TINA4_MQTT_KEEPALIVE=60
TINA4_MQTT_TLS_VERIFY=true
TINA4_MQTT_CA_FILE=/run/secrets/mqtt-ca.pem
```

Use `mqtt://` or `tcp://` for plain TCP and `mqtts://` for TLS. Put credentials in URL user information or constructor options. There are no username or password environment variables.

```
import { Mqtt } from "tina4-nodejs";

const mqtt = new Mqtt();
await mqtt.connect();
const packetId = await mqtt.publish(
  "fleet/meter-42/telemetry",
  JSON.stringify({ kwh: 12.5 }),
  1,
);
await mqtt.disconnect();
```

Node.js connects explicitly. QoS 0 waits for no acknowledgement. QoS 1 returns the packet id after PUBACK. Tina4 refuses QoS 2.

```
const mqtt = new Mqtt({ cleanSession: false });
await mqtt.connect();

for await (const message of mqtt.consume("fleet/+/telemetry", 1)) {
  process(message.topic, message.text());
}
```

`consume()` acknowledges after successful loop handling. Use `receive(undefined, false)` followed by `message.acknowledge()` for manual control. Messages expose topic, byte payload, QoS, packet id, retained and duplicate flags, acknowledgement, `text()`, and `toObject()`.

Retained publishes store current state; an empty retained payload clears it. Configure `willTopic`, `willPayload`, `willQos`, and `willRetain` for an unclean disconnect. `disconnect()` suppresses the will; `kill()` lets the broker publish it. TLS verification defaults to true. `tls()`, `cipher()`, and `tlsVersion()` report the connection. One client has one socket reader.